DCU编程实战2026版

DCU编程实战

愿本文稿能够帮助读者更高效的了解 DCU , 使用 DCU 。

感谢所有为本文稿贡献力量的朋友们。

本文稿非正式出版书籍 , 仅为交流学习使用。本书所涉及的源 码及更多关于 DCU 的信息请访问 https://developer.sourcefind .cn/

对于本书的编写工作 , 各位作者付出了极大的心血和努力 , 然 而编写时间仓促 , 精力有限 , 书中难免会有所疏漏敬请读者批评 指正 。

目录

- 第1章 概述
  - 1.1 从传统计算到异构计算的发展历程
  - 1.2 人工智能时代对AI加速卡提出新需求
  - 1.3 智能体的火爆为AI算力带来新的机遇
  - 1.4 海光DCU的价值升级：国产算力底座的战略突破
- 第2章 DCU软件栈简介
  - 2.1 DCU软件栈（DCU ToolKit, DTK）
  - 2.1.1 DTK中支持的编程语言
  - 2.1.2 DTK中的库
  - 2.1.3 DTK中的工具链
  - 2.2 DTK安装与使用
  - 2.3 DAS人工智能软件系统(DCU AI Software Stack)
  - 2.3.1 产品概述
  - 2.3.2 产品特点
  - 2.3.3 产品功能
  - 2.3.4 软件优化能力
  - 2.3.5 OpenDAS
- 第3章 使用C/C++编写DCU程序
  - 3.1 初识并行计算
  - 3.1.1 从一个串行程序开始
  - 3.1.2 为程序计时
  - 3.1.3 使用多个CPU核
  - 3.1.4 为DCU编写代码
  - 3.1.5 编译运行DCU程序
  - 3.1.6 核函数执行是异步的
  - 3.1.7 小节
  - 3.2 DCU编程总览
  - 3.2.1 异构编程模型
  - 3.2.2 从模型到硬件
  - 3.2.3 小节

- 3.3 并行规约
- 3.3.1 原子操作
- 3.3.2 利用共享内存优化原子加
- 3.3.3 利用线程束洗牌函数优化原子加
- 3.3.4 另一种可扩展的核函数写法
- 3.3.5 用空间换时间
- 3.3.6 小节
- 3.4 异步并发执行
- 3.4.1 用核函数掩盖主机端的求和
- 3.4.2 流与异步拷贝
- 3.4.3 主机-设备间拷贝与页锁定内存
- 3.4.4 事件
- 3.4.5 核函数并发执行
- 3.4.6 默认流
- 3.4.7 小节
- 3.5访存优化——以矩阵乘法为例
- 3.5.1 使用DTK中的库
- 3.5.2 初步编写核函数
- 3.5.3 使用共享内存优化
- 3.5.4 让每个线程计算更多的矩阵元
- 3.5.5 局部内存中的数组
- 3.5.6 共享内存的bank冲突
- 3.5.7 小节

- 第 4章 使用OpenMP/OpenACC编写DCU程序
  - 4.1 OpenMP与OpenACC简介
  - 4.2 OpenMP的设备端并行
  - 4.2.1 编写第一个设备端OpenMP程序
  - 4.2.2 OpenMP的核函数
  - 4.2.3 OpenMP任务并发
  - 4.3 OpenMP数据传输
  - 4.3.1 数据传输子句
  - 4.3.2 数据传输结构
  - 4.3.3 数据映射的优先级和生存周期
  - 4.4 OpenACC的设备端并行
  - 4.4.1 OpenACC的编译运行
  - 4.4.2 OpenACC的计算构件
  - 4.4.3 OpenACC异步
  - 4.5 OpenACC数据传输
  - 4.5.1 data指令
  - 4.5.2 enter data指令和exit data指令
  - 4.5.3 update指令
  - 4.5.4 declare/routine指令
  - 4.5.5 host_data指令
  - 4.5.6 private私有数组
  - 4.5.7 结构体、结构体数组、结构体内部多级指针
  - 4.6 调试方法
  - 4.6.1 核函数启动信息
  - 4.6.2 异构数据传输信息

- 4.6.3 核函数执行时间
- 4.6.4 核函数内部数据调试方法
- 4.6.5 hipprof性能日志
- 4.6.6 VMFault
- 4.6.7 编译过程信息Minfo
- 4.6.8 核函数计算完成后各数组求和
- 4.7 性能优化思路
- 4.7.1 创建核函数
- 4.7.2 合理使用数据驻留指令
- 4.7.3 使用异步多流
- 4.7.4 使用多卡、多进程
- 4.7.5 acc routine并行函数
- 4.7.6 规约
- 4.7.7 优化访存顺序

- 第 5 章 使用 DCC编译DCU程序
  - 5.1 DCC 概述与定位
  - 5.1.1 DCC 总体介绍
  - 5.1.1 DCC 在 DTK 工具链中的角色
  - 5.1.2 DCC 支持的语言与编程模型
  - 5.1.3 DCC 工具链介绍
  - 5.2 安装与环境准备
  - 5.3 快速上手：从源码到可执行
  - 5.3.1 C/C++ 基本编译
  - 5.3.2 HIP 程序编译
  - 5.3.3 OpenMP Offload 编译
  - 5.3.4 常见编译产物与 Fatbinary 组织方式
  - 5.4 编译选项说明
  - 5.4.1 优化等级
  - 5.4.2 目标架构与 Offload 选项
  - 5.4.3 默认参数注入机制
  - 5.4.4 浮点、精度与快速数学选项
  - 5.4.5 GPU 执行与资源相关参数
  - 5.5 DCC 的工具链
  - 5.5.1 从“一个编译命令”到“一套工程链路”
  - 5.5.2 dcc：统一驱动、参数编排与默认构建机制
  - 5.5.3 dccobjdump：把编译结果变成可观察对象
  - 5.5.4 dccturing：面向汇编级实验与再编译的低层工具
  - 5.5.5 dcc-autotune：把参数搜索从手工试错变成系统流程
  - 5.5.6 dccprof：把运行时行为纳入编译工具链闭环
  - 5.6 DCC 的前端和 GCVM
  - 5.6.1 前端在 DCC 中的位置
  - 5.6.2 多语言、多模型入口如何在 DCC 中汇聚
  - 5.6.3 GCVM 的角色：它不是“另一个名字”，而是统一承接层
  - 5.6.4 GCVM 的接口模型与基本使用方式
  - 5.6.5 AI 编译器如何接入 GCVM：以 Triton 与 XLA 路径为例
  - 5.7 DCC 的 Lingshu 后端
  - 5.7.1 Lingshu 后端在整体编译链中的位置
  - 5.7.2 地址空间与存储层次

    - 5.7.3 SGPR、VGPR 与资源占用模型
    - 5.7.4 初始执行状态与 wavefront 执行模型
    - 5.7.5 Code Object 与装载信息
    - 5.7.6 资源预算、occupancy 与寄存器调优
    - 5.7.7 Lingshu 后端中的典型优化
    - 5.7.8 后端 lowering 与机器代码生成流程
    - 5.8 DCC 的调试与诊断
- 第六章 使用DCU开发AI应用
    - 6.1 光源平台
    - 6.2 深度学习基础环境的安装及部署
    - 6.3 使用PyTorch训练分类模型
    - 6.3.1 PyTorch简述
    - 6.3.2 PyTorch安装
    - 6.3.3 训练分类模型
    - 6.3.4 如何进行分布式训练
    - 6.3.5 基于DCU的PyTorch性能分析
    - 6.4 使用TensorFlow训练分类模型 –徐显冲
    - 6.4.1 TensorFlow简述
    - 6.4.2 TensorFlow安装
    - 6.4.3 训练分类模型
    - 6.4.4 如何进行分布式训练
    - 6.5 大模型DCU训练
    - 6.5.1 Megatron-LM简述
    - 6.5.2 大模型并行介绍
    - 6.5.3 大模型训练优化
    - 6.5.4 算子级 FlashAttention 优化实战
    - 6.6 大模型DCU推理
    - 6.6.1 vLLM推理框架简介
    - 6.6.2 SGLang推理框架简介
    - 6.6.3 大模型推理优化
    - 6.6.4 算子级MOE优化实战
    - 6.7 通用模型DCU推理
    - 6.7.1 MIGraphX简介
    - 6.7.2 安装MIGraphX
    - 6.7.3 MIGraphx推理
    - 6.7.4 算子级卷积优化实战
    - 6.8 AI 编译器编程
    - 6.8.1 AI编译器提出的背景
    - 6.8.2 Triton 简介
    - 6.8.3 Triton 核心编程模型
    - 6.8.4 Triton 算子的高级优化技术
    - 6.8.5 在 DCU 上使用 Triton 的实践
- 附录
    - A 错误检查
    - A.1 API返回错误
    - A.2 kernel函数非法地址访问
    - B HIP调试工具

- B.1 使用hipgdb调试程序
- B.2 核函数调试具体过程
- B.3 总结
- C hipGraph
- C.1 hipGraph简介
- C.2 函数接口说明
- C.3 示例
- D Cooperative Groups
- D.1 Cooperative Groups简介
- D.2 块内同步
- D.3 网格级同步
- D.4 多设备同步
- E开发者社区（https://developer.sourcefind.cn/）
- E1、社区定位与核心价值
- E2、核心功能与服务模块

第1章 概述

本章节主要讲述了处理器异构化发展趋势，以及异构编程技术在人工智能领域中的应用，解释了为什么异构编程越来越受到重视以及学习异构编程的必要性。

1.1 从传统计算到异构计算的发展历程

计算机系统的不断进步是人类最杰出的工程成就之一。人类通过 50
来年的努力,使现代手机的计算能力达到了阿波罗登月任务所用的计算机的100万倍。这种进步的关键在于半导体行业的发展，以及随之而来的对处理器速度、功率和成本的改进。

处理器是由晶体管组成的。晶体管是逻辑开关，构成了从原始逻辑函数(如与、或、非)到复杂算术(浮点加法、正弦函数)以及存储器(如ROM、DRAM)等所有组件。多年以来，晶体管的研发方向一直是如何做得更小，使得在有限的芯片面积上集成更多的晶体管。

1965年，戈登・摩尔发现，集成电路中的晶体管数量每1-1.5年翻一番。他预计这一趋势将持续至少数十年。虽然有人认为，这与其说是一个定律，不如说是一个行业趋势，但它确实持续了大约
50年，是历史上持续时间最长的人为趋势之一。然而除了摩尔定律还有一条鲜为人知却至关重要的定律。它被称为登纳德缩放比例定律，由罗伯特・登纳德在
1974年提出。虽然摩尔定律预测晶体管将逐年缩小，但登纳德问道:除了能够在单个芯片上安装更多晶体管之外，拥有更小的晶体管还有什么实际好处?他发现当晶体管以k为倍数缩小时，电流也会降低。此外，由于电子移动的距离更小，我们最终得到的晶体管快了k倍，最重要的是–它的功率下降到1/（k*k）。因此，我们可以装配的晶体管，逻辑函数将快大约k*k倍，但芯片的功耗不会增加。在这50年中，芯片的性能按照这两个定律不断增长，功耗相对可控，这是一个美好的阶段。

不幸的是，大约在2000年，登纳德缩放比例定律开始崩溃。具体来说，随着频率的提升，电压停止以相同的速率下降，功率密度速率也是如此。如果这种趋势持续下去，芯片发热问题将不容忽视。然而，强大的散热方案还不成熟。因此，供应商无法继续依靠提高CPU频率来获得更高的性能，需要想想其它出

路，例如多核心CPU。

通常，单个应用是以连续指令流的形式编写的，如果 CPU
同时运行四个应用，理论上计算性能也能提升四倍。因此，解决方案之一，不是加快单个处理器的速度，而是将芯片分成多个相同的处理内核，每个内核执行其指令流。多核执行的方式也常见于GPU
中，这是因为 GPU 当中拥有很多的简单计算核心(相比与 CPU来说)。通常来讲，GPU
侧重于图形计算应用。因为图形图像由数千个像素组成，可以通过一系列简单且预先确定的计算来独立处理。从概念上来说，每个像素可以被分配一个线程，并执行一个简单的迷你程序来计算其行为，每个线程可以单独放置在一个计算核心中运行。高度的像素级并行使得开发数千个处理内核变得很自然。

[图片]

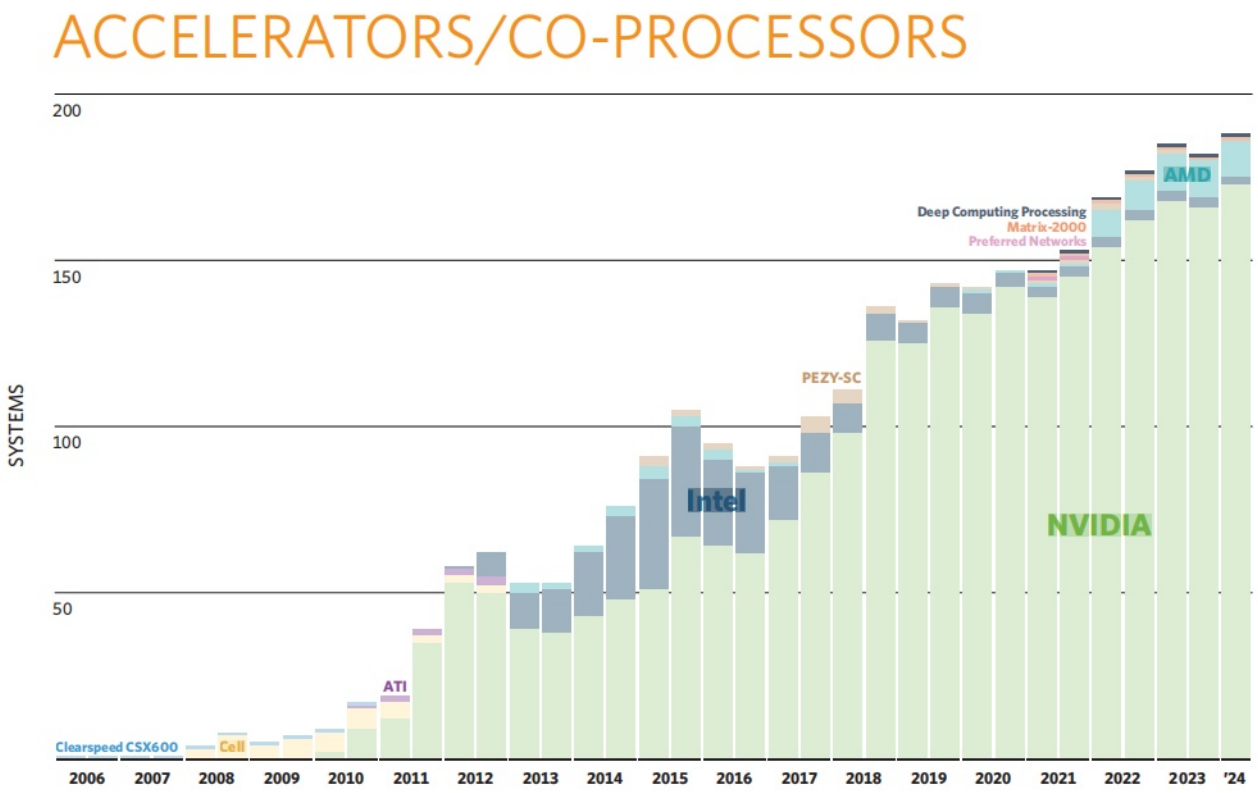


图1-1 GPU在TOP500中占比逐年增加

但是到了
2010年前后，事情变得更加复杂：登纳德缩放比例定律走到了尽头，因为晶体管的电压接近物理极限，无法继续缩小。虽然以前可以在保持相同功率预算的情况下增加晶体管数量，但晶体管数量翻倍意味着功耗也翻倍。此时，我们的芯片上有多少晶体管并不重要，只要有功耗限制，能够同时运行的晶体管数量是固定的，芯片的其余部分必须断电，这种现象也被称为“暗硅”。

暗硅本质上是摩尔定律终结的大预演–对处理器制造商来说，顺应时代要求变得具有挑战性。一方面，计算需求飞速增长，智能手机变得无处不在，而且拥有强大的计算能力，云服务器需要处理越来越多的服务。另一方面，人工智能重新登上历史舞台，并以惊人的速度吞噬计算资源。暗硅成为晶体管芯片发展的障碍，因此，当我们比以往任何时候都更需要提高处理能力时，这件事却变得比以往任何时候都更加困难。

[图片]

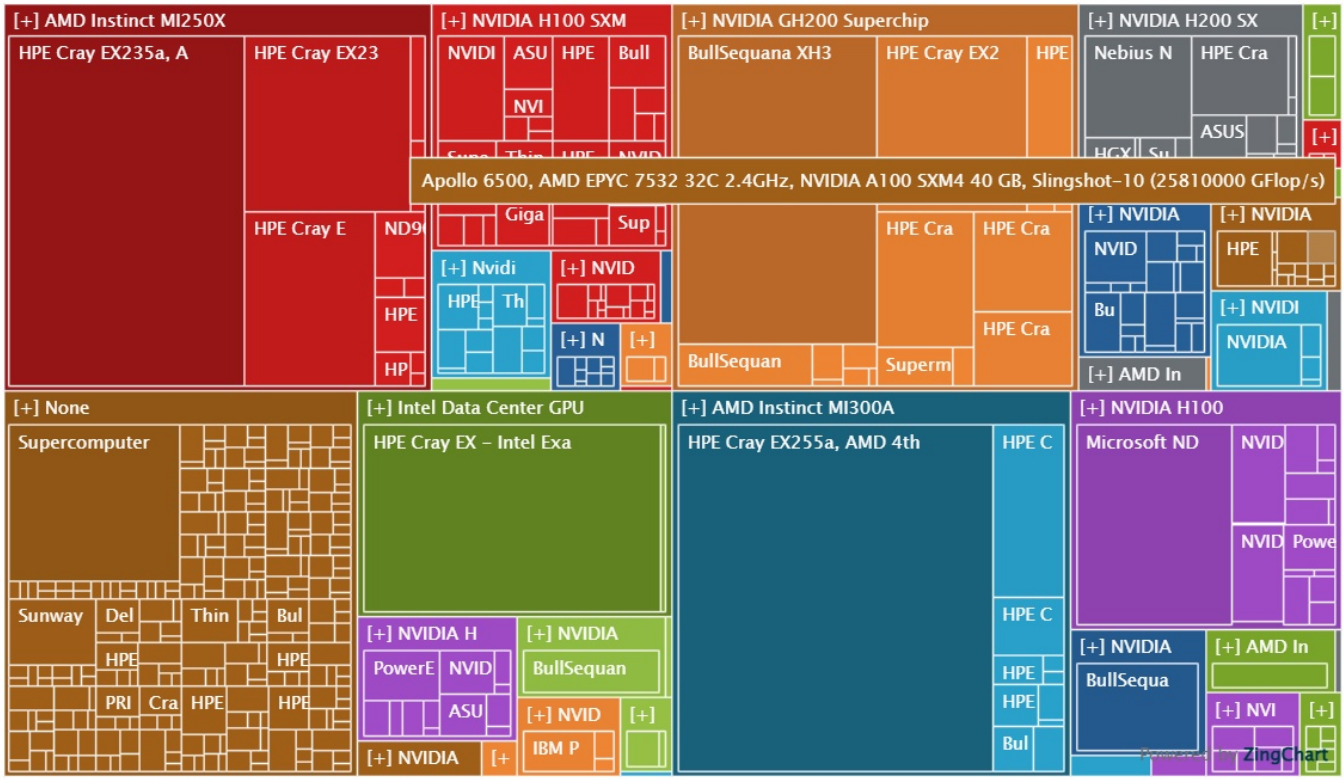


图1-2 SC25上TOP500中主要的加速卡，不包含国产卡

自从新一代芯片被暗硅束缚以来，计算机行业就开始把精力放到了硬件加速器上。他们的想法是:如果不能再增加晶体管，那就好好利用现有的晶体管吧。具体怎么做呢?答案是:专门化。

传统的 CPU
被设计成通用的，它们使用相同的硬件结构来运行我们所有应用。但是在图形处理中，计算问题往往是相似且简单的大量问题，如果把这些问题也放在CPU 中会使得 CPU
应顾不暇。所以将特定的图形处理问题交给 GPU 来，让 CPU 去调配 GPU 来进行计算，CPU
可以继续专注于自己擅长的事情。这种同时使用 CPU 和 GPU的计算模式也称之为“异构计算”。

随着这种异构组合应用的不断发展，GPU从最早的图形处理设备，慢慢发展为通用的异构计算设备。从核心芯片的架构角度来看，CPU是以低延迟为导向的计算单元通常由专为串行处理而优化的几个核心组成，而
GPU是以吞吐量为导向的计算单元由数以千计的更小、更高效的核心组成，专为并行多任务设计。

CPU 和 GPU 设计思路的不同导致微架构的不同。CPU的缓存大于 GPU，但在线程数，寄存器数和 SIMD(单指令多数据流)方面 GPU 远强于 CPU。微架构的不同最终导致 CPU
中大部分的晶体管用于构建控制电路和缓存，只有少部分的晶体管完成实际的运算工作，其功能模块很多，擅长分支预测等复杂操作。GPU的流处理器和显存控制器占据了绝大部分晶体管而控制器相对简单，擅长对大量数据进行简单操作，拥有远胜于 CPU
的强大浮点计算能力。因此，在相同的晶体管数量下，GPU 相对 CPU
峰值会高一个量级以上，功耗与晶体管的数量成正比，因此 GPU 的性能功耗比也会比 CPU
高一个量级以上。

综上所述，以通用 GPU(General-purpose
GPU，GPGPU)为主要计算部件的异构系统将会是解决功耗墙最好的方案之一。

1.2 人工智能时代对AI加速卡提出新需求

人工智能毫无疑问是当今科技领域最热门的技术，几乎没有之一。人工智能技术已经逐渐从实验室的研究走向了产业化，进入到了人们的日常生活中。继互联网和移动互联网引发第三次工业革命之后，以大数据驱动的人工智能技术正在推动第四次工业革命，这场技术革命的核心是网络化、信息化与智能化的深度融合。人工智能的时代正在到来。

人工智能的概念早在
1956年提出，经历了孕育、形成和发展3个阶段。在人工智能出现至今经历过几次寒冬，自深度学习算法出现后，近几年进人了快速爆发期。1957年，罗森布拉特发明感知机，是机器学习人工神经网络理论中神经元的最早模型；1967年，计算机可以通过KNN最近邻算法进行简单的模式识别。然而在人工智能的起步期后段，由于计算能力的限制，人工智能的发展进人了第一次寒冬。20 世纪
70年代中期开始，人工智能进人了以专家系统为代表的“知识期”，专家系统的能力来自于它们存储的专业知识，知识库系统和知识工程成为了80年代人工智能研究的主要方向，由于资源投人的缩减、知识获取困难和无法大规模扩展等因素，人工智能进人第二次低谷。之后人工智能进人了多元的发展周期，出现了决策树Boosting和 SVM 等多种算法，1997年，IBM
的超级计算机深蓝战胜了国际象棋世界冠军。从 2006年神经网络专家Hinton
提出神经网络深度学习算法之后，开启了深度学习在学术界和工业界的浪潮，例如无人驾驶汽车的上路、AlphaGo 运用深度学习战胜围棋冠军和 GPT-3
超大自然语言模型等等。这些新应用也将计算需求提升到一个全新高度。

[图片]

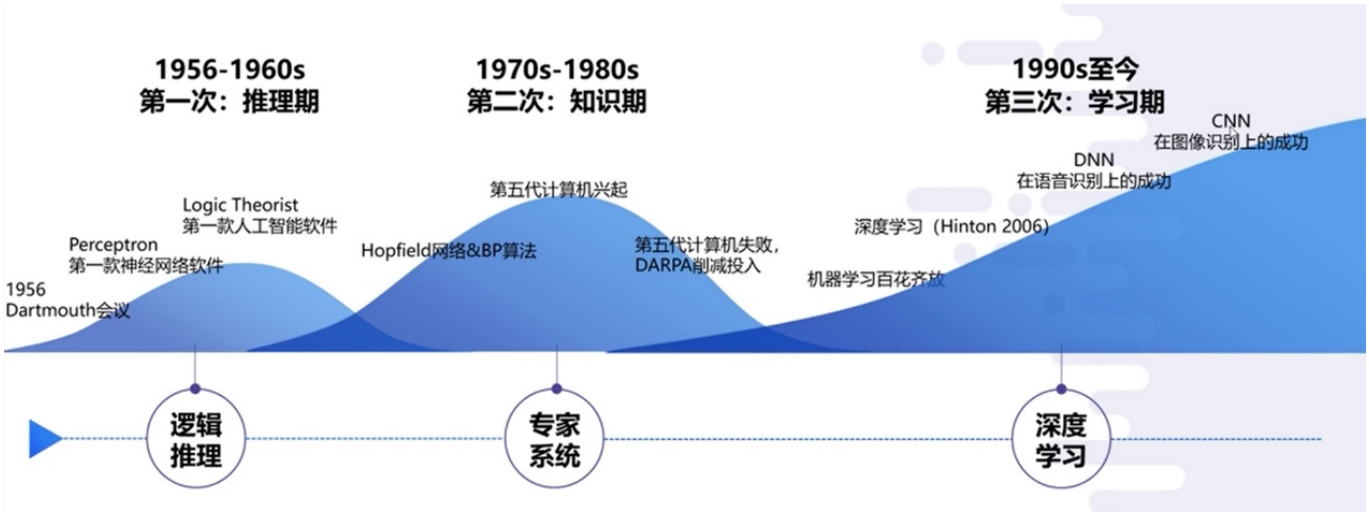


图1-3 人工智能发展的三次浪潮

近年来，人工智能的理论和技术都得到了飞速的发展，并在视频监控、语音识别图像分割、自然语言处理等领域都取得了突破。随着人工智能技术的发展，国内大批的高校、研究所和企业进人了智能化的浪潮中。中国科学院各大研究所以及清华大学、北京大学等高校也都已经开始了人工智能相关学科的建设以及人才培养工作。智能驾驶、智能诊疗智慧安防等领域已经逐渐进入到了人们的视野中，大量的智能化设备已经进人到了人们的生活中，大大提高了生活的便利性与舒适性。

[图片]

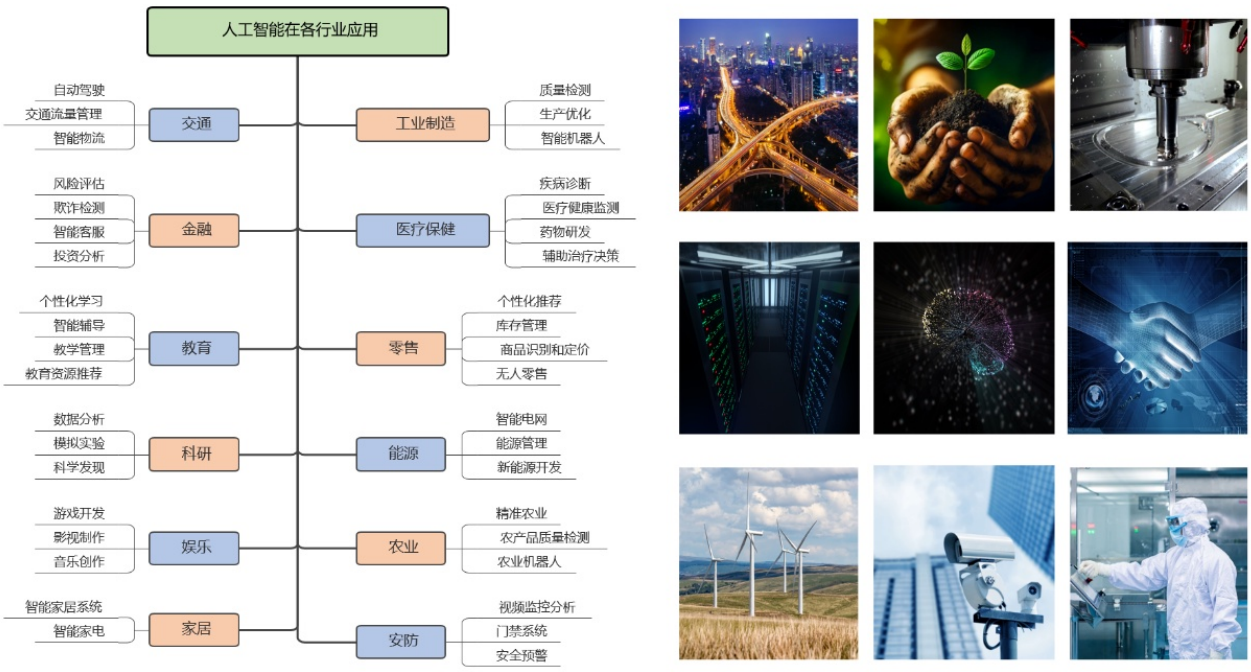


图1-4 人工智能主要应用领域

目前人工智能技术已经从基于数据的联网时代、基于算力的云计算时代发展到了基于模型的 AI 时代，将数据进行深度挖掘提升为超大规模的预训练模型。随着 ChatGPT发布，国内在超大规模预训练模型上也有了迅速发展，如百度文心大模型、阿里“通义千问”以及科大讯飞的“星火”大模型等等。另一方面，人工智能与传统的科学计算的联系越来越紧密，以深度势能为代表的AI 蛋白质预测方向，其研发的 Uni-Fold 不仅复现了 Alphafold2 的全规模训练，同时还解决了硬件支持单一、模型不可商用等问题、完善了训练和推理环节在 CPU 上的适配和性能优化工作，同时已经将相关工作集成至自主研发的药物设计平台 Hemmite 中。

目前有相当数量的算法依然调用传统数据中心的算力进行发展，但传统数据中心的面临新挑战：AI工作负载对电力、散热和网络提出极高要求（单机架功耗可达600千瓦）。传统数据中心模式难以适应，需重新设计基础设施。全球算力巨头们给出了他们的新解决方案：包括建设AI工厂，实现端到端架构，整合电力、数据、算力。

2025年开始，人工智能大模型进一步升级，模型的训练和推理对于AI算力有了更高的需求。很多人工智能初创企业都会把大量的融资或收入用于AI算力基础设施的打造。以Anthropic为例，这家硅谷企业已成为AI领域融资最多的初创公司之一。其融资主要用于训练大模型，GPU算力是核心支出。另外，全球的AI算法公司都在进行不断地优化实现以更低的算力成本实现同样的目的。而近年来，中国的Deepseek就是这样的代表，高效能可以更容易得到市场和资本的青睐，也是AI发展的必然。

1.3 智能体的火爆为AI算力带来新的机遇

智能体（Agent）是人工智能（AI）领域中的一个核心概念，指的是能够感知环境、进行决策并采取行动以实现特定目标的自主实体。它可以是软件程序（如聊天机器人）、硬件设备（如自动驾驶汽车或机器人），或者软硬结合的系统。

根据经典教材《人工智能：一种现代方法》的定义：“智能体是任何可以通过传感器感知环境，并通过

执行器对该环境产生作用的实体。”

智能体的主要特点包括：

1. 感知能力：通过传感器或数据输入（如摄像头、麦克风、文本）获取环境信息。
2. 决策能力：基于感知信息，利用算法、模型或规则进行推理和规划。
3. 行动能力：通过执行器（如机械臂、API调用、消息发送）影响环境。
4. 自主性：能在一定程度上独立运行，无需持续人工干预。
5. 学习与适应能力（部分智能体具备）：通过经验不断优化自身行为。

近年来智能体迅猛发展，对人工智能芯片的性能提出了前所未有的新要求。有以下两点值得关注：混合专家模型（MoE）架构已向十万亿参数演进，训练和推理对内存带宽、通信效率、并行计算能力提出极高要求。模型安全成为焦点：权重窃取、模型投毒等新型攻击频发，推动硬件级安全机制成为刚需。

最火的“Open claw小龙虾”智能体引爆Agent革命。OpenClaw（中文昵称“龙虾”）是一个开源的 AI 智能体框架，其核心目标是：让大语言模型（LLM）不仅能“说”，还能“做”——即在真实计算机环境中自动执行任务。作为通用AI智能体，其具备自主任务拆解、跨应用操作、长期在线执行等能力，实现从“会说”到“会做”的跨越。而智能体单次任务Token消耗可达传统对话模型的数十至千倍，引发算力需求指数级增长。行业正从单模型部署转向多Agent协作系统，进一步放大对集群调度、低延迟通信和高并发处理的需求。

“小龙虾”的爆火对AI芯片的新性能要求，包括以下几点：

1.
算力密度大幅提升：AI智算机柜功率已达240kW（是传统机柜的30倍），万卡集群年耗电超6亿度。
2.
能效比成为关键指标：在“算电协同”被写入2026年政府工作报告背景下，芯片需在提供高算力的同时降低单位功耗。
3.
安全能力内生于硬件：需支持机密计算、运行时数据加密（如SM4国密算法）、防算力劫持等安全特性。
4.
软硬协同生态韧性：要求芯片厂商提供完整的软件栈（如编译器、通信库、安全框架），以支撑复杂智能体部署。

综上可以看出，人工智能时代不可回避的一个现实就是，对于GPU不仅有数量上的需求更有性能上的要求。而一款GPU只有软硬件性能与量产供货都能保证才能成为真正意义上的AI算力基座。而海光DCU（GPGPU）可以很好地成为这一选择。

1.4 海光DCU的价值升级：国产算力底座的战略突破

作为国内高端处理器领域的标杆企业，海光信息深耕集成电路产业链中游设计环节，是国内少数同时具备高端通用处理器（CPU）和协处理器（DCU）研发能力的企业。

从战略优先级来看，公司采用 “技术自研 + 生态共建”
的双轮驱动模式：一方面坚持高端处理器自主设计，掌握核心技术话语权；另一方面通过与整机厂商、软件供应商深度协同，构建国产化算力生态。

[图片]

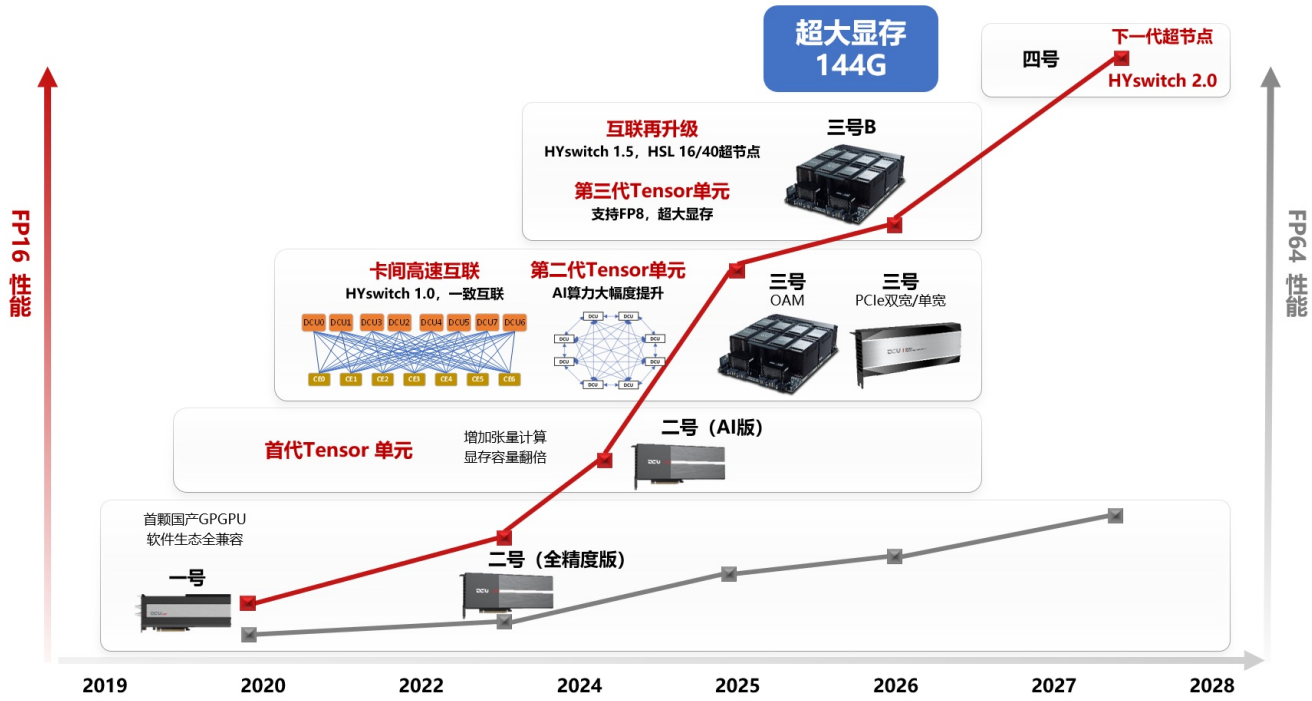


图1-5 海光DCU卡的发展

海光DCU 是一款加速计算芯片，基于 GPGPU 架构，支持 “训推一体” AI 场景全覆盖，其自主研发的 DTK 软件栈是国内最为完备的生态之一，已与多家头部互联网厂商完成全面适配，在 AI 大模型训练与推理、大数据处理等场景实现规模化应用。与其他GPGPU 一样，DCU 有强大的并行计算能力。为了顺应处理器异构化发展趋势，高效利用 DCU 的强大性能，系统性地学习DCU程序开发是非常必要的。本书将会逐步展开对 DCU 程序开发的介绍。

结语：AI大模型与“小龙虾”智能体的爆发，不仅是一场应用革命，更是一场底层算力基础设施的重构。在算力即国力的2026年，海光DCU凭借其安全原生、双芯协同与能效优化的综合优势，正从国产替代走向价值引领，成为支撑中国AI智能体时代的关键基石。

第2章 DCU软件栈简介

开发环境对于软件开发是必不可少的，本章我们来介绍DCU软件的开发环境——DTK软件栈。

2.1 DCU软件栈（DCU ToolKit, DTK）

DCU拥有自己的软件栈—DCU Toolkit（DTK），用来支持使用DCU进行异构计算，它是一种轻量级、模块化的软件开发环境，可以提供多种开发工具和运行时环境，适用于高性能计算和深度学习应用程序的开发。

[图片]

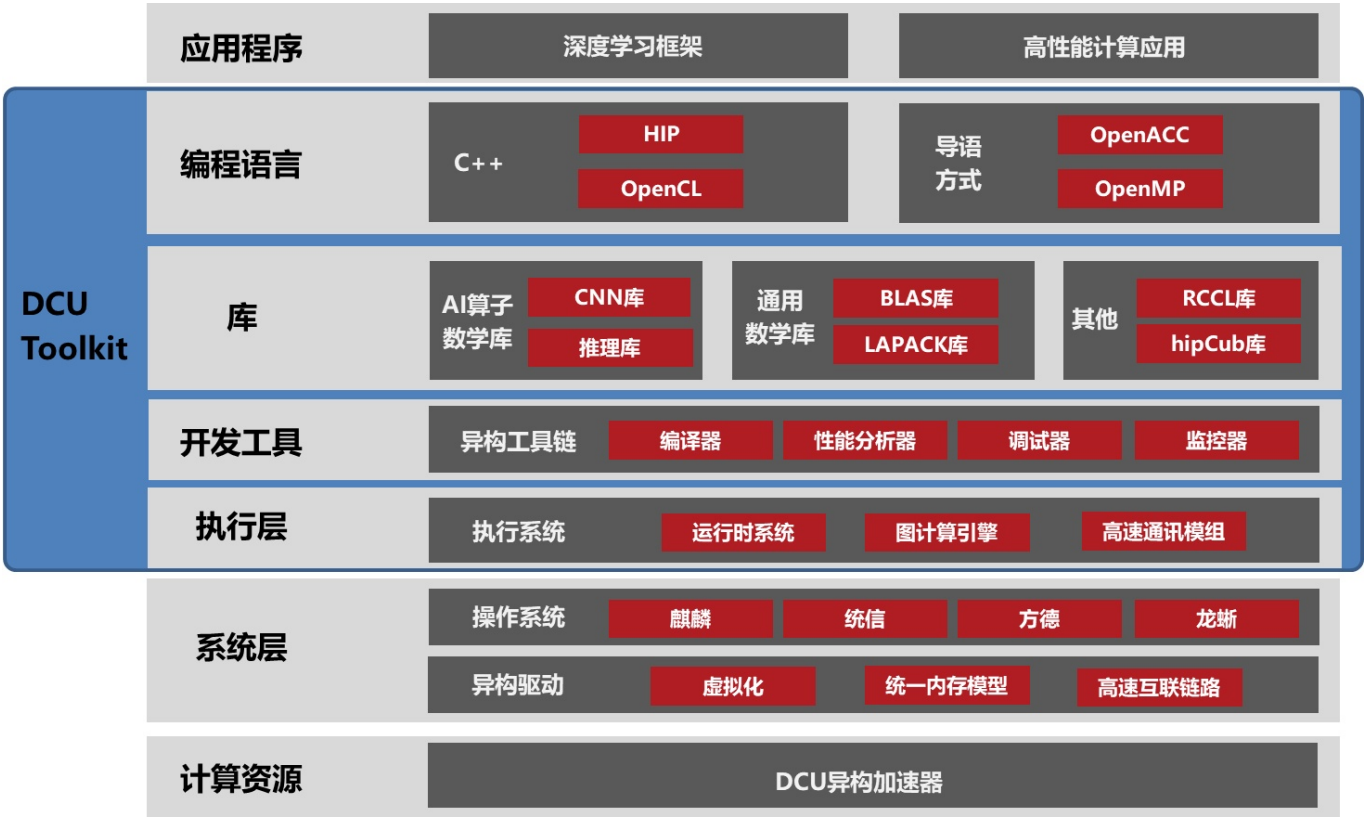


图2-1 DCU软件栈

图2-1展示了DTK在DCU异构计算系统中所处的位置。DTK处于异构计算系统中硬件与系统软件之上，DCU应用的开发与运行都以DTK为基础。DTK能兼容CentOS、Ubuntu等主流的Linux发行版，还能兼容麒麟、方德、统信等国产操作系统。DTK支持开发者在C++语言中使用HIP、OpenCL等编程接口开发DCU应用，还支持在C++或Fortran（部分支持）语言中使用OpenMP、OpenACC等编译器导语的方式进行开发。DTK中提供了编译器、性能分析工具、调试工具、系统状态监控等完整的开发工具链以保证便捷地开发。DTK中还包含了强大的运行时系统、图计算引擎和高速通讯模组等模块，以支持DCU应用的高效执行。

2.1.1 DTK中支持的编程语言

HIP（Heterogeneous-Compute Interface for Portability）是AMD公司在2016年提出的符合CUDA编程模型的、自由的、开源的C++编程接口和核函数语言。其设计核心是可移植性，让开发者使用HIP编写的同一套源代码，既可以编译运行在AMD的GPU上，也可以编译运行在NVIDIA的GPU上。在DTK中，HIP被进一步开发，使开发者用HIP编写的的代码可以编译运行在DCU上。在目前的DTK中，使用HIP编程是进行DCU应用开发最主要的方式。本书的第3章将介绍如何在C/C++中使用HIP编写DCU程序。

OpenMP（Open Multi-Processing）是共享内存的多处理器并行应用编程接口，它通过在C/C++或Fortran代码中加入编译器导语（directive）的方式让编译器将指定代码段并行化，此外还包含一系列函数形式的编程接口。OpenMP让开发者几乎不用改动原代码就能将串行程序修改为并行程序，发挥多核处理器的性能。OpenMP由OpenMP架构审核委员会（OpenMP Architecture Review Board）制定标准，各种编译器根据标准给出具体实现。OpenMP在1999年发布Fortran 1.0版本，最新版本为2024年发布的6.0版本。在OpenMP

4.0版本的标准中额外增加了对异构设备（主要是GPU）的支持。在目前的DTK中，支持全部的OpenMP 4.5标准及5.0、5.1、5.2中的部分标准，使用OpenMP可以便捷地将计算任务交给多个CPU核心和DCU，充分利用系统的各种硬件资源；使用OpenMP是进行DCU应用开发便捷有效的方式，我们将在第5章对此展开介绍。

OpenACC（Open ACCelerators）可以看作OpenMP针对异构设备的开发分支。使用OpenACC进行开发的方式与使用OpenMP是相同的，但其导语和编程接口与OpenMP是是不同的。近年来，OpenACC在逐步合并回OpenMP，OpenMP 4.0中对异构设备的支持标志着这种合并的开端。但OpenACC因其针对异构设备的出色设计仍保持其独立性，很多重要的高性能计算应用都通过OpenACC进行异构移植。在目前的DTK中，支持在C/C++中使用OpenACC的部分功能，相关介绍也在本书第5章。

此外在DTK中保持对OpenCL（Open Computing Language）的支持，在DTK中可以编译运行使用OpenCL编写的程序，但对OpenCL的支持不是DTK的开发热点，本书也不涉及对使用OpenCL编程的介绍。

高性能计算中很多领域的开发者保持着使用Fortran语言编程的习惯，但在目前的DTK中不支持使用Fortran语言编写DCU核函数。对于使用Fortran语言的开发者，目前有两种开发路线，其一是利用Fortran 2003标准中ISO_C_BINDING模块对C和Fortran互操作性的支持，在Fortran程序中调用HIP的编程接口（包括一系列数学库）和用HIP编写的核函数，本书第4章将介绍这种开发路线；另一种开发路线则是使用OpenMP，本书中对OpenMP的介绍虽然以C/C++为例，但所介绍的功能在Fortran语言中同样可以使用。

2.1.2 DTK中的库

在上节中介绍了HIP是AMD为了兼容CUDA平台而设计的，其中也包含了对CUDA平台上库的支持。对于CUDA平台上的大部分库，都有使用HIP编写的对应，这些库同样是开源开放的，使用这些HIP库可以实现代码在AMD GPU和NVIDIA GPU上的通用。

在DTK中，这些HIP库被进一步开发为可以在DCU上运行，领域涵盖了从科学/工程计算中常用的线性代数、快速傅里叶变换、随机数生成，到AI领域常用的卷积、池化等计算，再到异构并行中的通讯、算法等诸多领域用到的函数接口；这些函数接口针对DCU做了专门优化，通过这些函数接口可以实现各种应用场景下对DCU的高效使用。

下表描述了DTK中的库与CUDA平台中的库的对应关系与功能介绍。

表2-1 相关数学库介绍

HIP库	CUDA库	功能
hipBLAS	cuBLAS	基础线性代数库
hipFFT	cuFFT	快速傅里叶变换库
hipRAND	cuRAND	随机数生成库
hipSOLVER	cuSOLVER	矩阵求解库

hipSPARSE	cuSPARSE	稀疏矩阵线性代数库
Thrust	Thrust	C++STL形式的并行算法库
hipCUB	CUB	HIP形式的并行算法库
RCCL	NCCL	多DCU的通讯库
MIOpen	cuDNN	深度学习基础数学库
HIP RTC	NVRTC	运行时编译库
hipBLASLt	cuBLASLt	轻量级矩阵乘法（GEMM）专用
hipDNN	cuDNN	深度神经网络专用加速库

2.1.3 DTK中的工具链

DTK中包含针对DCU开发的LLVM编译器套件，DTK中对于HIP、OpenMP、OpenACC的支持都是由这套编译器套件提供。为方便编译HIP代码，DTK中还提供了脚本工具hipcc，hipcc可以在内部帮助开发者完成预编译器、编译器、链接器的参数配置与调用，使开发者只需要使用简单的编译命令即可完成编译。

DTK中还包含性能分析工具hipprof、调试工具hipgdb、以及DCU状态监控工具rocm-smi。hipprof能在DCU程序运行时抓取一系列性能指标，除统计信息外，还可以让开发者以可视化的方式分析程序的时间线，让开发者能快速找到程序的耗时较多的部分，发现性能瓶颈以开展优化。hipgdb是在GDB的基础上进一步开发，使其能够调试DCU程序，使开发者可以通过熟悉的插入断点，单步执行，打印变量等方式调试DCU程序。rocm-smi能够显示当前计算机中的DCU状态，如DCU个数，每个DCU的功率、内存使用情况和计算单元使用情况，还有查看系统信息的很多选项。

本书中有大量使用hipprof进行DCU程序性能分析的例子；对hipgdb的介绍在附录B中；rocm-smi的简单用法在下节中有介绍。。

2.2 DTK安装与使用

DTK在每年的4月和10月会进行大版本更新，包含各模块的升级、新功能特性的支持和已有问题修复，DTK保证向前兼容，开发者可根据需要切换到新版本以获得更高的运行性能，也可以重新编译DCU程序以获得更好的开发与使用体验。

DTK软件包可以从光合开发者社区（https://developer.sourcefind.cn/）下载，在页面上找到“资源中心”-“DCU
Toolkit”-“下载地址”。在打开的页面中找到需要的DTK版本，选择使用的操作系统，即可下载。

压缩包命名为DTK-版本号-操作系统版本-x86_64.tar.gz，解压后，目录结构如下：

```
├── amdgcn -> dcc/dccgcn
├── bin
├── cuda
├── dcc
```

```
├── dushmem
├── env.csh
├── env.sh
├── hip
├── hipblas
├── hipblaslt
├── hipcub
├── hipdnn
├── hipfft
├── hipprof_utils
├── hiprand
├── hipsolver
├── hipsparse
├── hsa
├── include
├── lib
├── lib64
├── llvm -> dcc
├── miopen
├── modulefile
├── opencl
├── rccl
├── rocalution
├── rocblas
├── rocfft
├── rocprim
├── rocprofiler
├── rocrand
├── rocsolver
├── rocsparse
├── rocthrust
├── roctracer
├── share
└── src
```

通过source根目录下的env.sh（对于bash）或env.csh（对于csh）即可加载DTK的环境。

在使用DCU的集群环境上，集群管理员通常都会安装配置好DTK，在使用Environment Modules软件进行环境变量管理的集群上，通常使用module load命令即可加载使用DTK（注意如果集群预加载了其他版本的DTK，需要使用module unload卸载），如：

```
module load compiler/dtk/26.04
```

使用DTK编译DCU程序时，节点上可以没有DCU，比如说可以在没有安装DCU集群的登录节点编译DCU程序；但要运行DCU程序，节点上就必须安装DCU了，即在使用DCU集群时必须申请安装了DCU的计算节点（涉及如何使用集群的调度系统，本书不过多涉及，请查看集群的使用文档）。

使用上节提到的rocm-smi工具（可以查看节点上的DCU，比如对安装了4个DCU的计算节点：

```
rocm-smi
==============System Management Interface ==========
================================================
DCU  Temp   AvgPwr  Fan   Perf    PwrCap  VRAM%  DCU%
0    49.0c  58.0W   0.0%  manual  450.0W    0%   0%
1    49.0c  59.0W   0.0%  manual  450.0W    0%   0%
2    50.0c  57.0W   0.0%  manual  450.0W    0%   0%
3    49.0c  58.0W   0.0%  manual  450.0W    0%   0%
================================================
==================End of SMI Log==================
```

从左至右依次为设备ID、温度、平均功率、风扇转速、性能级别、DCU最大功率、DCU内存使用率、DCU使用率。当DCU程序执行时，可以使用hy-smi查看DCU内存和DCU的实时使用情况，以判断DCU程序是否充分使用了DCU的硬件资源；在DCU执行计算时，温度、平均功率和风扇转速（对于浸没式液冷的节点风扇转速总是0.0%）也会相应提高。

DTK提供了一系列使用文档，供开发者参考。你可以在https://developer.sourcefind.cn/中DTK的下载页面里选择DTK版本，并找到“Document”目录。以DTK-26.04为例，打开后可以看到如图2-2所示的文档。

本书是使用DTK进行DCU程序开发的教程，而非详尽的使用说明。因此，若需要了解DTK中编程接口的详细说明以及DTK中各种工具的详细使用方法等信息，需要参考这些文档。

[图片]

每天定时更新

文件名

- DTK-26.04
- DTK 26.04 CUDA 程序适配使用手册.pdf
- DTK 26.04 DCC使用手册.pdf
- DTK 26.04 DUMMA使用手册.pdf
- DTK 26.04 DUSHMEM库使用手册.pdf
- DTK 26.04 DuImageCodec使用手册.pdf
- DTK 26.04 HIP C++编程指南.pdf
- DTK 26.04 HIP Runtime API开发手册.pdf
- DTK 26.04 HIP最佳实践手册.pdf
- DTK 26.04 MIOpen库使用手册.pdf
- DTK 26.04 OpenCL使用手册.pdf
- DTK 26.04 OpenMP和OpenACC使用手册.p
- DTK 26.04 RCCL库使用手册.pdf
- DTK 26.04 Release Note.pdf
- DTK 26.04 Rocprofiler API开发手册.pdf
- DTK 26.04 dccobjdump使用手册.pdf
- DTK 26.04 hipBLASLt库使用手册.pdf
- DTK 26.04 hipCUB库使用手册.pdf
- DTK 26.04 hipFFT库使用手册.pdf
- DTK 26.04 hipSOLVER库使用手册.pdf
- DTK 26.04 hipSPARSE使用手册.pdf
- DTK 26.04 hipgdb使用手册.pdf
- DTK 26.04 hipprof使用手册.pdf
- DTK 26.04 rocBLAS库使用手册.pdf
- DTK 26.04 rocPRIM库使用手册.pdf
- DTK 26.04 rocThrust库使用手册.pdf
- DTK 26.04 兼容性手册.pdf
- DTK 26.04 开发环境使用手册.pdf
- DTK 26.04 开发环境安装部署手册.pdf

图2-2 光合开发者社区中的DTK文档（以26.04版本为例）

2.3 DAS人工智能软件系统(DCU AI Software Stack)

2.3.1 产品概述

DAS（DCU AI Software
Stack，DCU人工智能基础软件系统）是针对海光DCU推出的一套完整的人工智能基础软件，包含基础算子层、框架工具层、扩展组件层，与DCU基础硬件系统、DTK开发工具栈、模型空间Model Zoo、光源等形成一套完整的DCU人工智能软硬件生态体系。支持新一代DCU加速卡K100_AI，实现AI的全场景覆盖和高效性能表现，为满足人工智能的快速发展需要，整体发版节奏以月度更新，同时为满足开源开放需求，在开发者社区建立OpenDAS开源空间，加速DCU的人工智能软件生态建设。

依托DAS，支持DCU在人工智能的全领域应用，如自然语言大模型、多模态大模型、MOE大模型、视觉大模型、通用模型、语音模型等，支持模型训练及推理落地，实现快速的DCU人工智能应用迁移、开发及迭代。

2.3.2 产品特点

[图片]

- 全面覆盖：实现人工智能的全场景覆盖，框架组件的全面适配
- 性能高效：支持新一代DCU K100_AI，性能全面超越，达到NV同级别平
- 快速迭代：支持月度发版节奏，满足人工智能行业的快速发展及需求变化
- 开源开放：打造开源开放社区，加速DCU生态建设

2.3.3 产品功能

基础算子层

该层主要提供定制化高级算子。融合算子库lightop，基于torch接入针对DCU深度优化的可融合稀疏计算类算子；算子模板库CK，支持基于DCU以tile级别进行算子编程及优化，依托模板库能够灵活控制DCU行为逻辑，更细粒度的进行自定义算子的编写及性能优化，类比Cutlass；AI编译算子生成，支持通过AI编译的方式进行算子生成，如Triton、XLA以及TVM，通过层级pass实现及优化生成需要的DCU算子。

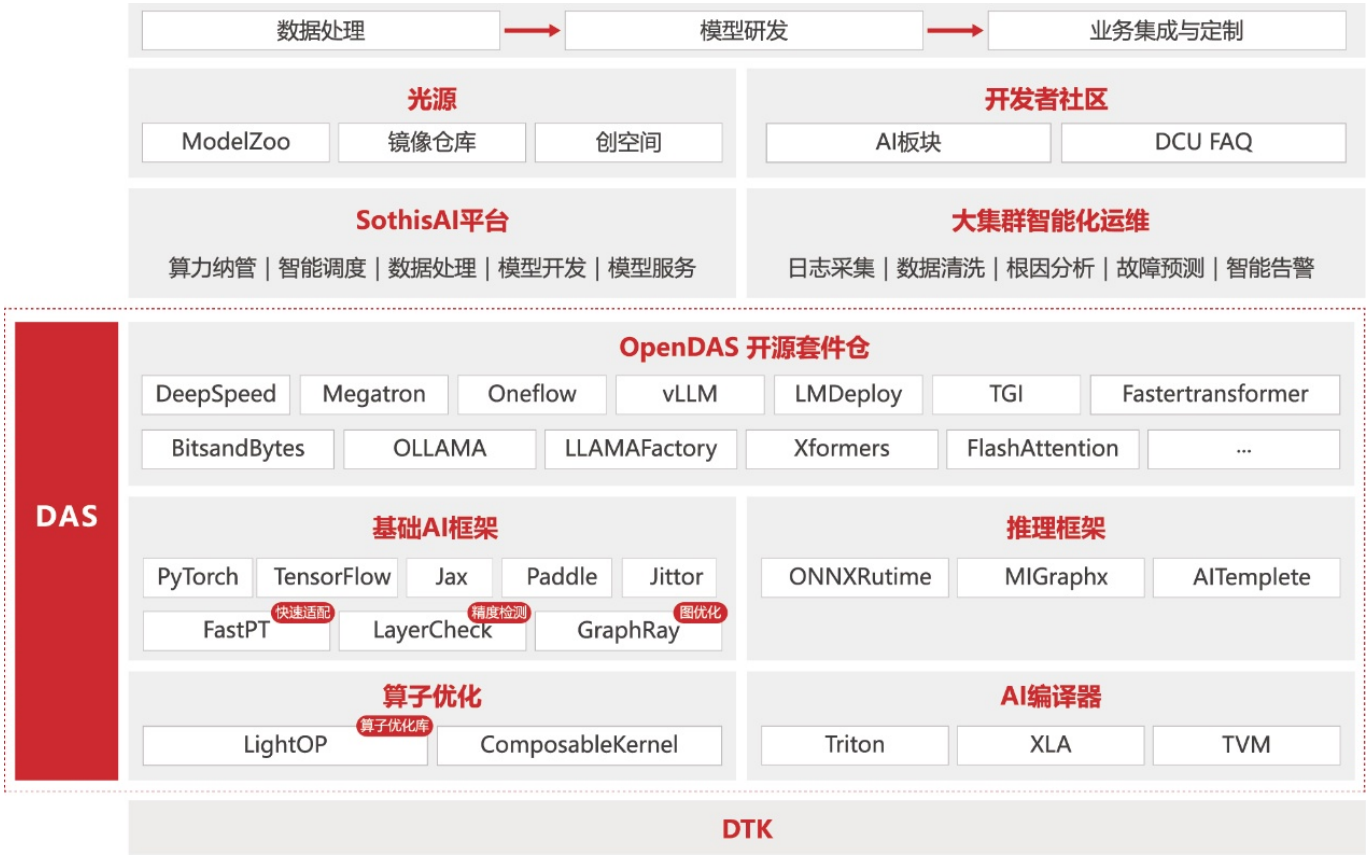
框架工具层

该层主要提供基础深度学习框架及开发工具。AI框架包含PyTorch、TensorFlow、JAX、Paddleh和OneFlow等、通用推理框架包含OnnxRuntime、MIGraphx和AITemplate等，为人工智能开发者提供基础的深度学习框架支撑。针对DCU的快速组件适配工具FastPT，支持基于Torch的组件快速适配；快速精度检测工具LayerCheck，通过单Layer层的精度检查，快速定位精度出问题的层，用于快速的精度检查及定位；图优化组件GraphRay，优化图后端，通过图算匹配快速找到最优的图执行路线，优化模型性能。

扩展组件层

该层主要由通用训练组件、大模型训练组件及大模型推理组件组成，其中通用训练组件包含支持DCU的Apex、Vision、Audio、Mmcv和Dgl等组件，大模型训练组件包含DeepSpeed、FastMoe、Bitsandbytes、FlashAttention、Megatron-DeepSpeed等组件，大模型推理组件包含FlashDecoding、vLLM、Xformers、TGI和Lmdeploy等组件，以上组件均基于DCU进行了适配及优化，其中部分组件已经在OpenDAS社区进行开源。

[图片]



2.3.4 软件优化能力

AI无论是通用模型还是新一代大模型场景，性能优化一直是其主要发展的方向之一，DAS在多个维度做了优化工作，构建起全栈的AI优化能力，包含基础算子优化能力、AI框架组件支撑优化能力、系统工程优化能力和基于模型的性能优化能力，整合了内部AI的系列基础软件及优化策略，应对AI的快速发展，进行快速更新迭代策略，使用者能够快速上手使用。

算子层面

引入BLAS，DNN，RCCL等基础算子库进行计算和通信的优化，对于访存密集型算子，通过大量的手写融合算子提升性能，构建了lightop算子库；为了更加适应更灵活的开发及高级开发者，引入了CK算子模板库，支持tensor
core指令，通过细粒度的tile级别运算模板化，提升算子性能，给高阶开发者提供了更灵活的算子开发模板，在AI编译器层面，整合了triton，XLA和TVM等AI编译组件用于算子生成，为flashattention、xformer、bitsandbytes等组件提供AI编译路线支持，为算子开发提供了更多的开发方式，最大化的利用硬件资源。

框架层面

实现了国内外框架的全面覆盖，包含Pytorch/tensorflow/Jax/Paddle/oneflow/jittor等国内外主流人工智能框架，为了满足更快的三方组件适配，开发了FastPT组件达到原生代码直接编译适配的能力，开发了基于模型层的验证工具LayerCheck，能够快速定位计算精度问题，同时还引入了动态及静态图优化能力，提供融合算子及ai编译等多路线最大化的使能硬件性能，同时还构建OpenDAS开源社区，其中包含了大量的经过适配优化的AI三方组件提供给高阶开发用户；

系统工程优化能力层面

针对通用模型训练推理、大模型训练推理的不同计算访存特点，组合不同的优化手段，构建系统工程优化能力，以大模型训练为例，一方面适配deepspeed、megtatron等分布式框架，另一方面采用topo优化、多维组合并行、激活重计算等策略，实现大模型场景的全面覆盖，在大模型推理方向，在flashattention、flashdecoding、kv
cache、pageattention等方向进行了针对性的适配优化，降低首字延时提高吞吐能力。

模型层面

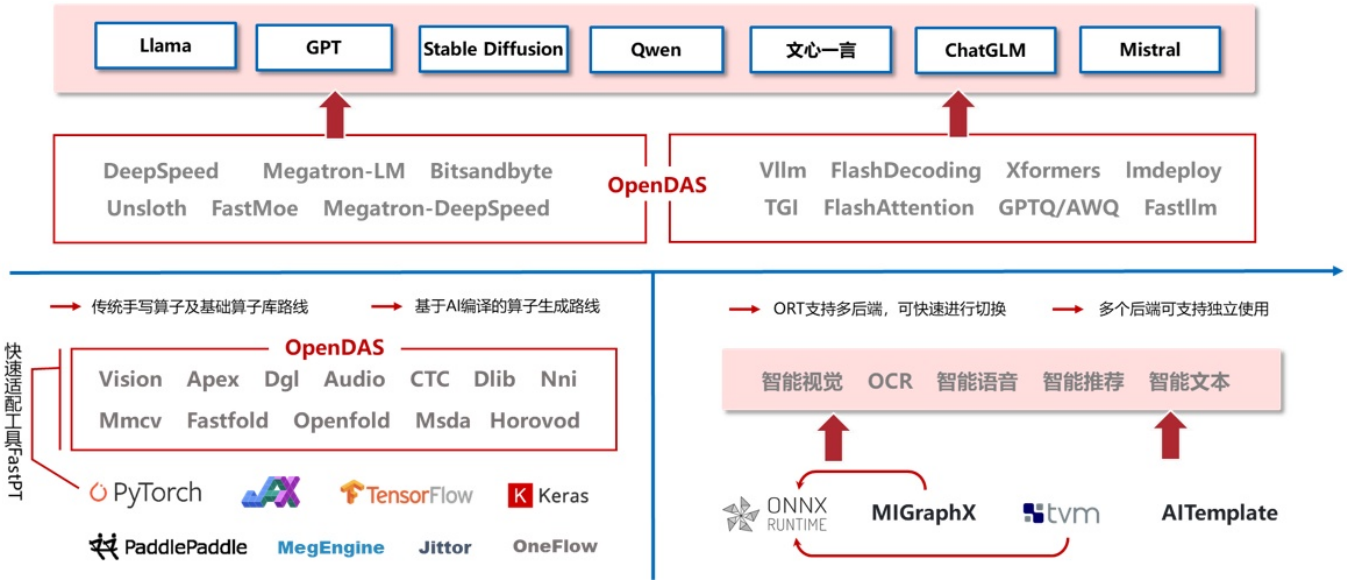
整合压缩、量化和剪枝蒸馏等技术，减少训练推理过程中的计算访存开销，在模型层进行性能优化。

2.3.5 OpenDAS

OpenDAS是一个源码仓库，提供适配DCU环境的第三方组件库源码，为高阶用户提供基于DCU的三方组件适配指导及自定义算子开发指导，支持自主编译安装。包括Vision、Audio、DeepSpeed、Megatron-LM、FastLLM等常用训练推理组件，提供更加灵活的开发模式，共同推进DCU的软件生态建设。

OpenDAS的地址为:
https://developer.sourcefind.cn/codes/OpenDAS，在使用过程中存在任何问题可以在对应仓库提交issue，会有专业的工程师第一时间解答。

[图片]



第3章 使用C/C++编写DCU程序

在C/C++中使用HIP编程接口编写程序是最主要的DCU程序开发方式，本章将从最简单的并行编程出发，逐步介绍如何开发高效的DCU程序。

3.1 初识并行计算

3.1.1 从一个串行程序开始

我们首先编写一个简单但有一些计算量的程序（quadFunc目录下cpu_origin.cc）。假设我们想对2G（$2×1024^{3}$）个x计算二次函数$y=ax^{2}+bx+c$，为此编写代码：

```
#include <cstdio>
#include <cstdlib>
int main()
{
  int N = 256 * 1024 * 1024;   // 数组长度256M
  int size = N * sizeof(float); // 内存空间1GB
  // 分配内存空间
  float *x = (float *) malloc(size);
  float *y = (float *) malloc(size);
  // 初始化a,b,c
  float a = 2.5, b = 2.0, c = 1.0;
  // 循环8次完成
  for (int j = 0; j < 8; j++) {
    // 初始化x
    for (int i = 0; i < N; i++) {
```

```
            x[i] = 2.0;
        }
        // 计算y = ax^2 +  bx + c;
        for (int i = 0; i < N; i++) {
            y[i] =  a * x[i] * x[i] + b * x[i] + c;
        }
        // 检查结果
        bool correct = true;
        for (int i = 0; i < N; i++) {
            if (y[i] != 15.0) {
                correct = false;
                break;
            }
        }
        if (correct) {
            printf("Loop %d: Correct results.\n", j+1);
        } else {
            printf("Loop %d: Wrong results.\n", j+1);
        }
    }
    // 释放空间
    free(x);
    free(y);
    return 0;
}
```

在上面的代码中，为了避免使用过大的内存，我们将从2G个x计算2G个y的流程分8次完成，每次处理256M个。我们首先通过malloc函数分配两个长度为256M的数组x和y（内存空间为$2×256M×4B=2GB$），之后初始化参数a、b、c。之后就进入大循环，在每个大循环中，首先初始化数组x，这在实际问题中可能是从文件中读取，或是以某种方式计算，但简单起见在这里我们让x中的256M个数全都等于2.0，之后计算这些数的二次函数，并将结果保存在数组y中，我们知道y中的所有数都应该等于15.0并据此来检验计算机是否给出了正确的结果。我们将这个大循环执行8次。最后我们释放了数组所占的内存。这是一个串行程序，计算机会使用一个CPU核顺序的执行上述过程。

我们用GNU编译器编译并运行，为了让程序执行得更快，让编译器以O3的级别优化：

```
g++ cpu_origin.cc -o cpu_origin -O3
./cpu_origin
```

在我的计算机上，逐行打印出了以下结果，每打印一行凭感觉用时不到一秒：

```
Loop 1: Correct results.
Loop 2: Correct results.
Loop 3: Correct results.
Loop 4: Correct results.
Loop 5: Correct results.
Loop 6: Correct results.
Loop 7: Correct results.
Loop 8: Correct results.
```

看来我们确实给了计算机施加了一定的压力。现在我们想知道计算机完成上面的工作具体用了多长时间，为此我们需要编写一个计时器。

> 题外话：优化代码对于现代编译器来说是非常重要的工作，想体会这一点，请尝试将编译命令中的-O3选项去掉（GNU编译器默认不优化）重新编译并运行。

3.1.2 为程序计时

我们创建一个名为timer.h的头文件，输入以下代码：

```
#ifndef TIMER_H
#define TIMER_H
#include <sys/time.h>
struct my_timer
{
  timeval ts, te; //起始时刻，终止时刻
  float dt; // 时间间隔,单位毫秒(ms)
  void start(){
    gettimeofday(&ts, NULL);
  }
  void stop(){
    gettimeofday(&te, NULL);
    long int dt_sec  = te.tv_sec - ts.tv_sec;
    long int dt_usec = te.tv_usec - ts.tv_usec;
    dt = dt_sec * 1.0e3 + dt_usec / 1.0e3;
  }
};
#endif
```

在上面的代码中，我们定义了一个简单的类my_timer，通过成员函数start()记录起始时刻，stop()记录终止时刻并计算时间间隔。代码中利用了sys/time.h头文件中的timeval类，其中包含以秒为单位成员变量的tv_sec和以微秒（μs）为单位的成员变量tv_usec。时间间隔dt是以终止时刻减去起始时刻，并以毫秒（ms）为单位。

现在我们可以用my_timer进行计时了，include头文件timer.h，定义一个计时器timer：

```
  // 计时器
  my_timer timer;
  double t_init = 0.0;
  double t_calc = 0.0;
  double t_chck = 0.0;
```

对初始化，计算和结果检验分别计时：

```
  for (int j = 0; j < 8; j++) {
    // 初始化x
    timer.start();
    for (int i = 0; i < N; i++) {
```

```
        x[i] = 2.0;
    }
    timer.stop();
    t_init += timer.dt;
    // 计算y = ax^2 +  bx + c;
    timer.start();
    for (int i = 0; i < N; i++) {
        y[i] =  a * x[i] * x[i] + b * x[i] + c;
    }
    timer.stop();
    t_calc += timer.dt;
    // 检查结果
    timer.start();
    bool correct = true;
    for (int i = 0; i < N; i++) {
        if (y[i] != 15.0) {
            correct = false;
            break;
        }
    }
    if (correct) {
        printf("Loop %d: Correct results.\n", j+1);
    } else {
        printf("Loop %d: Wrong results.\n", j+1);
    }
    timer.stop();
    t_chck += timer.dt;
}
```

并在大循环结束后输出这些时间：

```
printf("Initialization took %8.3f ms.\n", t_init);
printf("Calculation    took %8.3f ms.\n", t_calc);
printf("Validation     took %8.3f ms.\n", t_chck);
```

编译并运行（cpu_timer.cc）：

```
g++ cpu_timer.cc -o cpu_timer -O3
./cpu_timer
```

这次屏幕上显示了程序各部分的计时：

```
Initialization took  868.988 ms.
Calculation    took 1266.479 ms.
Validation     took 1909.695 ms.
```

这次我们有了比较精确的计时，程序的大循环部分总计耗时约4秒。这是个不错的结果，毕竟我们让计算机处理了2G个数据，但如果我们还想让程序运行地更快，需要怎样去做呢？

3.1.3 使用多个CPU核

现在CPU早已进入了多核时代，你所用的PC或服务器的配置单上一定会标注你的CPU是多个核的，而在上面的例子中我们只用了一个核心，现在要问的是，多个核能加速上面的程序吗？

观察我们的代码，很快我们发现对于初始化和计算二次函数这两个循环来说，虽然在刚才我们使用一个CPU核心时，循环需要一次接一次的做，但其实每次循环都是独立的，即第i次循环只会负责x[i]和y[i]，而不用处理比如说x[i+1]。在这种情况下，我们用n个CPU核心，每个核负责处理对i循环中的N/n次循环，因为这些核心彼此不必等待，可以让它们同时工作起来，期望速度会提升n倍。

打个比方，好比1千米的路，我们要在路边每隔10米种一颗树，我们让一位工人从头到尾种这100棵树，但这太慢了，所以我们决定多找几位工人来帮助他，比如说再找3位工人。我们给工人们指派任务后他们就开工了，工人A走到10米的位置种第一棵树、工人B走到20米的位置种第二棵树，工人C和D也分别在30和40米的位置开始工作；工人A种完第一棵树之后，走到50米的位置开始种下一颗树，差不多同时，工人B也走到60米的位置继续工作，以此类推。从每个工人的角度，他的同伴正在做什么对他来说并不影响，他们各自开着自己运树苗的车，拿着自己的铁锹和水桶。假设这几位工人的工作效率差不多，我们可以预期他们将以4倍于一个人的速度种完这100棵树。这种分工与使用多CPU核进行并行计算是同一思路，对于我们的程序来说，我们用多个线程（thread，相当于上面比喻中的工人），每个线程使用一个CPU核心（工人自身的劳动力以及工具）同时做计算（种树）。

> 思考：这种分工方式有什么好处（上面的工作，还可以让每个工人各负责一整段250米）。
> 提示：如果这段路变成了1.15千米会怎么样。

有了这个思路后，现在的问题是怎样让我们的程序使用多个CPU核，怎样告诉它们去分工。这里我们可以通过OpenMP简单地实现多核并行，OpenMP是一套基于共享内存模型的多线程编程接口，在后面我们还会专门介绍它，现在的使用方法是在初始化和计算两个for循环上面添加一句（以初始化的for循环为例）：

```
#pragma omp parallel for
for (int i = 0; i < N; i++) {
  x[i] = 2.0;
}
```

这是一句OpenMP指导语句，用来告诉编译器将下面的for循环并行执行，我们重新编译一下（cpu_omp.cc），这次我们需要加上选项-fopenmp，让编译器编译时考虑这些指导语句：

```
g++ cpu_omp.cc -o cpu_omp -O3 -fopenmp
```

我们配置一个环境变量OMP_NUM_THREADS为2，告诉计算机用两个线程执行OpenMP的并行部分，现在运行，如果这时你有两个空闲的CPU核心，就会让它们都开始工作：

```
export OMP_NUM_THREADS=2
./cpu_omp
```

这次的计时是：

```
Initialization took  429.644 ms.
Calculation   took  640.189 ms.
Validation    took 1915.407 ms.
```

可以看到初始化和计算的耗时缩短了一半。在我的计算机上有足够的CPU核心，可以进一步设置OMP_N

UM_THREADS=4，再次运行：

```
Initialization took  278.735 ms.
Calculation   took  435.696 ms.
Validation    took 1939.763 ms.
```

这次耗时是单个核心的三分之一，虽然没达到4倍的加速，但看来多个CPU核心并行计算确实有用。这时你可能想，如果我有更多的核心（设想一下成千上万个）会不会更快呢？

> 思考：现在花在检查结果的时间最长了，请尝试对它也使用并行加速。

3.1.4 为DCU编写代码

DCU是专为并行计算设计的，此刻你可以认为它是有几千个核心的处理器。但是目前的程序不会自动使用DCU，我们需要对程序做一些修改。在本章，我们介绍使用HIP编程接口为DCU编程，我们也可以通过OpenMP来使用DCU，在后文有单独章节介绍。

现在开工，首先DCU有自己的内存，和CPU使用的内存（下文称主存）是不一样的，所以需要对DCU的内存进行管理，为此我们在代码中添加以下内容：

```
  float *dx, *dy;
  hipMalloc(&dx, size);
  hipMalloc(&dy, size);
```

dx和dy分别是数组x和y在DCU内存中的对应，我们给它们分配主存上同样大小的内存空间，但这一次我们使用hipMalloc()而非malloc()。

为了给DCU编程，我们需要在代码中使用一些DCU专用的函数（专用名词：运行时应用编程接口，runtime
API，得名于这些编程接口的作用是与运行时环境进行交互），以hip前缀开头，包含在一个名为hip_runtime.h的头文件中，hipMalloc是其中之一，下面我们还会用到其他的。记得在文件开头加一行：

```
#include <hip/hip_runtime.h>
```

现在到初始化数组x的for循环了，但是DCU是无法直接执行这个for循环的，我们得为它专门写一个函数：

```
__global__ void init(float *x, int N)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < N) {
    x[i] = 2.0;
  }
}
```

这个函数在DCU编程中被称为核函数，它前面有一个修饰符__global__，表示这是可以被CPU调用，并在DCU上执行的函数。

观察一下这个函数，你会发现上面的for循环不见了，只保留了循环终止的判断i，i也不再从0开始递增，变成了用几个没有定义的变量计算（blockIdx.x、blockDim.x以及threadIdx.x）得到。回忆一下，刚才

我们说过使用多核CPU做并行计算时，是通过多个线程，每个线程使用一个CPU核心的方式同时计算。但我们编程的时候并没有遇到线程这个概念，只是在运行之前设置了一个OMP_NUM_THREADS的线程数，这是因为OpenMP让编译器帮助你做了很多工作，现在我们得自己来做这些。

DCU上能用更多的线程，这些线程是分块来管理的，称为线程块，blockIdx.x是线程块的索引（即编号），blockDim.x是线程块的大小，threadIdx.x是每个线程块内线程的索引，这些都是DCU的内置变量，因此你不用定义就可以使用（看到后面的.x你应该就能猜到线程块可以是多维的，我们会在后面提到），DCU在执行核函数之前，会为每个线程唯一确定这些变量。这时，我们看第一个线程块（blockIdx.x=0）的线程，对于每个线程来说都有一个私有的i，就等于线程索引threadIdx.x，比如索引是0的线程，它的i是0；索引是10的线程，它的i是10。假设线程块的大小是256（blockDim.x=256），对于第二个线程块（blockIdx.x=1）中索引为0的线程，i是256；索引是15的线程，它的i是271，以此类推。每个线程负责把数组x的一个元素x[i]写成2.0。所以方式还是一样，每个线程使用一个DCU核心完成这N次循环中的一次，一大堆线程同时开始工作，直到把整个数组初始化完。当然我们保留了判断（i），以此避免有线程对超过数组边界的内存写入数据。

现在我们调用这个核函数，这也需要特殊的写法：

```
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;
    // 初始化x
    timer.start();
    init<<<numBlocks, blockSize>>>(dx,N);
    timer.stop();
    t_init += timer.dt;
```

我们按上面的想法，把线程块blockSize定为256，线程块的个数numBlocks通过代码中的公式确定，用以保证在N无法被blockSize整除时总的线程数大于N。调用函数时init时，在函数后加上由三对尖括号扩起来的线程块个数和线程块大小，最后再跟上函数的参数。这是在程序中调用在DCU上运行的核函数的固定写法。

> 不必担心总线程数256M是否过大，我们在下一节会介绍DCU启动线程的方式。但注意线程块大小（blockSize）是不能大于1024的。

按照相同的方式，我们再写一个负责计算二次函数的核函数，将另一个for循环替换掉：

```
__global__ void calculate(float *x, float *y, float a,
                          float b, float c, int N)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < N) {
    y[i] = a * x[i] * x[i] + b * x[i] + c;
  }
}
```

并调用：

```
    calculate<<<numBlocks, blockSize>>>(dx,dy,a,b,c,N);
```

至此，我们可以检查结果了，我们暂时还是让CPU串行的来做，但问题是刚才提到了，DCU内存和计算机主存是不同的，虽然我们在DCU有了结果，但CPU却还不知道，因此我们需要将DCU内存上的数组dy的内容传递到主存上的数组y，通过以下编程接口：

```
    hipMemcpy(y, dy, size, hipMemcpyDeviceToHost);
```

我们把它包含在检查结果的计时范围之内（毕竟如果不需要检查结果也就用不着传输了）。再检视一下整个代码，我们还需要在程序最后释放掉DCU内存中的dx和dy（虽然在这里程序马上就结束了，释放并不重要，但用好的编程习惯规范地编程会让我们少犯很多错误）：

```
  hipFree(dx);
  hipFree(dy);
```

另外我们还发现其实有了数组dx之后，x也不再需要了，我们将对x的malloc和free都删去，大功告成！

方便起见，将DCU版代码完整展示如下（dcu.cc）：

```
#include <cstdio>
#include <cstdlib>
#include "timer.h"
#include <hip/hip_runtime.h>
__global__ void init(float *x, int N)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < N) {
    x[i] = 2.0;
  }
}

__global__ void calculate(float *x, float *y, float a,
                  float b, float c, int N)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < N) {
    y[i] = a * x[i] * x[i] + b * x[i] + c;
  }
}

int main()
{
  int N = 256 * 1024 * 1024;   // 数组长度256M
  int size = N * sizeof(float); // 内存空间1GB
  // 分配内存空间
  float *y = (float *) malloc(size);
  float *dx, *dy;
  hipMalloc(&dx, size);
  hipMalloc(&dy, size);
  // 计时器
  my_timer timer;
  double t_init = 0.0;
  double t_calc = 0.0;
  double t_chck = 0.0;
  // 初始化a,b,c
  float a = 2.5, b = 2.0, c = 1.0;
```

```
    // 循环8次完成
    for (int j = 0; j < 8; j++) {
        // 初始化x
        int blockSize = 256;
        int numBlocks = (N + blockSize - 1) / blockSize;
        timer.start();
        init<<<numBlocks, blockSize>>>(dx,N);
        timer.stop();
        t_init += timer.dt;
        // 计算y = ax^2 +  bx + c;
        timer.start();
        calculate<<<numBlocks, blockSize>>>(dx,dy,a,b,c,N);
        timer.stop();
        t_calc += timer.dt;
        // 检查结果
        timer.start();
        bool correct = true;
        hipMemcpy(y, dy, size, hipMemcpyDeviceToHost);
        for (int i = 0; i < N; i++) {
            if (y[i] != 15.0) {
                correct = false;
                break;
            }
        }
        if (correct) {
            printf("Loop %d: Correct results.\n", j+1);
        } else {
            printf("Loop %d: Wrong results.\n", j+1);
        }
        timer.stop();
        t_chck += timer.dt;
    }
    printf("Initialization took %8.3f ms.\n", t_init);
    printf("Calculation    took %8.3f ms.\n", t_calc);
    printf("Validation     took %8.3f ms.\n", t_chck);
    // 释放空间
    hipFree(dx);
    hipFree(dy);
    free(y);
    return 0;
}
```

3.1.5 编译运行DCU程序

现在该编译我们的DCU版代码了（dcu.cc），GNU编译器是无法完成的，我们需要DTK软件栈中的hipcc工具，这次不需要-O3因为对hipcc这是默认的：

```
hipcc dcu.cc -o dcu
```

我们运行程序，得到以下结果：

```
Initialization took   1.571 ms.
Calculation   took   0.070 ms.
Validation    took 3387.858 ms.
```

这个结果看起来很棒，我们的计时器显示除了验证结果的耗时增长了不少之外，初始化和计算的部分几乎都不耗时了。但我必须告诉你，这里我们的计时器显示的结果是有问题的。我们用DTK软件栈中的性能分析工具来看看真实的性能，使用命令：

```
hipprof ./dcu
```

这次屏幕的最后多了两张表格（图3-1）表格里记录了我们调用的HIP编程接口的耗时以及DCU核函数的耗时，结果是以纳秒（ns）为单位的，我们换算一下，数据传输hipMemcpy总耗时1.8秒，核函数init和calculate耗时都是约34毫秒。根据这些真实的时间，单从核函数和for循环对比，相比单核CPU分别提速了大约25倍和37倍。

[图片]

HIP PROF:hip API statistics

Name	Calls	TotalDurationNs	AverageNs	Percentage
hipMemcpy	8	1771907091	221488386	99.8761067754043
hipLaunchKernel	16	1504923	94057	0.0848271622142077
hipMalloc	2	431050	215525	0.0242967568921694
hipFree	2	224920	112460	0.0126779412137495
__hipPushCallConfiguration	16	25443	1590	0.00143413150587511
__hipPopCallConfiguration	16	11660	728	0.000657232769661744
Total	60	1774105087	29568418	100.0

HIP PROF:hip kernel statistics

Name	Calls	TotalDurationNs	AverageNs	Percentage
calculate(float*, float*, float, float, float, int)	8	34129570	4266196	50.0973505213397
init(float*, int)	8	33996927	4249615	49.9026494786603
Total	16	68126497	4257906	100.0

图3-1 hipprof统计的耗时

> 你可能觉得传输的时间也对不上，但这是因为编译器不同造成的，hipcc同样可以编译CPU版的代码（使用clang编译器），编译后运行的检查结果耗时大约1.6秒，DCU版多了传输时间1.8秒，总耗时约3.4秒。

3.1.6 核函数执行是异步的

现在你可能会问，我们的计时器为什么不准呢？这是因为DCU执行核函数与CPU执行其他部分是异步的

。回到我们的代码：

```
int blockSize = 256;
int numBlocks = (N + blockSize - 1) / blockSize;
// 初始化x
timer.start();
init<<<numBlocks, blockSize>>>(dx,N);
timer.stop();
t_init += timer.dt;
```

上面这些代码其实都是由单个CPU核串行执行的，首先设置两个变量存储线程块的大小和线程块的个数，让计时器记下初始时刻。随后调用核函数init，CPU会执行一系列操作，把init函数的相关信息打包发送给DCU。此时CPU就不再管init函数了，完全由DCU独立执行，CPU会继续往下，让计时器记录终止时刻并计算时间间隔。

说的更清楚一点，如果CPU执行和DCU核函数执行是同步的，CPU会等待DCU核函数执行完毕后，再继续执行timer.stop()；但实际情况是异步的，CPU把任务交给DCU之后，不待DCU运行完毕就直接继续执行timer.stop()了。这就解释了为什么我们的计时器不准，因为它实际记录的是CPU把核函数init打包交给DCU的时间，这段耗时相比DCU执行核函数是很短的。

为了让计时器记录准确的核函数执行时间，我们需要在timer.stop()之前让CPU与DCU同步，通过调用HIP运行时编程接口hipDeviceSynchronize()：

```
timer.start();
init<<<numBlocks, blockSize>>>(dx,N);
hipDeviceSynchronize();
timer.stop();
```

当CPU执行到这个接口时会等待DCU上的现有的任务执行完毕后再继续。对于核函数calculate我们同样加上hipDeviceSynchronize()，用hipcc重新编译并运行，这次的计时是正确的：

```
Initialization took   35.847 ms.
Calculation    took   34.363 ms.
Validation     took 3321.971 ms.
```

在介绍了上面的异步问题之后，你可能会担心，比如会不会DCU内存没有申请完就开始执行核函数；核函数init没有执行完，数组dx还有一部分没有初始化就开始执行核函数calculate；calculate还没执行完就开始执行dy到y的拷贝；dy中的内容还没有完全拷贝到y中就开始检查结果。

对于目前的代码，不需要有这些顾虑，因为hipMalloc、hipFree、hipMemcpy这几个编程接口都是同步的，它们会等待DCU上现有的任务全部执行完再执行，并且要等它们自身的操作（分配/释放内存、拷贝）执行完成后才允许CPU继续执行后面的代码。以hipMemcpy为例，即使我们没有改进我们的计时器（没有在calculate后面加hipDeviceSynchronize接口），当程序执行到hipMemcpy时也会等待calculate执行完再执行从DCU上的dy到主存中的y的拷贝，并且会在这个拷贝执行完成后才进入检验y的每个元素是否等于15.0的for循环（因此我们的计时器对检查结果部分的计时*基本*正确）。此外，DCU会顺序执行核函数，先执行完init之后再执行calculate。

但需要指出，DCU编程有很大的自由度，我们可以让拷贝异步的执行，也可以让多个核函数彼此异步的执行，这些是3.4节中的内容。

> 思考：请对比核函数calculate后加hipDeviceSynchronize对hipMemcpy耗时的影响并思考原因。

3.1.7 小节

在本节中我们编写了一个包含大量二次函数计算的程序，并使用单个CPU核运行。我们尝试用并行计算对程序进行加速，首先通过OpenMP使用了多个CPU核，之后修改了代码使得程序能使用DCU。在这个过程中，介绍了并行计算的思路，初步尝试了OpenMP编程和DCU编程，并测试了加速效果。

在传统计算机上，一个程序的所有指令都是由单个CPU核心执行的，但在现代计算机上，常常会根据指令进行专门化的分工，将原本由CPU处理的部分工作卸载（offload）给其他硬件或硬件单元。这些专门设计的硬件或硬件单元在处理这部分工作时相比CPU往往有更快的性能，卸载这部分工作之后，还能使CPU继续处理其他工作。对于我们上面的例子而言，通过OpenMP使用多个CPU核其实就是把单个CPU核的两个for循环部分卸载给其他CPU核心，而使用DCU则是直接将CPU的对两个for循环的计算全部卸载给DCU。另一个CPU工作卸载的例子是CPU与DCU之间的数据传输，这类工作通常也不会使用CPU执行，而是卸载给DCU上被称为DMA（Direct Memory Access，直接内存访问）的专用硬件单元。

DCU是专为并行计算设计的硬件。如本节中的DCU版代码所示，要使用DCU，或者说让CPU将程序中适合并行计算的部分卸载给DCU来执行，有两个方面的工作要做，其一是使用以hip为前缀的DCU编程接口，其二就是编写DCU核函数让DCU执行计算。

3.2 DCU编程总览

在上一节中，我们对并行计算做了简要的介绍，并初步尝试了DCU编程，你现在应该对一个DCU程序的代码样式有所了解了。为了能更有效的使用DCU做并行计算，需要对DCU编程做一个系统性的概述。首先介绍DCU编程模型，这里的编程模型是指对计算机系统本身以及计算机如何执行程序的抽象，特别是对DCU硬件以及DCU如何执行核函数；之后简要介绍从编程模型到DCU硬件的映射关系。

3.2.1 异构编程模型

我们的程序主体是用CPU执行的，在程序的某些阶段，CPU会把一些工作交给（卸载）DCU执行，因而我们的程序是用两种处理器执行的，这称为异构计算（heterogeneous computing）。CPU和DCU的关系是主从关系，因此在后文我们会用一个抽象的概念主机（host）来指代CPU，设备（device）来指代DCU。主机使用主存，设备有单独的设备内存。在一台计算机上可以有多个主机与多个设备相互连接（从硬件讲是通过PCIe总线），可以多个主机使用一个设备，也可以一个主机使用多个设备。

主机通过调用编程接口来操作设备，如在多个设备时选择使用的设备；为存储在设备内存上的数据分配/释放/清空设备内存；发起主机与设备、单个设备上、设备与设备间的数据拷贝等等。

在前面的例子中已经使用了三个DCU编程接口：hipMalloc、hipMemcpy和hipFree，它们从名称和参数上都尽量做的与C语言标准库函数malloc、memcpy和free相似。这是为了让熟悉C语言的开发者快速上手。后面我们还会用到更多的接口以实现更复杂的功能。

还可以在主机端调用核函数，让设备执行核函数进行并行计算。对于设备端执行的核函数，我们需要单独编写代码，并在核函数前加__global__修饰符以区分，核函数中还可以调用在设备端执行的函数，这些函数前需要加__device__修饰符。主机调用核函数后会继续执行后面的主机端代码，而设备端则会根据配置启动多个线程同时执行核函数。

仍以我们上节中的核函数init自身实现及其调用为例，

```
// 实现
__global__ void init(float *x, int N)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < N) {
    x[i] = 2.0;
  }
}

// 在main中的调用
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;
    init<<<numBlocks, blockSize>>>(dx,N);
```

我们在main函数中调用核函数init时，设置了线程块的个数为1M，每个线程块中的线程数为256，它们合起来被称为核函数启动参数（Kernel launch configuration），我们通过将它们置于核函数后的三个尖括号之中，让设备以这样的线程组织结构，启动256M个线程，每个线程都执行一遍init函数，换言之将init函数执行了256M次。

设想一下如果把核函数中的x[i]改成x[0]，则调用核函数会对x[0]赋值256M次，每次都赋值为2.0，这当然是冗余且无意义的；这里的关键是每个设备线程有不同的i，每个线程执行一遍核函数会把数组的不同元素x[i]赋值为2.0，核函数执行256M次就会把长度为256M的数组的所有元素初始化为2.0。即利用对于每个设备线程各不相同的内置变量（blockIdx、blockDim、threadIdx等）作为索引，控制所有线程同时独立地做不同的事。

设备的线程具有层次结构，一组线程可以构成线程块，一组线程块可以构成线程网格，它们由核函数启动参数配置，设备会根据启动参数在核函数启动时给每个线程设置内置变量，包括线程网格包含的线程块数目（gridDim）、线程块在线程网格中的编号/索引（blockIdx）、线程块包含的线程数（blockDim）、线程在线程块中的编号/索引（threadIdx）。这些维度和索引都是三维的，以适应程序中的二维、三维问题。它们有专门的数据类型dim3，可以用1到3个参数来初始化（未指定的维度默认为1），比如我们想设置16×16的二维线程块，可以声明如下：

```
dim3 dimBlock(16,16);
```

如果想修改，可以：

```
dimBlock = dim3(256,1,1);
```

[图片]

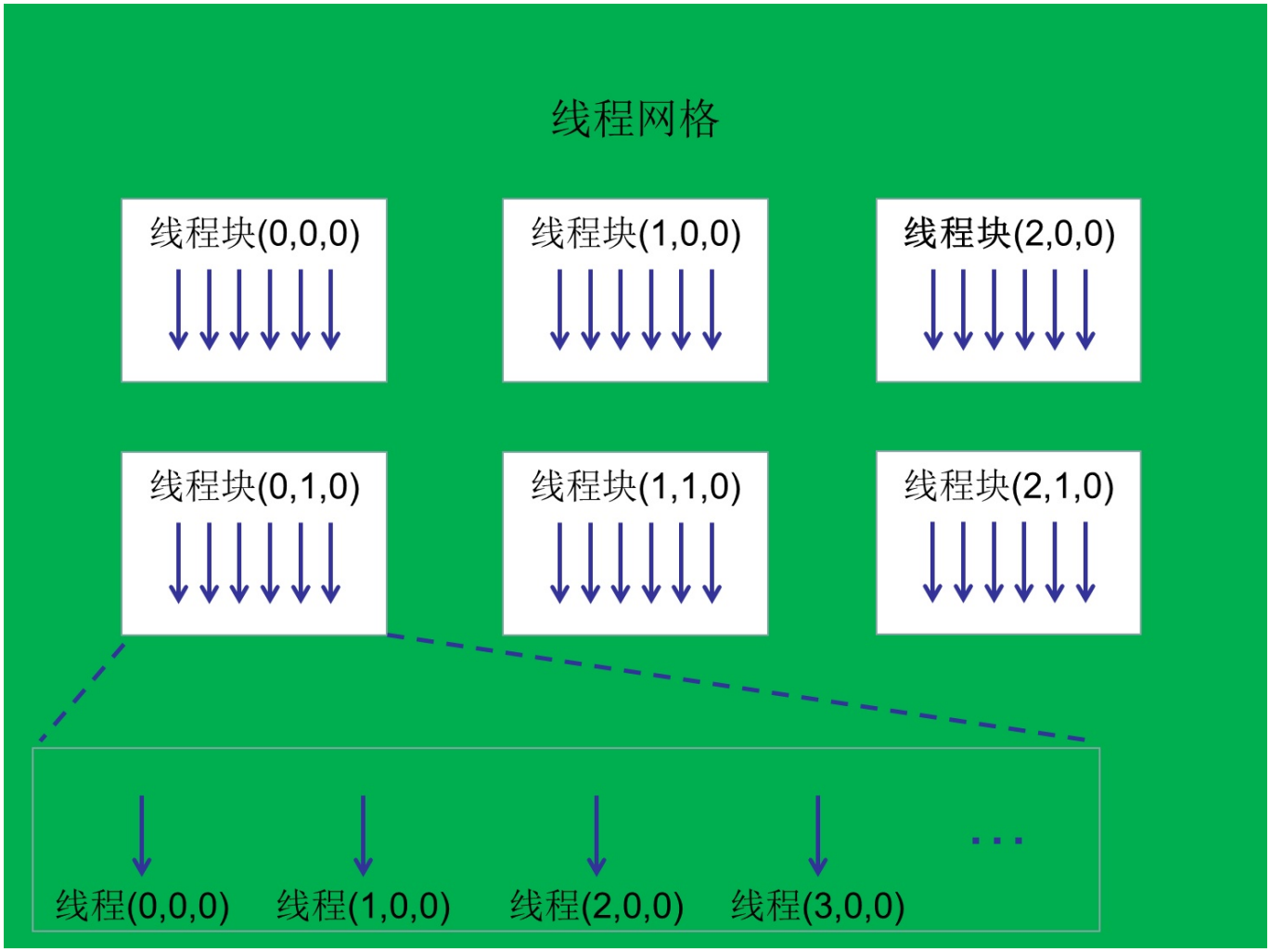


图3-2 线程层次结构

每个线程块最多包含1024个线程，一个线程块内的所有线程可以同步。不同线程块之间则是彼此独立的。

对应线程层次结构，设备内存也具有层次结构，每个线程有私有的局部内存；同一个线程块中的线程有共享内存（变量前加修饰符__shared__）；所有线程都可以访问同一全局内存。

[图片]

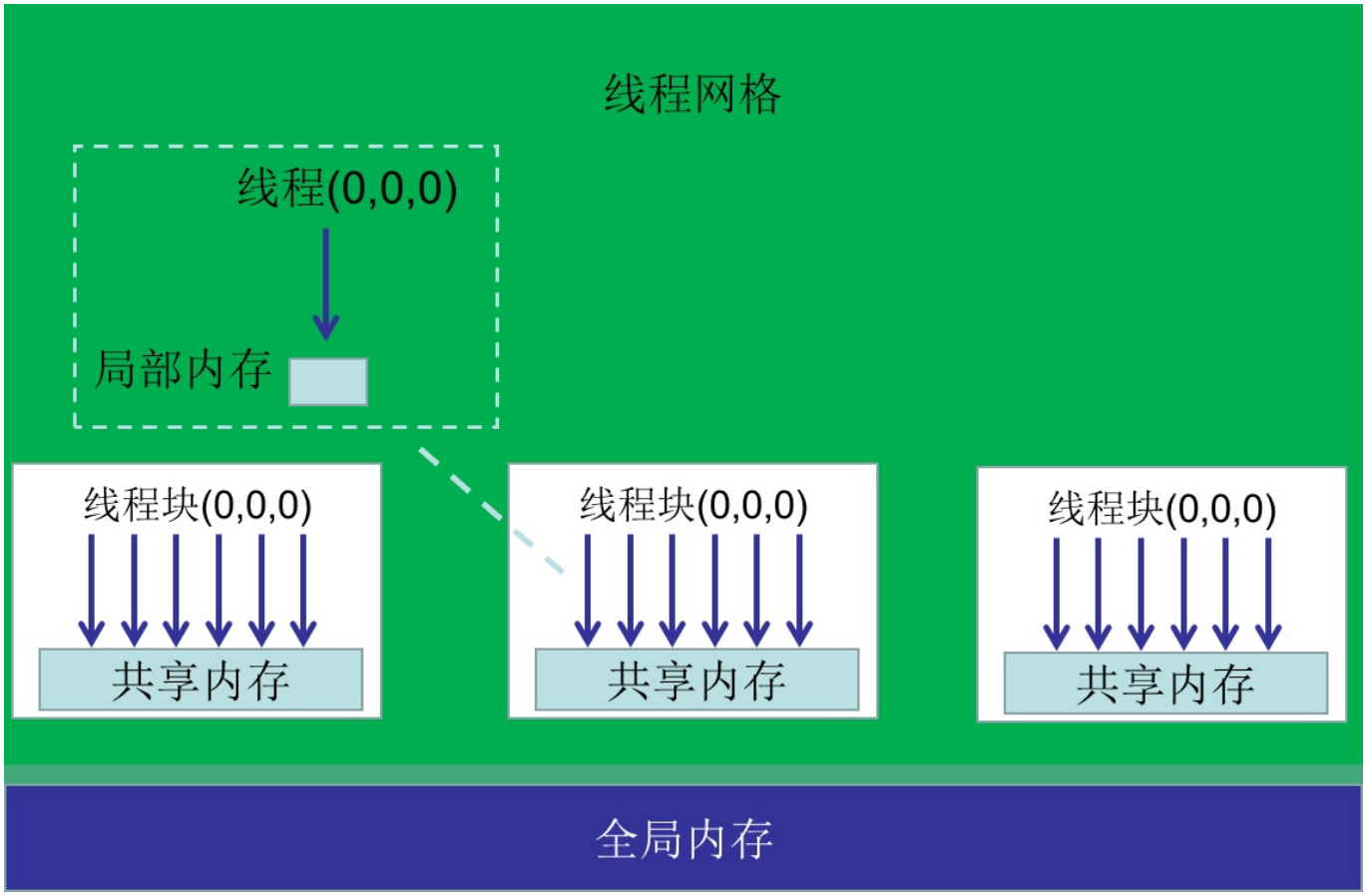


图3-3 内存层次结构图

3.2.2 从模型到硬件

DCU芯片上有64个独立的计算单元，每个计算单元有4个独立的SIMD（Single Instruction，Multiple Data，单指令、多数据）单元，每个SIMD单元又有16个向量ALU（算数逻辑单元），通俗的讲，可以认为一块DCU芯片上有4096个核心，每个向量ALU就对应一个线程。

虽然线程从概念上是独立执行的，但从DCU硬件来说是64个线程一组并行执行的，即一条指令发布后，一个SIMD单元中的16个ALU执行4轮完成，这64个线程被称为线程束（warp）；一个SIMD单元有10个线程束的指令缓冲，所以最多可以同一时间段内并发执行10个线程束（受其他硬件资源的限制）。这样算64个计算单元×4个SIMD单元×10线程束×64线程=163840，这是DCU硬件最大同时支持的线程数。此外，这10个线程束可以来自不同的线程块或者核函数，意味着我们的DCU支持多个核函数并发（在3.4节我们会介绍如何编程实现）。

每个计算单元配有一个标量ALU，负责线程束的流程控制，这里的标量是指ALU对整个线程束处理一个数，而向量ALU则对每个线程处理一个数。举例来说，在流程控制时，每个线程会根据判断条件得到一位（bit）以进行分支或循环，整个线程束是64位，标量ALU负责处理这个64位数。

计算单元上有高速的寄存器文件，每个SIMD单元有64KB的向量寄存器配额供向量ALU使用，负责存储我们在核函数中每个线程独有的局部变量。每个计算单元有64KB的局域数据共享（Local Data Share），负责存储核函数中以__shared__修饰符声明的变量，以实现同一线程块或同一线程束内的数据共享。此外，同现代CPU一样，DCU上还有其他复杂的多级缓存结构以实现对片外存储器的高速访问，但程序开发者是无法直接控制这些缓存的，只能通过修改算法提高访存的连续性以提升缓存命中。上

面所介绍的这些存储器件和计算单元一起（当然还包括其他的元件）封装在一块芯片内，构成片上存储。

DCU一般配有16GB或32GB的片外存储，对应我们上面说的设备端全局内存。计算单元请求片外内存上的数据是非常耗时的过程，通常需要几百个时钟周期，因此我们需要充分利用内存层次结构以提高程序访存性能，这也是后文重点要介绍的。

下面我们从硬件角度介绍核函数的执行。编译器在编译时会确定每个核函数的线程束需要多少硬件资源（比如说多少个寄存器，多大的共享内存），在程序运行时，DCU上的调度元件会根据所有计算单元的运行状态将核函数的线程束轮询的发送到空闲的且满足资源需求的计算单元上。这种轮询机制使得即使核函数启动参数中设置的线程数很多（如上节中的256M），仍然能够执行，只是不可以同时并发。同一个线程块中的线程束肯定会发送到同一计算单元上，以利用计算单元上的局域数据共享和专门的同步指令，DCU允许最多16个线程束构成一个线程块，这就是前面我们反复提到单个线程块中的线程最多是1024个的原因。单个线程块中的线程有唯一的ID，按以下公式计算：

$thread ID = threadIdx.x + threadIdx.y ×blockDim.x + threadIdx.z ×blockDim.x ×blockDim.y$

线程按线程ID以64个为一组划分在不同的线程束中，比如对于一维的（128，1，1）线程块，threadIdx.x为0到63的线程在同一线程束，64到127的在另一线程束；对于二维的（16，16，1）线程块，按threadIdx.y从0到3，4到7，8到11，12到15划分在4个线程束上。了解这个划分对访存是有用的，比如说我们知道在这样的二维线程块中，threadIdx.x连续的线程是硬件上连续的，threadIdx.y连续的线程则不然，在后面我们会介绍合并访存的概念，需要连续的线程访问连续的地址；此外还有助于针对线程束的函数的使用。顺便再强调一次，在编程模型中允许有小于线程束的线程块，但在这种情况下DCU依然会启动一个线程束来执行。

3.2.3 小节

在本节中我们简要介绍了DCU编程模型，模型的重点在于主机和设备的主从关系，设备线程的层次结构和设备内存的层次结构。了解DCU编程模型有助于我们构思设计DCU程序，也便于我们后文的介绍。之后我们深入到DCU的硬件，对DCU的架构做了简要的介绍，并说明了硬件单元与编程模型中抽象概念的对应。这方面我们没有过于深入，只需对线程束、线程ID这些源自硬件的概念有所了解即可，在后文中我们会看到这些概念与DCU程序性能的关系。其他硬件方面的信息我们也将在后文中根据需要适时介绍。

3.3 并行规约

在对DCU编程有个概述之后，我们继续使用DCU做并行计算，这次打算计算一个复杂的定积分，以此为例进一步探究DCU编程。我们在网络上搜索积分表，找到如下积分公式：

$sin^{2}axcos^{2}bxdx=\frac{x}{4}-\frac{sin2ax}{8a}-\frac{sin2a-bx}{16a-b}+\frac{sin2bx}{8b}-\frac{sin2a+bx}{16a+b}$

不要在意上面的公式，我们只想给计算机一些复杂的计算任务。

我们令a=2，b=1，化简一下：

$sin^{2}2xcos^{2}xdx=\frac{x}{4}+\frac{sin2x}{16}-\frac{sin4x}{16}-\frac{sin6x}{48}$

我们取积分限为$π$的整数倍，这样sin都没了，容易判断结果是否正确。我们先计算一个从0到40000$π$的积分，编写代码如下（integral目录下的cpu.cc）：

```
#include <cstdio>
#include "timer.h"
#include <cmath>
// 定义精度
#ifdef DOUBLE_PREC
typedef double real;
#define SIN(x) sin(x)
#define COS(x) cos(x)
#else
typedef float real;
#define SIN(x) sinf(x)
#define COS(x) cosf(x)
#endif

// 用积分公式计算
double F(double x)
{
  double F = x/4 + sin(2*x)/16 - sin(4*x)/16 -sin(6*x)/48;
  return F;
}

int main()
{
  double lower_bound = 0.0;
  double upper_bound = 40000 * acos(-1); //acos(-1) = pi
  int N = 1e8;
  real dx = (upper_bound - lower_bound) / N;
  double Integral = 0.0;
  my_timer timer;
  timer.start();
  // 数值计算 \int sin^2(2x)*cos^2(2x) dx
  for (int i = 0; i < N; i++) {
    real x = (i + 0.5) * dx;
    // 减少乘法次数
    real sin_2x = SIN(2 * x);
    sin_2x *= sin_2x;
    real cos_x = COS(x);
    cos_x *= cos_x;
    Integral += sin_2x * cos_x * dx;
  }
  timer.stop();
  printf("Numeric result is: %.11f\n", Integral);
  printf("Result form formula is: %.11f\n",
      F(upper_bound) - F(lower_bound));
```

```
    printf("Elapsed time: %8.6f ms\n",timer.dt);
    return 0;
}
```

在上面的代码中，0到40000$π$的区间被分为了10的8次方（一亿）段，取每段的中点x计算$sin^{2}2xcos^{2}bxdx$，最后累加到变量Integral上，并通过一点技巧，使计算sin、cos和乘法的次数减少了一些。我们对积分限和变量Integral采用双精度浮点数存储，利用积分公式计算的F(x)函数使用双精度，其他变量以及sin、cos函数的计算则可以通过条件编译选择不同的精度。

单精度版本编译运行如下：

```
g++ cpu.cc -o cpu -O3
./cpu
Numeric result is: 31415.92652882586
Result form formula is: 31415.92653589793
Elapsed time: 2773.030029 ms
```

双精度版本编译运行如下：

```
g++ cpu.cc -o cpu_dp -O3 -DDOUBLE_PREC
./cpu_dp
Numeric result is: 31415.92653618794
Result form formula is: 31415.92653589793
Elapsed time: 8468.580078 ms
```

> 尝试：
> （1）使用单精度存储变量Integral并测试结果。
> （2）编写DCU版本的代码（*先不要看下文*）。

3.3.1 原子操作

我们编写DCU版代码：

```
#include <cstdio>
#include "timer.h"
#include <cmath>
#include <hip/hip_runtime.h>
// 定义精度
#ifdef DOUBLE_PREC
typedef double real;
#define SIN(x) sin(x)
#define COS(x) cos(x)
#else
typedef float real;
#define SIN(x) sinf(x)
#define COS(x) cosf(x)
#endif
```

```
// 用积分公式计算
double F(double x)
{
   double F = x/4 + sin(2*x)/16 - sin(4*x)/16 -sin(6*x)/48;
   return F;
}

// DCU 核函数
__global__
void integral_kernel(double* Integral, double lb, int N, real dx)
{
   int i = blockIdx.x * blockDim.x + threadIdx.x;
   int tid = threadIdx.x;
   if (i < N) {
      real x = lb + (i + 0.5) * dx;
      real sin_2x = SIN(2 * x);
      sin_2x *= sin_2x;
      real cos_x = COS(x);
      cos_x *= cos_x;
      *Integral += sin_2x * cos_x * dx;
   }
}

int main()
{
   double lower_bound = 0.0;
   double upper_bound = 40000 * acos(-1); //acos(-1) = pi
   int N = 1e8;
   real dx = (upper_bound - lower_bound) / N;
   double Integral = 0.0;
   double* d_Integral;
   hipMalloc(&d_Integral,sizeof(double));
   // 核函数启动配置
   int blockSize = 256;
   int numBlocks = (N + blockSize - 1) / blockSize;
   // 开始计时
   my_timer timer;
   timer.start();
   hipMemcpy(d_Integral,&Integral,sizeof(double),
          hipMemcpyHostToDevice);
   integral_kernel<<<numBlocks, blockSize>>>
              (d_Integral,lower_bound,N,dx);
   hipMemcpy(&Integral,d_Integral,sizeof(double),
          hipMemcpyDeviceToHost);
   timer.stop();
   printf("Numeric result is: %.11f\n", Integral);
   printf("Result form formula is: %.11f\n",
        F(upper_bound) - F(lower_bound));
```

```
    printf("Elapsed time: %8.6f ms\n",timer.dt);
    hipFree(d_Integral);
    return 0;
}
```

在上面的代码中，我们在设备上申请内存用来保存积分结果，这块内存通过指针d_Integral访问，在主机端将Integral初始化为0并拷贝到设备上，将循环改为设备端的核函数，以256为线程块大小执行核函数，之后将结果拷贝回Integral中并释放设备内存。

编译运行如下：

```
hipcc dcu_error.cc -o dcu_error
./dcu_error
Numeric result is: 0.00072969924
Result form formula is: 31415.92653589793
Elapsed time: 4.727000
```

注意，DCU计算的结果是错误的！仔细检查一下，看起来代码写的并没有什么问题。

造成错误的原因其实是并行计算中的一个常见场景，让多个线程并行做计算，但最后需要把所有结果归并为一个值，这类问题被称为并行规约问题。在上面的代码中，多个线程会同时从d_Integral（所保存的地址）中取值，把本线程的结果加到这个值上再存回d_Integral中。以两个线程为例，假设此时d_Integral的值还是0，线程1算出结果dF1，线程2算出结果dF2，线程1从d_Integral处取值0，0加上dF1还是dF1，此时线程1把dF1写回d_Integral；如果线程1还没完成写回，线程2就从d_Integral处取值，仍然得到0，0加dF2等于dF2，线程将dF2写回d_Integral。这种情况下如果线程1先完成写回，d_Integral最终就是dF1，如果线程2先完成写回，结果就是dF2，但无论如何不是正确的结果dF1+dF2。

解决方法是使用原子操作，所谓“原子”的意思是上面叙述的一个线程对d_Integral的读-加-写回这三个动作是不可分的，借用了“原子”这个词不可再分的原意。当一个线程执行这个操作时，必须得把读-加-写回这三个操作全部做完，在此期间别的线程不能执行此操作。

修改一下代码，把核函数中的

```
*Integral += sin_2x * cos_x * dx;
```

修改为：

```
atomicAdd(Integral,sin_2x * cos_x * dx);
```

并把分段数N从1e8改为1e6，这是为了使核函数执行不至于太慢。编译并运行：

```
hipcc dcu_error.cc -o dcu_error.cc
./dcu_error
Numeric result is: 31415.92749216406
Result form formula is: 31415.92653589793
Elapsed time: 9127.658203 ms
```

从正确性的角度看，虽然精度低了一些，但结果好了很多；从性能的角度看，惨不忍睹，在计算量少了100倍的情况下还用了9.1秒。我们还需继续改进。

3.3.2 利用共享内存优化原子加

在我们的程序中d_Integral保存的是设备端全局内存中的地址。在3.2.2节，我们介绍了线程访问DCU片外存储是非常耗时的过程，此外对于原子操作只有对整型的硬件支持，对浮点数的原子操作则是通过软件实现的，这就是上面让所有线程对d_Integral做原子加性能很差的原因。

在3.2.2节中介绍过，DCU编程的一个重点就是合理利用内存层次结构，在全局内存与局部内存之间，还有一个线程块内所有线程公用的共享内存（对应硬件的LDS），线程对这块内存的访问速度相比全局内存要快很多，下面我们就利用它来改善原子加的性能。

将核函数修改如下：

```
__global__
void integral_kernel(double* Integral, double lb, int N, real dx)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  int tid = threadIdx.x;
  __shared__ double tmp_sh;
  if (tid == 0) tmp_sh = 0.0;
  __syncthreads();
  if (i < N) {
    real x = lb + (i + 0.5) * dx;
    real sin_2x = SIN(2 * x);
    sin_2x *= sin_2x;
    real cos_x = COS(x);
    cos_x *= cos_x;
    atomicAdd(&tmp_sh,sin_2x * cos_x * dx);
  }
  __syncthreads();
  if (tid == 0 ) atomicAdd(Integral,tmp_sh);
}
```

在这段代码中，首先定义了线程ID的变量tid，这是一种常用的命名，它的值在一维的情况下就等于threadIdx.x。此外多了共享内存中变量tmp_sh的声明（通过__shared__修饰符），我们让线程ID为0的线程（tid
==0）将其初始化为0。后面的__syncthreads()是一个内置函数，作用是令线程块同步，即等待线程ID为0的线程初始化tmp_sh之后，所有线程才能继续往下执行。在最后的原子加阶段，我们让线程块内的所有线程向共享内存做原子加，等完成之后，再由线程ID为0的线程对全局内存中的d_Integral做一次原子加即可。

这里需要多解释几句，前面介绍过，DCU是按64个线程一组的线程束来调度的，当遇到像if (tid ==
0)这样的条件判断时，标量ALU会把线程束的所有线程ID与0做比较得出一个64位的掩码，在这里代表线程ID是0的那一位是1，其他都是0，当线程束执行对共享内存中的变量tmp_sh赋值时，通过这个掩码使得只有线程ID为0的线程实际执行。当然在这个过程中，线程束内的其他线程并不能继续向下执行，而需要等线程ID为0的线程执行完。

使用__syncthreads()进行线程块同步对于线程数大于64的线程块是必要的，因为必须将做累加的变量tmp_sh初始化为0，因为线程块中线程束的调度顺序是不确定的，如果不包含线程ID为0的线程束先执行，在不加线程块同步的情况下很可能这些线程束将计算结果加到未初始化的tmp_sh中，之后等包含线程ID为0的线程束执行时，又会把tmp_sh置为0，最后的结果当然是不对的。同理对于最后的原子加阶段，如果线程块内还有线程未完成对共享内存中tmp_sh的原子加，线程0就把tmp_sh的结果加到全局内存d_Integral中去，同样会造成错误。

做了以上修改后，可以把N改回1e8了，编译并运行（shared.cc）：

```
hipcc shared.cc -o shared
./shared
Numeric result is: 31415.92577126694
Result form formula is: 31415.92653589793
Elapsed time: 185.567001 ms
```

这样，通过利用共享内存，我们的性能有了大幅的提高。

3.3.3 利用线程束洗牌函数优化原子加

到了这一步，每个线程块对全局内存做一次原子加，如果我们想继续减少全局内存的原子加次数就需要减少线程块的个数，对此，一个自然的想法是增加线程块的大小。将blockSize修改为1024（对于使用大于256的线程块启动的核函数，需要在声明前加__launch_bounds__，这与编译器优化有关，详见下面的核函数代码），编译并运行，得到：

```
Numeric result is: 31415.92577126688
Result form formula is: 31415.92653589793
Elapsed time: 616.575989 ms
```

结果并未如我们所愿，耗时反而更长了。我们猜测这可能与对共享内存的原子加增多有关，当blockSize为256时，每个线程块的256个线程向共享内存的同一地址做原子加，blockSize增大为1024时，则是1024个向同一个地址。是否有办法优化对共享内存的原子加呢？

实现的方法有很多种，在此我们通过一组被称为线程束洗牌函数（warp shuffle）的内置函数来实现。它们的作用是让线程获取到线程束内其他线程的局部变量的值，从硬件层面讲仍然是利用共享内存实现的。我们将核函数修改如下：

```
__global__ __launch_bounds__(1024)
void integral_kernel(double* Integral, double lb, int N, real dx)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  int tid = threadIdx.x;
  double tmp = 0.0;
  __shared__ double tmp_sh;
  if (tid == 0) tmp_sh = 0.0;
  __syncthreads();
  if (i < N) {
    real x = lb + (i + 0.5) * dx;
    real sin_2x = SIN(2 * x);
    sin_2x *= sin_2x;
    real cos_x = COS(x);
    cos_x *= cos_x;
    tmp += sin_2x * cos_x * dx;
    int delta = 1;
    for (int j = 0; j < 6; j++) {
      tmp += __shfl_down(tmp,delta,64);
```

```
        delta += delta;
      }
      if (tid%64==0) atomicAdd(&tmp_sh,tmp);
      __syncthreads();
      if (tid == 0 ) atomicAdd(Integral,tmp_sh);
    }
}
```

在这段代码中，我们使用了__shfl_down这个函数，它的作用是使一个线程束内，线程ID为id的线程，获取到线程ID为（id+delta）的线程的局部变量tmp的值，64是一个线程束中线程的个数，对于线程id+delta超过64的线程，还是得到自己的tmp值。举例来说，当delta=1时，对于线程0，这个函数返回线程1的tmp；对于线程2，这个函数返回线程3的tmp；对于线程63，这个函数则返回自己的tmp。当delta=2时，对于线程0，此时它的tmp已是进行洗牌前的线程0和线程1的tmp之和，再用__shfl_down把线程2的tmp（洗牌前的线程2和线程3的tmp之和）传给它，加在一起就是洗牌前线程0-3的tmp之和。当delta=4时，会把线程4的tmp（洗牌前线程4-7的tmp之和）传给线程0，加在一起是洗牌前线程0-7的tmp之和。我们对上述过程示意如下，图中的方块代表进行洗牌前的线程上的tmp值，方块中数字是线程ID：

[图片]

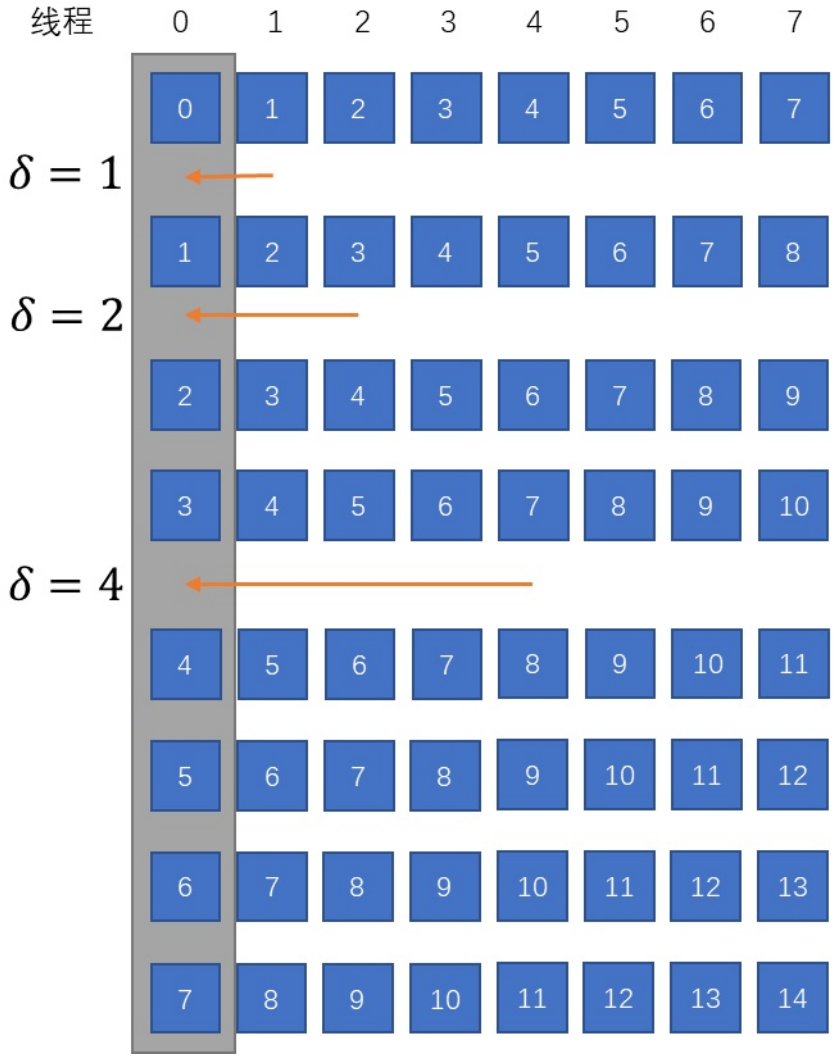


图3-4 使用__shfl_down函数规约示意图

以此类推，随着j不断循环，delta=8时，线程0的tmp为洗牌前线程0-15的tmp之和，delta=16时，是线程0-31的tmp之和，delta=32时，是线程0-63的tmp之和。

通过调用6次__shfl_down，线程束内的线程0的tmp为所有线程的tmp之和，然后再由每个线程束中的线程0（tid模64为0）将tmp值原子加到共享内存tmp_sh中，这样就把每个线程束中64个线程一起对tmp_sh做原子加改成了6次__shfl_down之后由一个线程对tmp_sh做原子加。我们测试一下效果，编译运行（warpshuffle.cc，注意此时blockSize=1024），结果如下：

```
hipcc warpshuffle.cc -o warpshuffle
./warpshuffle
Numeric result is: 31415.92577126683
Result form formula is: 31415.92653589793
Elapsed time: 49.224998 ms
```

现在的效果已经优于使用256的线程块但不使用__shfl_down函数的版本了。

3.3.4 另一种可扩展的核函数写法

现在要问，我们的核函数还有进一步的优化空间吗？答案是肯定的，我们可以进一步的减少线程块的数量，你可能会觉得，每个线程块的线程数已经到1024的上限了，再减少线程块的数量，就没有足够的线程来执行N次核函数了。

解决方法很简单，通过循环让每个线程多执行几次核函数的内容就可以了，将核函数改写如下：

```
__global__ __launch_bounds__(1024)
void integral_kernel(double* Integral, double lb, int N, real dx)
{
  int index = blockIdx.x * blockDim.x + threadIdx.x;
  int stride = gridDim.x * blockDim.x;
  int tid = threadIdx.x;
  double tmp = 0.0;
  __shared__ double tmp_sh;
  if (tid == 0) tmp_sh = 0.0;
  __syncthreads();
  for (int i = index; i < N; i += stride) {
    real x = lb + (i + 0.5) * dx;
    real sin_2x = SIN(2 * x);
    sin_2x *= sin_2x;
    real cos_x = COS(x);
    cos_x *= cos_x;
    tmp += sin_2x * cos_x * dx;
  }
  int delta = 1;
  for (int j = 0; j < 6; j++) {
    tmp += __shfl_down(tmp,delta,64);
    delta += delta;
  }
  if (tid%64==0) atomicAdd(&tmp_sh,tmp);
  __syncthreads();
  if (tid == 0 ) atomicAdd(Integral,tmp_sh);
}
```

上面的核函数中，CPU版本的for循环又回来了，但这次循环计数i并不是从0开始每次递增1，而是对于每个线程来说从index开始，每次递增stride，index是线程在这个核函数启动的所有线程中的索引，stride是这个核函数启动的总线程数。当核函数启动的总线程数少于循环次数N时，每个线程就从循环计数为index的循环开始，负责循环计数相差stride的所有循环，直到把N次循环全部做完。这是另一种常用的可扩展的核函数写法，区别于我们之前在主机代码中根据工作量动态的确定总线程数，这种写法可以用任意的总线程数启动核函数完成工作。

注意线程束洗牌函数和原子加并不在for循环中，因为我们完全可以让每个线程将它所负责的几次循环结果都累加到tmp之后再进行线程束内的求和，以及向共享内存和全局内存做原子加。

我们不改动主机代码中的启动参数，仅修改核函数并编译执行，效果与之前的完全一样的：

```
hipcc loop_in_kernel.cc -o loop_in_kernel
./loop_in_kernel
Numeric result is: 31415.92577126695
Result form formula is: 31415.92653589793
Elapsed time: 50.818001 ms
```

现在可以用更少的线程块来执行核函数了，我们尝试让每个线程做100次循环，为此令：

```
  int numBlocks = (N / 100 + blockSize - 1) / blockSize;
```

再次编译运行（loop_in_kernel.cc），得到结果：

```
Numeric result is: 31415.92577126687
Result form formula is: 31415.92653589793
Elapsed time: 4.646000 ms
```

对这个核函数的优化暂时告一段落。值得一提的是，这种优化方式实际是一种权衡，即到底是让每个线程多做一些计算，而让原子加的次数减少，还是让每个线程少做一些计算，而让原子加的次数增加。这是优化中常常遇到的一种两者不可兼得的场景，需要结合问题具体分析。

> 练习：使用hipprof工具查看上面几个版本的核函数运行时间。

3.3.5 用空间换时间

对于我们的程序还有一种可行的思路，既然对全局内存做原子加比较慢，我们可以申请长度为线程总数的数组，这样每个线程可以把计算结果保存在不同的位置，避免多个线程对同一位置的原子加，之后再单独对这个数组中的元素求和。这是一种用空间换时间的方法，同样是一种权衡。

首先编写一个使用CPU串行求和的版本，让blockSize等于线程束的大小，这样numBlocks就是线程束的个数，设置长度为numBlocks的数组res_warp（及其设备端对应d_res_warp）用于存储每个线程束的求和结果，最后在让CPU用一个循环将所有线程束的结果求和。这里我们保留了速度较快的线程束洗牌函数将64个线程的结果做一次求和，这样也将数组长度由总线程数减少到了线程束的个数。核函数改写如下：

```
__global__
void integral_kernel(double* res_warp, double lb, int N,
                     real dx)
```

```
{
    int index = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;
    int tid = threadIdx.x;
    double tmp = 0.0;
    for (int i = index; i < N; i += stride) {
        real x = lb + (i + 0.5) * dx;
        real sin_2x = SIN(2 * x);
        sin_2x *= sin_2x;
        real cos_x = COS(x);
        cos_x *= cos_x;
        tmp += sin_2x * cos_x * dx;
    }
    int delta = 1;
    for (int j = 0; j < 6; j++) {
        tmp += __shfl_down(tmp,delta,64);
        delta += delta;
    }
    if (tid == 0) res_warp[blockIdx.x] = tmp;
}
```

将主程序改写如下：

```
int main()
{
    double lower_bound = 0.0;
    double upper_bound = 40000 * acos(-1); //acos(-1) = pi
    int N = 1e8;
    real dx = (upper_bound - lower_bound) / N;
    double Integral = 0.0;
    // 核函数启动配置
    int blockSize = 64;
    int numBlocks = (N / 100 + blockSize - 1) / blockSize;
    // 用于存储每个warp的结果
    int size = numBlocks * sizeof(double);
    double* res_warp = (double *) malloc(size);
    double* d_res_warp;
    hipMalloc(&d_res_warp,size);
    // 开始计时
    my_timer timer;
    timer.start();
    integral_kernel<<<numBlocks, blockSize>>>
                (d_res_warp,lower_bound,N,dx);
    hipMemcpy(res_warp,d_res_warp,size,hipMemcpyDeviceToHost);
    for (int i = 0; i < numBlocks; i++) {
        Integral += res_warp[i];
    }
    timer.stop();
    printf("Numeric result is: %.11f\n", Integral);
```

```
  printf("Result form formula is: %.11f\n",
      F(upper_bound) - F(lower_bound));
  printf("Elapsed time: %8.6f ms\n",timer.dt);
  hipFree(d_res_warp);
  free(res_warp);
  return 0;
}
```

编译上面的代码（sum_on_cpu.cc）并运行，结果如下：

```
hipcc sum_on_cpu.cc -o sum_on_cpu
./sum_on_cpu
Numeric result is: 31415.92577126692
Result form formula is: 31415.92653589793
Elapsed time: 4.539000 ms
```

可以看到性能表现也非常出色，和前面几小节持续优化单个核函数的效果持平了。

> 题外话：现在的核函数integral_kernel是以计算为主了，请尝试在编译时通过定义-DDOUBLE_PREC控制计算精度，查看程序使用不同精度时的结果，并通过hipprof工具查看核函数性能。DCU还为单精度计算提供了一组内置快速数学函数，以双下划线__为前缀，如__sinf()、__cos()等，请尝试使用并查看精度与性能。

还可以将CPU求和这步也用DCU来做，为此编写一个核函数sum_kernel，代码如下：

```
__global__
void sum_kernel(double* res_warp, int numBlocks,
          double* Integral)
{
  int index = blockIdx.x * blockDim.x + threadIdx.x;
  int stride = gridDim.x * blockDim.x;
  int tid = threadIdx.x;
  double tmp = 0.0;
#if 1
  index *= 2;
  stride *= 2;
  // 如果numBlocks是奇数
  // 用0号线程把res_warp[numBlocks-1]单独读取
  if (numBlocks%2==1) {
    if (tid==0) tmp = res_warp[numBlocks-1];
    numBlocks -= 1;
  }
  for (int i = index; i < numBlocks; i += stride) {
    // 从全局内存一条指令读取128位，即两个double型
    tmp += res_warp[i] + res_warp[i+1];
  }
#else
  for (int i = index; i < numBlocks; i += stride) {
    tmp += res_warp[i];
  }
```

```
#endif
  int delta = 1;
  for (int j = 0; j < 6; j++) {
    tmp += __shfl_down(tmp,delta,64);
    delta += delta;
  }
  if (tid == 0) *Integral = tmp;
}
```

在这段核函数的编写中我们额外展示了一个小技巧，即DCU存在可以从全局内存或共享内存一次性读取连续的128位的指令，我们通过代码

```
    tmp += res_warp[i] + res_warp[i+1];
```

明确的告诉编译器每个线程会读取两个连续的双精度浮点数，刚好128位，编译器就会采用这条指令，达到更好的访存性能。我们保留了普通的写法，可以通过修改条件编译的#if 0和#if 1在这两种方式间切换，通过hipprof性能分析工具的计时来对比性能的差别。

言归正传，在主机代码中，现在不再需要主机内存中的空间res_warp，仅在设备端为d_res_warp分配内存即可，但这次最终的积分结果d_Integral需要拷贝回主机。至于sum_kernel，我们只使用一个线程束来执行，这样在核函数的最后直接用线程0对d_Integral赋值即可。

篇幅所限仅列出计时起止间的主机代码（其余部分仅删除了对res_warp的malloc()和free()）：

```
  timer.start();
  integral_kernel<<<numBlocks, blockSize>>>
            (d_res_warp,lower_bound,N,dx);
  sum_kernel<<<1,blockSize>>>
        (d_res_warp,numBlocks,d_Integral);
  hipMemcpy(&Integral,d_Integral,sizeof(double),
        hipMemcpyDeviceToHost);
  timer.stop();
```

这次编译运行的结果为（sum_on_dcu.cc）：

```
Numeric result is: 31415.92577126689
Result form formula is: 31415.92653589793
Elapsed time: 4.341000 ms
```

3.3.6 小节

本节我们通过一个数值计算定积分的例子介绍了使用DCU做并行计算时的规约问题，即计算虽然可以用很多线程并行，但最终所有线程的计算结果需要归并在一起。为了保证规约的正确，需要使用内置原子函数，即在一个线程做读-改-写这三个为一体的不可分的操作时，其他线程均需等待。大量线程对全局内存中的个别地址同时做浮点数原子操作的性能很慢，但是可以利用DCU的内存层次结构，即利用共享内存、线程束洗牌函数进行优化，效果显著。

之后我们还介绍了另一种编写可扩展的核函数的方法，通过使单个线程执行循环的方式实现核函数对计算量的可扩展：即使核函数的启动参数固定，仍然允许计算量动态扩展。我们利用这种写法通过增加单个线程的计算次数，减少了规约次数，使核函数性能进一步优化。

在本节最后，我们介绍了通过空间换时间优化规约的方法，将一部分规约任务或是直接使用CPU、或是让DCU另外运行一个核函数来执行，对于本节的例子同样都有不错的性能表现。这几种优化规约问题的方法，请读者编程时根据具体问题灵活使用。

3.4 异步并发执行

在本节我们将介绍DCU编程中的流水线技术。仍以上节中的积分计算为例，但在本节中将积分区间扩大8倍，即从0到320000$π$。当然根据sin函数和cos函数的周期性，我们知道结果就是上节中从0到40000$π$的8倍，但在这里我们不利用这个性质，而是让DCU挨个区间的算出来。

我们在3.3.5节中使用CPU做一部分求和的版本（sum_on_cpu.cc）基础上稍加修改，DCU核函数不变，只修改主函数main：

```
int main()
{
  double lower_bound = 0.0;
  double upper_bound = 40000 * acos(-1); //acos(-1) = pi
  double range = upper_bound - lower_bound;
  int N = 1e8;
  real dx = range / N;
  int numIntervals = 8;
  double Integral = 0.0;
  // 核函数启动配置
  int blockSize = 64;
  int numBlocks = (N  + blockSize - 1) / blockSize;
  // 用于存储每个warp的结果
  int size = numBlocks * sizeof(double);
  double* res_warp = (double *) malloc(size);
  double* d_res_warp;
  hipMalloc(&d_res_warp,size);
  // 开始计时
  my_timer timer;
  timer.start();
  for (int j = 0; j < numIntervals; j++) {
    integral_kernel<<<numBlocks, blockSize>>>
              (d_res_warp,lower_bound,N,dx);
    hipMemcpy(res_warp,d_res_warp,size,
          hipMemcpyDeviceToHost);
    for (int i = 0; i < numBlocks; i++) {
      Integral += res_warp[i];
    }
    lower_bound += range;
  }
  timer.stop();
  printf("Numeric result is: %.11f\n", Integral);
  printf("Result form formula is: %.11f\n",
      F(range*numIntervals) - F(0));
```

```
    printf("Elapsed time: %8.6f ms\n",timer.dt);
    hipFree(d_res_warp);
    free(res_warp);
    return 0;
}
```

在这段代码中定义了积分区间数numIntervals，积分区间的长度range，此外将调用核函数integral_kernel、设备到主机的拷贝以及主机的求和部分循环8次，每次循环之后将积分区间的下界递增range。此外，为了方便展示本节中要讨论的主题，将线程块个数numBlocks改回到(N + blockSize - 1) / blockSize，这样数组res_warp会更长，因而增加拷贝时间。

3.4.1 用核函数掩盖主机端的求和

在前面我们使用hipprof工具进行性能分析时，只看了统计数据，这次要深入一些，利用它来可视化地观察程序的时间线。编译（integral_async路径下的sum_on_cpu.cc）并使用性能分析工具运行：

```
hipcc sum_on_cpu.cc -o sum_on_cpu
hipprof ./sum_on_cpu
```

可以看到路径下生成了几个文件，可能你之前使用时就注意到，其中 hip-prof-进程号.db 是hipprof记录的原始数据文件，result_进程号.hiptrace.csv和result_进程号.hipkernel.csv分别是 CSV 格式的 API 和核函数的统计，result_进程号. pftrace则是 perfetto 格式的程序详细记录
(你可能会发现文件名中的进程号与可视化界面中的主机线程号Thread并不相等，因为文件名中的进程号是hipprof进程，而我们程序是作为其子进程启动的)，除上述文件格式外，也可以通过hipprof指定参数生成json、html格式的文件。

通过浏览器 (地址栏输入https://ui.perfetto.dev/)打开这个perfetto文件 (界面左上角的Open trace file选项):

[图片]

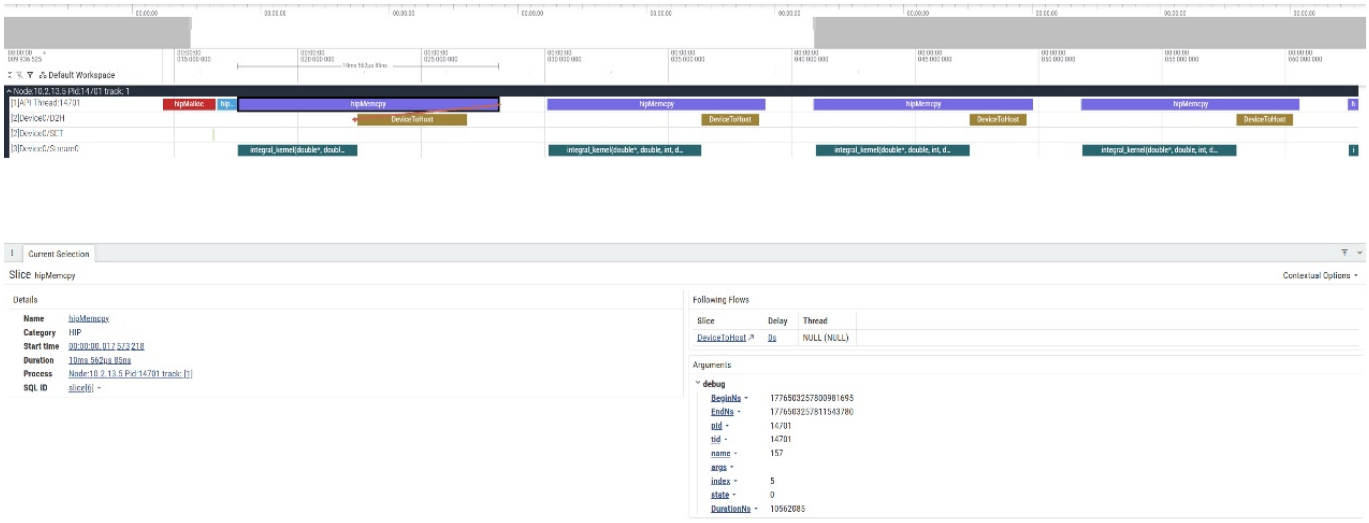


图 3-5 使用浏览器打开hipprof生成的perfetto文件的界面

可以看到界面分上下两部分，上半部分包括时间标尺和四行时间线，每行时间线都是 hipprof 记录的一系列事件，用一个个色块表示，色块的长度就是其执行的时间。这四行中的第一行是运行在线

程上的API接口，第二行和第三行是数据拷贝和数据设置，第四行则是核函数。具体内容区域包含每个操作所占用的时间和流程，称之为timeline，可以通过键盘上的（w，s，a，d）按键进行放大缩小和前后滚动查看的操作，点击彩色单个timeline可以在下方区域显示出函数的详细信息。

从时间线可以看出，主机在调用核函数之后就开始执行设备到主机的拷贝，但因为拷贝是同步的，因此实际的拷贝是在核函数执行完之后才开始，并且要等拷贝结束后，hipMemcpy才算执行完，这与3.1.6节中对同步异步行为的叙述是一致的。在hipMemcpy之后会有一段空隙（用红框标出），根据代码可知这段是主机在执行res_warp到Integral的累加。在主机执行这段工作时，DCU上也是空闲的，直到下一个循环调用核函数之后才会再次工作起来。

[图片]

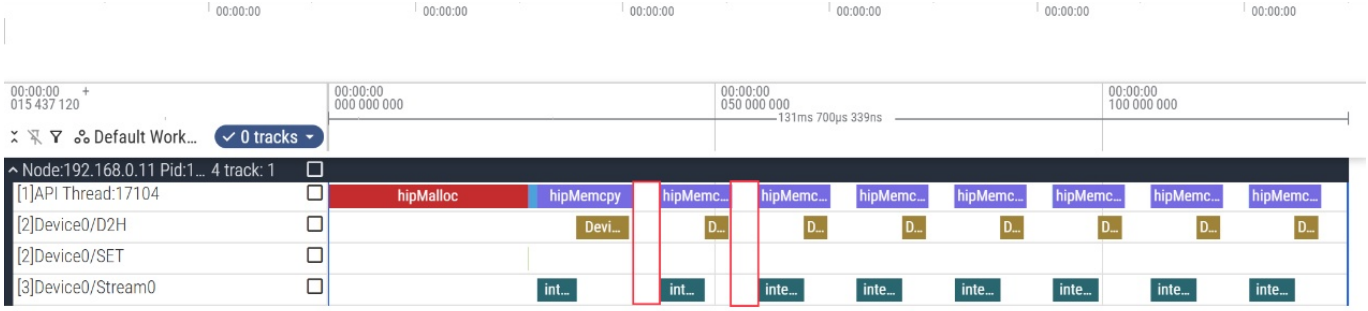


图3-6 程序sum_on_cpu的时间线

思考一下，DCU等待这段时间是没有必要的，它可以立刻继续执行核函数integral_kernel，这只会改变设备上d_res_warp的内容而不会改变主机上res_warp的内容；在DCU执行核函数的同时主机来做对Integral的累加，完成后再将DCU新计算的d_res_warp的结果拷贝过来，这样才更有效率。

当前的代码无法实现这种想法，因为主机是串行执行，只有在当前循环的累加执行完毕后，才能做下一次循环中的核函数调用。为此，一种思路是使用主机的多线程并行，一个线程负责累加，另一个线程调用核函数。但其实使用一个主机也可以完成，修改主机代码如下（只需修改timer.start()和timer.stop()之间的代码）：

```
timer.start();
// 对第一个区间计算并拷贝后，不立刻在主机端求和
// 而是先启动下一个区间计算的核函数
integral_kernel<<<numBlocks, blockSize>>>
          (d_res_warp,lower_bound,N,dx);
hipMemcpy(res_warp,d_res_warp,size,
       hipMemcpyDeviceToHost);
lower_bound += range;
for (int j = 1; j < numIntervals; j++) {
  // 启动核函数计算当前区间
  integral_kernel<<<numBlocks, blockSize>>>
            (d_res_warp,lower_bound,N,dx);
  // 主机端同时对上一个区间求和
  for (int i = 0; i < numBlocks; i++) {
    Integral += res_warp[i];
  }
  // 求和完毕后，拷贝当前区间的结果
  hipMemcpy(res_warp,d_res_warp,size,
```

```
            hipMemcpyDeviceToHost);
    lower_bound += range;
  }
  // 最后一个区间的求和
  for (int i = 0; i < numBlocks; i++) {
    Integral += res_warp[i];
  }
  timer.stop();
```

改动很简单，在对j的循环中将拷贝和求和调换一下顺序即可。这样每个循环开始时直接调用核函数在DCU上对当前区间做计算，调用之后主机开始对上一个区间求和（此时res_warp还是上一个区间的结果），求和完毕后再调用hipMemcpy将当前区间的结果拷贝到主机端，这样就实现了DCU上核函数与主机端求和在时间上的重叠。第一个区间和最后一个区间比较特殊，因为对于第一个区间来说没有上一个区间，因此在调用核函数之后就进行拷贝；而最后一个区间的求和没有包含在j的循环中，因此在对j的循环结束后需要补上。

编译运行 (hide_cpu.cc) 并使用hipprof生成perfetto文件:

```
hipcc hide_cpu.cc -o hide_cpu
hipprof ./hide_cpu
```

打开后时间线显示如下：

[图片]

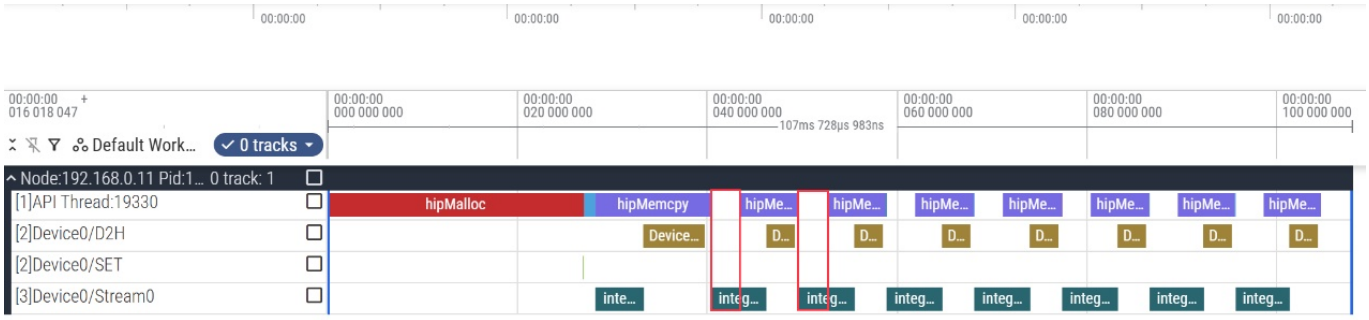


图3-7 程序hide_cpu的时间线

仍以红框标出主机上做求和的时间，可以看到在这些时间段内DCU不再空闲，而是在执行integral_kernel核函数了。从整段时间来看，DCU始终在不间断的执行核函数与拷贝，优化效果体现为总耗时从131 ms缩短到了107 ms。

3.4.2 流与异步拷贝

在上节中，我们利用核函数执行与主机执行的异步性来优化程序，使得当主机执行上一个区间求和的同时，DCU也在执行当前区间的计算，使得相邻区间的计算在时间上有重叠，因此缩短了总的计算时间。思考一下，是否能让相邻区间的计算在时间上有更多的重叠呢？比如说让当前区间的核函数与上一个区间的拷贝重叠，甚至与上一个区间的核函数重叠。

我们在3.1.6节的最后提到过，在DCU编程中主机与设备间的拷贝可以是异步的，多个核函数也可以异步执行，在本节我们介绍如何实现这些异步行为，并利用它们进一步优化程序。

在DCU编程中，可以创建被称为“流（stream）”的设备端队列，并将拷贝以及核函数发布在流上。同一个流上的任务是按入列的顺序串行执行的，不同流上的任务则是异步并发的，当然如果一个流的任务已占据了所需的软硬件资源，另一个流的任务则需要等待。

现在看如何将拷贝和核函数发布在指定流上。此前我们使用的拷贝API是hipMemcpy，它有一个异步版本hipMemcpyAsync，相比同步版本多一个流参数。对于核函数，在前面的例子中三个尖括号“<<<>>>”中的核函数启动配置仅包含线程网格和线程块的配置，但启动配置还包括动态共享内存的大小以及流，分别是三个尖括号中的第三和第四个参数。这两个参数的默认值都是0，所以之前没有配置也不影响核函数调用；动态共享内存将在后文介绍，而流参数的默认值代表所使用的是默认流。此前我们的核函数总是发布在默认流上，如上面时间线第四行所标的“Stream0”。

回到我们的程序，简单思考一下你会发现问题，如果将当前区间的核函数与上一个区间的拷贝重叠，会导致上一个区间从d_res_warp向res_warp拷贝的过程中当前区间的核函数integral_kernel同时在修改d_res_warp的内容，这样拷贝的结果很可能出问题；如果将当前区间的核函数与上一个区间的核函数也重叠，则会导致两个核函数同时修改d_res_warp，这同样不行。

有了上面的分析，解决方法自然是为每个区间分配不同d_res_warp，这里我们准备使用双缓冲（double buffering）的方法。定义缓冲结构体Buffer如下：

```
struct Buffer
{
  double* res_warp;
  double* d_res_warp;
  hipStream_t stream;
  void alloc(int size) {
    hipHostMalloc(&res_warp,size);
    hipMalloc(&d_res_warp,size);
    hipStreamCreate(&stream);
  }
  void dealloc() {
    hipHostFree(res_warp);
    hipFree(d_res_warp);
    hipStreamDestroy(stream);
  }
};
```

我们将res_warp和d_res_warp都包含在Buffer中，此外还有流变量stream。在Buffer中还有两个成员函数alloc和dealloc，用于申请/释放空间以及创建/销毁流。对于使用hipMalloc和hipFree申请和释放设备内存无需多言；对于主机内存，这里使用了hipHostMalloc和hipHostFree，而非之前的malloc和free，原因是对于涉及主机端内存的拷

贝，如果想要异步，则主机端内存必须是页锁定的（page-locked），否则即使使用异步版本的拷贝hipMemcpyAsync，拷贝仍然是同步的。目前用到的hipHostMalloc和hipHostFree是专门用来申请与释放页锁定主机内存的API，关于页锁定主机内存会在下节专门介绍。剩下的hipStreamCreate和hipStreamDestory两个API则是关于流的，分别用于创建与销毁。

定义结构体Buffer后，我们将代码中的res_warp和d_res_warp的创建与内存分配替换为：

```
  Buffer buffer[2];
  buffer[0].alloc(size);
```

```
buffer[1].alloc(size);
```

将释放res_warp和d_res_warp替换为：

```
buffer[0].dealloc();
buffer[1].dealloc();
```

并且将计时器起止间的代码如下修改：

```
timer.start();
integral_kernel<<<numBlocks, blockSize,0,buffer[0].stream>>>
        (buffer[0].d_res_warp,lower_bound,N,dx);
hipMemcpyAsync(buffer[0].res_warp,buffer[0].d_res_warp,size,
        hipMemcpyDeviceToHost,buffer[0].stream);
lower_bound += range;
int b1, b2; // buffer索引
for (int j = 1; j < numIntervals; j++) {
  b1 = j%2; // 当前区间使用的buffer
  b2 = (j-1)%2; // 上一个区间使用的buffer
  integral_kernel<<<numBlocks, blockSize,0,buffer[b1].stream>>>
          (buffer[b1].d_res_warp,lower_bound,N,dx);
  hipMemcpyAsync(buffer[b1].res_warp,buffer[b1].d_res_warp,size,
          hipMemcpyDeviceToHost,buffer[b1].stream);
  // 等待上个区间的结果拷贝完毕
  hipStreamSynchronize(buffer[b2].stream);
  for (int i = 0; i < numBlocks; i++) {
    Integral += buffer[b2].res_warp[i];
  }
  lower_bound += range;
}
// 最后一个区间的求和
hipStreamSynchronize(buffer[b1].stream);
for (int i = 0; i < numBlocks; i++) {
  Integral += buffer[b1].res_warp[i];
}
timer.stop();
```

在这段代码中，我们根据循环计数j确定使用的Buffer，在每次循环中主机先将当前积分区间的核函数和拷贝发布在该Buffer中的流上，之后调用流同步接口hipStreamSynchronize让主机等待上一个区间的核函数和拷贝结束（使用另一个Buffer中的流和内存空间）。这里的流同步是必要的，因为此时的拷贝是异步的；如果不加同步，主机会在d_res_warp中的结果还未拷贝到res_warp前就执行res_warp到Integral的累加。hipStreamSynchronize只会让主机等待指定的流，只要指定流上的任务完成后主机就可以继续往下执行，而与其他流的执行状态无关。

对于第一个区间和最后一个区间同样需要单独处理，不再赘述。

> 尝试：将对上一个区间的流同步与当前区间的核函数和拷贝调换顺序，并使用hipprof观察时间线。

编译（double_buffer.cc），并使用hipprof运行，因为我们使用了两个流，我们通过选项—group-stream将时间线按流来分组，即将拷贝和核函数按流显示在同一行：

```
hipcc double_buffer.cc -o double_buffer
hipprof --group-stream ./double_buffer
```

这样生成的perfetto文件命名为result_进程号_stream.pftrace，使用浏览器打开:（在图中，为了清晰区分，我们修改了拷贝的颜色，原本与核函数是相同的配色）：

[图片]

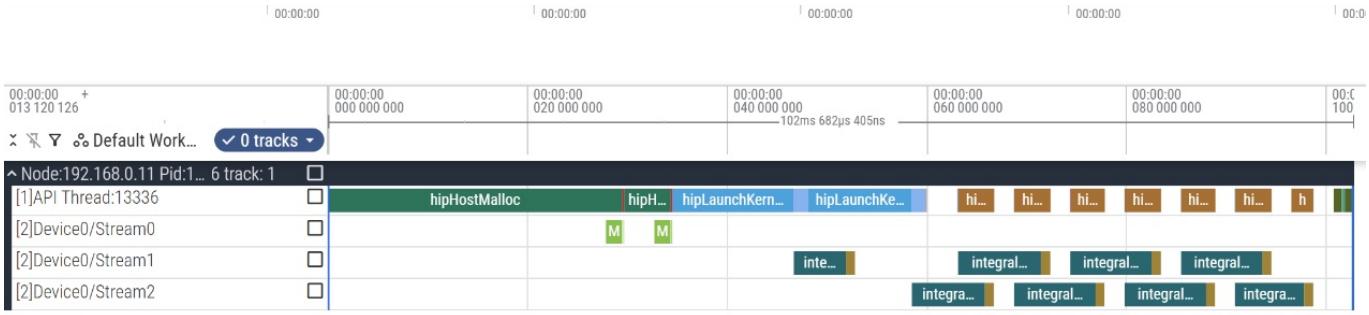


图3-8 程序double_buffer的时间线

这次的耗时进一步缩短了，而且的确实现了当前区间的核函数在时间上与上一个区间的核函数和拷贝重叠。附带提醒一下主机端的空当是每次循环中使用主机求和的部分。

你可能觉得耗时从107 ms缩短到102
ms这样的优化效果并不明显，其中有一部分因素在于当使用多流执行时，会有额外的软硬件调度开销，需要一定的“预热”时间；表现为图中Stream1上第一个核函数从主机端调用到设备端开始执行之间大约12 ms的空当，以及Stream1上第一个核函数与第一次拷贝之间大约6.57
ms的空当。在预热之后，两个流就维持高效执行了。由于这里的例子总的执行时间较短，使得预热占比较大，确实对优化效果产生了一定影响，对于执行时间较长的场景（比如增加积分区间数），优化效果会更加显著。

3.4.3 主机-设备间拷贝与页锁定内存

现在我们来仔细分析上节使用两个流异步并发的具体优化效果，本节先从设备到主机的拷贝开始。

> 建议：将hide_cpu.cc和double_buffer.cc编译并使用hipprof工具运行，在浏览器中打开生成的perfetto文件，按照本节实际查看一下。

使用可视化界面中的选择模式，选择DeviceToHost的色块查看具体耗时，对于我这次的执行，8次拷贝的耗时基本上都是0.9
ms；打开hide_cpu的perfetto文件，同样查看拷贝的时间，得到第一次拷贝大约6.8
ms，后续7次拷贝大约是3.2
ms，对比下来我们使用异步拷贝要快不少。但这里需要说明，导致拷贝变快的原因并不在于同步或异步，而在于我们使用了页锁定主机内存。

要说明这个问题，我们需要详细介绍主机与设备间的内存拷贝。在3.1.7节我们提到过主机与内存间的拷贝会被CPU卸载给DMA（直接内存访问）硬件，DMA在获取到hipMemcpy指定的源地址和目的地址后，不需要CPU参与就可以通过PCIe传递数据。这与将核函数卸载给DCU类似，都是相对CPU异步执行的。但这里的问题是现代操作系统大都使用虚拟内存技术进行内存管理，虚拟内存被分成称为“页”的固定大小的块，操作系统会根据需要（比如缺页时）将页面从内存换出到磁盘，或将磁盘中的页面换入到内

存。DMA是无法禁止操作系统进行页面调度的，因此在DCU执行主机与设备之间的拷贝时，都会将拷贝的内容通过主机上一块页面锁定的缓存区进行中转，以保证传输过程中操作系统不会移动内容所在的页面。换句话说，在DCU内存与用malloc申请的、可以被页面调度的主机内存之间进行内存拷贝，存在主机端将数据在可以被页面调度的主机内存与页面锁定的主机内存间搬运的开销；因为主机上的搬运工作需要CPU来做，导致拷贝是同步的，也使得整个拷贝过程耗时更长。具体到我们的代码，如果没有给res_warp分配页锁定内存，当发布从d_res_warp到res_warp的拷贝后，主机肯定得等待这段拷贝执行完毕，因为DMA只是完成了d_res_warp到页锁定缓存区的拷贝，主机还需要将数据从缓冲区拷贝到res_warp中。

清楚了主机与设备间的拷贝机制后，使用页锁定内存拷贝快的原因就很清楚了。当我们使用hipHostMalloc给res_warp分配内存后，这些内存本来就是页锁定的，因此DMA可以直接将数据从d_res_warp拷贝到res_warp中，省去了主机从缓存区的搬运；此外，由于不需要主机的参与，拷贝也可以是异步的了，这正是上节中强调涉及主机端的异步拷贝，主机端内存必须是页锁定的。顺带提一下，使用页锁定内存还有一种方式，可以通过hipHostRegister接口将使用malloc分配的内存页锁定，使用hipHostUnregister解锁。

现在你可能觉得，既然页锁定内存有这些优势，为什么不把主机端的内存都申请为页锁定内存呢？我们不建议这么做，原因有二，首先是申请页锁定内存本身有一定的时间开销，查看主机端API的耗时，对于我这次运行中两次hipHostMalloc的时间分别是29ms和4.6
ms；另一个原因是如果在主机端分配大量页锁定内存会导致主机端没有足够的可调度的页面，从而影响主机的性能。我们的建议是，仅为需要经常在主机与设备端拷贝的变量分配页锁定内存，分配也尽量在程序的开头与结尾，避免在程序运行中反复分配与释放。

此外，还有两点需要说明，第一是DCU有两个DMA引擎，可以支持同时从DCU传入传出数据，这意味着不同流上的HostToDevice和DeviceToHost拷贝是可以并发的，但如果是同向的拷贝则不能并发；第二，因为拷贝是通过DMA，不需要计算单元参与，因此异步并发拷贝和核函数不会对彼此的性能造成影响，这也正是我们让核函数与拷贝在时间上重叠的意义所在。为了确认这一点，可以观察我们程序的时间线，前7次拷贝的同时都有核函数执行，最后一次拷贝时则没有核函数执行了，但这8次拷贝的时间几乎相同；至于核函数耗时增加（仅使用默认流版本中大约5.85
ms，而使用两个流的版本中核函数最长耗时大约为8 ms）则是由于多个核函数并发所导致的。

3.4.4 事件

在上节的介绍之后，你可能想验证一下，如果仅是核函数与拷贝并发时，核函数是否确实与仅使用默认流时的耗时一致。但目前看想控制核函数仅与另一个流上的拷贝并发也并不容易，因为不同流上的任务在发布之后，执行顺序是不一定的。比如说从程序逻辑来看，主机端肯定先发布当前区间的核函数与拷贝，再发布下个区间的核函数与拷贝；但从时间线来看，下一个区间的核函数却比上一个区间的拷贝先执行，导致与上一个区间的核函数有部分重叠。

对于这个问题，我们可以通过在流中插入事件（event）的方式来解决，它相当于流执行的一个节点。我们不做过多抽象的解释，还是结合代码来说明。首先，我们在Buffer结构体中添加事件成员event，并在alloc和dealloc中加入对它的创建与销毁：

```
struct Buffer
{
  double* res_warp;
  double* d_res_warp;
  hipStream_t stream;
```

```
    hipEvent_t event;
    void alloc(int size) {
        hipHostMalloc(&res_warp,size);
        hipMalloc(&d_res_warp,size);
        hipStreamCreate(&stream);
        hipEventCreate(&event);
    }
    void dealloc() {
        hipHostFree(res_warp);
        hipFree(d_res_warp);
        hipStreamDestroy(stream);
        hipEventDestroy(event);
    }
};
```

之后，我们对在第一个区间的核函数调用之后插入事件：

```
    integral_kernel<<<numBlocks, blockSize,0,buffer[0].stream>>>
            (buffer[0].d_res_warp,lower_bound,N,dx);
    hipEventRecord(buffer[0].event,buffer[0].stream);
    hipMemcpyAsync(buffer[0].res_warp,buffer[0].d_res_warp,size,
            hipMemcpyDeviceToHost,buffer[0].stream);
```

在j循环中调用核函数之前，我们插入对事件的等待，并在核函数调用之后也插入事件：

```
for (int j = 1; j < numIntervals; j++) {
    b1 = j%2; // 当前区间使用的buffer
    b2 = (j-1)%2; // 上一个区间使用的buffer
    //等待上一个区间的核函数执行完毕
    hipStreamWaitEvent(buffer[b1].stream,buffer[b2].event);
    // 或使用
    // hipEventSynchronize(buffer[b2].event);
    integral_kernel<<<numBlocks, blockSize,0,buffer[b1].stream>>>
            (buffer[b1].d_res_warp,lower_bound,N,dx);
    hipEventRecord(buffer[b1].event,buffer[b1].stream);
    hipMemcpyAsync(buffer[b1].res_warp,buffer[b1].d_res_warp,size,
            hipMemcpyDeviceToHost,buffer[b1].stream);
    // 等待上个区间的结果拷贝完毕
    hipStreamSynchronize(buffer[b2].stream);
    for (int i = 0; i < numBlocks; i++) {
        Integral += buffer[b2].res_warp[i];
    }
    lower_bound += range;
}
```

先看一下效果，编译（hide_copy.cc）并使用hipprof运行：

```
hipcc hide_copy.cc -o hide_copy
hipprof --group-stream ./hide_copy
```

打开时间线（清晰起见，仍然修改了拷贝的颜色）：

[图片]

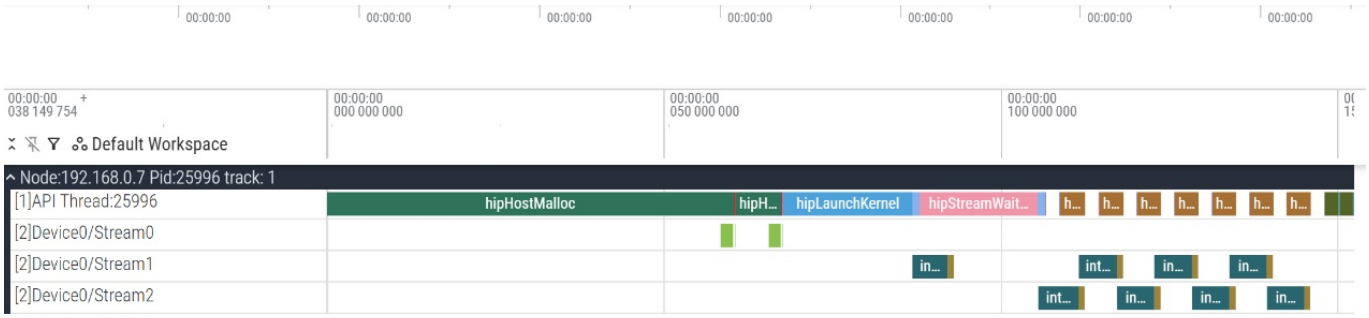


图3-9 程序hide_copy的时间线

可以看到时间线符合我们的初衷，抛开前面的预热，后面的核函数只与前一个区间的拷贝在时间上重叠，而与前一个区间的核函数错开了。选中核函数可以看到执行时间基本都是5.85 ms，和仅使用默认流版本（hide_cpu）的执行时间一致，这也就验证了核函数与拷贝重叠确实不会影响彼此的性能。

现在来介绍上面用到的事件相关API就比较容易了，hipEventRecord的作用是在指定流上记录一个事件，用于监视主机在调用hipEventRecord之前发布到该流上任务的状态，在上面的代码中，我们调用hipEventRecord的作用是监视它前面发布到同一流上的integral_kernel是否完成。hipStreamWaitEvent则是保证主机在调用hipStreamWaitEvent之后发布到该流上的任务必须等待事件前的任务完成后再执行，在上面的代码中，当前区间（使用流buffer[b1].stream）的核函数integral_kernel调用在hipStreamWaitEvent之后，因此它必须等待上一个区间buffer[b2].event记录前发布在buffer[b2].stream上的任务，也就是上一个区间的integral_kernel执行完毕，因此相邻区间的integral_kernel在执行时间上就总是错开的；而主机发布上一个区间的拷贝是在调用hipEventRecord记录事件之后，因此当前区间的核函数不必等待上一个区间的拷贝结束就可以执行，两者在时间上是重叠的。

当然还可以使用时间同步接口hipEventSynchronize，它的作用是让主机等待事件记录前相应流上的任务完成，之后再去执行后续的发布，对于我们的代码，这相当于让主机等上一个区间的核函数执行完再发布当前区间的核函数，同样可以避免两者的重叠。

归结起来，在我们的代码中实际上是利用事件来处理了两个流之间的依赖问题，即我们要求一个流上的核函数要等另一个流上的核函数执行完才能开始。除了这方面的功能外，事件的另一个功能是计时，有一个专门用来计时的API：

```
hipError_t hipEventElapsedTime(float* ms, hipEvent_t start,
                               hipEvent_t stop)
```

它将事件start和stop记录的时间间隔以毫秒为单位写入ms所指的地址中，注意在调用这个API之前需要先使用hipEventSynchronize等待事件stop完成。

> 练习：使用hipEventElapsedTime计时，并与hipprof的计时做比较。

3.4.5 核函数并发执行

现在我们将核函数与拷贝并发搞清楚了，下面将专注于核函数与核函数并发。在3.2.2节中我们提到计算单元中的每个SIMD单元支持并发10个线程束，这些线程束可以来自不同的核函数，这正是多个核函数并

发的硬件基础。前面已经对比了核函数不重叠（hide_cpu和hide_copy）中的核函数性能（5.85 ms）与核函数部分重叠时（double_buffer）中的核函数性能（5.2 ~ 8.3 ms）。我们暂且不考虑预热期间的第一个核函数，测量从第 2 个核函数开始到第 8 个核函数结束的耗时，对于 hide_copy 这段耗时是51.244ms，而 double_buffer 中则是 50.984ms。这样看来，我们让核函数与核函数部分重叠获得的收益并不高，出现这种情况的原因是单个核函数已经将计算单元占的比较满了（单看线程数需要1e8个，这就已经大于支持的最大并发线程数163840），这时再并发更多的核函数也很难得到更高的性能，因为硬件资源需要分配给不同核函数。

我们再尝试一个例子，记得再3.3.6节中，我们还编写了使用核函数sum_kernel求和每个线程束的计算结果的版本（sum_on_dcu.cc），我们也将它的积分区间扩大8倍。

> 练习：先不看下面的代码，仿照对sum_on_cpu.cc的修改，完成对sum_on_dcu.cc的修改，再与下面提供的代码对照。

核函数同样不需要改动，主机代码修改如下：

```
int main()
{
  double lower_bound = 0.0;
  double upper_bound = 40000 * acos(-1); //acos(-1) = pi
  double range = upper_bound - lower_bound;
  int N = 1e8;
  real dx = range / N;
  int numIntervals = 8;
  double Integral[numIntervals];
  double* d_Integral;
  hipMalloc(&d_Integral,numIntervals*sizeof(double));
  // 核函数启动配置
  int blockSize = 64;
  int numBlocks = (N + blockSize - 1) / blockSize;
  // 用于存储每个warp的结果
  int size = numBlocks * sizeof(double);
  double* d_res_warp;
  hipMalloc(&d_res_warp,size);
  // 开始计时
  my_timer timer;
  timer.start();
  for (int j = 0; j < numIntervals; j++) {
    integral_kernel<<<numBlocks, blockSize>>>
              (d_res_warp,lower_bound,N,dx);
    sum_kernel<<<1,blockSize>>>
          (d_res_warp,numBlocks,&d_Integral[j]);
    lower_bound += range;
  }
  hipMemcpy(&Integral,d_Integral,numIntervals*sizeof(double),
        hipMemcpyDeviceToHost);
  // 对所有区间的积分求和
  for (int i = 1; i < numIntervals; i++) {
    Integral[0] += Integral[i];
```

```
  }
  timer.stop();
  printf("Numeric result is: %.11f\n", Integral[0]);
  printf("Result form formula is: %.11f\n",
      F(range*numIntervals) - F(0));
  printf("Elapsed time: %8.6f ms\n",timer.dt);
  hipFree(d_Integral);
  hipFree(d_res_warp);
  return 0;
}
```

在上面的代码中，我们为d_Integral分配了8个双精度浮点数的空间，原因在于核函数sum_kernel中最后是对d_Integral直接赋值而非累加，因此我们对每个积分区间的结果分配一个双精度浮点数。在所有区间计算结束后，使用hipMemcpy一次性将这些结果都拷贝到主机端，最后由主机将结果累加在一起。你完全可以修改一下sum_kernel使其最后对d_Integral累加，这样为d_Integral分配一个双精度浮点数的空间就够了（但别忘了通过hipMemcpy或hipMemset初始化d_Integral为0）。我们采用上面的写法，是想展示在申请设备内存与拷贝时应尽量合并。对于上面的d_Integral，高效的做法是一次性申请64字节的空间，并一次性拷贝这64字节；而非分8次，每次申请/拷贝8个字节，因为在3.4.3节已经介绍过，每次拷贝都会有一系列开销。

言归正传，这次仍然让integral_kernel的总线程数是1e8，编译运行（integral_async路径下的sum_on_dcu.cc）：

```
hipcc sum_on_dcu.cc -o sum_on_dcu
hipprof ./sum_on_dcu
```

打开时间线如下（我们修改了sum_kernel色块的颜色）：

[图片]

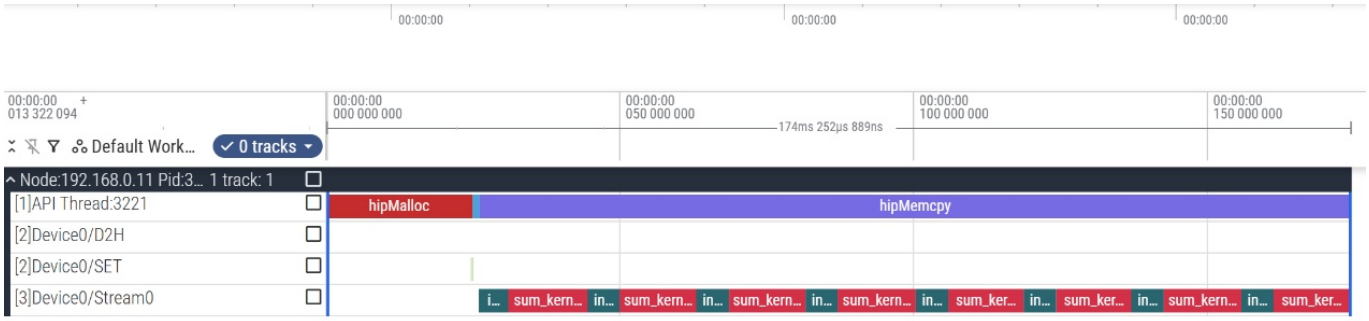


图3-10 程序sum_on_dcu的时间线

统计数据中sum_kernel平均耗时是12.834 ms，integral_kernel是5.542 ms。

下面我们同样使用双缓冲的方法将程序改为使用两个流。这次的结构体Buffer只需要d_res_warp和stream两个成员即可：

```
struct Buffer
{
  double* d_res_warp;
  hipStream_t stream;
  void alloc(int size) {
```

```
    hipMalloc(&d_res_warp,size);
    hipStreamCreate(&stream);
  }
  void dealloc() {
    hipFree(d_res_warp);
    hipStreamDestroy(stream);
  }
};
```

对于主机端代码的修改，相比于double_buffer.cc要容易许多：

```
// 用于存储每个warp的结果
int size = numBlocks * sizeof(double);
Buffer buffer[2];
buffer[0].alloc(size);
buffer[1].alloc(size);
// 开始计时
my_timer timer;
timer.start();
int b; // buffer索引
for (int j = 0; j < numIntervals; j++) {
  b = j%2; // 当前区间使用的buffer
  integral_kernel<<<numBlocks, blockSize,0,buffer[b].stream>>>
            (buffer[b].d_res_warp,lower_bound,N,dx);
  sum_kernel<<<1,blockSize,0,buffer[b].stream>>>
        (buffer[b].d_res_warp,numBlocks,&d_Integral[j]);
  lower_bound += range;
}
hipMemcpy(&Integral,d_Integral,numIntervals*sizeof(double),
      hipMemcpyDeviceToHost);
// 对所有区间的积分求和
for (int i = 1; i < numIntervals; i++) {
  Integral[0] += Integral[i];
}
timer.stop();
printf("Numeric result is: %.11f\n", Integral[0]);
printf("Result form formula is: %.11f\n",
    F(range*numIntervals) - F(0));
printf("Elapsed time: %8.6f ms\n",timer.dt);
buffer[0].dealloc();
buffer[1].dealloc();
return 0;
```

上面的代码不需要像double_buffer.cc中那样包含复杂的逻辑，在j循环中既发布当前区间的任务又等待上一个区间的任务，这是因为这里的求和是在设备端而非主机端，不需要在循环中添加同步，因而不必担心主机因等待设备端的执行导致无法继续发布任务。上面代码的j循环中，只需要让主机一前一后的调用计算当前区间的integral_kernel和sum_kernel，并通过索引b让相邻区间的计算发布在不同流上即可。

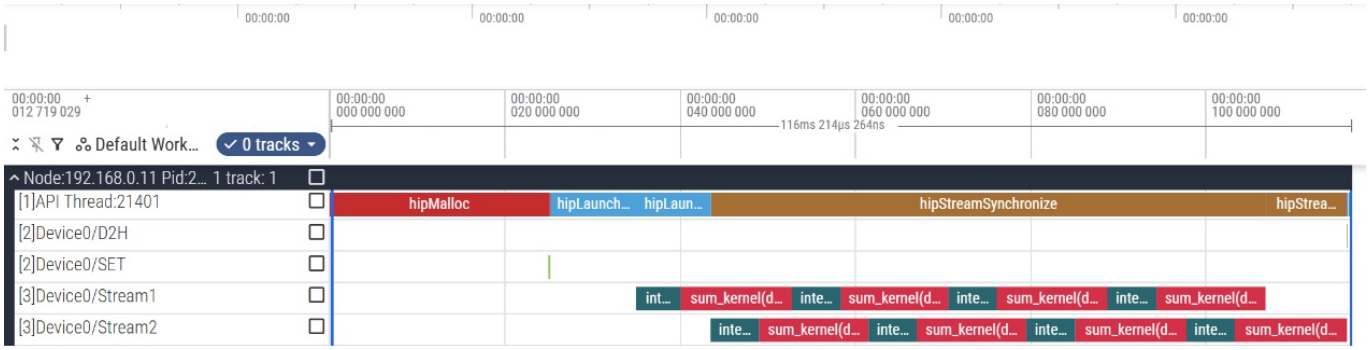
编译并运行（kernel_overlap.cc）：

```
hipcc kernel_overlap.cc -o kernel_overlap
hipprof ./kernel_overlap
```

打开时间线：

[图片]

图3-11 程序kernel_overlap的时间线



这次使用两个流的收益就大多了，如果抛开预热的因素，只算第一次sum_kernel开始到最后一次sum_kernel的耗时，这里是76.870 ms，而sum_on_dcu中则是145.208 ms，几乎快了一倍。从统计数据来看，sum_kernel平均耗时是12.625 ms，integral_kernel是6.215 ms；integral_kernel的平均耗时相比于sum_on_dcu中的只增加了约0.1 ms，却换来与sum_kernel在执行时间上完美的重叠。这里的原因在于sum_kernel使用硬件资源很少（只用了一个线程束），因此与integral_kernel并发时对integral_kernel的性能影响也很小（相当于DCU上少了一个线程束）。

> 尝试：通过修改numBlocks使sum_on_dcu和kernel_overlap中integral_kernel的总线程数是1e5（小于最大并发线程数163840），编译运行并查看并发效果并分析原因。

最后，对于多个核函数并发，我们做一些总结说明。当核函数通过多流在DCU上并发执行时，硬件会根据核函数的资源需求和硬件当前使用情况进行调度，当硬件资源已占满时，硬件会以在并发的核函数间来回切换的方式执行，这样核函数的执行时间相比单独执行时要长，具体长多少取决于并发的几个核函数各自占用的硬件资源，在这种情况下多个核函数并发并不一定会带来更高的性能。顺便提一句，CPU在有大量进程同时运行时也会使用这样的方式，出现多个进程仍在同时在运行，但进程都变得卡顿的现象，不过相比于CPU，DCU在核函数间切换的开支要小得多。但当一个核函数没有占满硬件资源时，多个核函数并发通常能带来性能的大幅提升。

当我们在编写DCU程序时，很多时候未必清楚一个核函数是否能占满硬件资源，而且这种情况还常常随执行时问题规模的动态变化而改变，这时将多个核函数并发执行，将问题扔给硬件调度通常是不错的选择，如果硬件资源已满，结果通常也就是提速不明显，如果硬件资源没用满，就会有很好的收益。

不过仍需提醒，这里依旧存在着权衡，比如在本节中见到的，多流调度起始阶段有一定的预热时间，此外使用两个流时需要双缓冲区，如果使用三个流则需要三缓冲区，等等，仍需仔细权衡。

3.4.6 默认流

在本节的最后，我们来看将核函数发布到默认流会怎样。我们将kernel_overlap.cc中sum_kernel的核函数执行配置改为：

```
sum_kernel<<<1,blockSize,0,0>>>
```

编译运行并打开时间线：

[图片]

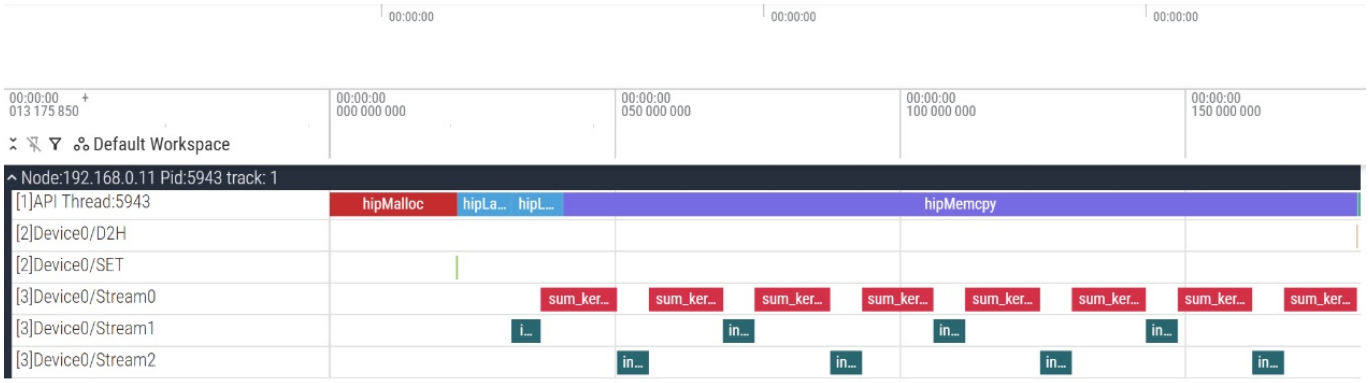


图3-12 将sum_kernel发布到默认流时kernel_overlap的时间线

可以看到，在默认流执行核函数并不会导致主机端同步，即主机线程发布核函数后会继续执行并发布其他任务，而不会等待默认流的核函数执行完毕后再继续；但默认流会让设备端的执行同步，一旦核函数开始在默认流执行，其他流上的核函数必须等待其执行完毕后才能开始。当然你可以利用默认流的性质，使得个别核函数总是单独执行，比如说以此保证它在执行时数据不会被其他核函数搞乱，但是在异步并发的场景下使用默认流会让设备上的执行流程变得更为复杂。

3.4.7 小节

在本节中，我们介绍了如何利用异步并发执行来优化程序。首先，利用了核函数执行与主机执行的异步性，避免了在主机执行计算时DCU处于不必要的空闲状态，以此提升了程序性能。之后，介绍了如何将核函数和拷贝发布在流上以实现异步并发，我们介绍了对于拷贝和核函数由于使用的是不同的硬件单元，因此在并发执行时两者可以在时间上重叠而不会影响彼此的性能，达到很好的优化效果；而对于核函数与核函数的并发，则相当于我们将多个核函数一块交给硬件，由硬件来调度，对于使用硬件资源较多的核函数，效果不一定很好，但如果并发的核函数都只用部分的硬件资源则可以得到很好的收益。

本节中为了能异步执行主机与设备间的拷贝，我们还介绍了页锁定主机内存，并以此为引介绍了主机与设备间拷贝的详细机制；为了处理不同流间的依赖，我们介绍了事件的用法；最后我们还展示了默认流上执行的核函数的行为，这些都是比较重要的。

最后我们建议如果是初学者在程序中使用多流并发时，尽量少的包含同步（除hipStreamSynchronize、hipDeviceSynchronize外，还包括hipMalloc、hipMemcpy、hipMemset、hipFree这些同步的接口），它们会导致主机无法持续向设备发布任务，使并发状态被破坏，还使得代码本身的逻辑以及实际执行的逻辑变得复杂。对于初学者来说，本节中kernel_overlap程序是比较好的范例。

3.5访存优化——以矩阵乘法为例

本节我们要介绍如何使用DCU计算矩阵乘，很多领域的计算都会涉及到矩阵乘。对于一个$m×n$的矩阵A，其矩阵元记为$A{ij}$，一个$n×p$的矩阵B，其矩阵元为$B{ij}$，A与B相乘，得到$m×p

$的矩阵$C$，其矩阵元为：

$C{ij}=k=1nA{ik}B_{kj}$

即每计算C的一个矩阵元，就需要做n次乘法和$n-1$次加法，所以整个矩阵乘需要mnp次乘法和$mpn-1$次加法，因此这是一个On^{3}复杂度的问题，计算量随着矩阵规模以三次函数增长。观察上面的公式，我们发现每个矩阵元C_{ij}的计算是相互独立的，因此非常适合用并行计算来加速，但想要最大化的利用DCU的性能，在编写DCU程序时又有很多值得关注的点。像前几节一样，我们会在本节中编写DCU程序计算矩阵乘并不断改进，以此进一步介绍DCU编程。

我们先简单编写一版CPU代码（matrixMul目录下的cpu.cc），使用my_timer计时看看效果，简单起见，让矩阵都是N维方阵：

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "timer.h"

void matrix_multiply(int N, double *A, double *B, double *C)
{
  int i, j, k;
  for (j = 0; j < N; j++) {
    for (i = 0; i < N; i++) {
      double Cij = 0.0;
      for (k = 0; k < N; k++) {
        // C_{ij} = \sum_k A_{ik} * B_{kj}
        Cij += A[i + k*N] * B[k + j*N];
      }
      C[i + j*N] = Cij;
    }
  }
}

void print_matrix(int N, double* M)
{
  int i, j;
  int max_size = 5; // 打印的最大维度
  for (i = 0; i < N && i < max_size; i++) {
    for (j = 0; j < N && j < max_size; j++) {
      printf("%13.6f ", M[i + j*N]);
    }
    if (N > max_size) { // 超过了最大维度，打印省略号
      printf("... ");
    }
    printf("\n");
  }
  if (N > max_size) { // 超过了最大维度，打印省略号
    printf(".\n.\n.\n");
  }
```

```
}

void generate_matrix(int N, double* M)
{
   int i, j;
   for (j = 0; j < N; j++) {
      for (i = 0; i < N; i++) {
         M[i + j*N] = (double) rand() / (double) RAND_MAX;
      }
   }
}

int main(int argc, char **argv)
{
   if (argc != 2) {
      fprintf(stderr, "Usage: %s N\n", argv[0]);
      return 1;
   }
   int N = atoi(argv[1]);
   if (N < 2) {
      fprintf(stderr, "N must larger than 2.\n");
      return 1;
   }

   // 初始化随机数发生器
   srand(time(NULL));

   // 计时器
   my_timer timer;

   // 分配矩阵内存空间
   double *A = (double *) malloc(N * N * sizeof(double));
   double *B = (double *) malloc(N * N * sizeof(double));
   double *C = (double *) malloc(N * N * sizeof(double));

   // 随机生成矩阵A和B
   timer.start();
   generate_matrix(N, A);
   generate_matrix(N, B);
   timer.stop();
   printf("Generating Matrices took %8.3f ms.\n", timer.dt);

   // 打印矩阵A,B
   printf("Matrix A:\n");
   print_matrix(N, A);
   printf("\n");
   printf("Matrix B:\n");
   print_matrix(N, B);
```

```
    printf("\n");

    // 矩阵乘法，将结果存储在矩阵C中
    timer.start();
    matrix_multiply(N, A, B, C);
    timer.stop();
    printf("Matrix multiplication took %8.3f ms.\n", timer.dt);

    // 打印矩阵C
    printf("Matrix C:\n");
    print_matrix(N, C);

    // 释放内存
    free(A);
    free(B);
    free(C);

    return 0;
}
```

在这段代码里，我们没有使用C语言中的二维数组来存储矩阵，而是视为一维数组，并且采用了列主序的存储方式（注意函数print_matrix和matrix_multiply中引用矩阵元的方式），即同一列的元素在内存中是连续存储的。这是有历史原因，因为诸如BLAS、LAPACK这些线性代数库都是使用Fortran语言编写的，而Fortran语言是列主序的。我们使用列主序可以方便在后续代码中使用BLAS接口风格的库。

矩阵A和B是使用函数generate_matrix随机生成的，利用了C语言标准库中的rand()函数（包含在头文件stdlib.h中，作用是产生0到RAND_MAX间的伪随机数），使得所有矩阵元都是0~1之间的双精度浮点数。在生成矩阵后，使用函数print_matrix将矩阵输出到屏幕上（对于大矩阵，只打印前5行5列的子矩阵）。之后使用matrix_multiply函数将A和B相乘并将结果保存在矩阵C中，之后打印结果。在matrix_multiply中没有采用任何优化手段，仅是按照矩阵乘法的公式编写了最简单的实现。

编译并使用我们的程序计算两个$2×2$矩阵相乘：

```
g++ cpu.cc -o cpu -O3
./cpu 2
Generating Matrices took    0.001 ms.
Matrix A:
    0.041022    0.575630
    0.975030    0.516855

Matrix B:
    0.170363    0.635518
    0.882355    0.999212

Matrix multiplication took    0.001 ms.
Matrix C:
    0.514898    0.601246
    0.622158    1.136096
```

可以使用计算器验证一下，我们的程序是没有问题的。再试试两个$1000×1000$的矩阵相乘：

```
./cpu 1000
Generating Matrices took   18.755 ms.
Matrix A:
   0.404158   0.718073   0.138790   0.999525   0.904998 ...
   0.200418   0.410106   0.639114   0.602230   0.319843 ...
   0.208472   0.098193   0.212182   0.299061   0.239892 ...
   0.331144   0.516972   0.145787   0.030868   0.595335 ...
   0.812551   0.437160   0.547514   0.278571   0.824977 ...
.
.
.

Matrix B:
   0.647890   0.089632   0.320305   0.104900   0.003846 ...
   0.721201   0.027903   0.387046   0.265118   0.312013 ...
   0.775124   0.951991   0.262372   0.277300   0.874698 ...
   0.663479   0.147379   0.351739   0.047627   0.450676 ...
   0.369385   0.029823   0.998284   0.239315   0.682138 ...
.
.
.

Matrix multiplication took 1257.347 ms.
Matrix C:
 256.739709  251.385397  271.003417  258.734088  251.304090 ...
 248.442402  237.363746  258.863536  246.663164  244.148036 ...
 235.748062  238.223707  248.997626  243.717928  242.310350 ...
 249.627830  246.672617  260.516637  255.078815  246.545102 ...
 240.161690  238.072973  249.207898  249.527630  243.100153 ...
.
.
.
```

> 尝试：使用我们的程序计算更多方阵的乘积（建议不要过大），并尝试不同的编译器优化级别，观察耗时与矩阵规模N的关系，思考当矩阵规模越来越大时程序还有哪些性能瓶颈。

3.5.1 使用DTK中的库

下面我们开始编写DCU程序，在2.1.2节中，我们提到过DTK软件栈中有很多库，这里我们就先使用DTK软件栈中的hipRAND库来随机生成矩阵，再使用hipBLAS库来计算矩阵乘。这样，一来简单的展示一下怎样使用DTK软件栈中的库，二来也看看我们在DCU上能达到的上限，并作为我们自己编写DCU程序的优化目标。

编写DCU程序如下（use_libs.cc）：

```
#include <stdio.h>
#include <hip/hip_runtime.h>
```

```
#include <hiprand/hiprand.h>
#include <hipblas.h>
#include <time.h>

void print_matrix(int N, double* M)
{
  int i, j;
  int max_size = 5; // 打印的最大维度
  for (i = 0; i < N && i < max_size; i++) {
    for (j = 0; j < N && j < max_size; j++) {
      printf("%13.6f ", M[i + j*N]);
    }
    if (N > max_size) { // 超过了最大维度，打印省略号
      printf("... ");
    }
    printf("\n");
  }
  if (N > max_size) { // 超过了最大维度，打印省略号
    printf(".\n.\n.\n");
  }
}

int main(int argc, char **argv)
{
  if (argc != 2) {
    fprintf(stderr, "Usage: %s N\n", argv[0]);
    return 1;
  }
  int N = atoi(argv[1]);
  if (N < 2) {
    fprintf(stderr, "N must larger than 2.\n");
    return 1;
  }

  // 初始化hipRAND随机数发生器
  hiprandGenerator_t gen;
  hiprandCreateGenerator(&gen, HIPRAND_RNG_PSEUDO_DEFAULT);
  hiprandSetPseudoRandomGeneratorSeed(gen, time(NULL));

  // 计时器
  hipEvent_t start, stop;
  hipEventCreate(&start);
  hipEventCreate(&stop);
  float dt;

  // 分配矩阵内存空间
  double *A = (double *) malloc(N * N * sizeof(double));
  double *B = (double *) malloc(N * N * sizeof(double));
```

```
double *C = (double *) malloc(N * N * sizeof(double));
double *dA, *dB, *dC;
hipMalloc(&dA, N * N * sizeof(double));
hipMalloc(&dB, N * N * sizeof(double));
hipMalloc(&dC, N * N * sizeof(double));

// 随机生成矩阵A和B
hipEventRecord(start);
hiprandGenerateUniformDouble(gen, dA, N * N);
hiprandGenerateUniformDouble(gen, dB, N * N);
hipEventRecord(stop);
hipEventSynchronize(stop);
hipEventElapsedTime(&dt, start, stop);
printf("Generating Matrices took %8.3f ms.\n", dt);

// 打印矩阵A,B
hipMemcpy(A, dA, N * N * sizeof(double),
      hipMemcpyDeviceToHost);
hipMemcpy(B, dB, N * N * sizeof(double),
      hipMemcpyDeviceToHost);
printf("Matrix A:\n");
print_matrix(N, A);
printf("\n");
printf("Matrix B:\n");
print_matrix(N, B);
printf("\n");

// 创建hipBLAS句柄
hipblasHandle_t handle;
hipblasCreate(&handle);

// 矩阵乘法，将结果存储在矩阵C中
const double alpha = 1.0;
const double beta = 0.0;
// 预热
hipblasDgemm(handle, HIPBLAS_OP_N, HIPBLAS_OP_N, 2, 2, 2,
        &alpha, dA, N, dB, N, &beta, dC, N);
// 正式计算
hipEventRecord(start);
hipblasDgemm(handle, HIPBLAS_OP_N, HIPBLAS_OP_N, N, N, N,
        &alpha, dA, N, dB, N, &beta, dC, N);
hipEventRecord(stop);
hipEventSynchronize(stop);
hipEventElapsedTime(&dt, start, stop);
printf("Matrix multiplication took %8.3f ms.\n", dt);

// 打印矩阵C
hipMemcpy(C, dC, N * N * sizeof(double),
```

```
        hipMemcpyDeviceToHost);
    printf("Matrix C:\n");
    print_matrix(N, C);

    // 清理
    hipFree(dA);
    hipFree(dB);
    hipFree(dC);
    hipEventDestroy(start);
    hipEventDestroy(stop);
    hiprandDestroyGenerator(gen);
    hipblasDestroy(handle);
    free(A);
    free(B);
    free(C);

    return 0;
}
```

在上面的代码中，我们为矩阵在DCU上申请了内存dA、dB、dC。对于库的使用方面，代码中首先include了用到的hipRAND库和hipBLAS库的头文件，创建了hipRAND的随机数生成器gen，并为其设置了伪随机数种子，之后使用了hiprandGenerateUniformDouble函数，其作用是产生0~1之间均匀分布的双精度伪随机数，我们用它生成了dA和dB的矩阵元；对于矩阵乘，我们先创建了hipBLAS的句柄handle，之后使用hipBLAS的库函数hipblasDgemm计算dA与dB的乘积。hipblasDgemm的功能更具普遍性，可以对$m×n$的矩阵OpA、$n×k$的矩阵OpB，和$m×k$的矩阵C计算形式为$C⊠αOpAOpB+βC$的问题，其中$Op(
)$可以是转置或转置复共轭（Herimtian共轭），也可以不做操作，这里我们只是用它计算了很简单的情况。

我们没有使用之前的my_timer计时器，而是利用了3.4.4事件一节中提到的hipEventElapsedTime接口来计时。注意到，我们在调用参数m、n、k都为N的hipblasDgemm之前，先令m、n、k都为2调用了一次，这是为了计时准确而做的“预热”。因为矩阵乘法非常重要，而其性能又同矩阵规模密切相关，因此hipBLAS中包含了大量的核函数，用于在程序运行时，根据hipblas?gemm（“?”代表不同精度，目前支持半精度H、单精度S、双精度D、单精度复数C和双精度复数Z）的规模选择性能最优的核函数来执行，而当前版本的hipBLAS库会在第一次调用hipblas?gemm时将这些核函数全部加载到内存里，在我的计算机里这需要大约3秒的时间，我们需要将这段时间排除。

使用hipcc编译，这次需要链接hipBLAS和hipRAND的动态库（此外低于23.04的DTK版本需要额外指定头文件路径-I$ROCM_PATH/rocrand/include）：

```
hipcc use_libs.cc -o use_libs -I$ROCM_PATH/rocrand/include -L$ROCM_PATH/lib -lhipblas -lhiprand
```

运行程序use_lib，令N等于4096：

```
Generating Matrices took    5.450 ms.
Matrix A:
    0.798041    0.043402    0.178880    0.356740    0.575366 ...
    0.836317    0.664488    0.877761    0.791172    0.515096 ...
    0.499405    0.304049    0.046138    0.740579    0.088408 ...
    0.500394    0.360916    0.580063    0.067178    0.363289 ...
```

```
   0.010420    0.664991    0.032631    0.443933    0.858081 ...
.
.
.

Matrix B:
   0.708751    0.067173    0.965586    0.871611    0.974055 ...
   0.552622    0.625481    0.081746    0.607098    0.587923 ...
   0.742625    0.765052    0.133508    0.116854    0.348009 ...
   0.044137    0.759527    0.606604    0.640436    0.216237 ...
   0.386603    0.200040    0.233368    0.187415    0.674187 ...
.
.
.

Matrix multiplication took   28.288 ms.
Matrix C:
 1015.087370  1021.600700  1007.313120  1033.348788  1026.125069 ...
 1048.421677  1032.788823  1011.338992  1035.188871  1021.101648 ...
 1010.991994  1013.646259  1018.475126  1015.856618  1020.496487 ...
 1004.955157  1020.468667  1015.666630  1010.192140  1015.513084 ...
 1012.398788  1012.408923  1029.628456   997.582603  1015.931811 ...
.
.
.
```

我们来计算一下，按DCU在28.288
ms内计算了4096^{3}次乘加运算（乘加算做了两次浮点运算），DCU达到了4.858 TFLOPS（floating point operations per second）的性能。

令N等于16384，hipblasDgemm耗时：

```
Matrix multiplication took 1736.475 ms.
```

这次DCU达到了5.065 TFLOPS的性能。

> 尝试使用BLAS、OpenBLAS、MKL等使用CPU计算的线性代数库中的Dgemm函数，与hipBLAS做一些性能对比。

3.5.2 初步编写核函数

我们编写一个简单的核函数，看看与hipblasDgemm的性能差距。

```
__global__
void matrix_multiply(int N, double *A, double *B, double *C)
{
   int tidx = blockIdx.x * blockDim.x + threadIdx.x;
   int stridex = gridDim.x * blockDim.x;
```

```
  int tidy = blockIdx.y * blockDim.y + threadIdx.y;
  int stridey = gridDim.y * blockDim.y;
  int i, j, k;
  for (j = tidy; j < N; j += stridey) {
    for (i = tidx; i < N; i += stridex) {
      double Cij = 0.0;
      for (k = 0; k < N; k++) {
        // C_{ij} = \sum_k A_{ik} * B_{kj}
        Cij += A[i + k*N] * B[k + j*N];
      }
      C[i + j*N] = Cij;
    }
  }
}
```

删去打印矩阵的函数，转而添加一个主机端函数来比较核函数与hipblasDgemm的计算结果：

```
void compare_matrices(int N, double* C1, double* C2)
{
  int i, j;
  double max_error = 0.0;
  for (j = 0; j < N; j++) {
    for (i = 0; i < N; i++) {
      double error = fabs(C1[i+j*N] - C2[i+j*N]);
      if (error > max_error) {
        max_error = error;
      }
    }
  }
  printf("Max error: %.4e\n", max_error);
}
```

在main函数中，申请C_kernel和C_hipblas分别存储核函数与hipBLAS的计算结果：

```
  double *C_kernel = (double *) malloc(N * N * sizeof(double));
double *C_hipblas= (double *) malloc(N * N * sizeof(double));
```

计算部分改动如下：

```
  // 正式计算
  hipEventRecord(start);
  hipblasDgemm(handle, HIPBLAS_OP_N, HIPBLAS_OP_N, N, N, N,
          &alpha, dA, N, dB, N, &beta, dC, N);
  hipEventRecord(stop);
  hipEventSynchronize(stop);
  hipEventElapsedTime(&dt, start, stop);
  printf("Matrix multiplication (hipblasDgemm)"
      "took %8.3f ms.\n", dt);

  // 将hipBLAS计算的结果拷贝回主机
```

```
    hipMemcpy(C_hipblas, dC, N * N * sizeof(double),
            hipMemcpyDeviceToHost);

    // 将dC清零
    hipMemset(dC, 0, N * N * sizeof(double));

    // 使用核函数计算
    dim3 blockSize = dim3(Nsub, Nsub, 1);
    int Nblk = (N + Nsub - 1) / Nsub;
    dim3 numBlocks = dim3(Nblk, Nblk, 1);
    hipEventRecord(start);
    matrix_multiply<<<numBlocks, blockSize>>>(N, dA, dB, dC);
    hipEventRecord(stop);
    hipEventSynchronize(stop);
    hipEventElapsedTime(&dt, start, stop);
    printf("Matrix multiplication (kernel)"
           "took %8.3f ms.\n", dt);

    // 将核函数计算的结果拷贝回主机并与hipBLAS的结果做比较
    hipMemcpy(C_kernel, dC, N * N * sizeof(double),
            hipMemcpyDeviceToHost);
  compare_matrices(N, C_kernel, C_hipblas);
```

先使用hipblasDgemm计算dA与dB相乘，保存在dC中，计算完成后将dC中的结果拷贝到C_hipblas；接着使用核函数matrix_multiply再算一遍，计算完成后将结果拷贝到C_kernel中，最后用compare_matrices逐个元素的比较两者的计算结果。

图3-13示意了核函数matrix_multiply的线程分工，我们使用2维的线程块和线程网格，线程索引的x分量对应矩阵的行，y分量对应矩阵的列，每个线程负责计算C的一个矩阵元C_{ij}，为此需要读取矩阵A的i行和矩阵B的j列，并做乘累加。目前我们先使用（8，8，1）的线程块（一个线程束）来启动核函数，为此在文件一开始定义Nsub为8：

```
#define Nsub 8
```

我们启动足够多的线程块，让每个线程计算一个矩阵元即可。从分块矩阵乘的角度，每个线程块计算了矩阵A的$8×N$的子矩阵与矩阵B的$N×8$的子矩阵的乘积，得到矩阵C的$8×8$的子矩阵（因此将线程块单个维度命名为N_{sub}）。

[图片]

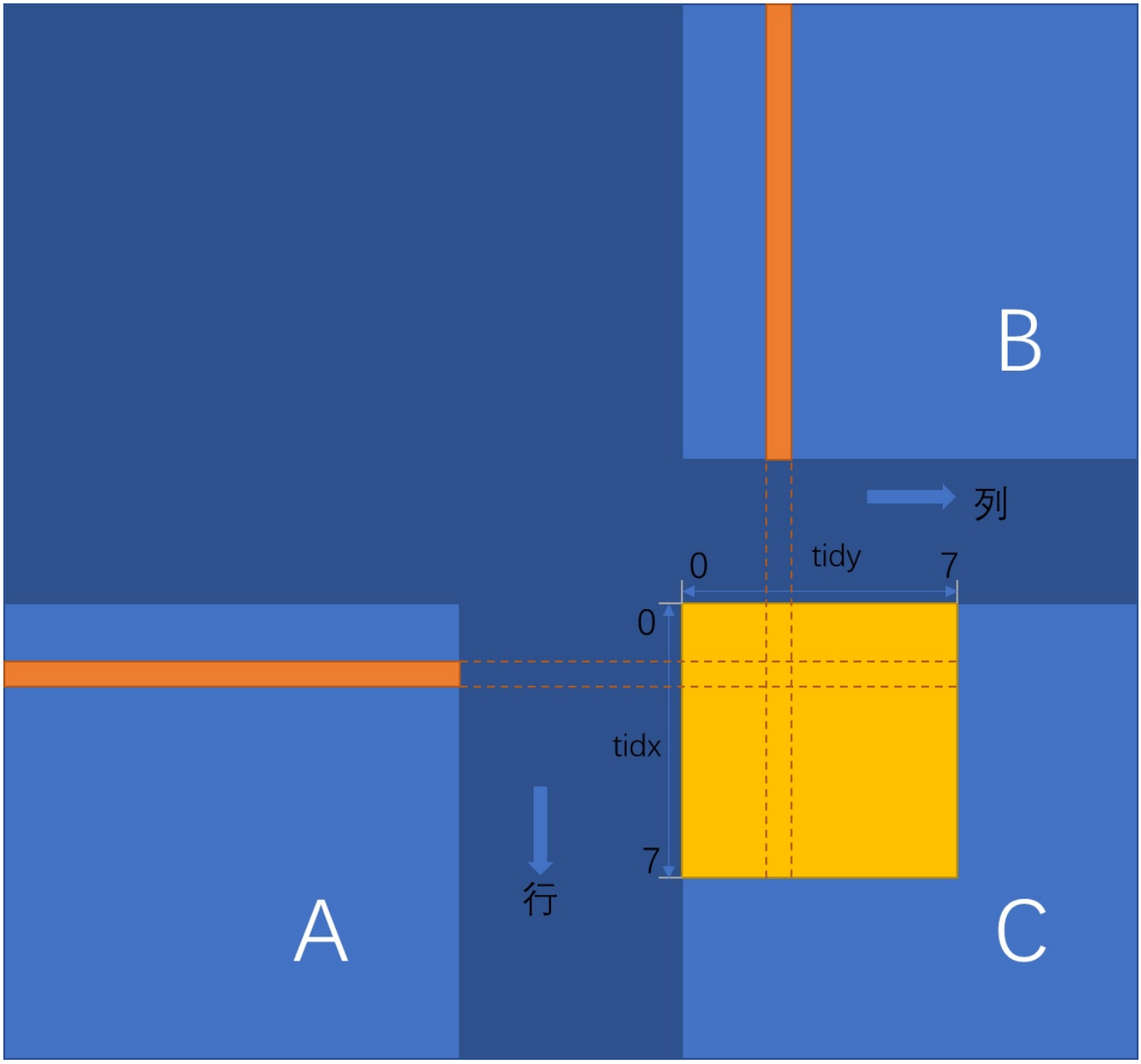


图3-13 核函数matrix_multiply

编译运行如下（[dcu.cc](https://vscode-remote+wsl-002bubuntu.vscode-resource.vscode-cdn.net/home/cwoody/dcu_developer/DeveloperGuide/dcu_programming_2023/codes/matrixMul/dcu.cc)）：

hipcc dcu.cc -o dcu -I$ROCM_PATH/rocrand/include -L$ROCM_PATH/lib -lhipblas -lhiprand

```
./dcu 4096
Generating Matrices took    7.533 ms.
Matrix multiplication (hipblasDgemm) took   28.389 ms.
Matrix multiplication (kernel) took  478.200 ms.
Max error: 0.0000e+00
```

我们编写的核函数的计算结果与hipBLAS库的完全一致，但是性能相去甚远，需要设法改进。观察图3-13，我们很快会发现多个线程会使用矩阵A和B的同一部分，即同一行的N个线程（tidx相同，tidy不同）都需要矩阵A的i行，因此A的每一行都需要从全局内存读取N次，同一列的N个线程（tidy相同，tidx不同）都需要矩阵B的j列，因此B的每一列也都需要从全局内存读取N次。这样算下来，整个核函数执行下来

，矩阵A和B的每个矩阵元都需要从全局内存读取到寄存器N次，当然在这些读取中DCU的多级缓存结构会起到作用，这些矩阵元并非每次都得从片外内存获取，但这一过程完全由硬件决定，我们是无法控制的。如果我们想要自己将反复用到的矩阵元缓存下来，需要使用共享内存。

3.5.3 使用共享内存优化

在上节中，我们看到按目前的核函数写法和执行配置（包含一个线程束的$8×8$的线程块），每个线程块是在做矩阵A的8行构成的子矩阵（$8×N$）和矩阵B的8列构成的子矩阵（$N×8$）的矩阵乘，得到矩阵C的一个$8×8$的子矩阵。在计算过程中A的行和B的列会被反复使用，每次都需要从全局内存中读取。本节我们尝试使用共享内存利用这种数据复用的特点对程序进行优化，先将子矩阵从全局内存读取到共享内存，之后多个线程用到同一矩阵元都从共享内存中读取，以利用共享内存相比全局内存更快的访存速度。

对于我们目前的程序，一个简单的想法是将上述这两个子矩阵全部保存到共享内存中，首先计算一下所需的内存大小，对于矩阵元为双精度浮点数的两个子矩阵需要$2×8×N×8B=128NB$的空间。在3.2.2节中我们提到每个计算单元有64KB的共享内存，所以只有当N小于等于512时才有可能一次性将这两个子矩阵读入共享内存，对于N更大的情况，需要将子矩阵进一步分块，因此我们不妨直接从分块矩阵乘的角度来考虑。

对于分块矩阵乘法，公式

$C{ij}=k=1N{block}A{ik}B{kj}$

依然成立，但此时$C{ij}$、$A{ik}$、$B{kj}$不是一个矩阵元，而是子矩阵。这些子矩阵不能随意划分，要求$A{ik}$的列数与$B{kj}$的行数相等，这样两者才能相乘，得到的$C{ij}$的行数与$A{ik}$的行数相等，列数与$B{kj}$的列数相等。我们按这个公式重新编写核函数，将$N×N$的矩阵A、B、C划分为$N{block}×N{block}$个子矩阵，每个子矩阵都是$N{sub}×N{sub}$的方阵。

我们先编写一个设备端函数用于将矩阵M的元素从全局内存读到共享内存中的子矩阵$M{ij}$中，每个线程负责将矩阵M中Row行Col列的元素读取到$M{ij}$中的row行col列，编写这个函数的关键是搞清楚Row与row、Col与col的关系：

```
__device__
void read_matrix(double* M, int N, int i, int j,
                 double* M_ij, int LD)
{
  // 矩阵元在子矩阵M_ij中是row行col列
  int row = threadIdx.x;
  int col = threadIdx.y;
  // 矩阵元在M_ij中的偏移
  int offset = row + col * LD;
  // 矩阵元在M中是Row行Col列
  int Row = i * Nsub + row;
  int Col = j * Nsub + col;
  // 矩阵元在M中的偏移
  int Offset = Row + Col * N;
  // 没有越界则读取，否则置零
  if ((Row < N) && (Col < N)) {
```

```
    M_ij[offset] = M[Offset];
  } else {
    M_ij[offset] = 0.0;
  }
}
```

在函数read_matrix中，我们使用了LD（Leading Dimension）这个变量，这同样是按照BLAS中命名方式，在列主序中，这个量是同一行相邻两列元素在内存中的间隔。

注意到，在read_matrix中我们让线程索引的x分量threadIdx.x对应矩阵的行row，这是因为在3.2.2中提到的，让线程ID连续的线程访问内存中连续的地址时，DCU可以将这些访存请求合并，以此提高访存效率。如果反过来，让threadIdx.x对应矩阵的列col，则线程ID连续的线程访问的内存地址间隔可能很远（当N增大时），DCU就无法将这些访存请求合并了。此外，我们考虑了$N{sub}$不能整除N的情况，将$M{ij}$中超过M边界的元素都置为0。现在可以将核函数改写如下：

```
__global__
void matrix_multiply(int N, int Nblk, double *A, double *B,
                     double *C)
{
  // 声明A_sub,B_sub用于保存A,B的子矩阵
  __shared__ double A_sub[Nsub * Nsub];
  __shared__ double B_sub[Nsub * Nsub];
  // 每个线程块负责计算子矩阵C_ij
  for (int i = blockIdx.x; i < Nblk; i += gridDim.x) {
  for (int j = blockIdx.y; j < Nblk; j += gridDim.y) {
    // 每个线程负责计算子矩阵C_ij中row行col列的矩阵元
    int row = threadIdx.x;
    int col = threadIdx.y;
    double C_sub = 0.0;
    for (int k = 0; k < Nblk; k++) {
      // 将子矩阵A_ik从全局内存读到共享内存
      read_matrix(A, N, i, k, A_sub, Nsub);
      // 将子矩阵B_kj从全局内存读到共享内存
      read_matrix(B, N, k, j, B_sub, Nsub);
      // 计算矩阵A_ik和B_kj的乘
      for (int l = 0; l < Nsub; l++) {
        C_sub += A_sub[row + l * Nsub]
              * B_sub[l + col * Nsub];
      }
    } // k
    // 将矩阵元写到C中
    int Row = i * Nsub + row;
    int Col = j * Nsub + col;
    if ((Row < N) && (Col < N)) {
      C[Row + Col * N] = C_sub;
    }
  } // j
  } // i
```

```
}
```

在这个版本的核函数中，每个线程块负责计算子矩阵$C{ij}$，每个线程负责计算子矩阵中的一个矩阵元（变量C_sub,位于子矩阵$C{ij}$的row行col列）。计算流程是先将子矩阵$A{ik}$和子矩阵$B{kj}$从全局内存读取到共享内存，将这两个子矩阵相乘后累加到C_sub上，将k从0至Nblk循环最终完成计算，将结果写回矩阵C中。

这样，对全局内存的读取变成了线程网格中同一行的Nblk个线程块（blockIdx.x相同，blockIdx.y不同）都需要矩阵A的i行子矩阵，因此A的每一行子矩阵都需要从全局内存读取Nblk次，同一列的Nblk个线程块（blockIdx.y相同，blockIdx.x不同）都需要B的j列子矩阵，因此B的每一列子矩阵也都需要从全局内存读取Nblk次。整个核函数执行下来，矩阵A和矩阵B的每个子矩阵都需要从全局内存读取到共享内存Nblk次，也就是每个矩阵元都需要从全局内存读取到共享内存Nblk次，对比上节最后的分析，可知对全局内存的访问由N次减少为Nblk次，变为之前的$1/N_{sub}$，而线程块内多个线程对同一矩阵元的重复读取都是从共享内存中读的。

注意核函数的参数中增加了矩阵在单个维度的分块数Nblk，因此修改main函数中对核函数的调用为：

```
  matrix_multiply<<<numBlocks, blockSize>>>
            (N, Nblk, dA, dB, dC);
```

编译运行（shared.cc）如下：

```
hipcc shared.cc -o shared -I$ROCM_PATH/rocrand/include -L$ROCM_PATH/lib -lhipblas -lhiprand
./shard 4096
Generating Matrices took    5.643 ms.
Matrix multiplication (hipblasDgemm) took   28.706 ms.
Matrix multiplication (kernel) took  465.965 ms.
Max error: 0.0000e+00
```

效果不明显，我们试试增大线程块的大小Nsub，进一步减少对全局内存的读取，定义Nsub为16。但注意此时我们需要在核函数中加上同步了，因为$16×16$的线程块包含4个线程束，我们需要让所有线程束完成从全局内存到共享内存的读取之后（这里的关键是一个线程束会用到由其他线程束读取到共享内存的数据），再开始利用共享内存中的数据进行计算，还需要让当前k循环计算结束后才能进入下一个k循环做下一组全局内存到共享内存的读取。为此，加入两个同步：

```
    __syncthreads();
    // 计算矩阵A_ik和B_kj的乘
    for (int l = 0; l < Nsub; l++) {
      C_sub += A_sub[row + l * Nsub]
            * B_sub[l + col * Nsub];
    }
    __syncthreads();
```

再次编译运行，结果如下：

```
./shared 4096
Generating Matrices took    5.777 ms.
Matrix multiplication (hipblasDgemm) took   28.971 ms.
Matrix multiplication (kernel) took  174.049 ms.
Max error: 0.0000e+000
```

可以看到速度提升了两倍多，再将Nsub定义为32（记得为核函数加上__launch_bounds__(1024)），

结果如下：

```
./shared 4096
Generating Matrices took    5.597 ms.
Matrix multiplication (hipblasDgemm) took   28.669 ms.
Matrix multiplication (kernel) took  130.636 ms.
Max error: 0.0000e+00
```

> 相比shared.cc中Nsub的大小对核函数性能的显著影响，Nsub的大小对于dcu.cc中核函数性能的影响则小很多，可以尝试修改并测试。

3.5.4 让每个线程计算更多的矩阵元

当你想继续增加Nsub时，可能会想到$32×32=1024$已经达到单个线程块中线程数的极限了。但有了之前章节的经验，你应该知道解决的办法，即每个线程不必只计算子矩阵$C{ij}$的一个矩阵元，而是可以多算几个，这样子矩阵就可以比线程块更大了。比如说每个线程负责将子矩阵$A{ik}$和$B{kj}$的两行两列的4个元素从全局内存读到共享内存，计算时负责从共享内存中的$A{ik}$中取两行，$B{kj}$中取两列，算出子矩阵$C{ij}$的4个元素，这样就可用$32×32$的线程块处理$64×64$的子矩阵了。

不过$64×64$的子矩阵又达到了共享内存的极限，因为此时A_sub和B_sub加起来所需的空间为$2×64×64×8B=64KB$刚好为单个计算单元中共享内存的配额。此时如果仍想继续增加Nsub，需要将矩阵A与矩阵B重新分块，令$A{ik}$为$N{sub}×N{sub}^{‘}$，$B{kj}$为$N{sub}^{’}×N{sub}$的矩阵，这样矩阵C的分块仍保持不变（线程网格也保持不变），通过$N_{sub}^{‘}$可以调整所需的共享内存空间。

有了这些编程思路，我们可以进一步优化程序了，但为了让程序不显得太复杂，突出我们的优化重点，从本节开始我们会省去边界检查，只考虑$N{sub}$和$N{sub}^{’}$都能整除N的情况。

> 作为练习，你可以尝试编写包含边界检查的版本，以处理N任意取值的情况。类似我们在shared.cc的read_matrix中所写的，如果行列没有超过全局内存中矩阵M的边界则将元素读入共享内存中的子矩阵的适当位置，否则将子矩阵该位置的元素置零。但我们之前的写法是有优化空间的，因为只有当子矩阵包含矩阵的最后一行或列时才需要做边界检查，而不需要对所有子矩阵做边界检查。

现在开始优化，首先通过宏定义一些常数，包括$N{sub}$、$N{sub}^{‘}$（记为Nsub_）、线程块的维度BLOCKDIM_X和BLOCKDIM_Y以及每个线程需要处理的A的子矩阵的行数Nx、B的子矩阵的列数Ny：

```
#define Nsub  64
#define Nsub_ 64
#define BLOCKDIM_X 32
#define BLOCKDIM_Y 32
#define Nx Nsub / BLOCKDIM_X
#define Ny Nsub / BLOCKDIM_Y
```

比如上面的定义就表明我们想用$32×32$的线程块来处理矩阵A、B、C都划分成一系列$64×64$的子矩阵的情况，每个线程需要处理子矩阵$A{ik}$中的两行（Nx等于2）与子矩阵$B{kj}$中的两列（Ny也等于2）的内积，算出子矩阵C_{ij}的4个（Nx乘Ny）元素。

接下来是改写read_matrix函数，因为现在$A{ik}$和$B{kj}$的行列数可以是不同的，此外着眼于性能，

我们希望在编译时就把更多的值确定下来，因此使用函数模板：

```
template <int nrows, int ncols>
__device__
void read_matrix(double* M, int N, int i, int j,
            double* M_ij, int LD)
{
  int row, col, offset, Row, Col, Offset;
  // 子矩阵行列数与线程块行列数的比值
  constexpr int Rrow = nrows / BLOCKDIM_X;
  constexpr int Rcol = ncols / BLOCKDIM_Y;
  for (int J = 0; J < Rcol; J++) {
    for (int I = 0; I < Rrow; I++) {
      // 矩阵元在子矩阵M_ij中的行列
      row = threadIdx.x + I * BLOCKDIM_X;
      col = threadIdx.y + J * BLOCKDIM_Y;
      offset = row + col * LD;
      // 矩阵元在矩阵M中的行列
      Row = i * nrows + row;
      Col = j * ncols + col;
      Offset = Row + Col * N;
      // 将矩阵元从全局内存读入共享内存
      M_ij[offset] = M[Offset];
    } // I
  } // J
}
```

在这一版的read_matrix中，每个线程总共需要读取的线程数由子矩阵行列数与线程块行列数的比值Rrow和Rcol来确定（R代表ratio），constexpr表示我们要求这两个值在编译时就确定下来。举例来说，当子矩阵的行数nrows和列数ncols都为64，线程块的行列数BLOCKDIM_X和BLOCKDIM_Y都为32时，Rrow和Rcol都等于2，因此对I和J的两重循环总共循环四次，所以每个线程读取了4个矩阵元。

将子矩阵读取到共享内存之后，下一步可以计算子矩阵的乘积了，由于现在每个线程需要计算多个矩阵元，相比之前的代码要复杂一些，因此我们也专门编写一个设备端函数multiply_submatrices：

```
__device__
void multiply_submatrices(double* A_sub, double* B_sub,
                double* c)
{
  for (int k = 0; k < Nsub_; k++) {
    for (int J = 0; J < Ny; J ++) {
      for (int I = 0; I < Nx; I ++) {
        // 每个线程计算Nx * Ny个c
        int i = threadIdx.x + I * BLOCKDIM_X;
        int j = threadIdx.y + J * BLOCKDIM_Y;
        c[I + J * Nx] += A_sub[i + k * Nsub]
                *B_sub[k + j * Nsub_];
      } // I
    } // J
```

```
  } // k
}
```

在这个函数中，我们让每个线程负责Nx行，Ny列，行与行之间间隔BLOCKDIM_X，列与列间隔BLOCKDIM_Y，计算出Nx乘Ny个C的元素，保存在局部内存中的数组c中。

类似的，再编写一个用于将矩阵元写回矩阵C中的函数write_matrix：

```
__device__
void write_matrix(double* C, int N, int i, int j, double* c)
{
  int row, col, Row, Col;
  for (int J = 0; J < Ny; J++) {
    for (int I = 0; I < Nx; I++) {
      row = threadIdx.x + I * BLOCKDIM_X;
      col = threadIdx.y + J * BLOCKDIM_Y;
      Row = i * Nsub + row;
      Col = j * Nsub + col;
      if ((Row < N) && (Col < N)) {
        C[Row + Col * N] = c[I + J * Nx];
      }
    }
  }
}
```

此时的核函数matrix_multiply简单明了：

```
__global__ __launch_bounds__(BLOCKDIM_X * BLOCKDIM_Y)
void matrix_multiply(int N, int Nblk, double *A, double *B,
                     double *C)
{
  // 共享内存中的A_sub,B_sub用于保存A,B的子矩阵
  __shared__ double A_sub[Nsub * Nsub_];
  __shared__ double B_sub[Nsub_ * Nsub];
  // 每个线程块负责计算子矩阵C_ij
  for (int i = blockIdx.x; i < Nblk; i += gridDim.x) {
  for (int j = blockIdx.y; j < Nblk; j += gridDim.y) {
    // 每个线程负责计算子矩阵C_ij中Nx*Ny个矩阵元
    double c[Nx * Ny];
    for (int k = 0; k < Nx * Ny; k++) {
      c[k] = 0.0;
    }
    // 假设Nsub_能整除N
    for (int k = 0; k < N / Nsub_; k++) {
      // 将子矩阵A_ik从全局内存读到共享内存
      read_matrix<Nsub, Nsub_>(A, N, i, k, A_sub, Nsub);
      // 将子矩阵B_kj从全局内存读到共享内存
      read_matrix<Nsub_, Nsub>(B, N, k, j, B_sub, Nsub_);
      __syncthreads();
      // 计算矩阵A_ik和B_kj的乘
```

```
        multiply_submatrices(A_sub, B_sub, c);
        __syncthreads();
      } // k
      // 将矩阵元写到C中
      write_matrix(C, N, i, j, c);
    } // j
  } // i
}
```

流程与之前版本是一致的，区别主要有三点：A_sub与B_sub的大小不同；需要一个局部内存中的数组c用于存储每个线程负责的多个矩阵元；k的循环范围。注意将主机代码中的线程块配置改为：

```
dim3 blockSize = dim3(BLOCKDIM_X, BLOCKDIM_Y, 1);
```

编译运行如下（more_elements.cc，Nsub和Nsub_都为64，BLOCKDIM_X和BLOCKDIM_Y都为32）：

```
hipcc more_elements.cc -o more_elements -I$ROCM_PATH/rocrand/include -L$ROCM_PATH/lib
-lhipblas -lhiprand
./more_elements 4096
Generating Matrices took    5.634 ms.
Matrix multiplication (hipblasDgemm) took   29.711 ms.
Matrix multiplication (kernel) took   72.035 ms.
Max error: 0.0000e+00
./more_elements 16384
Generating Matrices took   11.246 ms.
Matrix multiplication (hipblasDgemm) took 1734.654 ms.
Matrix multiplication (kernel) took 4598.284 ms.
Max error: 4.7294e-11
```

调整一下这些规模，当Nsub为128，Nsub_为16，BLOCKDIM_X和BLOCKDIM_Y都为16时：

```
./more_elements 4096
Generating Matrices took    5.592 ms.
Matrix multiplication (hipblasDgemm) took   28.318 ms.
Matrix multiplication (kernel) took   23.790 ms.
Max error: 0.0000e+00
./more_elements 16384
Generating Matrices took   11.211 ms.
Matrix multiplication (hipblasDgemm) took 1734.478 ms.
Matrix multiplication (kernel) took 1515.077 ms.
Max error: 5.1841e-11
```

> 你可能会觉得我们编写的核函数比hipBLAS库的性能更好了。只能说在目前版本（DTK-22.10）中，针对本节中的简单情况（又未做边界检查），是这样的。

你可能想，我们继续增加Nsub岂不更好，Nsub等于256，Nsub_等于16对于共享内存容量来说是允许的，但这样修改之后（线程块保持16×16）运行，会出现这样的结果：

```
Generating Matrices took    5.635 ms.
Matrix multiplication (hipblasDgemm) took   29.254 ms.
Matrix multiplication (kernel) took 2057.917 ms.
```

```
Max error: 0.0000e+00
```

这里性能骤降的原因在于我们碰到了又一条限制，即寄存器的数量是有限的，在3.2.2节中提到过，每个SIMD单元有64KB的向量寄存器配额，一个计算单元总计256KB，每个寄存器是32位的（4B），即所有线程总共可以使用64K个向量寄存器，对于大小为1024的线程块，这意味着每个线程最多只能使用64个向量寄存器；对于我们这里大小为256的线程块（也包括更小的线程块），每个线程最多只能使用256个向量寄存器。但注意到此时局部内存中的数组c的长度是256了（Nx和Ny都是16），考虑到数组元素是双精度浮点数来说，仅这个数组就需要512个寄存器，这是不允许的，编译器会将这个数组分配到片外内存中去，结果函数multiply_submatrices变成了反复对全局内存而非局部内存做累加，如此性能下降就是意料之中的了。

3.5.5 局部内存中的数组

本节我们详细分析一下上节中的程序more_elements.cc，取性能最优的配置（Nsub为128，Nsub_为16，BLOCKDIM_X和BLOCKDIM_Y都为16）。

首先，你可能会对函数read_matrix和multiply_submatrices中循环的写法有疑问，按照之前章节中的写法，这两个函数中的循环应该是这样的：

```
template <int nrows, int ncols>
__device__
void read_matrix(double* M, int N, int i, int j,
                 double* M_ij, int LD)
{
  int row, col, offset, Row, Col, Offset;
  for (col = threadIdx.y; col < ncols; col += blockDim.y) {
  for (row = threadIdx.x; row < nrows; row += blockDim.x) {
      offset = row + col * LD;
      Row = i * nrows + row;
      Col = j * ncols + col;
      Offset = Row + Col * N;
      M_ij[offset] = M[Offset];
    } // row
  } // col
}
```

以及这样的：

```
__device__
void multiply_submatrices(double* A_sub, double* B_sub,
                          double* c)
{
  for (int k = 0; k < Nsub_; k++) {
    for (int j = threadIdx.y; j < Nsub; j+= blockDim.y) {
    for (int i = threadIdx.x; i < Nsub; i+= blockDim.x) {
      int I = i / blockDim.x;
      int J = j / blockDim.y;
      c[I + J * Nx] += A_sub[i + k * Nsub]
```

```
                *B_sub[k + j * Nsub_];
    } // i
    } // j
  } // k
}
```

我们按上面的写法来修改read_matrix并编译运行：

```
./more_elements 4096
Generating Matrices took    5.624 ms.
Matrix multiplication (hipblasDgemm) took   28.977 ms.
Matrix multiplication (kernel) took   35.649 ms.
Max error: 0.0000e+00
```

结果没问题，但性能下降了。再按上面的写法修改multiply_submatrices，运行结果如下：

```
./more_elements 4096
Generating Matrices took    5.680 ms.
Matrix multiplication (hipblasDgemm) took   28.496 ms.
Matrix multiplication (kernel) took 2097.088 ms.
Max error: 0.0000e+00
```

这次导致了很严重的性能下降，原因是什么呢?

答案是编译器不知道该如何为局部内存中的数组c的元素分配寄存器，所以只能将它放到片外了。仔细思考一下，因为我们是使用C语言编程，数组实际上是一块连续的内存，通过指针来访问数组中的元素，但寄存器是不同于其他内存的，不存在“寄存器的地址”这个概念，如果你熟悉X86架构的CPU，你可能知道每个寄存器都有特殊的名称；对于DCU的向量寄存器，每个都有特定的编号，但这种编号只可以理解为代号，不能理解为索引，不存在某个寄存器保存着其他寄存器的编号这种概念。

从这个角度来讲，本质上是不可以把数组放到寄存器上的。上一节中的写法之所以能让编译器把数组元素分配到一个个寄存器上，是因为编译器在编译时就能解析语句的索引，知道c[I + J * Nx]具体是c的哪一个元素。更通俗的讲，编译器只能将“假”的数组放到寄存器上，数组中每一个元素就是一个的独立变量，不具备根据索引访问元素的功能。

具体来说，按上节中的写法，编译器在编译时可以做循环展开，可以将I从0到7，J从0到7直接代入，等价于我们写类似这样的代码（将其他的宏也替换掉）：

```
  // I = 0, J = 0
  int i = threadIdx.x;
  int j = threadIdx.y;
  c[0] += A_sub[i + k * 128] * B_sub[k + j * 16];
  // I = 1, J = 0
  int i = threadIdx.x + 16;
  int j = threadIdx.y;
  c[1] += A_sub[i + k * 128] * B_sub[k + j * 16];
  // I = 2, J = 0
  int i = threadIdx.x + 32;
  int j = threadIdx.y;
  c[2] += A_sub[i + k * 128] * B_sub[k + j * 16];
// ...
```

此时，我们用三个变量c0、c1、c2替换上面的c[0]、c[1]和c[2]是没有任何问题的。编译器可以按给c0、c1、c2分配寄存器的方式，给c[0]、c[1]、c[2]分配寄存器。但如果按本节的写法，虽然逻辑上看没有任何问题，可是在编译时，编译器是无法做循环展开的。i和j的初始值分别是threadIdx.x和threadIdx.y，这是在运行时才能确定的，而且对每个线程各不相同；i和j在编译时无法确定，I和J是多少就更不知道了。对于

```
c[I + J * 8] += A_sub[i + k * 128] * B_sub[k + j * 16];
```

这样的代码，应该用c?来替换掉c[I + J *
8]，这在编译时是不知道的，因此数组c的元素没法分配到寄存器上。

使用局部内存中的数组是有好处的，可以让我们的代码简洁而灵活，设想一下，对于我们目前的配置，每个线程需要计算64个矩阵元，如果一句句的写：

```
  c0  += ...;
  c1  += ...;
  c2  += ...;
  ...
c63 += ...;
```

代码会非常繁杂，而且随着Nx和Ny的改变，变量的数目还会发生变化。但按照上节的写法，使用局部内存中的数组加上循环展开，简单的几行代码就可以达到效果。

最后我们顺带提一下循环展开，在hipcc默认-O3的优化级别下，大部分情况下编译器会自动将可以展开的循环进行适当的展开，如果需要手动指定，则在循环之上添加编译器指导语句#pragma
unroll，或用#pragma
nounroll要求编译器不做展开。循环展开除了能让我们简洁的使用局部内存的数组之外，还有很多优点，比如说不必反复判断循环条件，根据条件进行跳转；展开后的代码也更有利于编译器进行指令级的优化。我们在本书中不想涉及汇编层次的机器级代码，因此对这部分也不做更详细的介绍了，值得一提的是循环展开对算法上有一定要求，比如对于本节中的矩阵乘法问题，由于算法的仔细设计，循环次数在编译时就完全给定了；对于一些一般性的算法，循环指标的初始化、循环次数等在运行时才能确定，就难以利用循环展开进行优化了。

3.5.6 共享内存的bank冲突

在本节的最后部分，我们来讨论共享内存的访存问题。在3.2.2节，我们提到过共享内存在硬件上对应每个计算单元上容量为64KB的局域数据共享存储（方便起见，下文直接用共享内存来称呼它），它有32个bank，每个bank有512个32位的条目。每个bank在一个时钟周期内可以做一次32位的读或写，32个bank可以同时进行，因此共享内存有很高的带宽，此外共享内存上还有专门处理32位整型原子加的硬件单元。

DCU的一个线程束有64个线程，因此如果线程束访问共享内存中连续的64个4字节数据，则每半个线程束中的32个线程刚好与共享内存中的32个bank一一对应，在两个时钟周期就可以完成访存请求，这种情况下对共享内存的访存是最高效的。但如果64个线程都访问一个bank中的不同条目时，则是最低效的情况，这种情况下会产生严重的bank冲突，访存只能串行进行，需要64个时钟周期才能完成。因此在使用共享内存时，需要注意数据在共享内存中的布局和线程的访存方式，避免多个线程在一个时钟周期内访问同一bank中的不同条目（多个线程访问同一bank中的同一条目不会带来bank冲突，因为共享内存支持广播方式的访存），这样才能充分利用共享内存的带宽。

下面仍以我们的函数multiply_submatrices为例，分析一下子矩阵大小以及线程块大小对于共享内存访存效率的影响：

```
__device__
void multiply_submatrices(double* A_sub, double* B_sub,
                  double* c)
{
  for (int k = 0; k < Nsub_; k++) {
    for (int J = 0; J < Ny; J ++) {
      for (int I = 0; I < Nx; I ++) {
        // 每个线程计算Nx * Ny个c
        int i = threadIdx.x + I * BLOCKDIM_X;
        int j = threadIdx.y + J * BLOCKDIM_Y;
        c[I + J * Nx] += A_sub[i + k * Nsub]
                  *B_sub[k + j * Nsub_];
      } // I
    } // J
  } // k
}
```

子矩阵A_sub和B_sub是共享内存中的两个数组，由于是双精度浮点数（64位），因此每一个元素占用连续的两个bank。对于$16×16$的线程块，刚好在一个周期内threadIdx.x从0到15的16个线程可以从32个bank中读取A_sub的16个元素，不存在bank冲突，最大化的利用了共享内存的带宽；对于B_sub，也是16个线程访问同一个元素，属于广播方式的访存，同样不存在bank冲突。

再分析一种情况，使用$8×8$的线程块，Nsub、Nsub_也都等于8时，这与我们使用共享内存优化的最初版本的shared.cc中Nsub等于8时的程序基本一致。可以修改Nsub、Nsub_、BLOCKDIM_X和BLOCKDIM_Y后重新编译，测试一下这种情况下的性能：

```
./more_elements 4096
Generating Matrices took    5.654 ms.
Matrix multiplication (hipblasDgemm) took   28.996 ms.
Matrix multiplication (kernel) took  428.407 ms.
Max error: 0.0000e+00
```

在这种情况下，对于读取A_sub的8个元素来说也不存在bank冲突，因为这8个元素保存在连续的16个bank中，同行不同列（threadIdx.x相同、threadIdx.y不同）的线程通过广播方式读取同一元素；对于B_sub，同一列（threadIdx.y相同）的8个线程读取相同的元素，不存在bank冲突，但对于不同列读取的元素则存在bank冲突。考虑threadIdx.y等于0到3的四列32个线程，分别读取B_sub[k]、B_sub[k+8]、B_sub[k+16]和B_sub[k+24]四个元素，因为每16个元素就会占用全部的32个bank，所以B_sub[k]与B_sub[k+16]是同一bank中的不同元素，B_sub[k+8]和B_sub[k+24]也是同一bank中的不同元素。如何解决这种bank冲突呢?

一种简单的处理方式是将B_sub的空间多申请一行，即为B_sub申请9行8列的空间，但只使用8行8列：

```
__shared__ double B_sub[(Nsub_ + 1) * Nsub];
```

修改read_matrix的参数LD：

```
// 将子矩阵B_kj从全局内存读到共享内存
read_matrix<Nsub_, Nsub>(B, N, k, j, B_sub, Nsub_ + 1);
```

并修改multiply_submatrices中对于B_sub的引用：

```
c[I + J * Nx] += A_sub[i + k * Nsub]
            *B_sub[k + j * (Nsub_ + 1)];
```

分析一下，此时threadIdx.y等于0到3的四列32个线程，分别读取B_sub[k]、B_sub[k+9]、B_sub[k+18]和B_sub[k+27]四个元素，均在不同的bank中，就不会冲突了。编译运行，结果如下：

```
./more_elements 4096
Generating Matrices took    5.628 ms.
Matrix multiplication (hipblasDgemm) took   28.953 ms.
Matrix multiplication (kernel) took  355.662 ms.
```

> 练习：分析其他线程块规模与子矩阵规模下的共享内存bank冲突情况。

最后还要补充一点，一个线程在一个周期内最多产生两个4字节（64位）的访存请求，如果访存请求不满4字节（比如只读取一个半精度浮点数，FP16），仍然会占用一个bank的32位带宽；如果是128位的访存请求，比如读取两个双精度浮点数、双精度复数等，也只能用到共享内存一半的带宽。举例来说，假设线程ID为0的线程需要读取128位的数据在0~3号bank中，线程ID为1的线程需要的数据在4~7号bank中，这些访存请求需要拆到两个时钟周期，第一个周期线程ID为0的线程读取到0~1号bank的数据，线程ID为1的线程读取到4~5号bank的数据，此时2~3号、6~7号bank没有访存请求，因此有一半的带宽没有用上，第二个周期同理，只有2~3号、6~7号bank在处理访存请求，0~1号、4~5号bank没有用上。

3.5.7 小节

在本节中，我们以双精度矩阵乘法为例进一步介绍了DCU编程与优化方法。我们首先使用hipRAND库随机生成了两个矩阵，利用hipBLAS库计算了矩阵乘，以此作为在DCU程序中使用库的简单范例，并对DCU在处理这些问题时的性能有了初步的印象。之后我们自己编写了矩阵乘法的程序，通过分析，发现矩阵乘法中存在大量的数据复用。抓住这一点，我们尽可能地利用共享内存进行优化，在维持代码相对简洁可读的情况下，达到了不错的性能。

在优化的过程中，我们也看到共享内存的大小，共享内存的访存方式，局部内存的有效利用，还有程序的编写方法对编译器的影响都会对程序的性能产生影响，根据算法的不同，这些因素的影响或多或少，有时还会彼此关联、彼此制约，导致问题很复杂。本节也仅仅是抛砖引玉，希望读者参考本节的优化案例，在编程解决自己的问题时，根据算法特点，首先抓住少量影响性能的主要因素进行优化，再处理其他因素。

> 思考：在本节的程序中，有哪些硬件资源的使用情况有关联。
> 举例来说：当核函数使用64KB的共享内存时，每个计算单元最多并发多少个线程束。

补充一点，本节中申请共享内存的大小在编译时就确定了（Nsub、Nsub_都通过宏定义替换为具体的数），这种方式称为静态共享内存。有时共享内存的大小可能难以在编译时确定，此时还可以动态申请共享内存，在调用核函数时通过三个尖括号中的第三个参数来指定，例如：

```
int shmemSize = 2 * Nsub * Nsub_ * sizeof(double);
matrix_multiply<<<numBlocks, blockSize, shmemSize>>>
          (N, Nblk, dA, dB, dC);
```

此时Nsub和Nsub_可以是变量。但在核函数中使用共享内存会麻烦一些：

```
  // 获取动态共享内存的指针
  extern __shared__ float shmem[];
  // 声明A_sub,B_sub用于保存A,B的子矩阵
  double* A_sub = (double*)shmem;
double* B_sub = (double*)&shmem[2 * Nsub * Nsub_];
```

需要通过指针手动实现。

第 4章 使用OpenMP/OpenACC编写DCU程序

4.1 OpenMP与OpenACC简介

OpenMP是用于多处理器并行系统程序设计的编程模型，程序员通过在源代码中加入编译制导指令#pragma行来指明程序的并发属性。当选择忽略这些#pragma，或者编译器不支持OpenMP，程序又可退化为串行程序，代码仍然可以正常、正确的编译执行。

OpenMP本身不是一种独立的并行语言，而是为多处理器上编写并行程序而设计的、指导多线程并行的编译制导指令和应用程序编程接口（API）。OpenMP
4.0规范加入了对异构计算的支持。即使用host（CPU端）指导device（DCU等加速卡）的运行，host将计算密集型区域放到device上执行。需要控制device的内存分配、数据传输、代码声明、循环分摊、内存释放等操作。如下图，当需要使用OpenMP进行卸载时，host会将需要处理的数据映射到device上，开启一个device主线程，使用分裂指令可以开启device上的多个线程，共同完成一个计算任务，完成任务后，数据会映射到host端。

[图片]

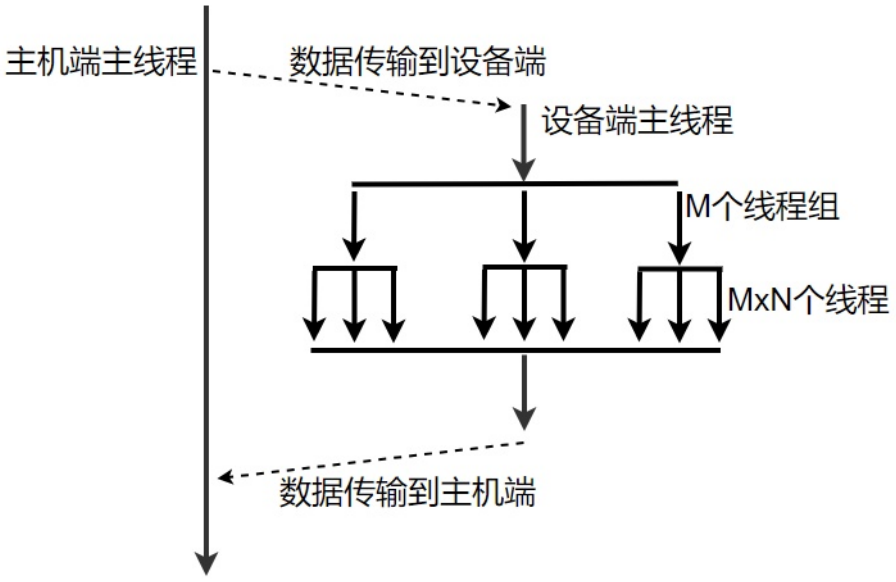


图1.1 OpenMP线程分配

OpenACC是一种基于导语的高级异构编程模型。OpenACC具有高性能、可编程性和跨平台可移植性等特点。其对刚接触异构并行计算的人十分友好，只需要在程序中加入指定导语，编译器就可以处理大部分

的翻译的复杂细节，将程序计算部分卸载到异构加速器中并行计算。若无异构加速器或者编译器不支持OpenACC,则也可以正确的编译和运行。OpenACC的发布降低了对异构加速卡使用的编程门槛。

OpenMP和OpenACC既有相似性，也有规则上的区别。

1. OpenMP和OpenACC的使用方法类似，都可以在代码中加入#pragma omp或#pragma acc来指示编译器开启并发功能，规则中都包含导语+子句的使用模式，且导语和子句中存在大量实现类似的功能。
2. OpenMP和OpenACC的支持语言相同，都支持C/C++/Fortran语言，且其支持语言的语法也是相似的。
3. OpenMP可以同时支持在主机端和设备端加入并发属性，在主机端并发时，能够分配出多个线程同时执行代码，在设备端并发时，可以根据指令的配置，分配出多个线程组和线程执行。OpenACC主要支持在设备端进行并发。
4. OpenMP和OpenACC虽然其导语与子句存在很多类似的功能，但是也有很多实现逻辑和细节不同的功能。
5. 由于开发进度不同，不同的编译器对于OpenMP和OpenACC的支持不同，当前DTK对于C/C++语言OpenMP的支持高于OpenACC，对于Fortran语言，OpenMP和OpenACC均有较好的支持。支持C/C++语言OpenMP5.1版本，支持Fortran语言OpenMP4.5版本，支持Fortran语言OpenACC3.0版本。

表5.1 OpenMP与OpenACC对比

	OpenMP	OpenACC
使用平台	主要主机端少量异构	主要异构
使用方法	#pragma omp!$omp	#pragma acc!$acc
使用标准	1.0-5.2版本	1.0-3.3版本
功能实现	支持大部分C/C++指令支 Fortran主流指令	支持少量C/C++指令支 Fortran主流指令

4.2 OpenMP的设备端并行

本章将详细介绍OpenMP在DCU上的编程方法，以及如何高效的使用OpenMP进行异构程序编程。

4.2.1 编写第一个设备端OpenMP程序

下面是一段用C语言实现的数组相加的程序，C=A+B。这个足够简单的程序能让你掌握OpenMP在DCU进行异构编程以完成任务并行的基本方法以及了解OpenMP
DCU编程和CPU编程以及HIP之间的重要差别，还有编译运行方法。

首先我们来看原始CPU端串行代码（ori_code.c）。

```
void vec_mult(int N)
{
    int i;
    float A[N], B[N], C[N];
    init(A,B,C,N);
    for (i=0; i<N; i++)
    {
      C[i] = A[i] + B[i];
    }
}
```

程序使用了一个for循环，将数组的每个元素都遍历一遍，相加，赋值。下面一段代码则是将其改造成OpenMP的异构计算代码（omp_code1.c）：

```
void vec_mult(int N)
{
    int i;
    float A[N], B[N], C[N];
    init(A,B,C,N);
    omp_set_num_threads(N);
    #pragma omp target map(tofrom:A[0:N],B[0:N],C[0:N])
    {
      #pragma omp parallel for
      for (i=0; i<N; i++)
      {
        C[i] = A[i] + B[i];
      }
    }
}
```

使用如下方法编译OpenMP异构代码：

```
Fortran代码：
flang -fopenmp --offload-arch=gfx936 test.f90 -O3
C/C++代码：
clang++ -fopenmp --offload-arch=gfx936 test.cpp -O3
```

下面我们来简单分析一下代码，看看我们使用OpenMP做了什么。

1. 分配资源

在初始化数组后，程序使用omp_set_num_threads(N)给kernel分配了N个线程资源来执行并行任务。

2.异构并行制导语句

使用OpenMP target结构制导语句#pragma omp
target和一对大括号“{}”来确定并行区域，并形成kernel。程序执行到该语句，数组相加的计算将由主机(CPU)转移到指定设备(DCU)中。一个target结构的程序段相当于HIP中的一个kernel核函数。

在target结构中使用#pragma omp parallel
for语句来控制线程的分配和任务，使任务并行计算，该部分的意义将会在接下来的章节一一讲述。

3.数据传递

在异构并行计算中很重要的一部分就是数据的传递，HIP使用hipMemcpy来完成这一操作，并使用最后一个参数hipMemcpyHostToDevice或者hipMemcpyDeviceToHost来控制数据的传递方向。

相比而言，OpenMP的操作方式就比较多样化，本例子使用target结构中的map子句来进行数据的传输，tofrom代表将列表中的A,B,C三个数组在设备端设置同名的副本参数，并在kernel刚开始时将参数传递至设备端，在kernel结束后将设备端的参数值传输到主机端。其余的数据传递操作将在第三章详细讲述。

4.2.2 OpenMP的核函数

在一个异构计算基本算例中，包含主机端和设备端处理，主机端处理包括内存申请、变量初始化、数据传输、线程配置等过程，设备端处理包括负载均衡、核函数计算等过程。OpenMP的优势是将中间大量繁复、变化较少的过程集成到编译器中，通过编译器提供的OpenMP指令和子句来调用这些过程，简化了程序处理。本节介绍OpenMP中核函数创建、使用方法、解决数据冲突和性能优化方面的内容。

4.2.2.1 核函数的创建和使用—target指令结构

在HIP编程中被global标识的函数又被称为核函数或“kernel”，核函数代表了设备端的入口，一旦进入核函数，函数的执行和控制就交给了设备端。而在OpenMP中使用target结构来标识核函数，与HIP类似，OpenMP异构计算将适合并行的计算量大，且计算元素互不干涉的do/for循环改造成kernel来使用异构加速卡并行计算，如下代码所示，target指令包围的代码块就成了一个kernel，在!$omp teams distribute parallel
do指令下的do循环被已分配的线程并行执行。teams、distribute、parallel等相关指令会在5.2.2.2小节详细解释。

```
!!!omp_target.f90
subroutine sample_omp(p, v1, v2, N)
  integer :: i,N,j
  real    :: p(N), v1(N), v2(N)
  call init(v1, v2, p, N)
  !$omp target map(to: v1,v2) map(tofrom: p)
    !$omp teams distribute parallel do
    do i=1,N
      v1(i) = v1(i) + 1.0
      v2(i) = v2(i) + 2.0
      p(i) = v1(i) * v2(i) + p(i)
    end do
  !$omp end target
end subroutine
```

target结构由指令#pragma omp target（Fortran语言为!$omp
target）和一个target计算区域组成，当程序执行到该语句，相关计算将由主机转移到指定设备中，等到设备端完成计算后，再将相应数据从设备端内存映射到主机端内存中。

target结构除了主语句之外，还配备有多个子句，这些子句是用来额外显式设定数据的传输规则，线程资源的分配以及同步异步等操作。

target结构的语法如下所示：

```
C/C++:
#pragma omp target [clause[ [,] clause] ... ]
//核函数
Fortran:
!$omp target [clause[ [,] clause] ... ]
!核函数
!$omp end target
```

4.2.2.2 线程执行模型

在HIP编程中，将线程块分成一维、二维或三维，但是在OpenMP编程中，为了简化计算，线程块都为一维的形式，这些逻辑为内部实现逻辑。如下图所示，每个线程组和线程都按照一维逻辑排列。

[图片]

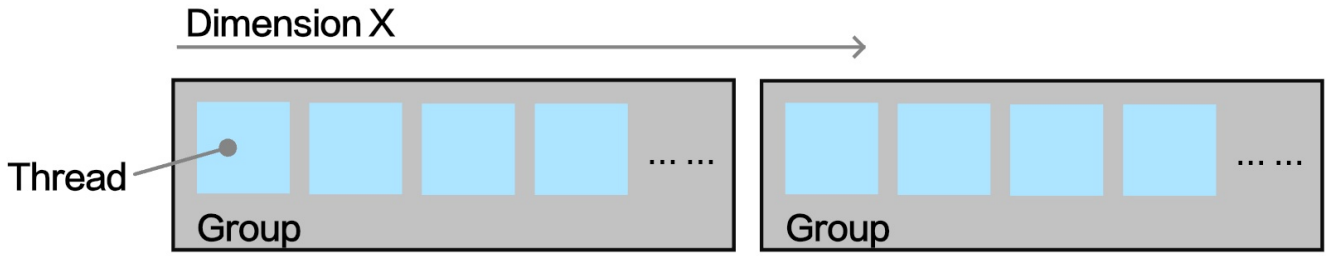


图1.2 线程实现逻辑

在OpenMP内部执行逻辑中，会将一个线程块划分成一定数量的线程束，每个线程束有64个线程，在硬件调度时，会按照64倍数的线程来调度。在执行时，OpenMP会自动将线程块中的线程设置为64的倍数，不需要使用者来设置线程束数量。

在OpenMP编程中，使用teams指令和parallel指令能够在执行时启动大量线程来执行核函数，这些线程在程序中被分为多个线程组，每个线程组中存在多个线程。可以使用distribute指令和for指令来将一个for循环中的任务分发到多个线程中，即将多次循环的计算工作量进行任务划分，让每个线程各自负责其中的一部分循环工作。

注意：以下1~5小节中的例子仅为说明指令的工作原理和使用方法，若要进行极致的性能优化，需使用第6小节中的方法。

1. 多线程指令parallel、for/ do
   在OpenMP的相邻的fork、join操作之间称为一个并行域，并行域是一块能被多个线程执行的代码。指令parallel的作用就是构造一个并行域，它也可以和其他指令如for、sections等配合使用形成复合指令。

指令for/do指定紧随的循环语句由多个线程并行执行，用来将一个for循环任务分配到多个线程，各个线程各自分担其中一部分工作。for指令一般需要使用在parallel指令的并行域中，若不在并行域中使用for/do指令，程序将会以串行的方式执行。

在使用指令parallel时，必须注意：一个并行区域必须是一个结构块，不能跨越多个程序或者代码文件；不能使用进入或者离开一个并行区域的分支。可以使用omp_get_num_threads函数获得程序在执行过程中正在运行的线程的数量，可以使用函数omp_get_thread_num获取当前正在运行的线程号。

在一般情况下，使用默认配置方式，即不需要手动配置线程数量，执行效率最高。

- 若单独使用for/do语句，其for/do循环都会在一个线程中执行，for/do指令要和parallel指令结合起来使用才有效果，即for/do出现在并行域中才能有多个线程来分担任务。
- 如果没有使用collapse子句，for/do指令只能作用在最邻近的一层循环。
- 当循环次数较小时，采用并行计算，执行OpenMP runtime程序进行调度的时间可能占比较高，无法取得很好的加速效果。
- 在对循环执行并行执行之前，需要保证数据不存在循环依赖或者数据冲突，若存在数据冲突，尝试使用下一节的方法进行解决。

来看一个例子：

```
subroutine circulation(N,ut, vt)
   integer :: N
   real, intent(INOUT), dimension(N,N,N)::ut, vt
  !$omp target
    !$omp parallel do     ! 对第一层i循环并行
    do i = 1,N
      do j = 1,N
        do k = 1,N
          ut(i,j,k)=ut(i,j,k) *1.1
          vt(i,j,k)=vt(i,j,k) *1.1
        enddo
      enddo
    enddo
  !$omp end target
end subroutine
```

从circulation函数中能够看出，ut、vt两个三维数组进行3层do循环的乘法计算，使用target指令表示该核函数需要卸载到DCU上运行，使用parallel do指令表示在DCU上产生多个线程，同时将第一个do循环中的任务拆分到每个生成的线程中，每个线程执行循环的次数肯定少于单线程运行循环的次数，以此来提高性能。产生的线程数根据N的大小，由OpenMP程序自行配置合适的参数，最大能够产生256个线程同时并行。

1. 多线程组指令teams、distribute

在一个核函数中使用parallel for指令只会生成一个线程组，分配的线程数量有限，最多分配256个线程。这时可以使用teams指令在分配线程之前来生成多个线程组。

在teams指令中，可以使用num_teams子句控制生成的线程组数量的最大值，也可以使用thread_limit子句控制每个线程组中线程数量的最大值。在单独使用parallel指令时，不会生成线程组，可以使用thread_limit子句控制线程数量的最大值。还可以使用OMP_NUM_TEAMS环境变量直接控制所有的teams结构的线程组数量。当前OpenMP若由程序自动生成线程组，最多生成256个线程组，若使用OMP_NUM_TEAMS环境变量配置线程组数量，最多能够扩展到生成1024个线程组。

例如，使用如下指令：

```
#pragma omp teams num_teams(2) thread_limit(128)
```

表示指定生成2个线程组，每个线程组中最大有128个线程，OpenMP会根据接下来代码的循环次数来自动计算出应该使用的线程组和线程数。

distribute指令与for/do指令功能类似，作用是将紧随指令distribute的循环以隐式任务的形式分配给指令teams生成的线程组中。使用指令distribute需要注意如下事项：

- 如果不使用collapse子句，distribute指令只能够作用于最邻近的一层循环。
- 指令distribute需要嵌套在teams指令区域内使用。

根据上面的circulation例子，若只使用parallel
do指令，可以最大生成256个线程并行，当N比较大时，生成的线程数已经不能满足程序需要。这时需要使用teams distribute指令，如下所示：

```
subroutine circulation(N,ut, vt)
   integer :: N
   real, intent(INOUT), dimension(N,N,N)::ut, vt
  !$omp target
    !$omp teams distribute ！对第一层i循环并行
    do i = 1,N
    !$omp parallel do   ！对第二层j循环并行
      do j = 1,N
        do k = 1,N
          ut(i,j,k)=ut(i,j,k) *1.1
          vt(i,j,k)=vt(i,j,k) *1.1
        enddo
      enddo
    enddo
  !$omp end target
end subroutine
```

从上面的circulation函数能够看到，teams
distribute指令作用于第一层循环，目的是生成一定数量的线程组，parallel
do指令作用于第二层循环，目的是在每个线程组中生成一定数量的线程。这个例子中有两层循环作用在了并行执行中，执行速度更快。当然这种使用方法有一定的弊端，例如当第一层循环次数极小，且第二层循环次数极大时，线程组数量较少，并不能产生较好的加速效果，为了解决这个问题，需要使用指令组合和collapse子句来配置多层循环的执行。

1. 复合指令
OpenMP中，可以在每个omp行中使用一个指令，也可以将多个指令组合起来，放到一个omp行中使用。例如如下单行命令结构:

```
C/C++:
#pragma omp target
{
#pragma omp teams
{
#pragma omp distribute
for循环
```

```
  #pragma omp parallel
{
    #pragma omp for
    for循环
}
}
}
Fortran:
!$omp target
!$omp teams
!$omp distribute
do循环
    !$omp parallel
    !$omp do
    do循环
    !$omp end parallel
!$omp end teams
!$omp end target
```

可以写成如下复合指令的形式：

```
C/C++:
#pragma omp target teams distribute
for循环
  #pragma omp parallel for
  for循环
Fortran:
!$omp target teams distribute
do循环
  !$omp parallel do
  do循环
```

该结构的功能：target结构表示这一部分代码需要在DCU上执行，teams指令表示需要生成多个线程组，distribute指令表示根据第一层do循环中的循环数量拆分成多个隐式任务，分发到各线程组中；parallel指令表示在单个线程组中生成多个线程，for循环表示根据do循环中的循环数量拆分成多个隐式任务，分发到各线程中。如图，若第一层i有200次循环，且teams指令生成了4个线程组，那么每个线程组都会分配50次i循环，若第二层j有200次循环，且parallel指令生成了2个线程，那么每个线程都会分配100次j循环。

[图片]

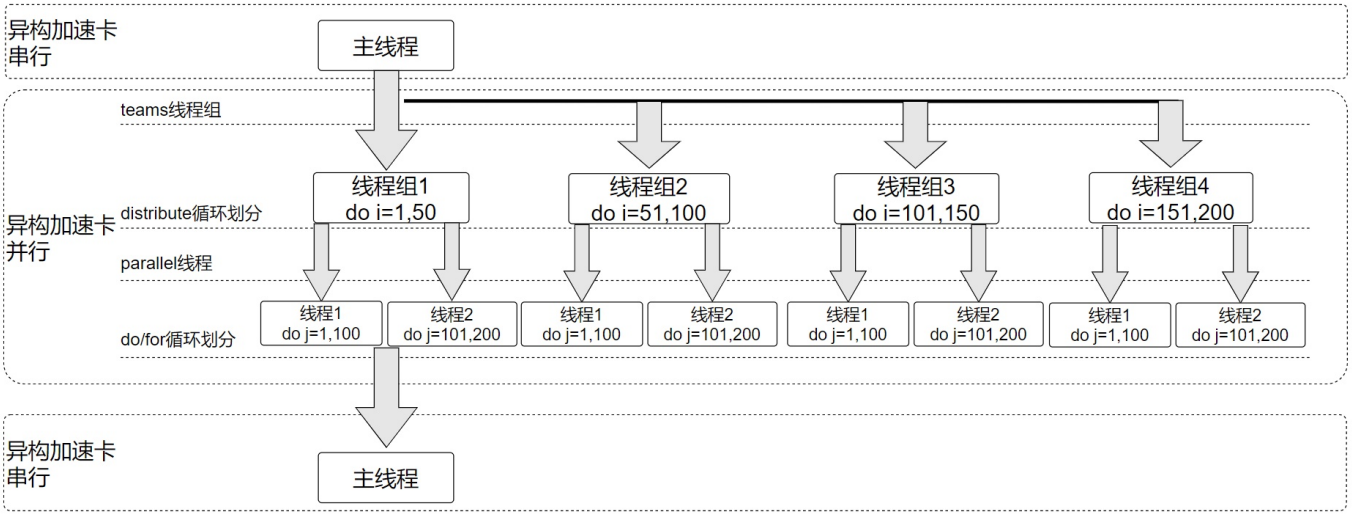


图1.3 teams distribute指令和 parralle指令分配线程

上面的指令分配方法可以由使用者手动控制拥有两层do循环的程序执行分配，如果循环只有一层，或者循环虽然有多层，但使用者不想手动分配线程组和线程数量，OpenMP还提供了一种指令组合形式，teams distribute指令可以与parallel
for指令同时使用，在日常优化程序时，更加推荐这一种形式，如下所示：

```
C/C++:
#pragma omp target
{
  #pragma omp teams distribute parallel for
  for(;;)
}
Fortran:
!$omp target
  !$omp teams distribute parallel do
    do
    enddo
!$omp end target
```

在此结构中，teams指令根据程序的循环次数创建了多个线程组，distribute指令将循环迭代的计算任务分配给teams区域中所有线程组的主线程；parallel指令在每个线程组中创建多个线程，for指令将循环迭代的任务再次分配给parallel区域中所有的线程。如图，若循环次数为4000，且teams指令生成了4个线程组，每个线程组都会分配到1000次循环；若parallel指令在每个线程组中生成了2个线程，每个线程都会分配到500次循环。

[图片]

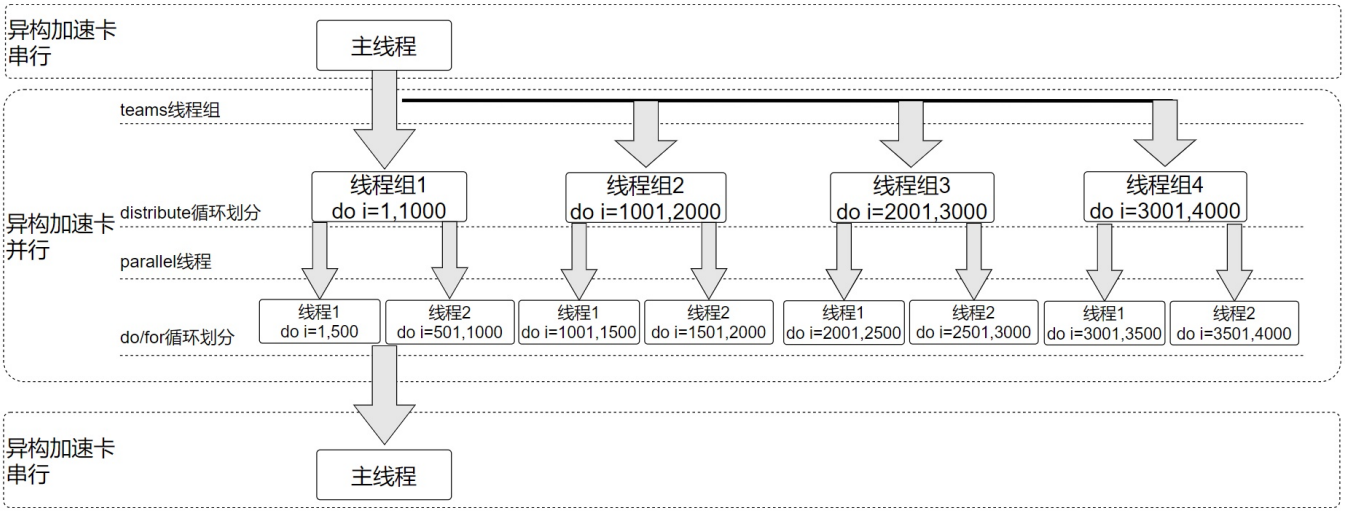


图1.4 teams distribute parralle复合指令分配线程

结合teams指令和distribute指令可以分配大量线程，因此5.2.1小节中omp_code1.c的程序可以优化为以下：

```
//omp_code2.c
void vec_mult(int N)
{
    int i;
    float A[N], B[N], C[N];
    init(A,B,C,N);
    omp_set_num_threads(N);
    #pragma omp target map(tofrom:A[0:N],B[0:N],C[0:N])
    {
      #pragma omp teams distribute parallel for
      for (i=0; i<N; i++)
      {
        C[i] = A[i] + B[i];
      }
    }
}
```

需要注意的是，复合指令omp teams distribute parallel
for同样只对相邻近的一层循环其作用，若想对连续的多层循环执行并行，需要使用collapse子句进行循环合并。

1. 多层循环合并，使用collapse子句。
子句collapse将一个多层循环进行合并后展开为一个更大的循环，从而增加将在线程组上进行划分调度的循环总数。当有多层for循环时，使用collapse子句能够将之后的size层for循环合并为一层。迭代次数为每层for循环迭代次数的乘积。使用collapse子句时，需要注意程序中不应有数据冲突，否则计算结果会出现错误。子句collapse的用法如下：

```
C/C++:
 #pragma omp teams distribute parallel for collapse(3)
  for (int i = 0; i < iep; i++)
    for (int j = 0; j < jep; j++)
```

```
      for (int k = 0; k < npz; k++)
        vt[i][j][k] = i + j * 4 + k * 16;
Fortran:
!$omp target
  !$omp teams distribute parallel do collapse(3)
   do i=1,iep
     do j=1,jep
       do k=1,npz
         vt(i,j,k) = i + j * 4 + k * 16;
       enddo
     enddo
   enddo
!$omp end target
```

上述程序具有如下特点：

- 子句collapse(3)对从最外层开始的i，j和k三层循环进行了合并，合并后的单循环长度为iep×jep×npz。
- 数组vt是共享变量，代码中利用了不同的线程对数组vt进行不同元素进行读写操作，不会产生数据冲突。

4.2.2.3 解决数据冲突的方法

在并行线程中共享数据会引起冲突，多个线程以不定的顺序访问同一数据，会导致不可预测的程序行为。下面介绍了几种方法，来解决在异构编程中出现的数据冲突的问题。

1. 使用规约来避免数据冲突。
规约reduction子句在实际的使用中不常用。通过指定一个操作符，在每个线程中都会创建该参数的private变量，执行完成并行区域后，将每个private变量通过操作符进行计算，得到最新的计算结果。使用方法如下：

```
reduction(operator: list)
```

2. 设置线程级临时变量，避免数据冲突。
我们可以通过一个数据冲突的例子来说明线程级变量的作用方式。

```
module test
contains
subroutine vec_mult(p, v1, v2, N)
  integer ::  i,N,j,k
  real:: v1(N), v2(N)
  real,intent(inout),dimension(1:N,1:N)::p
  call init(v1, v2, p, N)
  !$omp target map(tofrom: p) map(to:v1,v2)
    !$omp teams distribute parallel do collapse(2)
    do i=1,N
     do j=1,N
      v1(i) = v1(i) + 1.0     ! 一维变量写入
```

```
        v2(i) = v2(i) + 2.0
        p(i,j) = v1(i) * v2(i) + p(i,j)
       enddo
      end do
    !$omp end target
  print*,"p",p
  end subroutine
  subroutine init(v1, v2, p, N)
    integer,intent(in)::N
    real,intent(inout)::v1(N), v2(N)
    real,intent(inout),dimension(1:N,1:N)::p
     v1 = 2.
     v2 = 3.
     do i = 1,N
        do j = 1,N
           p(i,j) = i+j
        enddo
     enddo
  end subroutine
  end module test
  program main
  use test
    integer, parameter :: N=10
    real,allocatable, dimension(:,:) :: p
    real,allocatable, dimension(:)::v1, v2
    allocate( p(N,N))
    allocate( v1(N), v2(N) )
    call vec_mult(p, v1, v2, N)
  end program
```

上面的例子中出现了数据冲突，原因是使用了两层do循环，且使用了collapse子句提供进一步加速，在DCU上使用了100个线程，每个线程循环一次，而在循环内部，v1、v2变量为一维变量，且保存在全局内存中，多个线程同时执行会将计算结果互相覆盖，得出随机的计算结果。若想要解决这个问题，可以将v1、v2变量修改为标量临时变量。如下程序所示：

```
subroutine vec_mult(p, v1, v2, N)
  integer ::  i,N,j,k
  real :: l,m
  real:: v1(N), v2(N)
  real,intent(inout),dimension(1:N,1:N)::p
  call init(v1, v2, p, N)
  !$omp target map(tofrom: p) map(to:v1,v2)
    !$omp teams distribute parallel do collapse(2)
    do i=1,N
     do j=1,N
      l = v1(i) + 1.0  ！修改为线程级临时变量
      m = v2(i) + 2.0
      p(i,j) = l * m + p(i,j)
```

```
      enddo
     end do
   !$omp end target
   print*,"p",p
end subroutine
```

编译器在编译时，检查到l、m为标量数据，所以将其保存到寄存器中。寄存器内存是线程私有的，不会受到其他线程的影响，所以得到的结果是正确的，不会产生数据冲突。

1. 提升临时变量的数据维度。
还是如上面一节中提到的例子，出现数据冲突的原因是临时变量为一维，各线程数据互相覆盖，这时可以通过提升临时变量的数据维度，避免数据冲突。如下程序所示：

```
subroutine vec_mult(p, v1, v2, N)
integer ::  i,N,j,k
real,dimension(1:N,1:N) :: tmp1,tmp2
real:: v1(N), v2(N)
real,intent(inout),dimension(1:N,1:N)::p
call init(v1, v2, p, N)
!$omp target map(tofrom: p) map(to:v1,v2) map(alloc:tmp1,tmp2)
  !$omp teams distribute parallel do collapse(2)
  do i=1,N
    do j=1,N
     tmp1(i,j) = v1(i) + 1.0    ! 提升写入的数据维度
     tmp2(i,j) = v2(i) + 2.0
     p(i,j) = tmp1(i,j) * tmp2(i,j) + p(i,j)
    enddo
  end do
!$omp end target
print*,"p",p
end subroutine
```

在程序中，tmp1和tmp2都位于全局内存中，由于数据维度提升到了二维，各线程数据不会互相覆盖，计算出的结果是正确的。与设置线程级临时变量的方法相比，这个方法的缺点是使用了全局内存，数据延迟较高，优点是保存的数据量较多，可以在接下来的循环结构中继续使用。

1. 使用私有数组。
私有数组的使用方法可以见OpenACC private数组章节。

4.2.3 OpenMP任务并发

4.2.3.1 任务并发模型

当前版本Clang支持OpenMP任务并发，但是Flang不支持。该功能与OpenACC的异步行为完全不同，需要注意。

在HIP编程中，核函数的执行和数据拷贝可以以工作队列的形式进行编队，每一个队列就是一个流，使用

者可以手动创建和删除一个流，也可以使用默认流，每个流可以通过设置优先级来调整占用的资源先后顺序。在异步场景下，多流运行程序能够明显提升程序的执行效率。

在OpenMP中，也存在流的概念，用户无法手动指定某个核函数在哪个流上执行，也不能手动创建与删除流。而且由于当前LLVM中OpenMP中核函数执行都是同步的，在CPU单线程的情况下无法进行多流同步执行，只能通过在CPU端分配的多线程上实现多流并发。

在通常情况下，OpenMP异构编程需要先将用到的数据传输到设备，执行核函数结束后需要将结果传输到主机。如要使用OpenMP中的多流进行并发执行加速，其核心思想是在CPU端生成多个线程，然后在CPU端的线程上使用target指令执行异构操作。这里面有静态并发和动态并发两种类型。静态并发可以使用parallel/for指令或者parallel/section指令将循环或者一部分代码块拆分成多个静态任务，完成并发操作。

动态并发可以通过task指令，动态的生成CPU端任务，在任务中可以进行并发操作。使用任务task指令生成的任务只能在CPU上执行，若想进行异构运算，则需要与target指令配合使用。将异构计算嵌入到任务中，相当于创建了多个执行异构计算的任务。

在主机端的主线程中分配了并行环境，创建了多个CPU线程，主线程上创建了三个任务，工作线程由于是空闲的，就会取出任务进行执行。由于任务为异构计算任务，需要将数据传输到设备端，计算完成后将结果传输回主机端。

4.2.3.2 静态并发

使用parallel指令和target指令完成嵌套可以实现静态并发。如下例子所示：

```
#pragma omp parallel for  // 配置CPU多线程
 for (int i = 0; i < 16; ++i) {
  int *p = (int *)omp_target_alloc(i * sizeof(int), 0);
  #pragma omp target teams distribute parallel for is_device_ptr(p)   // 在多个CPU线程上同时执行卸载
   for (int j = 0; j < i; ++j) {
    p[j] = i;
   }
   int buffer[i];
  #pragma omp target teams distribute parallel for is_device_ptr(p) map(from : buffer)
   for (int j = 0; j < i; ++j) {
    buffer[j] = p[j];
   }
   omp_target_free(p, 0);
  }
 }
```

在这个例子中，在主机端使用parallel
for指令对一个for循环在CPU端执行并行，生成了多个CPU线程，然后在每个CPU线程中都会执行target指令进行卸载。同时生成且执行多个核函数，每个核函数再次进行DCU上的并行化，对for循环中的计算分配到每个DCU线程中。

OpenMP中也可以使用parallel section指令生成静态并发，使用方式与parallel
for类似，这里不再赘述。

4.2.3.3 动态并发

OpenMP规范中，使用for、section指令可以被认为是分配了任务，但是这种任务是基于比较规则的循环结构进行的静态的显式任务划分。OpenMP中定义了task指令，用于处理以动态形式产生或者不规则的多任务并行。

并行程序会用一个线程按照程序代码的顺序生成任务，这些任务将放入到任务池中，由空闲的线程取出执行。如果不加限制，任务的默认执行顺序是随机的。使用者可以通过配置任务的依赖关系来自行定义任务的执行顺序。

[图片]

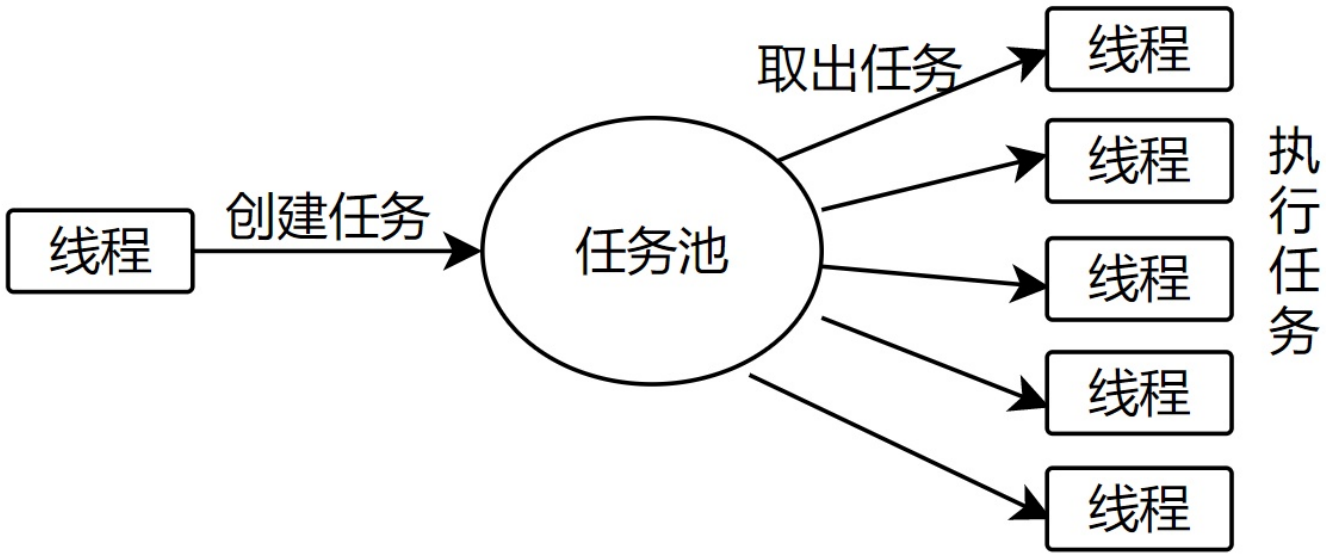


图1.6 并行环境中的任务划分

```
任务task指令的定义方法如下：
C/C++:
#pragma omp task[子句]
task内容
Fortran：
!$omp task [子句]
task内容
```

由上图可以看出，使用task指令生成任务需要处于并行环境中，由一个线程创建任务，且将创建的所有任务放入任务池中，同时若其他线程处于空闲状态，就会从任务池中取出任务执行。所以在一般情况下，需要task指令与parallel指令配合使用，使用single指令保证只有一个线程进行创建任务。创建出的任务在子句缺省的情况下，执行顺序是不确定的，指令parallel结尾处有一个路障，当所有线程都执行完成任务后才会继续向下执行。

注意，目前在Flang编译器环境下暂不支持task指令。

1. 使用for循环嵌套task是生成任务的一种方式。
   程序在使用parallel创建CPU多线程环境后，使用single指令保证只有一个线程执行区域中的代码，该线程使用for循环创建了大量任务，同时，其他空闲的线程同步执行创建的任务。主线程创建完成所有的任务后，执行后面的process(100)的任务。

```
#pragma omp parallel num_threads(4)
```

```
#pragma omp single
{
  int i;
  for (i = 0; i < N; i=i+a[i])
  {
    #pragma omp task
    process(a[i]);
  }
  process(100);
}
```

1. 使用sections和section指令嵌套task生成任务。
section指令创建任务时，仅保证在推出sections时任务已经生成，任务有可能是放在任务池中，还没有开始执行。
2. 使用任务循环结构。
任务循环结构taskloop允许对循环迭代进行分割并捆绑到多个任务中，这些任务由任务循环结构产生、调度执行，每个被创建的任务可以分配1个或多个迭代循环。子句grainsize表示可以手动分配给每个任务的逻辑循环迭代次数，需要大于子句表达式的值。

指令taskloop位于for循环之前，通过将循环划分为一个或多个任务来并行化循环。下面的例子使用了taskloop对1024次for循环进行任务划分，每个生成的任务循环次数不小于64：

```
#pragma omp taskloop grainsize(64)
for (int i = 0; i< 1024; i++)
func(i);
```

该代码表示的含义与下面使用taskgroup指令相同：

```
#pragma omp taskgroup
for (int j = 0; j < 16; j++)
  #pragma omp task
  for (int i = j * 64; i < j*64+64; i++)
func(i);
```

对于在并行区域中生成的所有任务，程序会保证这些任务在并行区域中遇到下一个路障前完成。任务的完成点是指在此点以前所有的任务都需要在此点执行完毕，可以使用指令taskwait与barrier保证这一点之前的任务在遇到该指令时等待其他任务完成，部分结构如parallel指令结束处的隐式路障也可以起到与barrier指令相同的作用。

并发实例。定义三个任务task1、task2、task3，每个任务都需要将数据传输到设备端进行计算，第三个任务依赖于前两个任务，前两个任务互不依赖。执行后能够发现，task1先执行，task2在task1未执行完成时开始执行，task3在task1和task2执行完成后才开始执行，task1和task2执行互不影响。

```
void vec_mult(int N)
{
int i,j;
  int tmp,tmp1;
  int task1[N],task2[N],task3[N];
  for(i=0;i<N;i++)
  {
    task1[i] = 0;
    task2[i] = 0;
```

```
    task3[i] = 0;
  }
  #pragma omp parallel //配置CPU多线程
  #pragma omp single  // 主线程上生成任务
  {
    #pragma omp task shared(task2) depend(out:task2)
    // 在每个任务中配置异构程序
#pragma omp target map(tofrom:task2[0:N])
    {
      for (i=0; i<N; i++) {
        task2[i] = 20;
        printf("task2 = %d\n",task2[i]);
      }
    }
    #pragma omp task shared(task1) depend(out:task1)
    #pragma omp target map(from:task1[0:N])
    {
      for (i=0; i<N; i++) {
        task1[i] = 10;
        printf("task1 = %d\n",task1[i]);
      }
    }
    #pragma omp task shared(task1,task2,task3) depend(in:task1,task2)
    #pragma omp target map(tofrom:task3[0:N],task2[0:N],task1[0:N])
    {
      for (i=0; i<N; i++)
      {
        task3[i] = task1[i] + task2[i];
        printf("task3 = %d\n",task3[i]);
      }
    }
    #pragma omp taskwait
  }
}
int main(void)
{
  int N=20;
  vec_mult(N);
  return 0;
}
```

使用parallel指令生成并行环境，且使用了single指令指定其中一个线程进行任务创建。程序中有三个任务，每个任务中使用的数组显式的设置为share属性。程序最后有一个taskwait指令，表示若要继续向下执行，必须将上面的任务执行完成。parallel指令最后结束处有一个隐式的barrier指令。

4.3 OpenMP数据传输

在OpenMP异构计算中，device端无法直接访问存储在host端的存储数据，因此我们需要将相关计算的参数进行数据管理。本章将会给大家介绍数据传输的几种方法，以及如何更合适，更有效率的进行数据传输。

4.3.1 数据传输子句

在梳理程序的数据传递时，我们首先要对每个kernel中的参数进行分析，理清楚哪些参数需要初始化（使用to/tofrom/firstprivate），哪些参数只需要在计算完成之后传回主机端(使用from)，哪些属于中间变量(使用alloc)，不同的参数使用合适的子句能使数据传输更加的合理，程序性能也会进一步被优化。

4.3.1.1 显式传输子句

1. 最常用子句–map子句
数据传输的子句主要为map子句。map子句可以设置数据在host和device上的移动方向，也能够控制device的数据持续时间。map语法格式为：

map [映射类型:变量列表]

映射类型有tofrom、to、from、alloc、release和delete。当kernel数据需要初始化，且计算之后需要传回host端时使用tofrom映射类型；当数据仅需要初始化，kernel结束后不需要传回host端时使用to映射类型；from映射类型适合kernel参数不需要初始化且计算结果需要传回host端；当参数仅为中间变量，使用alloc映射类型。

若没有显式指定，数组的默认映射类型是tofrom，指针的默认映射类型是具有tofrom类型的零长度的指针。标量变量的默认类型firstprivate变量。

[图片]

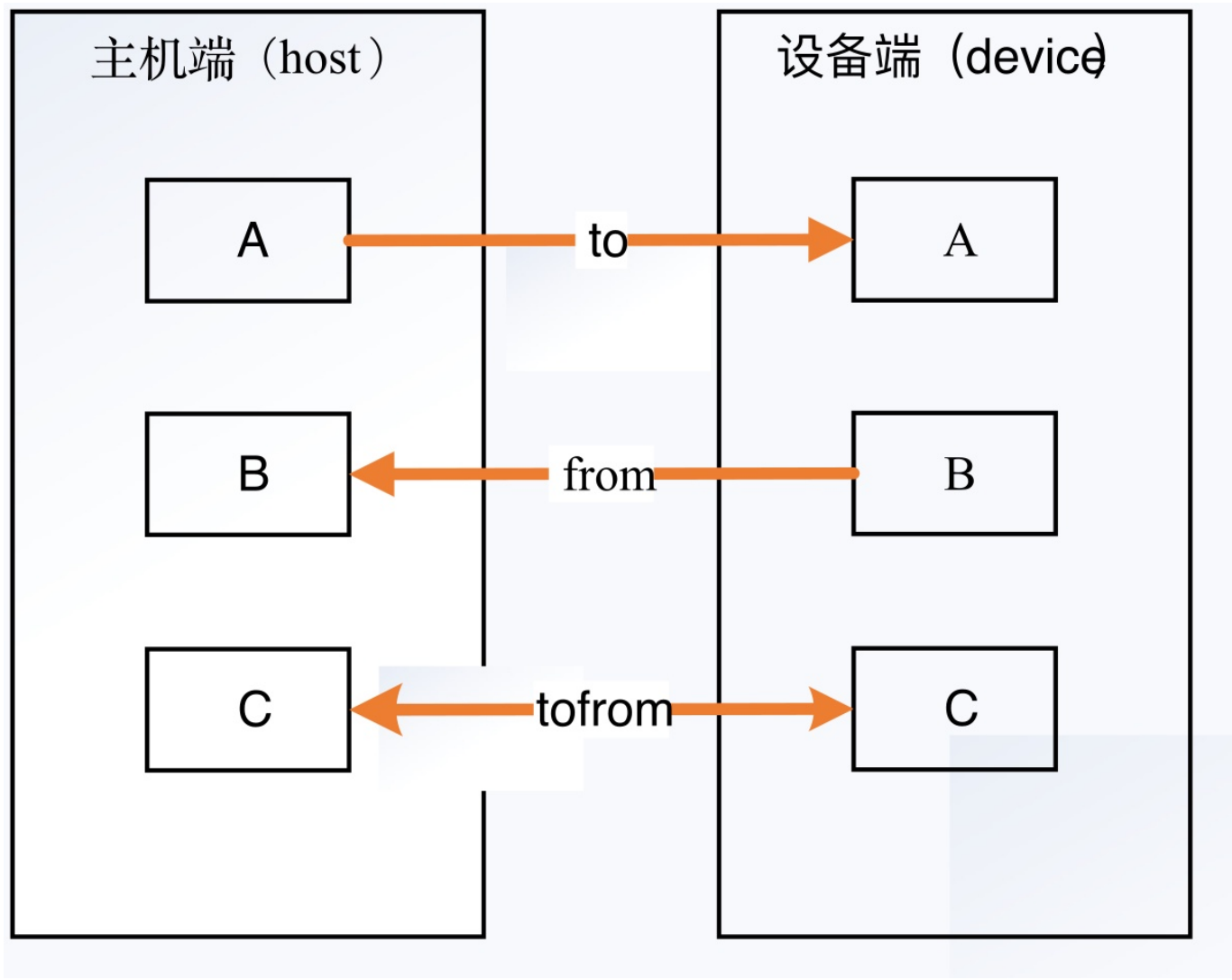


图1.7 map子句映射

一个target指令后面可以有多个map子句。了解了map指令的用法之后，我们要根据参数本身的使用特性，去使用map指令的映射类型，以此减少不必要的传输，提升程序运行速度。

1. 无映射线程私有变量–private子句
private子句将一个或多个变量声明为线程私有的变量，指定每个线程都有它自己的同名变量，其他线程无法访问私有副本。线程中的private变量不会初始化为主线程中原变量的值，且线程中的private变量在OpenMP区域结束后也不会更新主线程中原变量的内存。该子句适用于设定线程私有的中间变量参数。

```
//omp_private.c
int tmp = 3;
printf(“before device tmp = %d\n”,tmp);
#pragma omp target map(tofrom:A[0:N]) private(tmp)
{
  #pragma omp teams distribute parallel for
  for (i=0; i<N; i++)
  {
    tmp = tmp + 10;
     A[i] = tmp;
  }
```

```
}
printf(“after device tmp = %d\n”,tmp);
printf(“after device A[0] = %d\n”, A[0]);
```

输出结果为：

```
before device tmp = 3
after device tmp = 3
after device A[0]= 10
```

1. 只进不出标量传输指令–firstprivate子句
firstprivate子句将一个或多个变量声明为线程私有的变量，指定每个线程都有它自己的同名变量，其他线程无法访问私有副本。线程中的firstprivate变量通过主线程原变量来初始化，但线程中的firstprivate变量在OpenMP区域结束后不会更新原变量的内存。

target区域内所有未显示设定的变量都默认是firstprivate。

```
//omp_firstprivate.c
int tmp = 3;
#pragma omp target map(tofrom:A[0:N]) firstprivate(tmp)
{
  #pragma omp teams distribute parallel for
  for (i=0; i<N; i++)
  {
    tmp = tmp + 10;
     A[i] = tmp;
  }
}
printf(“after device tmp = %d\n”,tmp);
printf(“after device A[0] = %d\n”, A[0]);
```

输出结果为：

```
before device tmp = 3
after device tmp = 3
after device A[0]= 13
```

4.3.1.2 数据的隐式传输

当target结构中使用的数据没有在map子句或者private、firstprivate子句中出现时，OpenMP会将这些数据设置为隐式传输数据，其中数组、结构体等数据为隐式tofrom类型，整型、浮点型等基本类型数据为firstprivate类型。

defaultmap子句是target结构所独有的子句，作用是定义所有没有显式定义的scalar、aggregate、pointer类型参数的操作方式，它的参数有alloc、to、from、tofrom、firstprivate、none和default等。如defaultmap（tofrom：scalar），意思是将所有target区域中未显示指定的标量建立tofrom映射关系。

4.3.2 数据传输结构

4.3.2.1 多kernel的数据环境–target data结构

当同时存在多个target结构，且结构中有相同的元素时，会出现频繁的主机端和设备端之间的数据拷贝，非常耗时，此时可以用target data指令建立一个数据环境，减少数据的传输次数。

指令target data通过将host内存映射到device上，为结构target创建一个设备数据环境。指令target data后子句map使用到的数据在target data结构存在期间保持有效。使用方法如下：

```
C/C++:
#pragma omp target data clause[ [ [,] clause] ... ]
结构内容
Fortran:
!$omp target data clause[ [ [,] clause] ... ]
结构内容
!$omp end target data
```

其中，子句如下所示：

```
if([ target data :] scalar-expression)
device(integer-expression)
map([[map-type-modifier[,] [map-type-modifier[,] ...]] map-type: ] locator-list)
use_device_ptr(list)
use_device_addr(list)
```

target data仅建立一个数据环境，若target data结构中没有使用到target结构，则target data结构中的代码还是运行在host端。

如程序段中存在3个target结构（kernel），每个target结构在开始和结束处都会根据kernel内容和指令进行数据的传输，其中A和B参数在三个kernel中均存在，因此会传输多次，若数据一直驻留在设备端，无疑会减少数据的频繁传输，此时可以使用target data结构，优化后如下：

```
//omp_target_data.c
void vec_mult(int N)
{
  int i;
  float A[N], B[N], C[N];
  init(A,B,C,N);
  omp_set_num_threads(N);
  #pragma omp target data map(tofrom:A[0:N]) map(from:B[0:N],C[0:N])
  {
    #pragma omp target
    {
      #pragma omp parallel for
      for (i=0; i<N; i++)
      {
        A[i] = i;
```

```
      }
    }
    #pragma omp target
    {
      #pragma omp parallel for
      for (i=0; i<N; i++)
      {
        B[i] = A[i] + i;
      }
    }
    #pragma omp target
    {
      #pragma omp parallel for
      for (i=0; i<N; i++)
      {
        C[i] = A[i] + B[i];
      }
    }
  }
}
```

使用target
data结构将三个kernel包含在内，创建了一个持续存在的数据空间，因此两target结构之间不必重复拷贝数据，A,B,C数组只需在target data结构开始时拷贝一次，结束时再拷贝一次即可，节省了拷贝时间。

4.3.2.2 非结构化数据结构—target enter/ exit data指令

针对于全局变量，我们可以使用target enter/ exit data指令。相对于target
data指令必须包括一个完整的结构，target enter/ exit
data指令可以在代码中的任何位置导入导出或删除设备端数据。

target enter/ exit data指令的语法格式如下：

```
C/C++:
#pragma omp target enter data clause[ [ [,] clause] ... ]
#pragma omp target exit data clause[ [ [,] clause] ... ]
Fortran变量:
!$omp target enter data clause[ [ [,] clause] ... ]
!$omp target exit data clause[ [ [,] clause] ... ]
```

其中，子句如下所示：

```
if([ target data :] scalar-expression)
device(integer-expression)
map([[map-type-modifier[,] [map-type-modifier[,] ...]] map-type: ] locator-list)
nowait
depend([depend-modifier,] dependence-type : locator-list)
```

需要注意的是，对于map子句，target enter data指令拥有alloc, to, from, tofrom等属性，而target

exit data则拥有to, from, tofrom，release和delete等属性。

使用target enter/ exit data指令，再搭配target
update指令，可以灵活的解决绝大部分的数据传递问题。

4.3.2.3 全局参数声明–declare target指令

指令declare
target用于声明在target结构内函数（等同于HIP的device函数）和全局变量能够在默认的device上运行。指令declare target的语法格式如下：

```
C/C++:
#pragma omp begin declare target
函数定义或extern函数
全局变量
#pragma omp end declare target
Fortran变量:
module
!$omp declare target(list)
end module
Fortran函数:
function
!$omp declare target
end function
```

declare target指令的使用场景基本上有以下两方面，第一，声明设备端函数，类似于HIP
C中__device__开头的设备端函数，标明该函数可以运行在设备端，例子如下：

```
//declare_target1.c
#pragma omp begin declare target
extern void scalaradd(int m);
#pragma omp end declare target
void declare_test(int m)
{
  #pragma omp target
  {
     scalaradd(m);
}
}
```

第二个使用场景是解决以DCU为主的编程场景下的数据驻留问题，declare
target可以指定全局变量在所有默认设备上映射，并且在程序运行期间一直保持在设备端。需要注意的是，declare
target结构生命的设备端参数整个程序运行期间都保持在设备端，且该指令指定的参数属性优先级最高。
C语言相关例子如下：

```
//declare_target_sample.c
#include <omp.h>
#include <stdlib.h>
```

```
#include <stdio.h>
#include "declare_define.h"
void vec_mult(int);
void compute_target();
#pragma omp declare target
float de_v1;
int de_v2[5];
#pragma omp end declare target
int main(void)
{
   int N=10;
   vec_mult(N);
   return 0;
}
void vec_mult(int N)
{
   int i;
for(i=0;i<5;i++)
      de_v2[i]=i+1;
   printf("init de_v1 = %f,de_v2[0] = %d\n",de_v1,de_v2[0]);
#pragma omp target update to(de_v2)
   #pragma omp target data
   {
      #pragma omp target
      {
         de_v1 = de_v1 + 1.0f;
         de_v2[0] = de_v2[0] + 20;
        printf("first address de_v1 = %llx\n",&de_v1);
      }
      printf("before update de_v1 = %f, de_v2[0]=%d\n",de_v1,de_v2[0]);
      #pragma omp target update from(de_v1, de_v2)
      printf("after target1 de_v1 = %f, de_v2[0]=%d\n",de_v1,de_v2[0]);
      compute_target();
      printf("end  de_v1 = %f,de_v2[0]=%d\n",de_v1,de_v2[0]);
   }
}
```

程序运行结果为：

```
init de_v1 = 0.000000,de_v2[0] = 1
first address de_v1 = 2b82a9409020
before update de_v1 = 0.000000, de_v2[0]=1
after target1 de_v1 = 1.000000, de_v2[0]=21
other file before de_v1 = 1.000000,de_v2[0] = 21
other file de_v1 = 3.000000, de_v2[0] = 23
other file address de_v1 = 2b82a9409020
other file after de_v1 = 3.000000,de_v2[0] = 23
end  de_v1 = 3.000000, de_v2[0]=23
```

使用declare
target指令定义了全局变量de_v1，并对de_v1进行了一系列的操作，在此文件最后调用了另一个文件的compute_declare.c中的函数compute_target，且在compute_target函数中也有对de_v1的操作。计算结果如我们预想一样。

C语言使用declare target指令声明全局变量情况下，在其他文件想要调用该参数，则要使用declare target指令声明参数一在设备中，如compute_declare.c文件所示。且当我们在两个文件中都打印出de_v1参数的地址时，地址相同，证明是同一变量。

同样的，数组de_v2也是相同的操作。

C语言使用declare
target指令定义动态数组和指针的方法需要使用omp_target_alloc()等runtime函数，具体请参照5.3.3小节。

declare
target在指定声明全局变量时，也能够指定变量的生命周期，表示变量一直存在于device上，不会被释放。

4.3.2.4 随时随地传输数据–target update指令

target update是一个独立的指令，根据使用者的需要，显式地更新在target data区域内的host或device设备中的变量。语法格式如下：

```
C/C++:
#pragma omp target update clause[ [ [,] clause] ... ]
Fortran:
!$omp target update clause[ [ [,] clause] ... ]
```

其中，子句为如下语法：

```
if([ target update :] scalar-expression)
device(integer-expression)
nowait
depend([depend-modifier,] dependence-type : locator-list)
to([motion-modifier[,] [motion-modifier[,] ...]: ] locator-list)
from([motion-modifier[,] [motion-modifier[,] ...]: ] locator-list)
```

to子句表示从host端向device端更新数据，from子句表示从device端向host端更新数据，相同变量不能在两个子句中同时出现。

使用update的例子如下：

```
  subroutine vec_mult(p, v1, v2, N)
    real    :: p(N), v1(N), v2(N)
    integer :: i,j
    real    :: out1,out2
    call init(v1, v2, N)
    !$omp target data map(to: v1) map(from:p)
        !$omp target
```

```
        !$omp parallel do
        do i=1,N
          p(i) = v1(i)
        end do
      !$omp end target
      call init_again(v1, N) !重新初始化V1
      !$omp target update if(N<1) to(v1)
      !$omp target
        !$omp parallel do
        do i=1,N
          p(i) = v1(i) + p(i)
        end do
   !$omp end target
  !$omp end target data
  out1 = p(1)
  call init(v1, v2, N)
  !$omp target data map(to: v1) map(from:p)
      !$omp target
        !$omp parallel do
        do i=1,N
          p(i) = v1(i)
        end do
      !$omp end target
      call init_again(v1, N)
      !$omp target update if(N>1) to(v1)
      !$omp target
        !$omp parallel do
        do i=1,N
          p(i) = v1(i) + p(i)
        end do
   !$omp end target
  !$omp end target data
  out2 = p(1)
end subroutine
```

4.3.3 数据映射的优先级和生存周期

OpenMP异构并行计算中，有多种方式和指令操作数据在主机端和设备端的传输，若各个指令有数据传输的优先级和生命周期冲突，可以按照declare target最高，target data/ enter data/ exit data次之，target最低来理解。

需要注意的是，低优先级的变量属性不会修改高优先级属性。下面我们来看一个数据传输的例子，这个例子可以更好地帮助我们理解数据的传输情况和生命周期。

[图片]

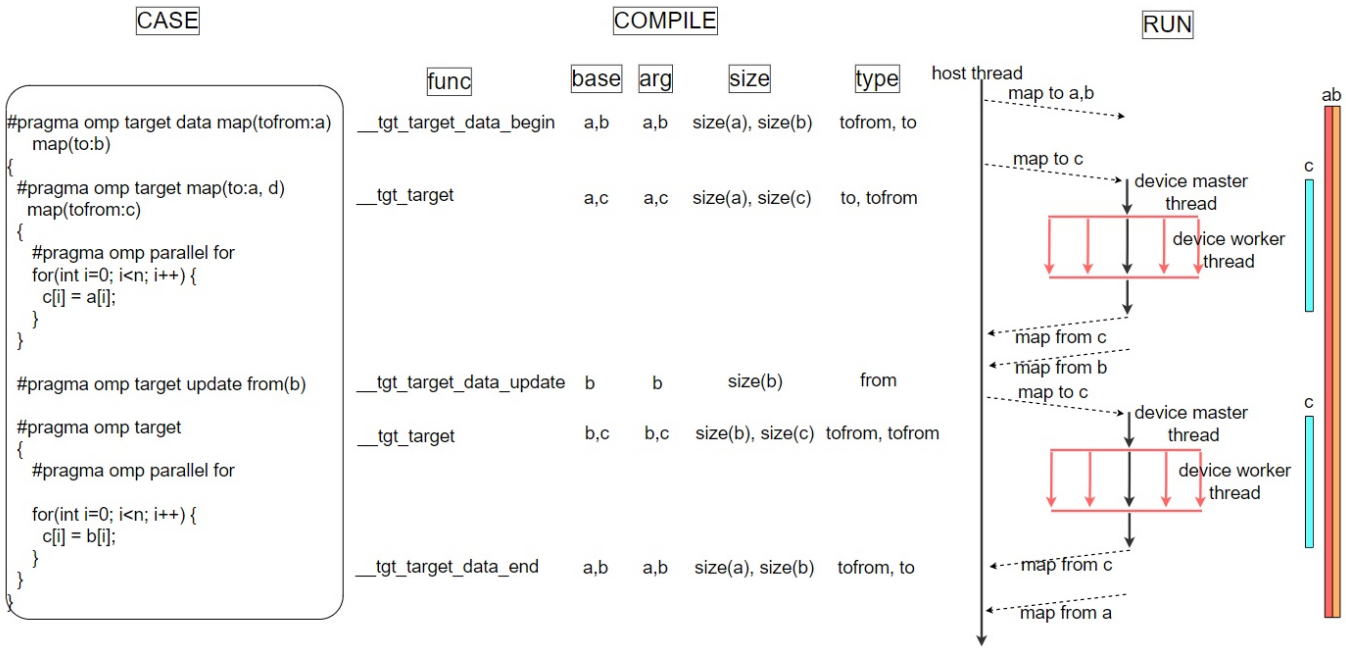


图1.8 OpenMP数据传递综合示例

如上图中的例子，用户在代码中使用了target data、target、target update指令进行数据的传输与计算，使用了target data指令先创建一个数据环境，期望将部分数据传入到device上。在target data区域中，使用了两个target指令和一个target update指令。在这个例子中，使用到的数据都是在栈中定义的一维临时数组。

在编译器编译这段指令时，将各类OpenMP指令转换成不同的函数调用，且函数参数主要有4类：基础地址、数据地址、数据长度、数据类型。在target data指令中，用户显式映射了a和b数据，且数据类型为tofrom和to，编译器会将a和b加入二进制文件的待传输参数中。在第一个target指令中，用户显式映射了a、c、d三组数据，编译器在检查时，发现target区域中没有使用到d数据，那么只会将a、c数据加入待传输参数中。在使用target update指令时，用户手动指定了需要传输b数据，则将b加入待传输参数中。在第二个target区域中，用户没有显式指定需要传输的参数，编译器在target区域中查找到使用了b和c数据，所以将b和c加入到待传输参数中，属性为默认的tofrom。

在执行这段例子时，由于device内存中不存在使用到的数据，target data指令首先在device上申请内存，将a和b数据复制到device上。执行到第一个target指令时，由于检查到a已经存在于device上，所以不传输a数据，在device上申请长度为c长度的内存，并将c数据复制到device上，target区域结束后将c从device复制到host，然后释放device上c的内存。在第二个target指令中，需要传输b和c数据，检查到b已经存在于device上，所以不传输b数据，在device上申请长度为c长度的内存，并将c数据复制到device上，target区域结束后将c从device复制到host，然后释放device上c的内存。在target data执行完成时，由于a数据属性为tofrom，需要将a从device复制到host，然后释放device上a和b的内存。

4.4 OpenACC的设备端并行

4.4.1 OpenACC的编译运行

对于OpenACC，在该章节中列举出的例子的编译方法如下：

```
flang -acc test.f90 -O3
flang -fopenacc --offload-arch=gfx936 test.f90 -O3
clang++ -fopenacc --offload-arch=gfx936 test.cpp -O3
```

4.4.2 OpenACC的计算构件

当前DTK版本已经支持Fortran语言大部分OpenACC指令和API。

OpenACC只要使用并行化指令parallel或serial，就默认进行核函数创建，执行时会卸载到GPU上执行。

与OpenMP相同，线程块都为一维的形式，不同的是OpenACC中线程组的名字是gang，线程名字是vector。线程组gang可以使用parallel指令和gang子句激活并行化，线程可以使用vector子句激活并行化。同时，可以使用loop指令将一个for循环中的任务分发到多个线程中。

4.4.2.1 计算构件parallel

与OpenMP的target指令类似，OpenACC
parallel指令的作用也是创造一个并行域相当于HIP或CUDA的核函数。

gang类似于HIP或CUDA中的线程块，每个gang包含一个或多个worker；worker类似于HIP或CUDA中的线程束；每个worker都有一个向量宽度vector，由一个或多个同时执行指令的向量元素组成。copy，create等数据传递子句将在数据传输章节详细讲述。

1.
程序在遇到parallel指令时，可能会创建一个或多个gang来执行这段并行区域。在并行区域内，每个gang都以冗余模式执行代码结构。
2.
与kernels指令不同的是，在没有使用loop指令时，parallel指令只有一个gang执行并行化，在一个gang中分配出多个worker和vector，循环体在一个gang上并行运行。
3.
若想使用多个gang来并行执行parallel区域中的循环，需要使用loop指令。使用者需要保证在loop指令中的数据不存在数据依赖。程序会自动分裂出多个gang和多个vector来并行执行代码。

4.4.2.2 串行计算构件serial

与parallel指令类似，OpenACC
serial指令的作用也是创造一个并行域相当于HIP或CUDA的核函数，不同的是，核函数在设备端为串行执行。其他子句的用法与parallel一致。

4.4.2.3 计算构件kernels

kernels指令功能与parallel指令类似，Parallel表示在结构中只产生一个核函数，kernels表示在结构中可能产生一个或者多个核函数，所以要考虑多个核函数的场景。Kernels结构在区域上更像是data指令，表示一个区域，在这个区域中所有的代码都是在设备端执行的，且需要尽可能并行化；在产生核函数功能上像parallel。使用格式如下：

1. 使用!$acc kernels开始，!$acc end
kernels结束，表示kernels区域，内部可能会产生一个或多个函数。
2. 使用!$acc kernels loop开始，下面接do循环。使用!$acc end kernels或!$acc end kernels loop结束；或者缺省acc指令，检测到do循环自动结束。内部只会产生一个循环的核函数。
3. 区域内部使用loop指令的格式与其他loop指令格式相同。
产生核函数方式如下：
4. 每个Do循环产生一个核函数，并行执行。

```
!$acc kernels num_gangs(a) num_workers(a)
!$acc loop
do i=1,N
 v1(i) = v1(i) + 1.
 v2(i) = v2(i) + 1.
end do
!$acc end loop
!$acc end kernels
```

5. 一个或多个连续的非循环语句合并到一个核函数中，串行执行。

```
!$acc kernels
v1(1) = v1(1) + 1.
V2(1) = v2(1) + 1.
!$acc end kernels
```

6. Do循环和非do循环混合，产生多个核函数。

```
!$acc kernels
v1(1) = v1(1) + 1.
!$acc loop
do i=1,N
  v1(i) = v1(i) + 1.
  v2(i) = v2(i) + 1.
end do
!$acc loop
do i=1,N
  v1(i) = v1(i) + 1.
  v2(i) = v2(i) + 1.
end do
v1(1) = v1(1) + 1.
!$acc end kernels
```

4.4.2.4 循环分配构件loop

loop指令可以用于kernels指令区域中，也可以用于parallel指令区域中。Loop指令作用于紧跟该指令的一个循环，可以描述执行循环的并行类型，也可以声明循环的私有变量、数组和规约操作。

1.
independent子句作用是告诉编译器在循环中，所有的数据都是独立的，均不会产生数据竞争。在parallel指令中，不带seq子句的loop指令都隐式包含一个independent。而在kernels指令中，使用loop指令，只是向编译器提供了一个参考，若发现数据竞争，则同样会拒绝并行化执行，若显式加入一个independent，则直接执行并行化。
2. reduction子句与OpenMP中的规约子句作用相同。
3. gang子句表示关联循环的每一次迭代需要在创建的所有gang上并行执行。
4.
若不带worker子句，loop指令在运行时只分配一个worker。带worker子句后，先激活额外的多个worker，然后将循环分摊到这些worker上。
5.
vector子句表示关联循环的每一次迭代都以向量化SIMD模式执行，vector子句先激活额外的多个向量通道，然后将循环分摊到这些vector上。
6. seq子语指明关联循环需要在加速器上串行地执行。本子语将阻止所有的自动并行化、向量化。

4.4.3 OpenACC异步

4.4.3.1 async子句

async子句可以出现在parallel、kernels、serial、enter data、exit data、update、wait导语中，都是可选子句。

没有async子语的时候，在执行任何后续代码之前，主机线程会一直等待直至异构指令操作完成；反之async子句存在时，在主机线程继续执行异构指令后面代码的同时，parallel、kernels指令区域或数据操作可以被异步地处理。

async子句可以有一个async参数，类型为非负数整型值或系统定义的特殊，参数值用于选择设备端的异步队列，其取值范围受限于硬件设备支持的异步队列，当前DCU设备支持的队列号为0~3，默认队列号为0，同一队列中的任务按顺序执行；

async特殊值及其含义：

```
#define acc_async_sync -1   //同步执行，等同于未使用async子句
#define acc_async_default -3   //异步执行，使用默认队列0，等同于async(0)
#define acc_async_noval -4   //异步执行，使用默认队列0，async(0)
```

以下示例中，由于async子句的存在，数组b在设备队列1中更新的同时，数组c也在设备队列2中异步更新。

```
program async_wait
  use openacc
  implicit none
  integer, parameter :: N = 4096
  integer :: i
```

```
  integer :: a(N), b(N), c(N)
  a = 1;
  b = 0;
  c = 0;
  !$acc data copyin(a) copyout(b, c)
  !$acc parallel loop async(1)  !kernels_1
  do i = 1, N
    b(i) = a(i) * -1
  end do
  !$acc end parallel loop
  !$acc parallel loop async(2)  !kernels_2
  do i = 1, n
    c(i) = a(i) + 1
  end do
  !$acc end parallel loop
  !$acc wait
  !$acc end data
  do i = 1, N
    print *, "b(i) = ", b(i);  ! b=-1
    print *, "c(i) = ", c(i);  ! c=2
  end do
end program async_wait
```

4.4.3.2 wait子句

wait子句可以出现在parallel、kernels、serial、enter data、exit data、update导语上，都是可选子句。没有wait子句时，在设备上关联的计算、数据更新等操作会被立即压入队列、启动或执行；反之，wait子句存在时，计算、数据更新操作可能不会被立即启动或执行，直到本线程在这个位置前压入关联异步设备活动队列的所有操作都已经完成。在实际应用中wait做子句的用法较少，多用wait指令，示例参见wait指令。

4.4.3.3 wait指令

wait指令将引发主机线程等待，直到在此之前压入队列的所有相关异步活动都已经完成。

使用语法如下：

```
C/C++:
#pragma acc wait [( int-expr-list )] [clause-list] new-line
Fortran:
!$acc wait [( int-expr-list )] [clause-list]
```

子句语法：

```
async [( int-expr )]
```

在以下示例中，由于数据存在依赖关系，d(i)值更新任务需要等待队列2中的c(i)值更新任务完成，同时无

依赖关系的b(i)值的更新继续在队列1中异步执行。

```
program wait_async
  use openacc
  implicit none
  integer, parameter :: N = 102400
  integer :: i, j
  integer :: errors = 0
  integer :: a(N), b(N), c(N), d(N)
  a(i) = 1;
  b(i) = 0;
  c(i) = 0;
  d(i) = 0;
  !$acc data copy(a, b, c)
  !$acc parallel loop async(1)
  do i = 1, N
    b(i) = a(i) * -1    ! b=-1
  end do
  !$acc end parallel loop
  !$acc parallel loop async(2)
  do i = 1, n
    c(i) = a(i) + 1     ! c=2
  end do
  !$acc end parallel loop
  !$acc wait(2) async(1)
  !$acc parallel loop async(3)
  do i = 1, n
    d(i) = c(i) + 1     ! d = 3
  end do
  !$acc end parallel loop
  !$acc wait
  !$acc end data
  do i = 1, N
    if (b(i) /= -1 .or. c(i) /= 2 .or. d(i) /= 3) then
      errors = 1
      print *, "b(i)=", b(i)
      print *, "c(i)=", c(i)
      print *, "d(i)=", d(i)
      exit
    end if
  end do
  if (errors /= 0) then
    print *, "acc wait async test failed."
  else
    print *, "acc wait async test pass."
  end if
end program wait_async
```

示例中的以下指令:

```
!$acc wait(2) async(1)
!$acc parallel loop async(3)
```

等同于：

```
!$acc parallel loop async(3) wait(2)
```

即为wait子句的用法，均实现队列3任务启动前等待队列2中的任务执行结束，同时队列1种的任务继续保持异步执行；

4.5 OpenACC数据传输

与OpenMP类似，OpenACC异构计算中也需要对主机与设备之间的数据传输进行管理和优化。

本章节将会介绍OpenACC中数据管理构件和子句是如何在主机内存和设备内存之间进行数据的传递，以及如何分配空间，释放内存，管理数据的生存周期和作用域。

OpenACC常用的数据导语包括data、enter data、exit data、update、declare等。

4.5.1 data指令

data指令与OpenMP中的target data指令功能类似，为结构创建一个设备数据环境。

4.5.2 enter data指令和exit data指令

enter data指令和exit data指令与OpenMP中的target enter data指令和target exit data指令功能类似。

4.5.3 update指令

update指令与OpenMP中的target update指令功能类似。

4.5.4 declare/routine指令

declare指令用于Fortran子程序、函数或模块的声明部分，或者在C或C++中的变量声明之后。可以指定在函数、程序的隐式数据区域期间在设备内存中分配一个变量，并指定是否在进入隐式数据区域时将数据值从本地内存传输到设备内存，以及在退出隐式数据区域时是否将数据值从设备内存传输到本地内存。routine指令表示声明的函数为设备端函数，可以在设备端调用和执行。

从功能来看OpenACC中的declare/routine指令与OpenMP中的declare target指令类似，实际使用时有些许不同。

```
MODULE module_sf
```

```
 REAL，DIMENSION(1:10 ) :: psim_stab
 !$acc declare create(psim_stab)
CONTAINS
 FUNCTION func_psim_stab(I)
  !$acc routine seq
  integer :: i
  func_psim_stab = psim_stab(I)
  return
 END FUNCTION func_psim_stab
 SUBROUTINE SFCL ()
  IMPLICIT NONE
  REAL,dimension(1:10) ::  buf_1, buf_2
  integer :: i
  !$acc parallel
  !$acc loop gang vector
  DO I=1,10
   buf_1(I)= psim_stab(I)
  ENDDO
  !$acc end parallel
  !$acc parallel
  !$acc loop gang vector
  DO I=1,10
   buf_2(I)= func_psim_stab(I)
  ENDDO
  !$acc end parallel
  if (abs(buf_1(1) - psim_stab(1)) < 0.0001) then
   print*, "PASS1"
  endif
  if (abs(buf_2(1) - psim_stab(1)) < 0.0001) then
   print*, "PASS2"
  endif
 END SUBROUTINE SFCL
END MODULE
program main
 use module_sf
 psim_stab = 10.1
 !$acc update device(psim_stab)
 call SFCL ()
end program main
```

如上示例，module中定义了一个全局变量，并且使用了declare声明，表示需要在设备端创建一个全局变量副本。程序初始化变量后，使用update指令更新全局变量内存。共有两个核函数，第一个核函数直接保存全局变量的值并且传输回主机端，第二个核函数通过调用一个routine声明的函数，保存全局变量的值，并且传输回主机端。执行程序，两个核函数返回的值为一致。

4.5.5 host_data指令

host_data指令使设备内存中的数据地址在主机端可以直接使用，避免了数据传输的复制过程，其功能与OpenMP指令 target
data的子句use_device_ptr相同；包含必选子句use_device和可选子句if、if_present。

使用host_data指令可以将OpenACC代码与hip代码混合使用。hip接口的参数一般需要传入设备端指针，而在OpenACC、OpenMP的代码中，很多场景都是只有主机端的指针。

OpenACC可以使用host_data
use_device指令，在该区域内，主机端的指针会被编译器转换为设备端指针，区域执行完成后，设备端指针又会被转换为主机端指针。OpenMP可以使用use_device_addr指令，也能达成上述效果。代码参考如下：

```
!$ACC HOST_DATA USE_DEVICE(fvlT, test)cufft_status = hipfftExecD2Z(plan, c_loc(fvlT),c_loc(test))if (cufft_status /= SUCCESS) print *, 'cufftExecD2Z, code=', cufft_status!$ACC END HOST_DATA
```

4.5.6 private私有数组

当程序从cpu串行移植到加速卡并行时，不可避免的会出现各类数据竞争的问题。用于为每个并行线程或向量分配独立的数组副本，避免线程间的数据竞争。

当多个线程并行执行时，private数组会为每个线程或线程组创建一份独立的数组副本，仅能访问自己的副本，修改不会影响其他线程，也不会影响主机端的原数组。

private子句位于loop的不同层级，其作用也不同。

private位于gang层，表示每个线程组中都维护了一个私有数组，线程组间互相隔离：

```
!$acc parallel  !$acc loop gang private(c)  DO x = 1, 10  !$acc loop vector  DO y = 1, LOOPCOUNT  c(y) = a(x,y) + b(x,y)  END DO  DO y = 1, LOOPCOUNT  d(x) = d(x) + c(y)  END DO  END DO!$acc end parallel
```

private位于vector层，表示每个线程中都维护了一个私有数组，线程间互相隔离：

```
!$acc parallel  !$acc loop gang  DO x = 1, 10  !$acc loop vector private(c)  DO y = 1, LOOPCOUNT  c(1) = a(x,y)  c(2) = b(x,y)  c(3) = 1  d(:, y, x) = c  END DO  END DO!$acc end parallel
```

private位于gang vector层，表示每个线程中都维护了一个私有数组，线程间互相隔离：

```
!$acc parallel  !$acc loop gang vector collapse(2) private(c)  DO x = 1, 10  DO y = 1, LOOPCOUNT  c(1) = a(x,y)  c(2) = b(x,y)  c(3) = 1  d(:, y, x) = c  END DO  END DO!$acc end parallel
```

4.5.7 结构体、结构体数组、结构体内部多级指针

用户可以直接在数据传输子句中进行结构体、结构体数组、结构体内部多级指针等数据的传输。编译器会自动识别类似数据类型，进行一系列操作。

编译器自动将结构体中的主机端指针替换为设备端指针，无需用户手动替换。支持多层结构体嵌套，结构体数组嵌套的H2D和D2H。支持传输结构体中的单个值、单个数组。支持在acc
routine指令中使用设备端的结构体元素。具体原理如下图所示：

[图片]

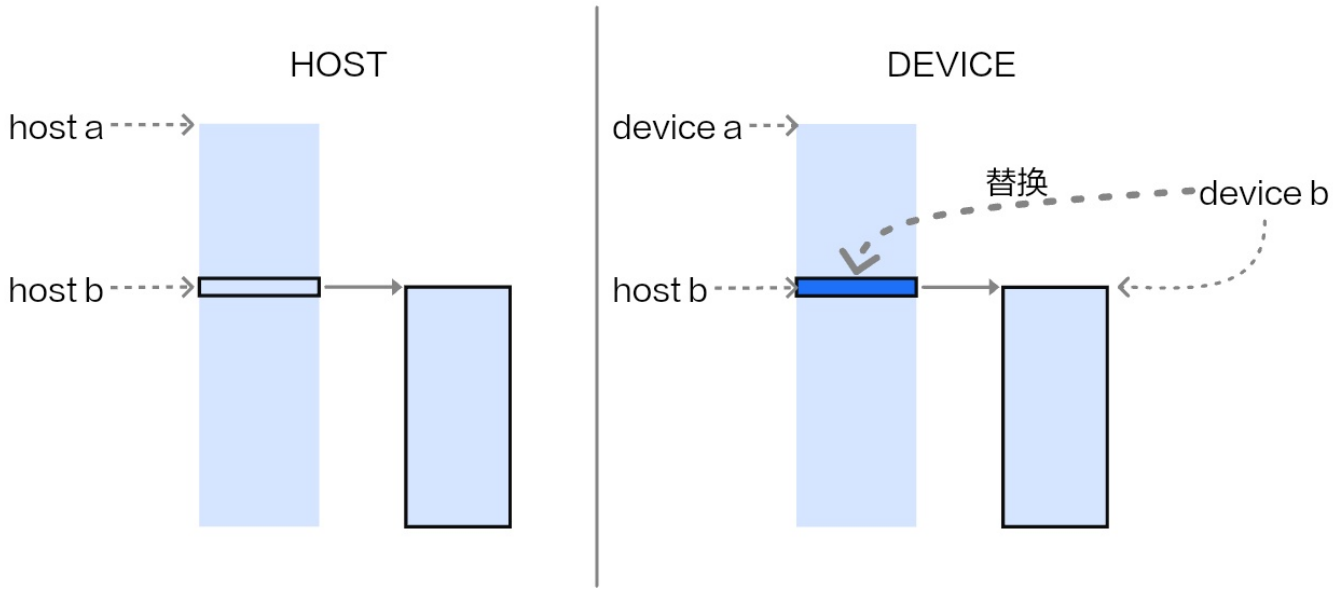


图1.9 编译器自动替换结构体中的指针

使用方式如下代码：

```
type :: C1   integer :: a   real(8), pointer :: b(:) => null()end typetype :: C   integer :: a   real(8), pointer :: b(:) => null()   type(C1) :: cend typetype :: D   integer :: e   type(C) :: f   integer :: hend typetype(D) :: sa(N)!$acc parallel copy(sa(2)%e, sa(2)%f%a, sa(2)%f%c%a)   sa(2)%e = 333   sa(2)%f%a = 444   sa(2)%f%c%a = 555   sa(2)%f%b(2) = 40.0D0!$acc end parallel
```

4.6 调试方法

OpenMP和OpenACC调试方法大致相同，但是c/c++语言与fortran语言，根据编译器的开发进度，有一定的区别。如下所述：

表 5.1 OpenMP和OpenACC的调试方法

语言	C/C++	Fortran	备注
核函数启动信息	LIBOMPTARGET_KERNEL_TRACE=1	LIBOMPTARGET_KERNEL_TRACE=1	
执行时间（所有函数）	LIBOMPTARGET_KERNEL_TRACE=2	LIBOMPTARGET_KERNEL_TRACE=2	OMPX_FORCE_SYNC_REGIONS=1同步执行
执行时间（只打印核函和同步）		LIBOMPTARGET_KERNEL_TRACE=32	OMPX_FORCE_SYNC_REGIONS=1同步执行
异构数据传输信息	export LIBOMPTARGET_INFO=59	export LIBOMPTARGET_INFO=59	编译时添加-g可以打印函数名称与文件行数

PROF性能统计		hipprof ./out	打印核函数统计、API计、火焰图
VMFault信息		核函数崩溃后自动打印	
核函数计算完成后各数求和		LIBOMPTARGET_INFO=256 OMPX_FORCE_SYNC_REGIONS=1	用于对比后查找到计算错的核函数和数组。执性能大幅下降。
配置默认加速卡		HIP_VISIBLE_DEVICES=4,5,6,7	可以设置多个，对应代中的卡号 0,1,2…
编译过程信息		编译命令添加 -Minfo，支持OpenACC	

4.6.1 核函数启动信息

添加如下环境变量

```
export LIBOMPTARGET_KERNEL_TRACE=1
```

例如，出现如下打印：

```
DEVID: 0 SGN:0 ConstWGSize:256  args:24 teamsXthrds:(1024X 256) reqd:(  0X  0) lds_usage:6224B
sgpr_count:58 vgpr_count:53 sgpr_spill_count:0 vgpr_spill_count:0 tripcount:0 rpc:0 n:
sw_core_mod_c_sw__TARGET_159_1_
DEVID: 0 SGN:0 ConstWGSize:256  args:31 teamsXthrds:(1024X 256) reqd:(  0X  0) lds_usage:6224B
sgpr_count:63 vgpr_count:42 sgpr_spill_count:0 vgpr_spill_count:0 tripcount:0 rpc:0 n:
sw_core_mod_c_sw__TARGET_191_2_
```

在这些日志打印中，DEVID表示执行的设备编号，teamsXthrds:(1024X 256)表示生成的线程组的数量为1024个，每个线程组中线程的数量为256个，sw_core_mod_c_sw__TARGET_159_1_为当前执行的核函数名称，这个核函数名称是编译器自动产生的，里面包含了module名称sw_core、函数名c_sw，159表示当前核函数位于文件的159行，最后的数字表示核函数编号，在flang中，这个编号是按照当前文件从上到下的顺序依次给核函数加的序号。

4.6.2 异构数据传输信息

若在编译时添加-g选项，表示添加debug信息，打印出的日志会包含变量名称、文件行号等信息。使用如下环境变量：

```
export LIBOMPTARGET_INFO=59
```

例如，出现如下日志打印：

```
Libomptarget device 0 info: Entering OpenMP kernel at loop-gangworkervector.f90:20:0 with 9
```

```
arguments:
Libomptarget device 0 info: firstprivate(j)[4] (implicit)
Libomptarget device 0 info: tofrom(p)[400000000]
Libomptarget device 0 info: Creating new map entry with HstPtrBase=0x00002b5aa695a020,
HstPtrBegin=0x00002b5aa695a020, TgtPtrBegin=0x00002b5abee00000, Size=400000000,
DynRefCount=1, HoldRefCount=0, Name=p
Libomptarget device 0 info: Copying data from host to device, HstPtr=0x00002b5aa695a020,
TgtPtr=0x00002b5abee00000, Size=400000000, Name=p
```

上面的日志中，前两行表示传输了两组数据，类型分别为firstprivate和tofrom，长度分别为8字节和4096字节，且都是隐式传输。Runtime对数据创建了一个映射map，且能够看到数据的主机端地址和设备端地址。在复制数据之前，发现在device端已经存在了一段数据，所以不进行数据的传输。最后一行表示程序结束，删除数据映射。

4.6.3 核函数执行时间

使用如下环境变量执行程序：

```
export LIBOMPTARGET_KERNEL_TRACE=2
export OMPX_FORCE_SYNC_REGIONS=1
```

[图片]

```
Call               __tgt_rtl_load_binary,    19771,us, 0x56530b7a2f40 (             0, 0x56530b772490)
Call                __tgt_rtl_data_alloc,     8676,us, 0x2b1301e00000 (             0,     4060300500, 0x2b0f30600020)
Call               __tgt_rtl_data_submit,   433157,us,              0 (             0, 0x2b1301e00000, 0x2b0f30600020,     4060300500)
Call                __tgt_rtl_data_alloc,     8832,us, 0x2b13f5e00000 (             0,     4060300500, 0x2b1022635060)
Call               __tgt_rtl_data_submit,   406718,us,              0 (             0, 0x2b13f5e00000, 0x2b1022635060,     4060300500)
Call                __tgt_rtl_data_alloc,     8793,us, 0x2b14e8200000 (             0,     4060300500, 0x2b111466a0a0)
Call               __tgt_rtl_data_submit,   407067,us,              0 (             0, 0x2b14e8200000, 0x2b111466a0a0,     4060300500)
Call                __tgt_rtl_data_alloc,     8797,us, 0x2b15da600000 (             0,     4060300500, 0x2b120669f0e0)
Call               __tgt_rtl_data_submit,   396751,us,              0 (             0, 0x2b15da600000, 0x2b120669f0e0,     4060300500)
Call                __tgt_rtl_set_device,        2,us,              1 (             0)
PID: 3214, DEVID: 0 Stream: 0 SGN:2 ConstWGSize:256  args:12 gridDims:(65535, 1, 1) blockDims:(256, 1, 1) reqd:(   0X   0) lds_usage:0B
sgpr_count:44 vgpr_count:26 sgpr_spill_count:0 vgpr_spill_count:0 tripcount:126253125 rpc:0 n:circulation__TARGET_6_1_
Call            circulation__TARGET_6_1_,    24496,us,              0 (             0, 0x56530b7c7d00, 0x56530b8133e0, 0x56530b81b090,
          12, 0x7fffb251e448, 0x7fffb251e454,      126253125)
Call               __tgt_rtl_set_device,        0,us,              1 (             0)
PID: 3214, DEVID: 0 Stream: 0 SGN:2 ConstWGSize:256  args:12 gridDims:(65535, 1, 1) blockDims:(256, 1, 1) reqd:(   0X   0) lds_usage:0B
sgpr_count:43 vgpr_count:27 sgpr_spill_count:0 vgpr_spill_count:0 tripcount:127263150 rpc:0 n:circulation__TARGET_22_2_
Call           circulation__TARGET_22_2_,    26367,us,              0 (             0, 0x56530b8102d0, 0x56530b8133e0, 0x56530b81b430,
          12, 0x7fffb251e4b8, 0x7fffb251e4c4,      127263150)
Call               __tgt_rtl_set_device,        0,us,              1 (             0)
PID: 3214, DEVID: 0 Stream: 0 SGN:2 ConstWGSize:256  args:11 gridDims:(65535, 1, 1) blockDims:(256, 1, 1) reqd:(   0X   0) lds_usage:0B
sgpr_count:42 vgpr_count:18 sgpr_spill_count:0 vgpr_spill_count:0 tripcount:1015075125 rpc:0 n:circulation__TARGET_36_3_
Call           circulation__TARGET_36_3_,    27352,us,              0 (             0, 0x56530b7c5220, 0x56530b81b430, 0x56530b8133e0,
          11, 0x7fffb251e528, 0x7fffb251e534,     1015075125)
```

图1.10 使用环境变量打印时间

能打印出初始化、显存申请、数据传输、执行核函数时间信息。而如果将LIBOMPTARGET_KERNEL_TRACE的值设置为32，则只打印执行核函数时间。

4.6.4 核函数内部数据调试方法

可以在OpenMP/OpenACC的核函数内部添加print、write进行数据打印。

或者可以使用update指令，将数据从DCU端传输到CPU端，然后打印出来。

```
!$omp target update from(ua, va)
print"(f35.2)",sum(va(:,:,:)),sum(ua(:,:,:))
```

4.6.5 hipprof性能日志

对于Fortran，当前支持使用hipprof打印OpenMP和OpenACC的性能日志，直接使用hipprof ./out记录程序性能。详情请查看hipprof使用手册。

[图片]

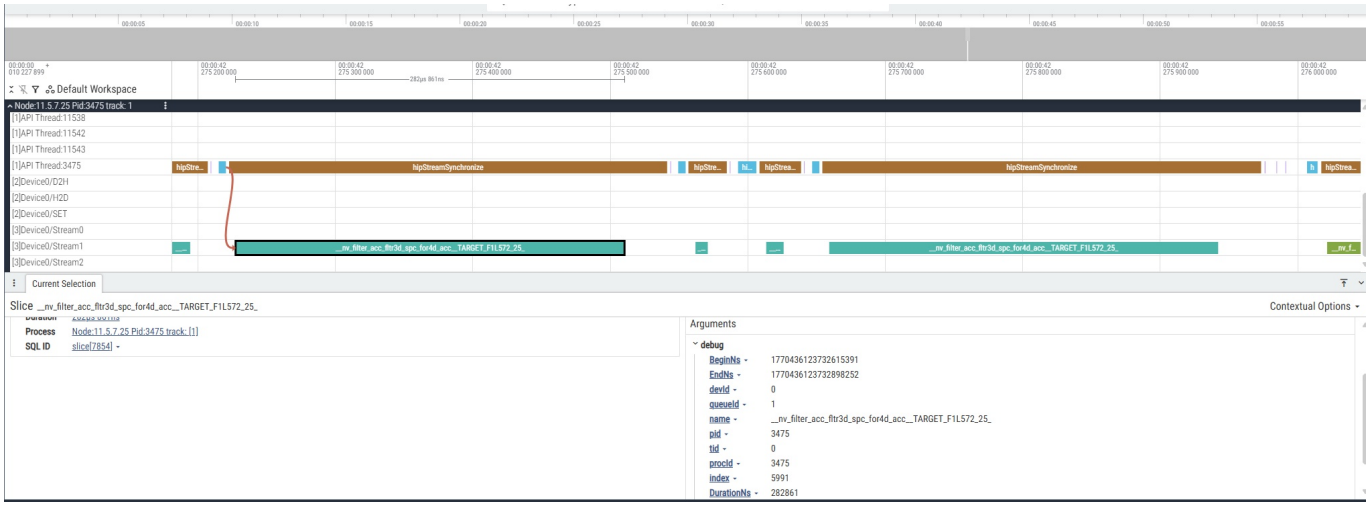


图1.11 使用hipprof统计异构程序性能

4.6.6 VMFault

对于Fortran，当核函数执行崩溃时，会自动打印出执行错误地址、核函数名称、核函数参数等信息，方便开发者进行进一步调试。

[图片]

```
Invalid address access: 0x2bb2580f9000, Error code: 1.
[GPU Memory Error] Addr: 0x2bb2580f9000 Reason: Page not present or supervisor privilege.
Target AMDGPU RTL -->
>>>>>>>> KERNEL VMFault !!!! <<<<<<

Target AMDGPU RTL --> >>>>>>>> PID: 32699 !!!! <<<<<<

Target AMDGPU RTL --> ================================================================================================
Target AMDGPU RTL --> =========> HOSTQUEUE <0x5580f5f8e9f0>: VMFault HSA QUEUE ANALYSIS <=========
Target AMDGPU RTL --> HOSTQUEUE <0x2ba4e9865000>: get hsa queue W/R ptr: write index: 2, read index: 1

Target AMDGPU RTL --> HOSTQUEUE <0x5580f5f8e9f0>: >>>>>>>> DUMP AQL 0x2ba4e9865000 PACKET WITH INDEX 1 <<<<<<<<<
Target AMDGPU RTL --> HOSTQUEUE <0x5580f5f8e9f0>: >>>>>>>> MATCH KERNEL COMMAND <<<<<<<<<
Target AMDGPU RTL --> HOSTQUEUE <0x5580f5f8e9f0>: header: 0x1402
Target AMDGPU RTL --> HOSTQUEUE <0x5580f5f8e9f0>: setup: 1
Target AMDGPU RTL --> HOSTQUEUE <0x5580f5f8e9f0>: workgroup: x:257, y:1, z:1
Target AMDGPU RTL --> HOSTQUEUE <0x5580f5f8e9f0>: grid: x:257, y:1, z:1
Target AMDGPU RTL --> HOSTQUEUE <0x5580f5f8e9f0>: group_segment_size: 9792
Target AMDGPU RTL --> HOSTQUEUE <0x5580f5f8e9f0>: private_segment_size: 16576
Target AMDGPU RTL --> HOSTQUEUE <0x5580f5f8e9f0>: completion_signal: 0x2ba4e98fef80
Target AMDGPU RTL --> HOSTQUEUE <0x5580f5f8e9f0>: kernel_object: 0x2ba4e99a0880
Target AMDGPU RTL --> HOSTQUEUE <0x5580f5f8e9f0>: device id: 0

Target AMDGPU RTL --> HOSTQUEUE <0x5580f5f8e9f0>: >>>>>>>> FIND MATCH KERNEL COMMAND <<<<<<<<<
Target AMDGPU RTL --> HOSTQUEUE <0x5580f5f8e9f0>: kernel name: __nv_vec_mult__TARGET_F1L25_1_
Target AMDGPU RTL --> HOSTQUEUE <0x5580f5f8e9f0>: >>>>>>>> DUMP KERNEL ARGS: size: 5 <<<<<<<<<
ARG 0, SCALAR, Arg_n_0.arg, value: 1024
ARG 1, SCALAR, Arg_i_1.arg, value: 0
ARG 2, ARRAY,  Arg_v1, ptr: 0x2ba907e00000, size: 4096, Invalid address: 0x2bb2580f9000, offset 40000000000
ARG 3, SCALAR, Arg_z_e_297_3.arg, value: 1024
ARG 4, ARRAY,  Arg_v2, ptr: 0x2ba907e01000, size: 4096, Invalid address: 0x2bb2580f9000, offset 39999995904

Target AMDGPU RTL --> HOSTQUEUE <0x5580f5f8e9f0>: >>>>>>>> DUMP KERNEL ARGS PTR INFO <<<<<<<<<



Target AMDGPU RTL --> ================================================================================================
Target AMDGPU RTL --> =========> HOSTQUEUE <0x5580f5f8ea20>: VMFault HSA QUEUE ANALYSIS <=========

Target AMDGPU RTL --> ================================================================================================
Target AMDGPU RTL --> =========> HOSTQUEUE <0x5580f5f8ea50>: VMFault HSA QUEUE ANALYSIS <=========

Target AMDGPU RTL --> ================================================================================================
Target AMDGPU RTL --> =========> HOSTQUEUE <0x5580f5f8ea80>: VMFault HSA QUEUE ANALYSIS <=========
Target AMDGPU RTL -->
>>>>>>>> KERNEL VMFault Analysis END !!!! <<<<<<
```

图1.12 VMFault信息

4.6.7 编译过程信息Minfo

编译代码时，添加-Minfo编译选项，则会打印出编译过程信息：

[图片]

图1.13 编译过程信息

能够打印出各函数名称，在某代码行数时添加OpenACC指令后生成了设备端代码、数据传输类型、数据传输的变量、生成核函数的模型等信息。

4.6.8 核函数计算完成后各数组求和

这个调试功能使用场景为应用添加openacc导语之后，通过打印每个openacc计算区域之后每个数组求和的值，进行主机端与设备端的求和值对比，可以定位到移植前后计算不精确或计算错误的核函数名称与变量名称。

4.6.8.1 不带数据类型的求和

根据数据的size，进行每4个字节的求和，最终结果没有具体意义，只表示该数据的二进制值，只能用于对比。只要数据为非标量，（数组、结构体、带指针的标量等），就会计算。适用于要求对比主机、设备完全一致的场景。

表 5.2 不带数据类型的数组求和

	设备端	主机端
文件	添加acc导语后的代码	添加acc导语后的，与设备端相代码
编译选项	-acc -g	-fopenacc-host-check -g
环境变量	LIBOMPTARGET_KERNEL_TRACE=1 OMPX_FORCE_SYNC_REGIONS=1 LIBOMPTARGET_INFO=128	LIBOMPTARGET_KERNEL_TRACE=1 OMPX_FORCE_SYNC_REGIONS=1 LIBOMPTARGET_INFO=128

注意事项	使用async子句的计算结构无法印出结果。 结构体的求和值可不准确。	结构体的求和值可能不准确。

将主机端与设备端的值作对比，如果值不同，则这个变量计算有差异。

Checksum精度比较高，如果有较小的误差，计算出的结果也会有差异。

4.6.8.2 带数据类型求和

根据数组的数据类型求和，计算出每个核函数结束后设备端数组的sum结果。对于浮点数类型，最终值会有一定的误差，适用于浮点数不完全一致的场景，若不一致，需用户自行判断是否正常。

只打印数组内部元素类型为int8/ int16/ int32/ int64/ float/ double的数组求和值。

表 5.2 带数据类型的数组求和

	设备端	主机端
文件	添加acc导语后的代码	添加acc导语后的，与设备端相代码
编译选项	-acc -g	-fopenacc-host-check -g
环境变量	LIBOMPTARGET_KERNEL_TRACE=1 OMPX_FORCE_SYNC_REGIONS=1 LIBOMPTARGET_INFO=256	LIBOMPTARGET_KERNEL_TRACE=1 OMPX_FORCE_SYNC_REGIONS=1 LIBOMPTARGET_INFO=256
注意事项	使用async子句的计算结构无法印出结果。	

设备端：

[图片]

```
PID: 30627, DEVID: 0 Stream: 0 SGN:2 ConstWGSize:256  args:11 gridDims:(7, 1, 1) blockDims:(256, 1, 1) reqd:(   0X   0) lds_usage:0B sgp
r_count:44 vgpr_count:57 sgpr_spill_count:37 vgpr_spill_count:72 tripcount:1600 rpc:0 n:__nv_rgradxy__TARGET_F1L17_1_
Libomptarget pid: 30627, device 0 info: Sum of rdenx, Type=float, Size=32000, Value=9600.000000
Libomptarget pid: 30627, device 0 info: Sum of rvsum, Type=int32, Size=32000, Value=35200
 rvsum        35200
 rdenx    9600.000
PID: 30627, DEVID: 0 Stream: 0 SGN:2 ConstWGSize:256  args:11 gridDims:(7, 1, 1) blockDims:(256, 1, 1) reqd:(   0X   0) lds_usage:0B sgp
r_count:44 vgpr_count:57 sgpr_spill_count:37 vgpr_spill_count:72 tripcount:1600 rpc:0 n:__nv_rgradxy__TARGET_F1L28_2_
Libomptarget pid: 30627, device 0 info: Sum of rvsum, Type=int32, Size=32000, Value=35200
Libomptarget pid: 30627, device 0 info: Sum of rdenx, Type=float, Size=32000, Value=9669.533203
 rdenx    9669.533
PID: 30627, DEVID: 0 Stream: 0 SGN:2 ConstWGSize:256  args: 4 gridDims:(1, 1, 1) blockDims:(256, 1, 1) reqd:(   0X   0) lds_usage:0B sgp
r_count:24 vgpr_count:32 sgpr_spill_count:26 vgpr_spill_count:36 tripcount:10 rpc:0 n:__nv_rgradxy__TARGET_F1L37_3_
Libomptarget pid: 30627, device 0 info: Sum of rdenx, Type=float, Size=32000, Value=9754.533203
 rdenx    9754.533
PID: 30627, DEVID: 0 Stream: 0 SGN:2 ConstWGSize:256  args:12 gridDims:(1, 1, 1) blockDims:(256, 1, 1) reqd:(   0X   0) lds_usage:0B sgp
r_count:38 vgpr_count:50 sgpr_spill_count:44 vgpr_spill_count:74 tripcount:80 rpc:0 n:__nv_rgradxy__TARGET_F1L42_4_
Libomptarget pid: 30627, device 0 info: Sum of rdenx, Type=float, Size=32000, Value=9754.533203
Libomptarget pid: 30627, device 0 info: Sum of drx, Type=double, Size=64000, Value=56043.695650100708
 rdenx    9754.533
 drx    56043.69565010071
PID: 30627, DEVID: 0 Stream: 0 SGN:2 ConstWGSize:256  args: 4 gridDims:(1, 1, 1) blockDims:(256, 1, 1) reqd:(   0X   0) lds_usage:0B sgp
r_count:24 vgpr_count:32 sgpr_spill_count:26 vgpr_spill_count:36 tripcount:10 rpc:0 n:__nv_rgradxy__TARGET_F1L54_5_
Libomptarget pid: 30627, device 0 info: Sum of rvsum, Type=int32, Size=32000, Value=35200
 rvsum        35200
```

图1.15 设备端调试日志

主机端：

[图片]

```
PID: 30903, Host args:14 n:__nv_rgradxy__TARGET_F1L17_1
Host pid: 30903 info: Sum of rdenx, Type=float, Size=32000, Value=9600.000000
Host pid: 30903 info: Sum of rvsum, Type=int32, Size=32000, Value=35200
 rvsum        35200
 rdenx    9600.000
PID: 30903, Host args:14 n:__nv_rgradxy__TARGET_F1L28_2
Host pid: 30903 info: Sum of rvsum, Type=int32, Size=32000, Value=35200
Host pid: 30903 info: Sum of rdenx, Type=float, Size=32000, Value=9669.533203
 rdenx    9669.533
PID: 30903, Host args: 5 n:__nv_rgradxy__TARGET_F1L37_3
Host pid: 30903 info: Sum of rdenx, Type=float, Size=32000, Value=9754.533203
 rdenx    9754.533
PID: 30903, Host args:15 n:__nv_rgradxy__TARGET_F1L42_4
Host pid: 30903 info: Sum of rdenx, Type=float, Size=32000, Value=9754.533203
Host pid: 30903 info: Sum of drx, Type=double, Size=64000, Value=56043.695650100708
 rdenx    9754.533
 drx    56043.69565010071
PID: 30903, Host args: 5 n:__nv_rgradxy__TARGET_F1L54_5
Host pid: 30903 info: Sum of rvsum, Type=int32, Size=32000, Value=35200
 rvsum        35200
```

图1.16 主机端调试日志

若没有主机端结果，也可以观察求和值是否出现了异常来定位问题。过长的数据，或者出现nan值，就是一个异常数据。

4.6.8.3 注意事项

设备端计算涉及到D2H操作较多，所以性能会变差很多。

如果日志打印过多，不好对比，可以写个脚本进行对比。

如果日志打印过多，想要筛选对应变量，可以使用环境变量export OMPX_INFO_SUM_FILTER=‘sdown$p:zq$p:dtsfc$p:dqsfc$p:grid’

4.7 性能优化思路

4.7.1 创建核函数

对于连续的多层循环，使用collapse子句子句是最优的性能优化方案，collapse(2)会将两层连续循环合并为一个扁平化的单层循环空间，由编译器自动分配到设备的计算单元中，最大化利用硬件并行资源，减少循环调度开销，因此能实现最佳的并行执行效率。

```
!$acc parallel loop collapse(2)do i = 1, M    do j = 1, N       A(i,j) = 10    enddoenddo
```

而对于循环逻辑不连续（如外层循环包含独立计算逻辑）的多层循环，需采用gang/vector嵌套的方式：外层gang子句将循环分配到不同的计算集群，内层vector子句实现细粒度并行。该方式性能略逊于collapse，且需注意vector维度的并行度默认256、最大1024，若内层循环次数远超该阈值，会引发分块调度开销，反而导致性能下降。

```
!$acc parallel loop gangdo i = 1, M    B(i) = 0    !$acc loop vector    do j = 1, N       A(i, j) = 10    enddoenddo
```

4.7.2 合理使用数据驻留指令

使用declare指令，其核心作用是将指定数据永久性创建在设备端，且程序运行周期内不会自动释放该数据资源。要求开发者必须通过显式的update指令，完成主机与设备之间的数据同步传输，可灵活控制数据更新时机，避免无意义的重复传输。

使用data/ enter data/ exit data指令，能够实现对设备端数据生命周期的精细化手动管控：enter data负责将主机数据迁移并创建到设备端，exit data则显式释放设备端的数据资源，适配不同计算阶段的内存需求。

若待传输的数据长度过长，需合理运用数组切片、分块传输策略，仅将当前计算所需的局部数据片段传输至设备端，大幅减少单次数据传输量，降低带宽占用，提升整体数据传输效率。

如果不使用上述指令，核函数启动时会触发隐式的数据管理机制：自动完成设备端数据创建、主机与设备的数据传输、计算结束后释放设备数据，但该过程会随核函数的每次调用重复执行，频繁的内存申请与释放会产生大量额外开销，导致程序整体性能显著下降。

4.7.3 使用异步多流

OpenMP与OpenACC的核心特性决定了程序运行时默认采用同步执行模式：设备端核函数执行期间，主机端代码会处于阻塞等待状态，直至设备端任务完全结束，这导致主机与设备的计算资源无法并行利用，整体执行效率受限。

[图片]

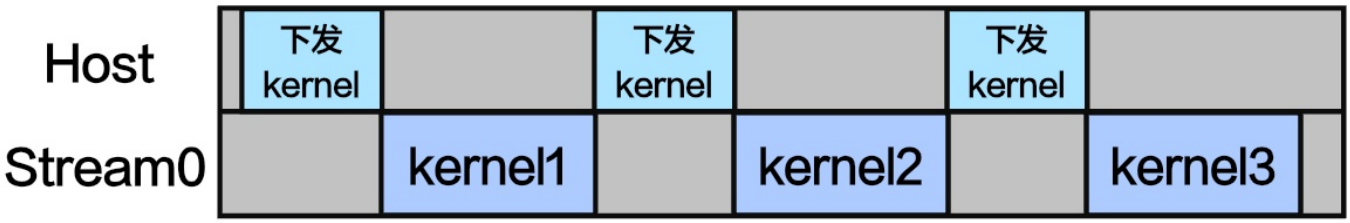


图1.17 同步执行模型

在核函数中添加async子句可将其配置为异步执行模式，且默认绑定到设备的默认流上，此时主机端无需等待设备端任务完成即可继续执行后续代码，主机与设备的计算逻辑并行推进，能显著提升程序整体性能。

Fortran环境下的OpenACC还提供了-facc-default-kernel-async编译选项，可一键将文件内所有核函数批量设置为异步执行，无需逐行修改代码，简化异步化配置流程。

[图片]

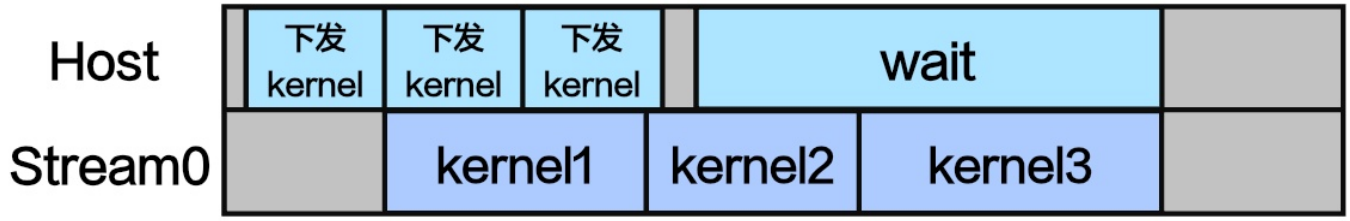


图1.18 默认流异步模型

若进一步使用async(var)子句手动指定不同的流标识，可让多个核函数分散执行在独立的设备流中，实现设备端任务的并行调度，性能提升更明显；但需严格确保多流核函数操作的变量无数据依赖关系，否则会引发计算结果错误或程序崩溃。

[图片]

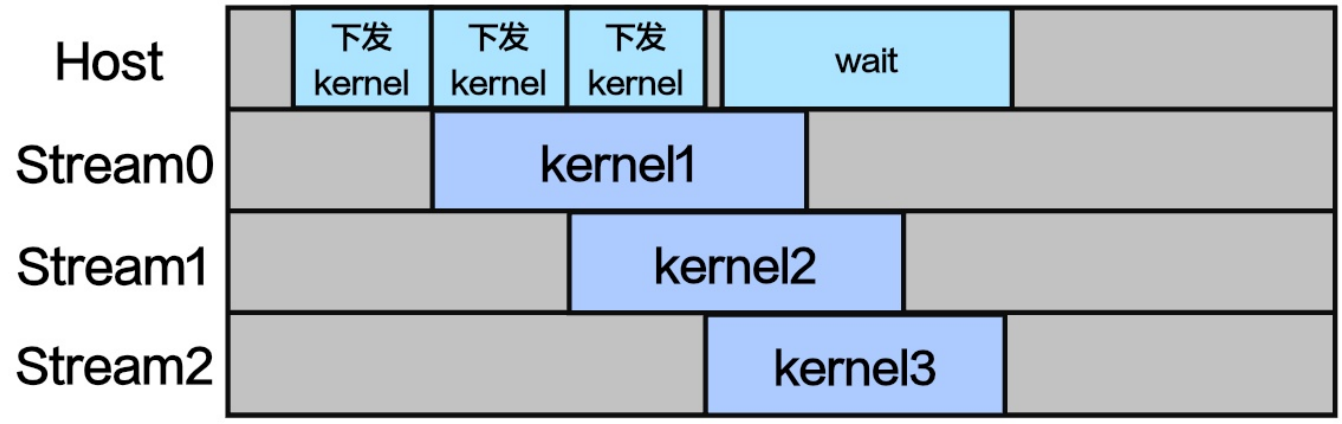


图1.19 多流异步模型

4.7.4 使用多卡、多进程

当待处理的数据量单张加速卡的显存容量无法容纳全部计算数据时，基于MPI的多进程分布式计算是最优

解决方案：通过MPI启动多个独立进程，并借助进程绑定技术将每个进程专属绑定到一张加速卡上，让每张卡仅负责处理分配给自己的数据集，充分利用多卡的显存资源和计算能力，避免单卡显存溢出或算力瓶颈。

具体实施时，需先按照数据的维度特征将整体数据集拆分为多份大小均衡的子数据集，确保每个进程分配到的数据量与对应加速卡的显存、算力相匹配，避免出现部分进程负载过高、部分闲置的情况。

数据拆分后，各进程可独立在绑定的加速卡上完成本地计算任务；当进程间需要共享中间结果或合并最终数据时，需调用标准MPI通信接口（如MPI_Send、MPI_Recv、MPI_Allgather等），或高性能通信库接口（如DUSHEMEM、RCCL）完成进程间的数据交换与同步，既保证计算的并行性，又能满足数据依赖场景下的通信需求。

[图片]

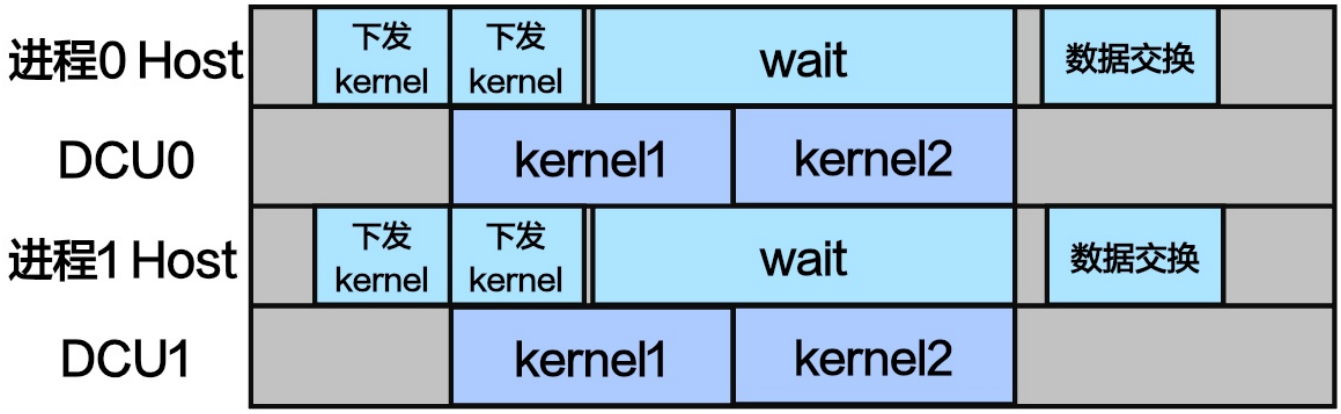


图1.20 多卡异步模型

4.7.5 acc routine并行函数

在OpenACC的routine指令中，可灵活搭配vector、gang等并行化子句，为设备端函数指定并行执行模式；当核函数需要调用其他设备端函数时，无需局限于串行执行，可根据函数内计算逻辑的并行潜力，对其内部循环做针对性并行化优化。

以如下示例代码为例，设备端函数func1通过!$acc routine
vector声明为向量级并行例程，并在内部循环前添加!$acc loop
vector子句，让循环迭代在设备的vector单元上并行执行；主调用函数SFCL同样声明为routine
vector，外层循环则用!$acc parallel loop
gang指定gang级并行，设备端能同时利用gang和vector维度的并行资源，最大化函数内循环的计算效率
。

```
 FUNCTION func1(I)  !$acc routine vector     ! 声明例程为设备端向量级并行  integer :: i
integer,dimension(1:10) :: psim  !$acc loop vector      ! 对函数内循环做vector并行化  DO I = 1,10
psim(I) = 10  ENDDO  func1 = psim(I)  return  END FUNCTION func1  SUBROUTINE SFCL ()  !$acc
routine vector     ! 声明子例程为设备端向量级并行  REAL,dimension(1:10) :: buf_1  integer :: i  !$acc
parallel loop gang  ! 外层循环按gang维度并行  DO I=1,10    buf_1(I)= func1(I)  ENDDO  END
SUBROUTINE
```

4.7.6 规约

HIP代码中的规约操作完全依赖开发者手动实现完整的规约逻辑，从线程块内的数据聚合、块间结果合并，到线程、块级别的同步控制（如__syncthreads()指令），均需显式编码完成；若要突破基础规约的性能瓶颈，还需掌握shuffle指令（线程束内数据交换）、共享内存分块优化、多级规约拆分等进阶技巧，但这类技巧涉及硬件架构的底层细节，对开发者的并行编程功底和硬件认知要求极高，编程门槛显著高于高层抽象框架。

而OpenACC与OpenMP作为高层并行编程模型，其规约功能仅需在循环或并行区域指令后添加reduction子句，即可完成规约声明：编译器会自动根据目标硬件的架构特性，适配最优的规约算法，包括自动分配共享内存存储临时聚合结果、优化线程的同步时机、拆分超大规模规约为多级聚合流程等，无需开发者关注底层实现细节；通过简单的子句声明，也能让规约操作达到接近手动优化的高性能，大幅降低并行规约的开发成本与学习门槛。

如下规约代码：

```
 !$acc parallel loop gang   do j = 1, m    !$acc loop vector reduction(max:gpu_max) reduction(+:gpu_sum)
 do i = 1, n      gpu_sum = gpu_sum + a(i,j)      gpu_max = max(gpu_max, a(i,j))    enddo    gpu_sum_col(j) =
gpu_sum    gpu_max_col(j) = gpu_max  enddo
```

4.7.7 优化访存顺序

C、C++
语言遵循行优先（Row-major）的数组访存规则，即数组元素会按行连续存储在内存中；而Fortran语言则采用列优先（Column-major）规则，数组元素按列连续存储。

这种底层存储方式的差异直接决定了循环嵌套的最优顺序。若循环遍历方向与语言的访存顺序相悖，会引发大量非连续的内存访问（跨步访存），导致CPU/GPU缓存命中率急剧下降，实测中性能差距可达10倍以上，甚至在大规模数组计算中差距会进一步扩大。

例如如下Fortran代码，其访存顺序不合理，导致执行性能较差：

```
!$acc parallel loop collapse(3)
 do i = 1,N
   do j = 1,N
     do k = 1,N
       ut(i,j,k)=ut(i,j,k) +1
     enddo
   enddo
 enddo
```

针对Fortran的列优先特性，Fortran编译器提供了-freorder-collapse编译优化选项。开启该选项后，编译器会自动扫描核函数内数组下标的访问模式，智能调整collapse子句覆盖的多层循环的嵌套顺序，无需开发者手动调整循环层级，即可让内存访存贴合列优先规则，最大化缓存利用率，从而使collapse并行后的循环达到最优执行性能。即便开发者未按列优先规则编写循环，该选项也能通过编译期优化修正访存路径，规避因循环顺序不当导致的性能损耗。

第 5 章 使用 DCC编译DCU程序

5.1 DCC 概述与定位

5.1.1 DCC 总体介绍

DCC 是针对 DCU 异构硬件体系（包含 CPU 与 GPU）的编译器工具链，作为 DTK（DCU Tool Kit）开发工具链的核心组成部分。DCC 基于 LLVM 编译基础设施构建，继承了 LLVM 成熟的三段式编译器架构——前端、中间表示与优化器、后端——并在此之上，面向 DCU 异构硬件做了系统级扩展：扩充了异构语言前端（HIP、CUDA、OpenMP、OpenACC、SYCL、OpenCL）、引入了 GCVM 作为 AI 编译器的接入层、增加了针对 DCU 执行模型的专项优化 Pass，以及定制了面向 GPU 目标的 Lingshu 后端，以适配已发布的多款 DCU 型号。

DCC 通过统一的编译驱动，覆盖从传统高性能计算到现代异构并行的多种语言与编程模型。C/C++ 通过 Clang 前端接入，适用于通用程序、运行时库与主机侧控制逻辑；Fortran 通过 Flang 前端接入，服务于科学计算与数值仿真等 HPC 场景；HIP 与 CUDA 入口更适合显式异构编程，可在同一源文件中同时描述主机逻辑与设备端 Kernel，并由 DCC 自动完成 Host/Device 代码拆分、设备端代码生成与打包；OpenMP 与 OpenACC 主要面向指令式卸载场景，允许开发者通过编译指示逐步把热点区域迁移到 DCU；SYCL 与 OpenCL 则提供另一类面向异构设备的单源或设备编程模型入口。它们虽然语法风格、工程组织方式和迁移路径不同，但最终都共享同一条优化与代码生成主干，使开发者可以在统一的编译体系下，面向不同 DCU 型号编写、编译并运行高效代码，从而提高代码的可移植性与可维护性。

随着 AI 编译技术的快速发展，越来越多的框架和编译器选择在前端优化之后，将计算图或算子下沉为 GCVM IR 或相关中间表示，再交由目标后端完成机器码生成。DCC 通过 GCVM（General Compiler Virtual Machine）承接这一路径，为 Triton、JAX、TensorFlow XLA、PyTorch Inductor 等主流 AI 编译器提供统一的后端接入能力。GCVM 的意义在于：AI 编译器生态种类繁多、演进迅速，各自拥有不同的前端表示与优化策略，但它们在进入硬件代码生成阶段时，都需要一个稳定、高效且与硬件深度适配的后端。GCVM 正是充当这一角色——屏蔽上层编译器的差异，将来自不同 AI 框架的中间表示统一引入 DCC 的优化与代码生成流程，最终生成面向 DCU 硬件的高质量设备端代码。这种设计使 DCC 不仅服务于传统 HPC 开发者，也成为 AI 框架在 DCU 硬件上落地的关键基础设施。

GCVM IR 是 GCVM 所采用的核心中间表示，可将不同前端编译器或 AI 编译器下沉后的程序，统一表达为一种与具体硬件解耦、但又便于后端继续优化和代码生成的 IR 形式。从定位上看，它并不是新的高级编程语言，而是位于“前端语义已经明确、目标机器码尚未生成”之间的编译接口层。对 DCC 而言，这一层的价值在于把来自不同来源的输入收敛到同一条后端流水线上，使后续优化、指令选择、寄存器分配和设备端封装都能够基于统一语义展开。

GCVM IR 基于 LLVM IR 17.0.0，并在此基础上加入了面向 DCU 目标和异构编译场景的扩展能力。因此，GCVM IR 在整体结构上继承了 LLVM IR 的模块、类型、常量、函数、指令、元数据等基本组织方式，便于复用成熟的 LLVM 分析与优化基础设施；与此同时，它又通过目标相关地址空间、内建函数与目标支持约定，把 DCU

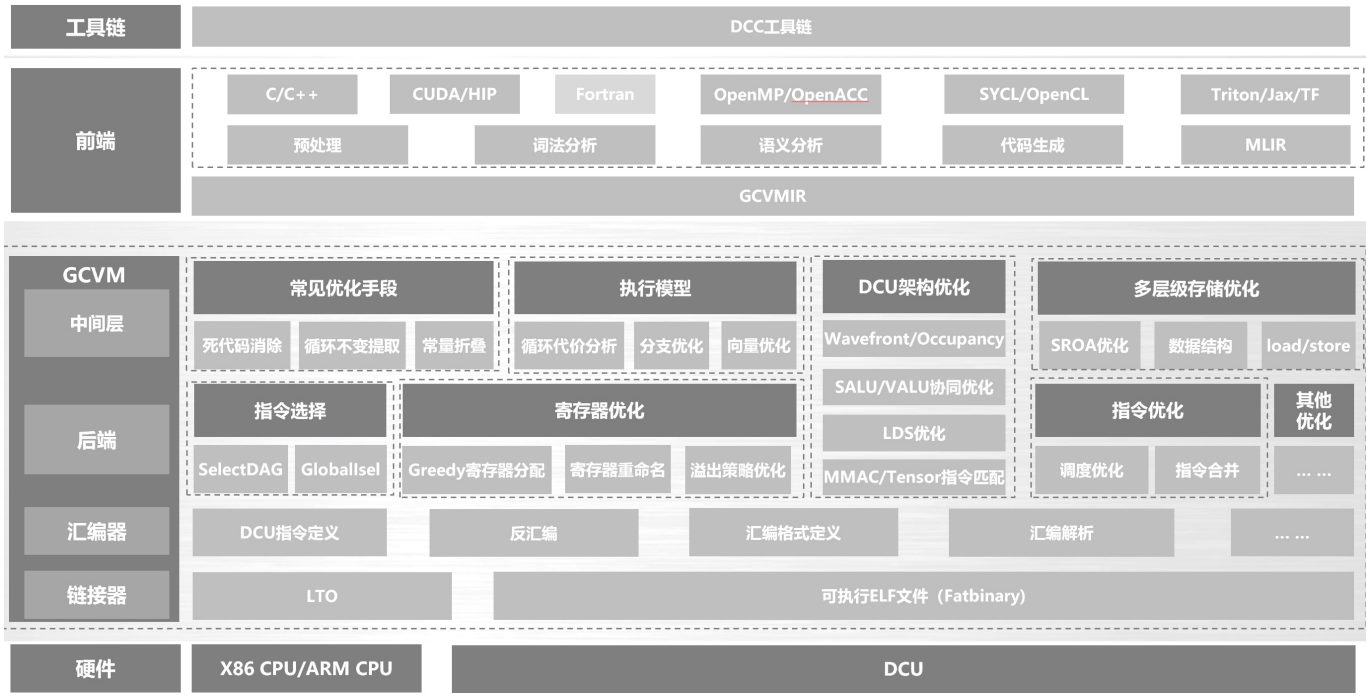
设备执行所需的信息显式保留下来。换句话说，GCVM IR 的设计目标不是脱离 LLVM 另起炉灶，而是在 LLVM IR 的通用表达能力之上，为 DCU 后端和 AI 编译接入补齐必要的目标语义。

从生态建设角度看，GCVM IR 还必须认真处理兼容性问题。参考 NVVM IR：它同样建立在 LLVM IR 之上，但长期保持相对稳定的设备侧语义边界，使上游 CUDA 编译生态能够围绕这一接口持续演进，而不必随着后端实现细节频繁调整。对 GCVM IR 而言，这意味着一方面要跟进 LLVM IR 的主线演进，适时吸收通用能力；另一方面又要尽量保持 DCU 相关扩展、地址空间约定、intrinsics 接口和目标属性描述的稳定性，避免因为 IR 频繁破坏性变化而增加 Triton、JAX、XLA、PyTorch Inductor 等上游系统的适配成本。只有把“可扩展”和“可兼容”同时控制好，GCVM IR 才能真正成为长期可用的编译接口，而不是阶段性的内部实现细节。

从编译流程上理解，GCVM IR 主要承担三类职责。第一，承接上游输入，把 Triton、JAX、TensorFlow XLA、PyTorch Inductor 等系统下沉后的表示，转换为 DCC 后端可以稳定消费的统一 IR。第二，作为优化载体，在进入目标代码生成之前，保留足够完整的数据流、控制流和内存语义，使 LLVM 通用优化与 DCU 专项优化都能够在这一层开展。第三，作为目标桥接层，通过地址空间模型、DCU 相关 intrinsics 和目标架构约定，为后续 Lingshu 后端生成面向 GFX906、GFX926、GFX928、GFX936、GFX938 等 DCU 目标的代码提供直接依据。

这也解释了 GCVM IR 在 DCC 架构中的实际意义：它一端连接多样化、快速演进的上游 AI 编译生态，另一端连接相对稳定、需要深度硬件适配的后端代码生成体系。借助这一中间层，DCC 可以把“前端多样性”和“后端统一性”同时保留下来。对开发者而言，这意味着无论输入来自传统语言前端还是 AI 编译器下沉 IR，只要进入 GCVM/GCVM IR 这一接口层，就能够复用 DCC 已有的优化能力、DCU 目标支持和最终产物封装流程。

[图片]



上图展示了 DCC 在 DTK 中的整体架构：上层是多来源输入（C/C++、Fortran、HIP、CUDA、OpenMP、OpenACC、SYCL、OpenCL、AI 编译器下沉 IR），中间是统一编译管线（Driver、前端、IR 优化、后端、链接与打包），底层是运行时与硬件执行环境。若按“输入程序 → 编译处理 →

运行落地”来理解，可以将其拆解为四个连续层次。

第一层是前端接入层。该层的核心职责是“理解不同来源的程序语义”。C/C++ 由 Clang 前端处理，Fortran 由 Flang 前端处理；HIP、CUDA、SYCL 与 OpenCL 负责承接显式设备编程或单源异构编程语义；OpenMP 与 OpenACC 负责识别 target、并行区域与数据映射等指令式卸载语义；AI 场景下则通过 GCVM 接入上游框架下沉后的中间表示。无论入口如何变化，前端层都会把输入统一降低为 GCVM IR 或相关可优化表示，为后续处理建立统一语义基础。

第二层是中间表示与优化层。该层负责把“可编译”推进到“可高效执行”。一方面执行 LLVM 通用优化（如 DCE、LICM、SROA、CSE、循环展开与向量化）；另一方面在 IR 阶段提前做一部分面向 DCU 执行模型的专项优化（如 Wavefront/Occupancy 平衡、SALU/VALU 协同、LDS 访存模式优化、矩阵模式向 MMAC/Tensor 指令映射）。需要说明的是，DCU 执行模型相关优化并不只发生在这一层，后端阶段仍会继续结合目标 ISA 与硬件资源进行细化优化。这一层本质上是性能上限的主要决定因素。

第三层是目标代码生成层（后端层）。该层负责将优化后的中间表示降低为目标 ISA 机器代码，关键步骤包括指令选择（SelectDAG/GlobalISel）、寄存器分配（如 Greedy 策略）和机器级调度优化。与 IR 层配合，后端还会继续执行与 DCU 执行模型强相关的目标优化，例如寄存器压力控制、指令调度与访存/发射平衡等。需要强调的是：DCC 既可以编译 CPU 程序，也可以编译 DCU 程序。对于纯 CPU 程序，后端仅生成主机侧目标代码；对于 DCU 程序，后端会并行生成主机侧控制代码与设备侧 Kernel 代码。

第四层是汇编、链接与封装层。该层负责把代码生成结果组织成可交付产物，并完成跨模块链接与必要优化（如 LTO）。在产物形态上：CPU 程序通常生成主机端对象文件或可执行文件；DCU 程序的编译结果包含两部分——主机端产物（负责设备初始化、Kernel 启动与同步）和设备端产物（HSACO/Code Object，可按需组织为 Fatbinary）。这一层决定了“编译结果如何部署与复用”。

从工程视角看，这四层架构的价值在于职责边界清晰、可演进性强：新增前端或新模型时，不必重写整个后端；新增硬件型号时，可复用既有前端与优化框架，仅在目标相关层做扩展。这也是 DCC 能同时服务传统 HPC、异构并行与 AI 编译接入场景的根本原因。

5.1.1 DCC 在 DTK 工具链中的角色

DCC 是 DTK（DCU Tool Kit）中的核心编译组件，承担“从源码或中间表示到目标二进制”的关键职责。对终端开发者而言，DCC 是统一命令入口；对工具链架构而言，DCC 是把前端、优化器、后端、链接、运行时与调优工具串联起来的中枢控制层。

在 DCU 异构计算场景下，一个工程通常同时包含两类代码：主机侧控制逻辑（CPU 执行）与设备侧并行计算逻辑（DCU 执行）。如果编译入口分散在多个工具中，工程会快速出现“参数体系不一致、目标架构不一致、产物组织不一致、调试链路不一致”的问题。DCC 的价值正是把这些异构构建动作收敛成一条统一、可追踪、可复用的编译流水线。

从 DTK 全局看，DCC 的角色可以概括为四层：

第一，统一入口层。 DCC 对上承接
Clang、Flang、HIP、CUDA、OpenMP、OpenACC、SYCL、OpenCL 与 GCVM
输入，对下屏蔽汇编、链接、设备端封装差异。开发者只需维护一套稳定命令接口，即可覆盖 CPU
程序、DCU 程序以及 AI 编译下沉程序。

第二，编排控制层。 DCC 不只是“调用编译器”，而是负责完整流程编排：识别输入类型、拆分
Host/Device 路径、分发多目标架构编译、注入运行时依赖、组织最终产物。尤其在
HIP、CUDA、OpenMP、OpenACC、SYCL、OpenCL
等异构场景中，这种编排能力决定了构建过程是否稳定可重复。

第三，优化落地层。 DCC 继承 LLVM 通用优化能力，并把 DCU 执行模型相关优化贯穿在 IR
与后端两个阶段。前者负责高层语义重写与模式规整，后者负责目标 ISA
相关优化、寄存器与调度细化，最终把“可运行”提升为“高效率运行”。

第四，生态枢纽层。 在 DTK 体系中，DCC
是后续分析和调优工具的上游源头。dccobjdump、dccturing、dcc-autotune、dccprof 等工具都依赖
DCC 产物作为输入。若编译阶段缺乏一致性，后续调优链路就难以形成闭环。

DCC
在工程生命周期中承担的是“从开发到交付”的连续职责。在开发阶段，提供统一命令与一致参数模型
，降低多语言/多模型协作成本。在构建阶段，保障 Host/Device
双通道构建的可重复性与可观测性。在发布阶段，支持单架构与多架构（Fatbin）产物组织。在运维与
性能阶段，为反汇编分析、参数搜索、性能归因提供稳定输入基线。

5.1.2 DCC 支持的语言与编程模型

DCC
的一个核心特点，在于它并不服务于单一语言或单一开发模式，而是试图把传统通用编程、高性能计算
、异构并行开发以及 AI
编译前端接入，统一到同一条代码生成链路中。从工程结果上看，这意味着不同来源的程序最终可以共
享同一套优化框架、同一套目标代码生成能力以及同一套产物组织方式；从开发体验上看，这意味着团
队不必因为语言或模型不同，就为每一类程序维护完全割裂的工具链。

在传统语言层面，DCC 首先支持 C/C++ 与 Fortran。C/C++ 通过 Clang
前端接入，是最通用、也是覆盖面最广的使用方式，既适用于业务逻辑、运行时组件和宿主侧控制代码
，也适用于需要较强工程抽象能力的基础库开发。Fortran 则主要面向科学计算和数值仿真场景。大量
HPC 历史代码长期积累在 Fortran 生态中，因此 DCC 对 Fortran
的支持，实际上决定了它能否进入真实的大型科研与工业计算环境。对这两类语言而言，DCC
的意义并不只是“能编译”，而是让它们能够继续进入统一的
IR、优化器和目标后端，从而与后续的异构执行能力衔接起来。

在编程模型层面，DCC 已经形成了较完整的异构前端支持面，覆盖
HIP、CUDA、OpenMP、OpenACC、SYCL 和 OpenCL。HIP 与 CUDA
更偏向显式异构编程，适合手写高性能
Kernel、算子开发以及需要精确控制线程组织、访存行为和设备端执行路径的场景。开发者可以在单一
源文件中同时描述主机侧与设备侧逻辑，而 DCC 负责将其拆分为 Host/Device
双路径，分别完成设备端代码生成与主机端控制逻辑构建。相比之下，OpenMP 与 OpenACC
更强调渐进迁移，适合在已有 CPU 并行程序基础上，通过

target、循环并行指示和数据映射语义逐步把热点计算区域卸载到 DCU。SYCL 与 OpenCL 则提供另一类面向异构设备的编程入口，更强调单源抽象、设备对象模型或显式设备接口。也正因为如此，这几类模型在 DCC 中并不是彼此替代的关系，而是分别面向“高控制力新开发”“低迁移成本旧工程演进”以及“跨设备抽象编程”这几类不同需求。

除了直接使用 dcc 作为统一编译驱动之外，DCC 还兼容 hipcc 与 nvcc 等常见异构编译入口。这一能力的重点不在于简单复刻命令名称，而在于尽量承接已有 HIP/CUDA 工程的构建习惯、参数组织方式与接口认知，降低迁移到 DCU 平台时对构建系统的大规模改造需求。对已有项目而言，这种兼容性意味着开发者可以在保留部分既有脚本与工程结构的前提下，逐步把底层编译和代码生成流程收敛到 DCC 体系中，从而缩短迁移路径并减少工程切换成本。

如果进一步从软件工程角度看，DCC 对编程模型的支持并不只停留在语法识别层。无论是 HIP/CUDA 中的 Kernel 函数、设备端内建变量、Host/Device 分离与启动语义，还是 OpenMP/OpenACC 中的数据映射、区域卸载、双端编译与运行时衔接，抑或 SYCL/OpenCL 中的设备对象、Kernel 提交和地址空间语义，最终都会进入统一的优化与代码生成流程。这意味着开发者虽然在源代码层面选择了不同模型，但在更底层的编译基础设施上，DCC 仍然尽量保持一致的行为边界和调优方式。这也是它能够在多模型共存的工程中保持稳定性的关键原因。

在 AI 编译器接入方面，DCC 的定位更偏向“统一后端承接者”。随着 AI 编译技术的发展，越来越多的系统不再直接从高级框架落到硬件代码，而是先经过图级优化、算子划分、调度变换和中间表示下沉，再把结果交给目标后端完成最终机器码生成。在这一路径中，DCC 通过 GCVM 承接来自不同 AI 编译器的输入，使 Triton、JAX、TensorFlow XLA、PyTorch Inductor 等上层系统可以复用 DCC 已有的目标代码生成与硬件适配能力。换句话说，DCC 在 AI 生态中的价值，不是替代这些上层编译器，而是为它们提供一个统一、稳定、与 DCU 硬件深度适配的后端落地点。

这种支持方式还有一个重要意义：AI 编译器生态变化很快，上层框架和 DSL 经常演进，如果每个系统都各自维护完整后端，成本极高且难以长期保持与硬件同步。而通过 GCVM 接入 DCC，前端差异可以保留在各自生态内部，后端的目标相关优化、代码生成、产物封装与运行时衔接则尽量收敛到统一基础设施中。也正因为如此，DCC 同时服务于传统 HPC 开发者、异构应用开发者以及 AI 框架后端适配需求，形成了一个覆盖面较广、但底层能力相对统一的编译生态承接面。

5.1.3 DCC 工具链介绍

从使用视角看，开发者最先接触的是 dcc 命令；但从工程视角看，DCC 并不是单一工具，而是一套覆盖“编译、观察、分析、调优”的协同工具链。dcc 本身负责把 C/C++、Fortran、HIP、CUDA、OpenMP、OpenACC、SYCL、OpenCL 和 GCVM IR 等不同输入组织成统一编译流程，自动完成前端处理、Host/Device 路径拆分、优化流水线编排、目标代码生成、链接与封装。对于 CPU 程序，它生成主机侧对象文件或可执行文件；对于 DCU 程序，它同时产出主机端控制代码与设备端 HSACO/Code Object，并可按发布需要组织为 Fatbinary。也就是说，dcc 解决的是“如何稳定地产生可运行产物”。

在拿到产物之后，dccobjdump

用于回答“编译器到底生成了什么”。它可以从符号、段信息、指令分布到设备端 code object 结构等多个维度解析编译结果，帮助开发者确认设备端代码是否正确打包、关键 Kernel 的静态指令形态是否符合预期，以及不同编译选项下产物差异是否合理。由于这些检查都发生在运行前，dccobjdump 往往是性能分析之前最重要的静态质量入口。

当问题深入到机器指令层，dccturing 的价值就会显现。它主要服务于底层性能攻关与后端验证：在不重构高层源码的前提下，对局部指令序列、调度策略或编码细节做实验性处理，用更低成本验证“某种底层实现是否更优”。因此，dccturing 不是日常开发必备工具，而更适合专家级调优或专项攻关场景。

如果目标是系统性寻找更优编译配置，dcc-autotune 可以把手工试错转化为可重复流程。它通过批量生成候选参数、自动编译运行并收集指标，完成参数搜索与结果筛选，适用于优化等级、目标参数和编译/链接策略的组合探索。对团队而言，dcc-autotune 的意义不仅在于节省时间，更在于把调参结论沉淀为可复验、可共享的工程资产。

与静态分析对应，dccprof 聚焦运行时行为，回答“为什么慢、慢在哪里”。它能够定位热点 Kernel、观察执行时间占比和资源瓶颈，验证参数调整是否真的带来收益，也能识别“静态上看合理、动态上却效率不高”的问题。实践中，dccprof 通常与 dccobjdump 配合使用：前者给出动态证据，后者提供静态结构，两者共同构成完整诊断视角。

从整体方法上看，DCC 工具链更适合按闭环使用：先用 dcc 构建产物，再用 dccobjdump 做静态检查，用 dccprof 做动态定位，随后借助 dcc-autotune 做参数搜索，必要时再通过 dccturing 做汇编级实验。通过这种“构建—观测—调优—验证”的循环，性能优化可以持续推进，并保持可重复、可追踪，而不依赖一次性的个人经验。

5.2 安装与环境准备

DCC软件包可以从光合开发者社区下载，在打开的页面中找到需要的 DCC 版本，(https://download.sourcefind.cn:65024/1/main/compiler)，选择对应的操作系统，即可下载。

压缩包命名为 DTK-版本号-DCC版本号-操作系统版本-x86_64.tar.gz，解压后，进入dcc目录，目录结构如下：

```
├—bin
├—— comgr
├—— dccgcn
├—— gcvm
├—— include
├—— lib
├—— pch
├—— share
├—— syclRT
```

通过 source根目录下的 env.sh (对于 bash) 或 env.csh (对于 csh),即可加载 DCC 的环境。

安装后建议执行版本验证：

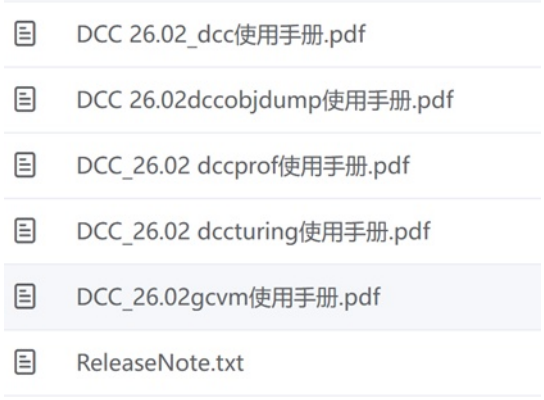
```
dcc --version
```

```
dcc --short-version
```

若版本信息输出正常，即可进入编译验证阶段。若命令执行异常，请优先检查：环境脚本是否生效,LD_LIBRARY_PATH 是否完整,DTK 软链接是否指向正确版本。

DCC
提供了一系列使用文档供开发者参考,你可以在(https://download.sourcefind.cn:65024/1/main/compiler)中 DCC 的下载页面里选择 DCC 版本,并找到document 目录，以 DCC_26.02为例，打开后可以看到如图所示的文档。

[图片]



5.3 快速上手：从源码到可执行

本节以 C/C++、HIP 和 OpenMP Offload 作为代表性示例展开说明。对
CUDA、OpenACC、SYCL、OpenCL 等其他异构前端入口而言，虽然源码形态和接口风格不同，但在 DCC 中同样会汇入统一的 Host/Device 编排、目标代码生成与产物封装流程。

5.3.1 C/C++ 基本编译

```
dcc -O3 main.cpp -o app
```

这条命令看起来很简单，但它实际上串联了完整的编译流水线：首先由前端把源代码解析为 GCVM IR；随后在中间层施加 -O3
级别的通用优化；再由后端生成主机侧机器代码；最后完成系统库与工具链依赖的链接，产出最终可执行文件。

对于刚接触 DCC 的用户，建议先从这类纯 C/C++
示例入手，因为它可以帮助用户确认两件事：第一，DCC 的基础驱动工作正常；第二，构建系统与 DCC 的整合方式清晰可控。

常用变体如下：

```
# 仅编译不链接
dcc -O2 -c main.cpp -o main.o

# 调试版本
```

```
dcc -O0 -g main.cpp -o app_dbg

# 保留中间产物
dcc -O3 --save-temps main.cpp -o app
```

其中，-c 适合大型工程的分步编译；-g 适合调试阶段；–save-temps
则适合需要观察中间文件或分析编译行为的场景。书写工程文档或排查构建问题时，后两种模式通常比直接生成最终产物更有信息量。

5.3.2 HIP 程序编译

```
dcc -O3 --offload-arch=gfx926 hip_demo.cpp -o hip_demo
```

与纯主机程序不同，HIP 程序的编译本质上是一条“双通道”路径。DCC
在处理这类输入时，会把同一源文件中的主机代码与设备端 Kernel
代码区分开来：设备端部分被编译为面向指定架构的 HSACO
或相关设备代码对象，主机端部分则生成负责设备初始化、参数传递、Kernel
启动与同步控制的可执行逻辑，最终再把两者组织为统一的程序产物。

命令中的 –offload-arch=gfx926
非常关键，它决定了设备端代码所针对的具体硬件架构。如果该参数与实际运行硬件不匹配，问题可能表现为编译通过但运行异常，或者性能明显偏离预期。因此，在 HIP
工程中，目标架构配置应当被视为与编译优化等级同等重要的参数。

实践中，推荐采用“先基线、后优化”的方式上手：先用 -O2 建立功能正确的基线程序，再切换到 -O3 比较性能收益，最后针对热点 Kernel 结合资源占用和运行期数据做增量调优。这样做的原因在于，HIP 程序的性能问题往往并不只与前端写法有关，还与寄存器压力、访存模式、线程组织和设备端代码生成策略共同相关。

5.3.3 OpenMP Offload 编译

```
dcc -O3 -fopenmp --offload-arch=gfx906 main.cpp -o app_omp
```

OpenMP Offload 提供了一条更适合存量 CPU 并行程序的迁移路径。与 HIP
相比，它不要求开发者显式编写 Kernel 启动代码，而是通过 target
等指令描述应当被卸载到设备端执行的区域。DCC 则负责把这些指令级语义拆解为双端编译流程。

这种方式的优点在于迁移成本较低，尤其适合已经大量使用 OpenMP
的项目；但它也有自己的工程约束。最典型的三个关注点分别是：数据映射策略是否合理、卸载粒度是否足够大、目标架构是否与设备一致。

其中，map(to/from)
语义直接影响数据传输成本。如果一个计算区域本身并不重，却在每次进入和退出时都触发大量主机与设备间数据拷贝，那么理论上的并行收益可能会被数据传输成本完全抵消。同样，如果被卸载的计算块过小，Kernel 启动开销和调度开销也可能使整体收益不明显。因此，在 OpenMP Offload
的实践中，热点识别与卸载边界设计往往比单纯切换编译开关更重要。

5.3.4 常见编译产物与 Fatbinary 组织方式

产物类型	说明	典型用途
可执行文件	含 Host 与 Device 装载信息的终产物	部署与运行
ELF 目标文件	主机侧中间/最终目标文件	链接、调试
HSACO / Code Object	设备端可加载二进制	Runtime 加载、内核分析
Fatbinary	多架构设备代码封装	一包多机型发布
汇编文件（.s）	目标指令文本表示	指令级验证
GCVM IR（.ll/.bc）	编译中间表示	优化过程分析

理解这些产物，对于构建系统设计、问题排查和性能分析都很重要。以异构程序为例，最终的可执行文件通常只是“最外层壳体”，其中可能已经包含主机端控制逻辑、设备端代码对象以及相关元数据。若只把它当作普通 ELF 文件看待，就很难理解设备端代码在其中的组织方式。

设备端的 HSACO（或 Code Object）更适合被视为“真正要在 DCU 上加载执行的对象”。当需要分析 Kernel 的寄存器使用、LDS
占用、目标架构对应关系或反汇编结果时，开发者往往更关注这部分产物。Fatbinary
则进一步把多个架构的设备端代码对象组织在一起，以支持同一程序在多种机型上部署。

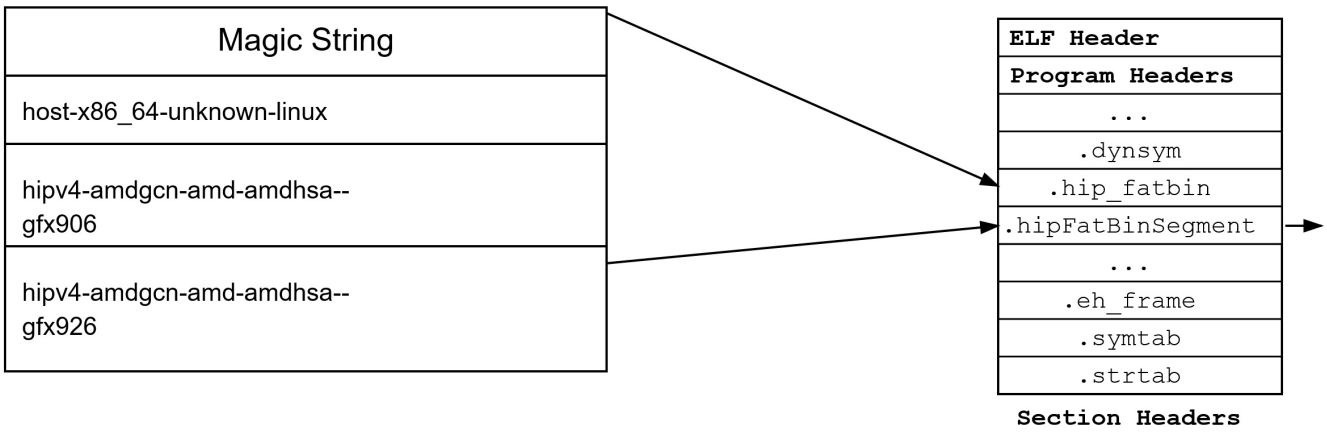
此外，汇编文件与 GCVM IR
文件虽然通常不是最终交付物，但在定位编译行为、验证优化是否生效、分析指令生成策略时非常有价值。实际调优中，推荐采用“静态 + 动态”的联合分析方式：先用 dccobjdump
查看静态资源占用和指令分布，再用 dccprof
观察运行期瓶颈。只有将两者结合起来，才能避免陷入“看起来汇编很漂亮，但运行表现并不理想”的局部判断。

在这些编译产物中，Fatbinary
是最容易在工程交付阶段被单独拿出来讨论的一类，因为它不只是“又一种文件格式”，而是直接对应多架构构建策略。

在单一设备型号环境中，开发者往往只需指定一个
–offload-arch；但在实际部署中，尤其是产品交付、集群发布和通用软件分发场景下，目标环境可能同时包含多个不同代际的
DCU。此时，如果为每种机型分别生成一个独立程序，维护成本会迅速增加。Fatbinary
的意义，正是在于为“单一交付物覆盖多种设备架构”提供统一封装能力。

下面这张示意图可以帮助理解 Fatbinary 在最终 ELF
产物中的大致组织方式：左侧可以把它看作被打包进去的多份目标描述与设备代码对象，右侧则表示这些内容在最终二进制中的承载位置。实际工程里，开发者通常不会直接手工构造这些
section，但在做产物检查、静态分析或排查多架构装载问题时，理解这种组织关系非常有帮助。

[图片]



从概念上看，Fatbinary 可以理解为“设备代码对象的集合容器”。每个设备架构会先独立生成自己的 Code Object/HSACO，随后这些对象与相应元数据一起被打包到统一载体中。运行时在程序启动或 Kernel
装载阶段，再根据当前硬件架构选择最匹配的那一份设备代码。这样，开发者不必在应用层手工分发不同机型的二进制文件，而可以把多架构支持前移到编译与打包阶段完成。

对用户来说，最直接的外在表现就是：一次编译命令可能对应多个设备端产物，而最终可执行文件中并不只包含单一设备镜像。也正因为如此，多架构构建虽然提升了交付灵活性，但也会带来产物体积增大、编译时间延长、调试复杂度上升等代价。工程上是否采用
Fatbinary，取决于部署策略。如果软件面向固定机型部署，单架构构建往往更简单直接；如果软件需要在多个型号间统一发版，Fatbinary
则能显著减少交付分叉。一个成熟的构建系统，通常会同时保留“单架构精简包”和“多架构通用包”两类产物，以满足开发调试与对外交付的不同需求。

5.4 编译选项说明

5.4.1 优化等级

优化等级是 DCC
最常用、也最容易被简单化理解的一组参数。很多用户会把它们理解为“优化开少一点”或“优化开多一点”，但对 DCC 这类建立在 LLVM/Clang 体系之上、又要同时处理 Host 与 Device
双路径生成的编译器而言，-O0、-O1、-O2、-O3、-Ofast 之间的差异，远不只是 Pass
数量的增减，而是编译器对内联、循环变换、向量化、冗余消除、访存重写、目标相关调度以及浮点语义收紧/放宽的整体策略变化。

总体来讲：-O0 更偏调试，-O2 更偏生产基线，-O3 更偏热点性能攻关，-Ofast
则意味着开始主动交换一部分严格语言/数值语义来争取更高吞吐。但在 DCC
里，这些优化等级的影响会同时落到主机侧和设备侧。也就是说，同一条 dcc -O3 –offload-arch=gfx926 app.cpp -o app 命令，既会改变主机侧普通代码的优化方式，也会改变设备端 Kernel 的 IR
优化、指令选择前准备和部分目标相关优化机会。因此，在异构工程里谈优化等级，不能只看主机程序经验，而必须把设备端代码生成质量一起纳入考量。

常见优化等级可以先用下面这张表做一个整体定位：

选项	大致语义	更适合的场景
-O0	基本关闭大部分优化，优先保留码与中间/目标代码的可对应性	问题复现、单步调试、前端语义位
-O1	启用较轻量优化，尽量控制编译本	开发迭代、快速试编
-O2	启用较完整的通用优化集合	生产默认、数值与性能平衡的主基线
-O3	在 -O2 基础上进一步打开更激进性能导向优化	热点模块、性能攻关
-Ofast	在 -O3 基础上进一步放宽部分严标准与浮点语义	对精度容忍度较高、吞吐优先场
-Os/-Oz	体积优先优化	镜像、分发、部署体积敏感场景

在实际工程里，-O0
的意义并不是“最慢，所以尽量别用”，而是“最适合作为诊断基线”。例如，当一个 HIP 程序在 -O3 下结果异常时，直接怀疑后端通常并不是最高效的路径，更实用的做法是先退回：

```
dcc -O0 -g --save-temps --offload-arch=gfx926 kernel.cpp -o kernel_dbg
```

这样可以最大程度保留前端语义、调试信息和中间产物，先回答“问题是否在无优化状态下就存在”。若 -O0 下结果正确、-O3 下才出现偏差，才更值得继续检查浮点融合、快速数学或目标相关优化路径。

-O2
往往是最值得推荐的团队默认基线。原因不只是它“性能和稳定性折中”，而是它通常能在不明显放大代码体积和不明显增加数值风险的前提下，提供一套比较成熟的通用优化收益。

```
dcc -O2 --offload-arch=gfx926 app.cpp -o app
```

这条命令在工程上的价值，不在于它一定最快，而在于它适合作为“功能正确 + 性能达标 + 易于回归”的公共起点。之后无论是要比较 -O3
的收益，还是比较更激进浮点开关的影响，都有一个稳定基线可做参照。

-O3
则更适合明确的性能攻关阶段。它通常会更积极地做代码膨胀型优化，例如更激进的循环变换、向量化、内联和调度准备。这在许多设备端热点 Kernel 上会带来真实收益，但也不是无条件成立：有些代码在 -O3 下会变得更大，寄存器压力更高，反而影响 occupancy 或导致局部退化。因此，-O3
最适合和静态、动态分析联合使用，例如：

```
dcc -O3 --offload-arch=gfx926 -mcpu=gfx926 kernel.cpp -o kernel_o3
dccobjdump --inputs=kernel_o3
```

如果 -O3
带来性能上升，同时静态资源占用没有出现异常膨胀，那么它才更适合被固化进正式构建；反过来，如果 -O3 只是让汇编更复杂，却让寄存器、LDS 或代码体积明显升高，就需要重新评估收益是否真实。

-Ofast 则必须单独看待。它在语义上并不是“比 -O3
再快一点”的普通升级，而通常意味着编译器开始系统性地放宽部分严格浮点语义，常常与后面 4.4
中讨论的 fast-math、FMA 融合、denormal 处理等行为一起作用。对 AI
推理、部分训练热点、以及更偏吞吐导向的数值场景，它可能值得尝试；但对科学计算、数值仿真、迭代求解器和要求结果严格可复现的场景，就必须谨慎。更稳妥的使用方式往往是局部评估，而不是全工程直接切换：

```
dcc -Ofast --offload-arch=gfx926 hotspot.cpp -o hotspot_fast
```

对体积敏感的场景，-Os 和 -Oz 也并非没有意义。虽然 DCC 更常被用于 HPC
和异构性能场景，但在需要把工具链集成进镜像、容器、边缘节点或交付包时，设备端与主机端体积同样可能成为问题。此时，-Os 和 -Oz 的关注点不再是纯性能，而是“交付形态是否更紧凑”。

5.4.2 目标架构与 Offload 选项

在 DCC 里，目标架构与 offload
相关选项的重要性通常不亚于优化等级，甚至在很多真实项目中更关键。原因并不复杂：优化等级决定的是“编译器尽多大努力优化”，而目标架构选项决定的是“编译器到底在为哪台机器生成设备端代码”。如果这一点配错，结果往往不是“稍微慢一点”，而是直接退化为运行异常、设备端代码不可用、指令选择不匹配，或者明明能跑但性能与预期严重偏离。

在 DCC 里，推荐使用 –offload-arch= 和 -mcpu=
来表达”为特定目标生成代码”。前者更像是在说“我要为哪类设备生成可加载代码”，后者更像是在说“在已经确定目标架构的前提下，后端应更贴近哪一类微架构特性做选择与调优”。对很多项目来说，这两个参数一起看，才构成完整的目标约束。

从功能上看，最常见的一组选项如下：

参数	主要作用	更常见的用途
–offload-arch=	指定设备端目标架构	标准异构构建
–no-offload-arch=	排除某一候选架构	多架构构建裁剪
–offload-device-only	仅生成设备端代码	单独分析 HSACO / Code Object
–offload-host-only	仅生成主机端代码	隔离主机侧问题
–offload-host-device	同时生成主机和设备端	标准 Host/Device 联合构建
–offload=	指定具体卸载目标名称	多目标或更复杂链路
-mcpu=	细化后端微架构优化	热点专项优化、固定硬件平台调

最基础、也最值得在团队里统一的一条规则是：关键目标架构尽量显式写进构建命令，而不要只依赖环境变量默认值。
某些目标变量虽然可以提供默认架构，但从工程维护角度看，显式命令更可读、更可审计，也更适合问题回溯。例如：

```
dcc -O2 --offload-arch=gfx926 app.cpp -o app
```

相比单纯依赖某个 shell 中事先 export AMDGPU_TARGETS=gfx926
的写法，这条命令更容易让后续读日志的人直接知道“这个产物就是按 gfx926 生成的”。

在理解 –offload-arch 时，还需要特别注意 DCC 的 Host/Device
双路径语义。对纯主机程序而言，架构参数的影响有限；但对 HIP、OpenMP Offload
或其他异构编译场景，–offload-arch 会决定设备端 IR 后续面向哪类
ISA、资源模型和运行时接口落地。如果它和实际节点硬件不一致，最常见的问题包括：程序可以编译，但运行期设备侧加载失败；程序运行正常，但性能显著偏低；某些指令路径或资源调度没有真正针对目标硬件展开。

在多机型部署场景下，DCC 的多架构构建能力就会变得非常重要。DCC 可以通过多次指定
–offload-arch 来生成同一程序的多架构设备端产物：

```
dcc -O3 --offload-arch=gfx926 --offload-arch=gfx928 kernel.cpp -o app_multi
```

这种写法更适合统一交付、镜像化发布或集群异构节点并存的场景。它的优点是部署时不用按机型分发不同二进制；缺点则是产物体积更大、编译时间更长、静态分析和问题定位也更复杂。因此，对本地开发和日常调试来说，通常更推荐先用单架构构建；对正式发版和公共软件包，再考虑多架构打包。

当需要单独拆解 Host 与 Device 行为时，–offload-device-only 与 –offload-host-only
会非常有用。例如，若你怀疑问题集中在设备端代码生成质量，而不想每次都经历完整主机链接流程，可以这样写：

```
dcc -O2 --offload-device-only --offload-arch=gfx926 kernel.cpp -c -o kernel_dev.o
```

相反，如果你想确认主机侧路径、构建系统或普通链接阶段是否正常，也可以用 –offload-host-only
先把设备端问题暂时排除。对复杂工程而言，这类拆分参数非常适合缩小问题范围。

在实际工程中，还常见另一类问题：项目脚本中同时存在环境变量、–offload-arch、甚至手工拼接的
-mcpu，最后到底哪个生效并不清晰。因此，推荐的治理方式是：正式构建命令里显式保留
–offload-arch，仅在需要更深入调优时再追加
-mcpu，而不要同时让环境变量和脚本参数在多个层面隐式决定目标。否则最容易出现的情况就是：本地开发机能跑、CI 能编、某些节点运行又异常，但没人能第一时间看出目标架构到底是谁决定的。

为了方便快速对应硬件平台，文档中常见的架构代号仍可以作为参考基线：

硬件平台	架构代号
VEGA20 / ZIFANG	gfx906
KONGMING	gfx926
KME / KME-ECO	gfx928
BW	gfx936
NMZ	gfx938

但在真正工程里，更重要的不是背下这些代号，而是把“硬件型号—构建脚本—发布产物”之间的映射固定下来。只有这样，4.2 这一节才不只是参数说明，而是设备端交付策略的一部分。

5.4.3 默认参数注入机制

DCC
与很多“只做参数转发”的薄驱动不同，它在编译和链接阶段会自动补充一组必要参数。对初学者而言，这种设计非常友好，因为用户不必像手写 clang++ / ld.lld
调用那样，逐个处理头文件路径、运行时库路径、基础依赖库和某些工具链内部目录；但对复杂工程而言，这也是最容易被忽视、最容易引发冲突的一环。很多看似“编译器行为异常”的问题，最后根因其实并不在前端、后端或优化器，而在于 DCC
自动注入了一套参数，而构建系统又手工补了一套相似或冲突的参数，最终让头文件优先级、链接顺序或运行库版本发生了偏移。

DCC
的这一机制并不难理解：它会主动补齐一部分构建所需的路径和库，使用户在常见场景下不必手工写出全部底层依赖。也就是说，DCC
的默认参数注入不是副作用，而是产品设计的一部分；真正的问题在于，团队是否知道它注入了什么、从哪里注入、什么时候应当复用、什么时候又必须显式覆盖。

从效果上看，DCC 自动注入的内容通常集中在三类。第一类是头文件搜索路径，常表现为 -isystem …
形式，用于把工具链内置或运行时依赖的头文件目录放进编译器搜索范围。第二类是库搜索路径，常表现为 -L …
形式，用于把工具链相关库、运行时库或某些默认依赖纳入链接器可见范围。第三类则是必要的默认链接库，例如某些基础系统库、线程库、数学库以及异构运行时相关库。这三类内容的组合，本质上就是让用户在多数简单场景下，不必显式手写一整串 include 和 link 选项。

可通过下面两个命令直接查看注入结果：

```
dcc --cxxflags
dcc --ldflags
```

这两个命令的用途远远不只是“打印一下看看”。在工程上，它们至少有三种非常现实的价值。第一，用来确认当前 DCC 会话到底绑定到了哪套工具链路径上。第二，用来判断自动注入的 -isystem、-L 与项目构建系统中显式写的 -I、-L 是否存在重叠或冲突。第三，用来帮助 CMake、Makefile、Bazel 等构建系统在接入 DCC 时建立更清晰的职责分工：哪些路径让 DCC
自动维护，哪些路径由项目自身显式维护。

举一个很常见的场景：项目的 CMakeLists 已经手工写了一套 include_directories(…) 和 link_directories(…)，构建前又额外 source env.sh，结果 DCC
本身也自动注入了一套系统路径。表面上看只是“多写了一点路径”，但一旦这些路径指向的目录版本并不一致，就可能出现：编译阶段用了新头文件，链接阶段却连到了旧库；或者主机侧链接顺序被改变，导致符号解析和预期不一致。此时，最有效的做法通常不是继续增删 -I/-L，而是先打印：

```
dcc --cxxflags
dcc --ldflags
```

把 DCC 已经承担的部分看清楚，再决定项目自身还需要补哪些内容。

默认参数注入还有一个经常被低估的影响：它会改变“构建系统应该显式负责什么”的边界。对简单项目来说，完全依赖 DCC
注入通常没有问题；但对大型工程来说，推荐把依赖分成两层。第一层是工具链公共依赖，例如 DCC 和 DTK 自带的头文件、运行时和基础库，让 DCC

继续自动维护。第二层是项目私有依赖和第三方业务库，例如本项目自己的
include/、自定义算子库、额外数学库或通信库，这些则应明确写在构建系统中。这样做的好处是，工具链升级时，只需要验证 DCC
的默认注入是否仍符合预期；项目自身的路径组织则不会因为工具链变化而被迫重写。

在实际排障中，最常见的几类问题都和这一机制密切相关。第一类是头文件优先级异常：项目本来期望使用本地某个头文件版本，但因为 -isystem
的默认注入，实际先命中了工具链目录。第二类是链接顺序异常：项目显式写的 -L 和 DCC 自动注入的 -L 混在一起，最终链接器优先找到了非预期版本库。第三类是“同一项目在不同机器上编译结果不同”：根因往往不是源码变化，而是某些机器上 env.sh 生效、某些机器上又额外叠加了本地 shell
的旧路径设置，导致 DCC 的自动注入结果实际不同。

从更实用的角度出发，可以把默认参数注入的处理方式总结为三条。第一，先观察，再覆盖。在没有看过 dcc –cxxflags 和 dcc –ldflags
之前，不要急着在构建系统里大面积手工补路径。第二，工具链依赖和项目依赖分层维护。让 DCC
负责工具链公共部分，让项目只负责自己的私有部分。第三，避免同时保留多套历史路径设置。如果已经通过 source env.sh 进入标准 DCC/DTK
环境，就不要再长期叠加另一套手工导出的旧路径变量或自定义 include/lib 路径。

若需要更接近实战的检查方式，可以直接把下面这组命令作为接入 DCC 时的起点：

```
dcc --version
dcc --cxxflags
dcc --ldflags
HIPCC_VERBOSE=1 dcc -O2 --offload-arch=gfx926 app.cpp -o app
```

前 3 条命令回答“这套工具链是谁、会自动补什么”；最后 1
条命令回答“在当前命令里这些自动注入最终如何参与真实构建”。把这两个问题先看清楚，再去谈参数治理，通常会比直接在 CMake 或 Makefile
里盲目堆叠选项更有效。对长期维护的异构工程而言，真正稳定的做法从来不是“把所有路径都写死”，而是把自动注入机制理解清楚，然后有边界地使用它。

5.4.4 浮点、精度与快速数学选项

前面三节已经把优化等级、目标架构和默认注入机制拆开说明了，因此从这一节开始，更适合进入“具体参数族”的层面。最值得先单独拿出来讲的，就是浮点、精度、舍入与快速数学相关选项。原因很现实：在很多实际项目中，真正让结果和性能发生明显变化的，往往不是 -O2 和 -O3 本身，而是
-Ofast、-ffast-math、FMA 融合、舍入模式和 denormal 处理这些更细的浮点开关。

在 DCC
中，这组参数的核心用法可以概括为两类：严格语义场景更适合显式约束浮点行为，吞吐优先场景则更适合逐步放宽这些约束。也就是说，这一节讨论的不是“哪些参数更高级”，而是“在结果一致性、IEEE 语义和设备端吞吐之间，编译器被允许做到什么程度”。

先看最常用的一组核心参数：

参数	作用	典型影响

-Ofast	在 -O3 基础上进一步放宽严格标语义	性能更激进，结果一致性风险更
-ffast-math	启用快速数学优化集合	允许更激进重排与近似
-fno-fast-math	关闭快速数学集合	更适合数值敏感场景
-ffp-contract=off	关闭浮点融合，如 FMUL+FADD → FMA	更接近逐步计算语义
-ffp-contract=fast	允许更积极的浮点融合	常提升性能，可能改变最低位结
-ffp-model=strict	更严格遵循 IEEE 浮点语义	调试、数值对齐、结果复现
-ffp-model=fast	更偏性能导向的浮点模型	更适合吞吐优先场景
-funsafe-math-optimizations	允许不完全保守的浮点优化	可能引入更大语义放宽
-ffinite-math-only	假设不存在 NaN/Inf	能帮助优化，但不适合异常值敏场景
-fno-signed-zeros	忽略 +0.0/-0.0 区别	便于某些数学重写
-freciprocal-math	允许用乘倒数替代除法	可能提升吞吐，影响精度边界
-fno-math-errno	不为数学函数保留 errno 语义	减少某些额外约束

这张表里最容易被误用的通常是 -Ofast。它看起来像是“只是比 -O3
更快一点”，但更准确的理解应当是：它会系统性放宽一部分严格浮点语义，因此它不是普通优化等级的线性升级，而更像一个“明确选择性能优先”的信号。对数值模拟、迭代求解、科学计算验证或要求 bitwise 稳定的场景，更稳妥的路径通常是先以 -O2 或 -O3 建立参考基线，再局部评估 -Ofast 或 -ffast-math 是否可接受。

下面给出几种更贴近实际工作的写法：

```
# 严格浮点语义，适合结果对齐和数值回归
dcc -O2 -ffp-model=strict main.cpp -o app_strict

# 关闭 FMA 融合，排查低位精度差异
dcc -O2 -ffp-contract=off main.cpp -o app_no_fma

# 快速数学，适合吞吐优先场景
dcc -O3 -ffast-math main.cpp -o app_fastmath

# 更激进的整体性能策略
dcc -Ofast main.cpp -o app_ofast

# 允许用乘倒数替代除法，常见于热点内核实验
dcc -O3 -freciprocal-math kernel.cpp -o kernel_rcp
```

如果你发现“同样的算法在 DCC
下最后几位有轻微差异”，最值得先看的通常不是源码本身，而是：是否启用了 -Ofast、是否允许了 FMA 融合、是否默认启用了 fast-math

风格语义，以及项目构建脚本里是否隐含传入了某些浮点相关开关。

如果继续往浮点行为更细的层面下钻，很多项目还会关心更具体的舍入、异常与 denormal 处理选项。它们不一定每个项目都会用到，但一旦涉及严格数值复现、跨平台对齐或定位“最后几位为什么不同”这类问题，就会变得很重要：

参数	关注点	更适合的场景
-frounding-math	假设程序依赖显式舍入模式，不意重排相关浮点运算	舍入模式敏感、严格数值验证
-fno-rounding-math	不把动态舍入模式当成强约束	常规性能导向场景
-ftrapping-math	保守处理可能触发浮点异常的运	严格异常语义、数值调试
-fno-trapping-math	放宽对浮点异常捕获语义的限制	大多数吞吐优先场景
-ffp-exception-behavior=strict	更严格保留浮点异常相关语义	调试、验证、与参考实现对齐
-ffp-exception-behavior=ignore	更偏性能导向，弱化异常语义约	训练、推理、吞吐优先场景
-fdenormal-fp-math=ieee	更严格保留 denormal 行为	极小值敏感计算
-fdenormal-fp-math=preserve-sign 或 positive-zero	更偏硬件友好路径	对尾部精度容忍度较高的性能场

把它们和前面已经提到的 -ffp-model、-ffp-contract 合在一起看，会更容易理解其分工：-ffp-model 是总策略，-ffp-contract 控制是否积极生成 FMA，-frounding-math 和 -ftrapping-math 更偏向运行时浮点语义约束，-fdenormal-fp-math 则更偏向 denormal 路径本身。也就是说，遇到数值问题时，不能只盯着 -Ofast 或 -ffast-math，还要看这些更细的浮点行为控制项是否参与了构建。

例如，下面几组命令分别对应几种典型诉求：

```
# 舍入模式和异常语义都更保守
dcc -O2 -ffp-model=strict -frounding-math -ftrapping-math main.cpp -o app_ieee_dbg

# 需要严格保留 denormal 行为
dcc -O2 -ffp-model=strict -fdenormal-fp-math=ieee kernel.cpp -o kernel_ieee_denorm

# 更偏吞吐，允许忽略异常约束
dcc -O3 -ffast-math -ffp-exception-behavior=ignore kernel.cpp -o kernel_fast_fp
```

5.4.5 GPU 执行与资源相关参数

与浮点语义相比，另一组更偏设备端执行行为的参数也非常常见，尤其常用于 Kernel 调优、资源限制实验以及运行期分析场景。它们不一定直接改变算法语义，但会明显改变 Kernel 在设备上的执行方式、资源约束或分析插桩方式：

参数	作用	常见用途
-fgpu-default-stream=	设置 DCU 上下文默认 stream	多 stream/默认 stream 行为控
-fgpu-flush-denormals-to-zero	将非规格化浮点数归零	提升某些计算路径吞吐
-fno-gpu-flush-denormals-to-zero	保留非规格化数行为	数值一致性要求更高场景
–gpu-max-threads-per-block=	指定每个 block 最大线程数	Kernel 配置实验与资源约束分
–gpu-instrument-lib=	注入 GPU 计数器或分析库	性能分析、运行期度量
-nogpuinc / -nocudainc	不引入某些默认 GPU/CUDA 头件	兼容复杂混合环境
-nogpulib / -nocudalib	不链接某些默认 GPU/CUDA 库	手工控制链接行为

这组参数在使用上更像“设备端行为控制开关”，而不是纯粹的通用优化开关。比如
-fgpu-default-stream= 更接近运行语义配置，–gpu-max-threads-per-block=
更接近资源模型实验，–gpu-instrument-lib=
则明显属于分析/度量配套能力。更建议把它们明确区分出“正式构建参数”和“专项实验参数”。

这组参数最典型的几个例子如下：

```
# 设置默认 stream
dcc -O2 -fgpu-default-stream=1 kernel.cpp -o kernel_stream1

# 打开非规格化数归零
dcc -O3 -fgpu-flush-denormals-to-zero kernel.cpp -o kernel_ftz

# 保留非规格化数语义
dcc -O2 -fno-gpu-flush-denormals-to-zero kernel.cpp -o kernel_no_ftz

# 显式约束 block 最大线程数
dcc -O3 --gpu-max-threads-per-block=1024 kernel.cpp -o kernel_1024

# 插入性能分析库
dcc --gpu-instrument-lib=libgpuinstrument.so kernel.cpp -o kernel_inst
```

这里尤其值得单独说一下非规格化数处理。-fgpu-flush-denormals-to-zero
是一个非常现实的性能开关，因为非规格化数会让某些硬件路径变慢。对吞吐优先场景，它可能是合理的；但对极小值敏感的数值计算，它同样可能改变尾部精度行为。因此，这个选项不应被简单理解成“一个默认应该开的加速开关”，而应视为明确的数值-性能权衡项。

另一个容易被忽视的点，是 -nogpuinc / -nogpulib
这类参数虽然平时不常作为正式构建默认项，但在混合环境排障时非常有价值。例如，当一台机器上同时存在多套相关头文件或库目录时，这类开关可以帮助开发者验证问题究竟是不是来自默认头文件/库的自动参与。类似地，–gpu-instrument-lib=
也更适合作为专项分析配置，而不宜默认保留在正式生产构建模板中。

5.5 DCC 的工具链

5.5.1 从“一个编译命令”到“一套工程链路”

如果只从命令行入口观察，DCC 最容易给人的印象是一个名为 dcc
的编译器驱动；但只要进入真实的异构开发场景，就会很快发现，DCC
实际上并不是单一命令，而是一套围绕“构建、观察、验证、调优”组织起来的工具链。理解这一点尤其重要：DCC 的价值不只是把源码编译成可执行文件，更在于它把设备端代码生成、Fatbinary
组织、静态反汇编、汇编级再编译、自动参数搜索以及运行时性能分析这些原本分散的动作，收敛到同一条工作路径中。

这种组织方式与普通 CPU
编译流程有明显差别。传统程序开发往往把“编译成功”视为一个重要阶段性结束；而在 DCU
异构程序中，编译成功通常只是开始。后面还会接着出现一连串更实际的问题：设备端代码是否真的进入了最终产物，目标架构是否匹配，Kernel
的寄存器和资源使用是否合理，多架构包是否按预期封装，运行时瓶颈究竟出现在内核执行、数据传输还是主机侧调度链路上，修改某个编译选项之后收益是否真实存在。DCC
工具链的意义，就在于为这些问题提供连续的回答路径。

如果把这套工具按职责划分，可以把它粗略看成四个层次。第一层是统一构建入口，也就是 dcc
本身；第二层是静态观测工具，以 dccobjdump 为代表；第三层是重构和搜索工具，即 dccturing 与 dcc-autotune；第四层是运行时分析工具
dccprof。这四层并不是简单并列关系，而更像一个递进闭环：先生成产物，再理解产物，再验证假设，最后把验证结果反馈回下一轮构建。也正因为如此，第五章讨论工具链时，不能只列工具名和命令格式，而要把它们放进同一个工程流程里理解。

DCC
工具链并不是一条从左到右一次走完的直线流程，而是一条会不断回流、不断收敛的工程闭环。构建、静态分析、动态分析和调优并不是互相割裂的阶段，而是围绕同一份程序反复交替进行。

5.5.2 dcc：统一驱动、参数编排与默认构建机制

dcc
是整套工具链的起点，也是把多种输入收敛到同一条构建链路上的总入口。对使用者来说，它表现为一条统一命令；对工具链内部来说，它承担的是流程编排：识别输入类型、划分主机侧与设备侧路径、补齐必要参数、驱动目标代码生成，并把最终产物组织成可执行文件、对象文件或设备端代码对象。真正重要的并不是“它能调用多少子工具”，而是它把原本容易分散的异构构建动作收到了同一套参数体系里。

这种编排能力在 HIP、CUDA、OpenMP、OpenACC、SYCL、OpenCL
等异构场景里最容易看清。纯主机程序通常沿着预处理、编译、链接一路向前；而异构程序会在构建过程中分出两条并行路径：一条生成主机侧控制逻辑，负责设备初始化、参数传递、Kernel
启动和同步；另一条面向指定架构生成设备端代码对象，并在后续阶段完成封装。表面上仍是一条 dcc 命令，实际已经包含了多目标生成、设备端打包和最终装配等步骤。也正因为如此，–offload-arch
这类参数在 DCC 里不是普通开关，而是决定设备侧产物形态的核心约束。

dcc
还有一个很典型、但经常在大型工程里被忽视的特征：它会自动补齐一部分构建所需的头文件路径、库路径和默认依赖。很多初次使用 DCC
的用户会觉得“命令写得很短却能直接编过”，原因并不神秘，而是驱动在背后完成了环境探测和参数注入。这种机制让日常编译更轻，也让工具链升级、脚本迁移和构建系统接入时更需要边界意识：哪些路径由 dcc
统一维护，哪些路径由项目自己显式维护，如果两套逻辑混在一起，最容易出现的并不是编不过，而是头文件优先级、链接顺序或运行库版本悄悄偏掉。

因此，在 DCC
环境里排查构建问题，第一步往往不是改源码，而是先看驱动到底展开成了什么。HIPCC_VERBOSE
适合看命令展开过程，–cxxflags 和 –ldflags
适合看默认注入参数。对复杂工程而言，这些入口的价值并不在于“多了几个调试选项”，而在于它们能够把驱动的真实行为摊开，让使用者区分：问题究竟出在源代码、构建脚本，还是出在驱动编排本身。

从这个角度再看，dcc
并不是后面几个工具前面的一个壳层，而是整个工具链闭环的起点。后续所有静态分析、再编译实验、自动调优和运行时分析，都是建立在它生成的产物和参数语义之上的。

5.5.3 dccobjdump：把编译结果变成可观察对象

在异构程序开发里，“编译成功”往往只说明语义层面过关，并不说明最终产物已经符合预期。设备端代码到底面向哪种架构生成、哪些 Kernel
被打包进了最终文件、关键路径的指令形态是否合理、静态资源占用是否过高，这些问题都无法只靠源码判断。dccobjdump
的意义，就是把这些已经写进二进制、但平时不可见的信息重新展开，让开发者真正看到“构建结果是什么”。

它最常见的用法并不是一上来就盯着某段汇编，而是先做一次整体摸底，再逐步缩小范围。先确认产物里带了哪些目标，再看有哪些 Kernel，再看某个热点函数的汇编、资源画像和
descriptor。这样的顺序更接近真实排障逻辑：先判断是不是打包问题，再判断是不是生成质量问题，最后才进入具体指令层。

一个很常见的起步组合如下：

```
# 先看程序里到底有什么
dccobjdump --inputs=app --show-target-triple --show-symbols

# 再看目标 kernel 的汇编和资源
dccobjdump --inputs=app --function=target_kernel --show-sass --show-resource-usage

# 如果要为后续再编译做准备
dccobjdump --inputs=app --recompile --output=./recompile_out
```

当整体结构确认无误后，再进入“聚焦热点对象”的阶段。例如先列符号，再只提取某个热点 Kernel：

```
# 列出程序中的 kernel / 符号
```

```
dccobjdump --inputs=app --show-symbols

# 只分析某个热点 kernel
dccobjdump --inputs=app --function=_Z15cuInsertionSortPfPiiii --show-sass
```

这样做的原因很简单：大型程序里设备端对象、主机侧符号和多架构镜像常常混在一起，如果不先缩小分析对象，就很容易得到一堆信息，却没有真正回答问题。相反，只要把范围收窄到某个 Kernel，很多判断就会立刻变得具体：这段代码是否真的生成了预期的指令，循环与访存模式是否自然，寄存器和 LDS 是否已经把执行资源压得过紧。

继续往下看时，最值得配合使用的是汇编、资源和 descriptor 三类信息：

```
# 看设备端汇编
dccobjdump --inputs=app --show-sass

# 汇编同时显示编码字节
dccobjdump --inputs=app --show-sass --show-instruction-encoding

# 看资源使用情况
dccobjdump --inputs=app --show-resource-usage

# 看 kernel descriptor
dccobjdump --inputs=app --show-kernel-descriptor
```

这三类输出回答的是三个层面的问题：指令到底长什么样、为了这些指令付出了多少静态资源、以及运行时在装载与启动阶段会如何看待这个 Kernel。很多性能问题只有把这三层对上，才不会误判。比如汇编看起来不长，却伴随很高的 VGPR 占用；或者资源并不夸张，但 descriptor 暗示了更严格的执行约束。这时真正要问的已经不是“汇编漂不漂亮”，而是“它在设备上是否容易跑得高效”。

dccobjdump 也很适合做多架构对比。对于同一程序在不同机型上表现差异较大的情况，最有价值的不是笼统地说“硬件不同”，而是把不同架构下的静态结果拆开看：

```
dccobjdump --inputs=app_multi --architecture=gfx906 --show-resource-usage
dccobjdump --inputs=app_multi --architecture=gfx926 --show-resource-usage
```

如果拆开以后发现资源画像和指令形态本身就已经不同，那么问题多半出在目标相关生成；如果静态结果基本一致，而运行表现却差很多，才更值得继续追运行时路径、内存行为或调度差异。

另外，dccobjdump 并不只盯着设备端。需要联合看主机侧调用路径时，也可以直接把主机端反汇编拉出来：

```
dccobjdump --inputs=app --disassemble-host --source
```

这在排查“设备端代码本身没问题，但程序整体行为不对”的场景里很有价值。很多时候真正的异常并不在 Kernel，而是在主机侧多做了一层分发、提前插入了同步，或者把参数准备路径拉得过长。只看设备端，很难把这类问题看完整。

更关键的是，dccobjdump 不是静态分析链路的终点。通过

–recompile，它还能把当前产物导出成更适合后续再编译实验的输入，为 dccturing 做准备：

```
dccobjdump --inputs=app --recompile --output=./recompile_out
```

所以，dccobjdump
最有价值的地方，不在于它能不能“看汇编”，而在于它把二进制真正变成了可分析对象。很多低层问题，都是从这里开始被缩小范围的。

5.5.4 dccturing：面向汇编级实验与再编译的低层工具

如果说 dccobjdump 负责把结果摊开，那么 dccturing
负责把局部实验重新装回去。它解决的不是“怎么写业务代码”，而是另外一类更低层的问题：当你已经确定疑点落在机器层，希望在不重写高层源码的前提下验证某个想法时，如何把这个实验做得有上下文、有输入边界，也有可回退的输出结果。

它最典型的工作流很清晰：先从现有程序导出可再编译的设备端输入，再只改你真正关心的那一小段汇编或目标文件，最后把修改后的内容重新嵌回原程序结构，生成一个新的可执行结果。这个过程的价值并不在于“允许手工改汇编”，而在于它把低层验证从一次性试验变成了可重复、可比较的工程动作。

常见命令模式如下：

```
# 单架构、单汇编文件重编译
dccturing --targets=gfx926 \
  --inputs=kernel-gfx926-0.s \
  --executable-file=app \
  --output=app_new

# 多架构重编译
dccturing --targets=gfx906,gfx926 \
  --inputs=app-gfx906-0.s,app-gfx926-0.s \
  --executable-file=app \
  --output=app_multi_new

# 直接以 .o 作为输入
dccturing --targets=gfx926 \
  --inputs=kernel-gfx926-0.o \
  --executable-file=app \
  --output=app_o_new \
  --type=o
```

这里最关键的不是记住参数名字，而是理解它的约束关系：–inputs、–targets 和原始 –executable-file 不是松散拼起来的三个参数，而是一组一一对应的重建上下文。只要输入和目标架构映射错位，即使工具成功生成了文件，结果通常也不具备分析价值。

一个最典型的工作流如下：

```
# 第一步：从原始程序导出可重编译输入
dccobjdump --inputs=test_app --recompile --output=./turing_in
```

```
# 第二步：编辑导出的 .s 文件
# 例如修改 turing_in/test-gfx926-0.s

# 第三步：重新生成新程序
dccturing --targets=gfx926 \
  --inputs=./turing_in/test-gfx926-0.s \
  --executable-file=test_app \
  --output=test_app_new
```

这种流程特别适合做小范围、强针对性的验证。比如，你已经在 dccobjdump 里看到某个热点 Kernel 的等待序列、访存排布或局部调度很可疑，希望只改一两处再看结果。如果直接回到高层源码，变量会一下子变多：前端展开、优化路径、目标相关选择都可能连带变化；而 dccturing 的价值，恰恰在于把变量尽量收窄到你当前怀疑的那一层。

例如，在多架构程序里，常见用法更像下面这样：

```
dccturing --targets=gfx906,gfx926 \
  --inputs=./turing_in/test-gfx906-0.s,./turing_in/test-gfx926-0.s \
  --executable-file=test_app \
  --output=test_app_multi_new
```

这里最容易出错的地方并不是命令格式，而是输入与目标的对应关系。一旦 gfx906 的汇编和 gfx926 的目标顺序写反，工具虽然仍可能完成输出，但新程序的语义就会变得不可靠。因此，凡是进入 dccturing 阶段的实验，最好都保留清晰的目录命名和修改记录，而不要把导出的多个文件临时堆在一起处理。

从工程角度看，dccturing 最适合三种事情：验证某个局部指令调整是否真的带来收益；测试某类特定问题能否通过机器层规避；以及帮助定位问题到底停在前面几层，还是已经落到后端与最终代码生成层。它不会替代正常的源码级开发，但在真正进入设备端执行形态分析之后，它往往是把“猜测”变成“可验证结论”的最快路径。

5.5.5 dcc-autotune：把参数搜索从手工试错变成系统流程

在调优工作里，最容易耗时间、也最容易依赖个人经验的部分，就是参数搜索。大家通常知道“某些开关会影响性能”，但并不知道当前程序真正受哪组组合影响，更不知道不同选项之间是否在相互放大或相互抵消。dcc-autotune 的意义，就是把这种原本靠手工试编、试跑、记结果的过程，改造成有目标函数、有日志、有收敛过程的系统搜索。

它的核心并不是“自动替你做决定”，而是把调优过程变成一套可重复的实验框架：先定义目标，再批量生成候选配置，之后通过稳定的编译与运行流程记录结果，最后把搜索结论落到配置文件里。对个人开发者来说，这意味着少做重复试错；对团队来说，这意味着调优结果终于可以沉淀成可以比较、可以复验的工程资产。

最常见的命令形式如下：

```
# 默认模式：总体参数搜索
dcc-autotune minimize default \
  --results-log RESULTS_LOG.txt \
```

```
  --results-log-details RESULTS_LOG_DETAILS.txt

# 循环模式：更聚焦循环优化
dcc-autotune minimize loop \
  --results-log LOOP_LOG.txt \
  --results-log-details LOOP_DETAILS.txt

# 对最优配置做精简
dcc-autotune impact-analysis final_config.yaml -o tuning_config_minimal.yaml
```

它最适合处理这样一类问题：你已经知道程序还有优化空间，但不确定是 -O2 还是 -O3 更好，不确定某些后端开关的组合是否值得保留，也不确定循环相关优化应该开到什么程度。此时如果完全依赖人工搜索，成本会迅速失控，因为每多一个选项，组合空间都会明显膨胀。dcc-autotune 的意义并不是“替人做决定”，而是让这种本该系统化的搜索真正系统化。

一个最基础的使用方式如下：

```
dcc-autotune minimize default \
  --results-log RESULTS_LOG.txt \
  --results-log-details RESULTS_LOG_DETAILS.txt
```

这条命令背后的含义是：以“执行时间最短”为目标，在默认调优模式下搜索一组更优的编译配置，并把搜索过程分别记录到概要日志和详细日志中。搜索结束后，通常会得到 final_config.yaml 这样的结果文件。对单次实验来说，这意味着你拿到了一份候选最优配置；对团队来说，这意味着以后再做同类程序调优时，可以从这份配置继续演进，而不是每个人重新从零试一遍。

如果问题更偏向循环相关优化，而不是总体后端选项搜索，那么可以把模式切换得更聚焦：

```
dcc-autotune minimize loop \
  --results-log RESULTS_LOG.txt \
  --results-log-details RESULTS_LOG_DETAILS.txt
```

这种用法更适合那些热点明确、计算密集、循环结构主导性能的程序。比如数值内核、矩阵变换或某些 stencil/迭代求解器，在这类场景里，直接搜索与循环和代码生成关系更密切的参数，往往比“对所有参数一视同仁地乱搜”更有效。

调优完成后，一个经常被低估、但实际非常关键的动作是结果精简。自动搜索给出的最优配置常常不短，其中有些选项是真正决定性能的，有些则只是和核心选项一同出现。此时继续做影响性分析，目的不是再找一个“更优答案”，而是把最终配置收缩成团队真正敢长期维护的版本：

```
dcc-autotune impact-analysis final_config.yaml -o tuning_config_minimal.yaml
```

真正落到工程里时，dcc-autotune 最适合放在一个受控实验环境中使用：输入数据固定、运行轮次固定、评价指标固定，必要时连运行节点和负载状态也尽量固定。否则搜索本身没有问题，结论也可能被噪声掩盖。更稳妥的做法通常是先用 dccobjdump 和 dccprof 理清结构与瓶颈，再决定自动调优要搜索哪一类空间。这样最终得到的配置既更有解释性，也更容易进入正式构建体系。

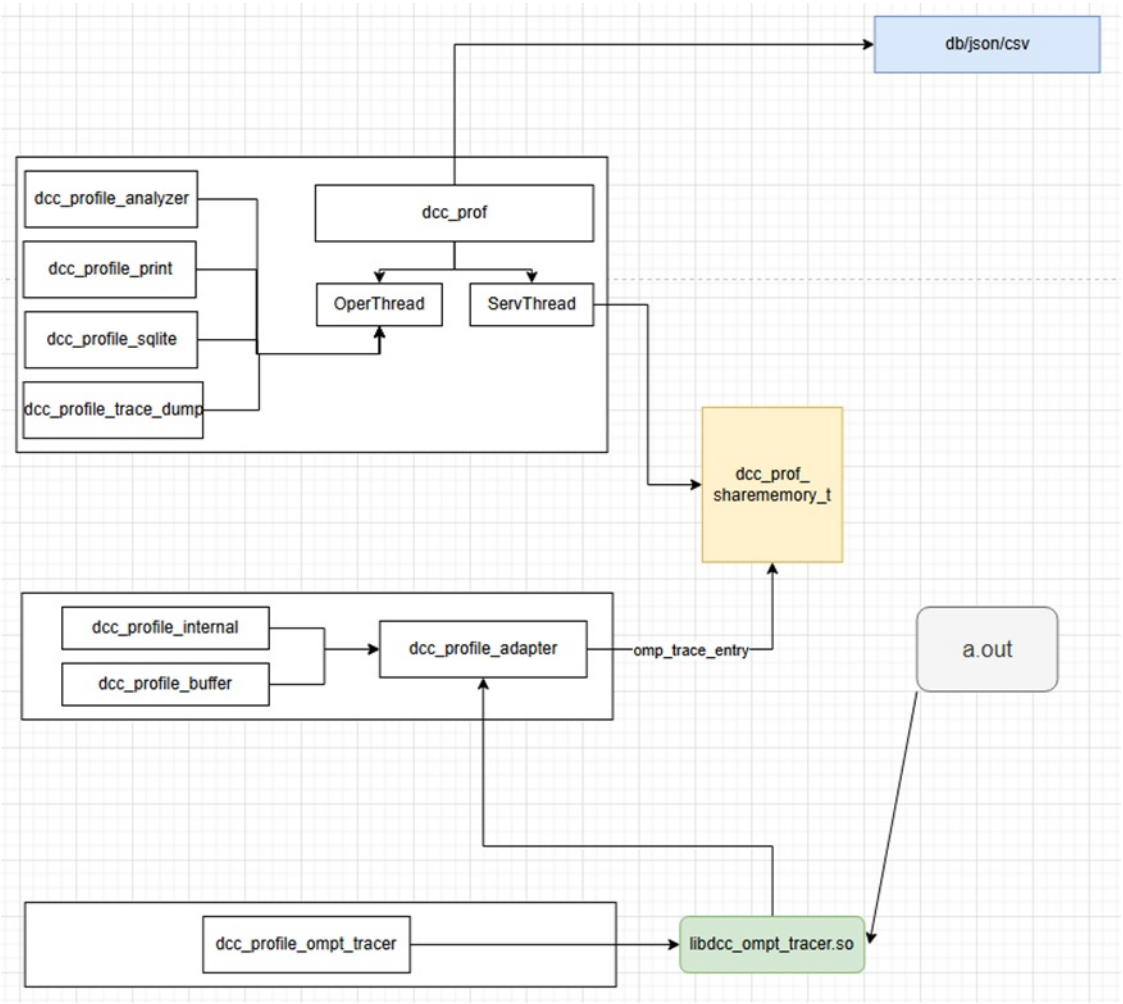
5.5.6 dccprof：把运行时行为纳入编译工具链闭环

只看静态产物，并不能回答所有性能问题。某个 Kernel
的汇编很规整、资源占用也不夸张，程序仍然可能跑得慢；反过来，有些程序设备端算得并不差，真正拖慢它的却是主机侧反复创建映射、同步过密，或者卸载粒度本身设计得太碎。dccprof
面向的，就是这种“静态结果没有直接给出答案，但运行时证据已经决定优化方向”的场景。

dccprof 的价值不只是计时，而是把运行过程组织成可观察、可归档的事件流。它可以把 OpenMP/offload 相关的区域执行、Kernel 调用、数据创建和数据传输记录下来，既支持直接在终端看 trace，也支持写入数据库做后续分析。对一次性排查来说，终端输出已经很有帮助；对需要跨版本、跨节点比较的团队来说，把结果落成数据库更重要，因为这意味着性能问题终于能被持续追踪，而不是每次都从现场重新抓起。

从实现上看，dccprof
可以理解为三层协同：前端控制层负责接收命令和组织输出，运行时采集层负责截获 offload
相关事件，中间的数据适配与缓冲层则把原始事件整理成统一记录，再交给终端输出或 SQLite
落盘。它的结构如下图所示：

[图片]



实际使用时，最常见的几组命令如下：

```
# 最基础的运行时跟踪
dccprof --omp-trace ./a.out

# 一边运行一边在终端打印详细过程
dccprof --omp-trace --print-trace ./a.out
```

```
# 写入数据库并展开更高层级的调用信息
dccprof --db dcc-prof-test.db --omp-trace --trace-level=4 ./a.out
```

这里最后的位置参数始终是被分析的程序，前面的选项则决定采集范围、输出粒度和结果落盘方式。一般来说，–omp-trace 是基础入口，–print-trace 适合在线排查，–db 更适合做持续分析，而 –trace-level 用来控制调用链展开到什么深度。

从调优角度看，dccprof
最重要的作用，是把“程序为什么慢”拆成几个可以真正讨论的问题：到底是算慢了，还是传慢了；到底是设备端热点本身重，还是主机侧调用链太长；到底是并行区域设置不合理，还是同步点过密。没有这些拆分，很多优化会自然滑向错误方向。最典型的误判就是：总时间高，于是继续卷 Kernel；结果一跑 trace 才发现，大头其实花在了 map/unmap 和反复启动的小粒度 offload 上。

基础用法通常如下：

```
# 直接跟踪程序运行时行为
dccprof --omp-trace ./a.out

# 打印更详细的 trace
dccprof --omp-trace --print-trace ./a.out

# 指定数据库输出
dccprof --db dcc-prof-test.db --omp-trace --print-trace ./a.out
如果需要更细地看运行时 API 是怎样层层展开的，可以继续提高 --trace-level：
dccprof --omp-trace --trace-level=4 ./test_omp_device
```

这个选项的意义，在于它能够把“用户看到的一次 offload”拆成“运行时实际发生的多步动作”。对简单程序来说，这个差异可能不显著；但对复杂程序来说，一个高层区域背后完全可能隐藏多次数据操作、同步和设备调用。

一个非常典型的场景，是判断 OpenMP Offload 程序的时间到底花在了哪里：

```
dccprof --omp-trace --trace-level=4 ./solver_omp
```

如果结果显示大部分时间并不在设备端计算，而是在数据映射、同步或主机侧组织阶段，那么接下来该改的通常不是汇编细节，而是卸载边界和执行粒度。对优化方向来说，这种判断比“总时间下降了多少”更关键。

另一个高价值用法，是把 dccprof 当作验证工具。你已经通过 dcc-autotune 找到一组候选参数，或者通过 dccturing 做了一次低层实验，下一步不能只看总时间，还要看时间分布是否真的朝预期移动。局部更快、整体更慢的情况在异构程序里并不少见，而 dccprof 的作用就是把这种误判尽可能提前挡掉。

一个更完整的使用节奏可以是：

```
# 首次摸底：直接看终端 trace
dccprof --omp-trace --print-trace ./app

# 需要存档和后续对比：写入数据库
dccprof --db app_trace.db --omp-trace --trace-level=4 ./app
```

```
# 之后可以围绕数据库继续分析
dccprof --db app_trace.db --print-trace --omp-trace
```

这种方式适合“先快速摸底，再持续沉淀”的调优过程。第一次运行先回答瓶颈大致在哪里；确认值得继续跟踪后，再把结果写入数据库，后续无论换参数、换版本还是换节点，都可以在相近口径下重复比较。只有把静态结构分析和动态行为分析连起来，编译链路的闭环才算真正形成。

5.6 DCC 的前端和 GCVM

5.6.1 前端在 DCC 中的位置

在传统编译器教材里，前端常被简化为“把源码读进来，再吐出 IR”；但在 DCC 这样的异构编译系统中，前端的职责远不止于此。它不仅要完成预处理、词法分析、语法分析和语义检查，还要把程序中的异构执行意图准确识别出来：哪些代码属于主机侧控制逻辑，哪些代码属于设备侧计算逻辑，哪些语义只影响普通函数调用，哪些语义会决定 Host/Device 拆分、数据映射和并行执行范围。换句话说，前端在 DCC 中承担的是“语义分流器”的角色，而不是单纯的“文本解析器”。

这一点在 DCU 程序中尤其关键。因为一个普通 C/C++ 工程与一个异构 HIP 工程，即使表面上都只是若干 .cpp 文件，前端真正要做的事情却完全不同。前者主要关注语言本身的正确性、类型系统和普通目标代码生成入口；后者则要识别 kernel 声明、内建线程索引、设备函数属性、Host/Device 代码边界以及启动参数等信息。再进一步，若程序采用 OpenMP Offload、OpenACC、SYCL/OpenCL 或 AI 编译器下沉 IR 这类入口，前端还必须识别更高层的并行语义、数据映射语义或 IR 级模块语义，并把这些内容统一降低到后续流水线能够接得住的形式上。

从整个编译架构看，前端其实决定了后续流水线“看到的世界”。如果前端没有把源程序里的异构含义清楚表达出来，后面的优化器和后端即使再强，也只能在一个已经失真的表示上继续工作。因此，第六章讨论前端时，不能只把它理解为 DCC 的最前面几步，而应把它看成连接“源码意图”和“目标代码质量”的第一道关口。

5.6.2 多语言、多模型入口如何在 DCC 中汇聚

在前端接入层面，DCC 面对的并不是一种语言或一种编程模型，而是一组来源差异很大的输入。C/C++ 通过 Clang 进入，Fortran 通过 Flang 进入，这两类入口负责承接传统通用编程与高性能数值计算代码；HIP 负责显式异构编程场景，OpenMP Offload 和 OpenACC 负责指令式卸载场景，SYCL/OpenCL 则代表另一类面向异构设备的模型入口；而 AI 编译器场景中，上游系统往往已经不再直接交付源码，而是交付 LLVM IR、汇编或更接近后端的中间表示。

从表面看，这些入口的差异极大：有的是高级语言源码，有的是带异构扩展的 C++，有的是编译器导语驱动的并行区域，有的是 DSL 或框架下沉出来的 IR。但 DCC 的目标并不是让每一类输入都走一套完全独立的链路，而是尽量在前端层把差异收敛。也就是说，前端层要做的不是“把所有语言都原样送下去”，而是尽可能把它们转换为统一、可优化、可继续降低的表

示，从而保证后续中端与后端能够工作在一个相对一致的语义平面上。

以几个典型入口为例，前端处理的重点各不相同。对 C/C++
而言，重点在于稳定承接通用代码、运行时组件和主机侧控制逻辑。对 Fortran
而言，重点在于保留数值计算语义、数组与循环结构，以及 HPC 传统代码的编程习惯。对 HIP
而言，重点在于识别 __global__、__device__、__host__、kernel
launch、线程索引与内建执行语义。对 OpenMP Offload 而言，重点在于识别
target、teams、distribute、parallel for、map 等指令，并构造明确的卸载区域和数据映射关系。对 AI
编译器输入而言，重点则从“语言解析”转向“模块接入与目标生成接口”，也就是让已经下沉的 IR
继续进入统一后端链路。

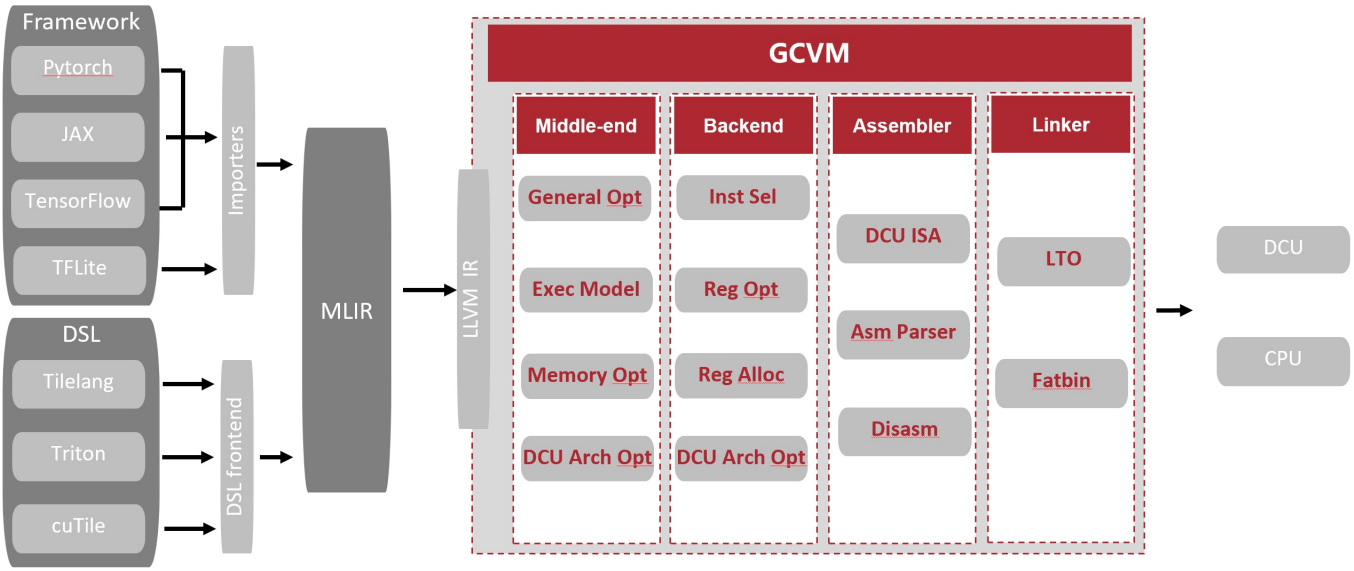
可以把这种汇聚理解成“入口多样，主干统一”。用户层面看到的是不同语言、不同模型的灵活选择；
而 DCC 内部真正追求的是：无论入口差异多大，尽量都能进入统一
IR、统一优化器和统一目标代码生成体系。这也是 DCC 能同时服务通用程序、异构程序与 AI
编译器后端接入的关键前提。

5.6.3 GCVM 的角色：它不是“另一个名字”，而是统一承接层

GCVM 在 DCC 中的角色，不能只理解成“又多了一种 IR”。更准确地说，GCVM 是 DCC
为异构代码生成和 AI
编译器接入建立的一层统一承接面。它位于前端与目标后端之间，向上承接来自不同来源的中间表示，
向下把这些表示交给统一的优化、后端、汇编器与链接器流水线。这个位置非常关键，因为它决定了上
游输入差异能否真正被收敛，下游目标生成能力能否被重复复用。

从整体结构上看，GCVM 不是孤立组件，而是一条完整链路的一部分。前端把
C/C++、Fortran、HIP、OpenMP/OpenACC、SYCL/OpenCL 以及 AI 编译器下沉表示统一送入 GCVM IR
或相关中间表示后，中端继续执行通用优化、执行模型优化与内存优化，后端继续做指令选择、寄存器
优化和机器级调度，汇编器与链接器再负责产物封装、LTO 和最终目标文件组织。也就是说，GCVM
真正承担的是“把多来源输入接入统一后端主干”的职责，而不是单独解决某一个小步骤。

[图片]



它的价值尤其体现在 AI 编译器场景中。对传统源码编译来说，前端到 LLVM IR

的路径已经比较成熟；但对 Triton、JAX、TensorFlow XLA、PyTorch Inductor
这类系统而言，上游真正需要复用的往往不是前端，而是面向目标设备的后端能力。不同框架拥有自己的图优化、调度和算子表达方式，如果每一家都单独维护一套 DCU
目标后端，成本会非常高，也难以跟随硬件和后端能力持续演进。GCVM
的作用，就是把这些上游系统下沉出的 IR 统一接进 DCC
的中后端能力里，让目标相关的优化、汇编生成、HSACO 生成与链接能力集中演进。

因此，GCVM 的意义有两层。第一层是技术上的统一：不同来源的 IR
可以共享同一条目标生成链路。第二层是工程上的降本：上游系统不必重复建设和维护一套完整后端，而可以把精力放在更靠近自身价值的图优化、调度与算子表达上。这种分工结构决定了 DCC
不只是一个面向源码用户的编译器，也是一套可以被更大编译生态复用的后端基础设施。

5.6.4 GCVM 的接口模型与基本使用方式

GCVM
并不是一个只能在架构图里出现的概念，它提供了一套可以直接嵌入上游系统的编译接口。典型流程很清晰：先创建 gcvmProgram，再把 LLVM IR
或汇编模块加入程序对象，然后设置目标架构与优化等级，最后执行编译并获取结果或日志。这个过程把原本分散的目标初始化、模块装载、编译执行、结果提取和日志获取收敛成一组较稳定的 API。

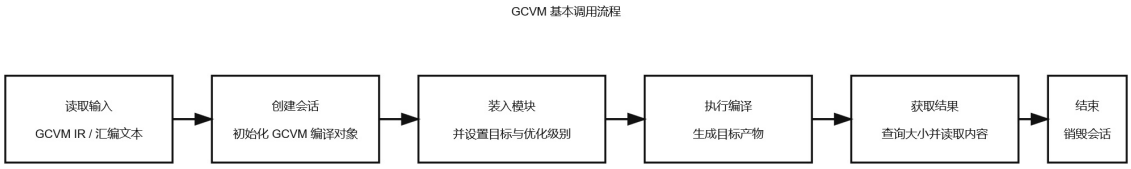
一个最基础的使用示例如下：

```
std::ifstream inputFile(InputFilename);
if (!inputFile.is_open())
  errMsg("Can't open file: " + InputFilename + "\n");

std::string source((std::istreambuf_iterator<char>(inputFile)),
                   (std::istreambuf_iterator<char>()));

gcvmProgram prog;

if (GCVM_SUCCESS != gcvmCreateProgram(&prog))
  errMsg("Error: create program failed\n");

if (GCVM_SUCCESS != gcvmAddModuleToProgram(prog, source.c_str(), source.size(), "",
SourceType::LLVMIR))
  errMsg("Error: add module to program failed\n");

if (GCVM_SUCCESS != gcvmCompileProgram(prog, 0, nullptr, ResultType::Assembly))
  errMsg("Error: compile failed\n");

size_t result_size;
if (GCVM_SUCCESS != gcvmGetCompiledResultSize(prog, &result_size))
  errMsg("Error: get compile result size failed\n");

char *result = (char*)malloc(result_size);
if (!result)
```

```
    errMsg("Error: fail to malloc for result\n");

  if (GCVM_SUCCESS != gcvmGetCompiledResult(prog, result))
    errMsg("Error: get compile result failed\n");

  std::ofstream outFile(OutputFilename, std::ios::binary);
  outFile.write(result, result_size);

  gcvmDestroyProgram(&prog);
```

如果把上面这段示例背后的逻辑压缩成一条更直观的调用链，可以概括为下面这张流程图：

[图片]



如果只需要快速理解 GCVM
的使用方式，可以把它看成一个“面向目标代码生成的后端会话对象”。用户或上游系统先向它装入模块，再配置目标信息，最后让它输出汇编、目标文件或 HSACO
等产物。相比直接在上游系统里拼接大量底层后端逻辑，这种接口模型更稳定，也更便于复用。

在很多场景下，编译不仅需要输入模块，还需要显式指定目标架构和优化级别。 gcvmSetArch()
决定目标架构，gcvmSetOptLevel() 决定编译策略，ResultType::Hsaco 则明确告诉 GCVM
最终要产出设备端可加载对象，而不是汇编文本。也就是说，GCVM 不只是“把 IR
过一遍”，而是提供了面向具体目标产物的稳定出口。

从接口分类看，GCVM 的 API 大体可以按下面几组理解：

生命周期管理：gcvmCreateProgram()、gcvmDestroyProgram()；

目标配置：gcvmSetArch()、gcvmSetOptLevel()；

模块输入：gcvmAddModuleToProgram()、gcvmLazyAddModuleToProgram()；

编译与校验：gcvmCompileProgram()、gcvmVerifyProgram()；

结果与日志获取：gcvmGetCompiledResultSize()、gcvmGetCompiledResult()、gcvmGetProgramLogSize()、gcvmGetProgramLog()；

目标与链接扩展：gcvmLookupTarget()、gcvmCreateTargetMachine()、gcvmRunOptCodegenPipeline()、LTO 相关接口等。

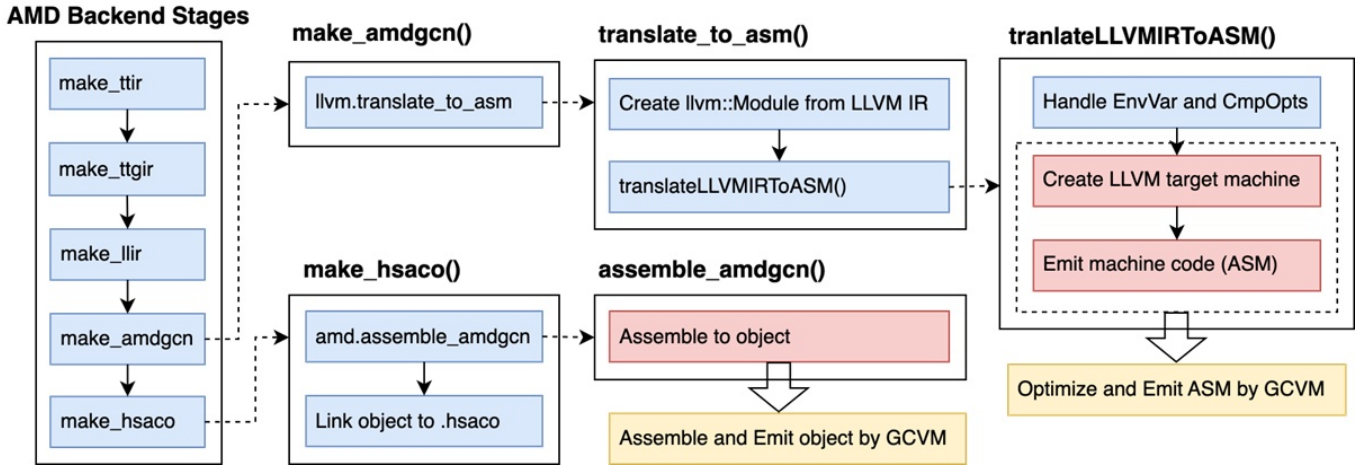
这些接口的设计思路很明确：上游系统不需要关心 DCC 后端内部的所有细节，但它们必须能通过稳定 API 控制“输入什么、朝哪个目标编、输出什么”。这也是 GCVM 能被不同编译系统接入的关键原因。

5.6.5 AI 编译器如何接入 GCVM：以 Triton 与 XLA 路径为例

Triton接入GCVM

1、整体流程说明

[图片]



Triton 接入 GCVM 的整体流程可以理解为在后端原有编译链路中，将部分 LLVM/汇编生成与组装环节替换或适配到 GCVM。

前端流程保持不变，仍然依次经过 make_ttir、make_ttgir、make_llir，生成 LLVM IR 。

随后在 make_amdgcn 和 make_hsaco 阶段，把原本由 LLVM 完成的“生成ASM && 生成HSACO”流程，需要改为由 GCVM 接管。

2、修改示例

2.1– LLVM IR 编译为 汇编（重写translateLLVMIRToASM函数）

```
 gcvmProgram prog;
 gcvmCreateProgram(&prog);
 gcvmSetArch(prog, proc.c_str());

 llvm::SmallVector<char, 0> buffer;
 llvm::raw_svector_ostream stream(buffer);
 module.print(stream, nullptr, false, false);
 gcvmAddModuleToProgram(prog, reinterpret_cast<char*>(buffer.data()), buffer.size(), "hello",
SourceType::LLVMIR);

 std::vector<std::string> ComFlags(flags);
 if (enable_fp_fusion){
  ComFlags.push_back("enable-fp-fusion");
 }
 if (triton::tools::getBoolEnv("TRITON_DEFAULT_ENABLE_NUM_VGPRS512")) {
  ComFlags.push_back("enable-num-vgprs-512");
```

```
}
std::vector<char*> InOptions;
InOptions.reserve(ComFlags.size());
for (const auto& flag : ComFlags) {
  char* buf = new char[flag.size() + 1];
  strcpy(buf, flag.c_str());
  InOptions.push_back(buf);
}

std::string amdgcn;
auto fileType = isObject ? ResultType::Object : ResultType::Assembly;
if (GCVM_SUCCESS == gcvmCompileProgram(prog, numOptions, (const char**) InOptions, fileType)) {
  size_t result_size;
  gcvmGetCompiledResultSize(prog, &result_size);
  char *result = (char*)malloc(result_size);
  gcvmGetCompiledResult(prog, result);
  amdgcn.assign(result, result + result_size);
  delete result;
}
gcvmDestroyProgram(&prog);
return amdgcn;
```

2.2 汇编 编译为 HSACO（重写assemble_amdgcn函数）

```
gcvmProgram asprog;
gcvmCreateProgram(&asprog);
gcvmSetArch(asprog, arch.c_str());

gcvmAddModuleToProgram(asprog, assembly.c_str(), assembly.size(), "hello",
SourceType::ASSEMBLY);

bool res = gcvmCompileProgram(asprog, 0, nullptr, ResultType::Hsaco);
if (res != GCVM_SUCCESS)  {
  return py::bytes();
}

size_t result_size;
gcvmGetCompiledResultSize(asprog, &result_size);
char *result = (char*)malloc(result_size);
gcvmGetCompiledResult(asprog, result);

gcvmDestroyProgram(&asprog);
return py::bytes(std::string(result, result_size));
```

2.3 引入GCVM头文件和库

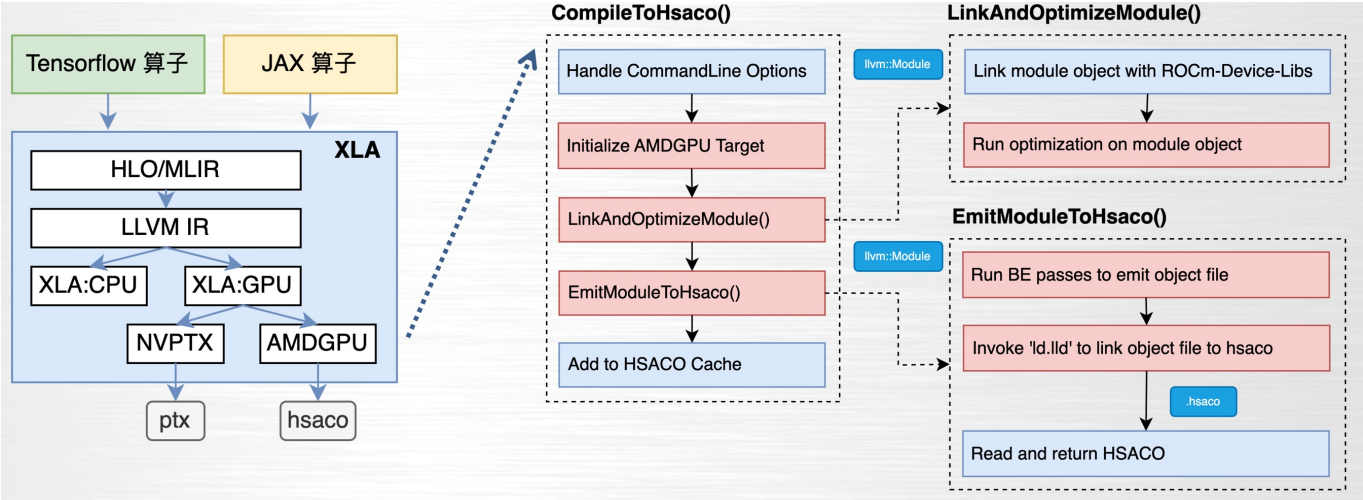
```
CMakeLists.txt
set(__GCVM__ "on")
if(DEFINED __GCVM__ AND __GCVM__)
 add_definitions(-D__GCVM__)
```

```
 if (NOT DEFINED ROCM_PATH )
   set ( ROCM_PATH "/opt/dtk"  CACHE STRING "Default ROCM installation directory." )
 endif ()

 include_directories(${ROCM_PATH}/llvm/gcvm/include/)
endif()
if(DEFINED __GCVM__ AND __GCVM__)
 include_directories(${ROCM_PATH}/llvm/gcvm/include/)
 set(GCVM_LIB ${ROCM_PATH}/llvm/gcvm/lib/libgcvm.so)
endif()
```

Jax/Tensorflow接入GCVM

1、整体流程说明

[图片]



Tensorflow 和 JAX 都是通过 XLA 编译器与 GCVM 库进行适配，XLA 中 AMDGPU 后端提供了 CompileToHsaco() 方法读入 LLVM IR 并输出 HSACO 对象。

GCVM 适配代码修改在 CompileToHsaco() 及其调用的相关方法中。

2、修改示例

2.1 LLVM IR 编译为 HSACO（重写OptimizeAndEmitHsacoByGCVM函数）

```
std::unique_ptr<llvm::raw_fd_ostream> ir_fs(
    new llvm::raw_fd_ostream(ir_path, ec, llvm::sys::fs::OF_None));
module->print(*ir_fs, nullptr);
ir_fs->flush();

std::ifstream input_ir_file(ir_path);
std::string ir_source((std::istreambuf_iterator<char>(input_ir_file)),
                      (std::istreambuf_iterator<char>()));
```

```
gcvmProgram gcvm_prog;
gcvmCreateProgram(&gcvm_prog);

gcvmAddModuleToProgram(gcvm_prog, ir_source.c_str(), ir_source.size(), "", SourceType::LLVMIR);

gcvmSetArch(gcvm_prog, gfx_version.c_str());
gcvmSetOptLevel(gcvm_prog, opt_level);

gcvmCompileProgram(gcvm_prog, 0, nullptr, ResultType::Hsaco);

gcvmGetCompiledResultSize(gcvm_prog, &result_size);
char *result = (char*)malloc(result_size);
gcvmGetCompiledResult(gcvm_prog, result);
std::ofstream out_file(hsaco_path, std::ios::binary);
out_file.write(result, result_size);
out_file.close();

gcvmDestroyProgram(&gcvm_prog);

input_ir_file.close();
```

2.2 引入GCVM头文件和库

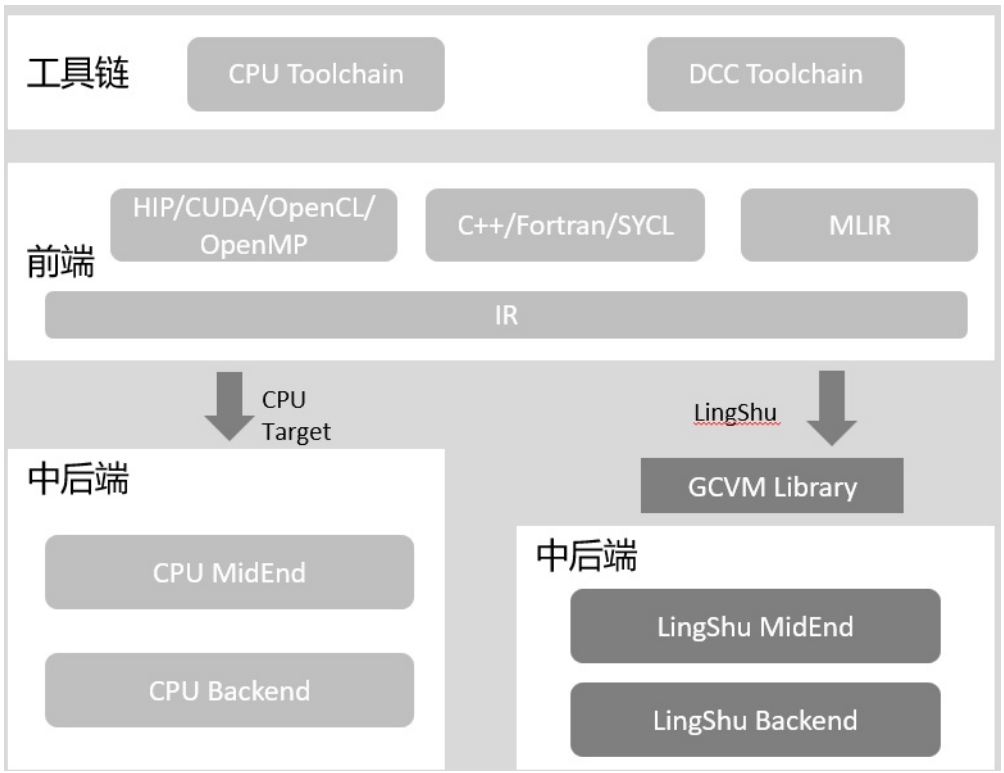
```
workspace_gcvm.bzl:
def workspace():
  native.new_local_repository(
    name = "gcvm_repo",
    path = "/opt/dtk/dcc/gcvm",
  build_file_content = """
cc_library(
  name = "gcvm_lib",
  hdrs = ["include/gcvm.h"],
  srcs = ["lib/libgcvm.so"],
  visibility = ["//visibility:public"],
)
""",
  )

tf_workspace_gcvm = workspace
BUILD:
config_setting(
  name = "use_gcvm",
  values = {"define": "GCVM_ENABLED=true"},
)

cc_library(
  name = "llvm_gpu_backend",
  ......
  copts = select({
```

```
    ":use_gcvm": ["-DUSE_GCVM=1"],
    "//conditions:default": [],
}),
deps = [
......
] + if_rocm_is_configured([
......
]),
]) + select({
    ":use_gcvm": [
    "@gcvm_repo//:gcvm_lib",
    ],
    "//conditions:default": [],
}),
```

5.7 DCC 的 Lingshu 后端

5.7.1 Lingshu 后端在整体编译链中的位置

[图片]



Lingshu 后端是 DCC 面向 DCU
设备进行目标代码生成的核心后端。它位于前端语言处理与设备端机器码生成之间：前端负责把
C、C++、Fortran、HIP、CUDA、OpenMP、OpenACC、SYCL、OpenCL
等输入整理成统一中间表示；中间层负责完成通用优化、循环与内存访问优化，以及一部分与执行模型相关的规整；Lingshu 后端则接管这份中间表示，把它逐步降低为符合 DCU 指令集、寄存器模型、ABI

约束和装载格式要求的机器代码。

从职责上看，Lingshu 后端并不是简单的“IR
到汇编”的最后一步翻译器，而是一个同时理解语言语义、机器资源和运行时约束的落地层。它需要回答一系列非常具体的问题：一个地址应当落在哪个地址空间；一个值应该放在 SGPR 还是
VGPR；一个同步语义应该映射成何种 scope 与 wait 组合；一个 kernel
需要向运行时暴露哪些元数据；一个函数是否会因为寄存器压力过高而降低并发驻留能力。这些决策都会直接影响最终 kernel 的正确性、可装载性和性能表现。

对于使用者来说，Lingshu 后端的意义主要体现在三个层面。第一，它决定了同一份高层程序在 DCU 上最终生成什么样的指令序列。第二，它决定了 kernel 的资源画像，例如 SGPR 数、VGPR 数、LDS 使用量、private segment 大小和 wavefront
配置。第三，它决定了这些资源画像如何与运行时装载模型对接，最终形成可以被调度执行的 code object。因此，第七章的重点不是罗列若干底层术语，而是把“机器模型如何进入编译器”这件事讲清楚。

5.7.2 地址空间与存储层次

Lingshu
后端最基础、也最重要的机器语义之一，就是地址空间。中间表示中的指针并不只是“一个地址”，它还携带“这是谁的地址、在哪层存储中、用什么访存方式访问”的信息。Lingshu
后端通过地址空间编号把这些语义显式保留下来，再根据不同空间的访问规则生成不同类型的目标指令。

Address Space Name	LLVM IR Address Space Number	Segment Name	Hardware Name	Address Size	NULL Value
Generic	0	flat	flat	64	0x0000000000000000
Global	1	global	global	64	0x0000000000000000
Region	2	N/A	GDS	32	not implemented for AMDHSA
Local	3	group	LDS	32	0xFFFFFFFF
Constant	4	constant	same as global	64	0x0000000000000000
Private	5	private	scratch	32	0xFFFFFFFF
Constant 32-bit	6	TODO	-	-	0x00000000

Buffer Fat Pointer	7	N/A	N/A	160	0
Buffer Resource	8	N/A	V#	128	0x00000000000000000000000000000

其中最常见的五类空间分别是 Generic、Global、Local、Constant 和 Private。它们与开发者在编程时接触到的“全局内存、共享内存、常量区、线程私有局部变量”一一对应，但后端关注的不只是语义名称，而是访问代价、可见范围和可使用的指令形式。

Global
表示设备可见的全局虚拟地址空间，典型对象是输入输出张量、显存缓冲区和长期存活的数据结构。它容量最大，但访问延迟通常也最高。Lingshu 在为 global 指针生成代码时，需要考虑 cache 层次、原子访问语义以及不同架构上的访存队列行为。Constant 在地址上与 global 同域，但语义上强调“单次 kernel 执行期间只读不变”。这使后端可以优先使用标量读取路径，把某些本来可能放在 VGPR 访存路径上的读取降成更轻量的标量访存。

Local 对应 LDS，也就是 work-group 级共享存储。它的生命周期与 work-group 一致，work-group 创建时分配，结束时释放；同一 work-group 中的所有 work-item 能看到相同的 LDS 地址，不同 work-group 之间则完全隔离。对 Lingshu 后端来说，Local 的意义不只在于“快”，还在于它强烈影响 barrier 和 wait 的选择：任何跨线程块内部协作、tile 缓存、block 级归约，都会把大量中间数据和同步点压到这一层。

Private 对应 scratch，也就是每个 lane 私有的线程局部存储。许多开发者会把它简单理解为“寄存器不够时的溢出空间”，这并不完全错，但还不够完整。内核中的大局部数组、动态栈、调用栈帧、寄存器 spill 等都可能落入 private segment。其地址宽度通常表现为 32 位 segment 地址，但背后对应的是运行时分配的每 wave scratch backing memory。对后端而言，private 的关键难点在于它并非线性共享内存，而是按 lane 交错映射的。简化理解时，可以认为相同的 private 地址在不同 lane 上会映射到不同的物理位置；当所有 lane 访问同一 private 偏移时，硬件往往会把这些访问组织成相邻的 dword 访问，以提升局部性。

Generic 是一个更值得注意的空间。它不是独立物理存储，而是一个统一视图。Lingshu 后端利用 flat 机制，让一个通用指针在运行时根据地址落点访问 global、local 或 private。这样做的好处是前端语言与中间优化可以用更统一的指针语义表达程序；代价则是后端需要正确处理 aperture、flat scratch 以及 segment 地址与 flat 地址之间的转换逻辑。也正因为如此，generic 指针看似抽象，实际上是后端必须严肃处理的硬件接口。

此外，Region、Buffer Fat Pointer、Buffer Resource 等空间则更多出现在特定场景或底层 intrinsic 中。Region 对应 GDS，属于设备级共享数据存储，使用频率通常低于 global/local/private。Buffer Resource 则是面向资源描述符的非整数指针表示，主要服务于 buffer intrinsic 与资源型访存；它不是普通源码层面会直接书写的地址空间，但在 Lingshu 后端中有明确的表示意义。

从优化角度看，地址空间本身就是一种强约束信息。后端可以据此判断某些值是否可以标量化、某些原子是否能够缩小同步范围、某些 fence 是否只需要工作组级可见性、某些局部数组是否值得尝试寄存器化。因此，地址空间并非“附带元数据

”，而是整个设备端代码生成的第一层语义骨架。

5.7.3 SGPR、VGPR 与资源占用模型

Lingshu
后端采用典型的标量寄存器与向量寄存器双寄存器文件模型。对于后端而言，这不是一个实现细节，而是几乎所有机器级优化的出发点。因为一旦一个值落入 SGPR 还是
VGPR，后续可选指令、活跃区间长度、可驻留 wave 数、spill 风险乃至最终
occupancy，都会随之变化。

SGPR 保存的是 wave 内统一的值，例如 kernel 参数基址、dispatch 信息、work-group
ID、循环不变量、某些统一地址计算结果以及各类 user/system 输入。VGPR 保存的是 lane
私有值，例如 work-item ID、逐线程计算结果、lane 特定地址偏移、条件中间值等。如果说 SGPR
更偏向“控制与公共状态”，那么 VGPR 更偏向“数据通路上的逐线程状态”。

这个区分对代码质量非常关键。如果一个本应 wave-uniform
的值没有被识别出来，而是错误地长期保存在 VGPR
中，那么它不仅多占用一份向量寄存器资源，还可能迫使后续指令走更重的向量执行路径。相反，如果某个值确实存在 lane 差异，却被过早尝试统一化，就会引入额外的 lane
交换或控制流修复成本。因此，Lingshu 在机器指令选择之前、过程中以及之后，都会不断尝试改进 uniform 分析与寄存器类别选择。

除了通用 SGPR 和 VGPR 外，后端还必须处理若干特殊寄存器或特殊状态，例如 EXEC 执行掩码、VCC 条件寄存器、FLAT_SCRATCH、与 private segment
相关的系统寄存器、以及不同架构上用于传递初始执行状态的保留寄存器组。这些寄存器虽然不总是直接暴露给源码，但它们决定了分支如何收敛、private memory
如何寻址、函数调用如何建立栈帧，以及某些特定 lowering 是否可行。

从资源角度看，寄存器数量不是单纯的“越多越好”。单个线程使用更多
VGPR，通常意味着它可以把更多中间结果保留在寄存器中，减少 spill
和访存；但与此同时，每个执行单元能容纳的并发 wave 数会下降。单个 wave 使用更多 SGPR
也有类似效果：它可能减少重复计算，却可能压缩驻留空间。当寄存器使用量越过某些阈值，硬件能够同时容纳的 wave 数就会减少，隐藏访存延迟和执行延迟的能力也会下降。Lingshu
后端在寄存器分配阶段的本质任务，就是在“单 wave 更强”与“多 wave 并发更高”之间寻找平衡点。

寄存器溢出是这个平衡失控时最常见的表现。VGPR 溢出通常会转化为额外的 scratch 访问，表现为更多 private segment 占用和额外的 load/store；SGPR
相关压力也会引发额外的保存、恢复或更复杂的指令序列。由于这些访问最终会落到片外或半片外资源上，性能退化往往非常明显。因此，在阅读 dccobjdump
或其他资源报告时，sgpr_count、vgpr_count、private_segment_fixed_size、vgpr_spill_count
等字段常常能快速说明一个 kernel 的主要瓶颈在哪一层。

5.7.4 初始执行状态与 wavefront 执行模型

Lingshu 后端面向的执行模型，可以概括为“以 wavefront 为基本调度单元的 SIMT/SIMD
融合执行”。源码中的 thread/work-item 是逻辑概念，硬件调度时真正进入执行单元的是 wave。一个

wave 中包含一组同时执行相同指令流的 lane，哪些 lane 当前有效，则由 EXEC 掩码控制。也就是说，后端处理控制流时，核心任务不是为每个线程单独生成完全独立的分支，而是把条件分支重写成掩码切换、路径执行与汇合恢复的组合。

对 Lingshu 来说，wavefront 模型至少带来四类直接影响。

第一，线程索引不是普通局部变量，而是硬件启动 wave 时写入初始寄存器状态的一部分。运行时在 kernel dispatch 过程中，会根据 kernel descriptor 的要求设置用户 SGPR、系统 SGPR 和系统 VGPR，例如 kernarg 指针、work-group ID、work-item ID、dispatch 指针、queue 指针等。后端必须保证机器代码对这些寄存器的期望与 descriptor 中声明的启用位一致，否则 kernel 即使能被链接，也会在装载或执行时出现语义错误。

第二，控制流发散会转化成掩码控制成本。源程序中的 if/else、循环退出条件、早停逻辑，在机器级并不是“某线程跳走、某线程继续”，而是不同 lane 的活跃位被写入和恢复。分支越发散，掩码管理、路径串行执行和汇合修复的成本越高。因此，Lingshu 后端会尽量在机器级保留或重建 uniform 条件，把 wave 内一致的分支转化成更轻的控制流形式。

第三，同步范围必须显式与执行层级对应。很多高层程序只写了 barrier、atomic 或 fence，但在后端看来，真正需要回答的是：同步发生在 wave 内、work-group 内、agent 范围还是 system 范围？不同范围决定需要什么样的 wait 指令、cache 失效或写回序列，以及是否必须触及 LDS、L0/L1/L2 等层级。

第四，wave 内通信是非常重要的优化基础。对于归约、scan、broadcast、ballot、投票、lane 间重排等模式，Lingshu 后端通常优先尝试生成跨 lane 通信指令，而不是把中间结果先写入 LDS 再同步读取。前者往往以寄存器和执行单元内部网络完成，延迟更低，也更容易规避额外 barrier。

在具体架构上，wavefront 大小、部分资源字段的编码方式、以及同步与调度细节都可能有差异。对开发者而言，不需要记住所有架构细节，但必须理解一条原则：DCC 中大量“看似语言级”的并行语义，实际上都是在 Lingshu 后端中被翻译成 wave 级机器行为的。只要理解 wave 是基本执行单位，很多性能现象就会更容易解释。

5.7.5 Code Object 与装载信息

Lingshu 后端生成的 Code Object，本质上是一个可装载的设备端 ELF 对象。它不是只保存指令文本的普通目标文件，而是把设备端机器代码、运行时需要的装载信息，以及与 Kernel 入口相关的资源描述统一收在一起。对 DCC 的异构编译流程来说，设备端真正进入运行阶段时，依赖的就是这份对象：前面所有 lowering、寄存器分配、资源计算和入口组织，最后都要落在这里。

从结构上看，一个完整的 Code Object 至少包含三层内容。第一层是 .text 中的设备端机器指令；第二层是 metadata，负责描述参数布局、段大小和资源画像；第三层是 kernel descriptor，负责把这些静态结果翻译成启动时真正要使用的配置。Lingshu 后端的职责，就是保证这三层信息彼此一致：指令怎么取参，metadata 就必须怎样描述参数；metadata 暴露了怎样的资源需求，descriptor 就必须给出相应的启动约束。只要其中任意一层失配，问题就不再只是“性能不理想”，而可能直接表现为装载失败、参数解释错误或运行行为异常。

因此，Code Object

不能简单理解成“设备端可执行文件”。它更像是设备端交付物的最小完整单元：既包含真正要执行的内容，也包含让运行时能够正确装载、传参、分配资源并启动 Kernel 的说明信息。对使用者来说，这种设计最直接的外在表现就是：很多平时在源码里看不到的约束，例如 kernarg 大小、LDS 固定占用、private segment 开销、wavefront 大小、寄存器使用量，最终都会在这份对象中留下明确痕迹。

在这些信息里，metadata 更像“运行时先读的说明书”。运行时在提交 dispatch 之前，需要先知道参数段有多大、是否需要固定的 group/private 段、这个 Kernel 的资源画像大致如何；静态分析工具之所以能够展示参数列表、资源占用和部分执行属性，也正是因为这些内容在 metadata 中有结构化记录。像 .kernarg_segment_size、.kernarg_segment_align、.group_segment_fixed_size、.private_segment_fixed_size、.sgpr_count、.vgpr_count、.wavefront_size、.max_flat_workgroup_size 以及 .args 这样的字段，本质上都在回答同一类问题：这个 Kernel 需要怎样的外部调用环境，运行时又该为它准备什么。

其中最容易被低估的是参数布局信息。很多人会把 kernarg 看成“只是把参数打包一下”，但对后端和运行时来说，它其实是一条严格的调用协议。参数的顺序、大小、对齐、地址空间语义，以及某些指针参数在设备侧应当被解释成 global、local、constant 还是 generic，都必须在 metadata 中说清楚。只有这样，运行时在准备参数段时才能和生成出来的机器代码保持一致。也就是说，metadata 不是额外附带的调试信息，而是装载链路本身的一部分。

如果说 metadata 负责“说明这是什么”，那么 kernel descriptor 负责“规定如何启动它”。descriptor 更接近真正的入口配置块：它会给出固定的 group/private 段大小、参数段大小、入口偏移、资源控制字段，以及若干与初始执行状态有关的使能位。运行时在把 Kernel 送上设备之前，最终要依赖这些信息去建立入口上下文。对后端来说，这一步的意义在于把内部分析结果真正落到可启动的格式上；对调试和调优来说，这一步的意义在于，很多原本只存在于编译器内部的决策，到了这里都会被固化成可观察、可对照的启动配置。

descriptor 中最值得关注的内容通常可以按三类理解。第一类是段大小和入口定位，例如 GROUP_SEGMENT_FIXED_SIZE、PRIVATE_SEGMENT_FIXED_SIZE、KERNARG_SIZE、KERNEL_CODE_ENTRY_BYTE_OFFSET，它们决定从哪里开始执行，以及启动前至少要为这个 Kernel 预留多少固定资源。第二类是资源控制字段，例如 COMPUTE_PGM_RSRC1/2/3 一类配置，它们把寄存器粒度、执行模式、work-group 组织方式和若干硬件行为选项固化下来。第三类是初始状态使能项，也就是要不要在入口时提供 dispatch 指针、queue 指针、kernarg 指针、work-group id、work-item id 以及 private segment 相关上下文。这些位看起来像一组零散开关，实际上决定了 Kernel 的入口协议是否完整成立。

在版本上，当前最值得重点理解的是 Code Object V5。它的重要性不在于“字段更多了”，而在于 metadata 与 descriptor 之间的对应关系比早期版本更完整、更直接。以前有些信息只能从资源字段里间接推断，有些又只在高层描述里出现；到了 V5，像动态栈、uniform work-group 大小等信息，可以在 metadata 和启动配置里找到更明确的落点。对使用者来说，这意味着通过 dccobjdump 看到的静态结果，与运行时实际装载和后端最终决策之间，更容易建立直接映射。

V4 → V5 至少有四个具体变化点。

一是 metadata 版本号继续前移，amdhsa.version 从 1.1 变成 1.2。这个变化本身并不重要，重要的是它对应着一批新键和新语义正式进入规范，而不再只是后端内部

约定。

二是 kernel metadata 新增了一批直接影响运行行为的字段。比如 .uses_dynamic_stack 与 .uniform_work_group_size：前者明确告诉工具和运行时，这个 kernel 不是只有固定的 private segment，而是真的会在执行过程中使用动态栈。早期版本里，遇到动态栈时，PRIVATE_SEGMENT_FIXED_SIZE 往往只能保守地给大，外部只能看到“private 很大”，却不容易区分这是静态局部对象、spill，还是动态栈导致的；V5 之后，这件事可以直接读出来。后者则明确表达 kernel 是否要求 global size 在每一维上都能被 work-group size 整除。这个条件过去往往来自前端属性、语言规则或者运行时假设，V5 之后可以直接在 code object metadata 中查询到。对 OpenCL 一类可能出现最后一个 work-group 为 partial 的场景，这一点尤其有区分度。

三是 V5 扩展了 kernel 参数的 value_kind，新增了 hidden_block_count_x、hidden_block_count_y、hidden_block_count_z 这一类隐藏参数类型。它们传递的是“非 partial work-group 的数量”，而不是简单的 grid size。这个变化很具体，也很重要，因为它意味着 V5 不只是多记了几个属性，而是把一部分过去只能靠隐式参数约定传递的调度信息，正式纳入了 kernarg 语义体系。对需要处理尾块、分块调度或语言层隐藏参数的场景，V5 的表达能力明显比 V4 更完整。

四是 descriptor 层终于把动态栈和预取参这些信息真正落成了可启动配置，而不只是高层描述。最典型的就是 USES_DYNAMIC_STACK 位在 V5 及以后才正式使用；另外，KERNARG_PRELOAD_SPEC_LENGTH 和 KERNARG_PRELOAD_SPEC_OFFSET 这类预加载字段，虽然与硬件能力相关，但从 V5 开始，隐藏参数通过预加载进入 User SGPR 的路径变得更正式、更容易和 metadata 对应。换句话说，V5 的价值不只在于“多了几个 metadata key”，而在于 metadata 和 descriptor 的两层信息终于开始说同一种话。

从工程角度看，理解 Code Object 的意义，并不是为了手工去拼装这些字段，而是为了知道设备端产物到底由什么构成。很多问题之所以难排查，就是因为大家只盯着最终 ELF 或可执行文件，却没有意识到设备端真正的交付单位不是“某段汇编”，而是一份同时携带代码、元数据和入口配置的完整对象。只有把这层关系看清楚，后面的静态分析、运行时装载理解和资源调优才会真正连起来。

5.7.6 资源预算、occupancy 与寄存器调优

内核性能分析中，大家经常会同时看到“寄存器更多以后 spill 变少了”和“寄存器更多以后 occupancy 反而下降了”这两种现象。它们并不矛盾，而是资源预算的两个侧面。Lingshu 后端在做寄存器分配时，实际上一直在回答一个问题：为了让每个线程更强，愿意牺牲多少并发驻留能力；反过来，为了保住更多并发驻留，又愿意接受多少额外的重计算或 spill。

在当前工具链中，单线程可用 VGPR 数量通常不是一个绝对固定值，而会受到编译器假设的最大 work-group 规模影响。work-group 越大，编译器越需要为更高并发做预算，于是单线程能够放心使用的寄存器上限通常就越低。反之，如果最大 block 规模更小，后端往往更愿意把更多中间值保留在寄存器里。

一个常见调节点是：

```
-mllvm -enable-num-vgprs-512=true
```

该选项打开后，后端会在特定目标上采用更高的 VGPR 上限假设，从而缓解高寄存器压力 kernel 的 spill 问题。在某些机型上，它对大 block kernel 的帮助会更明显，因为原本随着 block 规模增大而迅速下降的单线程寄存器预算，可以得到一定程度的抬升。经验上可以把它理解为：用更宽裕的向量寄存器空间，换取更少的 scratch 访问。

参考上限可以概括为下表：

每个 Block 线程最大数目	默认寄存器预算	开启 -mllvm -enable-num-vgprs-512=true 的预算
256（默认）	256	256
512	128	256
1024	64	128

但要强调，这张表更适合作为调优方向，而不是绝对的性能结论。因为真实收益仍然取决于 kernel 当前瓶颈是在寄存器、LDS、global memory，还是在并发隐藏能力本身。如果一个 kernel 本来就主要受全局内存带宽限制，单纯提高寄存器预算未必能带来收益；如果一个 kernel 因 spill 导致 private segment 爆炸，则适当增大寄存器预算往往非常有效。

除了编译开关，更常见的做法是通过源码属性和编译选项显式约束 work-group 规模。典型手段包括：

方式一：使用 amdgpu_flat_work_group_size 属性

示例源码如下：

```
#include <hip/hip_runtime.h>

__attribute__((amdgpu_flat_work_group_size(1, 256)))
__global__ void saxpy_kernel(const float* x,
          const float* y,
          float* out,
          float a,
          int n) {
 int tid = blockIdx.x * blockDim.x + threadIdx.x;
 if (tid < n) {
  out[tid] = a * x[tid] + y[tid];
 }
}
```

这段源码表达的含义是：编译器可以假设这个 kernel 的扁平 work-group 大小会落在 1 到 256 之间。对于 Lingshu 后端，这样的约束会产生两类影响。第一，它会影响 max_flat_workgroup_size 一类的资源边界估算。第二，它会影响寄存器使用策略，因为后端不再需要为更大的 block 规模预留过于保守的资源预算。

它的优点是作用精准、可读性强；缺点是如果项目里 kernel 很多，逐个加属性会增加维护工作量。

方式二：使用 __launch_bounds__ 属性

示例源码如下：

```
#include <hip/hip_runtime.h>

__global__ __launch_bounds__(256, 2)
void reduce_kernel(const float* in, float* out, int n) {
 __shared__ float smem[256];

 int tid = threadIdx.x;
 int gid = blockIdx.x * blockDim.x + threadIdx.x;

 smem[tid] = (gid < n) ? in[gid] : 0.0f;
 __syncthreads();

 for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
  if (tid < stride) {
   smem[tid] += smem[tid + stride];
  }
  __syncthreads();
 }

 if (tid == 0) {
  out[blockIdx.x] = smem[0];
 }
}
```

这里的 __launch_bounds__(256, 2) 可以理解为：该 kernel 预计每个 block 最大 256 个线程，并希望在每个执行单元上保留至少一定数量的并发驻留能力。Lingshu 后端会据此调整寄存器分配策略，避免单线程寄存器使用过度膨胀，从而破坏并发驻留目标。

与 amdgpu_flat_work_group_size 相比，__launch_bounds__ 更像是一种“调度与资源联合提示”，不仅告诉编译器 block 大小，还告诉编译器并发目标。

方式三：使用编译选项统一约束

如果不希望修改源码，或者希望在工程、CI、实验脚本中快速批量比较不同 work-group 假设，可以直接在编译命令行上给出统一上界。常见写法如下：

```
dcc -O3 --offload-arch=gfx926 -gpu-max-threads-per-block=256 vector_add.cpp -o vector_add
```

或者在构建系统中写成：

```
target_compile_options(my_kernel_target PRIVATE
 -gpu-max-threads-per-block=256
)
```

这种方式的最大优点是：不改源码、适合批量实验、方便在持续集成或自动调优脚本中反复切换参数。

不过，它也有明显局限：它是编译配置级别的统一假设。如果一个工程里不同 kernel 的最佳 block 上界并不相同，那么只用单个全局选项可能会“一部分 kernel 受益，另一部分 kernel 受损”。因此，当性能调优已经收敛到少数热点 kernel 时，通常还是源码级属性更精确。

这些设置本质上都是在告诉后端：这个 kernel
未来会以什么样的线程块规模运行，因此资源预算应该以什么边界条件来估算。max
设得越小，编译器通常越敢让单线程使用更多寄存器；但如果运行时实际配置与编译假设不匹配，就可能导致兼容性或性能预期偏差。因此，这类约束应当基于真实 launch 形态，而不是凭经验随意收紧。

5.7.7 Lingshu 后端中的典型优化

Lingshu 后端除了通用的代码生成流程之外，还会承接一些更贴近 DCU
执行特征的专门优化。它们通常不是简单的 peephole 规则，而是围绕原子操作、跨 lane
通信、局部数组落点和资源画像所做的针对性改进。

1) atomic接口统一

原子操作是最容易在前端、IR
和后端之间出现实现分裂的区域之一。不同数据类型、不同地址空间、不同硬件能力，往往会让前端头文件层面充满条件分支。Lingshu
的一个重要方向，就是把这类分散实现统一收敛到更标准、更可优化的 IR 原子语义上，例如
atomicrmw 和
cmpxchg。这样一来，前端接口层只负责表达“要做什么原子语义”，而不再承担过多目标能力分派。

Lingshu 的做法，是尽量把这种前端分叉收敛成统一形式。例如，从用户视角看，源码里可能是不同的 atomic API；但进入中间层之后，会尽量统一表达为 atomicrmw、cmpxchg 等标准 IR
原子操作，再根据类型、地址空间和同步语义进入后续
pass。这样一来，编译器就能在更靠后的阶段集中回答这些关键问题。第一，atomic
的地址空间是什么，落在 global、local 还是 generic。第二，它的内存序是 relaxed、acquire、release 还是更强。第三，当前目标架构对这种数据类型和操作组合是否有直接硬件支持。

统一链路之后，后续处理通常会沿着“识别与规范化 → 原子优化 → 必要时展开 → 目标
lowering”这条路线进行。这样做最大的好处，是把原本散落在前端头文件中的目标判断，下沉到了更适合做这件事的编译阶段。接口层只需表达语义，而后端负责判断能力与生成代码，整个职责边界会更清晰。

对使用者而言，这类统一优化虽不直接可见，但可以通过两个现象观察到它的价值。其一，atomic
相关问题更容易在 IR 和后端阶段定位，而不是在复杂头文件分支里追踪。其二，当工具链扩展对
i64、double
等类型的支持时，不必为每类前端接口单独补一套实现，只需要在统一原子链路中补足对应 lowering 与优化逻辑即可。

从工程维护角度看，这也是非常重要的一步。因为 atomic
是跨语言、跨运行时、跨目标架构都会反复出现的基础能力。如果没有统一链路，后续每增加一种类型、一个 builtin
或一个地址空间变体，复杂度都会近似乘法式增长；而一旦先统一到标准语义，再在后端集中处理，这个复杂度就会被显著压平。

统一之后，后端就可以把原子相关处理收敛到更稳定的几步：先进行目标相关 atomic
识别与优化，再做必要的 atomic expand，最后根据地址空间、同步范围和目标能力完成

lowering。这样做的收益主要有三点。第一，接口层实现更简单。第二，后端更容易集中处理同类模式。第三，增加新类型支持时，不必在多个前端分支里重复修改。当前对 double 和 i64 等类型的扩展，就属于这一思路下的自然结果。

2) 基于跨 lane 通信的热点 atomic 优化

当同一个 wave 中多个 lane 同时对同一地址做 atomicAdd、atomicMin、atomicMax 等操作时，如果每个 lane 都直接发起全局 atomic，就会形成热点冲突，最终表现为原子通路串行化。Lingshu 在某些模式下会先识别“这些 atomic 实际上可以先在 wave 内合并”，再利用 shuffle、permute、wave reduction 等跨 lane 通信，把多个 lane 的更新先归约成一个值，最后由单个 lane 发起一次 memory atomic。

这个优化能否成立，关键在于编译器要先证明两个事实：第一，多个 lane 访问的是同一个目标地址，或者至少能归并成少数几个热点地址；第二，atomic 操作本身满足可在 wave 内先做部分归约的数学性质，例如加法、最小值、最大值、某些计数型更新等。只有当这些条件成立时，Lingshu 才能安全地把“多 lane 各自原子更新”改写成“wave 内先做寄存器归约，再由一个代表 lane 提交结果”。

为了帮助理解，可以把原始模式简化成下面这样的逻辑：

```
// 每个 lane 都对同一地址 counter 做一次 atomicAdd(1)
atomicAdd(counter, 1);
```

如果一个 wave 中 64 个 lane 都执行这句，而它们访问的又是同一个 counter，那么最朴素的结果就是 64 次 global atomic。优化后，逻辑会更接近：

```
// wave 内先统计有多少 lane 需要更新
int wave_sum = wave_reduce_sum(1);

// 只让一个 lane 发起真正的原子更新
if (lane_id == 0) {
  atomicAdd(counter, wave_sum);
}
```

真实机器级实现当然比这个示意更复杂，还要处理活跃掩码、lane 条件、返回值语义以及不同 atomic 操作类型，但核心思想就是一致的：用 wave 内通信替代部分片外冲突。

这一优化特别适合以下几类场景。第一，直方图、桶计数、热点累加。第二，稀疏图算法中的热点顶点更新。第三，一些归约后写回单地址的统计型内核。第四，内部大量线程对少量共享计数器做更新的场景。

它之所以重要，是因为这类场景的瓶颈往往不在算术本身，而在“同一地址上的 atomic 冲突”。如果仍然让所有 lane 直接访问 global atomic 单元，那么 ALU 再快也帮不上忙；而跨 lane 通信大多发生在寄存器级或片上执行网络中，代价明显更低。因此，只要模式可识别，先归约再提交通常都会比直接高冲突 atomic 更划算。

不过，这个优化也有适用边界。若 atomic 返回值被每个 lane 立即用于后续不同逻辑，或者不同 lane 实际访问的地址并不一致，那么可合并空间就会显著下降。也就是说，跨 lane atomic

优化并不是“看到 atomic 就做”，而是一个对模式匹配、数据依赖和语义保持要求都很高的后端优化。

这个思路的本质，是用片上寄存器交换替代一部分片外原子访问。因为跨 lane 通信通常远比高冲突 global atomic
便宜，所以在热点统计、计数、归约类场景中，收益往往很直接。它特别适合那些“很多线程在做相同地址更新，但每次更新本身很轻”的内核。在这类 kernel
中，真正拖慢速度的不是算术，而是原子冲突；因此，减少 atomic 次数比进一步优化 ALU
指令更有效。

3) Kernel 局部变量数组优化

较大的局部数组也是 DCC 中很典型的后端调优主题。一个 kernel 中如果存在 float a[38]、float b[38] 这样的局部数组，编译器通常无法把它们完全保存在寄存器中。当元素数超过可寄存器化阈值时，这些对象就会转移到 LDS 或 scratch/private
memory。这样做虽然保证了正确性，但会带来更高的访问延迟、更大的
private_segment_fixed_size，以及热点路径上的重复访存。

这个问题之所以常见，是因为局部数组在源码层看起来很自然：写起来像普通局部变量，逻辑上也清晰；但在设备端，它们往往会迅速变成资源压力源。数组一旦无法完全标量替换，编译器就必须为它找到可寻址的存储位置。若对象是线程私有的，通常会优先落到
private/scratch；若分析结果允许，也可能放入其他更合适的存储层。但无论落点如何，只要离开寄存器，访存成本就会显著上升。

可以用一个简化示例来理解这个现象：

```
__global__ void stencil_kernel(float* out, const float* in, int n) {
  float window[64];

  int tid = blockIdx.x * blockDim.x + threadIdx.x;
  if (tid >= n) return;

  #pragma unroll
  for (int i = 0; i < 64; ++i) {
    window[i] = in[tid + i];
  }

  float sum = 0.0f;
  #pragma unroll
  for (int i = 0; i < 64; ++i) {
    sum += window[i];
  }

  out[tid] = sum;
}
```

如果后端无法把 window[64] 的大部分元素寄存器化，那么这段代码的热点路径就会多出大量 scratch 读写。尤其是在循环展开后，逻辑上看似“只是多了些局部缓存”，机器级却可能演变成大量 private memory 访问。

Lingshu
提供局部数组寄存器化阈值调节，本质上就是在告诉后端：对于一定规模以内的局部数组，可以更积极地尝试标量替换、寄存器分配或更激进的局部展开。单 kernel 级别可以通过属性控制：

Lingshu 针对这一问题提供了比较直接的调优入口。如果是单个热点 kernel，可以使用：

```
__attribute__((dcu_max_array_size(64)))
__global__ void example_kernel(...) {
  // ...
}
```

这个属性用于提示编译器提高局部数组寄存器化阈值，使数组在不超过阈值时更倾向于进入寄存器。对于工程级统一调优，则可以通过：

```
-mllvm -dcu-max-array-size=N
```

统一设置阈值，便于批量测试不同配置对性能、寄存器占用和 private segment 的影响。

这里要特别注意一个常见误区：不是所有局部数组都应该强行寄存器化。阈值升高后，虽然 scratch 访问可能减少，但 VGPR 占用也可能快速上升，进而压低
occupancy。也就是说，局部数组优化本质上是在“更多寄存器”与“更少访存”之间重新分配成本，它与前面讲过的资源预算问题是同一个系统里的两面，而不是独立开关。

在热点 kernel
上，这类优化往往非常有效，因为局部数组通常位于核心计算循环里，被高频反复访问。一旦把高频 scratch
访问换成寄存器访问，收益会直接反映在延迟和吞吐上；但在非热点路径或已经高度寄存器压力的 kernel
中，收益则未必稳定。因此，这个优化最适合结合静态资源分析与实际运行性能一起做，而不是仅凭“数组大就应该寄存器化”的直觉调整。

5.7.8 后端 lowering 与机器代码生成流程

从统一中间表示到最终 code object，Lingshu 后端内部还要经历一条相当长的 lowering 流程。虽然不同版本的具体 pass
顺序会调整，某些优化也会持续演进，但总体逻辑比较稳定：先把高层表示中与目标无关的语义，逐步压缩成目标相关的地址空间、调用约定、内建操作和资源约束；再进入机器层，对寄存器、调度、等待关系和启动信息做反复收敛；最后把这些结果固化成汇编、对象文件以及可装载的 code object。

首先是 IR 到目标语义的
lowering。这个阶段通常还没有真正进入最终机器指令，但已经开始把平台无关表示改写成 Lingshu 能理解的形式。例如，地址空间信息会被进一步具体化，某些通用 intrinsics
会被转成更贴近目标的内建形式，kernel 参数、隐式参数、work-item/work-group
相关访问，也会逐步从“语言级语义”转成更接近 ABI
的访问模式。可以把这一步理解为：先把前端和中端留下的高层语义，整理成“后端以后真正要处理的目标问题集合”。如果这里做得不充分，后面的指令选择与机器优化往往会被迫在更别扭的表示上工作。

然后进入指令选择阶段。这里的核心任务，是把平台无关的 IR

操作映射为目标平台支持的机器操作。某些 IR
节点可以直接一一映射，某些则需要先展开成更细粒度的机器模式，另一些还需要根据子目标特性选取不同变体。这个阶段不仅决定“生成什么指令”，也决定“哪些值会被标量化、哪些地址计算会被折叠、哪些 intrinsic
会被转成专用机器模式”。例如，同样是一次内存访问，在这个阶段就已经可能分化成更偏标量路径的读取、偏向量路径的读取，或者带有特定地址空间含义的访问模式。很多后续性能表现，其实在这里就已经埋下了伏笔。

指令选择之后，代码会进入机器 SSA
与机器级优化阶段。此时代码已经开始带有明确的目标寄存器类别和机器指令框架，但寄存器还未最终落到物理位置，因此后端仍有一定空间去做机器层面的合并、消除、传播与局部重写。这个阶段的目标通常不是追求“看起来更短”，而是让后面的调度与寄存器分配更容易得到好结果。比如缩短某些中间值的活跃范围、减少不必要的拷贝、让某些可折叠寻址在机器层面真正折进去，都会直接影响后续 SGPR/VGPR 压力。

接下来是机器级调度，而且通常不止一次。前置调度更多关注减少寄存器压力、缩短关键活跃区间、避免过早拉长依赖链；后置调度则更多关注在寄存器已经固定之后，如何减少流水线停顿、改善等待计数器行为、平衡不同类型的访存与算术指令。对于 DCU
这类延迟隐藏高度依赖并发和调度质量的体系结构，调度质量往往直接决定 kernel
是否能把理论吞吐转化成实际吞吐。很多时候，指令条数并没有变化，但只是把加载、算术、同步与等待的相对位置重新组织，最终性能就会出现明显差异。

然后是寄存器分配。这个阶段要把大量虚拟寄存器映射到有限的物理 SGPR/VGPR
空间中，并在必要时产生 spill。寄存器分配之后，后端才能知道真实的
sgpr_count、vgpr_count、是否需要额外 scratch、某些保留寄存器最终落在什么位置，以及 kernel descriptor
中相关字段应该如何填写。也就是说，前面很多阶段做的其实还是“候选决策”，而寄存器分配会把这些决策压缩成最终可执行结果。某个 kernel
在寄存器分配前看起来结构很好，不代表最后不会因为寄存器不足而产生
spill；反过来，前面若已经尽量缩短活跃区间、减少无效复制，寄存器分配阶段就更有机会维持高质量结果。

寄存器分配之后，后端通常还要做一轮更贴近真实机器状态的收尾工作。这里会涉及 frame lowering、prolog/epilog 插入、scratch/stack 相关设置、等待指令与 hazard
处理，以及某些必须在物理寄存器确定后才能做的修正。对 kernel 而言，这一步尤其关键，因为前面 7.4 和 7.5 提到的很多“初始执行状态”和 kernel descriptor
字段，最终都要在这里与真实机器代码对应起来：是否需要 private segment，是否要建立 flat scratch，哪些 SGPR/VGPR 假定在入口已就位，是否需要额外的 prolog 来处理 kernarg preload 或动态栈，这些都不是抽象概念，而是此时要真正落进代码和启动配置里的内容。

最后是汇编生成、目标文件封装和链接。Lingshu 会输出符合目标 ABI
的汇编或对象文件，再由后续工具完成链接，形成可装载的设备端产物。在这一阶段，kernel descriptor、metadata
note、符号、段布局、对齐和重定位信息都会被一起整理进对象文件中，使之既能被分析工具读取，也能被运行时直接装载。换句话说，机器码本身只是最终产物的一部分；如果没有与之匹配的 descriptor、metadata 和对象布局，这段机器码依然不能算“真正可运行的设备端交付物”。

因此，所谓“后端流程”并不是几个教科书式名词的串联，而是一条不断在“语义正确、资源受限、执行高效”三者之间权衡的过程。每个阶段看起来都只负责一小部分工作，但最终它们共同决定了设备端代码的形态；而真正理解 Lingshu 后端，也往往不是死记 pass

名称，而是理解这些阶段各自解决什么问题、又把什么约束传递给了下一阶段。

5.8 DCC 的调试与诊断

DCC
相关问题通常集中在四类：第一类是“命令能跑，但用错了工具链”；第二类是“编译阶段没按预期进入 Host/Device
双路径”；第三类是“编出来了，但产物结构不对”；第四类是“程序能运行，但运行结果、卸载行为或性能不对”。下面分别给出更接近一线使用的场景和命令。

场景一：明明装了新版本 DCC，实际调用的还是旧版本。
这是共享服务器和多人环境里最常见的问题之一。典型现象是：同一套命令在两台机器上行为不同，或者你明明刚切换了 DTK
版本，但日志里出现的路径还是旧目录。此时第一步不要重新编译，而是先看版本和默认注入参数：

```
dcc --version
dcc --short-version
dcc --cxxflags
dcc --ldflags
```

如果 –version 输出的版本不对，或者 –ldflags
里出现了旧目录中的库路径，问题通常就不在源码，而在环境本身。最常见的原因包括：没有重新 source env.sh、PATH 顺序里旧版本排在前面、CI 脚本里写死了旧 DTK 路径、或者集群模块系统切换后 shell 会话没有刷新。这个场景下，继续改 -O2/-O3
或继续怀疑后端都没有意义，先把当前会话真正绑定到预期工具链上才是关键。

场景二：纯 C/C++ 能编过，HIP/OpenMP Offload 却行为异常。
这种情况通常不是“编译器坏了”，而是异构参数没有进入正确阶段。最直接的查法是打开 Driver 展开日志：

```
HIPCC_VERBOSE=1 dcc -O3 --offload-arch=gfx926 hip_demo.cpp -o hip_demo
```

看这类输出时，重点不是一行一行追，而是抓三件事：有没有明显的设备端编译动作、–offload-arch=gfx926
有没有真正出现在设备端相关命令里、最终链接阶段有没有把设备端产物打包进去。如果展开结果里根本看不到设备端路径，通常说明参数没有进对阶段、源码没有被按异构模式解释，或者某个编译开关被前面的构建系统吞掉了。

场景三：OpenMP Offload 写了 target，但程序看起来还是在 CPU 上跑。
这类问题尤其容易误判成“OpenMP
性能差”。更实用的做法是先用较小样例和详细日志确认卸载是否真的发生：

```
HIPCC_VERBOSE=1 dcc -O2 -fopenmp --offload-arch=gfx906 omp_demo.cpp -o omp_demo
```

如果展开日志里没有设备端相关步骤，或者生成阶段根本没有出现 offload
目标，那么问题往往出在编译命令本身，而不是运行时。如果编译阶段看起来没问题，但运行时仍像在主机上执行，则需要继续检查环境、运行库和目标架构是否匹配。这个场景里最常见的错误并不是 OpenMP 语法写错，而是少了 -fopenmp、少了 –offload-arch，或者节点硬件和目标架构不一致。

场景四：编译通过，但想确认某个 Kernel 到底有没有被打包进去。
这时不必靠猜，也不要只看最终可执行文件名。更直接的方法是先保留中间产物，再对最终产物做静态分析：

```
dcc -O3 --offload-arch=gfx926 --save-temps hip_demo.cpp -o hip_demo
dccobjdump --inputs=hip_demo
```

这个组合特别适合查三类问题：第一，设备端中间文件到底有没有生成；第二，最终可执行文件里是否真的包含预期 Kernel；第三，产物中的目标架构到底是不是你指定的那个。如果 –save-temps 之后根本没有设备端中间文件，问题大多出在前面阶段；如果中间文件有，但 dccobjdump 看不到设备端对象或目标信息异常，则问题更可能在打包、链接或产物组织阶段。

场景五：同一份程序在一台机器上能跑，在另一台机器上运行时报库找不到。
这种问题在异构环境里非常常见，尤其是多人共享服务器、容器和集群节点。最典型的表现是：编译成功，启动时却报动态库找不到、设备端加载失败、或者不同节点上运行结果不一致。这个时候，先不要改源码，而要优先看环境：

```
HIPCC_VERBOSE=2 dcc -O2 main.cpp -o app
```

HIPCC_VERBOSE=2 更适合看环境相关信息，比如 HIP_PATH、HSA_PATH、平台识别以及部分运行时上下文。若再结合 dcc –ldflags 一起看，通常就能判断当前会话到底准备从哪些目录加载库。这个场景里，最常见的根因是 LD_LIBRARY_PATH 顺序不对、节点上软链接没有切换到同一版本、某个用户自己的 shell 启动脚本额外追加了旧路径，或者不同节点实际安装的 DTK 并不一致。

场景六：程序能编过也能运行，但结果和预期不一致。 这种问题不应直接从高优化等级开始查，因为 -O3 下同时叠加了前端、优化器和后端多个变量。更稳妥的查法，是先退回一个可观察基线：

```
dcc -O0 -g --save-temps main.cpp -o app_dbg
```

这样做的目的很实际：保留调试信息、减少优化重排、保留中间产物。若 -O0 下结果正确、-O3 下结果异常，才更值得继续怀疑优化相关行为；若 -O0 下结果也不对，那么问题更可能在源码、环境、链接或运行输入上，而不是优化器。这个场景里，很多人一开始就开最高优化级别反复试，结果把问题层次混在了一起，反而更难定位。

场景七：程序慢，但不知道慢在编译结果还是运行时。
对这类问题，最有效的办法通常不是只看总耗时，而是静态和动态一起看。一个实用组合是先静态拆产物，再动态看行为：

```
dccobjdump --inputs=app
dccprof --omp-trace --trace-level=4 ./app
```

前一个命令适合看目标架构、Kernel 静态形态、资源占用和产物结构；后一个命令适合看 OpenMP 区域、Kernel 调用和数据传输到底把时间花在了哪里。比如，有些程序慢并不是 Kernel 本身差，而是因为数据映射太频繁；有些程序静态上看设备端代码没问题，但动态上大量时间都消耗在同步和搬运上。若没有 dccprof，开发者很容易误把运行时问题当成后端代码生成问题。

场景八：怀疑构建系统把参数吞掉了。 这类问题在 CMake、Makefile 或脚本层层包装之后很常见。你以为自己传了 –offload-arch=gfx926、-fopenmp 或某个 -mcpu，但真正到 DCC 时已经被脚本改写或遗漏。此时最省时间的办法不是逐层读构建脚本，而是直接看原始入参与最终展开：

```
HIPCC_VERBOSE=4 dcc -O3 --offload-arch=gfx926 kernel.cpp -o app
HIPCC_VERBOSE=1 dcc -O3 --offload-arch=gfx926 kernel.cpp -o app
```

前者帮助确认“用户到底传了什么”，后者帮助确认“Driver
最终执行了什么”。这两个级别配合起来，最适合排查构建系统吞参、改参和重复注入的问题。

如果只想记一条最实用的排障顺序，可以直接按下面的经验流程走：先用 dcc –version、dcc –cxxflags、dcc –ldflags 确认当前工具链身份；再用 HIPCC_VERBOSE=1/2/4
看命令展开、环境和原始入参；若问题仍不清楚，就加 –save-temps
看中间产物；若怀疑产物结构异常，就用 dccobjdump
拆最终结果；若程序已经能跑但性能或卸载行为不对，再上 dccprof
看动态行为。这个流程最大的好处，是每一步都有明确观察对象，不需要一开始就从“编译器内部很复杂”出发空想原因。

最后需要强调的是，DCC
的调试与诊断不应被理解为“出问题时临时查几个开关”。对于长期维护的异构工程，更合理的做法是把这些诊断动作前移并固化：把版本信息和关键环境变量写入 CI
日志，把关键构建命令的展开结果纳入问题复现记录，把最小编译验证和最小异构验证作为节点验收的一部分。只有当诊断信息被持续记录下来，很多原本难以复现的“偶发构建问题”才会真正变成可追踪、可解释、可修复的工程问题。

第六章 使用DCU开发AI应用

近年来，随着人工智能技术的迅猛发展，DCU芯片在性能上取得了显著的突破，在硬件参数上与海外芯片的差距日益缩减。但在某些特定应用场景下，DCU仍需进一步优化和提升其计算能力，需要支持更多的软件框架和编程语言，以及需要支持持续更新迭代的软件版本，以满足不同用户的需求。DAS是针对海光DCU推出的一套完整的人工智能基础软件栈，包含框架工具层、扩展组件层、基础算子层。产品兼容CUDA软件生态并在各层面做了一系列编译、优化和快速部署，能够使能DCU芯片的特性，充分发挥DCU的算力。帮助企业用户快速构建基于DCU的AI应用和业务。同时，DAS采用敏捷的开发模式，快速响应需求变化，以支持不断更新的框架组件版本，帮助用户缩短推出解决方案的时间。

企业在DCU上进行AI应用开发最核心的两个阶段是模型训练和推理，训练阶段包括数据准备、模型构建、训练、验证与调优、测试与评估，推理阶段包括模型部署、输入处理、正向传播。整个过程中，缺乏完善的DCU软件生态系统和软件工具链，驱动、计算库、通信库、AI框架、组件、模型之间的兼容性差以及软件版本的快速变化，增加了开发和部署的难度，让企业开发者面临诸多挑战，需要投入更多的时间、人力和计算资源，大大增加了应用开发的成本。因此，企业迫切需要在每个阶段使用经过优化的软件来构建生产就绪型AI 应用。

为了支持用户基于DCU进行模型的高效训练，DAS在训练侧提供TensorFlow/PyTorch/PaddlePaddle等主流深度学习框架支持，帮助用户在DCU上进行数据处理、模型构建、高效训练等、验证调优等。针对通用模型和大模型训练的计算访存特点，采用多维组合并行策略提升模型的数据吞吐量，采用激活重计算策略，在不牺牲训练速度的前提下，大幅度降低显存占用，从而允许在更小的硬件上训练更大的模型。另外还采用了topo优化、数据并行、模型并行、流水线并行、ZeRO显存优化等，这些策略可以单独使用也可以组合使用，以应对不同规模和复杂度的模型训练任务，实现大模型场景的全面覆盖。

为了支持用户基于DCU进行模型的高效推理，DAS在推理侧提供vLLM、lmdeploy等主流推理引擎支持，帮助用户在dcu上进行模型量化部署、大规模数据集处理和高效推理。vLLM通过PageAttention高效管理kv内存、连续批处理和分布式推理等技术手段提高了推理速度和效率；而lmdeploy则通过动态Shape、

图优化、OP融合和ONNX导入等功能模块提供了高效性和灵活性的推理服务。用户可以根据具体的应用场景和需求选择合适的推理引擎。推出了推理加速工具组件LLMSlim，从剪枝、稀疏、量化等多种角度提供基于DCU的最优LLM推理优化策略，提供了丰富的轻量化工具链以及轻量化推理计算支持模块，并与Lmdeploy和VLLM框架深度结合，可直接对用户应用进行优化加速支持。另外，还在kv cache、连续批处理、pageattention、flashattention、flashdecoding等方向进行了针对性的适配优化。这些优化策略能够解决LLM模型高计算需求，高储存成本、高延迟等应用瓶颈，降低推理开销并加速推理过程，降低模型的首字延时病提高吞吐能力。

依托DAS，用户能快速构建基于DCU的AI应用和业务，支持DCU在人工智能的全领域应用，如自然语言大模型、多模态大模型、MOE大模型、视觉大模型、通用模型、语音模型等，支持模型训练及推理落地，实现快速的DCU人工智能应用迁移、开发及迭代。

6.1 光源平台

光合开发者社区的光源平台是专门为DCU自主算力生态打造的一站式 AI 开发与资源托管平台。它是光合开发者社区的核心板块，旨在通过高度集成的软硬件适配，降低国产加速卡的使用门槛，，提升团队协作效率。

光源平台通过提供五大核心资源库，构建了从算力到底层软件，再到顶层应用的全链路支撑体系：

- AI 框架镜像仓库：预置了深度适配DTK（DCU ToolKit）的 PyTorch、TensorFlow、飞桨（PaddlePaddle）等主流框架镜像。开发者可以直接获取已配置好底层驱动和算子库的容器，实现“开箱即用”。
- AI 模型仓库：提供了一系列经过海光 DCU 深度优化、验证过的成品模型，包括最近热门的 DeepSeek-V3、R1 系列模型。这些模型确保了在国产硬件上的运行稳定性和推理/训练性能。
- 代码仓库：托管模型实现的源代码和AI组件源码等，支持开发者在 DCU 硬件环境上进行深度的算法调优和二次开发。
- AI 数据集仓库：提供标准化的数据集和评估框架，特别是针对大模型的通用智能评估，帮助开发者快速完成模型的精度对标与验证。
- 技能库 (Skill Base)：面向智能体（Agent）开发，提供可复用的工程化技能模块，覆盖从自动化编码到复杂业务流的端到端场景。

平台价值

- 高度兼容性：基于海光 DCU 的“类 CUDA”架构（兼容 ROCm 生态），光源平台让原本运行在 NVIDIA GPU 上的 AI 项目可以几乎无缝地迁移到国产算力平台上。
- 软硬一体化优化：平台集成了海光自研的 DTK 软件栈，针对 DCU 的硬件特性对数学库、编译器和通信库进行了专项优化。
- 支持国产大模型生态：目前，光源平台已成为国产大模型（如 DeepSeek 等）在海光 DCU 上落地的主要分发阵地，助力政务、金融、智算中心等领域实现自主可控的 AI 升级。

** 使用指南**

光源平台是专为海光 DCU 开发者打造的资源宝库，平台访问地址：https://developer.sourcefind.cn/sourcefind。通过以下四个步骤，您可以快速完成从环境搭建到模型部署的全流程。

第一步：获取适配环境（镜像仓库）

在 DCU 上开发的第一步是解决底层驱动与框架的兼容性。

1、进入 “镜像仓库” 板块。

2、根据您的需求选择预装了 DTK (DCU ToolKit) 的官方镜像（支持 PyTorch、TensorFlow、PaddlePaddle 等）。

3、直接复制 docker pull 命令，在您的计算节点上拉取环境，实现“开箱即用”。

第二步：选择优化模型（模型仓库）

为了发挥海光算力的最佳性能，建议使用经过官方调优的模型。

1、访问 “模型仓库”，筛选您需要的任务类型（如 NLP、CV 或大语言模型）。

2、查看模型详情页，获取在海光 DCU 上的性能表现数据。

3、点击 “立即使用” 或查看关联的 “源码仓库”，获取适配后的代码实现。

第三步：调用标准化数据（数据集仓库）

1、在 “数据集仓库” 中，您可以根据任务类型下载标准化的测试集。

2、利用平台提供的评估框架，对您的模型进行“通用智能”测试，确保模型在国产算力上的精度与预期一致。

第四步：开发智能体应用（技能库）

如果您正在开发 AI Agent（智能体）：

1、进入 “技能库” 搜索可复用的组件。

2、参考编码、测试或工程化场景的技能模板，将这些技能集成到您的智能体工作流中，提升开发效率。

6.2 深度学习基础环境的安装及部署

在使用海光 DCU 执行训练或推理任务前，请按照以下方式准备基础环境：

1.快速部署（推荐）：镜像仓库

访问 光合开发者社区-光源-镜像仓库，直接选择预装了 DTK (DCU ToolKit) 及常用 AI 框架（如 vLLM、PyTorch、TensorFlow、PaddlePaddle）的官方容器镜像。这是最快捷的“开箱即用”方式。

2.手动安装：驱动与工具链

若需获取底层组件，请访问 光合开发者社区-资源中心，下载 驱动 (Driver) 和 DCU Toolkit (DTK) 等基础环境安装包。

3.生态包获取：DAS 平台
如需下载特定的 AI 生态组件或框架版本，可通过合开发者社区-资源中心-DAS，下载python安装包。

[图片]

图6-1-1 光合社区DAS资源下载

6.3 使用PyTorch训练分类模型

6.3.1 PyTorch简述

6.3.2 PyTorch安装

6.3.3 训练分类模型

6.3.4 如何进行分布式训练

6.3.5 基于DCU的PyTorch性能分析

6.4 使用TensorFlow训练分类模型

6.4.1 TensorFlow简述

1.什么是Tensorflow

TensorFlow 是由 Google Brain 团队开发并于 2015
年正式开源的端到端机器学习框架。它以”张量（Tensor）在计算图（Graph）中流动（Flow）”的核心思想命名，支持从研究原型到生产部署的完整机器学习生命周期。

TensorFlow
提供了丰富的工具、库和社区资源，使研究人员和开发者能够轻松地构建和部署机器学习驱动的应用程序。其核心优势体现在以下几个方面：

- 灵活性：支持从低级数值计算到高级神经网络 API 的全栈操作
- 可扩展性：可在单台设备、集群乃至超算上无缝运行
- 生产就绪：内置 TensorFlow Serving、TensorFlow Lite 等部署工具
- 生态丰富：拥有 Keras、TensorBoard 等完整的配套工具链

2. 核心概念

(1)张量(Tensor)

张量是 TensorFlow 中数据的基本单位，可以理解为多维数组。根据维度不同，张量分为：

0 阶张量（标量）：单一数值，如 3.14; 1 阶张量（向量）：一维数组，如 [1, 2, 3];2
阶张量（矩阵）：二维数组，如图像的灰度表示;N 阶张量：N 维数组，如彩色图像批次 [batch, height, width, channels].

在代码中，使用 tf.constant 或 tf.Variable 可以创建张量：

```
import tensorflow as tf

# 创建常量张量
scalar = tf.constant(3.14)              # 标量
vector = tf.constant([1.0, 2.0, 3.0])      # 向量
matrix = tf.constant([[1, 2], [3, 4]])     # 矩阵

# 创建变量张量（可训练参数）
weights = tf.Variable(tf.random.normal([128, 64]))
```

(2)计算图(Computation Graph)

TensorFlow 2.x 默认采用动态图（Eager Execution）模式，运算结果立即返回，便于调试。同时，通过 @tf.function 装饰器可将 Python 函数编译为静态计算图，显著提升执行效率。

```
@tf.function
def matrix_multiply(a, b):
  return tf.matmul(a, b)
```

3.TensorFlow 2.x 架构概览

TensorFlow 2.x 在 1.x
基础上进行了全面重构，以「用户友好、灵活强大、生产就绪」为核心设计目标。整体架构采用分层设计，从底层硬件抽象到上层用户 API

形成清晰的五层结构，开发者可以根据任务需求在不同层级自由选择切入点。

层级	主要组件	功能定位
应用与工具层	TensorBoard / TensorFlow Hub / Model Garden / TFX	可视化、模型共享与 ML 流水线程化工具
高级 API 层	tf.keras（Sequential / Functional / Subclassing）	用户最常直接使用的建模接口，置训练循环
中级 API 层	tf.data / tf.distribute / tf.Module / tf.function	数据管道、分布式训练、模块化件与计算图编译
低级 API 层	tf.GradientTape / tf.linalg / tf.random / tf.nn	自动微分、线性代数运算、底层经网络原语
硬件抽象层	CPU / GPU（CUDA）/ TPU / 边设备（TFLite）	跨硬件统一执行，无需修改上层码

1. 高级 API 层：tf.keras

tf.keras 是 TensorFlow 2.x 的官方高级 API，也是绝大多数用户的主要入口。Keras 在 TF 2.x 中被深度集成（而非外部依赖），提供三种建模风格以覆盖不同复杂度的需求：

- Sequential API：线性堆叠层结构，语法最简洁，适合入门与标准任务
- Functional API：支持多输入/多输出、共享层、残差连接等复杂拓扑
- Model Subclassing：继承 tf.keras.Model 实现完全自定义的前向传播逻辑

三种 API 可以混合使用，且与 tf.data、tf.distribute 等中级 API 无缝配合。Keras 内置的 model.fit()
训练循环封装了梯度下降、回调函数、指标计算等大量工程细节，极大降低了使用门槛。

Sequential API示例：

```
model = tf.keras.Sequential([
  tf.keras.layers.Dense(128, activation='relu', input_shape=(784,)),
  tf.keras.layers.Dropout(0.3),
  tf.keras.layers.Dense(10, activation='softmax')
])
```

2. 中级 API 层：数据、分布式与计算图

中级 API 层为高级 API 提供关键基础设施支撑，主要包含三个模块：

(1)tf.data —— 高性能数据管道

tf.data 提供声明式的数据加载与预处理
API，支持惰性求值、并行预处理和内存预取，是构建高吞吐量训练管道的核心工具。其典型使用链路如下：

```
dataset = tf.data.Dataset.from_tensor_slices((x, y))
dataset = (dataset
  .shuffle(buffer_size=10000)  # 随机混洗
```

```
    .map(augment_fn,          # 并行数据增强
         num_parallel_calls=tf.data.AUTOTUNE)
    .batch(64)                # 批处理
    .prefetch(tf.data.AUTOTUNE)) # 异步预取，与 GPU 计算重叠
```

(2)tf.distribute —— 分布式训练策略

tf.distribute.Strategy
提供统一的分布式训练接口，封装了设备间梯度同步、数据分片等复杂逻辑。只需用策略的 scope 包裹模型构建，训练代码几乎无需改动即可扩展到多 GPU 或多机器。

(3)tf.function —— 静态图编译加速

通过 @tf.function 装饰器，TensorFlow 会将 Python 函数追踪并编译为 XLA 优化的静态计算图，在保留 Eager 调试便利性的同时获得接近 TF 1.x 的执行效率。

```
@tf.function(jit_compile=True)  # 启用 XLA 编译
def train_step(x, y):
    with tf.GradientTape() as tape:
        pred = model(x, training=True)
        loss = loss_fn(y, pred)
    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))
    return loss
```

3.低级 API 层：自动微分与数学运算

低级 API 层提供对计算过程的精细控制，核心是 tf.GradientTape 自动微分机制。GradientTape 记录前向计算过程中的所有操作，并在反向传播时自动计算梯度，是实现自定义训练循环、元学习（Meta-Learning）等高级研究场景的基础工具。

```
x = tf.Variable(3.0)
with tf.GradientTape() as tape:
    y = x ** 2 + 2 * x + 1     # y = (x+1)^2
dy_dx = tape.gradient(y, x)     # dy/dx = 2x + 2 = 8.0
```

此外，低级层还包含：

- tf.linalg：矩阵分解、线性方程求解等线性代数运算
- tf.random：各类随机分布的张量生成
- tf.nn：卷积、池化、激活函数等神经网络原语
- tf.signal：FFT、短时傅里叶变换等信号处理函数

4.应用与工具层：生态配套

TensorFlow 围绕核心框架构建了完善的生态工具链，覆盖从实验到部署的全生命周期：

工具	用途	典型场景
TensorBoard	训练过程可视化	实时监控 loss/accuracy 曲线、算图结构、权重分布

TensorFlow Hub	预训练模型共享	一行代码加载 BERT、EfficientNet 等主流预练模型
TFX（TF Extended）	ML 生产流水线	数据验证、特征工程、模型分、自动部署一站式管理
TF Serving	模型线上推理	高并发 REST/gRPC 接口，支持型热更新
TF Lite	移动与边缘部署	模型量化压缩，部署到手机、嵌式设备、MCU
TF.js	浏览器端推理	在 Node.js 或浏览器中直接运行 TensorFlow 模型

5.Eager Execution 与 Graph Mode 的统一

TensorFlow 2.x 的一项重要设计决策是将动态图（Eager Execution）设为默认模式，同时通过 @tf.function 保留静态图的性能优势，实现了「开发体验」与「生产性能」的统一：

维度	Eager Mode（默认）	Graph Mode（@tf.function）
执行方式	逐行即时执行，结果立即返回	构建计算图后批量优化执行
调试体验	可直接 print/断点调试	调试相对复杂，需 tf.print
执行性能	有 Python 解释器开销	XLA 编译优化，速度显著提升
适用阶段	实验探索、模型开发	训练加速、生产部署

6.4.2 TensorFlow安装

- ** pip安装**

在光合开发者社区(https://download.sourcefind.cn:65024/)下载对应版本的tensorflow whl包。

裸机pip安装：

```
conda create -n tf_env python=3.10
conda activate tf_env
pip install /path/to/tensorflow*whl
```

容器内直接安装：

```
pip install /path/to/tensorflow*whl
```

- ** 直接拉取对应tensorflow镜像**

在开发者社区(https://developer.sourcefind.cn/servicelist)拉取可直接使用tensorflow环境的镜像。

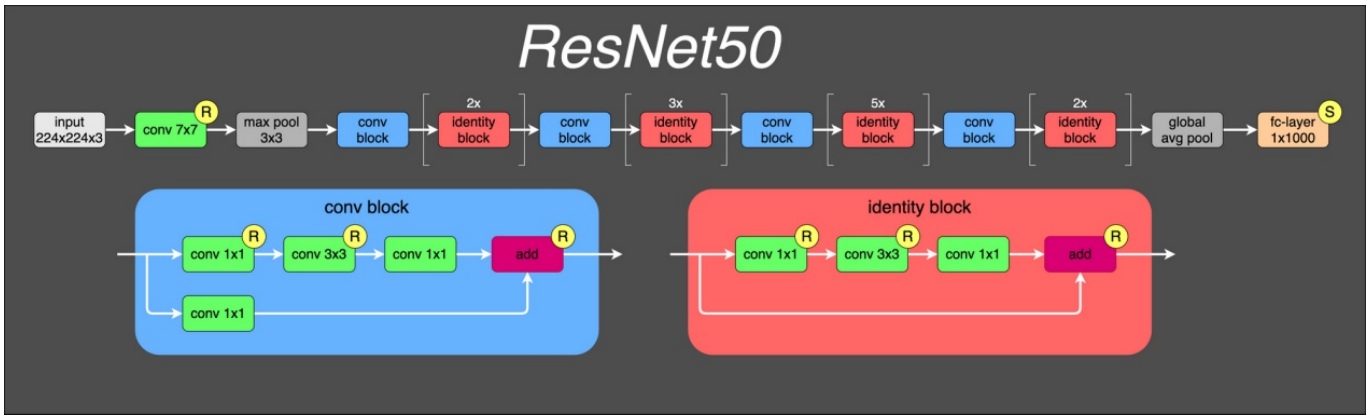
镜像实例化：

```
docker run -it --network=host --name=容器名  --restart=always --privileged --device=/dev/kfd
--device=/dev/dri --ipc=host --shm-size=16G --group-add video --cap-add=SYS_PTRACE -v
/opt/hyhal:/opt/hyhal:ro 镜像名称 /bin/bash
```

6.4.3 训练分类模型

以ResNet50网络为例，展示下如何使用tensorflow环境运行分类模型：

[图片]



- ** 拉取以ResNet50代码**

拉取对应代码到工作目录：

```
git clone -b master http://developer.sourcefind.cn/codes/modelzoo/resnet50_tensorflow.git
```

- ** 准备数据集**

(1)真实数据

使用ImageNet数据集，并且需要转成TFRecord格式
ImageNet数据集可以官网(https://image-net.org/)下载，ImageNet数据集转成TFRecord格式，可以参考以下script和README制作完成的TFRrecord数据形式如下：

```
tfrecord-imagenet
|
train-00000-of-01024
train-00001-of-01024
...
train-01022-of-01024
train-01023-of-01024
validation-00000-of-00128
validation-00001-of-00128
...
validation-00126-of-00128
validation-00127-of-00128
```

(2)合成数据

基于随机合成的数据，不需要下载ImageNet数据集，执行网络训练时只需要把程序执行语句中的–use_synthetic_data设置为true即可。

- ** 训练**

在ResNet50代码目录下，运行以下训练命令：

(1)单机单卡训练指令

```
export PYTHONPATH=/home/resnet50_tensorFlow:$PYTHONPATH
export HIP_VISIBLE_DEVICES=0
# 可选：开启 XLA 图优化以提升性能（如不需要可注释此行）
export TF_XLA_FLAGS="--tf_xla_auto_jit=1"

python3 official/legacy/image_classification/resnet/resnet_ctl_imagenet_main.py \
 --data_dir=/path/to/ImageNet-tensorflow_data_dir \
 --model_dir=/path/to/model_save_dir \
 --batch_size=128 \
 --num_gpus=1 \
 --use_synthetic_data=false \
 --train_epochs=90 \
 --dtype=fp16
```

(2)单机多卡训练指令

```
export PYTHONPATH=/home/resnet50_tensorFlow:$PYTHONPATH
export HIP_VISIBLE_DEVICES=0,1,...,N  # 指定使用的 GPU 编号
export TF_XLA_FLAGS="--tf_xla_auto_jit=1"

python3 official/legacy/image_classification/resnet/resnet_ctl_imagenet_main.py \
 --data_dir=/path/to/ImageNet-tensorflow_data_dir \
 --model_dir=/path/to/model_save_dir \
 --batch_size=$((128 * N)) \    # batch_size 随卡数线性扩展
 --num_gpus=N \
 --use_synthetic_data=false \
 --train_epochs=90 \
 --dtype=fp16
```

6.4.4 如何进行分布式训练

以两机和以ResNet50代码为例展示多机训练：

- ** 同步好环境**

(1)统一多机的软件环境和ssh配置

推荐使用 Docker 容器统一各节点环境（参考 2.3.2 中的镜像部署方式），确保所有节点的 TensorFlow 版本、Python 版本、驱动版本完全一致。

配置ssh：

```
# 生成 SSH 密钥对
ssh-keygen -t rsa
```

```
# 将本机公钥追加到 authorized_keys
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys

# 开启 sshd_config 中的 AuthorizedKeysFile（取消注释）
vim /etc/ssh/sshd_config
# 确认以下行已取消注释：
# AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

# 启动 SSH 服务（指定端口，容器内使用自定义端口避免冲突）
echo "/usr/sbin/sshd -p 6589" >> ~/.bashrc
source ~/.bashrc
```

将所有节点的公钥相互交换，验证连通性：

```
# 在每个节点上测试 SSH 连接
ssh <对端节点IP> -p 6589
```

- ** 多机训练**

(1)ResNet50代码代码调整

进行以下四处改动以支持多机训练：

```
# 改动 1：注入多机配置到主训练脚本
sed -i '100 r configfile' official/legacy/image_classification/resnet/resnet_ctl_imagenet_main.py

# 改动 2：修改训练脚本中的 data_dir 路径
# 编辑 scripts-run/single_process.sh，将 data_dir 修改为实际数据路径

# 改动 3：配置 hostfile（所有参与训练的节点 IP）
# 示例内容：
# 192.168.1.1 slots=8
# 192.168.1.2 slots=8

# 改动 4：配置 nodefile（仅填写当前节点的 IP）
# 示例内容：
# 192.168.1.1
```

(2)启动多机训练

```
chmod +x scripts-run/single_process.sh

mpirun \
 --allow-run-as-root \
 --bind-to none \
 -mca btl self,tcp \
 --hostfile hostfile \
 -np 16 \                    # 总进程数 = 节点数 × 每节点卡数（如 2节点×8卡=16）
 -mca plm_rsh_args "-p 6589" \    # 使用自定义 SSH 端口
 scripts-run/single_process.sh
```

6.5 大模型DCU训练

6.5.1 Megatron-LM简述

i. 框架的定位与演进
Megatron-LM 是由 NVIDIA 开源的大规模 Transformer
模型训练框架，专为突破单机显存与算力瓶颈、实现千亿至万亿参数语言模型的高效分布式训练而设计。该框架于2019年随论文* 《Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism》 *首次公开，系统验证了张量并行（Tensor
Parallelism，TP）在超大模型训练中的可行性。历经多次迭代，Megatron-LM
已从早期的单库脚本演化为模块化核心库 megatron-core（mcore），并深度集成至 NVIDIA NeMo
生态，成为工业界与学术界训练 GPT、BERT、T5、ViT 等架构的事实标准之一。

ii. 核心并行策略
Megatron-LM
通过将模型参数、计算图与数据在多个正交维度切分，构建了高度可扩展的分布式训练拓扑。随着模型架构从稠密向稀疏（MoE）演进，其并行体系已从经典的3D并行扩展为支持多模态与条件计算的N-D并行。核心策略如下：

[图片]

	并行维度	切分对象	
2	张量并行 (TP)	将FFN与Attention的权重/激活沿矩阵维度切分（Column/Row Parallel）	节点内
3	流水线并行 (PP)	按Transformer层切分模型，分配至不同节点/阶段	节点间
4	数据并行 (DP)	复制完整模型副本，各副本处理不同数据批次	
5	序列/上下文并行 (SP/CP)	沿序列长度维度切分长文本	局部
6	专家并行 (EP)	将MoE模型中的不同专家（Experts）分配至不同 GPU	

点击图片可查看完整电子表格

并行组合策略：现代千亿/万亿参数模型通常采用TP+PP+DP+EP混合拓扑。TP负责节点内高带宽计算，PP负责跨节点显存分摊，DP保障数据吞吐，EP则专用于 MoE路由分发。Megatron-LM通过统一的ProcessGroup管理与拓扑感知的网格映射（Mesh
Mapping），自动求解最优并行度配比，实现计算、显存与网络带宽的帕累托最优。

iii. 关键底层优化技术
Megatron-LM 通过系统级协同优化显存、通信与计算，核心模块如下：

i. 显存与精度优化

- 激活卸载：异步卸载非关键激活至CPU，结合预取与分层缓存，支持更大Batch或更长序列。
- 分布式优化器：按DP秩分片存储优化器状态，替代完整副本以显著降低显存占用。
-

低比特混合精度：原生适配FP8/FP4（Hopper/Blackwell），支持动态校准与专用算子，在无损精

度下减半显存与带宽开销。
•
选择性检查点：仅对高消耗层（Attention/FFN）保存激活，其余反向重计算，精细权衡显存与时间。
ii. MoE稀疏架构优化
• Grouped
GEMM：聚合多专家GEMM请求为单次Kernel调用，消除碎片化计算，提升SM利用率与访存局部性。
•
高效All-to-All：集成DeepEP等拓扑感知通信原语，优化拥塞控制与带宽调度，消除不规则路由的通信热点。
• 通信计算重叠：通过CUDA
Stream与异步NCCL，将Token路由通信与专家计算交错执行，隐藏通信延迟。
iii. 通信与计算图优化
• CUDA Graph：预捕获稳定计算图以固化Kernel序列，消除解释器开销与动态Launch延迟。
• 全局重叠调度：将梯度同步、参数更新与前向计算绑定至独立Stream，实现细粒度流水线并行。
•
算子融合与FlashAttention：深度融合LayerNorm/Dropout等算子；集成FlashAttention-2/3，降低Attention显存复杂度，突破长序列训练瓶颈。
Megatron-LM以”底层极致优化+架构灵活扩展”为核心，通过N-D混合并行（TP/PP/DP/EP/SP）、分布式优化器、通信-计算重叠及低比特适配，系统性突破显存墙与通信瓶颈。作为模块化、硬件感知的训练底座，其为千亿/万亿参数预训练、长上下文对齐及异构集群弹性训练提供高吞吐、低延迟的工程级解决方案，持续推动大模型训练向高效、经济、可扩展方向演进。

作为AI训练栈的核心底座，Megatron-LM（megatron-core）已发展为模块化、硬件感知的分布式训练事实标准。无论是千亿/万亿参数基座预训练、超长上下文对齐、多模态架构融合，还是跨地域异构集群的弹性训练，该框架均能提供高吞吐、低延迟、强容错的工程级解决方案，持续推动大模型训练向更高效、更经济、更易扩展的方向演进。

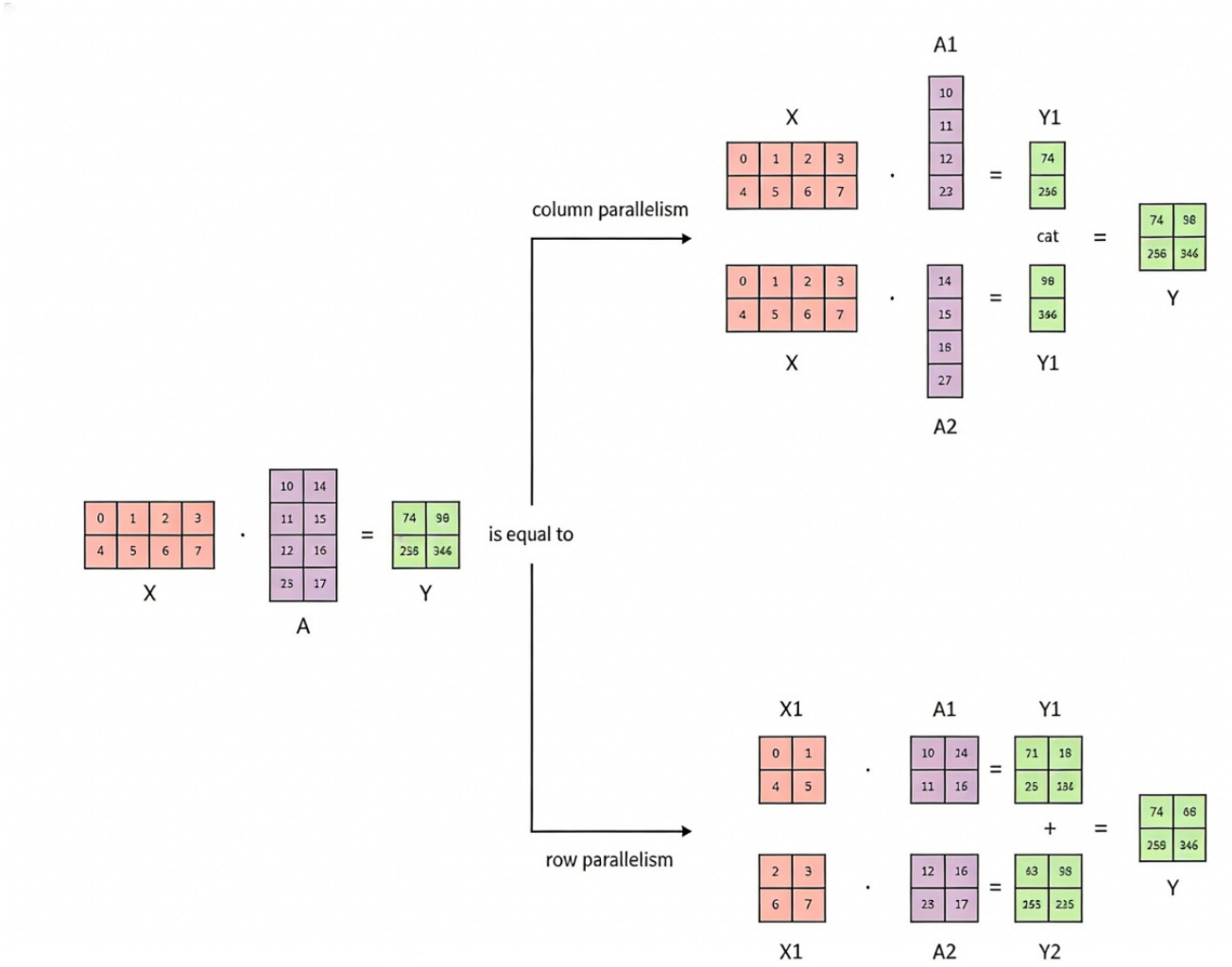
6.5.2 大模型并行介绍

6.5.2.1 张量并行（TP）

矩阵乘的张量并行充分利用矩阵分块乘法的原理。

举例来说，要实现如下矩阵乘法Y=X*A，其中X是维度为MxN的输入矩阵，A是维度为NxK的参数矩阵，Y是结果矩阵，维度为MxK。如果参数矩阵A非常大，甚至超出单张卡的显存容量，那么可以把参数矩阵A切分到多张卡上，并通过集合通信汇集结果，保证最终结果在数学计算上等价于单卡计算结果。

[图片]



a、行并行（Row Parallelism）

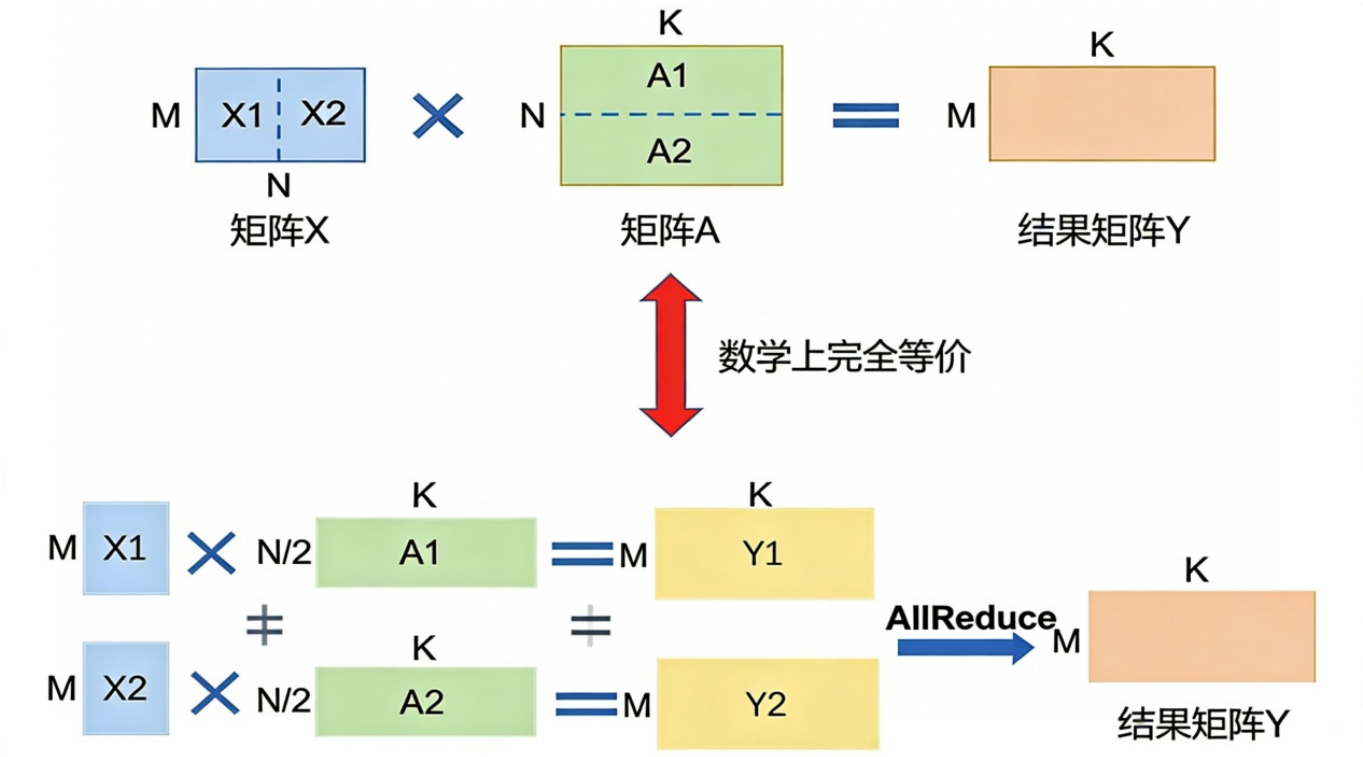
我们先看看Row
Parallelism，就是把A按照行分割成两部分。为了保证运算，同时我们也把X按照列来分割为两部分，这里X1的最后一个维度等于A1最前的一个维度，理论上是：

[图片]

$$XA = \begin{bmatrix} X_1 & X_2 \end{bmatrix} \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} = X_1A_1 + X_2A_2 = Y_1 + Y_2 = Y$$

为了满足矩阵乘法规则，输入矩阵X需要按列切分X=[X1 |
X2]。同时，将矩阵分块，分别放置在两张卡上，每张卡分别计算Y1=X1*A1，Y2=X2*A2。计算完成后，通过collective通信Allreduce_sum，归约其他卡上的计算结果，可以得到最终的结果矩阵Y。同样，这种切分方式，既可以保证数学上的计算等价性，并解决单卡显存无法容纳，又可以保证单卡通过拆分方式可以装下参数A的问题。

[图片]

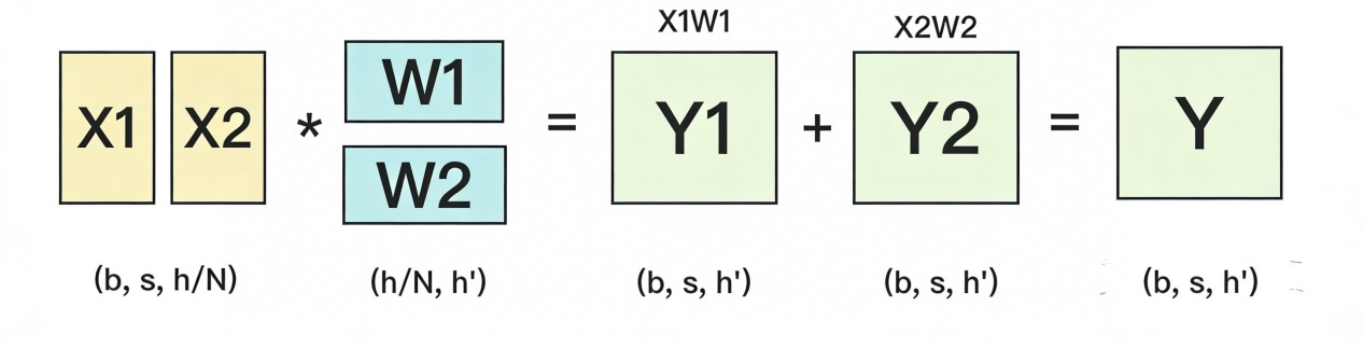


(1) forward

用N来表示GPU的数量。有几块GPU，就把W按行维度切成几份。

W按照行维度切开后，X的维度和它不对齐了，这可怎么做矩阵乘法呢？很简单，再把X“按列切开”就行了，如下图所示：

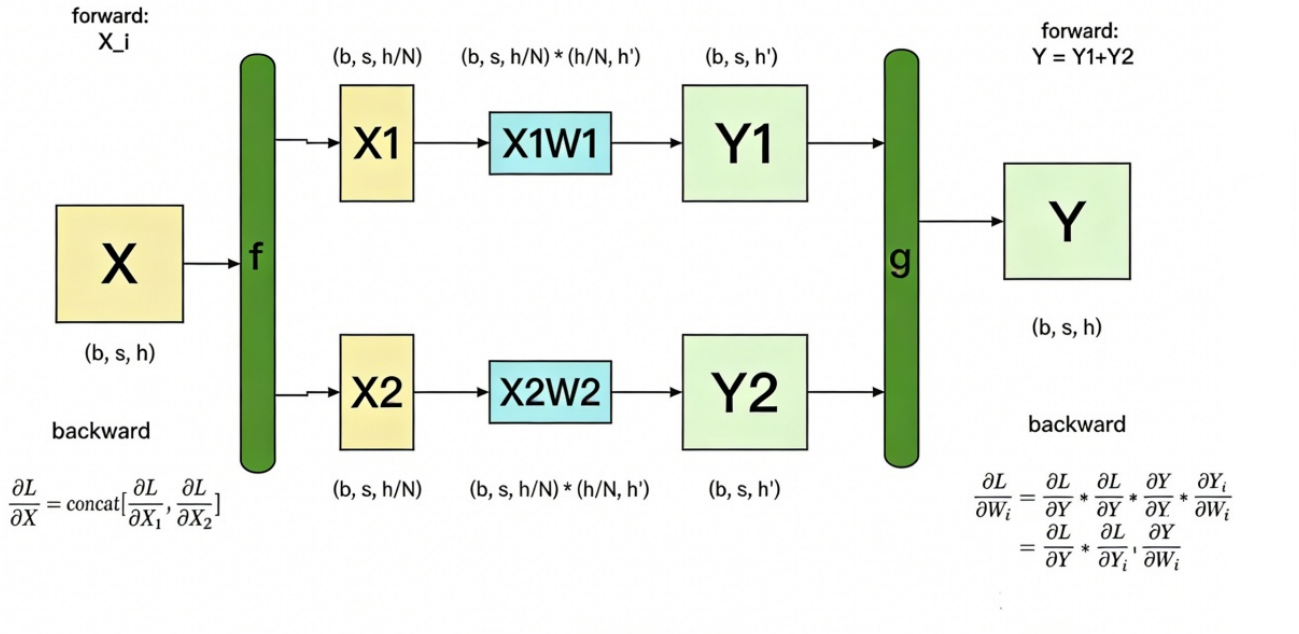
[图片]



(2) backward

做完forward，取得预测值Y，进而可计算出损失L，接下来就能做backward了。我们重画一下forward的过程，并在其中加入backward的部分，整体流程图如下：

[图片]



- f和g：分别表示两个算子，每个算子都包含一组forward+backward操作。forward操作已讲过，不再赘述。
- 图中的每一行，表示单独在一块GPU上计算的过程。
- g的backward：假定现在我们要对求$W{i}$梯度，则可推出:
$\frac{∂L}{∂W{i}}=\frac{∂L}{∂Y}\frac{∂Y}{∂Y_{i}}\frac{∂Y{i}}{∂W{i}}=\frac{∂L}{∂Y}*\frac{∂Y{i}}{∂W{i}}$

也就是说，只要把$\frac{∂L}{∂Y}$同时广播到两块GPU上，两块GPU就可以独立计算各自权重的梯度了。

- f的backward：在上图中，我们只画了模型其中一层的计算过程。当模型存在多层时，梯度要从上一层向下一层传播。比如图中，梯度要先传播到X，如何才能往下一层继续传递。这就是f的backward的作用。这里也易推出：
$\frac{∂L}{∂X}=concat[\frac{∂L}{∂X{1}},\frac{∂L}{∂X{2}}]$

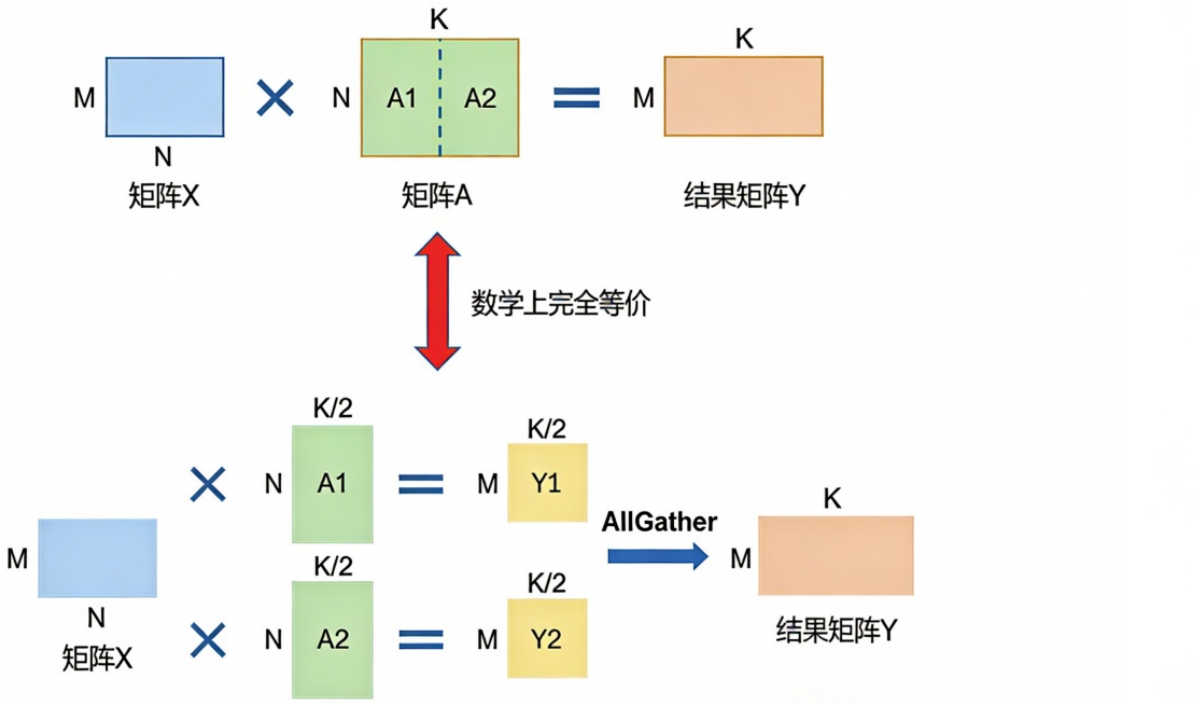
b、列并行（Column Parallelism）

另外一种并行方式Column Parallelism，就是把 A按照列来分割。

分别将A1，A2放置在两张卡上。两张卡分别计算Y1=X*A1和Y2=X*A2。计算完成后，通过collective通信AllGather，获取其它卡上的计算结果，拼接在一起得到最终的结果矩阵Y。

综上所述，通过将单卡显存无法容纳的矩阵A拆分，放置在两张卡上，并通过多卡间通信，即可得到的最终结果。该结果在数学上与单卡计算结果上完全等价。

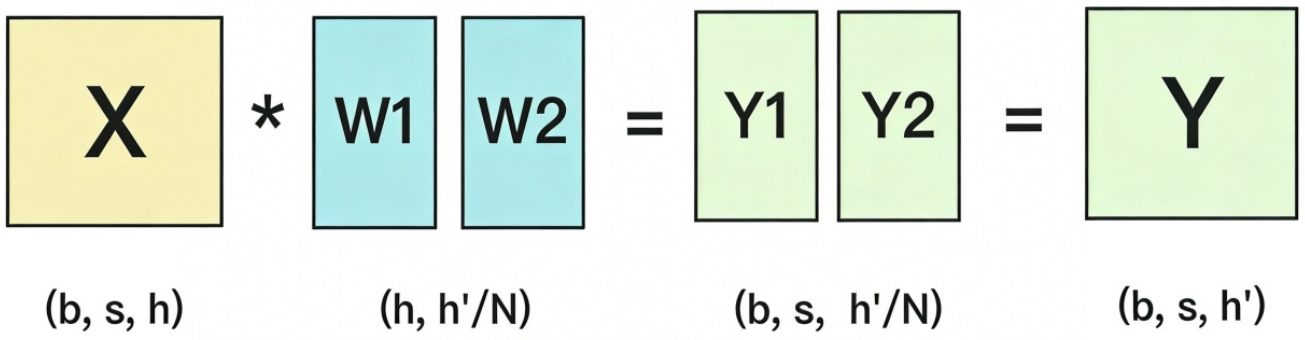
[图片]



（1）forward

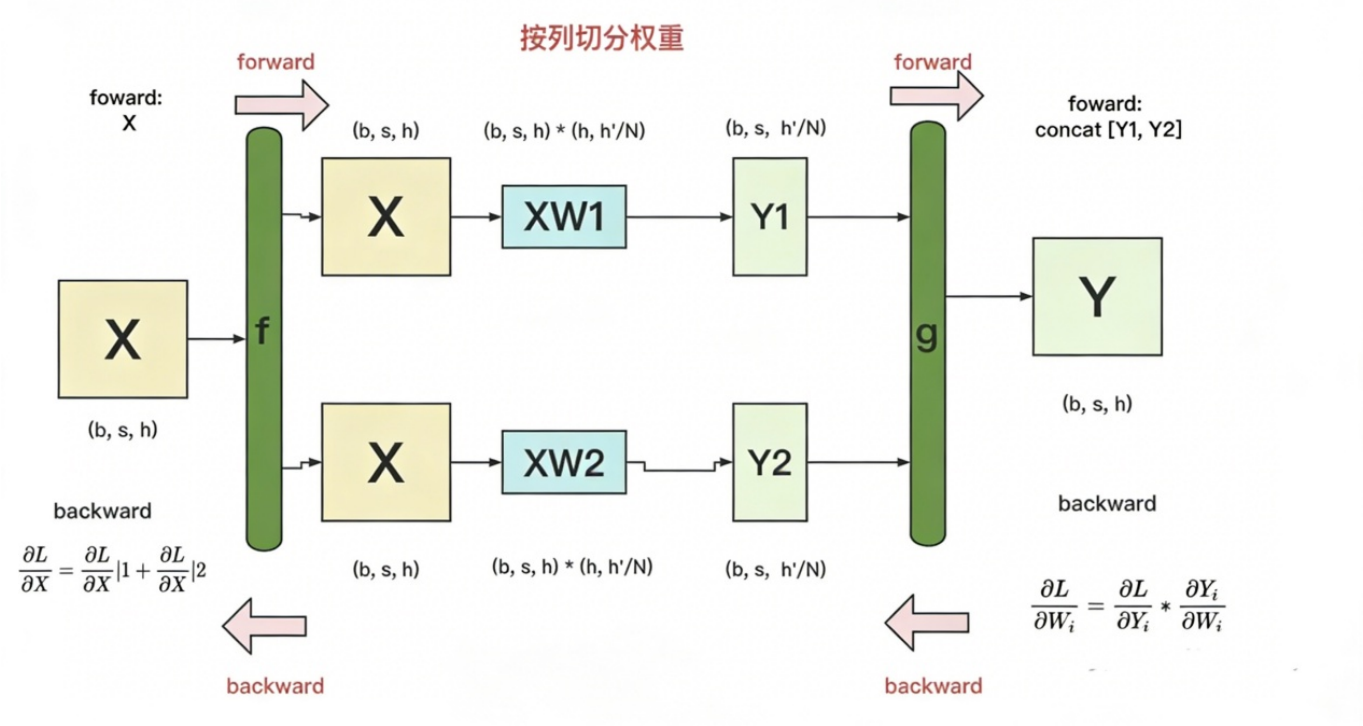
按列切分权重后，forward计算图如下：

[图片]



（2）backward

[图片]



・ g的backward：
$\frac{∂L}{∂W{i}}=\frac{∂L}{∂Y{i}}*\frac{∂Y{i}}{∂W{i}}$
・ f的backward：因为对于损失L，X既参与了XW1的计算，也参与了XW2的计算。因此有：
$\frac{∂L}{∂X}=\frac{∂L}{∂X}|1+\frac{∂L}{∂X}|2$

其中$\frac{∂L}{∂X}|i$表示第i块GPU上计算到X时的梯度。

介绍完了“按行”和“按列”切分权重的方法。在 Megatron-LM
中，权重切分由基础并行原语组合实现。针对 Transformer 架构，其并行策略如下：

分布式张量计算通过将算子划分至多设备以加速计算或扩展模型规模。FlexFlow
通过自动搜索优化并行策略，Mesh TensorFlow
则引入领域语言指定并行维度并编译生成计算图。Megatron-LM 借鉴 Mesh TensorFlow
的核心思想，利用 Attention Heads 的天然并行性切分
Transformer，但摒弃了专用编译器或框架重构，仅通过对 PyTorch
原生实现插入轻量级同步原语，即可实现高效的模型并行。

具体而言，Megatron-LM 对 Masked Multi-Head Self-Attention 与 Feed-Forward
Network进行结构化切分，结合插入的同步原语，以极简方式实现完整的模型并行，无需代码重写或额外编译流程。

c、切分MLP

MLP的第一部分是GEMM，后面是GeLU：

$Y=GeLU(XA)$

并行化GEMM的一个选项是沿行方向分割权重矩阵A，沿列切分输入X：

$X=[X{1}X{2}],A=[A{1}A{2}]$

结果就变成$Y=GeLU(X{1}A{1}+X{2}A{2})$，括号之中的两项，每一个都可以在一个孤独的GPU上完成，

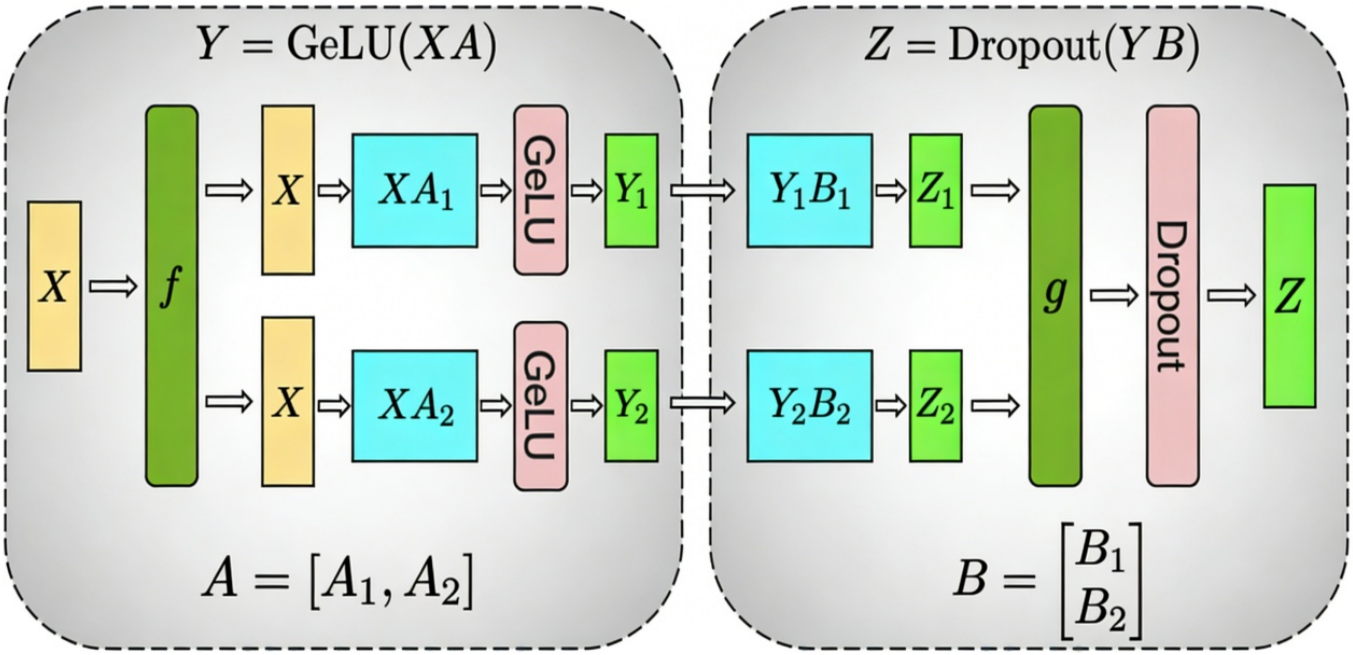
如何通过All-Reduce操作完成求和。既然GeLU是一个非线性函数，那么就有$GeLU(X{1}A{1}+X{2}A{2})≠GeLU(X{1}A{1})+GeLU(X{2}A{2})$，所以这种方案需要在GeLU函数之前加上一个同步点。这个同步点让不同GPU之间交换信息。

另一个选项是沿列拆分A，得到$A=[A{1},A{2}]$。该分区允许GeLU非线性独立应用于每一个分区GEMM的输出：

$[Y{1},Y{2}]=[GeLU(XA{1}),GeLU(XA{2})]$

这个方法更好，因为它删除了同步点，直接把两个GeLU的输出拼接在一起就行。因此，我们以这种列并行方式划分第一个GEMM，并沿其行分割第二个GEMM，以便它直接获取GeLU层的输出，而不需要任何其他通信（比如All-Reduce就不需要），如图所示：

[图片]



上图第一个是 GeLU 操作，第二个是 Dropout操作，具体逻辑如下：

1、MLP的整个输入 X 通过 f 放置到每一块 GPU 之上。

2、对于第一个全连接层：

使用列分割，把权重矩阵切分到两块 GPU 之上，得到 A1,A2。

在每一块 GPU 之上进行矩阵乘法得到第一个全连接层的输出 Y1和 Y2。

3、对于第二个全连接层：

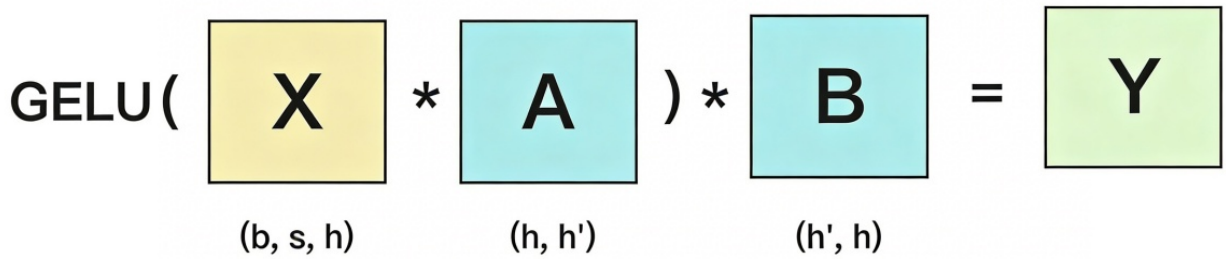
使用行切分，把权重矩阵切分到两个 GPU 之上，得到 B1,B2。

前面输出 Y1 和 Y2 正好满足需求，直接可以和 B
的相关部分（B1,B2）做相关计算，不需要通信或者其他操作，就得到了
Z1,Z2。分别位于两个GPU之上。

4、Z1,Z2通过g做 all-reduce（这是一个同步点），再通过dropout得到了最终的输出Z。

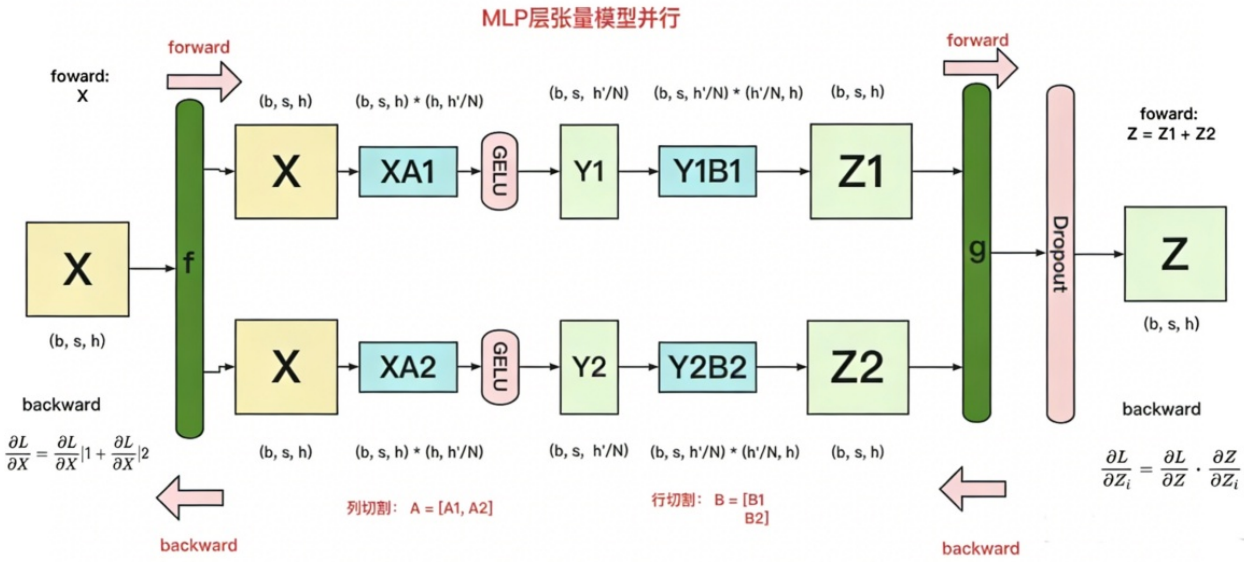
然后在GPU之上，第二个GEMM的输出在传递到dropout层之前进行规约。这种方法将MLP块中的两个GEMM跨GPU进行拆分，并且只需要在前向过程中进行一次all-reduce
操作（g操作符）和在反向过程中进行一次 all-reduce 操作（f操作符）。

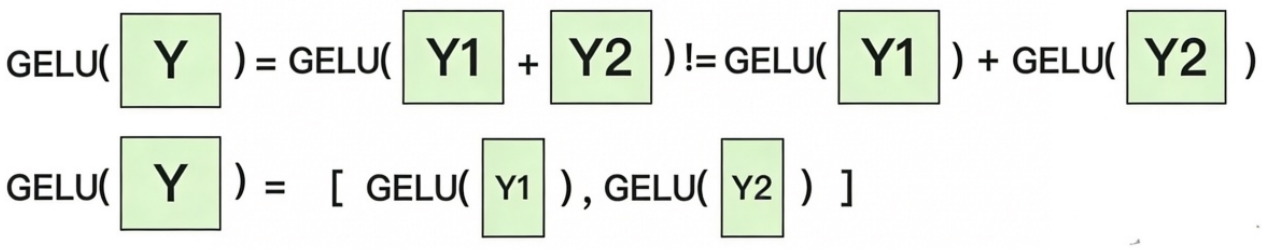
[图片]



其中，GELU是激活函数，A和B分别为两个线性层。在Transformer里，一般设h’ = 4h。假设现在有N块GPU，我们要把MLP层的权重拆到上面做计算，要怎么拆分呢？Megatron提供的拆分办法如下：

[图片]

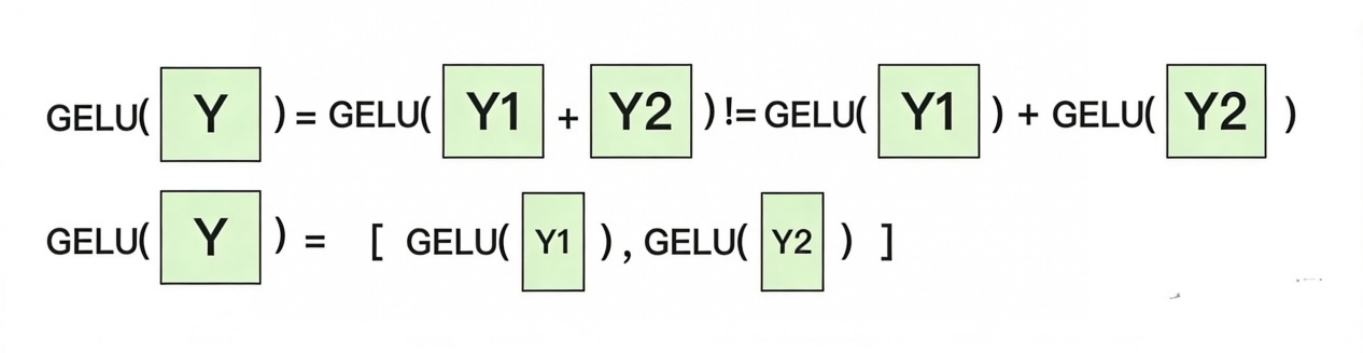




在MLP层中，对A采用“列切割”，对B采用“行切割”

・ f的forward计算：把输入X拷贝到两块GPU上，每块GPU即可独立做forward计算。
・
g的forward计算：每块GPU上的forward计算完毕，取得Z1和Z2后，GPU建做一次AllReduce，相加得到结果Z。
・
g的backward计算：只需要把$\frac{∂L}{∂Z}$拷贝到两块GPU上，两块GPU就能各自独立做梯度计算。
・
f的backward计算：当前层的梯度计算完毕，需要传递到下一层继续做梯度计算时，我们需要求得$\frac{∂L}{∂X}$。则此时两块GPU做一次AllReduce，把各自的梯度$\frac{∂L}{∂X}|1$和$\frac{∂L}{∂X}|2$相加即可。
为什么我们对A采用列切割，对B采用行切割呢？这样设计的原因是，我们尽量保证各GPU上的计算相互独立，减少通讯量。对A来说，需要做一次GELU的计算，而GELU函数是非线形的，它的性质如下：

[图片]



也就意味着，如果对A采用行切割，我们必须在做GELU前，做一次AllReduce，这样就会产生额外通讯量。但是如果对A采用列切割，那每块GPU就可以继续独立计算了。一旦确认好A做列切割，那么也就相应定好B需要做行切割了。

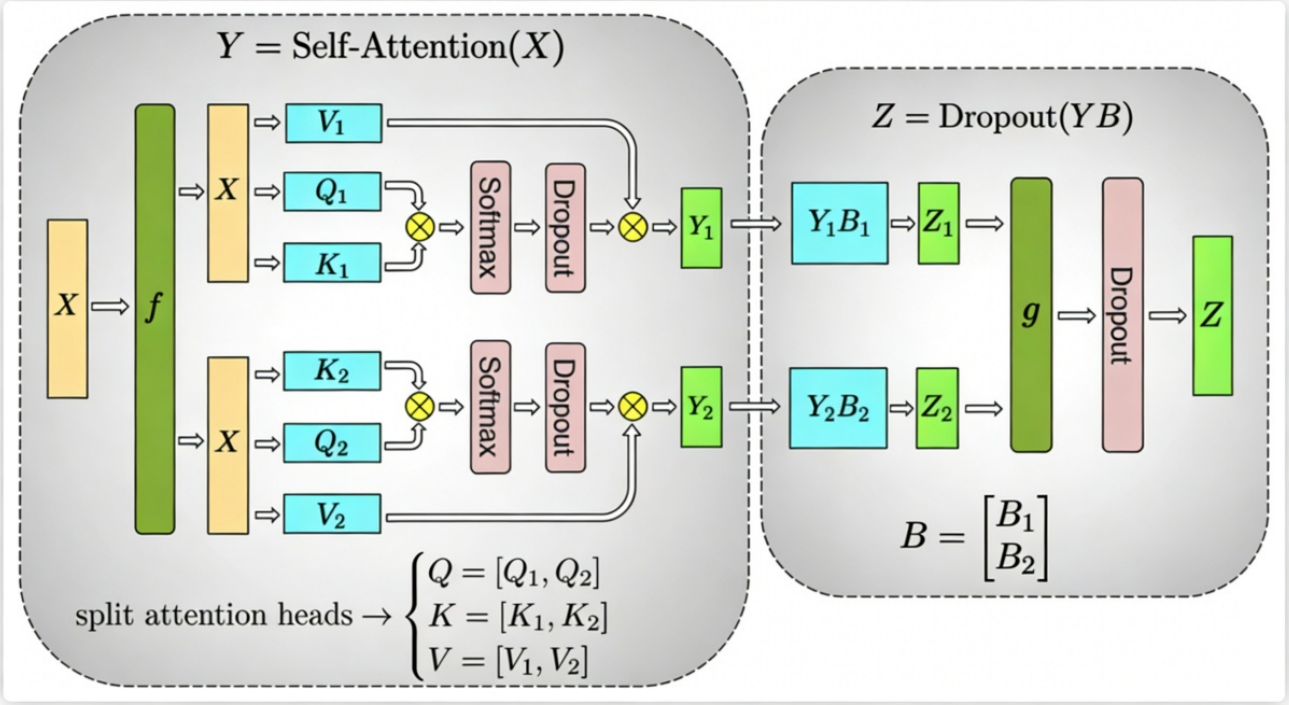
Transformer中的FFN结构均包含两层全连接（FC）层，即存在两个矩阵乘，这两个矩阵乘分别采用上述两种切分方式，对第一个FC层的参数矩阵按列切块，对第二个FC层参数矩阵按行切块。这样第一个FC层的输出恰好满足第二个FC层数据输入要求（按列切分），因此可以省去第一个FC层后的AllGather通信操作。

d、切分Self-Attention层

现在，我们来看稍微复杂一点的self-attention层切割方式（Transformer中Encode和Decoder之间还有做cross-attention，但计算逻辑和self-attention一致，因此这里只拿self-attention举例）。attention的多头计算简直是为张量模型并行量身定做的，因为每个头上都可以独立计算，最后再将结果concat起来。也就是说，可以把每个头的参数放到一块GPU上。

如下图所示。首先，对于自我注意力块，Megatron利用了多头注意力操作中固有的并行性，以列并行方式对与键（K）、查询（Q）和值（V）相关联的GEMM进行分区，从而在一个GPU上本地完成与每个注意力头对应的矩阵乘法。这使我们能够在GPU中分割每个attention
head参数和工作负载，每个GPU得到了部分输出。其次，对于后续的全连接层，因为每个GPU之上有了部分输出，所以对于权重矩阵B就按行切分，与输入的 Y1,Y2 进行直接计算，然后通过g之中的
all-reduce 操作和Dropout 得到最终结果 Z。

[图片]



具有模型并行性的transformer块。f和g是共轭的。f在前向传播中使用一个identity运算符，在后向传播之中使用了all reduce，而g在前向传播之中使用了all reduce，在后向传播中使用了identity运算符。

对三个参数矩阵Q，K，V，按照“列切割”，每个头放到一块GPU上，做并行计算。对线性层B，按照“行切割”。切割的方式和MLP层基本一致，其forward与backward原理也一致，这里不再赘述。

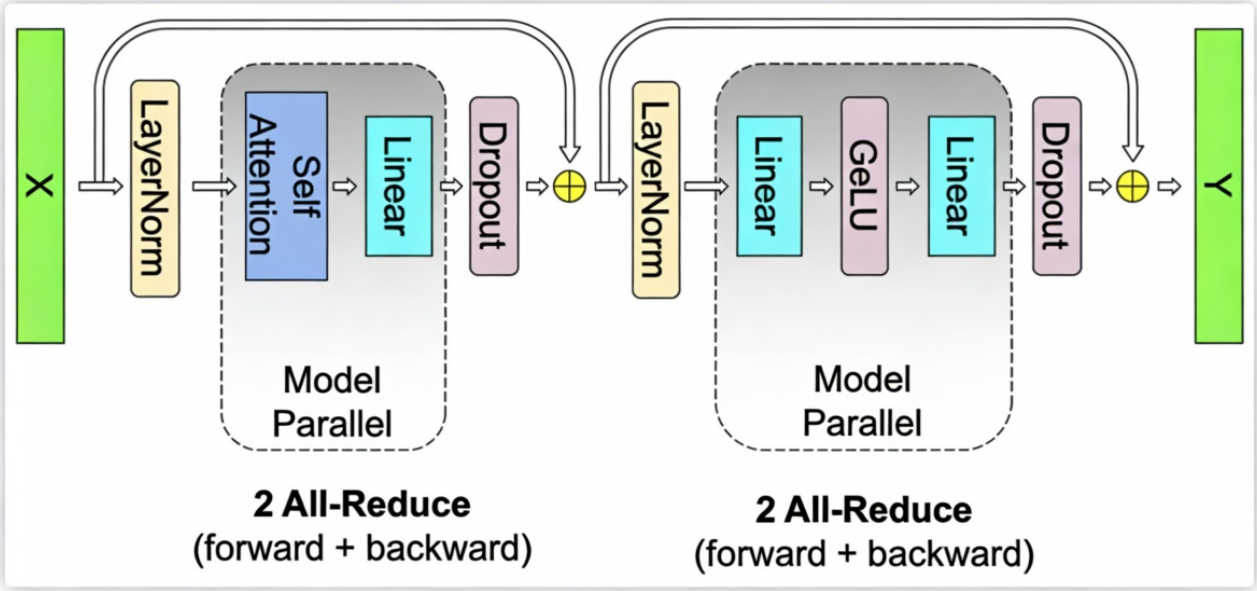
最后，在实际应用中，并不一定按照一个head占用一块GPU来切割权重，我们也可以一个多个head占用一块GPU，这依然不会改变单块GPU上独立计算的目的。所以实际设计时，我们尽量保证head总数能被GPU个数整除。

e、通信成本

来自线性层（在 self attention
层之后）输出的后续GEMM会沿着其行实施并行化，并直接获取并行注意力层的输出，而不需要GPU之间的通信。这种用于MLP和自我注意层的方法融合了两个GEMM组，消除了中间的同步点，并导致更好的伸缩性。这使我们能够在一个简单的transformer层中执行所有GEMM，只需在正向路径中使用两个all-reduce，在反向路径中使用两个all-reduce。

类比于MLP层，self-attention层在forward中做一次AllReduce，在backward中做一次AllReduce。总通讯量也是 4Φ ，其中 Φ=b⨂s⨂h

[图片]



图：transformer层中的通信操作。在一个单模型并行transformer层的正向和反向传播中总共有4个通信操作。

在 Megatron-LM 中，Transformer
语言模型的输出嵌入维度为H×V（V通常为数万）。为高效并行化并支持输入/输出嵌入权重共享，框架沿词汇表维度对嵌入矩阵 EE 进行列切分 [E1,E2][*E*1,*E*2]：输入嵌入后执行 All-Reduce 聚合特征；输出端通过并行 GEMM 计算局部 Logits [Y1,Y2]，并将 All-Gather 与交叉熵损失融合，避免显式聚合b×s×V维张量，将通信量降至 b×s 级别。

需注意，张量并行（TP）在每层前/后向均需两次 All-Reduce，对设备间互联带宽极度敏感，通常不建议跨慢速网络进行跨节点 TP。以 BLOOM 训练为例，若节点内 GPU 通过高速互联（如 NVLink）而节点间带宽受限，建议将 TP 度限制为节点内 GPU 数量（如 4）；若需更高 TP 度，则须使用单节点内集成更多 GPU 的配置。

综上，Megatron-LM 对 Transformer 各模块（MLP、Self-Attention、Embedding）采用统一的层内参数均分策略，每层通过 All-Reduce 同步输出，在保持计算等效性的同时实现高效张量并行。

6.5.2.2 流水线并行（PP）

所谓流水线并行，就是由于模型太大，无法将整个模型放置到单张GPU卡中；因此，将模型的不同层放置到不同的计算设备，降低单个计算设备的显存消耗，从而实现超大规模模型训练。

比如，模型共包含四个模型层（如：Transformer层），被切分为三个部分，分别放置到三个不同的计算设备。即第 1 层放置到设备 0，第 2 层和第三 3 层放置到设备 1，第 4 层放置到设备 2。相邻设备间通过通信链路传输数据。具体地讲，前向计算过程中，输入数据首先在设备 0 上通过第 1 层的计算得到中间结果，并将中间结果传输到设备 1，然后在设备 1 上计算得到第 2 层和第 3 层的输出，并将模型第 3 层的输出结果传输到设备 2，在设备 2 上经由最后一层的计算得到前向计算结果。反向传播过程类似。最后，各个设备上的网络层会使用反向传播过程计算得到的梯度更新参数。由于各个设备间传输的仅是相邻设备间的输出张量，而不是梯度信

息，因此通信量较小。

a、朴素流水线并行

朴素流水线并行是实现流水线并行训练的最直接的方法。我们将模型按照层间切分成多个部分（Stage），并将每个部分（Stage）分配给一个GPU。然后，我们对小批量数据进行常规的训练，在模型切分成多个部分的边界处进行通信。

我们将计算分配给两个 GPU，如下所示：

- GPU1 computes: intermediate=L2(L1(input))
- GPU2 computes: output=L4(L3(intermediate))

为了完成前向传播，我们在GPU1上计算中间值并将结果张量传输到 GPU2。
然后，GPU2计算模型的输出并开始进行反向传播。
对于反向传播，我们从GPU2到GPU1的中间发送梯度。
然后，GPU1根据发送的梯度完成反向传播。
这样，流水线并行训练会产生与单节点训练相同的输出和梯度。
朴素流水线并行训练相当于顺序训练，这使得调试变得更加容易。

朴素流水线并行存在的问题：

1、GPU利用率极低（Bubble严重）： 任意时刻仅1个GPU工作，其余空闲。朴素流水线的 Bubble 的时间为$O(\frac{K-1}{K})$，随GPU数量增加趋近1，计算资源严重浪费。

2、通信开销大：
设备间数据传输耗时。例如4张6GB卡虽能容纳24GB卡模型，但因传输开销，训练反而比单卡更慢。

3、通信计算未重叠： 传输中间输出(FWD)和梯度(BWD)期间，GPU处于完全空闲状态。

4、首卡显存压力高： 首个GPU需保留整个Batch的激活值直至最后，大Batch易导致显存溢出。

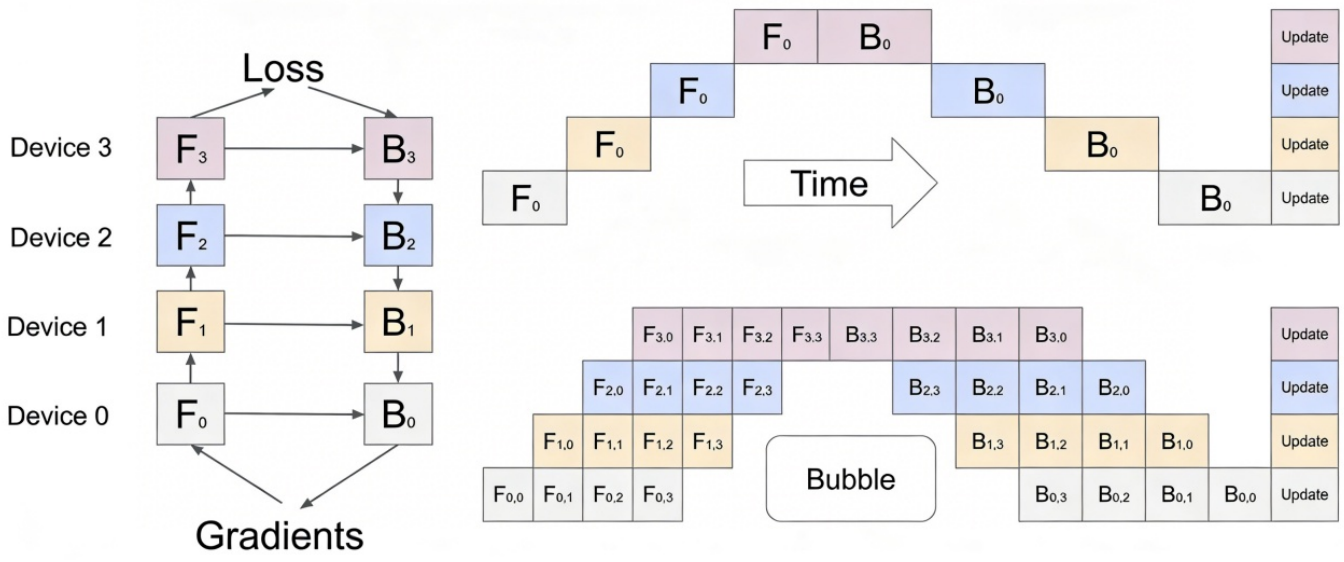
b、微批次流水线并行

微批次（MicroBatch）流水线并行与朴素流水线几乎相同，但它通过将传入的小批次（minibatch）分块为微批次（microbatch），并人为创建流水线来解决GPU空闲问题，从而允许不同的GPU同时参与计算过程，可以显著提升流水线并行设备利用率，减小设备空闲状态的时间。目前业界常见的流水线并行方法 GPipe 和 PipeDream 都采用微批次流水线并行方案。

Gpipe 流水线并行主要用来解决这两个问题：

第一，提高模型训练的并行度。Gpipe在朴素流水线并行的基础上，利用数据并行的思想，将mini-batch细分为多个更小的micro-batch，送入GPU进行训练，来提高并行程度。

[图片]



GPipe通过微批流水线显著降低Bubble比例。F的第一个下标表示GPU编号，F的第二个下标表示micro-batch编号。若将mini-batch切分为M份，Bubble时间降为：$O(\frac{K-1}{K+M-1})$。当M>>K的时候，这个时间可以忽略不计。

缺点是Batch
Normalization等依赖统计量的层需特殊处理：GPipe在训练时使用micro-batch统计量，并同步追踪全mini-batch的移动平均/方差供推理使用，而Layer Normalization不受影响

第二，通过重计算（Re-materialization）降低显存消耗。采用“以算代存”策略。前向传播时丢弃中间激活值，反向传播时按需实时重算，避免全量存储。每卡仅保留最后一层输出，大幅压降显存峰值，使同等模型显存需求显著降低。

简而言之，GPipe
通过纵向切分模型突破单卡显存墙，结合微批流水线提升多卡并行度，并引入重计算技术优化显存占用，构成高效的大模型并行训练基座。

上面讲述了 GPipe 流水线并行方案，接下来讲述一下PipeDream 。讲述
PipeDream之前，我们先来看看流水线并行策略。

c、流水线并行策略

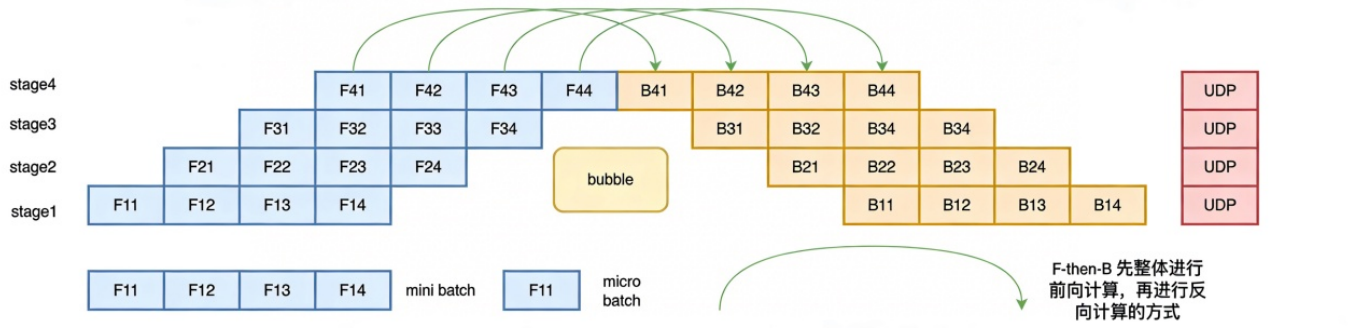
流水线并行根据执行的策略，可以分为F-then-B和1F1B两种模式。之前讲述的朴素流水线并行以及GPipe都是F-then-B模型，而后续讲述的 PipeDream 则是1F1B模式。

（1）F-then-B 策略

F-then-B 模式，先进行前向计算，再进行反向计算。

F-then-B 模式由于缓存了多个 micro-batch 的中间变量和梯度，显存的实际利用率并不高。

[图片]

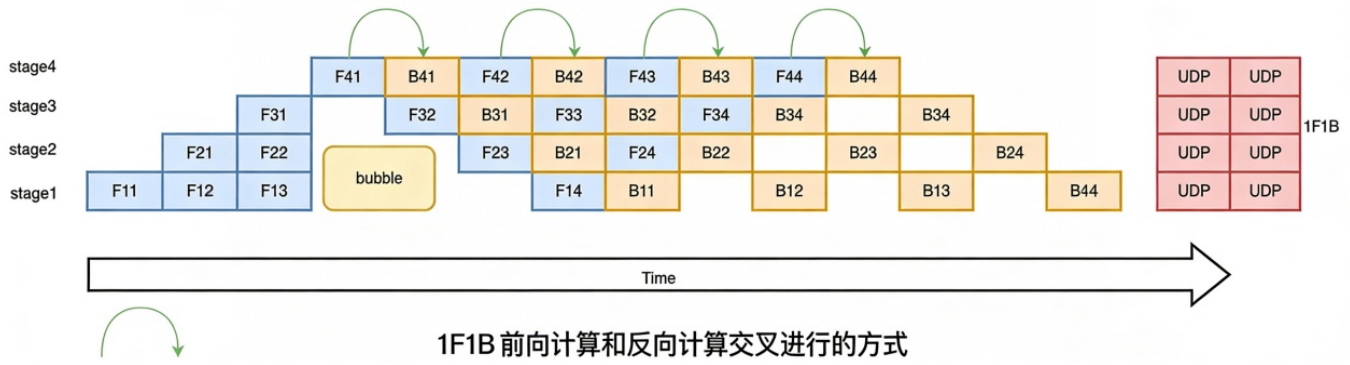


（2）1F1B 策略

1F1B（One Forward pass followed by One Backward pass）模式，一种前向计算和反向计算交叉进行的方式。在1F1B模式下，前向计算和反向计算交叉进行，可以及时释放不必要的中间变量。

1F1B示例如下图所示，以stage4的F42（stage4的第2个micro-batch的前向计算）为例，F42在计算前，F41的反向B41（stage4的第1个micro-batch的反向计算）已经计算结束，即可释放F41的中间变量，从而 F42可以复用F41中间变量的显存。

[图片]



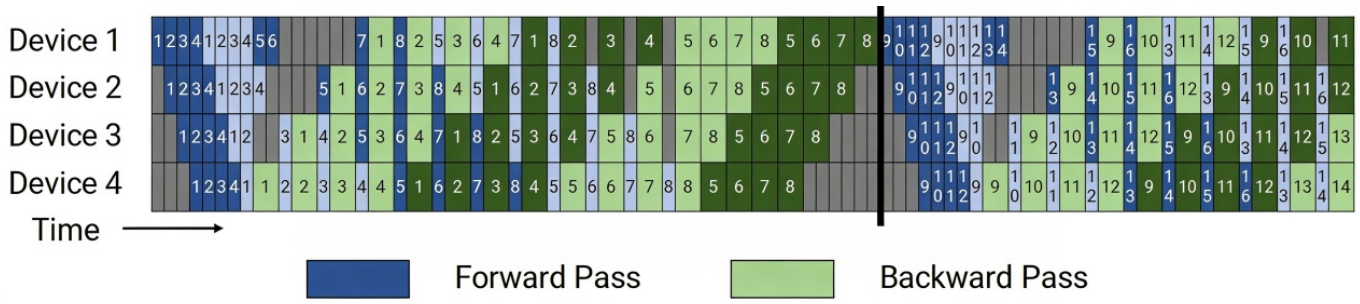
研究表明，1F1B方式相比于F-then-B方式，峰值显存可以节省37.5%，对比朴素流水线并行峰值显存明显下降，设备资源利用率显著提升。

d、PipeDream（交错式1F1B）-Megatron-LM

Megatron-LM 基于PipeDream-Flush提出了一个小的Trick：交错式1F1B调度，而交错式1F1B调度也是Megatron-LM 论文《*Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM），virtual pipeline》*中最主要的一个创新点。

传统的流水线并行通常会在一个设备（Device）上放置几个连续的模型层（如：Transformer层）。但Megatron 这篇论文采用虚拟流水线（virtual pipeline），进行交错式1F1B并行。在设备数量不变的情况下，分出更多的流水线阶段（pipeline stage），以更多的通信量，换取流水线Bubble比率降低。

[图片]



此外，该方案要求一个小批次中的微批次数量是管道并行大小（流水线中的设备数量）的整数倍。
例如，对于4个设备，一个小批次中的微批次数量必须是 4 的倍数。

那虚拟流水线（virtual pipeline）是怎么做到的呢?

对照上面示例图举例说明，若网络共16层（编号 0-15），4个Device，前述谷歌的 GPipe 和微软的PipeDream是分成4个stage， 按编号0-3层放Device1，4-7层放 Device2 ，以此类推。

英伟达的virtual
pipeline则是按照文中提出的virtual_pipeline_stage概念减小切分粒度，以virtaul_pipeline_stage=2为例，将 0-1 层放Device1,
2-3层放在Device2，…，6-7层放到Device4，8-9层继续放在Device1，10-11层放在Device2，…，14-15层放在 Device4。

按照这种方式，Device之间的点对点通信次数(量)直接翻了virtual_pipeline_stage
倍，但空泡比率降低了，若定义每个 Device 上有 v 个 virtual stages，或者论文中也叫做 model chunks，在这个例子中 v=2，这样一来，空泡比率为：

$Bubble time fraction (pipeline bubble size)=\frac{t{pb}^{int.}}{t{id}}=\frac{1}{v}⊠\frac{p-1}{m}$

从上面公式可以看出空泡比率和v成反比，降低了v倍。但交错调度导致通信量同增
v倍，需依赖高速网络（如多网卡 InfiniBand）缓解。相比 GPipe，PipeDream 虽降低显存却未减少Bubble；Megatron-LM 引入交错式 1F1B 调度进一步压低 Bubble 率，但同样带来额外通信开销，亦需IB 网络支持。

6.5.2.3 数据并行（DP）

所谓数据并行，就是由于训练数据集太大；因此，将数据集分为N份，分别装载到N个GPU节点中，同时，每个GPU节点持有一个完整的模型副本，分别基于每个GPU中的数据去进行梯度求导。然后，在GPU0上对每个GPU中的梯度进行累加，最后，再将GPU0聚合后的结果广播到其他GPU节点。

数据并行(torch.nn.DataParallel)，这是Pytorch最早提供的一种数据并行方式，它基于单进程多线程进行实现的，它使用一个进程来计算模型权重，在每个批处理期间将数据分发到每个GPU。

DataParallel 的计算过程如下所示：

- 将 inputs 从主 GPU 分发到所有 GPU 上。
- 将 model 从主 GPU 分发到所有 GPU 上。
- 每个 GPU 分别独立进行前向传播，得到 outputs。
- 将每个 GPU 的 outputs 发回主 GPU。

・ 在主 GPU 上，通过 loss function 计算出 loss，对 loss function 求导，求出损失梯度。
・ 计算得到的梯度分发到所有 GPU 上。
・ 反向传播计算参数梯度。
・ 将所有梯度回传到主 GPU，通过梯度更新模型权重。
・ 不断重复上面的过程。
但是它的缺点也很明显：
・ 单进程多线程局限：受 GIL
锁限制，性能开销大、速度慢；仅支持单机多卡，无法分布式扩展；且不支持 Apex 混合精度训练。
・
效率低下与负载不均：数据需经主进程分发、参数与输出需频繁在主卡（GPU0）聚合，通信易成瓶颈；主卡额外承担协调与显存开销，易导致显存不均与 OOM。
・ 无法组合模型并行：架构局限使其难以与模型并行策略协同使用。
因此，PyTorch 官方推荐优先使用 DistributedDataParallel (DDP)，即便在单机多卡场景下。

a、分布式数据并行（PyTorch DDP）

分布式数据并行(torch.nn.DistributedDataParallel)，基于多进程进行实现的，每个进程都有独立的优化器，执行自己的更新过程。每个进程都执行相同的任务，并且每个进程都与所有其他进程通信。进程（GPU）之间只传递梯度，这样网络通信就不再是瓶颈。

具体流程如下：

・ 首先将rank=0进程中的模型参数广播到进程组中的其他进程；
・ 然后，每个DDP进程都会创建一个local Reducer来负责梯度同步。
・
在训练过程中，每个进程从磁盘加载batch数据，并将它们传递到其GPU。每个GPU都有自己的前向过程，完成前向传播后，梯度在各个GPUs间进行All-Reduce，每个GPU都收到其他GPU的梯度，从而可以独自进行反向传播和参数更新。
・
同时，每一层的梯度不依赖于前一层，所以梯度的All-Reduce和后向过程同时计算，以进一步缓解网络瓶颈。
・ 在后向过程的最后，每个节点都得到了平均梯度，这样各个GPU中的模型参数保持同步 。
而**DataParallel 是将梯度reduce到主卡，在主卡上更新参数，再将参数 broadcast
给其他GPU，这样无论是主卡的负载还是通信开销都比DDP大很多**)，相比于DataParallel，DistributedDataParallel方式可以更好地进行多机多卡运算，更好的进行负载均衡，运行效率也更高，虽然使用起来较为麻烦，但对于追求性能来讲是一个更好的选择。

b、DP 与 DDP 的区别

DP 和 DDP 的主要差异有以下几点：

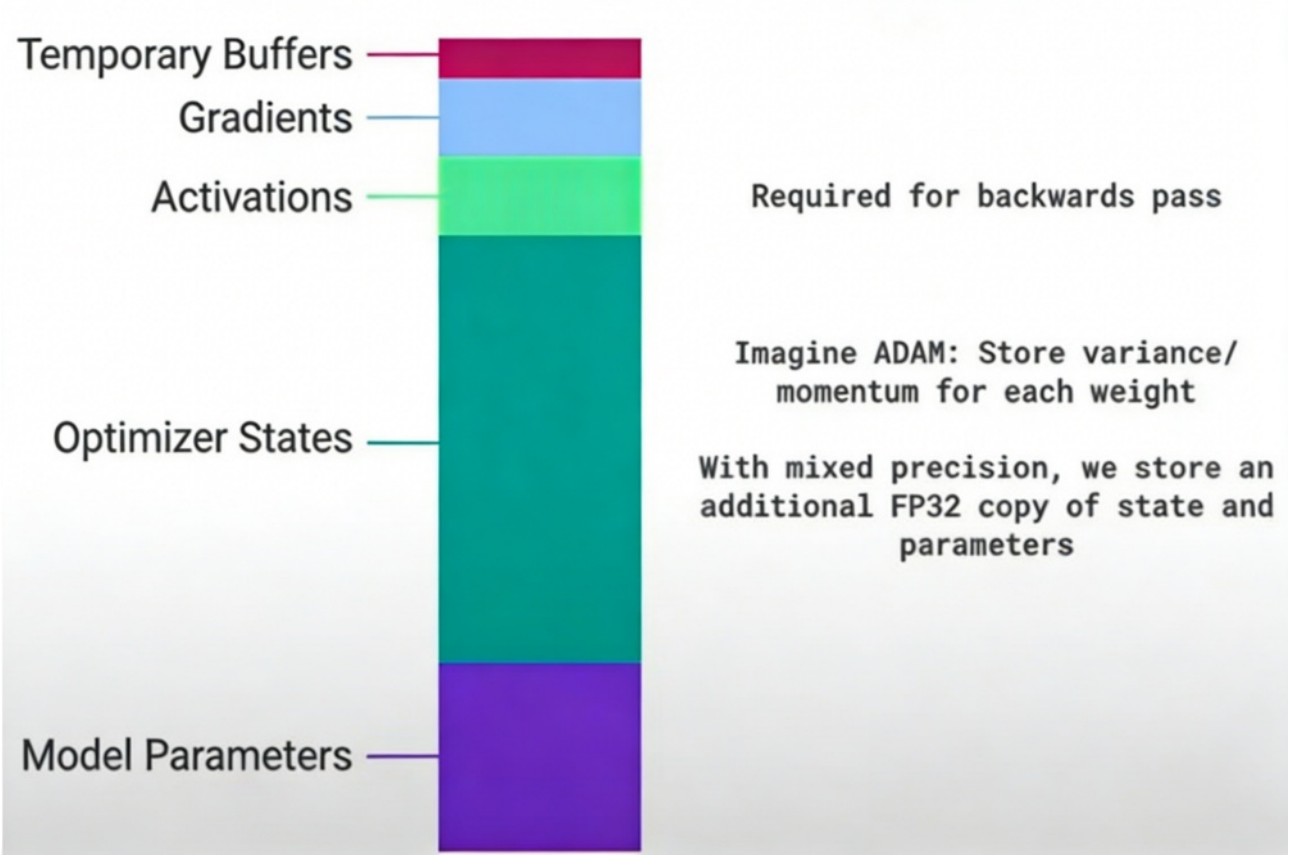
・ 架构与扩展性：DP 为单进程多线程，受 GIL 限制且仅支持单机；DDP
为多进程（一卡一进程），支持单机/多机分布式，彻底规避 GIL 开销。
・ 同步机制与通信：
・ DP：输出 Gather 至 GPU0 算 Loss → Scatter 回各卡 → 梯度 Gather 至 GPU0 更新参数 →
Broadcast 同步权重。全程单优化器，通信约 4 次，主卡易成瓶颈且显存严重不均。
・ DDP：各卡独立前向/算 Loss → 梯度 All-Reduce 汇总 →
各卡独立更新参数。初始仅同步一次，通信仅 1 次（梯度），无主卡瓶颈，负载均衡。
・ 模型并行支持：DDP 原生兼容，DP 不支持。
c、完全分片数据并行(PyTorch FSDP)

由于 PyTorch FSDP 受 DeepSpeed ZeRO 启发而获得灵感，因此，下面先简要介绍下 ZeRO。

补充说明：ZeRO

通常来说，在模型训练的过程中，GPU上需要进行存储的参数包括了模型本身的参数、优化器状态、激活函数的输出值、梯度以及一些零时的Buffer。各种数据的占比如下图所示：

[图片]



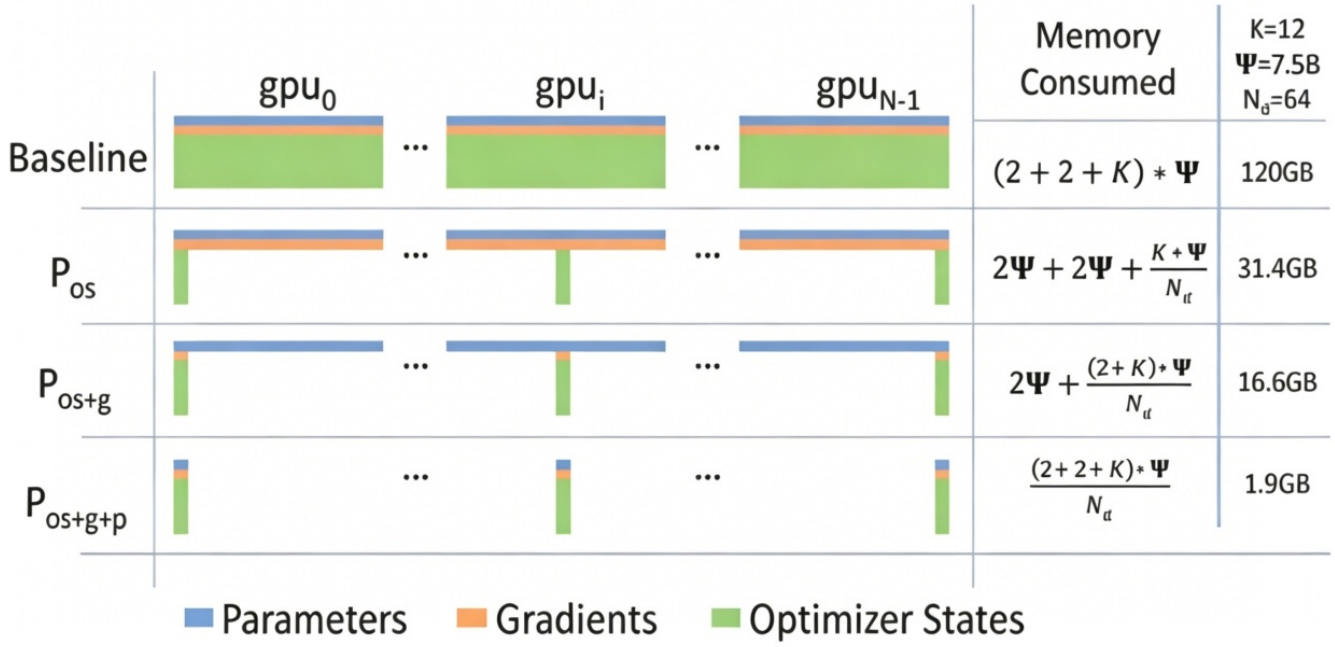
可以看到模型参数仅占模型训练过程中所有数据的一部分，当进行混合精度运算时，其中模型状态参数(优化器状态 + 梯度+
模型参数）占到了一大半以上。因此，我们需要想办法去除模型训练过程中的冗余数据。

针对模型状态的存储优化（去除冗余），DeepSpeed
提出了ZeRO，ZeRO使用的方法是分片，即每张卡只存1/N的模型状态量，这样系统内只维护一份模型状态参数。

ZeRO对模型状态（Model States）参数进行不同程度的分割，主要有三个不同级别：

- ZeRO-1（优化器分片）：仅切分 Optimizer State（如 Adam 的 m/v），不分片模型与梯度。Forward 独立执行，Backward 通过 All-Reduce 同步梯度；各 Rank 仅更新负责的分片，最后 All-Gather 同步参数。适合 Adam 等重型优化器，可消除 Optimizer 冗余，但不适合 SGD。
- ZeRO-2（优化器+梯度分片）：在 ZeRO-1 基础上增加梯度分片。Backward 阶段用 Reduce 替代 All-Reduce，将梯度归约至对应 Rank，显著降低通信开销；各 Rank 独立更新后广播同步。
- ZeRO-3（全状态分片）：进一步切分模型参数。利用 All-Reduce 可拆解为 Reduce+All-Gather 的特性，各 Rank
仅保留当前计算所需的层参数，其余释放。前向计算时动态获取参数，实现“数据并行+模型并行”效果，显存节省最大化。

[图片]



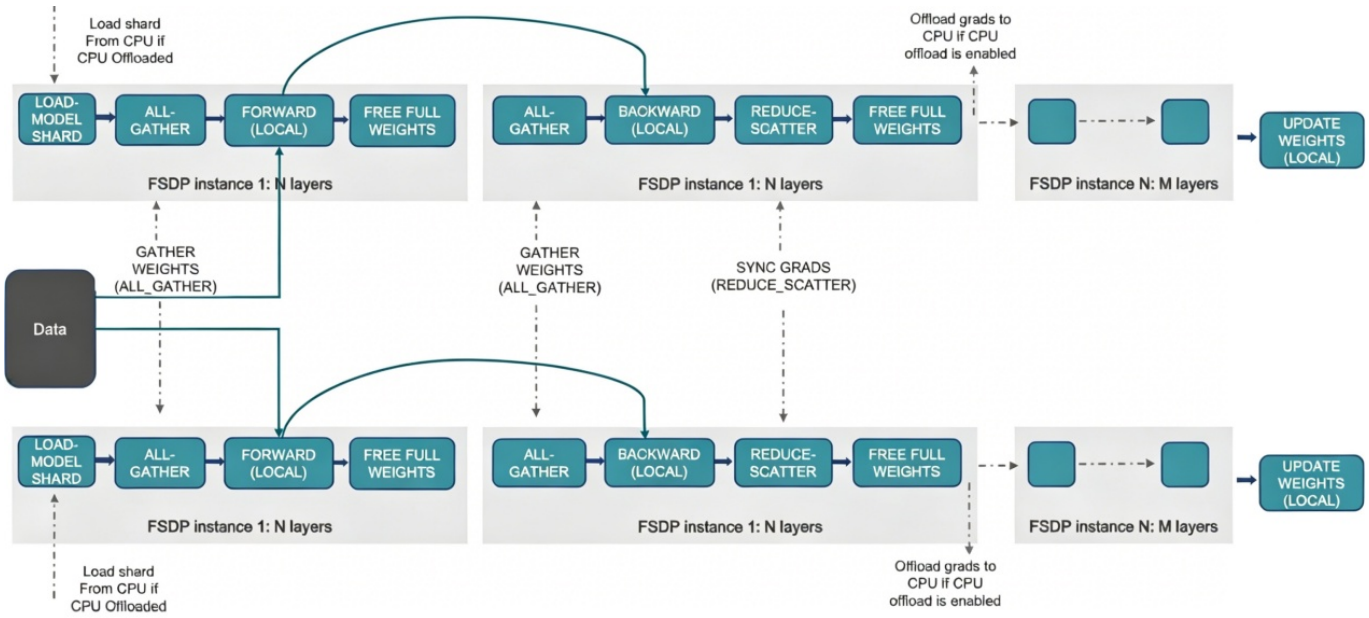
FSDP

完全分片数据并行(torch.distributed.fsdp.FullyShardedDataParallel)，是Pytorch最新的数据并行方案，在1.11版本引入的新特性，目的主要是用于训练大模型。我们都知道Pytorch DDP用起来简单方便，但是要求整个模型加载到一个GPU上，这使得大模型的训练需要使用额外复杂的设置进行模型分片。因此，为了打破模型分片的障碍（包括模型参数，梯度，优化器状态）；同时，仍然保持了数据并行的简单性，该新特性应运而生。

FSDP
是一种新型数据并行训练方法，但与传统的数据并行不同，传统的数据并行维护模型参数、梯度和优化器状态的每个GPU副本，而FSDP将所有这些状态跨数据并行工作线程进行分片，并且可以选择将模型参数分片卸载到CPU。

下图显示了FSDP如何在 2 个数据并行进程中工作流程：

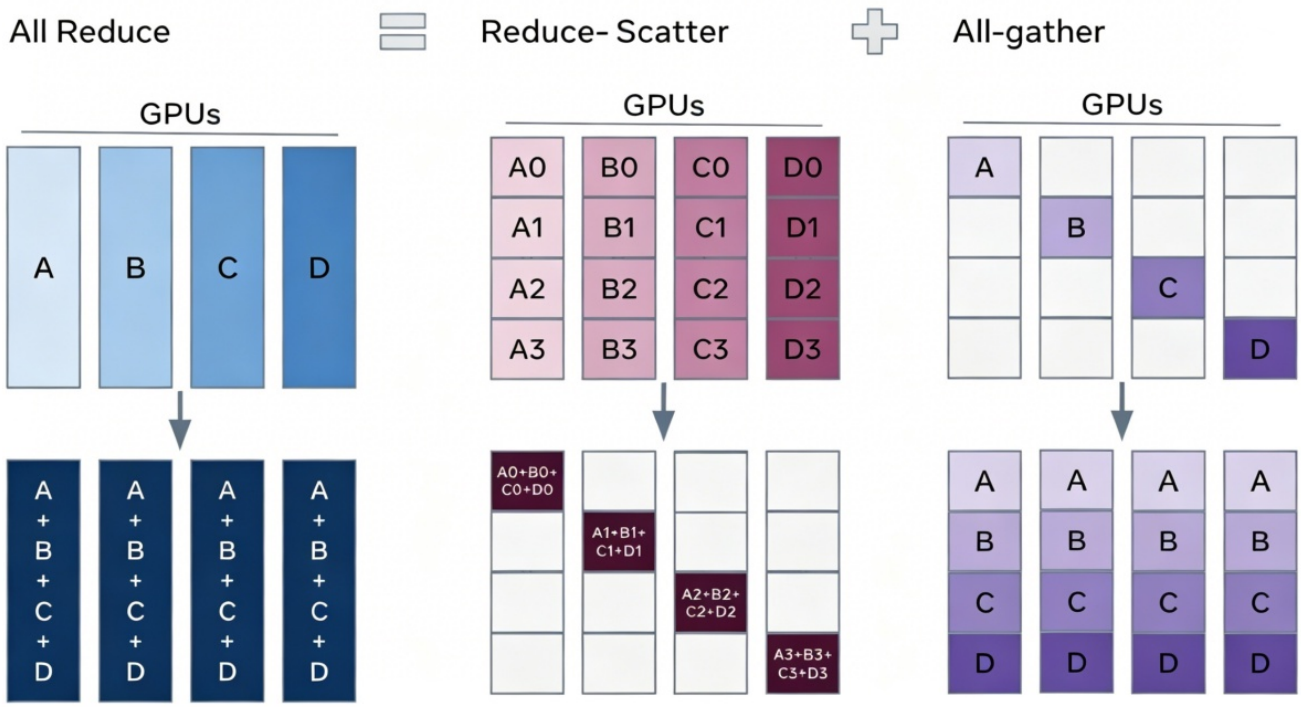
[图片]



通常，模型层以嵌套方式用FSDP包装，因此，只有单个FSDP实例中的层需要在前向或后向计算期间将完整参数收集到单个设备。
计算完成后，收集到的完整参数将立即释放，释放的内存可用于下一层的计算。
通过这种方式，可以节省峰值GPU内存，从而可以扩展训练以使用更大的模型大小或更大的批量大小。
为了进一步最大化内存效率，当实例在计算中不活动时，FSDP可以将参数、梯度和优化器状态卸载到CPU。

解锁ZeRO/FSDP的关键是我们可以把DDP之中的All-Reduce操作分解为独立的Reduce-Scatter和All-Gather操作。

[图片]



All-Reduce是Reduce-Scatter和All-Gather的组合。聚合梯度的标准All-Reduce操作可以分解为两个单独的阶段。

- Reduce-Scatter阶段，在每个GPU上，会基于rank索引对rank之间相等的块进行求和。
- All-Gather阶段，每个GPU上的聚合梯度分片可供所有GPU使用。

通过重新整理Reduce-Scatter和All-Gather，每个DDP worker只需要存储一个参数分片和优化器状态。

在PyTorch中使用FSDP包装模型有两种方法。

- 自动包装（Auto Wrapping）是DDP的直接替代品；
- 手动包装（Manual Wrapping）需要对模型定义代码进行少量的更改，并且能够探索复杂的分片策略。

自动包装（Auto Wrapping）

模型层应以嵌套方式包装在FSDP中，以节省峰值内存并实现通信和计算重叠。
最简单的方法是自动包装，它可以作为DDP的直接替代品，而无需更改其余代码。

fsdp_auto_wrap_policy参数允许指定可调用函数以使用FSDP递归地包裹层。 PyTorch FSDP提供的default_auto_wrap_policy函数递归地包裹参数数量大于100M的层。当然，也可以根据需要提供自己的包装策略。

此外，可以选择配置cpu_offload，以便在计算中不使用包装参数时将这些参数卸载到 CPU。
这可以进一步提高内存效率，但代价是主机和设备之间的数据传输开销。

手动包装（Manual Wrapping）

通过有选择地对模型的某些部分应用包装，手动包装对于探索复杂的分片策略非常有用。
总体设置可以传递给enable_wrap()上下文管理器。

DDP 与 FSDP 的区别

DDP（DistributedDataParallel）：每卡维护完整模型副本，本地完成前/后向传播后，通过All-Reduce同步梯度并独立更新参数，多进程架构避免GIL开销，支持单机/多机部署。

FSDP（FullyShardedDataParallel）：将模型参数、梯度及优化器状态分片存储于各卡。前/后向计算前通过All-Gather临时聚合完整权重，计算后通过Reduce-Scatter聚合梯度并分片，每卡仅更新本地权重分片；启用reshard_after_forward=True可在层间释放冗余权重以优化显存。

DP的缺陷：单进程多线程存在锁竞争；梯度同步依赖0号节点汇聚-平均-广播，导致通信瓶颈与负载不均；通信开销随卡数线性增长；不支持多机训练，现已基本弃用。

DDP的局限：虽实现高效分布式训练且参数一致性有保障，但要求单卡容纳完整模型，大模型需额外分片策略。

FSDP的优势：在保持数据并行简洁性的同时，通过状态分片与可选CPU卸载，突破单卡显存限制，支持超大规模模型训练。

6.5.2.4 序列/上下文并行（SP/CP）

Megatron-LM的第三篇论文*《Reducing Activation Recomputation in Large Transformer Models》*是2023年出的。在大模型训练过程中显存占用过大往往成为瓶颈，一般会通过recomputation重计算的方式降低显存占用，但会带来额外的计算代价。这篇论文提出了两种方法，分别是sequece

parallel和selective activation
recomputation，这两种方法和TP是可以相结合的，可以有效减少不必要的计算量。

下图中绿色部分表示不同模型中需要用于保存activation需要的显存大小，蓝色部分表示不同模型中需要用于保存parameter和optimizer state需要的显存大小。红色线表示A100的显存大小80G。

[图片]

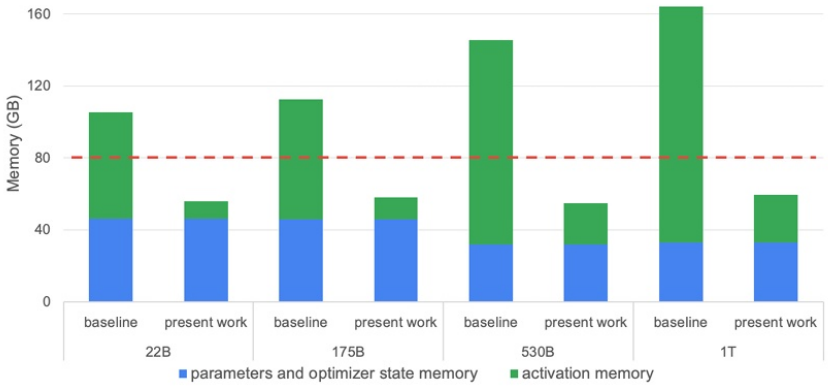


Figure 1. Parameters, optimizer state, and activations memory. The dashed red line represents the memory capacity of an NVIDIA A100 GPU. Present work reduces the activation memory required to fit the model. Details of the model configurations are provided in Table 3.

a、Pipeline Parallel详细介绍

i. 估算Transformer Activation Memory大小
以Transformer结构为例估算Activation
Memory大小，这里的Activation定义是指前向和反向梯度计算中创建的所有tensor。按这个定义来说，计算不包含模型参数大小和优化器中状态大小，但是包含dropout op用到的mask tensor。

[图片]

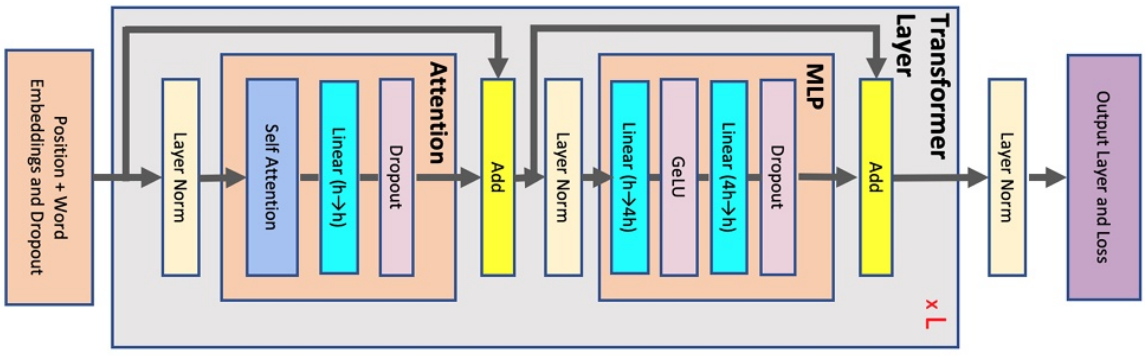


Figure 2. Transformer Architecture. Each gray block represents a single transformer layer that is replicated L times.

一个Transformer块中由一个Attention块和一个MLP块组成，中间通过两个LayerNorm层进行连接。在Transformer中用到的参数表示如下：

[图片]

a	# attention heads	p	pipeline parallel size
b	microbatch size	s	sequence length
h	hidden dimension size	t	tensor parallel size
L	# transformer layers	v	vocabulary size

Table 1. Variable names.

Attention模块的计算公式如下：

$Attention(Q,K,V)=softmax(\frac{QK^{T}}{\sqrt{d_{k}}})V$

[图片]

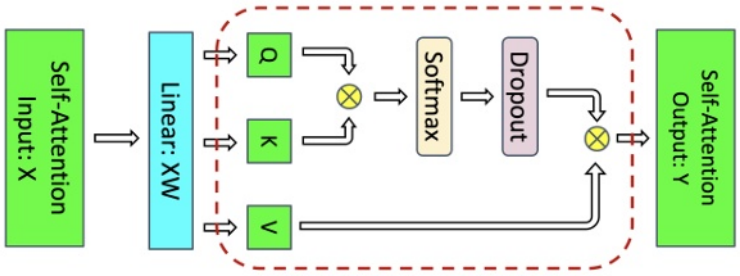


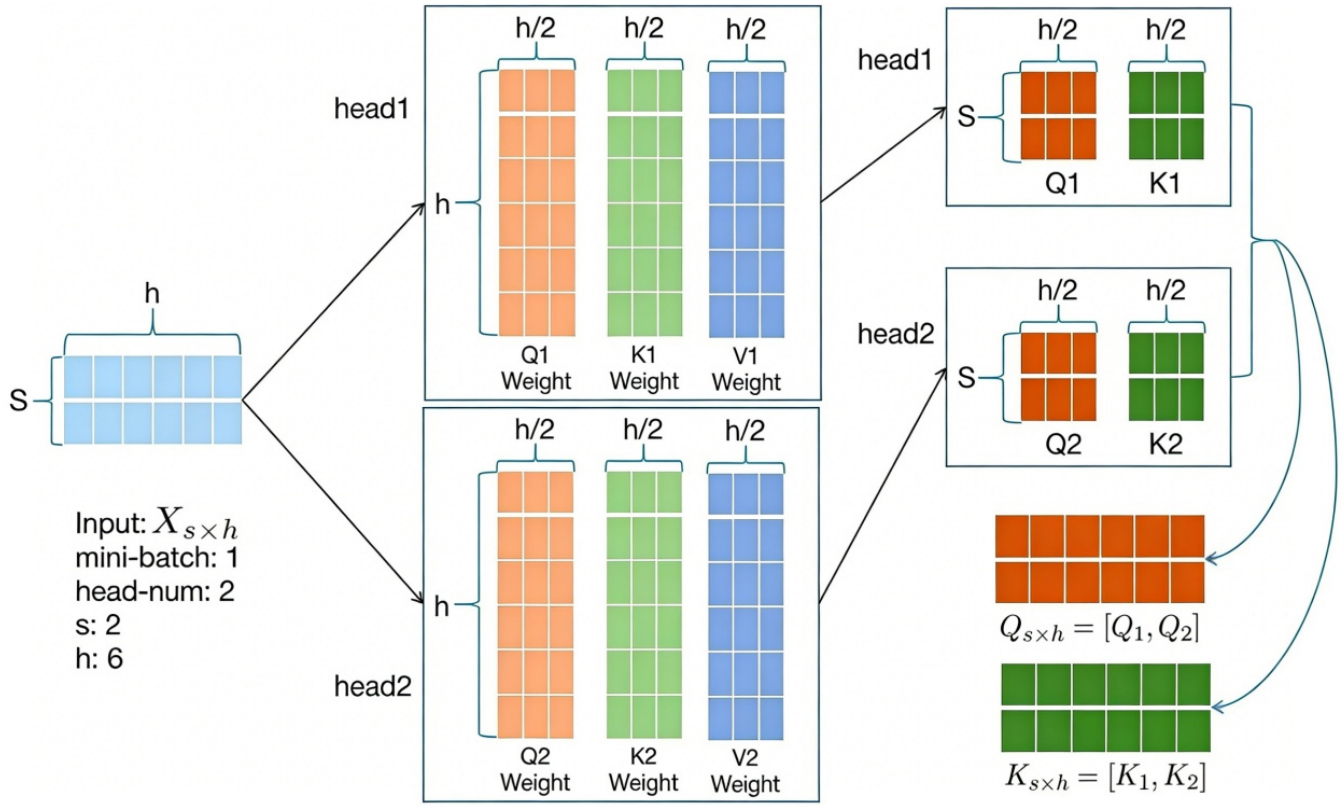
Figure 3. Self-attention block. The red dashed line shows the regions to which selective activation recomputation is applied (see Section 5 for more details on selective activation recomputation).

对于Attention块来说，输入的element个数为sbh个，每个element以16-bit的浮点数(也就是2 bytes)来进行存储的话，对应输入的element大小为2sbh bytes，后续计算默认都是按bytes为单位进行计算。

Attention块中包含一个self-attention块、一个linear线性映射层和attention dropout层。对于linear线性映射层来说需要保存输入的Activation大小为2sbh, 对于attention dropout层需要mask的大小为sbh(对于一个元素的mask只用1个bytes即可)，对于self-attention块的Activation Memory的计算有以下几块：

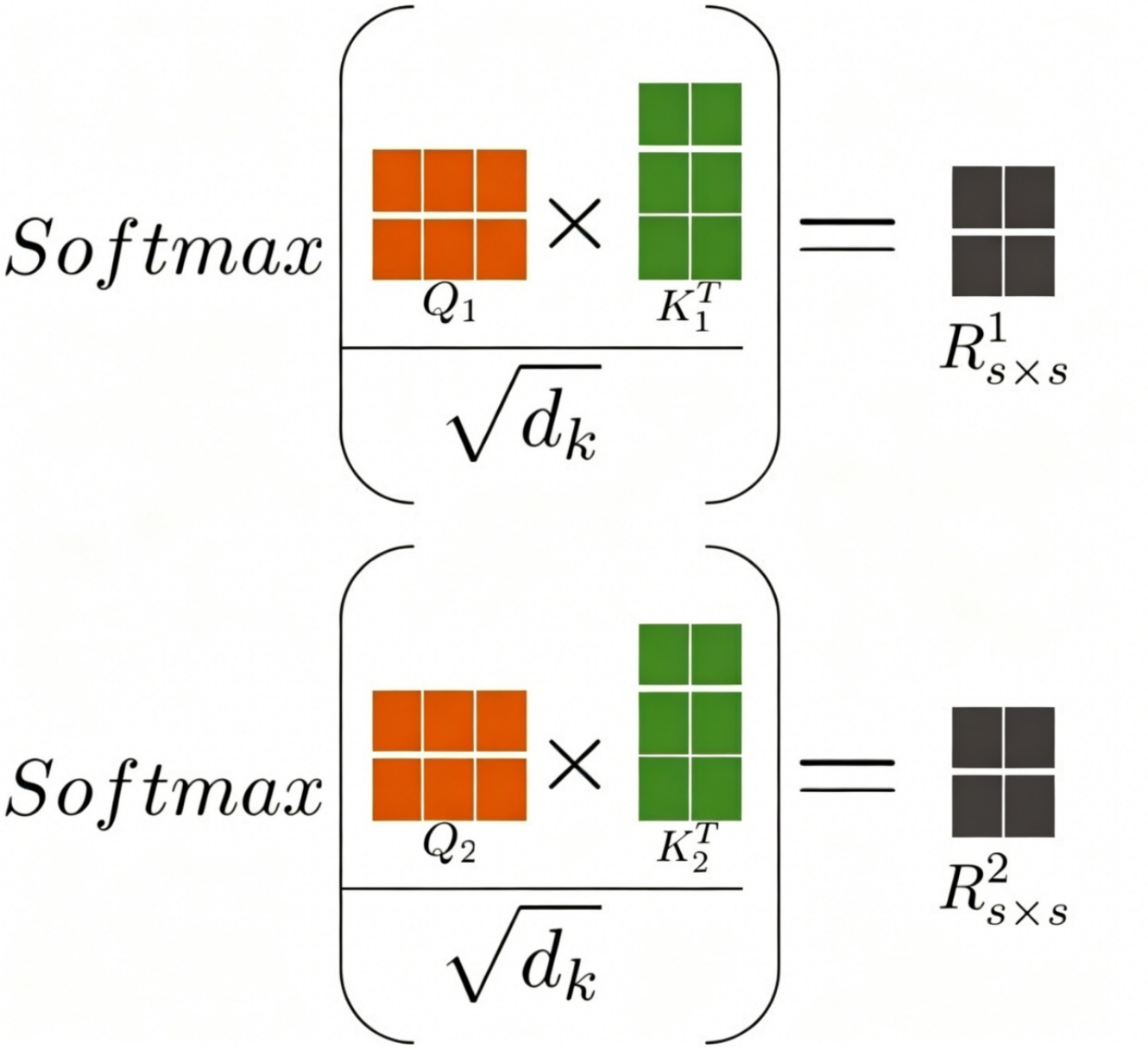
- $Query(Q)$, $Key(K)$, $Value(V)$ 矩阵相乘：输入$input$是共享的，元素个数为 sbh 个，总大小是 $2sbh$ bytes。
- QK^{T} 矩阵相乘：需要分别创建保存Q和K的矩阵，每个矩阵元素总大小为$2sbh$bytes，总共大小为$4sbh$bytes。
  如下图以b=1, s=2, h=6为例，输入X元素个数为1 * s * h = 12个，计算完后Q和K的矩阵中元素个数各有1 * s * h = 12个，总元素大小为2 * 2 * b * s * h = 48bytes。

[图片]



・ $softmax$ 的输出总的元素大小为$2as^{2}b$bytes，分别计算每个 Head 头的$Q{n}×K{n}$的乘积。计算公式如下，图中计算以b=1，s=2，h=6，a=2为例：

[图片]



·

在$softmax$后还有dropout的mask层大小，mask矩阵的大小与$softmax$的输出一样，元素个数都是$as^{2}b$个，但mask单个元素的大小只用1 bytes即可，总的大小为$as^{2}b$bytes

· softmax的输出也会用于反向的计算，需要缓存下来，对应大小也是$2as^{2}b$

· V矩阵的大小之前没有统计，和Q、K矩阵一样，大小也是$2sbh$ bytes

综上，Attention Block总的大小为 11sbh + 5as^2b bytes。

MLP的Activation大小计算：MLP中有两层线性layer，分别存储输入矩阵大小为2bshbytes和8sbhbytes；GeLU的反向也需要对输入进行缓存，大小为8sbhbytes; dropout层需要sbhbytes; 总大小为19sbh。

LayerNorm的Activation大小计算：每个LayerNorm层的输入需要2bsh大小，有两个LayerNorm层，总大小为4sbhbytes.

最终transformer网络中一层(含Attention/MLP/LayerNorm)的Activation总的大小为：

$ActivationMemoryPerLayer=sbh(34+5\frac{as}{h})$

注意: 这里公式计算的Activation总和是在没有应用模型并行策略的前提下进行的。

ii. ** Tensor Parallel的Activation Memory计算**
如下图，在Tensor模型并行中只在Attention和MLP两个地方进行了并行计算，对于Attention(Q/K/V)和MLP(Linear
Layer)的输入并没有并行操作。图中f和f互为共轭(conjugate)，f在前向时不执行操作，反向时执行all-reduce；f在前向时执行all-reduce，反向时不执行操作。

[图片]

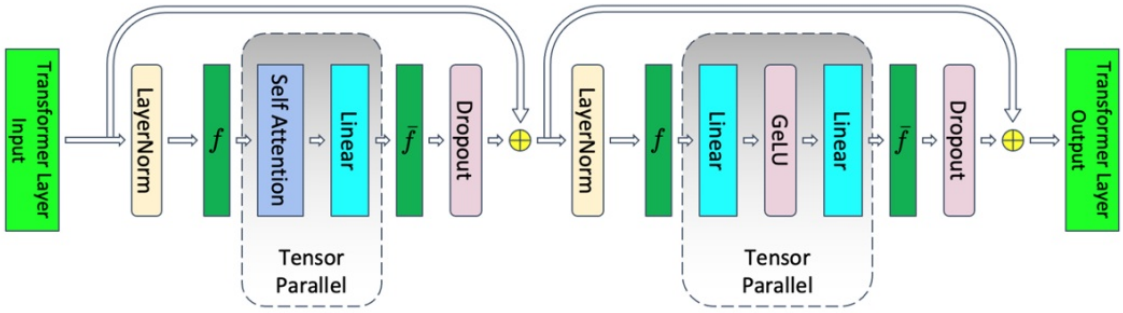


Figure 4. Transformer layer with tensor parallelism. f and $\bar{f}$ are conjugate. f is no operation in the forward pass and all-reduce in the backward pass. $\bar{f}$ is all-reduce in the forward pass and no operation in the backward pass.

参看上Tensor并行的话(Tensor并行度为 t)，并行部分有MLP的Linear部分(18sbh bytes)和Attention的QKV部分(6sbh bytes)，ActivationMemoryPerLayer 为:

$ActivationMemoryPerLayer=sbh(10+\frac{24}{t}+5\frac{as}{h})$

iii. Sequence Parallel
在Tensor模型并行基础上提出了Sequence
Parallel，对于非Tensor模型并行的部分在sequence维度都是相互独立的，所以可以在sequence维度上进行拆分(即sequence
parallel)。拆分后如下图，f和f替换为g和g，g和g也是共轭的，g在前向是all-gather通信，反向是reduce-scatter通信；g在前向是reduce-scatter，反向是all-gather通信。

[图片]

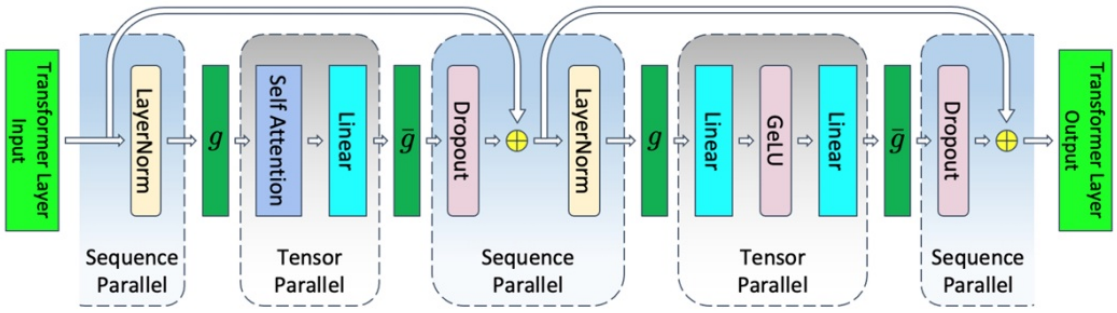


Figure 5. Transformer layer with tensor and sequence parallelism. g and $\bar{g}$ are conjugate. g is all-gather in the forward pass and reduce-scatter in the backward pass. $\bar{g}$ is reduce-scatter in forward pass and all-gather in backward pass.

接下来以MLP为例，详细说明拆分步骤。MLP层由两个Linear层组成，对应的计算公式如下，其中X的大小为 $s×b×h$ ； A 和 B 是Linear的权重weight矩阵，大小为 $h×4h$ 和 $4h×h$ 。

$Y=LayerNorm(X)Z=GeLU(YA)W=ZBV=Dropout(W)$

如下图，切分时说明如下：

1、对X按sequence维度切分，$X=[X{1}^{s},X{2}^{s}]$，LayerNorm的结果 $Y=[Y{1}^{s},Y{2}^{s}]$；

2、由于接下来的GeLU不是线性的，所以要进行all-gather操作，计算 $Z=GeLU(YA)$；

3、对A进行列切分的tensor并行，得到结果 $YA{1}^{c}$ 和 $YA{2}^{c}$

4、对B进行行切分的tensor并行，得到结果 $Z{1}^{h}B{1}^{r}$ 和 $Z{2}^{h}B{2}^{r}$

5、得到$W{1}$和$W{2}$后进行累加操作(reduce-scatter)

[图片]

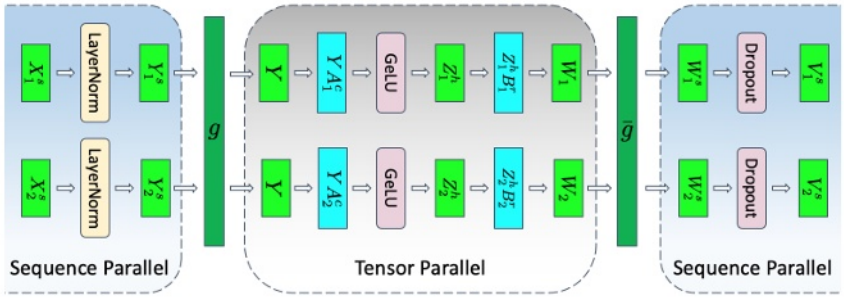


Figure 6. MLP layer with tensor and sequence parallelism. g and $\bar{g}$ are conjugate. g is all-gather in forward pass and reduce-scatter in backward pass. $\bar{g}$ is reduce-scatter in forward pass and all-gather in backward pass.

对应的计算公式如下：

$=LayerNorm([X{1}^{s},X{2}^{s}])Y=g(Y{1}^{s},Y{2}^{s})[Z{1}^{h},Z{2}^{h}]=[GeLU(YA{1}^{c}),GeLU(YA{2}^{c})]W{1}=Z{1}^{h}B{1}^{r}W{2}=Z{2}^{h}B{2}^{r}[W{1}^{s},W{2}^{s}]=g(W{1},W{2})[V{1}^{s},V{2}^{s}]=[Dropout(W{1}^{s}),Dropout(W{2}^{s})]$

Tensor并行在一次前向和后向总共有4次的all-reduce操作，在Sequence并行一次前向和后向总共有4次all-gather和4次reduce-scatter操作。ring all-reduce
执行过程中有两步，先是一个reduce-scatter然后跟着一个all-gather，Sequence并行相比没有引入更多的通信代价。一个使用reduce-scatter和all-gather实现all-reduce的Python代码示例如下：

```
import torch
import torch.distributed as dist

# 初始化进程组
dist.init_process_group(backend='gloo')

# 获取进程组中的进程数和当前进程的排名
world_size = dist.get_world_size()
rank = dist.get_rank()

# 定义输入和输出张量
x = torch.tensor([1, 2, 3, 4])
result = torch.zeros_like(x)
```

```
# 使用 reduce_scatter 将每个进程的输入张量的部分和归约到每个进程的输出张量
dist.reduce_scatter(input_list=[x], output=result)

# 使用 all_gather 将每个进程的输出张量收集到所有进程中
output_list = [torch.zeros_like(result) for _ in range(world_size)]
dist.all_gather(output_list, result)

# 在每个进程上打印结果
print(f"Process {rank}: {output_list}")

# 清理资源
dist.destroy_process_group()
```

通过使用 sequence parallel 和 tensor parallel 以后，ActivationMemoryPerLayer为：

$ActivationMemoryPerLayer=sbh(\frac{10}{t}+\frac{24}{t}+5\frac{as}{ht})=\frac{sbh}{t}(34+5\frac{as}{h})$

iv. Pipeline Parallel
加上Pipeline Parallel后，对具有L层的 layer 的transformer来说，Pipeline Parallel 并行度为p，对应会分为$\frac{L}{p}$组（即 stage 个数）。以 PipeDream 中的1F1B调度为例，要完成初始化的话，第1个stage必须处理完p个micro-batch，让其他 stage 至少有1个micro-batch在处理，也就是要缓存p个micro-batch的activation。由于每个stage都有$\frac{L}{p}$个 layer，一共需要$p\times\frac{L}{p}=L$个layer的activation信息，对应总的计算如下：

$TotalActivationMemory=\frac{sbhL}{t}(34+5\frac{as}{h})$

当然这里的公式ActivationMemory的计算没有加上EmbeddingLayer、最后的LayerNorm和输出的OutputLayer。加上这三部分的结果会略大于以上公式，但以22B参数模型来说只增加0.01%的大小，这部分可忽略，证明请参考原论文。未计算部分如下图红色部分：

[图片]

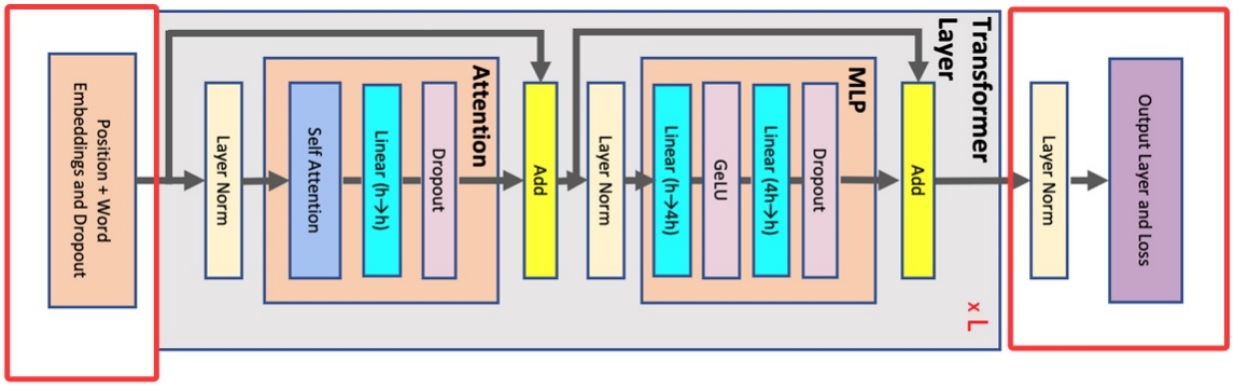


Figure 2. Transformer Architecture. Each gray block represents a single transformer layer that is replicated L times.

v. 可选Activation重计算介绍(Selective Activation Recomputation)
在反向过程中通过重计算方式重新计算前向结果来节省显存大小，这种方式文中称为full activation recomputation，以 transformer 为例会增加 30%~40% 的计算量。selective 的方式主要思路是选择 FLOPs 计算量小，且
activation占用大的算子进行重计算，这里的FLOPs的衡量标准是GEMM的计算量大小。以TotalActivationMemory公式为例，针对大模型来说$5as/h>34$，如果重计算这部分layer的话可以减少快一半的acti

vation大小。对于GPT-3来说，这种方式可以减少70% 的activation显存大小，同时只增加了 2.7% 的FLOPs计算量。采用Selective Activation Recomputation后，TotalActivationMemory公式的结果可以减少为：

$Totalrequiredmemory=34\frac{sbhL}{t}$

以下是不同方法组合下 Activation Memory 占用情况：

[图片]

Configuration	Activations Memory (bytes)	FLOPs	Bytes Communicated
no parallelism	$sbh\left(34+5\frac{as}{h}\right)$	$72sbh^2\left(1+\frac{s}{6h}\right)$	0
tensor parallel (baseline)	$sbh\left(10+\frac{24}{t}+5\frac{as}{ht}\right)$	$\frac{72sbh^2}{t}\left(1+\frac{s}{6h}\right)$	$16\frac{t-1}{t}sbh$
tensor + sequence parallel	$sbh\left(\frac{34}{t}+5\frac{as}{ht}\right)$		
tensor parallel + selective activation recomputation	$sbh\left(10+\frac{24}{t}\right)$	$\frac{72sbh^2}{t}\left(1+\frac{2s}{9h}\right)$	
tensor parallel + sequence parallel + selective activation recomputation	$sbh(\frac{34}{t})$		
full activation recomputation	$sbh(2)$	$\frac{96sbh^2}{t}\left(1+\frac{s}{6h}\right)$	$24\frac{t-1}{t}sbh$

Table 2. Activations memory, FLOPs, and communication bytes per transformer layer for different techniques.

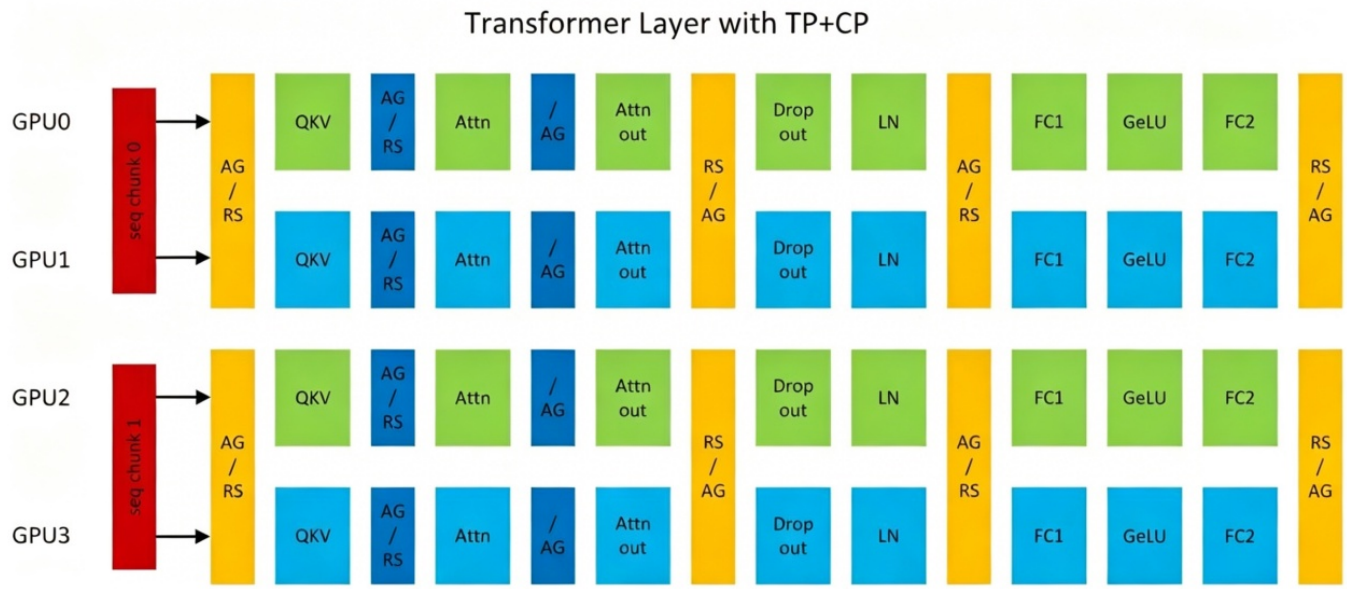
b、上下文并行（context parallel）

i. 原理介绍
megatron中的context并行(简称CP)与sequence并行(简称SP)不同点在于，SP只针对Layernorm和Dropout输出的activation在sequence维度上进行切分，CP则是对所有的input输入和所有的输出activation在sequence维度上进行切分，可以看成是增强版的SP。除了Attention模块以外，其他的模块(Layernorm、Dropout)由于没有多token的处理，在CP并行时都不用任何修改。

为什么Attention模块是个例外?
因为Attention计算过程中每个token的Q(query)要跟同一个sequence中其他token的K(key)和V(value)一起进行计算，存在计算上的依赖，所以通过CP并行后，在计算Attention前要通过allgather通信拿到所有token的KV向量，在反向计算时对应需要通过reduce_scatter分发gradient梯度。

为了减少显存占用，在前向时每个gpu只用保存一部分KV块，反向时通过allgather通信拿到所有的KV数据。KV的通信发生在相邻TP通信组相同位置的rank之间。allgather和reduce_scatter在ring拓扑架构实现时，底层会通过send和recv来进行实现。

[图片]



以上图TP2-CP2的transformer网络为例，在Attention前的是CP的通信算子，其他都是TP的通信算子。AG表示allgather, RS表示reduce_scatter, AG/RS表示前向allgather反向reduce_scatter, RS/AG表示前向reduce_scatter反向allgather。

这里TP2对应为[GPU0, GPU1], [GPU2, GPU3], CP2对应为TP组相同位置的rank号，也就是[GPU0, GPU2], [GPU1, GPU3]。CP并行与Ring Attention类似，但是提供了新的OSS与FlashAttention版本，也去除了low-triangle causal masking的冗余计算。

LLM经常由于sequence长度过长导致显存OOM，这时之前的一种方式是通过重计算的方式保存中间的activation产出，全量重计算的劣势会带来30%的计算代价；另外一种方式是扩大TP(tensor parallel)的大小，扩大TP的劣势在于会对tensor切的更小，从而导致linear fc的计算时间变少，从而与通信很难进行计算的掩盖。

通过CP可以更好解决OOM的问题，每个GPU只用处理一部分的sequence, 同时减少CP倍的通信和计算，但保持TP不变，同时activation也会减少CP倍。CP优化的性能参考如下图，在Megatron中(Megatron-Core>=0.5.0 && **Transformer Engine **>=1.1)通过指定--context-parallel-size可以进行使用。

$total_gpu_count=TP×CP×PP×DP$

6.5.2.5 专家并行（EP）

MoE核心：采用“分而治之”，将FFN拆分为多个独立专家。每个Token仅由Top-K专家处理并经门控聚合，使参数量随专家数线性增长，而计算量仅由少数激活专家决定（计算稀疏）。

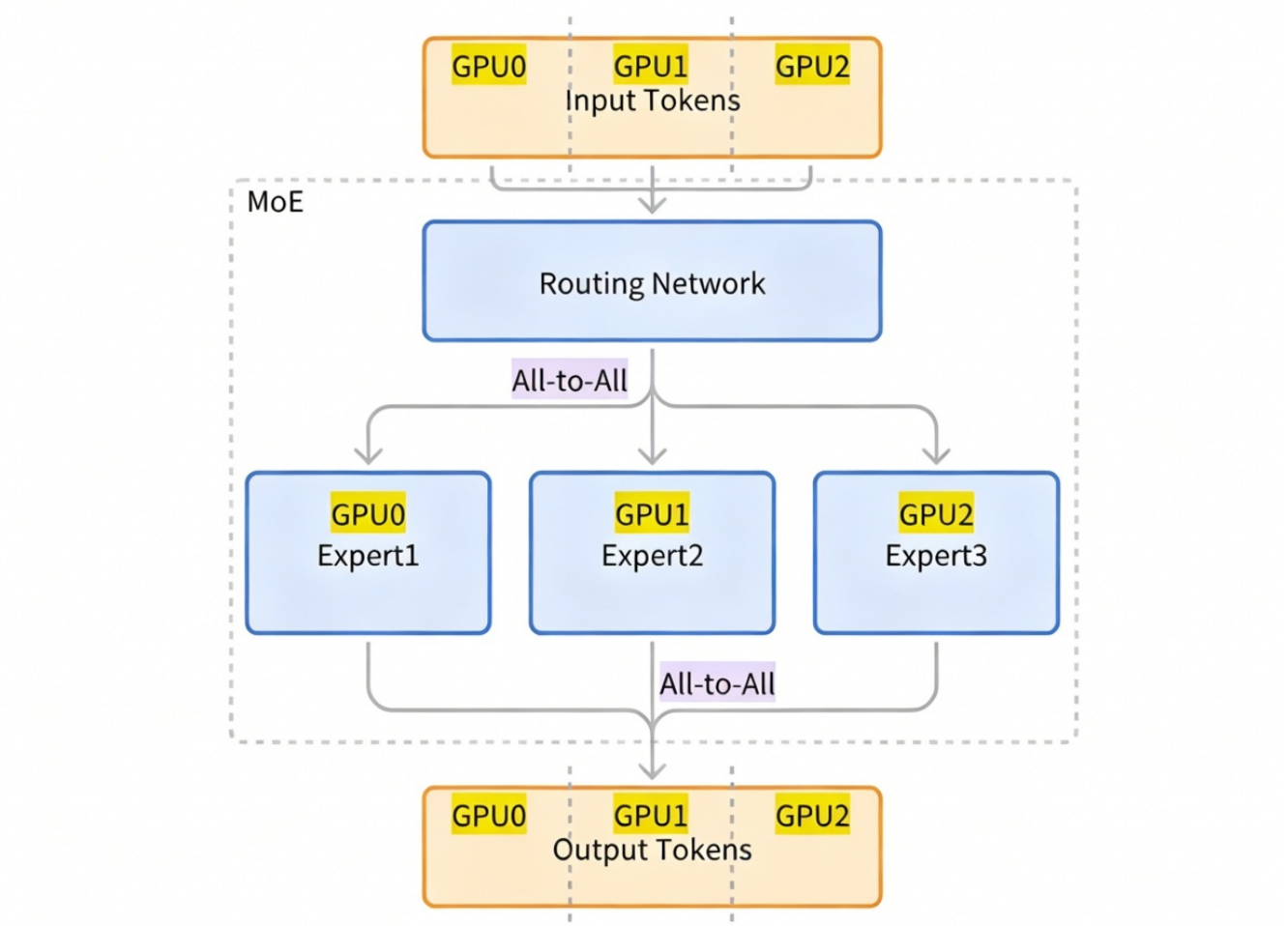
专家并行（EP）：当专家规模达数千/上万时，单卡显存无法容纳，需将专家均匀分片至各GPU（每卡存专家数/world_size 个）。其参数分布思路与TP相似，但粒度与实际效果截然不同。

All-to-All通信（EP灵魂）：因Token路由动态随机，前向传播必须跨卡搬运数据，依赖All-to-All通信解决两大关键步骤：

- Dispatch：将输入Token精准分发至目标专家所在GPU；
- Combine：将专家输出回传至原始GPU并完成结果聚合。

我们具体看看EP的过程。简单起见，这里我们假设有3个expert，3张GPU。这样，每个GPU负责一个expert。

[图片]



在进入MoE部分前，我们已经通过TP和SP得到了attention输出的全部token，并且每个GPU上都有全部的token。但是每个GPU都对所有的token计算routing network并没有必要。这里，一般会约定,GPU 0只负责处理序列的前三分之一部分Token（比如 Token 0-15）；GPU 1负责中间三分之一（比如 Token 16-31）；GPU 2负责最后三分之一（比如 Token 32-47）。这里从逻辑上来看，类似SP。

之后，routing network会对每个Token进行计算，得到这个Token需要去到的expert ID以及对应的score。

但问题来了，当GPU0拿到一批数据，假设16个token，但routing network计算后，发现不同的token要去不同的专家，而这些专家都分布在不同的GPU上。与此同时，其他GPU上的数据也想去GPU 0负责的Expert 0 那里。

这里每张卡都要给其他的卡发数据，就是我们所称的All-to-All操作。在 MoE模型的推理中，All-to-All是通信开销最大的环节之一。

我们先阐述All-to-All的具体实现，重点阐述一下，如下图，在一个moe layer里面，完整的过程包括all-to-all dispatch，MoE function（即各个专家MLP），all-to-all combine三阶段。

[图片]

每个方块代表1个token，每个颜色用来更好看一下这些token在dispatch和combine前后的设备变化。举个例子，A1一开始分布在rank1，它选中的专家在rank0，经过dispatch后，它就跑rank0去了，跑过去之后，就和rank0上的专家斗地主，乘了个1，然后又回到自己家rank1去了（combine）。每个token都遵循类似的过程。

下面我们跟着代码来看看实现细节，完整代码比较长，这里摘重点

1、 一些数据的初始化

初始化时，
明确专家数量、设备数及每个设备承载的专家数（num_local_experts），并预留足够的缓冲区以应对动态 token 数量（max_recv）。

```
class AllToAll:
  # 元数据维度：global_exp, src_rank, src_token, src_k, 记录token从哪儿来的，用于combine把token发回

  META_DIM = 4
  def __init__(self, cfg: MoEConfig, rank: int, world_size: int):
    self.cfg = cfg
    self.rank = rank  # 当前设备编号
    self.world_size = world_size  # 总设备数
    self.num_local_experts = cfg.num_experts // world_size  # 每个设备上的专家数
    self.max_recv = cfg.max_num_tokens * world_size  # 最大接收token数
```

2、Dispatch 阶段：将 token 分发到目标专家

Dispatch 的目标是：让每个 token 到达其选中的专家所在设备。代码中dispatch方法分为 6 个关键步骤：

步骤 1：计算发送 / 接收计数

首先统计当前设备需要向其他设备发送的 token 数量（send_counts），以及需要从其他设备接收的 token 数量（recv_counts）：

```
# 统计每个目标设备需要接收的token数
send_counts = [0] * self.world_size
token_map = [[] for _ in range(self.world_size)]  # 记录每个设备接收了哪些token
meta_map = [[] for _ in range(self.world_size)]  # 记录每个token的元数据
for t, expert_list in enumerate(indices.tolist()): # indices：记录了每个token选中的专家列表
  for k, e in enumerate(expert_list):
    dst_rank = e // self.num_local_experts  # 专家e所在的目标设备
```

```
        send_counts[dst_rank] += 1
        token_map[dst_rank].append(t)  # 记录token索引t需要发送到dst_rank
        # 元数据：专家ID、源设备、源token索引、选中的专家序号（k）
        meta_map[dst_rank].extend([e, self.rank, t, k])
```

通过torch.distributed.all_to_all_single交换send_counts，得到每个设备需要接收的 token 数recv_counts_t：

```
send_counts_t = torch.tensor(send_counts, dtype=torch.long, device=device)
recv_counts_t = torch.empty(self.world_size, dtype=torch.long, device=device)
dist.all_to_all_single(recv_counts_t, send_counts_t)  # 交换计数
```

步骤 2：构建发送缓冲区

将需要发送的 token 和元数据整理为连续的缓冲区：

```
# 构建token发送缓冲区（按目标设备拼接）
send_buf = torch.cat([dp_x[idx_list] for idx_list in token_map], dim=0)

# 构建元数据发送缓冲区（按META_DIM维度整理）
send_meta = torch.tensor(
    [v for sub in meta_map for v in sub], dtype=torch.int32, device=device
).view(-1, self.META_DIM)
```

步骤 3：All-to-All 数据交换

通过torch.distributed.all_to_all_single将send_buf和send_meta发送到目标设备，同时接收来自其他设备的数据：

```
# 接收token的缓冲区（大小为总接收数×隐藏维度）
total_recv = int(recv_counts_t.sum().item())
recv_buf = torch.empty(total_recv, cfg.hidden_dim, dtype=cfg.in_dtype, device=device)

# 接收元数据的缓冲区
recv_meta = torch.empty(total_recv, self.META_DIM, dtype=torch.int32, device=device)

# 交换token数据
dist.all_to_all_single(
    recv_buf, send_buf,
    output_split_sizes=recv_counts_t.tolist(), # 每个源设备接收的数量
    input_split_sizes=send_counts_t.tolist()  # 每个目标设备发送的数量
)

# 交换元数据（元数据长度是token数×META_DIM）
dist.all_to_all_single(
    recv_meta.view(-1), send_meta.view(-1),
    output_split_sizes=[c * self.META_DIM for c in recv_counts_t.tolist()],
    input_split_sizes=[c * self.META_DIM for c in send_counts_t.tolist()]
)
```

步骤 4：分发到本地专家

接收完成后，将recv_buf中的 token
按元数据分配到当前设备的本地专家的token缓冲区expert_x，它和expert_meta,expert_num_tokens也正是dispatch的输出：

```
# 初始化本地专家的token缓冲区
expert_x = torch.empty(
  (self.num_local_experts, self.max_recv, cfg.hidden_dim),
  dtype=cfg.in_dtype, device=device
)
expert_meta = torch.empty(
  (self.num_local_experts, self.max_recv, self.META_DIM),
  dtype=torch.int32, device=device
)
expert_num_tokens = torch.zeros(self.num_local_experts, dtype=torch.int32, device=device)

# 按元数据分配token到对应专家
for i in range(total_recv):
  global_eid = int(recv_meta[i, 0].item()) # 全局专家ID
  local_eid = global_eid % self.num_local_experts # 本地专家ID（相对于当前设备）
  # 记录token数据、元数据和计数
  expert_x[local_eid, expert_num_tokens[local_eid]] = recv_buf[i]
  expert_meta[local_eid, expert_num_tokens[local_eid]] = recv_meta[i]
  expert_num_tokens[local_eid] += 1
```

至此，Dispatch 阶段完成：每个 token
已到达其选中的专家所在设备，并被正确分配到本地专家的缓冲区

3 Combine 阶段：聚合专家输出结果

Combine 的目标是将专家处理后的结果回传到原始 token
所在设备，并按门控权重聚合。combine方法的核心逻辑与 Dispatch 对称，但方向相反：

步骤 1：统计回传计数

首先确定当前设备的专家输出需要回传到哪些设备：

```
send_counts = [0] * self.world_size
y_map = [[] for _ in range(self.world_size)] # 专家输出的token
meta_map = [[] for _ in range(self.world_size)] # 回传的元数据
for local_eid in range(self.num_local_experts):
  cnt = int(expert_num_tokens[local_eid].item()) # 该专家处理的token数
  for j in range(cnt):
    meta = expert_meta[local_eid, j] # 提取元数据（包含原始设备信息）
    dst_rank = int(meta[1].item()) # 原始token所在设备
    send_counts[dst_rank] += 1
    y_map[dst_rank].append(expert_y[local_eid, j].unsqueeze(0)) # 专家输出
    meta_map[dst_rank].extend(meta.tolist()) # 回传元数据
```

步骤 2：All-to-All 回传数据

通过dist.all_to_all_single将专家输出和元数据回传到原始设备：

```
# 构建发送缓冲区
send_buf = torch.cat([torch.cat(sub_list, dim=0) if sub_list else torch.empty(0) for sub_list in y_map],
dim=0)
send_meta = torch.tensor([v for sub in meta_map for v in sub], dtype=torch.int32,
device=device).view(-1, self.META_DIM)

# 接收缓冲区
total_recv = int(recv_counts_t.sum().item())
recv_buf = torch.empty(total_recv, cfg.hidden_dim, dtype=cfg.out_dtype, device=device)
recv_meta = torch.empty(total_recv, self.META_DIM, dtype=torch.int32, device=device)

# 回传数据和元数据
dist.all_to_all_single(recv_buf, send_buf, ...)
dist.all_to_all_single(recv_meta.view(-1), send_meta.view(-1), ...)
```

步骤 3：加权聚合

最后，根据元数据找到原始
token，并按门控权重（weights）聚合结果，out_tokens即combine的输出：

```
for i in range(total_recv):
    src_token = int(recv_meta[i, 2].item())  # 原始token索引
    src_k = int(recv_meta[i, 3].item())     # 该token选中的第k个专家
    w = weights[src_token, src_k].to(torch.float32)  # 门控权重
    out_tokens[src_token] += recv_buf[i].to(torch.float32) * w  # 加权累加
```

至此，Combine 阶段完成：所有专家的输出已按权重聚合到原始
token，并且原始token也回到了原始rank，形成最终结果。

我们回顾上述GPU分配，假设Expert 0在GPU
0上已经算出结果了。其他Expert在其他GPU上也算完了。但此时的Token是按专家分组的，而不是按原始句子顺序。无法直接用来进行下一层的计算。

重组的过程也是一次All-to-All
，这是第一次All-to-All的逆向过程。通信结束后，每个GPU重新拿回了它最初那一批 Token
的计算结果。注意此时拿回来的数据虽然在正确的 GPU
上了，但顺序还是乱的。比如，它可能先排列了去过 Expert 1 的 Token，再排列了去过 Expert 2
的。所以会根据一开始 Router 记录的索引，把这些乱序的 Token
结果一个个插回到原来的序列位置上。

最后，每个专家把我们把得到的Token乘以它的置信度
Score，然后加到原始输入X上。至此，一个完整的 MoE 层EP计算结束，数据又变成了标准的形状
[Batch, Seq_Len, Hidden_Dim]，准备进入下一个 Transformer Block。

6.5.3 大模型训练优化

6.5.3.1 重计算(recompute)

a、recompute参数配置

在megatron/arguments.py中有重计算的参数配置如下：

```
group.add_argument('--recompute-activations', action='store_true',
                   help='recompute activation to allow for training '
                   'with larger models, sequences, and batch sizes.')
group.add_argument('--recompute-granularity', type=str, default=None,
                   choices=['full', 'selective'],
                   help='Checkpoint activations to allow for training '
                   'with larger models, sequences, and batch sizes. '
                   'It is supported at two granularities 1) full: '
                   'whole transformer layer is recomputed, '
                   '2) selective: core attention part of the transformer '
                   'layer is recomputed.')
group.add_argument('--distribute-saved-activations',
                   action='store_true',
                   help='If set, distribute recomputed activations '
                   'across model parallel group.')
group.add_argument('--recompute-method', type=str, default=None,
                   choices=['uniform', 'block'],
                   help='1) uniform: uniformly divide the total number of '
                   'Transformer layers and recompute the input activation of '
                   'each divided chunk at specified granularity, '
                   '2) recompute the input activations of only a set number of '
                   'individual Transformer layers per pipeline stage and do the '
                   'rest without any recomputing at specified granularity'
                   'default) do not apply activations recompute to any layers')
group.add_argument('--recompute-num-layers', type=int, default=1,
                   help='1) uniform: the number of Transformer layers in each '
                   'uniformly divided recompute unit, '
                   '2) block: the number of individual Transformer layers '
                   'to recompute within each pipeline stage.')
```

说明：

- --recompute-activations:
  设置recompute_activations等同于recompute_granularity为selective；selective运行效率更高，大部分场景只设置这个就可以。如果显存更紧张时，再通过recompute-granularity来进行full的设置。
- --recompute-granularity:
  支持不同颗粒度的重计算，设为full会重计算整个transformer层，设为selective只会重算transformer中的core_attention部分。
- --distribute-saved-activations: 按TP并行度分开存储activation。
- --recompute-method: uniform计算会把所有的transformer layer分为若干组，分别把每组的input activation保存在内存中，GPU显存不足时，可通过设大每个组内的layer数来运行更大的model；block是针对pipeline并行的每个stage，checkpoint部分transformer layer的input activation，剩余部分不进行checkpoint缓存，对于一个pipeline

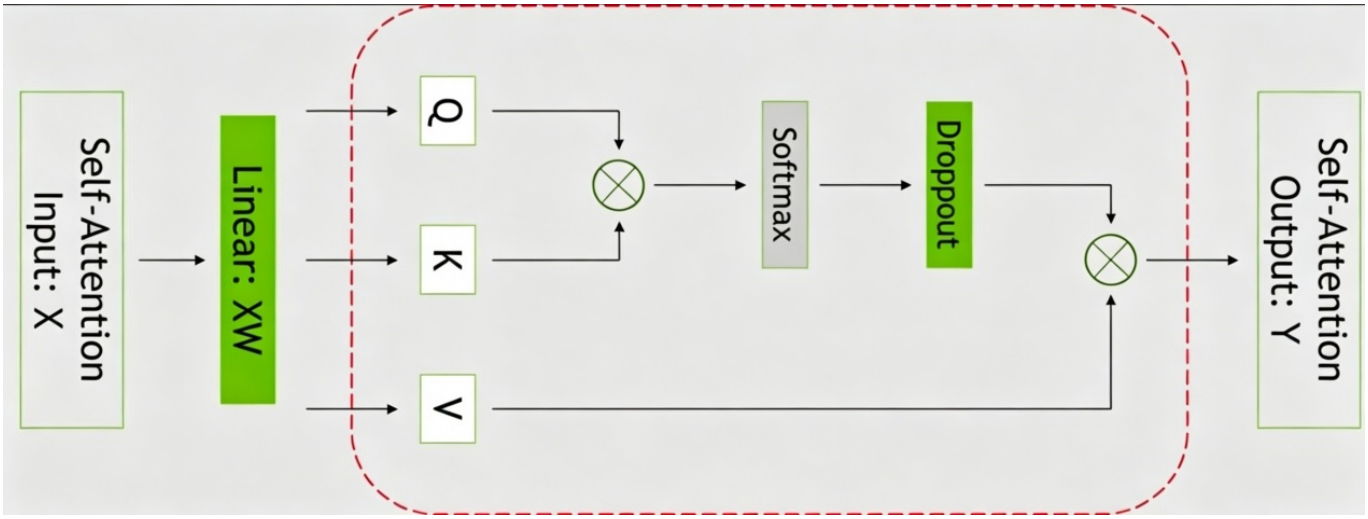
stage中有8层的来说，当设为5时，前5层中每一层的input
activation都会被缓存，后3层在反向的时候正常计算。
• --recompute-num-layers: 对于uniform类型，表示设置在每个重计算的transformer layer
group中的层数, 默认为1表示对每一层transformer
layer都分别进行checkpoint；对于block类型，设为N表示单个pipeline
stage中的前N个layers会缓存input activation。
b、源码详解

i. recompute-activations
设置recompute_activations等同于recompute_granularity为selective，设置后会覆盖recompute_granularity的值。

```
if args.recompute_activations:
  args.recompute_granularity = 'selective'
del args.recompute_activations
```

ii. recompute-granularity
支持不同颗粒度的重计算，设为full会重计算整个transformer层，设为selective只会重算transformer中的core_attention部分。下图红框中为选择重计算的部分：

[图片]



入口是在megatron/model/transformer.py文件中的ParallelTransformer函数中，
在forward中如果为full的话会对整个前向进行checkpoint操作。

```
class ParallelTransformer(MegatronModule):

  def forward(self, hidden_states, attention_mask,
        encoder_output=None, enc_dec_attn_mask=None,
        retriever_input=None,
        retriever_output=None,
        retriever_attn_mask=None,
        inference_params=None,
        rotary_pos_emb=None):
        ...
        # Forward pass.
        if self.recompute_granularity == 'full':
```

```
            hidden_states = self._checkpointed_forward(hidden_states,
                                                       attention_mask,
                                                       encoder_output,
                                                       enc_dec_attn_mask,
                                                       rotary_pos_emb,
                                                       is_first_microbatch)
        ...
```

具体执行checkpoint的方法有两种，分别是uniform和block。对于uniform方法来说，每次会过self.recompute_num_layers个layer保存一次input activation。custom(l, l + self.recompute_num_layers), custom函数是用于执行自定义的前向计算。对于block方法，对于小于self.recompute_num_layers的layer执行input activation的checkpoint,
对于大于等于self.recompute_num_layers的layer还执行原有操作。

iii. distribute-saved-activations

只支持设置TP>1, 并且是full粒度的重计算, 对于uniform和block重计算都支持。

```
# Activation recomputing.
if args.distribute_saved_activations:
    assert args.tensor_model_parallel_size > 1, 'can distribute ' \
        'recomputed activations only across tensor model ' \
        'parallel groups'
    assert args.recompute_granularity == 'full', \
        'distributed recompute activations is only '\
        'application to full recompute granularity'
    assert args.recompute_method is not None, \
        'for distributed recompute activations to work you '\
        'need to use a recompute method '
    assert TORCH_MAJOR >= 1 and TORCH_MINOR >= 10, \
        'distributed recompute activations are supported for pytorch ' \
        'v1.10 and above (Nvidia Pytorch container >= 21.07). Current ' \
        'pytorch version is v%s.%s.' % (TORCH_MAJOR, TORCH_MINOR)
```

实现是在megatron/core/tensor_parallel/random.py文件的CheckpointFunction中，前向计算时使用no_grad()的上下文，不保存grad，在存activation的时候会把tensor展成一维的，每个rank只存自己的那一部分。

```
class CheckpointFunction(torch.autograd.Function):
    def forward(ctx, run_function, distribute_saved_activations, *args):
        ...

        with torch.no_grad():
            outputs = run_function(*args)

        # Divide hidden states across model parallel group and only keep
        # the chunk corresponding to the current rank.
        if distribute_saved_activations:
            ctx.input_0_shape = args[0].data.shape
            safely_set_viewless_tensor_data(
                args[0],
```

```
            split_tensor_into_1d_equal_chunks(args[0].data, new_buffer=True))
        ...

        # Store everything.
        ctx.save_for_backward(*args)

        return outputs
```

反向会在model_group中先通过gather_split_1d_tensor函数进行all_gather操作,
再进行backward的计算。

```
class CheckpointFunction(torch.autograd.Function):
   def backward(ctx, *args):
      if not torch.autograd._is_checkpoint_valid():
         raise RuntimeError("Checkpointing is not compatible with .grad(), "
                            "please use .backward() if possible")
      inputs = ctx.saved_tensors
      if ctx.distribute_saved_activations:
         safely_set_viewless_tensor_data(
            inputs[0],
            gather_split_1d_tensor(inputs[0].data).view(ctx.input_0_shape))
      ...
      # Compute the forward pass.
      detached_inputs = detach_variable(inputs)
      with torch.enable_grad():
         outputs = ctx.run_function(*detached_inputs)
      ...
      torch.autograd.backward(outputs, args)
      grads = tuple(inp.grad if isinstance(inp, torch.Tensor) else inp
                    for inp in detached_inputs)
      return (None, None) + grads
```

c、配置选择

i. selective方式
只设--recompute-activations相当于只用了selective, 只对core_attention的input
activation进行缓存。selective优点是新增的计算量相比较其他方案较少，性价比高。

```
--recompute-activations
```

ii. block方式
使用block策略，前N层每一层都保存对应的input
activation。可在pipeline并行中配合使用。--distribute-saved-activations可看情况使用。N等于pipeli
ne stage中的layer数，可以最大限度使用重计算，节省显存。

```
--recompute-granularity full \
--recompute-method block \
--recompute-num-layers [N] \
--distribute-saved-activations \
```

iii. uniform方式

使用uniform策略，每N个layer进行一次分组，每组会缓存输入input activation用于后续重计算，默认N为1表示对所有层都会，N等于1可以最大限度使用重计算，节省显存。在使用uniform基础上增加使用--distribute-saved-activations, 存activation时可以一个TP组进行分开存储。

```
--recompute-granularity full \
--recompute-method uniform \
--recompute-num-layers 1 \
--distribute-saved-activations \
```

6.5.3.2 Distributed-Optimizer分布式优化器

在megatron中指定--use-distributed-optimizer就能开启分布式优化器,
参数定义在megatron/arguments.py中。分布式优化器的思路是将训练中的优化器状态均匀地分布到不同数据并行的rank结点上，相当于开启ZERO-1的训练。

```
group.add_argument('--use-distributed-optimizer', action='store_true',
                   help='Use distributed optimizer.')
```

在使用--use-distributed-optimizer, 同时会check两个参数 args.DDP_impl == 'local'(默认开启)和args.use_contiguous_buffers_in_local_ddp(默认开启)。

```
# If we use the distributed optimizer, we need to have local DDP
# and we should make sure use-contiguous-buffers-in-local-ddp is on.
if args.use_distributed_optimizer:
  assert args.DDP_impl == 'local'
  assert args.use_contiguous_buffers_in_local_ddp
```

分布式优化器节省的理论显存值依赖参数类型和梯度类型，以下是每一个 parameter 对应占用的理论字节数（d 表示数据并行的大小，也就是一个数据并行中的卡数，等于 $TP \times PP$）：

[图片]

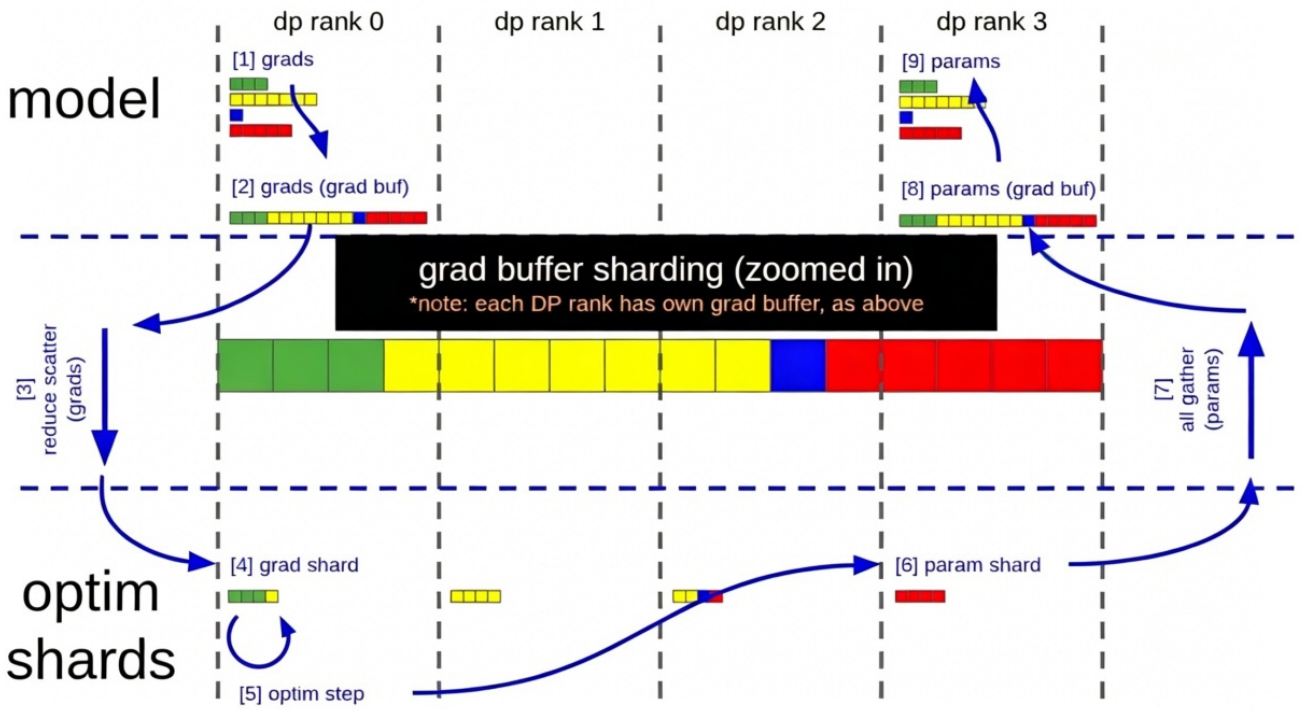
	训练数据类型	Non-distributed optim（单位 Byte）	Distributed o
2	float16 param, float16 grads	20	4
3	float16 param, fp32 grads	18	6
4	fp32 param, fp32 grads	16	8

点击图片可查看完整电子表格

Distributed-Optimizer分布式优化器的主要实现是通过连续的grad buffer来进行的，grad buffer中用于模型状态和优化器状态之间进行parameter参数和grad梯度的通信。grad buffer中使用reduce-scatter和all-gather进行通信。

数据流如下：

[图片]



1、在每个dp的rank上计算完grad后，组成待更新的grad buffer数组

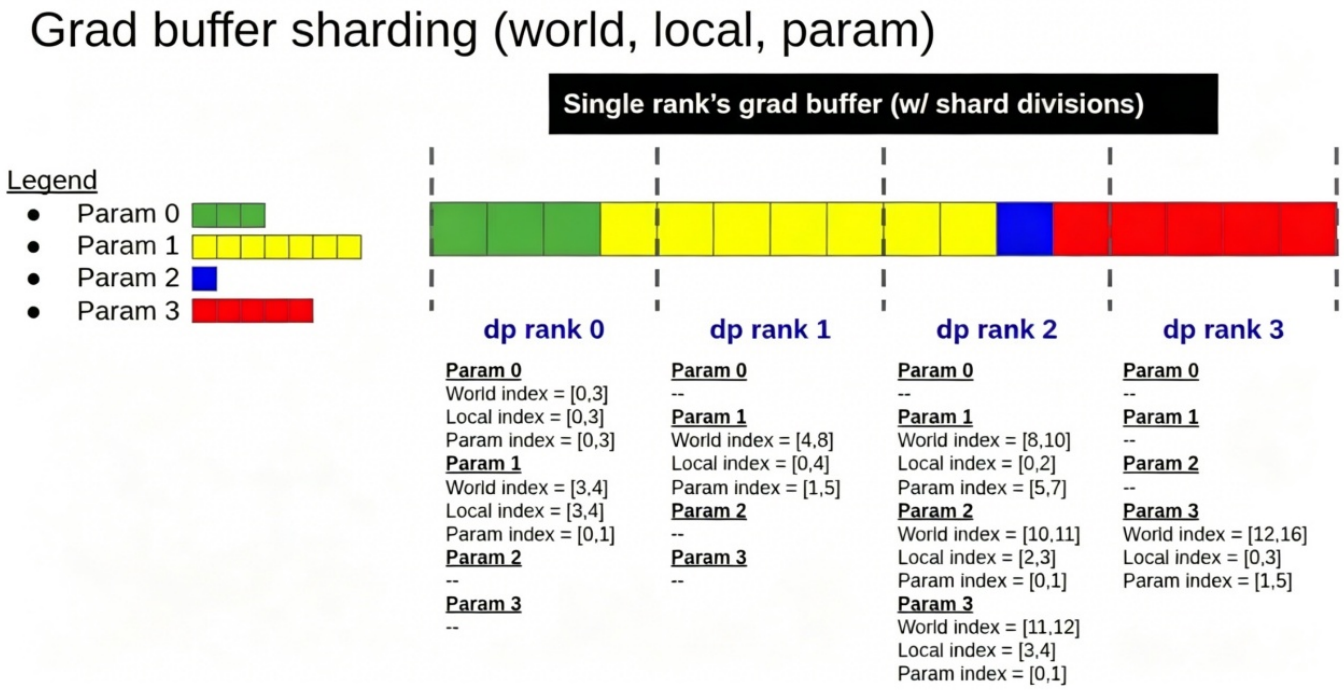
2、更新的时候通过reduce-scatter将grad buffer切分到各个rank上

3、在每个rank上完成优化器的step操作

4、最后将所有结果执行allgather操作得到更新后的grad buffer。

以fp16类型grad为例，grad buffer分片说明如下：

[图片]



- 一共有4个参数，分别用绿/黄/蓝/红表示；总参数大小为16个fp16类型数据
- 按DP中rank的个数对总数据均匀切分
- 如果参数过大，每个rank可能会只包含部分参数的数据，所以要考虑参数的偏移
- 每个DP

rank中的每个param参数都对应有3个偏移，一个是world_index表示总的数据偏移，一个是local_index表示在当前rank中的数据偏移，一个是param_index相对于param来说，表示当前rank结点存的数据的偏移。

-

以黄色参数Param1为例，在rank0存了Param1的一个元素，rank1存了Param1的4个元素；world_index来说rank0上黄色部分的元素是总数据的[3,4], rank1上黄色部分的4个元素是总数据的[4,8]; local_index来说在rank0上表示[3,4]，rank1表示当前结点全部的4个元素，范围也就是[0,4];param_index来说，对于rank0上的Param1的param_index就是[0,1]，在rank2上的param_index就是[1,5];

关键步骤详解：

- 上图中每个方块看成是一个grad buffer中的一个fp16类型元素，在反向结束以后，grad buffer中有16个fp16类型的元素
- 在每一个DP rank上调用reduce-scatter操作
- 每个DP rank的grad buffer中都有4个fp16类型元素经过了reduce-scatter操作更新，没更新的12个fp16类型元素等待后续垃圾回收
- 每个DP rank从grad buffer中拷贝更新后的4个fp16类型元素到fp32类型的main grad buffer中，准备开始后续的更新操作，例如
- DP rank0拷贝[0:4]个元素
- DP rank1拷贝[4:8]个元素
- DP rank2拷贝[8:12]个元素
- DP rank3拷贝[12:16]个元素
- 执行Optimizer.step(), step()操作必须通过fp32类型来进行计算
- 每个DP rank从main grad buffer中拷贝step()更新后的4个fp32类型元素到fp16类型的grad buffer中
- 执行allgather操作, 这样每个grad buffer就都是最新更新后的数据了
- 基于grad buffer来更新各个模型的fp16类型的参数
- 开始进行下一轮的更新

初始化的入口在文件megatron/training.py的get_model函数中，在创建LocalDDP的实例中会传入args.use_contiguous_buffers_in_local_ddp。

```
from torch.nn.parallel.distributed import  as torchDDP

def get_model(model_provider_func, model_type=ModelType.encoder_or_decoder,
wrap_with_ddp=True):
  ...
  if wrap_with_ddp:
    if args.DDP_impl == 'torch':
      ...
    elif args.DDP_impl == 'local':
      model = [LocalDDP(model_module,
                args.accumulate_allreduce_grads_in_fp32,
                args.use_contiguous_buffers_in_local_ddp)
           for model_module in model]
  ...
```

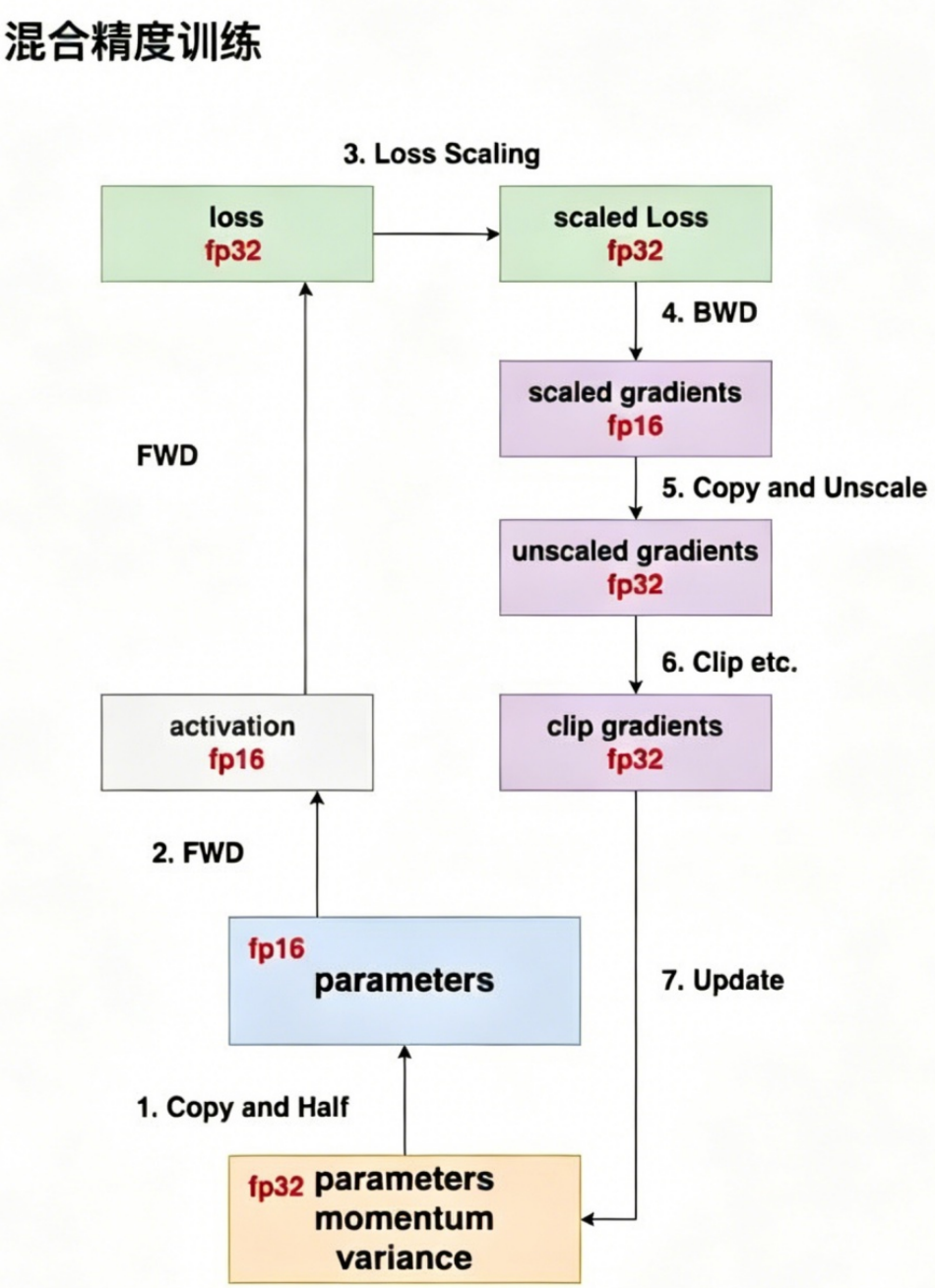
训练的入口定义在train_step函数中, 基本流程如下：

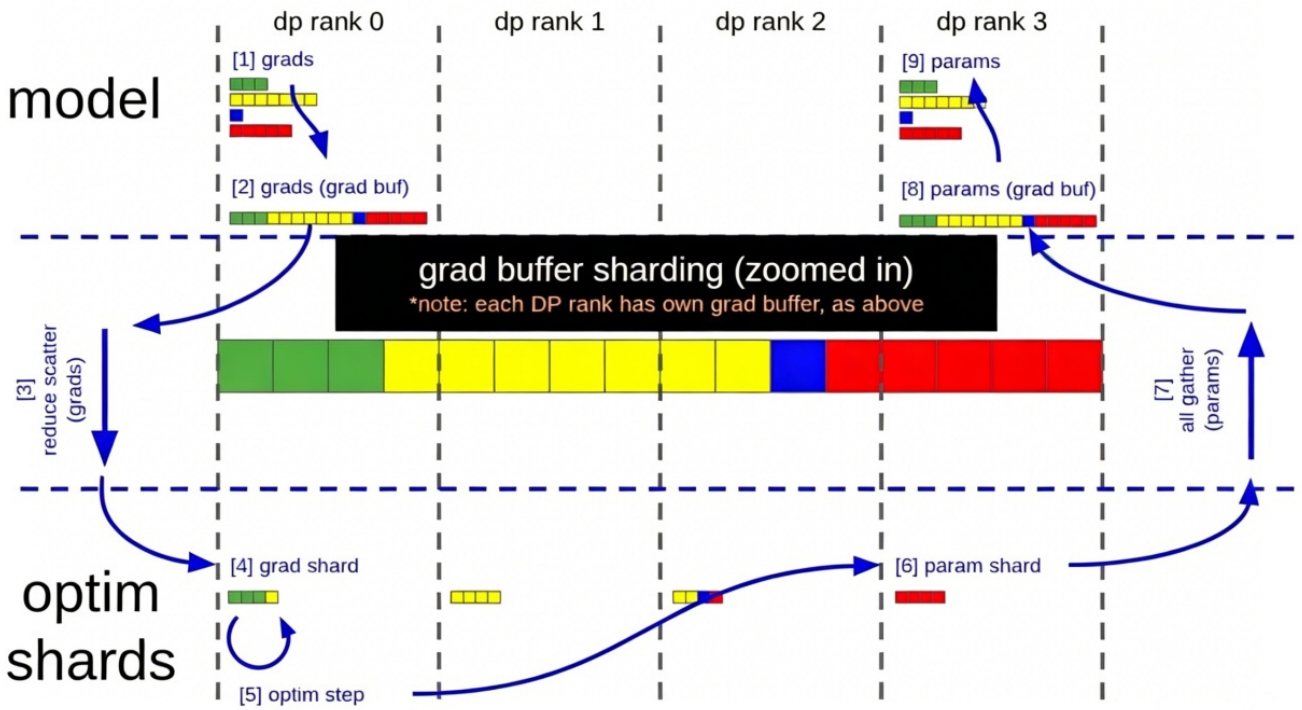
```
def train_step(forward_step_func, data_iterator,
           model, optimizer, opt_param_scheduler):
  ...

  # 清除grad
  if args.DDP_impl == 'local' and args.use_contiguous_buffers_in_local_ddp:
     for partition in model:
        partition.zero_grad_buffer()
  optimizer.zero_grad()
  ...

  # 执行前反向计算
  losses_reduced = forward_backward_func(...)
  ...

  # 对梯度执行Reduce-Scatter操作
  optimizer.reduce_model_grads(args, timers)
  ...

  # 更新梯度
  timers('optimizer', log_level=1).start(barrier=args.barrier_with_L1_time)
  update_successful, grad_norm, num_zeros_in_grad = optimizer.step(args, timers)
  timers('optimizer').stop()
  ...

  # 对更新后的param执行gather操作
  if update_successful:
     optimizer.gather_model_params(args, timers)
  ...

  # 通过scheduler更新学习率
  if update_successful:
     increment = get_num_microbatches() * \
                 args.micro_batch_size * \
                 args.data_parallel_size
     opt_param_scheduler.step(increment=increment)
     skipped_iter = 0
  else:
     skipped_iter = 1
  ...
```

6.5.3.3 混合精度训练

根据Megatron的源码，混合精度训练流程如下图所示：

[图片]


混合精度训练
3. Loss Scaling
loss
fp32
scaled Loss
fp32
4. BWD
scaled gradients
fp16
5. Copy and Unscale
unscaled gradients
fp32
6. Clip etc.
clip gradients
fp32
FWD
activation
fp16
2. FWD
fp16
parameters
7. Update
1. Copy and Half
fp32 parameters
momentum
variance



（1）计算准备：我们存储一份fp32的parameter，momentum和variance。然后，我们将parameter复制一份，再将其精度减半，得到一份fp16的parameter。

- fp32的parameter相当于“主权重”(在Megatron源码中被称为main_param)。在模型训练的过程中，我们执行optimizer.step()更新的就应该是这份权重。当模型训练完毕后，我们保存的也是它。取得高精度的权重是我们的最终目标。
- fp16的parameter相当于“训练权重”（在Megatron源码中被称为model_param），也就是在训练过程中实际参与前向传播过程的权重。混合精度训练的核心思想也体现在这：在整个训练过程里，我们不需要维持单精权重，通过半精权重+一些策略，我们依然也能顺利训练，还能达到节省显存和加速训练的目的。

（2）FWD：使用fp16的parameter做前向计算，在这过程中我们会得到fp16的activation（将在反向传播过程中被使用）。特别注意的是，如果没有采取重计算等操作，activation占据的存储会非常大（可能大过模型本身）。

计算出来的loss我们用fp32精度来表示，这样做是为了保证反向传播计算梯度的精确性。

（3）Loss计算：为了防止梯度溢出（主要是下溢情况），我们对loss做scale处理，得到fp32的scaled loss。我们会在后文详细阐述loss scale的原理。

（4）BWD：利用fp32的scaled loss做反向传播计算梯度。因为loss做过scale了，那自然得到的也是scaled gradients，为了节省显存，scaled gradients以fp16的形式存储。

（5）Unscaled gradients：梯度以fp16的形式存储，但等到需要用梯度去更新模型权重时，就必须转换成fp32的形式了。在转换的同时，我们需要对梯度做unscale操作，让其恢复到原始值。（4）和（5）的步骤可以总结为：在混合精度训练中，我们用fp16的形式存储梯度，等到实际更新模型时，再将其转换成fp32的梯度。

（6）Clip
gradients：在转换为fp32的梯度后，我们还可以执行clip等操作，来进一步预防梯度爆炸/消失。相关的细节我们会在后文详述。

a、混合精度训练下的存储计算

我们先忽略 fp16 的
activation（因为正如前文所说，它是个不稳定因素，由于它总可以通过重新做前向计算得到，因此你可以视情况决定要不要保存它，而保存它的目的是为了让 BWD
过程计算更快），只看其余更必要的部分，则它们占据的显存大小如下（$Φ$
表示模型参数量，表格中的显存大小的单位是 byte），计算方式来自 DeepSpeed ZeRO 原论文：

[图片]

	分类	大小	大小总计
必存	parameter (fp32)	4Φ	12Φ
	momentum(fp32)	4Φ	
	variance (fp32)	4Φ	
中间值	parameter (fp16)	2Φ	4Φ
	gradients (fp16)	2Φ	
总计			16Φ

b、Loss Scale

我们来看一组训练后期激活函数梯度分布的情况：

[图片]

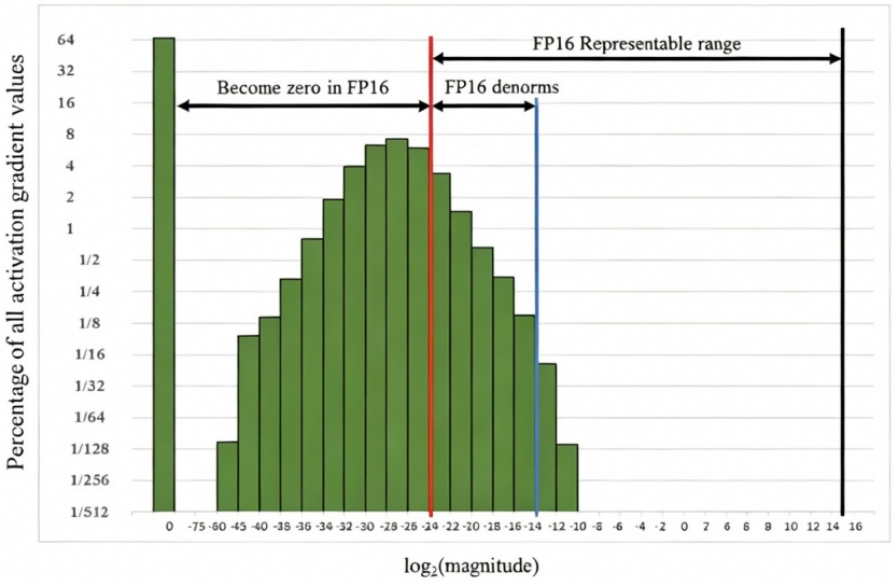


Figure 3: Histogram of activation gradient values during the training of Multibox SSD network. Note that the bins on the x-axis cover varying ranges and there's a separate bin for zeros. For example, 2% of the values are in the $[2^{-34}, 2^{-32})$ range, 2% of values are in the $[2^{-24}, 2^{-23})$ range, and 67% of values are zero.

可以发现，在训练后期，有 67% 的梯度的值会小于2^{-24}（复习一下，这是 fp16 所表示的数值范围的下界，也就是说梯度可能比你想得还要小），也就是说，如果全程用 fp16 做训练，在训练后期就会频繁出现梯度下溢的问题，使得整个训练过程不能正常进行。

而对于梯度下溢问题，常用的办法是 **loss scale。Loss Scale 的核心就在于：如果我把 Loss 放大 N 倍，那这样计算出来的梯度就能放大 N 倍，自然而然就不会发生梯度下溢的问题了。**

基于此，混合精度训练诞生了。另外需要再说明的一点是，混合精度训练并不意味着所有的模型参数都是以fp16的形式参与训练的，例如做LN/BN时的参数一般就保持fp32的形式，Loss也保持fp32的形式。主要原因还是这些数值的精度对模型训练过程影响较大，同时它们占据的存储也不大，因此维持原始形式，越高精越好。

接下来，我们来看两种做Loss Scale的方法：常量损失放大和动量损失放大。

（1）常量损失放大

Loss
Scale的核心是将Loss放大N倍，使得计算出来的梯度也放大N倍，进而解决fp16精度表示下的梯度下溢问题。

如果我们训练全程，我们只采用一个固定的缩放因子loss_scale来缩放loss的话，就被称为“常量损失放大”，其流程如下：

- Scale
up阶段：在backward阶段，将loss值放大2^(loss_scale)倍，用放大后的loss计算梯度，然后正常用fp16存储梯度
- Scale down阶段：在我们需要用梯度进行更新时，
-
先检查放大后的梯度值是否出现上溢(inf/nan)的情况（毕竟放大loss后下溢问题是解决了，但副作

用是可能出现上溢），如果有，则跳过本step权重更新
•
如果不存在梯度上溢情况，则将fp16的梯度unscale，即将梯度值缩小2^(loss_scale)倍，并恢复成fp32的梯度，用于做本step权重更新。
这方法听着不错，但它有个显而易见的问题：这个loss_scale我该设成多少合适呢？太小了，解决不了梯度下溢问题；太大了，又容易发生梯度上溢的情况。有没有一种办法，让模型探索性地去寻找和更新这个loss_scale？

（2）动量损失放大

该方法的总体目标是，动态地找到一个尽可能大的loss_scale，使得其在解决梯度下溢的同时，尽可能避开梯度上溢情况。

（a）首先，先用一个非常大的loss_scale，比如2^24，然后用放大后的loss计算梯度

（b）检查放大后的梯度是否出现上溢（inf/nan）：

• 没有出现上溢，则把梯度unscale成fp32，正常做本step权重更新
• 出现上溢，则跳过本step权重更新，同时将loss_scale缩小F倍（F的默认值为2）
• 在训练后期，梯度的波动幅度逐步稳定，此时可以尝试放大loss
scale，例如每N次iteration就放大F倍（N默认为2000）。如果放大loss
scale后再次出现梯度上溢情况，可以将梯度回退成放大前的结果，以此类推
以上步骤的核心就是，遇到梯度上溢，就缩小loss
scale并跳过相关step更新；连续若干次没有遇到梯度上溢，就尝试增大loss_scale。

所以，如果你在训练初期发现模型偶尔出现inf/nan的情况，其实这是正常的，因为动量策略本身就是一个探索性的策略。

（3）Megatron中的损失放大

Megatron同样提供了常量和动态两种损失放大方法。常量方法我们就不说了，着重来介绍下动态方法。Megatron基本维持了动量损失放大的核心思想，在此基础做了一些变种：

• self._scale：表示初始化loss scale
• self.min_scale：表示loss scale的最小值
• self.growth_interval：表示连续无梯度上溢的迭代次数
• self.growth_factor：当连续self.growth_interval次未出现梯度上溢时，就将loss
scale扩大self.growth_factor倍
• self.hysteresis：表示最多允许出现梯度上溢的迭代次数
•
self.backoff_factor：当累计出现self.hysteresis次梯度上溢的情况时（注意是累计不是连续），则将loss scale缩小self.backoff_factor倍。缩小公式为self._scale = torch.max(self._scale * self.backoff_factor, self.min_scale)。缩小后重新开始计算梯度上溢的次数。
c、Clip gradients

（1）对单个梯度，固定阈值剪裁。比如阈值设定成min=-1，max=1，那么对于某个梯度值，如果小于这个范围，就置为-1，大于这个范围，就置为1。这个做法的难点是阈值难定。

（2）根据全量梯度组成的梯度向量的范数来剪裁。

[图片]

这里介绍梯度裁剪（Gradient Clipping）的方法，对梯度进行裁剪，论文提出对梯度的L2范数进行裁剪，也就是所有参数偏导数的平方和再开方。

让$g_1 = \frac{\partial J(\mathbf{w})}{\partial w_1}$，$g_2 = \frac{\partial J(\mathbf{w})}{\partial w_2}$，设定裁剪阈值为$c$，$\|\mathbf{g}\|_2 = \sqrt{g_1^2 + g_2^2}$。

当$\|\mathbf{g}\|_2$大于c时：

$$\mathbf{g} = \frac{c}{\|\mathbf{g}\|_2} \cdot \mathbf{g}$$

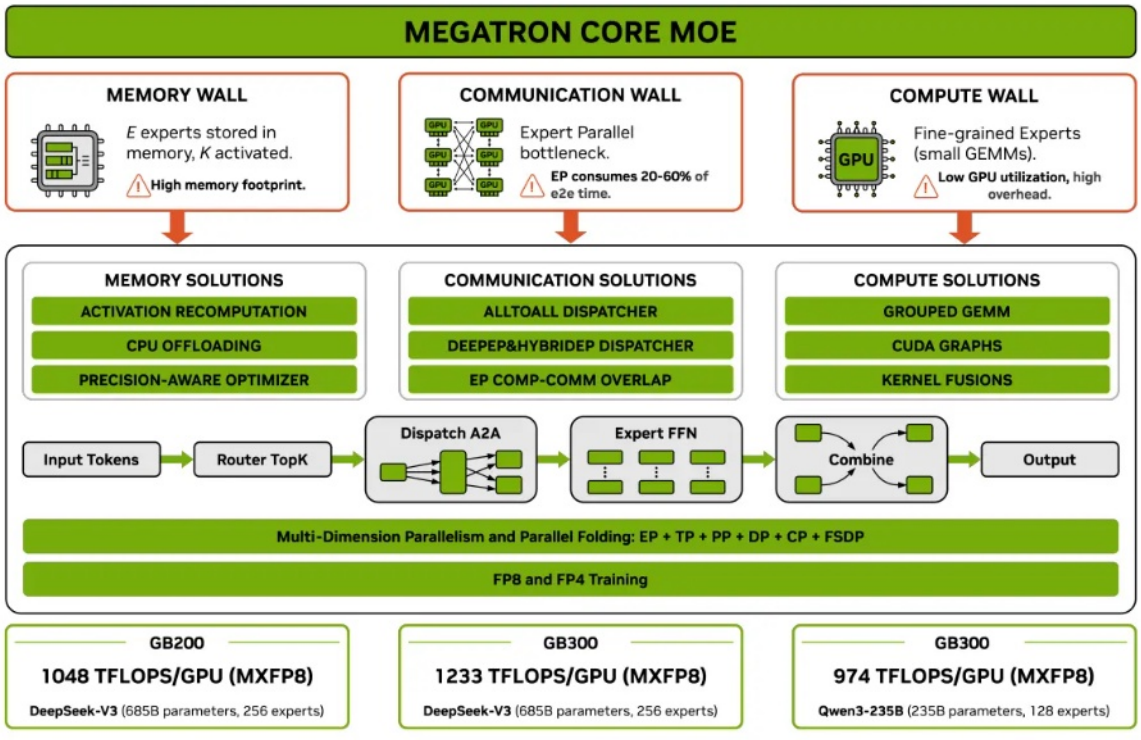
当$\|\mathbf{g}\|_2$小于等于c时：$\mathbf{g}$不变。

其中，$\frac{c}{\|\mathbf{g}\|_2}$是一个标量，大家有没有觉得这个跟学习率$\alpha$很类似?

6.5.3.4 [MOE优化](https://r0ddbu55vzx.feishu.cn/file/WFEWbtcQTokktxxfdfgcj0lMn9e)

MoE模型凭借其高参数容量与低激活计算量的优势，成为构建超大规模语言模型的关键路径。然而，其稀疏激活机制引入了独特的“参数-计算失配”问题，导致在扩展至万亿参数级别时面临三大系统级瓶颈——即Memory Wall、Communication Wall和Compute Efficiency Wall。这三者相互耦合，单一维度的优化往往引发其他维度的恶化，因此必须通过系统级协同设计实现端到端的可扩展训练。

[图片]



为应对上述挑战，NVIDIA
Megatron-Core框架提出了一套完整的解决方案体系，从并行架构、内存管理、通信协议到计算内核进行全面重构。

MoE训练核心瓶颈（源于模型稀疏性）：

- 参数-计算不匹配：总参数远超激活参数（如DeepSeek-V3 685B vs 37B，差18倍）。存算失衡使通信主导，引发三重瓶颈：

1. 内存墙：全量状态存储 + Top-K Token复制导致显存剧增。
2. 通信墙：EP All-to-All通信量庞大，跨节点时耗时占比可达60%。
3. 计算墙：细粒度小GEMM、路由重排、负载不均与频繁Kernel启动严重拉低GPU利用率。

- 密集-稀疏不匹配：Attention（密集）层偏好高TP，MoE（稀疏）层偏好EP且高TP会破坏效率。传统框架强制全局统一并行度，导致配置僵化（EP≤DP）、最低GPU门槛高且易触发跨节点瓶颈。
通过 Parallel Folding
解耦Attention与MoE层的并行配置，结合多级内存优化、HybridEP通信协议、Grouped GEMM与CUDA
Graphs，从架构、通信到计算内核全面重构，系统性突破上述瓶颈，为万亿级MoE模型高效训练提供底层支撑。

a、并行架构创新：Parallel Folding

在大规模MoE模型训练中，传统并行瓶颈（密集/稀疏需求冲突 + EP⊆DP强约束）：

- Attention层：偏好高TP/CP以优化大矩阵与长序列计算。
- MoE层：依赖高EP以分散海量小型专家。
旧框架强制EP组必须嵌套于DP组内，引发三大问题：

1. 资源膨胀：扩EP必同步扩DP，最小GPU门槛激增（如CP=8, EP=8需≥64卡）。
2. 通信低效：CP/EP易跨节点部署，错失节点内NVLink高带宽优势。
3. 配置僵化：无法独立调优两类层的并行粒度，系统灵活性受限。
为破解这一困局，Megatron-Core提出Parallel Folding架构，其核心思想是解耦Attention层与MoE层的并行语义映射，允许两者采用独立且最优的并行组合。

[图片]

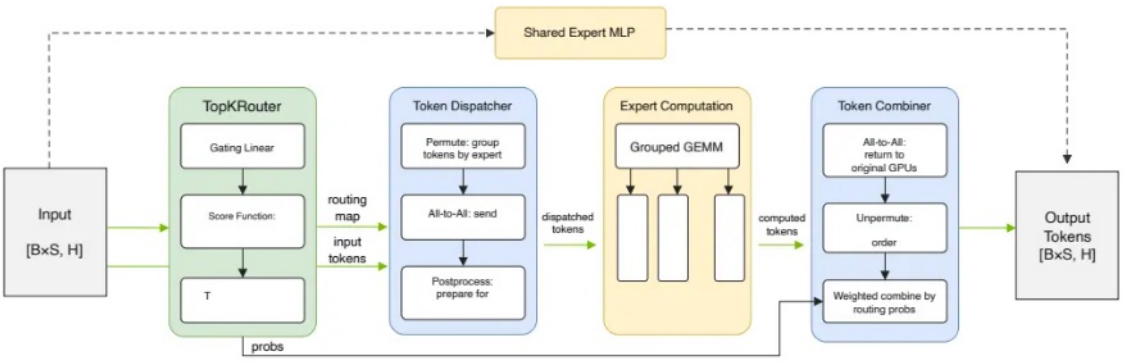
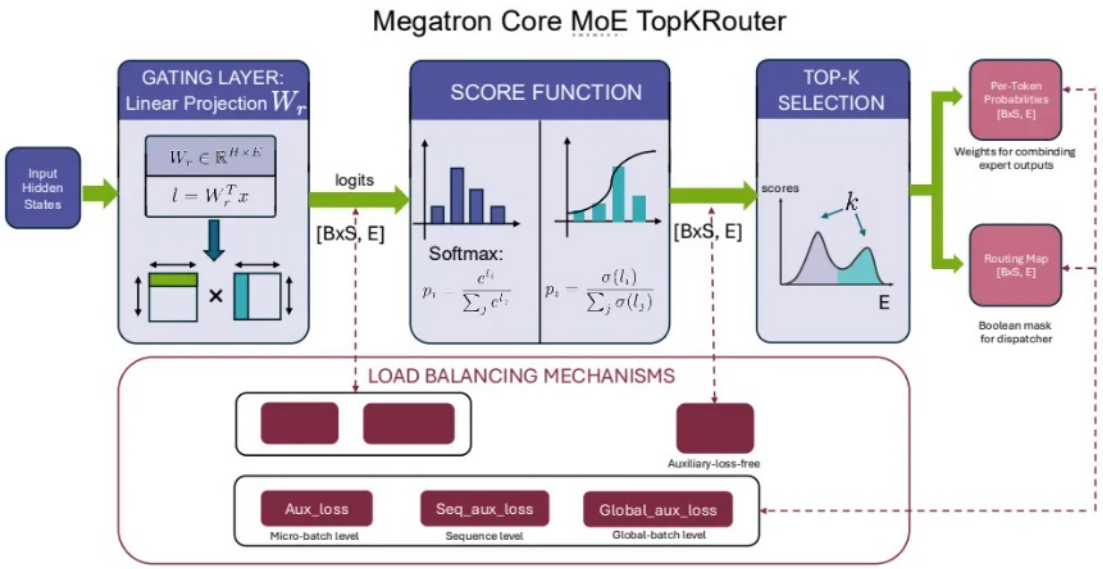


Figure 1: Data flow through an MoE layer: Route, Dispatch, Compute, and Combine stages.

[图片]



核心机制设计

该方案通过构建两套独立的并行视图实现灵活资源配置：

[图片]

	层类型	支持的并行维度	优化目标
2	Attention层	TP × CP × DP × PP	大矩阵分片、长序列切块、
3	MoE层	ETP × EP × EDP × PP	专家分布均衡、All-to-All通

点击图片可查看完整电子表格

其中，“Folding”体现在：MoE层的EP不再受限于全局DP，而是通过折叠TP×CP形成的子组来实现更大规模的专家分布。例如，在TP=4、CP=2的配置下，可自然形成8个NVLink域内子组，从而支持EP=8甚至更高，而无需扩展DP。

b、多层次内存优化体系

为突破MoE模型训练中的Memory
Wall，Megatron-Core构建了一套可组合、多层次的内存优化技术体系。该体系覆盖激活（activation）、优化器状态（optimizer
states）与参数存储等多个层面，通过精细化控制内存生命周期，在保证训练稳定性的前提下显著降低每GPU显存占用。以下为七类核心技术的综合对比：

[图片]

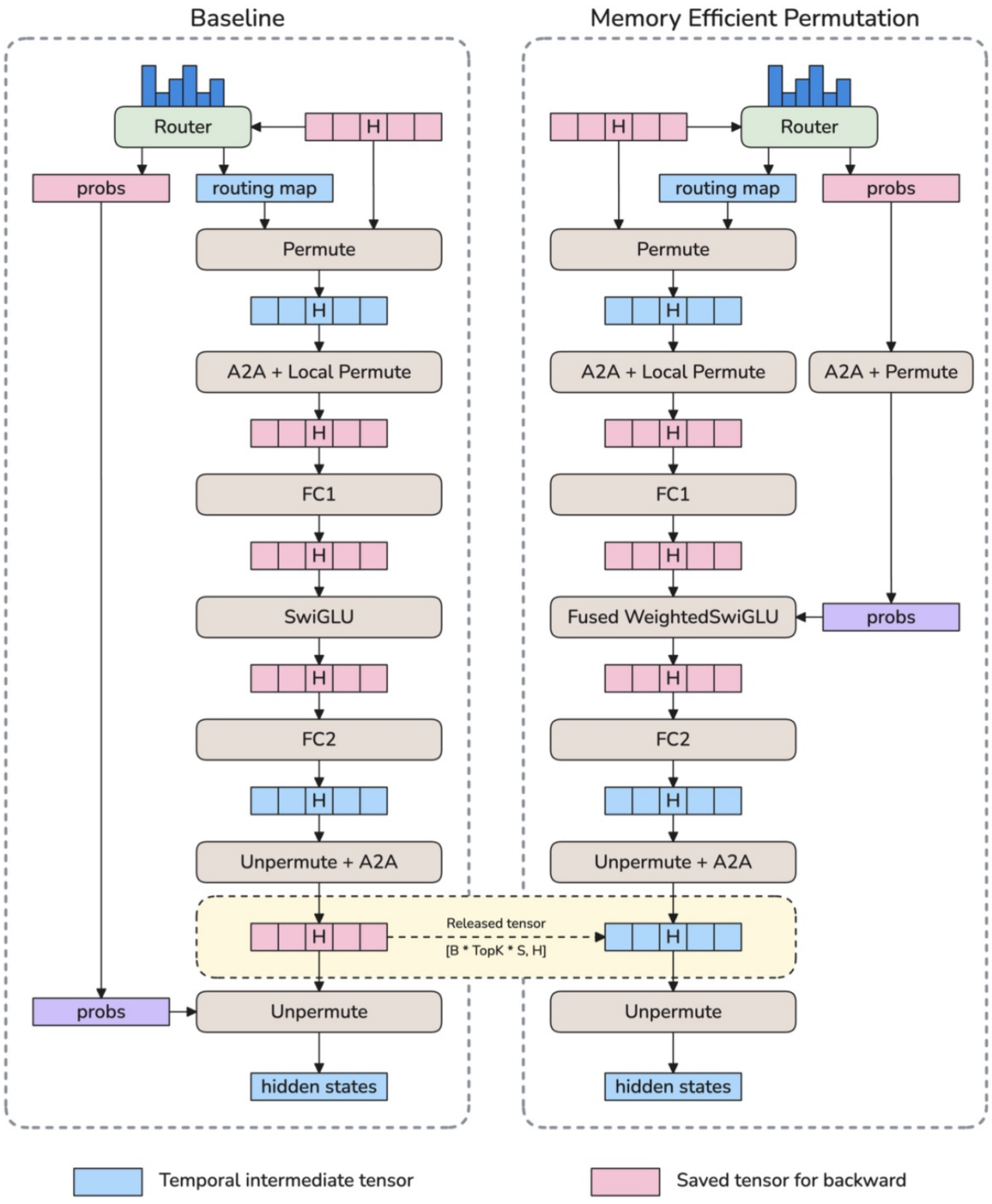
	技术	目标组件	节省量（DeepSeek-V3示例）	
2	Memory-Efficient Permutation	激活	~26.3 GB/GPU	零
3	FP8/FP4训练	激活输入	~16 GB/GPU（FP8）	数值稳
4	Fine-grained Recomputation	激活	总计42.4 GB/GPU	计算开
5	Fine-grained Activation Offloading	激活	10–18%内存降幅	PCIe带
6	Precision-aware Optimizer	优化器状态	~10–12 GB/GPU	存储精

点击图片可查看完整电子表格

- Memory-Efficient Permutation

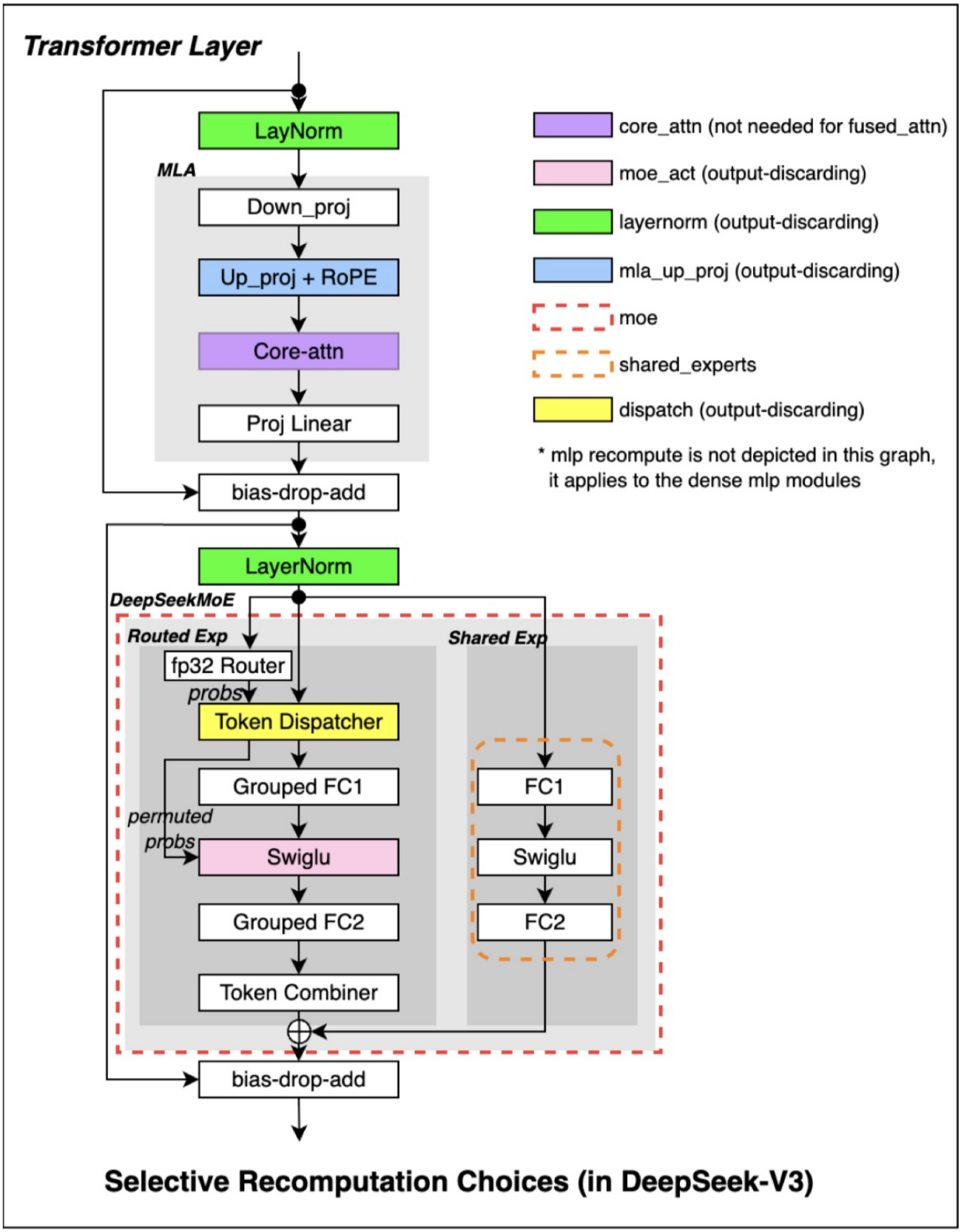
通过改变路由权重的乘法位置（将权重乘在激活函数之后），消除了为计算路由梯度而保存的中间张量，实现零开销的内存节省。对 DeepSeek-V3 可节省约 26.3 GB。

[图片]



・细粒度重计算（Fine-grained Recomputation）
只选择内存占用大但计算代价小的模块（如 MLA 的上投影、SwiGLU
激活、LayerNorm）进行重计算，而非整个层。结合“输出丢弃”策略，进一步减少内存。对 DeepSeek-V3 可节省约 42.4 GB。

[图片]



• 细粒度激活卸载（Fine-grained Activation Offloading）
将计算代价大的模块（如注意力、专家）的输入激活，在计算完后立即异步卸载到 CPU 内存，需要时再预取回来。通过专用 CUDA 流与计算重叠，隐藏传输延迟。模块级粒度允许混合策略：轻量模块用重计算，重量模块用卸载。在 PP/VPP 场景下，通过 ChunkOffloadHandler

管理多个微批次和虚拟阶段的卸载/重载。相比完全重计算，卸载可将峰值内存从 L 个 layer_input + 1 个中间激活降低到 1 个layer_input + 1 个中间激活，与层深度无关。对于 DeepSeek-V3，卸载可减少 10-18% 内存，仅带来 1.6-2% 吞吐量开销。

- 权重和优化器优化
- 精度感知优化器（Precision-aware Optimizer）

Adam 的动量项（一阶矩和二阶矩）可存储在 BF16 或 FP8 中，计算更新时再动态转换为 FP32，减少优化器状态内存约 50%。可与分布式优化器结合，进一步降低每 DP 秩的内存。

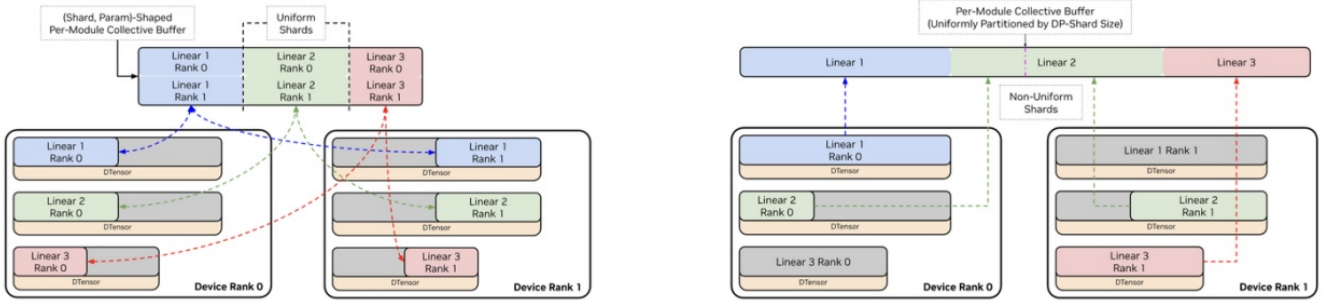
- 优化器状态卸载（State Offloading）

在 forward/backward 期间将优化器状态和主权重卸载到 CPU，需要更新时再加载回来，可节省 15-20 GB 内存，但引入少量传输开销。两者结合效果更好

- FSDP + EP (Dual DeviceMesh)

结合 FSDP 和 EP，通过双重 DeviceMesh 设计，对稠密层和专家层应用不同的分片策略。使用非均匀分片（将模块内参数扁平化后分片，减少通信开销）、持久双缓冲和NCCL 用户缓冲区注册，实现零拷贝通信，降低 SM 占用，并支持通信-计算重叠。

[图片]



(a) FSDP2 per-parameter uniform sharding.

(b) Megatron-FSDP per-module non-uniform sharding.

[图片]

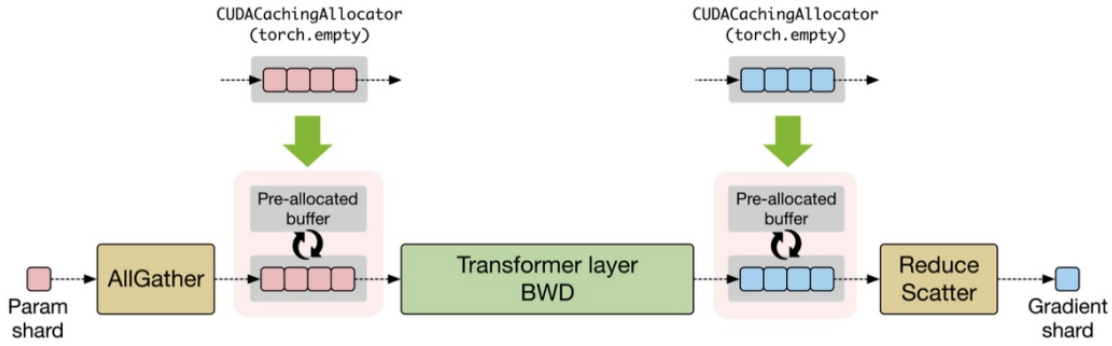


Figure 12: Persistent double-buffer design: two pre-allocated buffers are cycled across FSDP collectives, eliminating allocation overhead and enabling NCCL User Buffer Registration.

c、高效通信机制

EP 引入了大量 all-to-all 通信，在跨节点时可能占训练时间的 40-60%。

[图片]

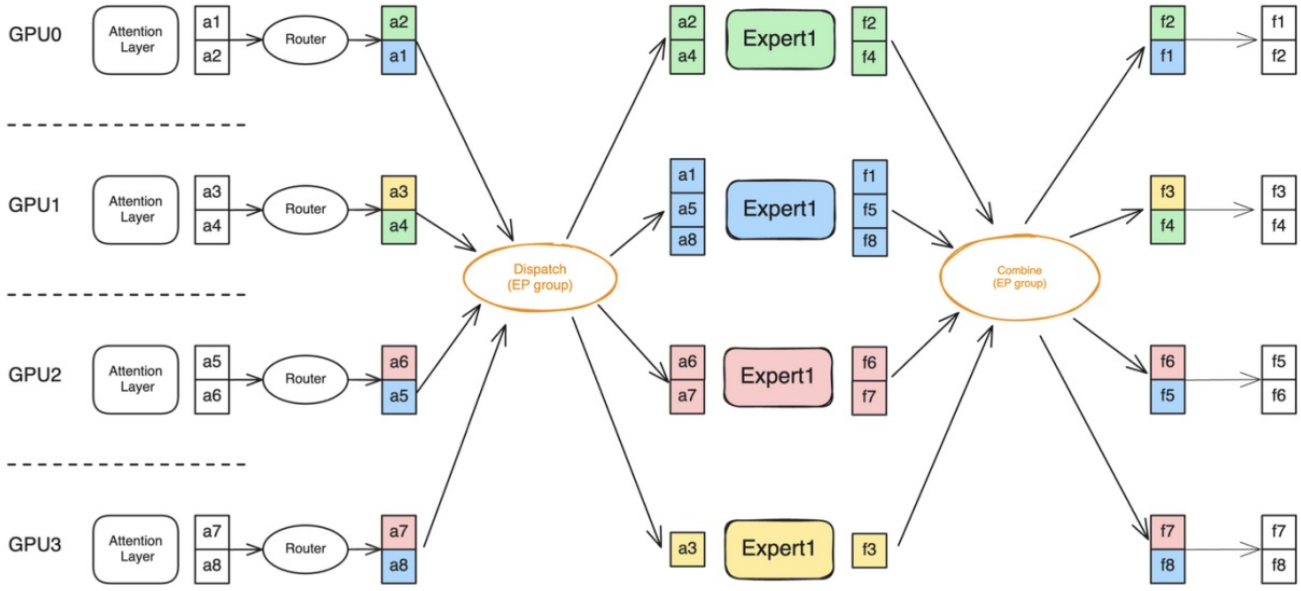


Figure 13: Expert parallelism across 4 GPUs with 4 experts.

- HybridEP
  优化的 token 调度器，取代标准 NCCL all-to-all。HybridEP 利用 TMA、RDMA 等技术，实现 token 级通信，通过减少 permutation 操作、避免 token 的重复通信、融合归约操作，显著提高带宽利用率，并减少 SM 占用。在跨节点场景下，HybridEP 先通过 RDMA 在相同本地索引的 GPU 间交换数据，再在节点内转发，减少跨节点流量。对于 combine，HybridEP 将归约融合进通信内核。

HybridEP通过三项核心技术重构了传统All-to-All的执行流程，显著降低通信延迟与带宽消耗：

Device-side Permutation Pipeline

将原Host端的token重排序逻辑迁移至GPU内核，利用FIFO队列与RDMA warp group直接交换目标子集，消除冗余数据复制。

Reduction-Fused Communication Kernel

在Combine阶段将梯度规约（reduction）操作融合进通信内核，避免额外启动unpermute kernel，减少SM占用。

Node-level First Aggregation

跨节点部署时优先在本地节点完成部分规约，再进行跨节点通信，有效缓解低带宽链路压力。

下表对比了HybridEP与标准All-to-All的关键特性差异：

[图片]

	特性	Standard All-to-All	HybridEP
2	通信前处理	Host-side permute，复制k倍token	Device-side FIFO + RDMA
3	冗余流量	是（发送重复token）	否（直接交换目标
4	Combine阶段	单独unpermute kernel	Reduction fused into con kernel
5	SM占用	较高	更低
6	跨节点优化	无	支持node-level reduc

点击图片可查看完整电子表格

- EP 通信重叠
将 all-to-all 与计算重叠。采用合并的前向-后向调度（例如将微批次 1 的前向与微批次 2 的后向合并）和权重/数据梯度拆分技术，打破依赖关系，使通信能与计算并行。结合交错流水线并行（Interleaved PP），可最大化重叠机会。通过流分离和 W/D 拆分，将 EP 通信开销从 30-40% 降至 5% 以下。在 DeepSeek-V3 训练中，实现了 93% 的通信延迟重叠。

关键优化手段包括：

- Stream Separation：划分独立的Compute Stream与Comm Stream，并行执行计算与通信任务
- W/D Split in
Backward：将MLP反向拆分为权重梯度（W/mlp）与数据梯度（D/mlp），前者可提前执行以延长重叠窗口
- Interleaved PP + Flexible
VPP：结合虚拟流水线技术，提升微批次调度灵活性，进一步扩大重叠机会
该方案将EP相关通信的时间占比从典型的30–40%降至%，实测通信-计算重叠率高达93%，从根本上缓解了通信对整体吞吐的制约。

[图片]

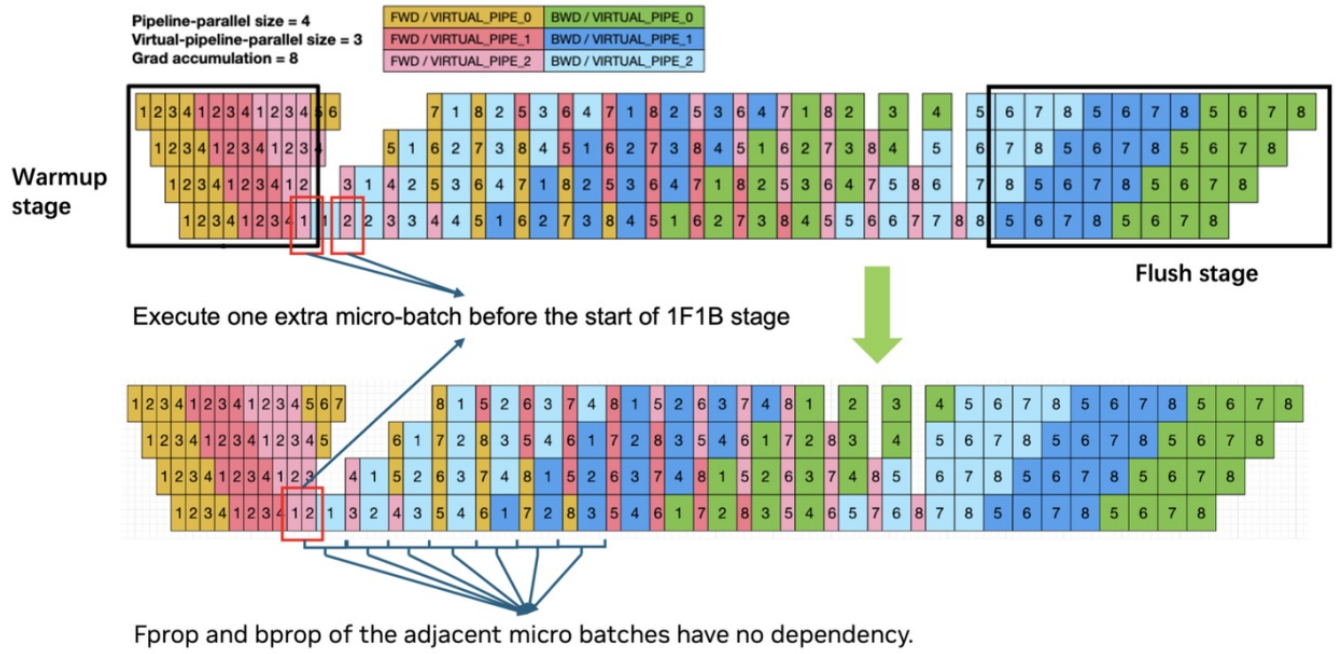


Figure 19: Interleaved PP Timeline with all-to-all Overlapping.

d、计算效率提升路径

为突破MoE模型中因细粒度专家导致的Compute Efficiency Wall，Megatron-Core从Kernel执行效率与Host调度开销两个层面实施系统性优化，彻底消除“CPU Bound”现象，最大化GPU算力利用率。

通过重构GEMM执行模式与融合关键计算流程，显著提升SM占用率与吞吐性能：

- Grouped GEMM 与内核融合：
- Grouped GEMM：将多个专家的小 GEMM 批量执行。提供多种实现：多流并发 cuBLASLt、CUTLASS Grouped GEMM、cuBLASLt Grouped GEMM（支持设备端形状信息，可融入 CUDA Graph）、cuteDSL Grouped GEMM（融合激活和量化）。
- 排列融合：将排列预处理、排列、反排列操作融合，减少内核数量。

[图片]

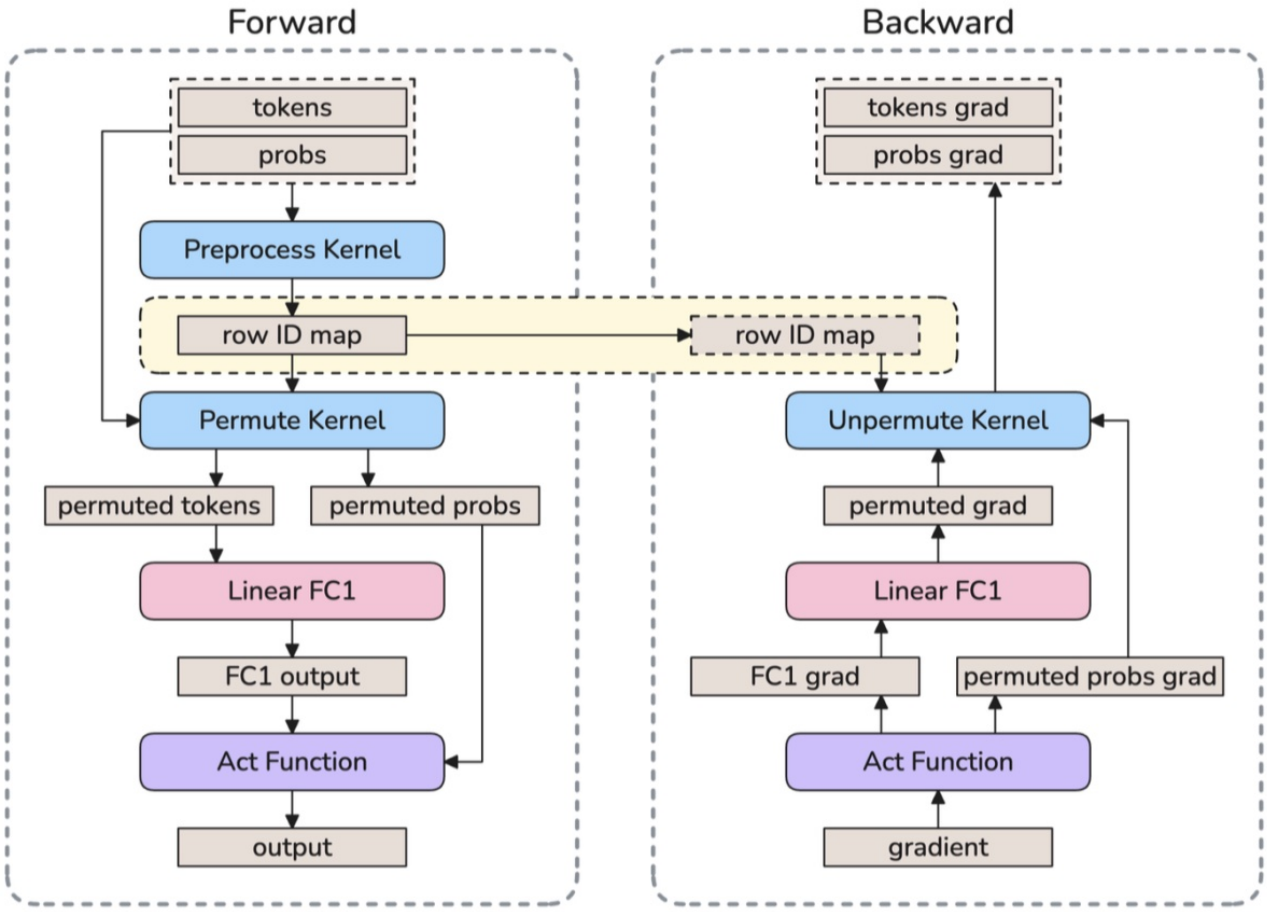


Figure 20: The pipeline for permute fusion in the training process.

•

路由器和辅助损失融合：将路由计算（softmax/sigmoid、top-k）和辅助损失计算融合为单个内核。

[图片]

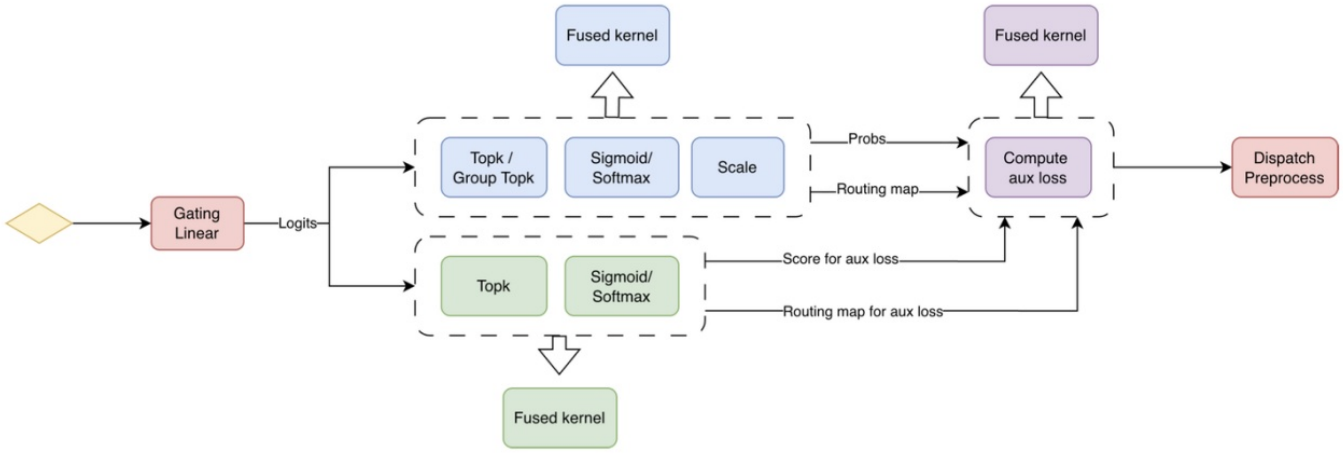


Figure 21: The workflow of the router fusion.

• CUDA Graph：消除主机开销。对于纯 dense 模型或者 token dropping with padding场景的 MoE（静态形状），可用 Full CUDA Graph。对于无 token 丢弃场景的 MoE（动态形状），采用Partial CUDA Graph（Layer-wise CUDA

Graph），仅捕获静态部分（注意力、路由器预处理），动态部分（专家计算）留在图外，仍能大幅减少 CPU 开销。在 DeepSeek-V3 GB200 训练中，Partial CUDA Graph 带来 10% 的端到端加速，额外内存约 7 GB。

[图片]

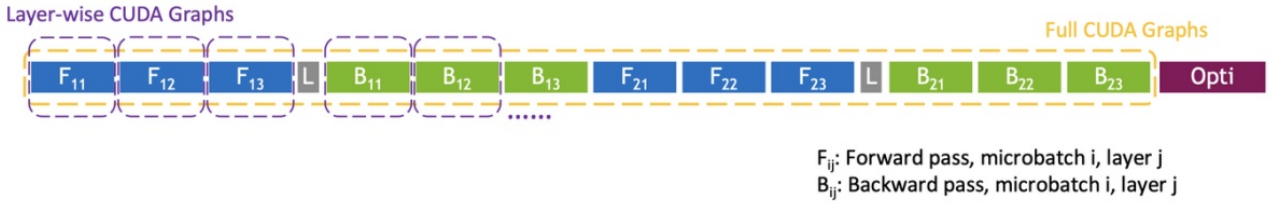


Figure 23: Full versus layer-wise CUDA Graphs in one training iteration (three layers, two microbatches).

[图片]

Figure 25: Transformer layer forward pass: without (upper) and with (lower) partial CUDA Graphs. CPU overhead is largely eliminated for static components.

・针对无 token 丢弃 MoE 的Full CUDA Graph覆盖：通过三种技术使 CUDA Graph也能覆盖动态shape部分：

・设备启动内核：针对 kernel 启动前因问题规模未知导致不可避免的主机同步问题，可以通过重新设计 kernel，让 Grouped GEMM和HybridEP支持动态输入形状，并从设备内存读取形状信息，无需主机同步。

・

ECHO（弹性克隆热门专家）：针对如何申请不定大小内存的问题，可以动态地将热点专家复制到负载低的 GPU 上，平衡负载，减少内存碎片化，减轻 straggler 问题。ECHO 规划器通过 bin-packing 算法最小化克隆数量。

[图片]

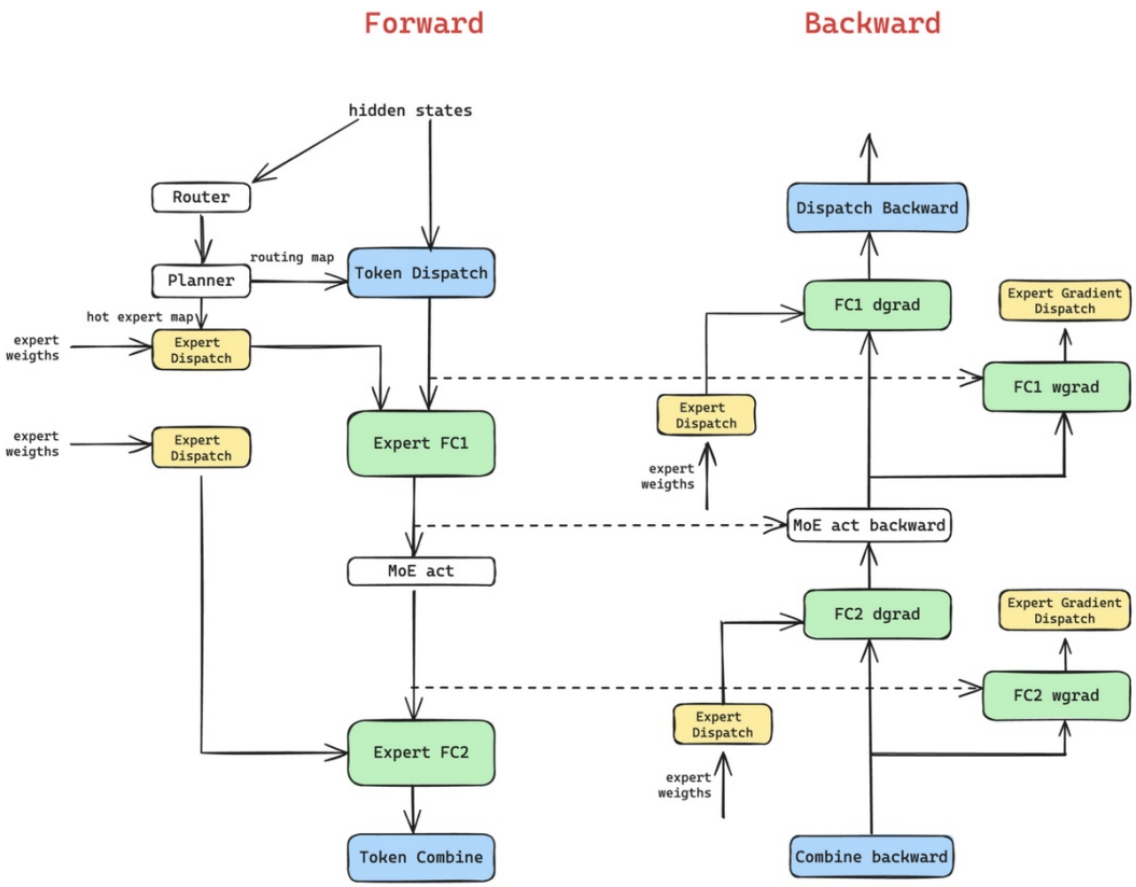


Figure 27: ECHO workflow for forward and backward passes. The planner generates routing and hot expert maps. Expert Dispatch clones hot expert weights to spare slots; Expert Gradient Dispatch reduces gradients back to home experts.

- 分页暂存：使用一个共享的临时缓冲区进行计算，并将实际需要的激活以分页形式存放到一个独立的暂存缓冲区中，将内存占用从 O(层数×最坏情况) 降至 O(最坏情况+实际总量)。通过流重叠隐藏分页操作延迟。

[图片]

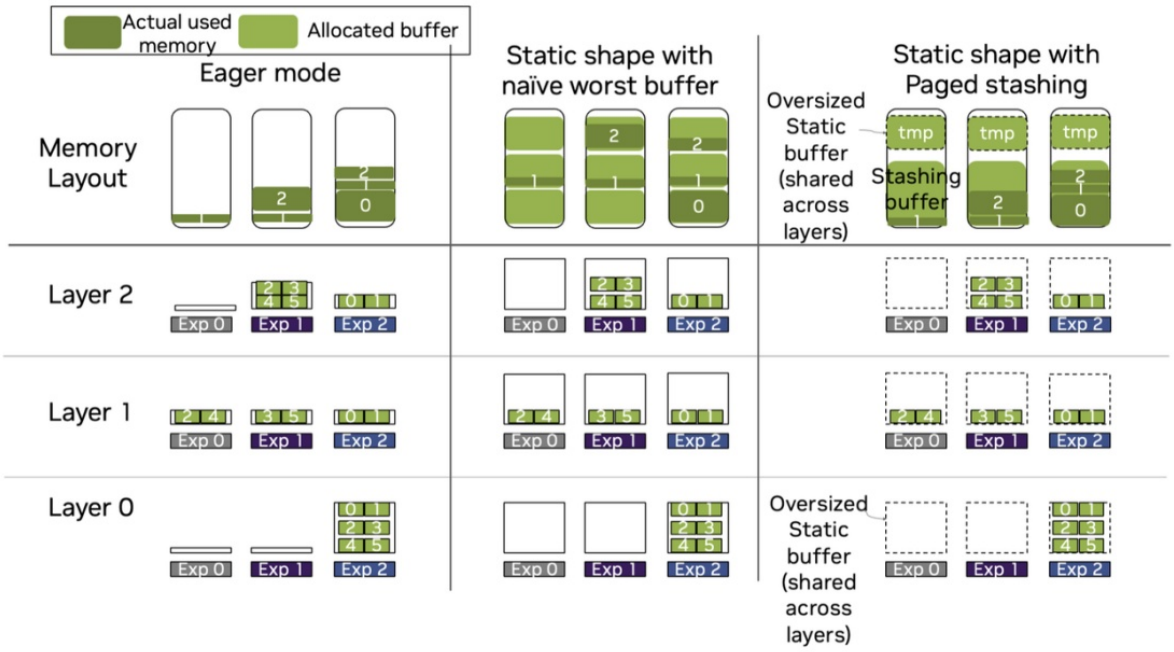


Figure 28: Memory layout comparison across three execution modes. **Left**: Eager mode allocates memory dynamically based on actual usage. **Middle**: Baseline static shape requires worst-case sized buffers for each layer independently, causing severe fragmentation when actual usage is lower. **Right**: Paged Stashing uses a single worst-case tmp buffer shared across layers for computation, while a paged stashing buffer stores only the actual tokens, significantly reducing total memory footprint.

e、低精度训练支持与量化融合

低精度训练同时影响三大瓶颈：减少内存、加速通信（参数 AllGather
减半）、提升计算。但引入量化内核开销，可能影响稳定性。策略是选择性精度：保护数值敏感部分（路由器、嵌入层、输出层、梯度、优化器状态）保持高精度，对大部分计算（专家
GEMM）进行低精度量化。

- 方案：
- Per-Tensor FP8：每个张量一个缩放因子，简单适合入门。
- Blockwise FP8（Hopper 推荐）：以 128×128 的块为单位缩放，精度更高，已在 DeepSeek-V3 等大规模训练中验证。
- MXFP8（Blackwell 推荐）：以 1×32 的细粒度缩放，Blackwell
张量核心原生支持，性能最佳。
- NVFP4（Blackwell）：4
位微缩放格式，结合随机哈达玛变换、二维缩放和随机舍入以保证训练稳定。

[图片]

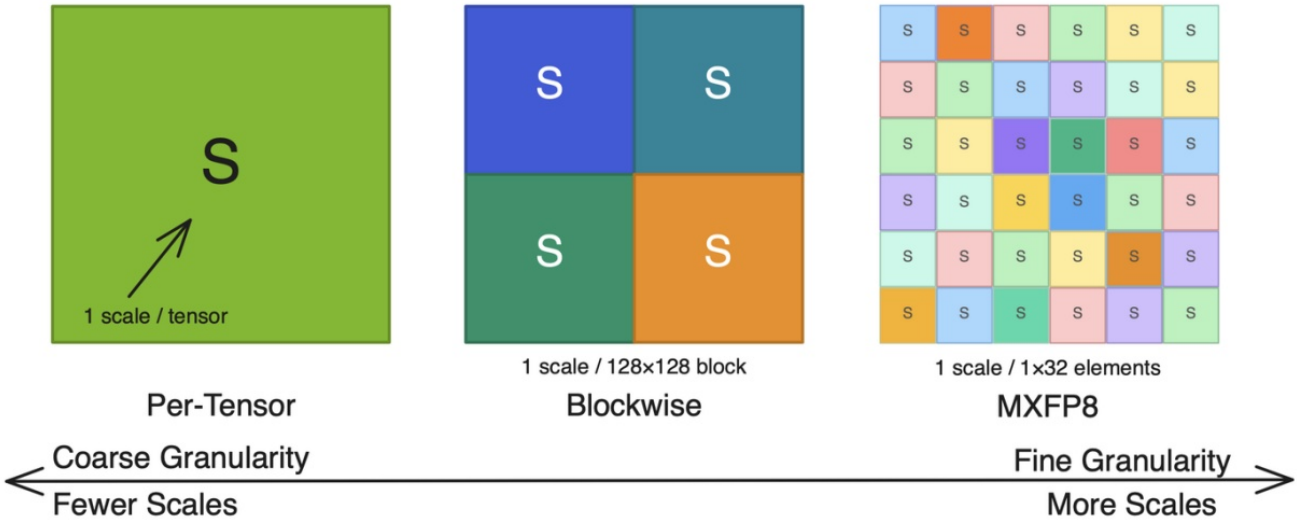
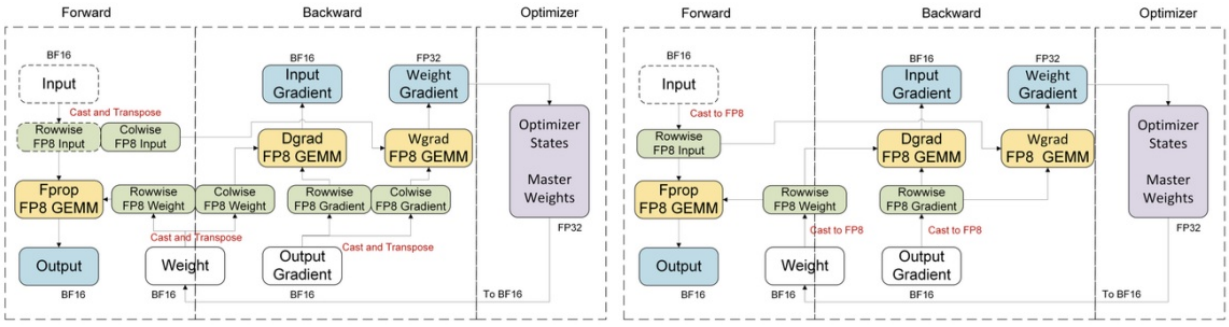


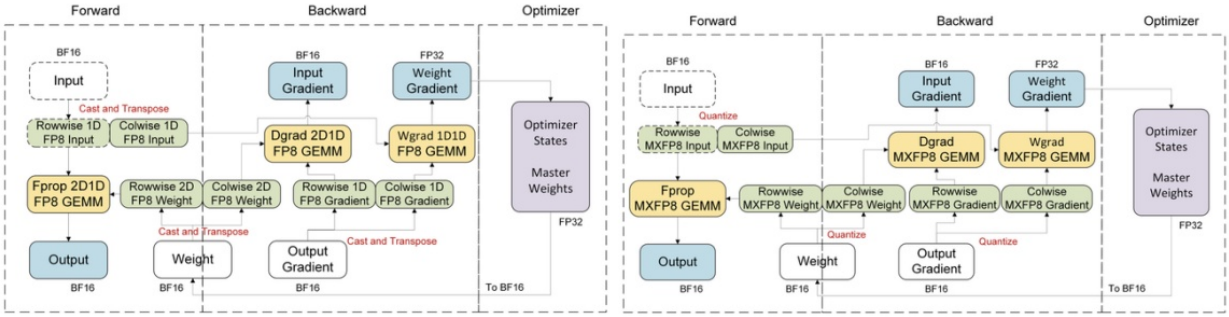
Figure 30: FP8 training recipes: Per-Tensor Scaling, Blockwise FP8, and MXFP8.

[图片]



(a) Per-tensor Current scaling on the Hopper platform.

(b) Per-tensor Current scaling on the Blackwell platform.



(c) Blockwise FP8 recipe on Hopper.

(d) MXFP8 recipe on the Blackwell platform.

Figure 31: The computation of a linear layer with various FP8 recipes. Note the differences in quantization granularity and tensor layout requirements across platforms.

- MoE 特定挑战与解决：
-

动态形状对齐：为满足量化粒度要求，需要填充。通过将填充融合到排列内核中，或填充路由映射而非 token 本身，来减少开销。

- 分组量化：将多个输入张量的量化融合为一个内核，减少内核启动开销，支持 CUDA Graph。
- NVFP4

量化融合：将随机哈达玛变换、转置、量化和随机舍入等复杂操作融合到一个内核，避免中间内存读写。对于 MoE，需实现分组 NVFP4 量化，确保 128 的 token

对齐，并支持每个专家的独立缩放因子。

f、上下文MOE训练

长序列训练核心挑战：注意力层复杂度O(s^2)远超MoE层O(s)，SDPA已高度优化，瓶颈集中于显存与通信。

显存管理：

- CP/TP扩维：提升并行度维持单卡子序列长度恒定（建议4K/8K）。
- 优化器卸载：状态移至CPU释放显存（256 H100/16K seq下开销仅~2%）。
- 选择性重计算：避开SDPA重算（计算占比高，重算致16%性能损耗仅省9GB）；优先重算非注意力模块（省89.8GB且性能影响极小）。

CP策略与部署：

- P2P CP：通算天然重叠，适配跨节点。
- All-to-All CP：序列分片转头分片，提升SDPA内核效率，但增加通信。
- 实践：节点内采用All-to-All CP + TP提效降参存；跨节点采用P2P CP保重叠。Megatron-Core支持二者融合的分层CP。

变长序列与动态并行：

- 无Padding打包：采用THD格式(总Token×头×维)拼接变长序列，依累积长度追踪边界，消除填充浪费（如混合序列可省~60%算力），原生支持RL与多模态。
- Dynamic-CP：按微批次实际长度动态切换CP度（短序列低CP提效，长序列高CP防OOM）。预建CP组实现轻量切换，结合打包策略均衡DP/CP负载，有效压缩同步气泡。

[图片]

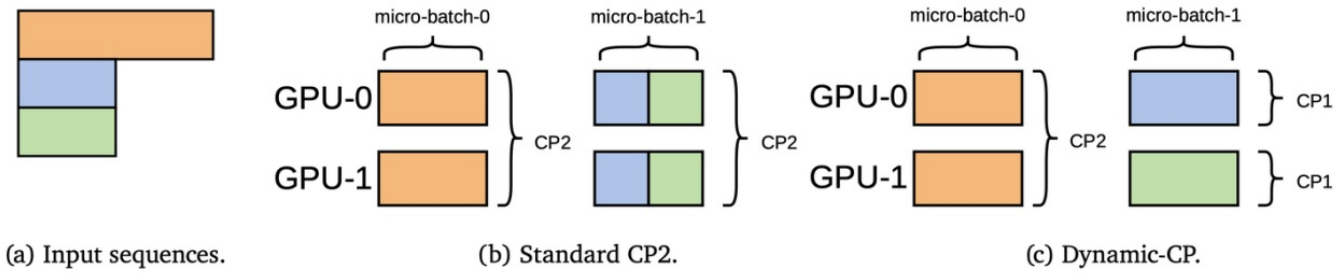


Figure 38: Dynamic Context Parallelism for Packed Sequences.

这些优化共同构建了一个高效、紧凑的低精度执行流，使万亿参数MoE模型能够在保持训练稳定性的前提下，充分释放现代AI硬件的算力潜能。

h、Deep EP

跨机EP场景下需要完成 MoE
模块的专家计算。在MoE场景下，每个输入token的特征向量会交给一个或多个专家（Expert），每个专家通常是一个独立的神经网络（例如
FFN），不同专家可以处理不同的模式或领域的数据，由此可以提升模型应对各类任务的表现。

在EP场景下，专家分布在不同的 rank 间，因此在 MoE 计算前需要进行跨 rank 间的 token 通信。

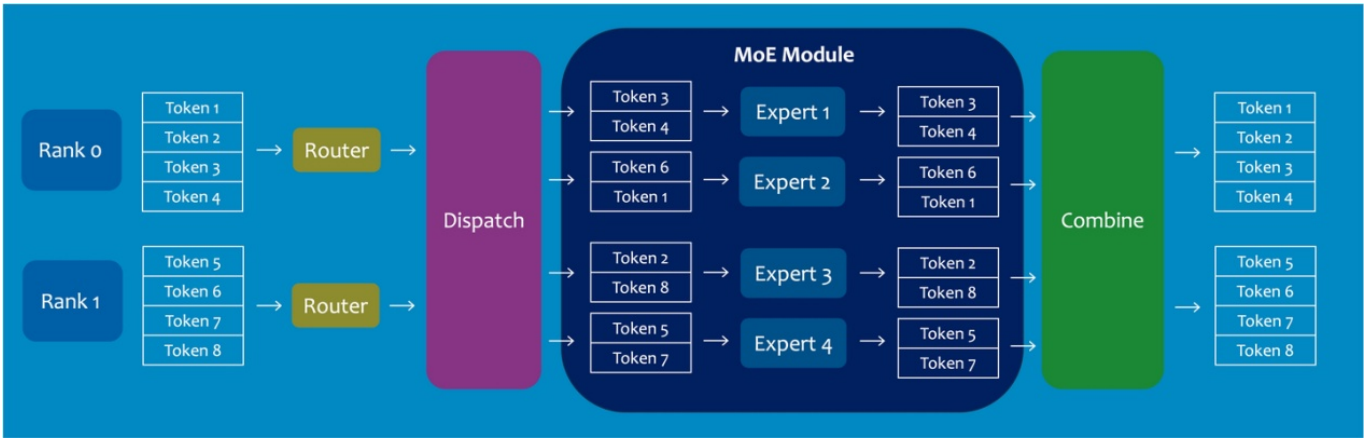
- Dispatch（前向分发）：因专家跨设备分布，各Rank经Router计算目标路由后，需通过All-to-All将

Token精准投递至对应专家所在节点。

- Combine（反向聚合）：专家计算完成后，执行逆序All-to-All将输出结果回传至Token原始Rank，完成数据对齐。
- 本质：通过双向All-to-All实现“Token按路由找专家”与“结果原路返回”，是跨机MoE协同计算的通信基石。

下图展示了 EP = 2 时，Rank 0 和 Rank 1 之间的 Dispatch 和 Combine 通信流程。

[图片]



MoE 场景的 All-to-All 通信与传统集合通信的 All-to-All 具有明显的区别。

[图片]

	集合通信 A2A	MoE 场景 A
核心算法	数据转置：每个 rank 将数据分发给所有其他 rank，主要用于矩阵转置或特定的并行排序算法。	负载均衡与路由：将 token 果，发送到对应的专家
通信的对称性	对称，每个 rank 发送和接收的数据量均相等。	非对称，每个 rank 发送和接相等，不同 rank 间发送和接不相等。
数据规模与特征	每个 rank 发送给其他 rank 的数据量通常是固定的、均匀的。	通信时需要动态计算每个 ra 据量。
先验假设	假定每一份数据仅传递给 1 个 rank。	无强假设，因此一份数据可 个 rank 上。

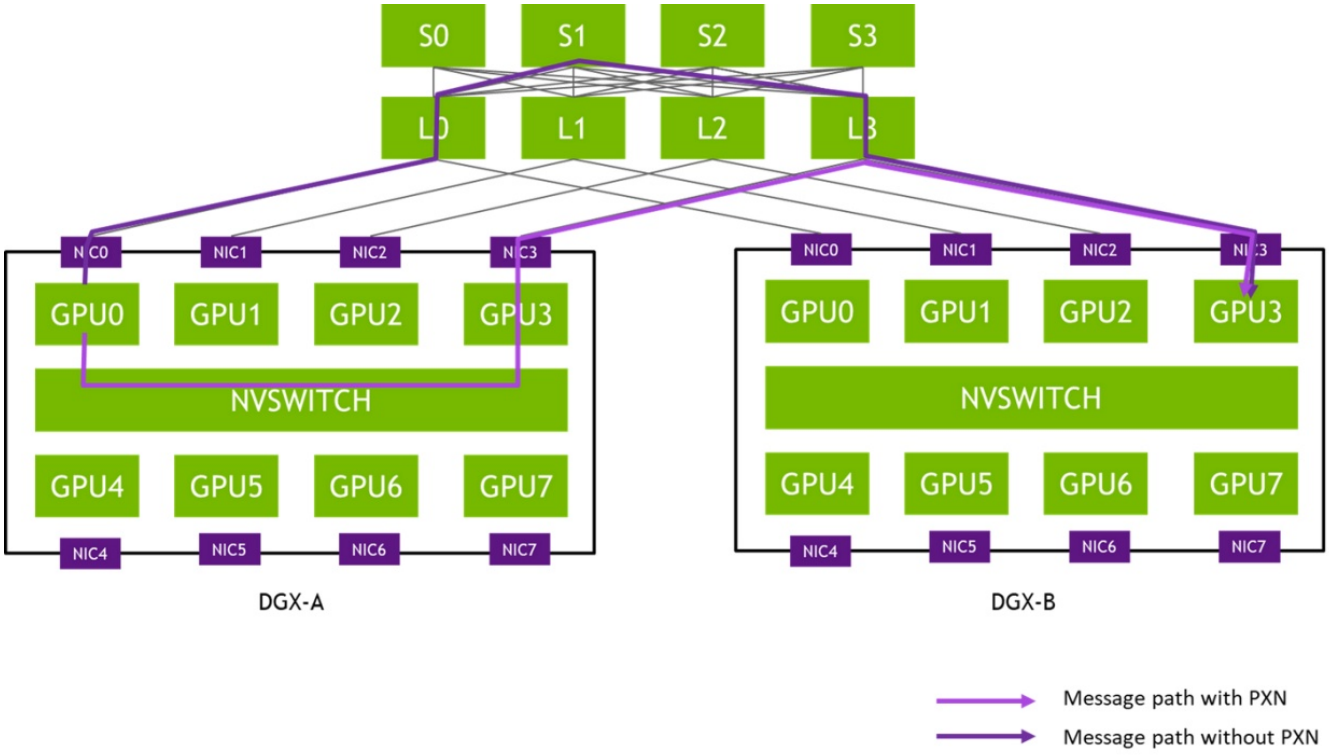
点击图片可查看完整电子表格

传统A2A的场景下，一个节点的rank要向另一个节点的若干rank发送数据时，每一次点对点的数据传输均走直连的 RDMA 链路，因此非同号
rank下的跨机通信需要经过二级交换机。在通信数据规模较大的场景，二级交换机容易成为整个系统的通信瓶颈。

NCCL 2.12引入了对集合通信场景的PXN优化，即PCI ×

NVLink。通过将数据经由NVLink链路进行中转，可以让RDMA通信双端均为同号rank，这样通信时仅需要经过一级交换机，同时还可以将多个rank的数据进行消息聚合，实现网络流量优化。

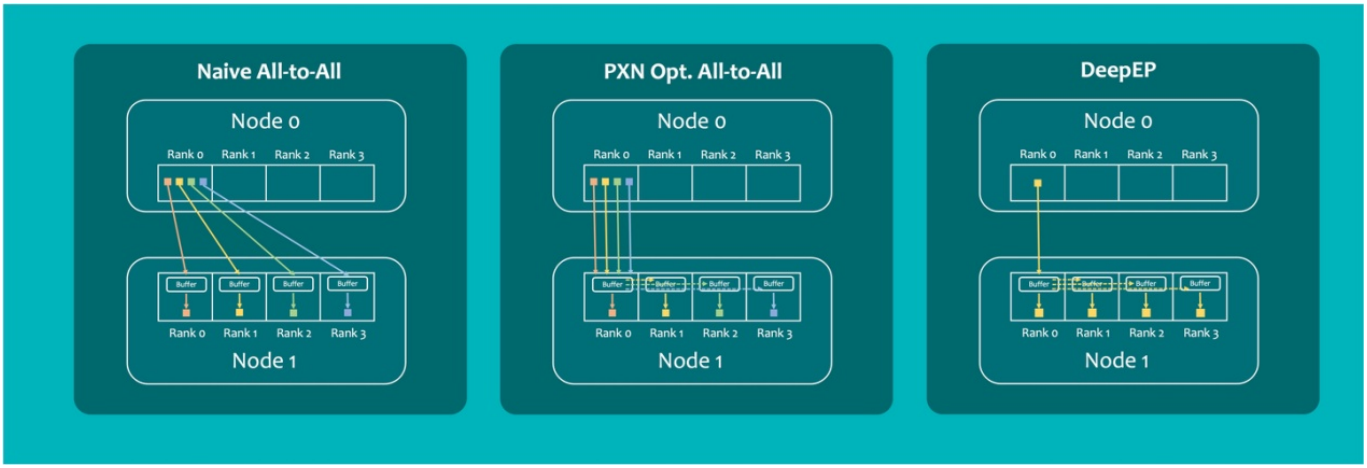
[图片]



MoE 场景下，一个 token 可能要 broadcast 到跨机的 N 个 rank。如果使用传统集合通信的 A2A 算法，则需要产生 N 次 RDMA 传输。DeepEP 的优化方法是，将这个 token 发送给跨节点的同号 rank，再在该 rank 下通过 NVLink 链路 broadcast 到其他 rank，从而仅需要 1 次 RDMA 传输即可完成通信。

下图展示了 Native A2A、PXN 优化和 DeepEP 通信的区别：

[图片]



DeepEP通信算子设计：

- Normal模式（训练/Prefill）：面向高吞吐场景，聚焦带宽最大化。通过优先利用NVLink域内高带宽、最小化跨节点RDMA流量，显著降低总通信开销。

- Low-Latency模式（推理/Decode）：面向低吞吐时延敏感场景，聚焦时延最小化。因数据量小、二级交换非瓶颈，采用Rank间直连RDMA，规避多级Buffer中转与线程同步开销。
- 并行优化：细粒度拆解通信任务，映射至GPU的SM、Warp、Thread多级并行维度，实现通信流水线并行与带宽饱和利用。

DeepEP 通信机制详解

（1）初始化通信 Bootstrap

通过torch.distributed建立的通信组，在NVLink域传递共享内存句柄，在RDMA域broadcast rank 0的unique id，最终建立起机内的共享内存和机间同号 GPU 的 RDMA 通信组。

（2）创建通信 Buffer

每个 rank 在 GMEM 开辟两个buffer：RDMA buffer 和 NVL buffer。

i. RDMA buffer
RDMA buffer为NVSHMEM注册的对称内存，即读写远端内存时仅需传入本地的地址。其构造如下图所示：

[图片]

1、rdma_channel_data：存放参与RDMA通信的数据。分为Send和Recv两个分块，每块形状为 (K1, rdma_ranks, channels)。其中K1 = rdma_recv_chunk_token * num_bytes_per_token。

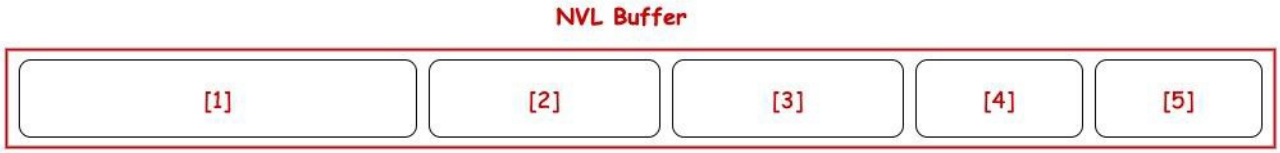
2、rdma_channel_meta：存放通信的meta数据，包括RDMA域和NVL域收发数据的头尾指针。分为Send和Recv两个分块，每块形状为 (K2, rdma_ranks, channels)。其中 K2 = nvl_ranks * 2 + 2。

3、rdma_channel_head：存放远端消费数据的进度，远端消费本rank发送的数据后，需要写入该区域，本地用于控制数据发送速度。形状为 (1, rdma_ranks, channels)。

4、rdma_channel_tail：存放远端发送数据的进度，远端发送数据到本rank后，需要写入该区域，本地可以得知同步数据是否到达。形状为 (1, rdma_ranks, channels)。

ii. NVL Buffer
NVL Buffer为机内的非对称内存，其构造如下图所示：

[图片]



1、 nvl_channel_data：存放参与NVLink通信的数据。形状为 (K3, nvl_ranks, channels)。其中 K3 = nvl_recv_chunk_token * num_bytes_per_token。

2 、nvl_channel_prefix_start：存放通信的meta数据，此处为NVL域收发数据的头指针。形状为 (rdma_ranks, nvl_ranks, channels)。

3 、nvl_channel_prefix_end：存放通信的meta数据，此处为NVL域收发数据的尾指针。形状为 (rdma_ranks, nvl_ranks, channels)。

4、nvl_channel_head：存放NVL
Buffer远端消费数据的进度，远端消费本rank发送的数据后，需要写入该区域。形状为 (1, nvl_ranks, channels)。

5、nvl_channel_tail：存放NVL
Buffer本地填充数据的进度，远端发送数据到本rank后，需要写入该区域，本地可以得知同步数据是否到达。形状为 (1, nvl_ranks, channels)。

（3）任务划分

GPU 一大特点是多层级的并行计算，通信场景下也应当利用这种并行能力。

i. Channel维度的任务划分
DeepEP 将参与通信的数据在token维度平均划为10份，每份数据对应一个channel来处理。数据在不同 rank 间的同号channel之中进行通信。

ii. SM维度的任务划分
由于每个 channel 均要在RDMA域和NVL域完成一次收发，单个SM并行压力太大。DeepEP中每个 channel 使用2个SM并行处理，总共需要20个SM执行通信算子。

考虑到RDMA Receive和NVL Send需要串行执行，SM任务划分情况为：

- SM 0：负责RDMA Receive 和 NVL Send
- SM 1：负责RDMA Send 和 NVL Receive

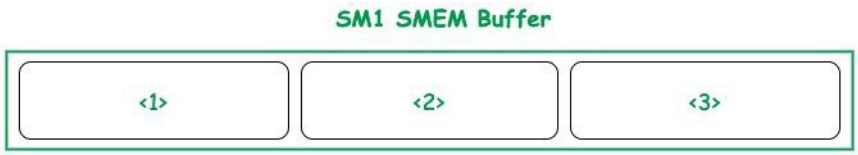
iii. Warp和Thread维度的任务划分
这里先以Dispatch过程为例，2 个 SM的数据通信在warp维度总共划分为5种角色。
- RDMA Sender（SM 1，7W）：源端封装。7个Warp按Token维并行，每Warp前8线程并行处理8个远端Rank，负责将待路由数据预填充至本地RDMA Send Buffer。
- RDMA Sender Coordinator（SM 1，1W）：发件调度。监测Buffer水位，攒足阈值后触发RDMA Write；前8线程并行管控8个远端Rank的发送流。
- RDMA & NVL Forwarder（SM 0，8W）：跨域桥接。8个Warp各负责1个NVL Rank，每Warp前8线程并行拉取8个RDMA源数据；将RDMA落地数据搬运至远端NVL Buffer。
- Forward Coordinator（SM 0，1W）：转发流控。8线程并行监控8个RDMA Rank的消费进度，实时同步至远端，实现背压与流量对齐。
- NVL Consumer（SM 1，8W）：终端落盘。8个Warp各对接1个远端NVL Rank，每Warp前8线程并行混排多RDMA源数据；从NVL Buffer提取并精准写入目标Tensor（如recv_x）。

架构本质：SM 0专注“RDMA↔NVL跨协议转发与流控”，SM 1专注“源端填充与终端消费”。通过Warp/Thread两级细粒度切分与异步状态同步，构建无阻塞的本地→RDMA→NVL→远端Tensor 高效数据流水线。

（4）创建SMEM空间

以Dispatch为例，SM1中角色1、2需通过SMEM同步RDMA Buffer填充状态。布局如下

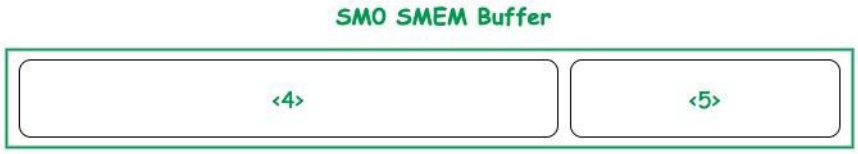
[图片]



<1> rdma_send_channel_lock： 管理<2><3>的锁。形状(rdma_ranks,)。
<2> rdma_send_channel_tail： 存RDMA Buffer填充进度（角色1写/角色2读）。形状(rdma_ranks,)。
<3> rdma_send_channel_window： 用于transaction window机制，解决多warp并行填充进度报告问题。形状(rdma_ranks,)。

SM0中，角色3、4需通过SMEM同步RDMA数据消费状态。布局如下：

[图片]



<4> forward_channel_head： 存RDMA消费进度（即NVL Buffer填充进度）。形状(rdma_ranks, nvl_ranks)。
<5> forward_channel_retired： 任务完成信号。对应NVL Rank写1代表数据填充完毕。形状(nvl_ranks,)。

（5）执行通信流程

下面分 5 个角色解析 Dispatch 过程的通信机制。

- RDMASender（7W）：源端封装。并行写入头尾指针与预发量；按Token轮询统计累计发送规模，检测远端RDMA Buffer空间，数据就绪后填充本地Send区，经transaction window写<2>信号触发调度。
- RDMASenderCoordinator：RDMA分片调度。同步SMEM后获取总量；轮询<2>，攒足chunk即通过RDMA发往远端Recv区，写远端<4>通知抵达，循环至全量发完。
- RDMAAndNVLForwarder：协议桥接。接收路由并同步指针；双重就绪检查（NVL目标空间可用 + RDMA数据达本地）后，将数据直写远端NVL [1]。同步更新RDMA消费进度<4>与NVL填充标记远端[5]，置位<5>上报。
- ForwardCoordinator：全局协调。监控<5>判定阶段完成；实时捕获<4>消费信号，写回远端(3)反向通知源端，实现跨域背压与流量控制。
- NVLConsumer：终端消费。等待指针就绪并计算接收量；轮询[5]确认数据到达后，从本地[1]提取

至Tensor，写远端[4]同步消费状态，闭环终结。

注：()/[]为GPU显存Buffer，<>为SMEM同步标志位。全链路通过多级Warp并行与异步信号握手，实现RDMA发送 → 跨协议转发 → NVLink消费 的无阻塞流水线。

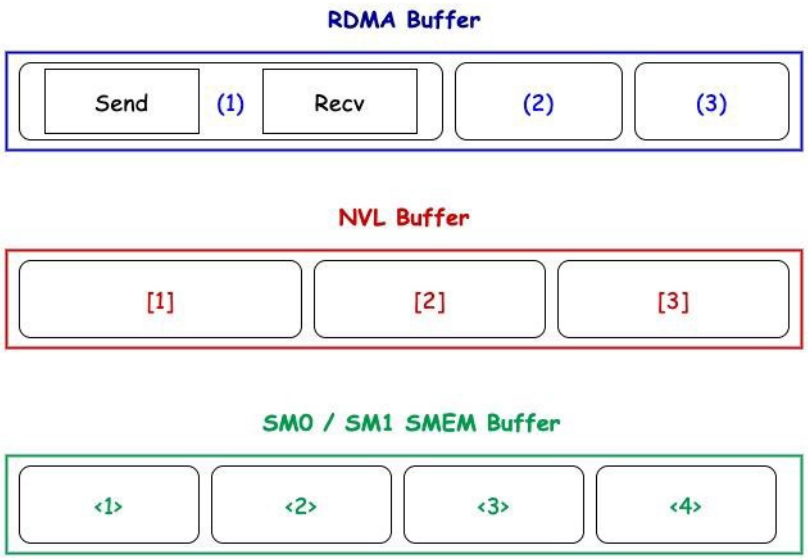
（6）Combine通信流程

Combine流程为Dispatch逆过程，数据流向改为NVL→RDMA。共10个channel，每channel含2个SM：

- SM 0：NVL Send + RDMA Receive
- SM 1：NVL Receive + RDMA Send

Buffer 情况如下图所示：

[图片]



(1)-(3)与[1]-[3]（RDMA/NVL Buffer）参数均与前阶段一致。
SMEM同步区：
<1> forwarder_nvl_head：NVL消费进度。形(numForwards, nvl_ranks)
<2> forwarder_retired：完成信号。形(numForwards,)
<3> rdma_receiver_rdma_head：RDMA消费进度。形(numRDMAReceivers, rdma_ranks)
<4> rdma_receiver_retired：完成信号。形(numRDMAReceivers,)
其中，numForwards=24（对应【角色2】warp数），numRDMAReceivers=16（对应【角色3】warp数）。

Combine阶段四大角色职责与执行流：

- NVLSender（SM0，8W）：NVLink直发器

每Warp负责1个NVL Rank，前8 Thread并行处理8个RDMA
Rank。按Token轮询，命中目标后等待远端NVL
Buffer空闲，将数据写入远端[1]，并通过远端[3]触发接收通知。

- NVLAndRDMAForwarder（SM1，24W）：NVL→RDMA桥接器

每3 Wap为一组负责1个RDMA
Rank（Token维并行）。初始化同步后，循环读取本地[2]的NVL数据暂存至Send区；待RDMA
Buffer就绪，由组内末位Warp通过RDMA发往远端[1]，写远端[3]通知到达；全程通过<1>向Coordinator上报NVL消费进度。

・ RDMAReceiver（SM0，16W）：RDMA数据回收器
16
Wap在Token维并行。初始化同步后，通过<3>上报进度并轮询本地[3]，待数据就绪后从本地[1]的Recv区提取数据写入最终Tensor。
・ Coordinator（SM0/SM1，各1W）：全局流水线协调器
・ SM1侧（NVL同步）：轮询<1>获取消费更新，写入本地[2]通知NVLSender释放/覆盖NVL数据。
・
SM0侧（RDMA同步）：轮询<3>获取消费更新，写入远端[2]通知Forwarder推进下一Chunk发送。
核心作用：打通NVL与RDMA两条通信链路的消费-生产信号，确保多级Buffer无缝流转与Warp级严格同步。
DeepEP核心架构（专为MoE EP定制）：
・ 双层拓扑管理：机内NVLink与跨机RDMA独立配置对称/非对称缓冲。借CUDA IPC/VMM实现跨进程显存共享，依托NVSHMEM接管RDMA域通信。
・ 四级并行分解：数据按Channel切分 → 每通道独占2 SM → SM内分配专属Warp（收发/转发/协调/消费）→ Warp内Thread并行处理多Rank。以极细粒度实现非对称高效传输。
・
正反向对称设计：Combine直接复用Dispatch的缓冲结构与通信原语，仅调整数据流向与Warp配比，确保双向带宽严格对等。
・
范式价值：实证了深入算子层的细粒度通信调度可有效突破NCCL等传统库的抽象瓶颈，为异构链路专用通信算子设计提供了可复用的工程范式。

6.5.4 算子级 FlashAttention 优化实战

6.5.4.1内容概览

Flash-Attention（以下简称 FA）的算法主线并不复杂：把 attention 分块，边算 QK 边做 online softmax，再继续完成 PV。但当实现平台从 A100/H100 切换到海光 DCU 时，性能差异的关键往往已经不再是“公式怎么写”，而是“Q、K、V 如何在 HBM、LDS、VGPR、MFMA 之间流动”。这也是同样的 FA 前向在两代 DCU 上会形成两套不同实现的原因。

这一节围绕仓库中两支已经上线的前向 kernel 展开：

BMZ 路径：compute_attn_1rowblock_16x64，位于 flash-attention-cutlass/flash-attention-cutlass/csrc/flash_attn/src/flash_fwd_kernel.h 第 595 行，基于 CuTe 抽象，Tile 为 16×64；

NMZ 路径：compute_attn_mha_1rowblock_gfx938，位于 flash-attention-v2.6.3/csrc/flash_attn/src/flash_fwd_b16_fa.h 第 875 行，手写寄存器布局，Tile 为 128×128。

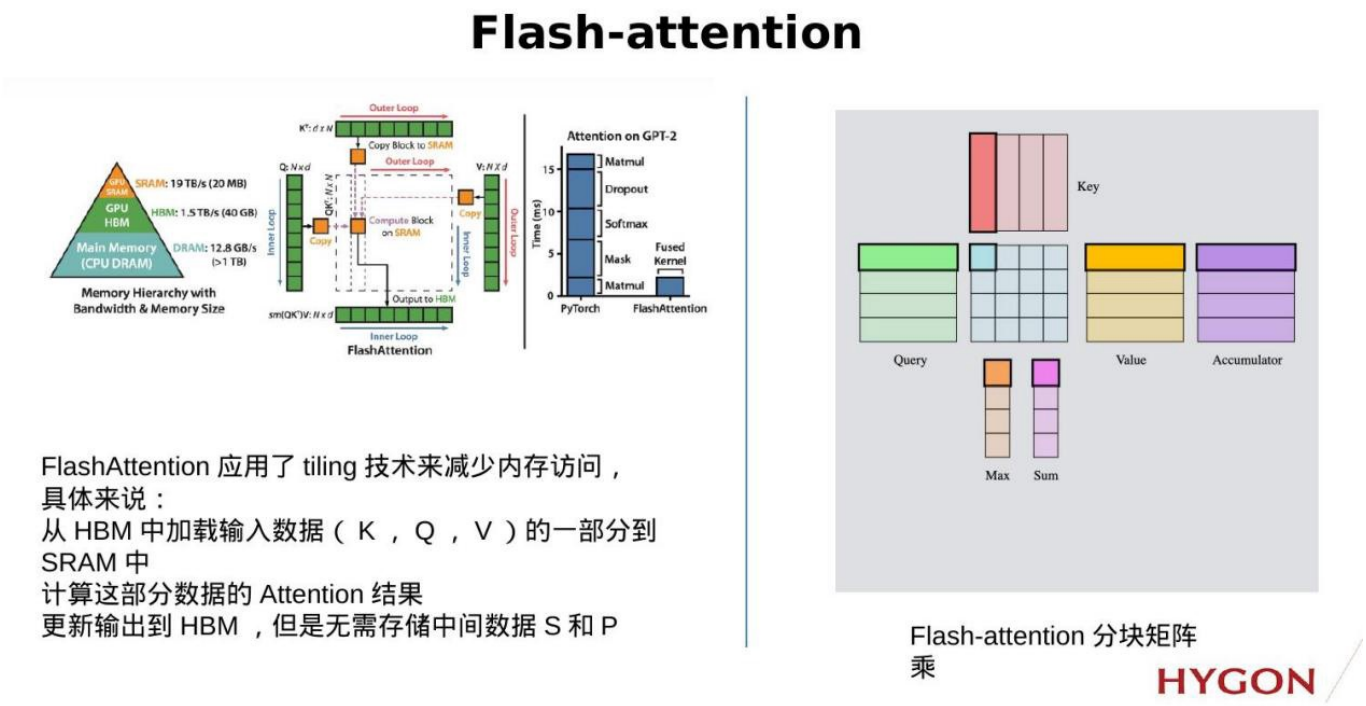
核心结论可以概括为：

BMZ 的核心是“把通用指令和软件调度用到极致”；

NMZ 的核心是“围绕 MLS 和寄存器布局复用，把数据流尽量做成零拷贝双流水”。

建议读者先熟悉 FA v1/v2 的 online softmax 推导，以及 CuTe 中 TiledMma / TiledCopy 的基本概念。这一节聚焦“优化实战”，不再展开 attention 数学本身，而把重心放在数据通路、流水组织和硬件约束上。文中代码片段均来自上述两个源文件，只做极小幅度裁剪，便于读者对照源码定位。

[图片]



6.5.4.2 BMZ 与 NMZ 为什么会形成两套 FA 实现

Flash-Attention 的数学框架在两代卡上没有变化，真正决定实现风格的主要有三点：

LDS -> VGPR 的搬运能否直接对齐 MFMA 期望的寄存器布局；

QK 的输出在 softmax 之后，能否低成本接到 PV 的 A 端；

掩码、预取、同步这些“控制流细节”，会不会把主流水冲散。

BMZ（gfx936）与 NMZ（gfx938）在这些点上的能力差异，直接决定了优化路线。下面这张表只保留与 FA 前向关系最密切的几项能力，用来先建立整体图景。

维度	BMZ（gfx936）	NMZ（gfx938）	对 FA 前向的直接影响
exp2 延迟	约 4 cycle	约 2 cycle	NMZ 的 online softmax 关键路径更短
矩阵读路径	主要依赖 buffer_load -> LDS -> ds_read	新增 MLS / ds_read_matrix	NMZ 可直接按 MFMA 望布局取数

QK 结果接 PV	需要 float2half + Register2MMAC	4-Interleaved 输出更复用	NMZ 更容易把 QK -> softmax -> PV 串成一链
更大 Tile 的可行性	会明显放大搬运与重排本	直读和复用能力更强	NMZ 更适合 128×128 别的大 Tile
低精度扩展	以 FP16 为主	额外支持 FP8 路径	NMZ 为后续低精度推留出空间
从实现角度看，可以把代卡的优化目标概括为句话：			

BMZ：在没有 MLS
的前提下，把旧数据通路的每一级都压紧，尽量减少“等内存、等同步、等重排”的时间；

NMZ：围绕新指令重组数据流，尽量把 QK、softmax、PV 组织成一条连续的寄存器接力链。

下面先给出硬件差异图，再把原先“优化要点总览”的内容整理成 BMZ / NMZ
并列对照表。相较于图片中的简化表述，表格对几处容易误解的内容作了进一步澄清。例如，BMZ
并不能把 P=QK 完全“直接”送入 PV；NMZ 也不是额外依靠 mmac_f32_16x16x16_f16_lit
去“做一次寄存器重排”，而是利用其更容易复用的输出布局来降低重排成本。

[图片]

Flash-attention 优化

硬件单元	BMZ	NMZ	备注
LDS	64KB per CU	64KB per CU	
L1	32KB per CU	32KB per CU	
L2	8MB	8MB	
vgpr	256x3	256x3	
超越函数 (exp2)	指令周期 4Qcycle	指令周期 2Qcycle	
MLS + ds_matrix	无	有	
FP8 数据类型	不支持	支持	
mmac_f32_16x16x16_f16_lit	不支持	支持	Matrix C/D 4-Interleaved layout (NOTE: LIT is supported by V_MMAC_F32_16x16x8_F32/TF32, V_MMAC_F32_16x16x16_F16/BF6, V_MMAC_I32_16x16x32_I/U8 and V_MMAC_I32_16x16x64_I/U4)

HYGON

维度	BMZ	NMZ

核心思路	以通用指令为基础，把 HBM -> LDS -> VGPR -> MFMA 这条传统据通路压紧	围绕 MLS 与 ds_read_matrix 组数据流，尽量做成零拷贝双流
Tile 与任务切分	采用较小 Tile（64×64），并结 K/V 反向遍历，优先提升 L2 命与局部性	采用更大 Tile（典型为 128×128×32），提升单个 block 的吞吐密度
关键访存路径	依赖 buffer_load_dwordx4 … lds、ds_read 等传统路径，把 HBM 数据高效搬到 LDS 再送入 MFMA	以 MLS + ds_read_matrix 为核，让 LDS -> VGPR 更接近 MFMA 期望的寄存器布局
LDS / 共享存储处理	通过 swizzle 缓解 bank conflict，尤其服务 PV 阶段读取	通过更规整的矩阵读取路径与 LDS 复用，减少显式转置和额外排
QK -> PV 衔接	需要尽量把 P 的布局组织成便于续消费的形态，但仍需 float2half + Register2MMAC，非完全直接进入 PV	利用 mmac_f32_16x16x16_f16_lit 容易复用的 4-Interleaved 输，减少或省去显式重排
流水组织	依赖内联汇 、s_waitcnt、sched_barrier、s_setprio 手工编排访存与计算水	尽量把 QK、softmax、类型转、PV 串成连续双流水，并把 mask 尾段拆出主循环
并发与资源约束	更偏向在既有硬件能力下维持稳吞吐，重点控制 LDS 等待与寄器中转对 MFMA 的影响	追求每个 CU 4 个 block 并发、个 block 4 个 wave，并控制 LDS <= 16 KB / block、VGPR <= 192 / thread

6.5.4.3 BMZ 路径：把通用指令和软件流水压到极致

BMZ 路径的出发点很明确：既然没有 MLS，就需要把 HBM -> LDS -> VGPR -> MFMA 这条传统路径尽量压缩到位。它的重点不是“发明一条新链路”，而是尽可能减少每一级之间的额外成本。

Tile 切分与基础数据通路

BMZ 版 kernel 采用“细 M、粗 N”的 Tile 组织方式：每个 thread block 处理 (kBlockM=16, kHeadDim) 的 Q 行块，以及 (kBlockN=64, kHeadDim) 的 K/V 列块。这样切有两个直接好处：

kBlockM=16 能较自然地贴合 MFMA 的 M 向消费方式，避免在 Q 端引入额外的外层展开；

kBlockN=64 的 K/V 块大小适合配合 buffer_load_dwordx4 做规整搬运，使主循环节奏更容易排齐。

Q 在 prologue 预取后尽量常驻寄存器；K 和 V 共用一段 LDS；V 再额外通过转置视图供 PV

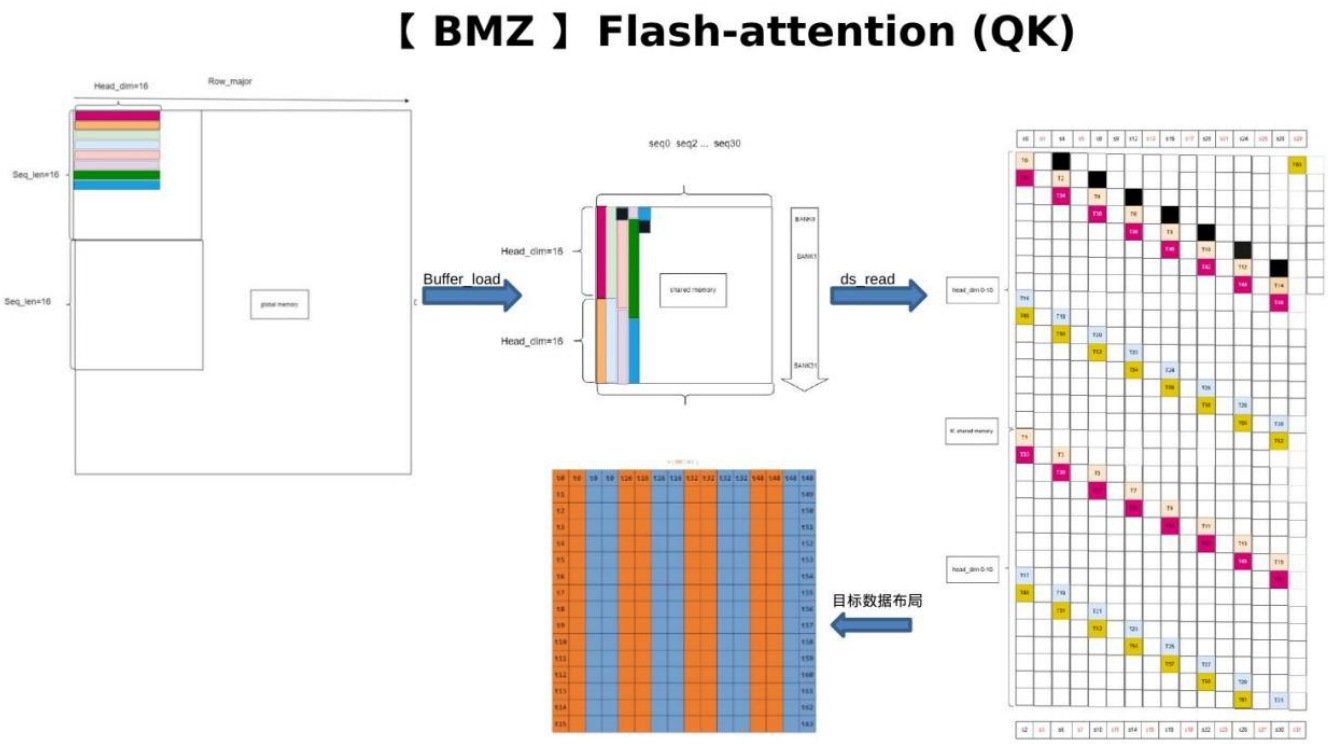
阶段读取。BMZ
的基本原则并不是“所有数据都常驻”，而是优先让生命周期更稳定的数据留在寄存器中。

BMZ 主要张量与布局声明：

```
typename Kernel_traits::TiledMma16x64 tiled_mma;                // QK GEMM
typename Kernel_traits::TiledMma16x32 tiled_mma_for_gemm1;      // PV GEMM
Tensor sQ = make_tensor(make_smem_ptr(smem_), SmemLayoutQ{});
Tensor sK = make_tensor(sQ.data(),          SmemLayoutKV{});    // Q/K 复用 LDS
Tensor sV = make_tensor(sK.data() + (Share_K_V_smem ? 0 : size(sK)),
                        SmemLayoutV{});
Tensor sVt = make_tensor(sV.data(), SmemLayoutVtransposed{});   // PV 零成本转置视图
```

从宏观上看，BMZ 在 QK 阶段的数据流可以概括为：HBM 先通过 buffer_load 落到 shared memory，再经 ds_read 进入 MFMA 所需的寄存器布局；PV 阶段则继续复用同一段 LDS，并通过 sVt 视图避免显式转置。

[图片]



反向遍历 K/V，优先保住 L2 局部性

BMZ 主循环里一个很容易被忽略、但收益很扎实的细节，是把 n_block 的起点设为 n_block_max - 1，然后向前递减。

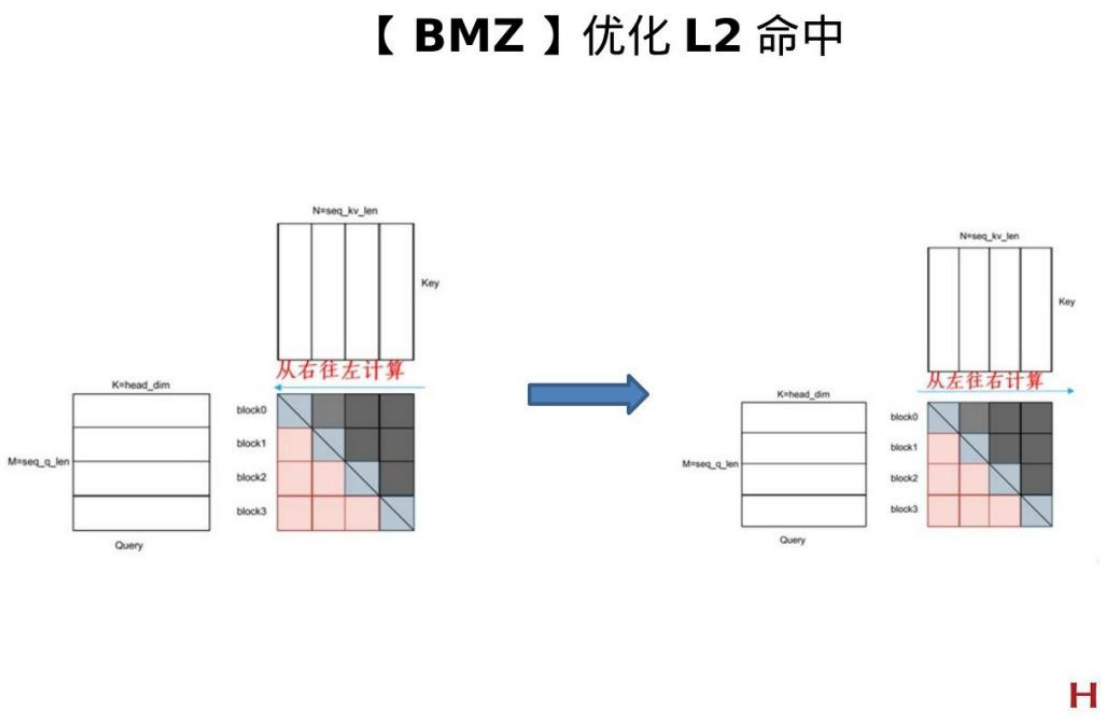
```
int n_block = n_block_max - 1;
flash::copy<Is_even_MN, Is_even_K>(gmem_tiled_copy_QKV,
                                   tKgK(_, _, _, n_block),
                                   tKrK, tKVcKV, tKVpKV,
                                   binfo.actual_seqlen_k - n_block * kBlockN);
```

背后的原因并不复杂：相邻 m_block 处理的 K/V 区间在尾部往往高度重叠，尤其 causal 场景更明显。若采用正向扫描，上一轮刚命中的 K/V

可能很快就被后续访问淘汰；改成反向扫描后，前一个 m_block 刚读过的尾部 K/V 更容易继续留在 L2 中，被后一个 m_block 复用。

这类优化在代码层面看起来只是“循环方向变了”，本质上却是在降低 HBM 压力。对长序列场景来说，这类收益通常比单独优化某一条指令更稳定。图 5 中左侧是反向扫描，右侧是作为对照的正向扫描。

[图片]



QK / PV 的软件流水：把等待藏进下一拍

BMZ 没有 MLS，LDS -> VGPR 的搬运只能靠传统 ds_read 体系完成。因此真正的难点不是“能不能算”，而是“如何把等待尽量压进下一拍，而不是停在 s_waitcnt 上”。

gemm_rs_swait 的组织方式可以概括为一句话：先发下一拍的 ds_read，再计算当前拍的 MFMA，同时用调度屏障把这段顺序钉死，避免编译器把关键路径重新排坏。

gemm_rs_swait 的核心循环：

```
cute::copy(smem_tiled_copy_B, tCsB(_, _, _0{}), tCrB_copy_view(_, _, _0{}));
asm volatile("s_waitcnt lgkmcnt(0)\n\t");
#pragma unroll
for (int i = 0; i < size<2>(tCrA); ++i) {
  if (i < size<2>(tCrA) - 1) {
    cute::copy(smem_tiled_copy_B, tCsB(_, _, i + 1), tCrB_copy_view(_, _, i + 1));
    asm volatile("s_waitcnt lgkmcnt(0)\n\t");
  }
  cute::gemm(tiled_mma, tCrA(_, _, i), tCrB(_, _, i), acc);
  if (i == 0) {
    __builtin_amdgcn_sched_barrier(0);
```

```
    __builtin_amdgcn_s_setprio(1);  // 提升 MFMA 在调度器中的优先级
    __builtin_amdgcn_sched_barrier(0);
  }
}
__builtin_amdgcn_s_setprio(0);
```

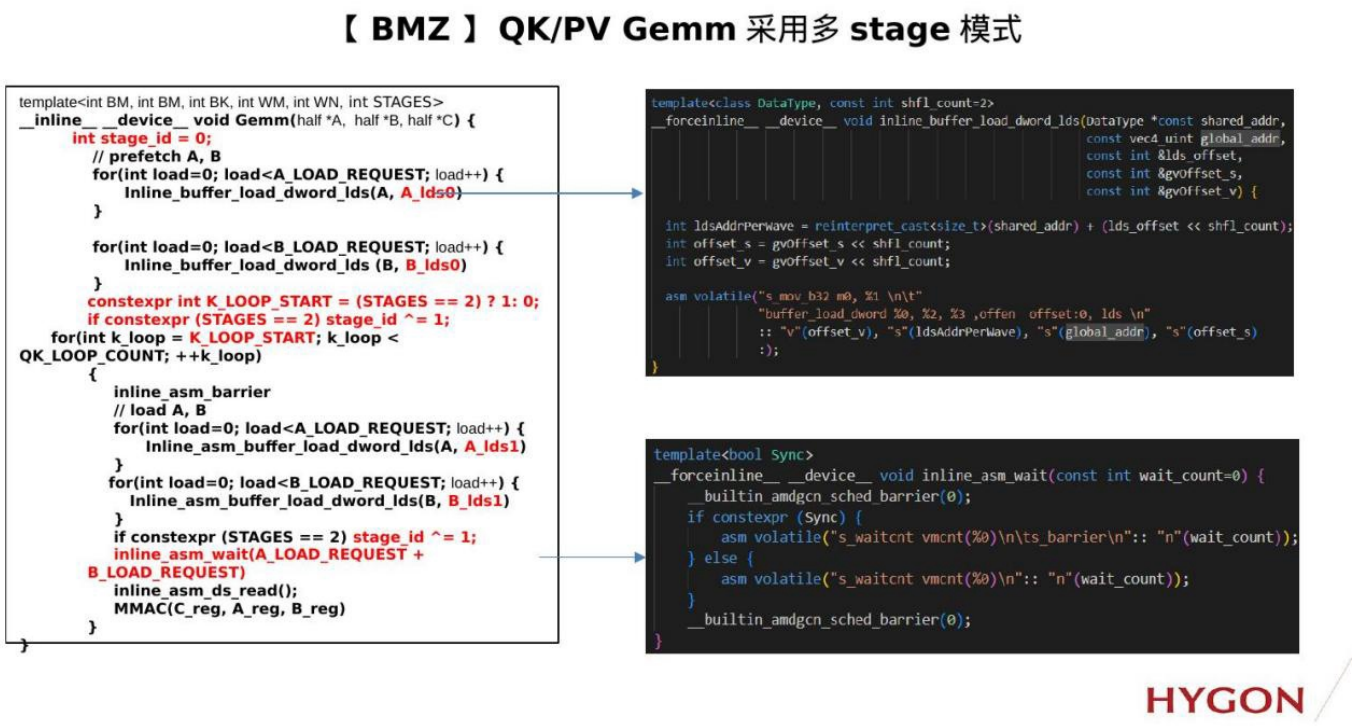
这里最关键的不是双缓冲本身，而是下面两点：

s_waitcnt lgkmcnt(0) 是强同步，一旦放错位置，MFMA 就会空等；

sched_barrier(0) + s_setprio(1)
的组合能显式告诉编译器，这一小段调度窗口不要随意重排，并优先保证 GEMM 主干发射。

BMZ 这一套做法的本质，是用软件手工替代部分硬件流水能力。NMZ
上虽然等待成本更低，但“关键路径要显式看护”这一原则仍然成立。

[图片]



buffer_load_dwordx4 ... lds：将 HBM 数据直接写入 LDS

BMZ 上最关键的一条指令是 buffer_load_dwordx4 ... lds。它将全局内存数据直接写入
LDS，中间不再经由 VGPR 做临时中转。对于寄存器本就紧张的 FA
前向来说，这相当于同时减少了数据搬运和寄存器占用。

HBM -> LDS 直通的内联汇编：

```
#if (defined(__gfx936__) || defined(__gfx938__))
asm volatile(
  "s_mov_b32 m0, %1 \n\t"
  "buffer_load_dwordx4 %0, %2, %3 ,offen  offset:0, lds \n"
  ::"v"(offset_v), "s"(ldsAddrPerWave), "s"(global_addr), "s"(offset_s)
);
```

```
#endif
```

这段汇编有三个细节值得单独记住：

m0 必须先被设置成 wave 内目标 LDS 基址，否则数据会落到错误的 LDS 地址；

offset_v = -1 可以用来表示越界线程，配合模板参数把边界处理折叠到同一条指令里；

offen 让 offset 来自 VGPR，适合 paged attention、变长序列这类不完全规整的访存场景。

如果走传统路径，即先 buffer_load_dwordx4 到 VGPR，再 ds_write_b128 落 LDS，那么既要多占一份寄存器，又要多发一条 LDS 写指令。BMZ 上这条“全局直通 LDS”几乎是必须拿到手的基础收益。

LDS 布局与 bank conflict控制

BMZ 的 K/V 在 LDS 中采用 swizzle 布局，并在最后一维加入 8 元素 padding。这样做的目标只有一个：避免 PV 阶段把大量线程压到同一组 bank 上。

PV 阶段 V 的读取 Atom：

```
auto smem_tiled_copy_V = make_tiled_copy_B(
   Copy_Atom<GFX928_DS_READ_DS_M32x16_B16, Element>{}, tiled_mma_for_gemm1);
auto smem_thr_copy_V = smem_tiled_copy_V.get_thread_slice(tidx);
Tensor tOsVt = smem_thr_copy_V.partition_S(sVt);
```

这里需要额外说明一个容易引起误解的点：GFX928_DS_READ_DS_M32x16_B16 是仓库内部沿用的 Atom 命名，关键在于它描述的是“32×16、b16 的 DS 读取形态”，并不意味着本节目标 ISA 变成了 gfx928。

最重要的一点是：PV 读取的不是原始 sV，而是转置视图 sVt。如果既不做 swizzle，也不在 LDS 侧准备好适合 PV 的读取形态，那么最坏情况下会出现严重 bank conflict，吞吐会出现明显的台阶式下跌。

online softmax：BMZ 上最容易露出延迟的那一段

在 BMZ 上，exp2 的延迟比 NMZ 更高，所以 softmax_rescale_o 往往是除 MFMA 之外最值得盯的热点。

代码层面采取了两个直接策略：

统一调用 __builtin_amdgcn_exp2f，避免在 kernel 内做额外的换底开销；

将缩放因子预先整理为 log2(e) * scale_softmax，即 params.scale_softmax_log2，把运行期计算压到最少。

online softmax 的调用入口：

```
masking_step == 0
 ? softmax.softmax_rescale_o</*Is_first=*/true,  Check_inf>(acc_s, acc_o, params.scale_softmax_log2)
 : softmax.softmax_rescale_o</*Is_first=*/false, Check_inf>(acc_s, acc_o, params.scale_softmax_log2);
```

第一轮与后续轮次分开模板实例化，也是为了把 row_max 初始化等额外判断压到最低。

此外，scores_sum 的行内归约没有走 shuffle，而是借助 __builtin_amdgcn_mmac_f32_16x16x4f32 完成 wave 内累加。这是一个很典型的“DCU 风格”优化：把 MMAC 暂时当作 reduce 单元来使用，用硬件矩阵指令替代传统 lane shuffle。

PV 阶段的隐藏成本：float2half + Register2MMAC

BMZ 路径还有一个很容易在整体图里被忽略的成本：QK GEMM 的结果是 FP32 累加寄存器，而 PV 的 A 端需要 FP16，且寄存器排列方式也不同。因此 QK 到 PV 之间还要补一段格式与布局变换。

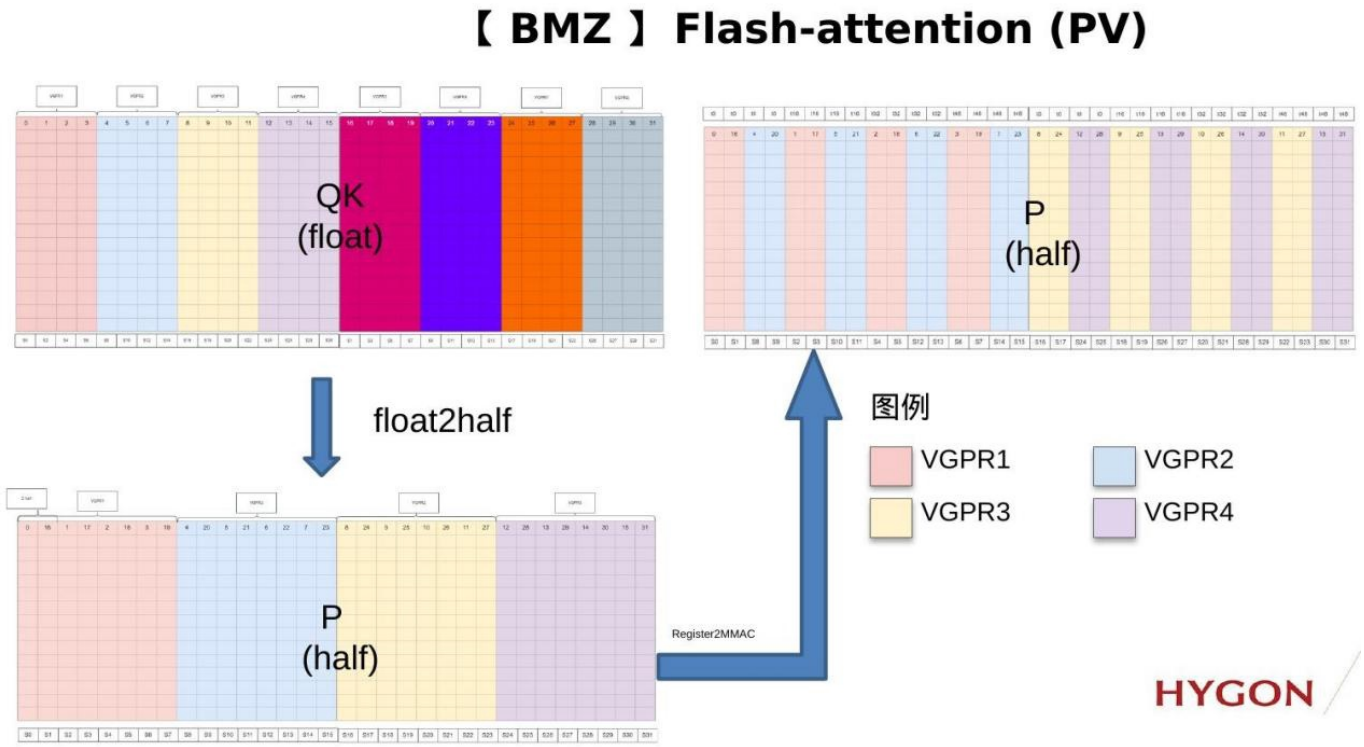
这段变换通常分成两步：

先把 QK(float) 压成 P(half)；

再通过 Register2MMAC 做一次 lane-to-lane 的寄存器重排。

这一步不会像 HBM 访问那样在 profile 里“特别显眼”，但它是真实存在的链路成本，也是 BMZ 无法像 NMZ 一样自然做成零拷贝接力的关键原因之一。

[图片]



小结

BMZ 路径的优化点虽然较多，但主线相当统一：在缺少 MLS 的前提下，尽可能充分利用每一级通用能力。反向遍历提升 L2 命中、buffer_load_dwordx4 ... lds 节省寄存器中转、sched_barrier + s_setprio 维持 MFMA 主干优先级、swizzle + padding 规避 bank conflict、MMAC 兼做 reduce，这些手段最终都服务于同一目标：尽可能减少传统数据通路中的停顿。

6.5.4.4 NMZ 路径：围绕 MLS 组织零拷贝双流水

如果将 BMZ 的关键词概括为“补齐能力缺口”，那么 NMZ

的关键词可以概括为“围绕新硬件能力组织数据流”。NMZ 不再局限于优化旧流水，而是直接围绕 MLS、ds_read_matrix 和可复用寄存器布局来重组整条数据流。

也正因为如此，NMZ 版本放弃了 CuTe 的抽象，转而使用 union_vec4_f16x2、vec4_Accum 这类更接近物理寄存器布局的 POD
类型。原因并不只是“手写更底层所以更快”，而是新指令要求布局高度对齐，抽象层越厚，越难把收益完整兑现出来。

更大的 Tile 与更高的吞吐密度

NMZ 的典型配置是 kBlockM=128, kBlockN=128, kBlockK=32, kWaveM=32, kWaveN=32。这意味着一个 block 会用 4 个 wave 并行处理一块 128×128 的 score 子矩阵。

NMZ 的 Kernel 配置与共享内存复用：

```
constexpr int WARP_M  = Kernel_traits::kWaveM;     // 32
constexpr int WARP_N  = Kernel_traits::kWaveN;     // 32
constexpr int STAGES  = Kernel_traits::STAGES;
constexpr int WARP_NUM = kBlockM / WARP_M;         // 4

extern __shared__ Element smem[];
Element* q_lds = smem;
Element* k_lds = q_lds;                    // Q/K/V 轮流复用同一段 LDS
Element* v_lds = k_lds;
```

BMZ 上 Tile 做大，往往意味着搬运、重排和等待一起膨胀；NMZ 则不同，MLS 让“更大 Tile”第一次真正变成了可控选项。因为读取形态已经更贴近 MFMA 所需布局，所以 Tile 增大后，额外成本不会像 BMZ 那样成倍外溢。

MLS 的基本思路：Q 常驻寄存器，K/V 按阶段推入 LDS

在 NMZ 路径里，Q、K、V 的放置策略更明确：

Q 在一个 m_block 的生命周期内不变，因此优先一次性预取到 VGPR；

K 和 V 则分别在各自 GEMM 之前预取到 LDS，由后续 ds_read_matrix 直接消费。

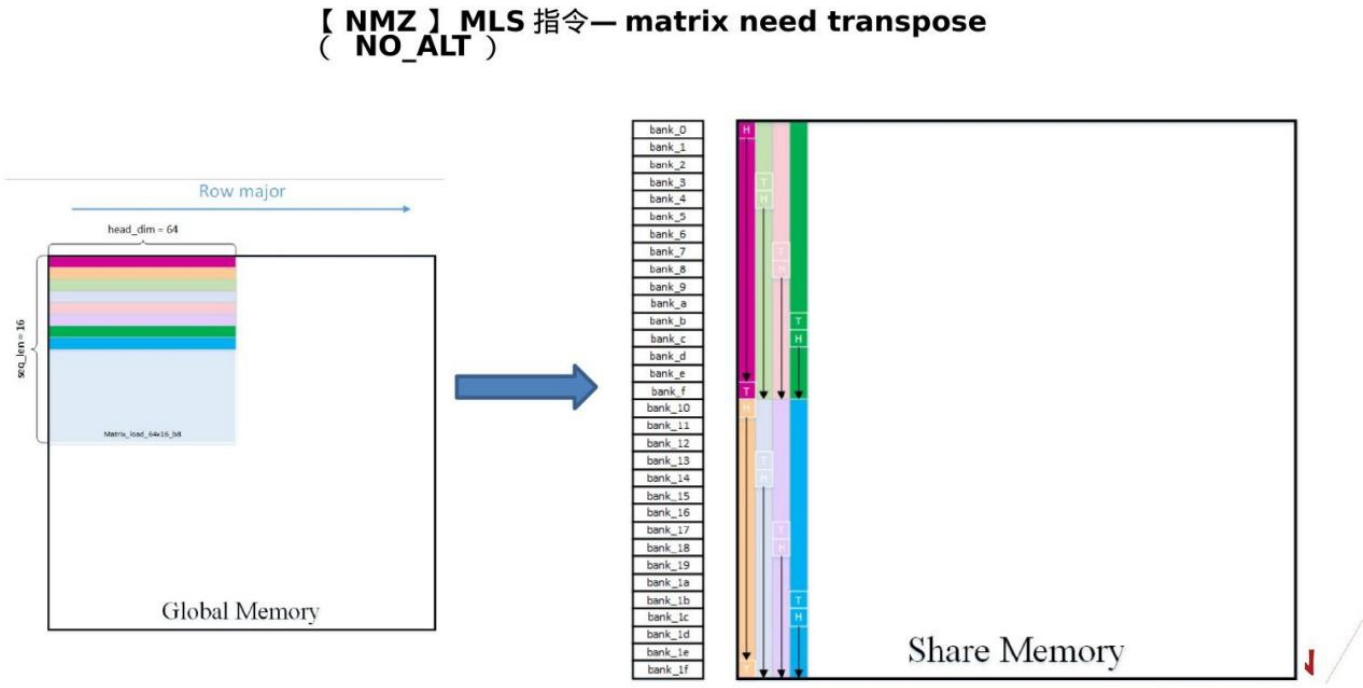
Q 与 K 的 MLS 预取入口：

```
union_vec4_f16x2<Element> q_reg[(kHeadDimQKCompute / kBlockK)
                    * (WARP_M * kBlockK) / (32 * 32) * 2];
prefetch_q_to_vgpr_mls_ds<kHeadDimQKCompute, kBlockM, kBlockK,
                  WARP_M, Element, Is_even_MN>(
  q_ptr, q_lds, q_reg, warp_id, seqlen_q_stride,
  Is_even_MN ? 0 : binfo.actual_seqlen_q - m_block * kBlockM);

// 开启 PREFETCH_K 时主循环第一拍就已经在搬运下一块 K
if constexpr (PREFETCH_K and ALLOW_PREFETCH) {
  if (n_block_min < n_block_max - n_masking_steps) {
    prefetch_k_to_lds_mls_ds<kHeadDim, kBlockN, kBlockK,
                    WARP_NUM, WARP_N, Element, Is_even_MN>(
```

```
        k_ptr, k_lds, warp_id, seqlen_k_stride,
        Is_even_MN ? 0 : binfo.actual_seqlen_k - n_block_min * kBlockN);
    }
}
```

这里体现了 NMZ 的一个典型取舍：预取深度需要结合具体场景控制。PREFETCH_K 只在部分 head_dim 和非 causal 场景下打开，是因为尾段 mask 逻辑会削弱过深预取的收益。也就是说，NMZ 的优化虽然更依赖新硬件能力，但仍然高度重视控制流的规整性。

[图片]



QK -> softmax -> PV：尽量做成一条寄存器接力链

NMZ 相对 BMZ 最大的结构性收益，在于它更有条件把 QK GEMM、softmax、FP32 -> FP16 转换、PV GEMM 串成一条连续链路，让寄存器布局在上下游之间尽量直接复用。

NMZ 非 mask 主循环：

```
for (int n = n_block_min; n < n_block_max - n_masking_steps; ++n) {
    int warp_offset_in_seqkv = n * kBlockN;
    int warp_seqkv_limit = Is_even_MN ? 0 : binfo.actual_seqlen_k - warp_offset_in_seqkv;

    if constexpr (not PREFETCH_K) {
        prefetch_k_to_lds_mls_ds<...>(k_ptr, k_lds, warp_id, seqlen_k_stride, warp_seqkv_limit);
    }

    vec4_Accum<ElementAccum> s_reg[(WARP_M/32) * (kBlockN/WARP_N)][4];
    QK_GEMM_FUNC(k_ptr, v_ptr, k_lds, v_lds, q_reg, s_reg,
            warp_id, seqlen_k_stride, seqlen_v_stride, warp_seqkv_limit);

    softmax_rescale_o_gfx938<false, Is_causal || Is_local, ...>(
```

```
    s_reg, scores_max, scores_sum, acc_o, params.scale_softmax_log2);

  union_vec2_f16x2<Element> p_reg[(WARP_M/32) * (kBlockN/32)][4];
  convert_pk_type<WARP_M, kBlockN, Element, ElementAccum>(p_reg, s_reg);

  *(uint64_t*)&k_ptr += kBlockN * params.k_row_stride * sizeof(Element); // 预偏移准备下一轮 K
  PV_GEMM_FUNC(v_ptr, k_ptr, v_lds, k_lds, p_reg, acc_o,
          warp_id, seqlen_k_stride, seqlen_v_stride, warp_seqkv_limit);
  *(uint64_t*)&v_ptr += kBlockN * params.v_row_stride * sizeof(Element);
}
```

这段代码最值得留意的，不是某一条单独指令，而是“前一步的输出，是否刚好是后一步能接住的输入”：

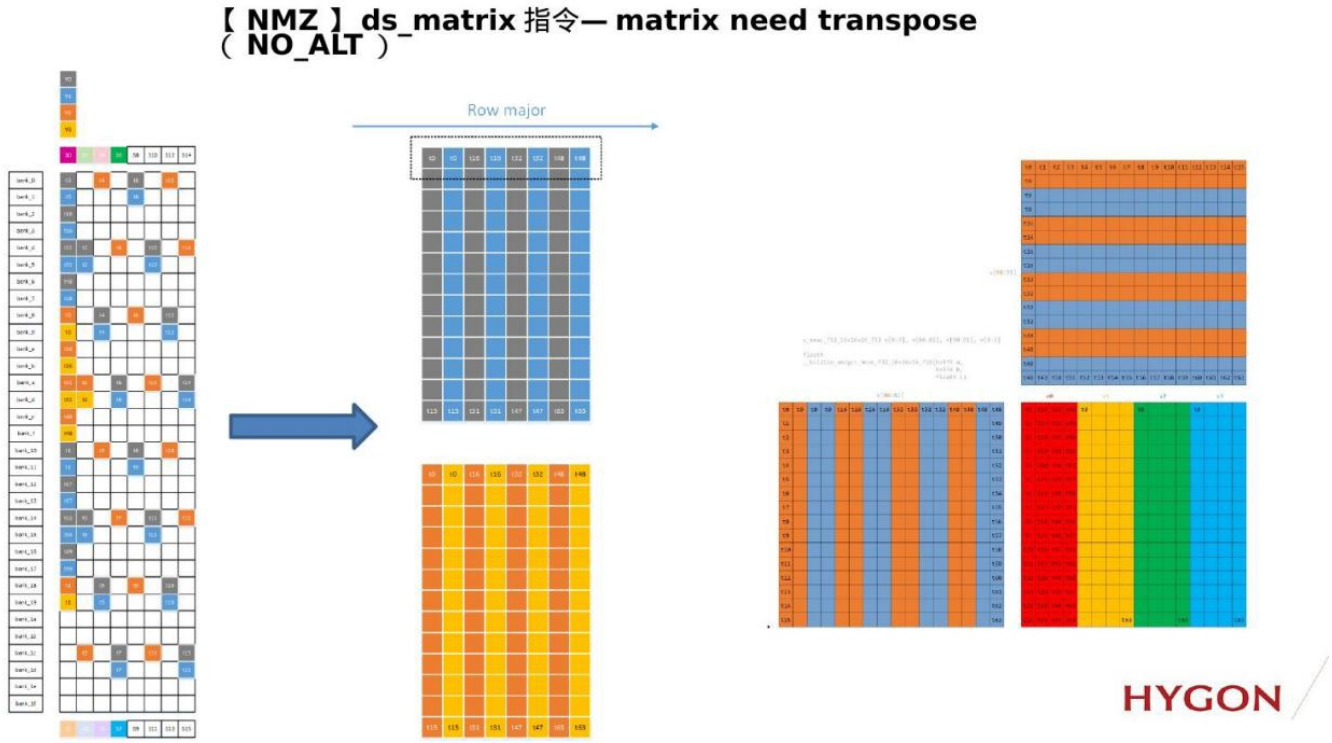
ds_read_matrix 读出的 fragment 更接近 MFMA 期望布局；

mmac_f32_16x16x16_f16_lit 的输出布局更容易作为下一次 GEMM 的 A 端输入；

因此从 s_reg 到 p_reg 的过渡里，只保留了必要的类型转换，而不再需要 BMZ 那种额外的 Register2MMAC 重排。

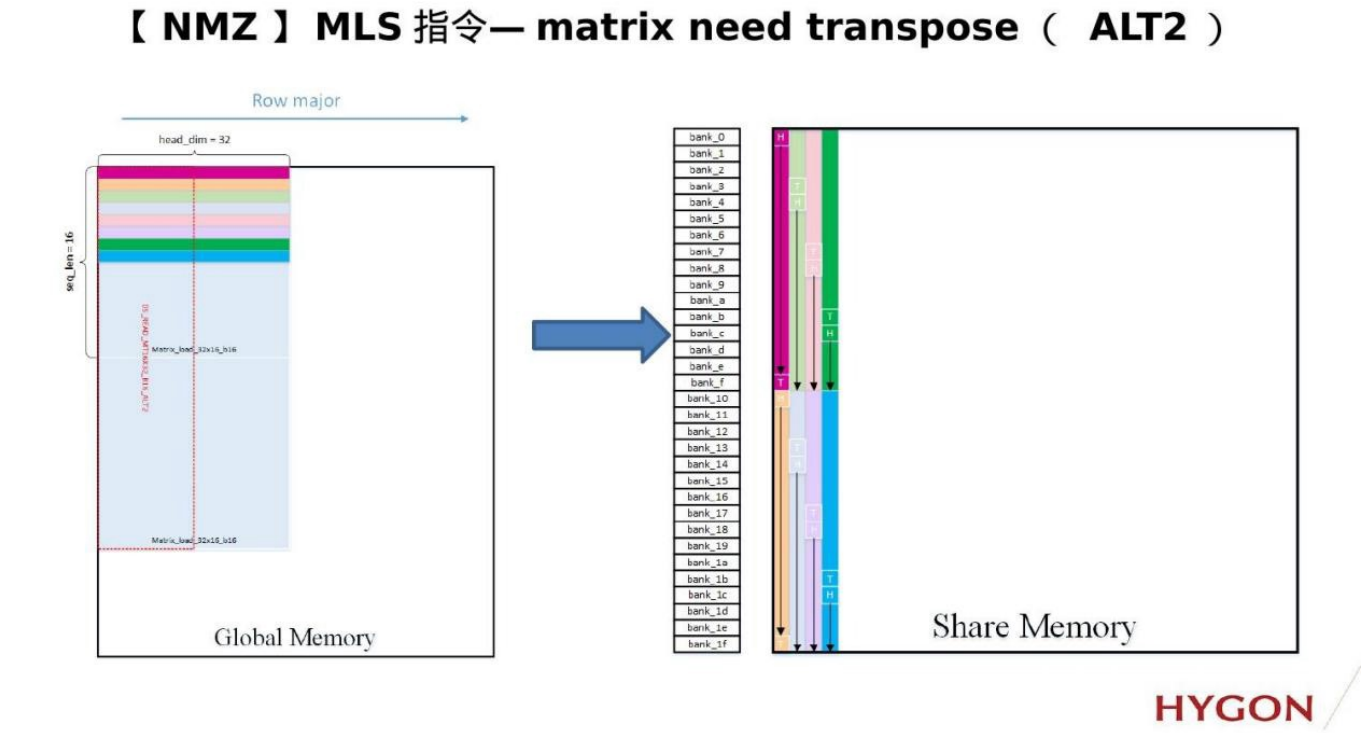
这就是 NMZ
所谓“零拷贝双流水”的核心含义：不是绝对没有任何转换，而是尽量不做多余的搬运与重排，让链路中每一级都尽量直接接住上一级。

[图片]



对 V 的读取，NMZ 还会使用 MLS 的 ALT2 模式，使其更贴合 PV 的 B 端消费形态。V 在 HBM 中同样是行主序，但 ALT2 下可以用两次 Matrix_load_32x16_b16 把 head_dim=32 的两块按“奇偶 VGPR 交错”的方式搬到更利于后续 PV 消费的形态。图 10 主要展示的是这一步 HBM -> Share Memory 的排布关系；结合后续 PV 读取路径，可以进一步把显式重排成本压到很低。

[图片]



将 mask 尾段与主循环拆分

NMZ 对主循环中的复杂分支较为敏感，尤其是会打断 ds_read_matrix / MFMA 节奏的分支。因此 causal 或 local window 的处理没有并入统一主循环，而是拆成了单独的尾循环。

NMZ 的 mask 尾循环：

```
int n_block_loop = max(n_block_max - n_masking_steps, n_block_min);
#pragma unroll
for (int masking_step = 0; masking_step < n_masking_steps; ++masking_step, ++n_block_loop) {
  prefetch_k_to_lds_mls_ds<...>(k_ptr, k_lds, warp_id, seqlen_k_stride, warp_seqkv_limit);
  vec4_Accum<ElementAccum> s_reg[(kBlockN/32) * (WARP_M/32)][4];
  QK_GEMM_FUNC(k_ptr, v_ptr, k_lds, v_lds, q_reg, s_reg, ...);
  *(uint64_t*)&k_ptr += kBlockN * params.k_row_stride * sizeof(Element);

  if constexpr (Is_causal)     apply_mask_causal_gfx938<...>(s_reg, ...);
  else if constexpr (Is_local)  apply_mask_local_gfx938</*HasWSLeft=*/true, ...>(s_reg, ...);

  softmax_rescale_o_gfx938<false, Is_causal || Is_local, ...>(s_reg, scores_max, scores_sum, acc_o,
params.scale_softmax_log2);
  union_vec2_f16x2<Element> p_reg[(WARP_M/32) * (kBlockN/32)][4];
  convert_pk_type<WARP_M, kBlockN, Element, ElementAccum>(p_reg, s_reg);
  PV_GEMM_FUNC_IN_MASK(v_ptr, k_ptr, v_lds, k_lds, p_reg, acc_o, ...);
  *(uint64_t*)&v_ptr += kBlockN * params.v_row_stride * sizeof(Element);
}
```

拆开的收益有两层：

非 mask 主循环可以保持更规整的预取和 GEMM 节奏；

mask 尾段的边界处理、越界风险、分支开销被收敛到一个更短的局部区域里。

尤其要注意，PV_GEMM_FUNC_IN_MASK 会把 PREFETCH_K 关掉，避免尾段再去预取一块可能越界的 K。这不是“保守”，而是典型的尾段正确性优先。

Prefetch 收口：主循环结束前完成在途请求同步

当主循环结束时，如果此前预取到 LDS 的 K 还没有被完全消费掉，就可能让后续 softmax / PV 读到半成品。因此 NMZ 在退出前会显式做一次 prefetch 收口。

主循环退出前的 prefetch 收口：

```
if constexpr (PREFETCH_K) {
  buffer_load_lds_dwordx1_wait<0>();  // 主循环退出前把 prefetch 的 K 消费完
}
```

这一小段代码的意义不在于简单增加一次等待，而在于显式划定流水边界。缺少这类收口时，profile 中看起来可能只是“偶发性抖动”，但根源通常是数据尚未真正稳定。

输出阶段的寄存器复用

NMZ 的 epilogue
也延续了同样的思路：尽量复用已经在手上的寄存器，而不是重新组织一套临时缓冲。PV 的累加结果 acc_o 会被直接用于最终 rescale、LSE 写回与 O 写回。

这种写法的价值在于：

最后一轮 1 / sum 的缩放可以直接在现有累加寄存器上完成；

LSE 与 O 的写回能尽量合并到更规整的 store 流里；

尾声部分不再额外拉长活跃寄存器链条。

对于一个前半段已经高度流水化的 kernel，epilogue
如果重新引入重排或临时缓存，就会抵消前面积累下来的收益。NMZ 在这一点上处理得相对克制。

causal 场景下的额外调度：LPT 方式

除了单个 block 内部的数据流，NMZ 在块级调度层面还有一项额外优化：compute_attn_gfx938 在 Is_causal 场景下会采用一种接近 longest-processing-time（LPT）的反向分派方式。

原因也比较直接：causal 场景里，不同 m_block 对应的有效 K 长度差异很大。靠前的 m_block 只需要处理较少的 K block，靠后的 m_block 则可能要处理完整长度。如果完全按自然顺序发 block，尾部那些计算量更大的 block 更容易拖慢整个 grid 的收尾。

通过优先调度计算量更大的 m_block，可以让各个 CU
的结束时间更接近，从而减轻明显的尾部不均衡现象。这不是单核指令级优化，但对整体吞吐同样重要。

小结

NMZ
路径的关键，不是“多了几条新指令”这么简单，而是新指令改变了整条数据流的组织方式。更大的

Tile、MLS 预取、ds_read_matrix、更容易复用的输出布局、拆开的 mask 尾段、块级的负载均衡，这些动作最终都在向同一个目标收敛：让 QK、softmax、PV 尽量成为一条连续、少重排、少回头的双流水链路。

6.5.4.5 从排查视角再看：BMZ 与 NMZ 的差别落在哪

前面的总览表回答的是“这两条路径各自如何优化”，这一节的对照表回答的是“当性能或行为出现异常时，应该优先从哪里查起”。也就是说，它不再重复总览信息，而是把重点重新按排查视角组织一遍。

观察角度	BMZ	NMZ
核心目标	把传统搬运链路压紧	围绕 MLS 做连续接力
Tile 风格	小 M、较保守的 N	大 Tile，追求吞吐密度
Q 的放置	尽量常驻寄存器，必要时借助 smem	一个 m_block 内优先常驻 VGPR
K/V 的组织	依赖 buffer_load -> LDS -> ds_read	依赖 MLS 预取 + ds_read_matrix
QK 结果接 PV	float2half + Register2MMAC 成明显	多数情况下只剩必要的类型转换
最怕的问题	L2 命中差、LDS 等待、bank conflict	预取没拼上、mask 分支污染、段未收口
代码风格	CuTe 抽象较多，靠调度细节补	手写布局更多，强调物理数据流
更适合的场景	兼容老卡、短序列也比较稳	长序列、DCU3、希望线性吃到 MLS 收益
从这张表可以看出，BMZ 与 NMZ 并不是“同一套路优化的同程度版本”，而是两种不同的程答案：		

BMZ 的思路是先接受硬件没有矩阵直读这一事实，再系统地处理缓存、流水、bank 和调度问题；

NMZ 的思路则是承认布局已经成了第一等公民，然后围绕“谁能直接接住谁”来组织 kernel。

理解了这一点，再去看源码风格为什么差异这么大，就不会觉得突兀。

6.5.4.6联调与性能验证

真正上线时，决定收益的往往不是某一条优化本身，而是多条优化叠加后是否互相打架。比如：

BMZ 开了更激进的 Q 预取后，可能会把 MFMA 需要的寄存器空间挤掉；

NMZ 开了预取后，如果 causal 尾段占比很高，反而可能被 mask 分支抵消掉收益。

因此更稳妥的验证方式，不是“一次性把所有开关全开”，而是做开关扫描：每次只打开一类优化，记录 TFLOPS、占用率和关键计数器变化，再看组合后是否出现负收益。

下面这张表可以作为常见问题的排查入口。

症状	优先排查原因	工具或观察点
实算 TFLOPS 明显低于峰值	MFMA 发射被 LDS 等待打断	rocprof 观察 VALUUtilization、LDSBankConflict
吞吐随 seqlen 异常线性下降	K/V 预取没拼上或局部性失效	核对循环方向、预取步 、rowoffset*
出现 NaN 或不稳定结果	softmax_rescale 或 mask 边界理有误	在 softmax_rescale 前后打印中寄存器/分块统计
占用率低于预期	VGPR 溢出、LDS 超额或 stage 深	反查寄存器数 、spill、Kernel_traits::STAGES
如果要进一步提高排查效率，建按下面顺序做：		

先看是否正确，确认 mask、softmax、边界处理没有问题；

再看是否“等内存”，重点盯 prefetch、L2 命中和 LDS 冲突；

最后再看是否“等调度”，包括寄存器压力、同步位置和优先级窗口。

这个顺序的价值在于，很多看起来像“算力没吃满”的问题，根源并不在 MFMA，而在更早的搬运和控制流。

在实际部署中，两条路径通常会长期并存，并由统一的 launcher 按目标 ISA 选择。若集群以 DCU2 为主，可优先采用 compute_attn_1rowblock_16x64；其 Tile 较小、占用相对保守，在小 batch、短序列场景下更容易维持稳定表现。若集群以 DCU3 为主，且主要服务长序列任务，则更适合优先采用 compute_attn_mha_1rowblock_gfx938，因为 MLS 与布局复用的收益通常会随着序列长度增长而更稳定地兑现。

如果需要同时覆盖两代卡，建议保留两套 Kernel_traits，用 __gfx936__ / __gfx938__ 在编译期区分，再由 launcher 根据设备架构码做运行时分派。两条路径共享同一个 Flash_fwd_params 结构，以及 scale_softmax_log2 这类上层预处理参数，因此上层 Python 接口通常不需要同步重写，迁移成本主要集中在底层 kernel。

6.5.4.7小结

BMZ 与 NMZ 并不是同一条优化路线的前后站，而是两套侧重点不同的 FA 实现范式：

BMZ 强在“通用指令 + 手工调度”，本质是在旧数据通路上尽可能减少等待；

NMZ 强在“MLS + 布局复用”，本质是在新数据通路上尽可能减少搬运与重排。

当我们把这两套思路并排放在一起，就能看清一个更普遍的结论：算子优化并不只是“把某条指令换快一点”，而是要根据硬件能否支持新的数据流组织方式，决定整条 kernel 应该如何生长。这也是后续讨论 MoE、RMSNorm、GEMM + epilogue 等算子时会反复出现的共同主题。

6.6 大模型DCU推理

6.6.1 vLLM推理框架简介

vLLM 是一个面向大规模语言模型（Large Language Models, LLM）的高性能推理框架，旨在为在线服务和离线推理提供高吞吐、低延迟以及高资源利用率的解决方案。随着大模型参数规模不断增长以及推理请求并发度的提升，传统推理框架在 GPU 利用率、内存管理以及调度效率方面逐渐暴露出瓶颈。vLLM 从系统设计层面对这些问题进行了重新建模和优化。

vLLM 的核心思想是围绕推理过程中最关键的两个资源展开优化：计算资源与显存资源。其中，计算资源主要体现在模型前向计算效率，而显存资源主要体现在 KV Cache 的组织与复用。通过引入分页注意力（Paged Attention）、连续批处理（Continuous Batching）以及精细化调度机制，vLLM 在保持模型输出质量不变的前提下显著提升了整体推理性能。

从系统架构角度，vLLM 可以划分为以下几个核心组件：

推理引擎（LLM Engine）

请求调度器（Scheduler）

模型执行层（Model Executor）

内存管理模块（KV Cache Manager）

服务接口层（兼容 OpenAI API）

这些组件协同工作，形成一个完整的高性能推理系统。

6.6.1.1 vLLM适用场景

适用于对延迟敏感且请求动态到达的应用场景，例如：

对话系统

智能客服

AI Agent

编程助手

该类场景通常要求较低的响应时间（如 p95 或 p99 延迟），同时需要支持较高的并发请求数。vLLM 通过连续批处理与调度优化，有效降低平均延迟并提升吞吐。

离线批量推理

适用于吞吐优先的任务，例如：

数据生成（Synthetic Data Generation）

模型评测

Embedding 批量计算

在该类场景中，vLLM 可通过大规模 batch 和 KV cache 复用机制实现接近硬件上限的吞吐能力。

多租户与多模型服务

在多用户或多模型共享资源的环境中：

不同请求具有不同优先级

prompt 长度差异较大

请求到达具有随机性

vLLM 的调度器支持队列管理与优先级控制，可提升系统公平性与资源利用率。

分布式推理与大模型部署

单机多卡部署

多机集群部署

超大模型（数百亿参数以上）

vLLM 支持张量并行（Tensor Parallel）与多进程执行模型，具备良好的扩展能力。

6.6.1.2 核心技术与机制

分页注意力（Paged Attention）

分页注意力是 vLLM 的核心创新之一，其设计灵感来源于操作系统中的虚拟内存分页机制。传统 KV Cache 通常为每个序列分配连续内存，这在动态场景下容易产生严重碎片。vLLM 将 KV Cache 切分为固定大小的 block（页），并通过类似页表的结构进行管理。

主要优势包括：

显著减少内存碎片

支持动态扩展序列长度

提高显存利用率

支持高并发场景

该机制使得 KV Cache 可以在多个请求之间灵活复用与调度。

连续批处理（Continuous Batching）

连续批处理是提升 GPU 利用率的关键机制之一。与传统静态 batching 不同，vLLM 允许在每一个解码 step 动态加入新请求，从而形成“流式 batch”。系统在每一步都会重新构建 batch，使得 GPU 始终处于高负载状态。

其核心特点包括：

动态 batch 构建

新旧请求混合执行

避免等待 batch 填满

该机制显著减少了 GPU 空闲时间，并提升整体吞吐能力。

调度器设计（Scheduler）

vLLM 的调度器负责管理所有请求的生命周期，包括：

请求入队

batch 构建

prefill 与 decode 分配

资源调度

调度器通常维护多个队列，例如 waiting queue 与 running queue，并可根据策略（如先来先服务或优先级调度）进行调度。

此外，调度器还需要在以下维度进行权衡：

延迟与吞吐

长序列与短序列公平性

prefill 与 decode 的资源分配比例

分块预填充（Chunked Prefill）

在处理长 prompt 时，prefill 阶段可能占用大量计算资源，导致其他请求被阻塞。vLLM 通过将 prefill 拆分为多个小块（chunk），实现以下效果：

降低单个请求对系统的占用时间

提高系统响应公平性

减少尾延迟（tail latency）

该机制在混合负载场景中尤为重要。

前缀缓存（Prefix Caching）

前缀缓存用于复用已有的 KV Cache，从而避免重复计算。当多个请求具有相同或相似的前缀时，系统可以直接复用已计算的 KV Cache，而无需重新执行模型前向计算。

该机制适用于：

多轮对话

RAG（检索增强生成）

模板化 prompt

推测解码（Speculative Decoding）

推测解码通过引入辅助模型来提升解码速度。

基本流程如下：

小模型生成一段候选 token（draft）

大模型对这些 token 进行验证

若通过验证则直接接受，否则回退

该方法可以减少大模型的调用次数，从而提高整体生成速度。

CUDA Graph 与算子优化

vLLM 在底层进行了多项 GPU 优化：

使用 CUDA Graph 减少 kernel launch 开销

定制 attention kernel 以适配分页机制

优化 memory access pattern

这些优化可以显著降低系统开销，提高吞吐。

分布式执行架构

vLLM 提供多种执行模式：

单进程执行（适用于单 GPU）

多进程执行（适用于多 GPU）

分布式执行（跨节点）

支持的多个并行策略包括：

Tensor Parallel（张量并行）

Pipeline Parallel（流水线并行）

执行器（Executor）负责协调各个 worker 的计算与通信。

6.6.1.3 安装vllm

使用源码编译方式安装

提供2种环境准备方式:

基于光源pytorch2.9.0基础镜像环境:根据pytorch2.9.0、python、dtk及系统下载对应的镜像版本。

基于现有python环境:安装pytorch2.9.0,pytorch whl包下载目录:https://download.sourcefind.cn:65024/4/main/pytorch,根据python、dtk版本,下载对应pytorch2.5.1的whl包。安装命令如下:

```
pip install torch* (下载的torch的whl包)
pip install setuptools wheel
```

源码编译安装

```
git clone http://10.16.6.30/dcutoolkit/deeplearing/vllm.git # 根据需要的分支进行切换
```

安装依赖:

```
pip install -r requirements/rocm.txt
```

提供2种源码编译方式(进入vllm目录):

```
1. 编译whl包并安装
python setup.py bdist_wheel
cd dist
pip install vllm*

2. 源码编译安装
python3 setup.py install （若调试，可使用python3 setup.py develop）
```

若需要添加git号，设置环境变量: export ADD_GIT_VERSION=1

运行基础环境准备

1、使用上面基于光源pytorch2.9.0基础镜像环境

2、根据pytorch2.9.0、python、dtk及系统下载对应的依赖包:

triton:[https://download.sourcefind.cn:65024/4/main/triton](https://download.sourcefind.cn:650

24/4/main/triton/)

flash_attn:
https://download.sourcefind.cn:65024/4/main/flash_attn

flash_mla:
https://download.sourcefind.cn:65024/4/main/flash_mla

lightop:
https://download.sourcefind.cn:65024/4/main/lightop

lmslim:
https://download.sourcefind.cn:65024/4/main/lmslim

注意事项

若使用 pip install 下载安装过慢,可添加源: -i https://pypi.tuna.tsinghua.edu.cn/simple/

验证

python -c “import vllm;
print(vllm.__version__)”,版本号与官方版本同步,查询该软件的版本号,例如0.15.1;

6.6.2 SGLang推理框架简介

6.6.2.1 大模型推理系统概述

随着大语言模型（Large Language Models,
LLMs）的广泛应用，其推理阶段逐渐成为系统性能与成本的核心瓶颈。与训练阶段相比，推理系统在设计目标和执行模式上存在本质差异。

在训练阶段，系统通常以离线批处理为主，目标是最大化吞吐（Throughput），并尽可能提高硬件利用率。而在推理阶段，系统需要直接面向用户请求，呈现出以下特点：

强实时性要求：用户期望在较短时间内获得响应（低延迟）

请求动态到达：请求到达时间不可预测，难以预先组织 batch

生成过程不规则：自回归生成（autoregressive decoding）导致计算流程逐 token 展开

因此，一个高效的大模型推理系统通常围绕三个核心指标进行设计：

Latency（延迟）：单个请求的响应时间

Throughput（吞吐）：单位时间内处理的请求数量

Cost（成本）：单位请求所消耗的计算资源

从系统结构上看，一个完整的推理系统通常可以抽象为三层：

调度层（Scheduling）：负责请求管理与批处理构建

执行层（Execution）：负责模型前向计算

内存层（Memory）：负责中间状态（尤其是 KV Cache）的管理

这三者之间的协同，是决定推理性能的关键。

6.6.2.2 推理框架的发展与动机

早期的大模型推理通常基于通用深度学习框架（如 PyTorch + Transformers）直接实现。这类方式虽然开发便捷，但在实际部署中存在明显不足：

使用静态 batch，难以适应动态请求

缺乏高效的 KV Cache 管理机制

GPU 利用率较低，资源浪费严重

难以扩展到分布式推理场景

随着需求增长，专门的推理框架逐渐出现，其核心演进方向包括：

动态批处理（Dynamic Batching）：根据实时请求构建 batch

连续批处理（Continuous Batching）：在执行过程中不断插入新请求

KV Cache 优化：减少重复计算，提高生成效率

分布式执行：支持大模型的并行推理

在这一背景下，SGLang 作为新一代推理框架，进一步将关注点从“高效执行”扩展到“可编程推理流程”。

SGLang 的核心定位可以概括为：

一个兼顾高性能与灵活控制能力的大模型推理框架

它不仅关注 GPU 利用率，还强调对复杂推理流程（如多轮对话、工具调用、Agent）的支持能力。

6.6.2.3 SGLang 核心设计理念

SGLang 是一个面向大语言模型与多模态模型的高性能推理框架，目标是在从单卡部署到大规模分布式集群的不

同场景下，均能提供低延迟、高吞吐的推理能力。

在工程实现上，SGLang
集成了多种针对推理场景的优化机制，例如前缀缓存（RadixAttention）、Prefill 与 Decode
阶段分离、连续批处理（Continuous Batching）、分页式注意力（Paged
Attention）、多种并行策略（Tensor / Pipeline / Expert Parallel）、量化（FP8 / INT4 / AWQ /
GPTQ）以及多 LoRA 批处理等。这些机制共同构成了其高性能运行时的基础。 同时，SGLang
具备较强的通用性与可扩展性：一方面兼容主流模型生态（如 Hugging Face 与 OpenAI
API），支持多种语言模型、嵌入模型和多模态模型；另一方面也能够运行在多种硬件平台上，包括主流
GPU、CPU 以及其他加速器。得益于开源社区的持续投入，SGLang 在工业界也得到了广泛应用。

在这些具体能力之上，SGLang
的整体设计遵循了若干关键原则，这些原则从系统层面决定了其性能与扩展能力。

执行与推理逻辑解耦。在传统推理框架中，推理流程往往直接绑定在具体的执行路径上，例如生成逻辑
与 GPU
计算过程紧密耦合。这种方式虽然实现简单，但限制了系统在调度和优化层面的灵活性。SGLang
将“推理逻辑”与“执行过程”进行了明确分离：
推理逻辑关注“要做什么”，例如生成流程、控制结构等
执行引擎关注“如何高效完成”，例如批处理、算子执行等
这种解耦使得系统可以在不改变上层逻辑的情况下，对底层执行策略进行独立优化，例如调整调度策略
或批处理方式。 以动态调度为核心。在实际服务场景中，请求是持续到达且彼此独立的，很难通过静态
batch 的方式高效组织计算。SGLang 将调度系统作为核心组件，通过动态构建和调整
batch，实现资源利用的最大化： 请求进入系统后无需等待固定 batch 形成
调度器根据当前负载与执行状态动态选择请求 batch 在执行过程中可以持续变化
这种以动态调度为核心的执行方式，是实现高吞吐与低延迟平衡的基础。
以内存为中心的设计。在大模型推理过程中，性能瓶颈往往不仅来自计算，还来自内存访问，尤其是在
Decode 阶段，对 KV Cache 的依赖尤为明显。SGLang 在设计上将 KV Cache
视为核心资源进行管理，其目标包括： 避免对历史 token 的重复计算
提高自回归生成过程中的数据复用效率 支持灵活的内存分配与回收 从系统角度来看，KV Cache
的组织方式与访问效率，往往直接决定了推理性能的上限。
扩展的并行设计。为了支持更大规模模型与更高吞吐场景，SGLang
在架构层面即考虑了分布式执行需求，并支持多种并行方式，例如： Tensor Parallel（张量并行）
Pipeline Parallel（流水线并行） Expert Parallel（专家并行，用于 MoE 模型）
这些并行策略使得系统能够在多节点环境下扩展模型规模与处理能力。虽然具体实现涉及复杂的通信与
同步机制，但在设计层面，SGLang 已经为这些能力提供了统一的抽象与接口。

6.6.2.4 SGLang 系统整体架构

从系统视角来看，SGLang
可以抽象为由多个核心模块协同构成的推理执行系统。各模块分工明确，并围绕请求处理流程形成一条
完整的数据通路。整体上，可以将其划分为接口层、调度层、执行层以及内存管理层四个部分。

6.6.2.4.1 接口层（Frontend）

接口层负责接收外部请求，并将其转换为系统内部的统一表示，是系统与用户交互的入口。其主要功能

包括：

提供兼容 OpenAI 的 API 接口 解析请求内容（如 prompt、生成参数等）
支持流式输出（streaming），实现逐 token 返回

该层主要承担协议转换与请求接入的职责，对整体性能影响相对间接，但直接决定了系统的易用性与兼容性。

6.6.2.4.2 调度层（Scheduler）

调度器是 SGLang 的核心组件，负责在动态请求环境中组织计算资源，其主要职责包括：

管理请求队列 动态构建与调整 batch 决定请求的执行顺序 跟踪并管理请求生命周期

在实际运行中，调度器需要在多个目标之间进行权衡，例如：

延迟优先（Latency-oriented）还是吞吐优先（Throughput-oriented） Prefill 阶段与 Decode 阶段的资源分配

调度策略的设计，直接影响系统的整体性能表现，是推理系统中的关键环节。

6.6.2.4.3 执行层（Execution Engine）

执行引擎负责具体的模型计算，是系统中最接近硬件的一层。其主要工作包括：

Transformer 模型的前向计算 Attention 计算 MLP（前馈网络）计算

该层的实现效率决定了单次计算的性能上限，是推理优化的重点关注对象之一。

6.6.2.4.4 内存管理层（KV Cache Manager）

在自回归生成过程中，KV Cache 用于存储历史 token 的 Key 和 Value，是避免重复计算的关键机制。

内存管理层的核心职责包括： KV Cache 的分配与回收 内存布局与组织方式管理
支持不同请求之间的数据隔离与复用

KV Cache 的设计对系统性能具有直接影响，主要体现在： 内存占用规模 内存带宽利用率 Decode 阶段的执行效率

从系统角度来看，KV Cache 的管理效率往往是决定推理性能的重要因素之一。

6.6.2.4.5 整体执行流程

基于上述模块，一个请求在系统中的处理流程可以抽象如下：

```
Client Request
```

```
    ↓
Frontend（接口层）
    ↓
Scheduler（调度层）
    ↓
Execution Engine（执行层）
    ↓
KV Cache（内存管理）
    ↓
Response（流式返回）
```

整个流程体现了 SGLang 的基本执行路径：

请求首先被解析并进入调度系统，随后由执行引擎完成计算，同时依赖 KV Cache
维护中间状态，最终以流式形式返回结果。

6.6.2.5 请求生命周期（Request Lifecycle）

为了更直观地理解系统的运行方式，我们从一个请求的视角，分析其在 SGLang 中的完整生命周期。

6.6.2.5.1 阶段 1：请求进入系统

用户请求通过 API 接口进入系统，包括：

输入 prompt 生成参数（max tokens、temperature 等）
请求被封装为内部数据结构，并加入调度队列。

6.6.2.5.2 阶段 2：调度与批处理

调度器从请求队列中选择一部分请求，构建 batch：

batch 大小不是固定的 会根据系统负载动态调整 不同请求可能处于不同阶段（prefill / decode）

6.6.2.5.3 阶段 3：Prefill 阶段

在 Prefill 阶段：

模型处理完整输入 prompt 计算所有 token 的 attention 构建对应的 KV Cache

这一阶段的特点是：

计算量大 并行性高

6.6.2.5.4 阶段 4：Decode 阶段

在 Decode 阶段：

每一步生成一个新 token 使用 KV Cache 避免重复计算 持续进行直到生成结束

这一阶段的特点是：

计算量小但频繁 强依赖内存访问

6.6.2.5.5 阶段 5：流式返回结果

生成的 token 会逐步返回给用户：

支持 streaming 输出 用户可以实时看到生成结果

6.6.2.5.6 阶段 6：请求结束与资源释放

当生成完成后：

请求从系统中移除 对应的 KV Cache 被释放 系统资源可用于处理新请求

本章从系统角度介绍了 SGLang 推理框架的核心内容，包括：

推理系统的基本结构与挑战 推理框架的发展背景 SGLang 的设计理念 系统架构与核心模块
请求在系统中的完整执行流程

通过这些内容，读者可以建立对大模型推理系统的整体认知。在此基础上，下一章将进一步探讨：

如何针对推理过程中的计算、内存与通信瓶颈进行系统性优化，从而实现更高性能的大模型推理。

6.6.3 大模型推理优化

6.6.3.1 为什么需要大模型推理优化

在实际部署中，大模型的价值不仅取决于模型能力，也取决于其在真实服务环境中的响应速度、吞吐能力、资源消耗和稳定性。即使模型在离线评测中表现良好，如果推理阶段缺乏系统优化，也难以满足在线服务对低时延、高并发、低成本和稳定运行的要求。

相比训练阶段，推理更直接受业务侧约束影响。用户关注首 token
返回速度、生成流畅度和整体响应时间，平台侧则关注 GPU
利用率、单位请求成本、并发能力和多机扩展效率。随着模型规模增大、上下文长度增长，以及多轮对话、RAG、Agent
等场景普及，推理系统面临的核心问题已不只是“能否完成计算”，而是“能否以足够低的成本稳定提

供服务”。

从系统角度看，大模型推理瓶颈通常由计算、显存、内存带宽、调度策略和通信开销共同决定。因此，推理优化并不是单个算子或 kernel
的局部加速，而是一套覆盖模型执行、缓存管理、批处理调度、并行策略和系统架构的综合工程体系。

因此，在介绍具体优化方法之前，需要先明确大模型推理阶段面临的主要问题，为后续讨论连续批处理、KV Cache 管理、长上下文优化、并行扩展和系统调度等技术建立基础。

6.6.3.1.1 大模型推理面临的核心问题

大模型推理的性能瓶颈主要体现在计算、内存和系统调度等多个层面。综合来看，常见问题可以归纳为以下几个方面。

首 token 时延高（Time To First Token, TTFT）

用户提交请求后，系统需要先完成输入 prompt 的整段前向计算，即 Prefill 阶段，之后才能生成第一个 token。该阶段开销较大，主要原因包括：

需要对整段输入执行完整的前向计算 Attention 计算会随输入长度增长而显著增加 需要完成 KV Cache 的初始化与写入 请求进入系统后还可能经历排队、组 batch、调度等额外开销

因此，TTFT 是用户最先感知到的性能指标之一。即使后续生成速度较快，如果首 token
返回过慢，用户仍会感受到明显的响应延迟。

单 token 解码速度慢（Decode Latency）

Decode 阶段以自回归方式逐 token 生成结果。每一步处理的数据规模较小，但当前 token
依赖之前所有 token
的结果，天然存在串行依赖，难以像训练阶段一样大规模并行。这一阶段的性能问题主要体现在：

每生成一个 token 都需要执行一次完整的解码步骤 每一步计算量不大，但调用频繁，容易产生调度和 kernel launch 开销 计算与显存访问频繁交替，很多场景下更容易受内存带宽限制 小 batch 或不规则 batch 下，GPU 利用率往往较低

因此，Decode 阶段虽然单步计算较轻，但执行次数多，往往成为影响整体生成速度的关键因素。

显存占用高，KV Cache 成为核心压力源

推理系统通常使用 KV Cache 保存 Attention 所需的 Key 和 Value 状态，以避免每步解码重复计算历史 token。KV Cache 是推理加速的基础，但也是主要显存消耗来源之一。KV Cache
的开销通常随着以下因素线性增长：输入和输出的总序列长度，模型层数，Attention head 数量及 head dimension，并发请求数量等。这带来的问题包括：

显存很快被历史上下文占满，限制可同时承载的请求数 长上下文场景下，单请求就可能占用大量显存 大量 KV 读写会增加显存带宽压力 动态申请和释放缓存还可能导致内存碎片化和管理开销

因此，现代推理框架通常会重点优化缓存分页、块管理、复用和迁移机制。

高并发场景下吞吐容易下降

在线服务通常需要同时处理大量请求，而这些请求在长度、到达时间和生成阶段上都可能不同。实际运行中，系统往往需要同时面对以下复杂情况：

不同请求的 prompt 长度差异很大 部分请求处于 Prefill 阶段，部分请求处于 Decode 阶段 batch 大小会随时间动态变化 某些请求很短，某些请求可能持续生成很久

如果调度策略设计不合理，就容易出现以下现象：

GPU 存在明显空转，硬件利用率不高 batch 中请求差异过大，导致执行效率下降 Prefill 和 Decode 相互干扰，拉高整体时延 短请求被长请求阻塞，影响服务公平性 平均吞吐和尾时延同时恶化

因此，高并发推理并不是简单堆叠请求，而是需要精细的批处理和调度机制，在吞吐与时延之间取得平衡。

长上下文推理成本显著增加

随着 32K、64K、128K
甚至更长上下文需求的出现，推理系统需要承受更高的计算和内存压力。但上下文变长并不只是“多存一点 token”这么简单，它会同时放大计算和内存两方面的压力。长上下文带来的挑战主要包括：

Attention 计算随序列长度增长迅速上升 KV Cache 占用显著扩大 历史 token 越多，内存访问成本越高 请求越长，系统中的缓存驻留时间越久，进一步挤压并发空间

因此，长上下文不仅意味着更高成本，也更容易暴露系统瓶颈。如果缺乏专门优化，模型即使支持长上下文，也可能因时延过高或显存不足而难以实际部署。

多机多卡扩展带来新的通信与调度问题

当模型规模超过单卡或单机承载能力，或单机吞吐无法满足业务需求时，系统需要采用分布式推理。分布式推理能够提升能力上限，但也会引入新的通信和调度复杂度。不同并行方式会带来不同类型的问题，例如：

Tensor Parallel（TP）需要频繁进行跨卡通信，容易受到互联带宽和通信延迟限制 Pipeline Parallel（PP）会带来阶段切分、流水调度和气泡开销问题 Expert Parallel（EP）在 MoE 场景下会引入动态路由、负载不均和跨设备 token
分发问题多机部署下还会叠加网络拓扑差异、节点异构和跨机通信抖动等问题

因此，分布式推理并不是简单增加 GPU 就能线性提速，通信与调度往往会成为新的性能瓶颈。

性能、成本与精度之间需要权衡

为了提升效率、降低成本，推理系统通常会引入量化、裁剪、稀疏化、近似计算和推测解码等优化手段。但这些优化并非没有代价。常见的权衡包括：

低比特量化可能降低数值精度，从而影响生成质量 近似计算可能带来输出偏差或不稳定性
激进的优化策略可能提升吞吐，但对某些任务的效果更敏感
不同模型、不同任务、不同输入长度下，优化收益和精度损失并不一致

因此，推理优化的目标并不是单纯追求“最快”，而是在性能、成本、精度和稳定性之间找到合适的平衡点。一个真正可用于生产环境的推理系统，必须能够在不同业务目标下做出合理取舍，而不是只在单一指标上表现突出。

小结

总体来看，大模型推理的挑战来自计算、显存、缓存管理、请求调度和分布式通信等多个环节。不同场景下瓶颈可能不同，但都会直接影响系统响应速度、吞吐能力和部署成本。

6.6.3.1.2 推理优化的主要目标

针对前述问题，大模型推理优化的目标并不是单点提速，而是在真实服务约束下，对时延、吞吐、资源占用和生成质量进行系统性优化。总体来看，推理优化通常围绕以下几个核心目标展开。

降低服务时延

时延优化通常包括两个最关键的指标：

首 token 时延（TTFT） 单 token 解码时延（Decode Latency / Inter-Token Latency）

TTFT 决定用户发起请求后多久能看到模型开始响应，单 token
解码时延则决定生成过程是否流畅。因此，降低时延不仅要让模型尽快开始回答，也要保证后续输出稳定连续，从而提升整体交互体验。

提高系统吞吐

在线服务通常需要在单位时间内处理尽可能多的请求，因此吞吐是推理系统的重要指标。吞吐优化的重点通常包括：

提高 batch 的有效利用率 减少 GPU 空转时间 提升高并发条件下的请求承载能力
降低由于请求长度差异带来的执行浪费

降低显存占用与缓存压力

显存是大模型推理部署中的硬约束，尤其在长上下文和高并发场景下，KV Cache
往往是主要开销来源。因此，推理优化的重要目标之一是尽可能降低显存压力，包括：

减少 KV Cache 的存储开销 优化中间状态与缓存的管理方式 提高单卡可承载的请求数量
为更长上下文和更高并发留出空间

显存占用下降不仅意味着资源成本更低，也直接关系到系统是否能够支撑更复杂的业务场景。

提升硬件利用率

推理性能不仅取决于模型计算量，也取决于系统是否能高效利用底层硬件。很多性能问题并非 GPU
算力不足，而是计算、访存和调度组织不充分。因此，推理优化还需要关注硬件利用率，包括：

提高 GPU 计算单元利用率 减少 kernel 启动和调度开销 优化算子执行效率
改善计算与显存访问之间的匹配关系

支持更长上下文

随着 RAG、多轮对话、代码生成和 Agent
等场景的发展，长上下文能力已成为推理系统的重要需求。因此，推理优化还需要关注如何在可接受的成本下支持更长序列，包括：

降低长序列 Attention 的计算代价 控制长上下文下 KV Cache 的增长速度
减少长序列带来的时延和显存放大效应

支持长上下文并不只是“模型能跑更长”，而是要求系统在长输入条件下仍然具备可用的时延和成本表现。

支持更大模型的高效部署

随着模型规模增大，单卡甚至单机往往无法满足部署需求，推理系统需要通过并行和分布式技术支持多设备协同运行。这包括：

支持多卡和多机协同推理 控制并行带来的通信开销 优化不同并行策略下的执行效率
在扩展模型规模的同时尽量保持性能可接受

在性能、成本与精度之间取得平衡

推理优化的最终目标不是让某一指标达到极致，而是在性能、成本和精度之间取得平衡。通常需要综合考虑以下三个维度：

性能：时延与吞吐是否满足服务要求 成本：算力、显存和部署资源是否可接受
精度：生成质量和任务效果是否保持稳定

例如，某些优化可能提升吞吐但增加时延，低比特量化可能降低成本但影响质量。因此，真实生产系统更关注整体收益最优，而不是单一指标最强。

小结

总体来看，大模型推理优化是一个多目标协同优化问题。不同技术方案虽然侧重点不同，但本质上都围绕时延、吞吐、显存、扩展性和精度之间的关系进行设计与取舍。

6.6.3.2 大模型推理流程与优化切入点

在明确大模型推理面临的核心问题后，还需要进一步分析：这些瓶颈具体出现在哪些执行环节，又应从哪些系统层面进行优化。

从工程实现看，大模型推理并不是单一的模型前向计算，而是一条从请求接入、批处理组织、模型执行到结果返回的完整链路。推理性能也通常不是由某个模块单独决定，而是由调度策略、计算执行、缓存管理、解码算法和分布式通信等多个环节共同影响。

因此，理解推理优化不能只关注单个算子或局部技巧，而需要放在完整推理流程和系统分层框架中分析。本节先概括推理基本流程，再从系统层面归纳主要优化切入点，为后续技术展开提供统一视角。

6.6.3.2.1 推理的基本流程

大模型推理的完整流程可结合第 1.5 节“请求生命周期”理解。

6.6.3.2.2 推理优化的主要切入层次

从系统实现角度看，大模型推理优化可以划分为多个相互关联的层次。不同层次关注的对象不同，有的面向服务组织，有的面向模型执行，有的面向底层算子和显存管理，还有的涉及分布式扩展与模型压缩。本文将其归纳为以下几个主要层次。

服务与调度层（Serving & Scheduling）

该层关注请求如何进入系统、如何组织成 batch，以及如何在 Prefill 和 Decode 等阶段之间进行调度，是实现高吞吐和低时延的基础。这一层典型的优化内容包括：

动态批处理与连续批处理 请求队列管理与调度策略 Prefill / Decode 分离调度 面向 tail latency 的服务策略设计

其核心作用是提高请求组织效率，减少资源空转，并在吞吐、平均时延和尾时延之间取得平衡。

计算图与编译层（Graph & Compilation）

该层关注模型执行图如何被更高效地表达和运行，对整体性能具有重要放大作用。常见的优化方式包括：

静态图或半静态图优化 图融合与算子融合 编译执行与代码生成优化 减少 Python 解释层带来的调度开销

其目标是降低框架层和执行层额外损耗，使计算过程更接近硬件高效执行形式。

算子与 Kernel 层（Operator & Kernel）

该层直接面向 GPU 执行，是影响推理性能最直接的层次。Attention、GEMM、Norm、激活函数和各类 fused ops 的实现效率，都会显著影响最终性能。典型优化包括：

高效 Attention 实现 Kernel 融合 自定义 CUDA / Triton Kernel 针对特定硬件特性的算子实现

其核心目标是减少不必要的访存和 kernel 启动开销，提高单算子执行效率，并尽可能接近硬件性能上限。

KV Cache 与显存管理层（Memory & KV Cache）

该层围绕缓存和显存使用展开，是长上下文和高并发场景下的关键优化方向。尤其在 Decode 阶段，系统性能往往不再主要受算力限制，而更多受 KV Cache 访问和管理效率影响。这一层通常包括：

KV Cache 的布局与分页管理 显存分配与复用策略 中间状态的生命周期管理 减少显存碎片与无效占用

其核心价值是降低显存压力、提高缓存访问效率，并支撑更高并发和更长上下文。

解码策略层（Decoding Strategy）

该层关注模型如何生成 token，偏向推理算法设计，但同样会显著影响性能。典型方法有：

Greedy、Sampling、Beam Search 等基本解码方式 Speculative Decoding 等加速方法 借助 Draft Model 或 Prefix 信息的辅助生成策略

其目标是在尽量不损害生成质量的前提下，提高生成效率，降低平均解码成本。

并行与分布式层（Parallelism & Distributed）

当模型规模或服务负载超出单卡、单机能力时，推理系统需要引入并行和分布式机制。该层关注模型如

何跨设备切分、如何协同执行，以及如何控制通信开销。主要包括：TP，PP，DP，EP，CP 。

其重点不仅是让模型能够运行在多设备上，更是要在扩展模型规模和吞吐能力的同时，降低通信、同步和负载不均带来的损失。

模型压缩与轻量化层（Model Compression）

除了优化系统执行路径，也可以从模型表示本身入手，降低推理所需的计算量和存储开销。

典型方法包括量化、剪枝、蒸馏和结构轻量化改造。这类方法通常可以降低部署门槛和推理成本，但也需要在性能、成本和精度之间进行权衡。

MoE 专项优化层（Mixture-of-Experts）

对于 MoE
模型，除通用推理优化外，还存在与专家路由、负载均衡和跨设备通信相关的专项问题。这些问题通常跨越调度、缓存、并行和通信多个层次，适合作为跨层专题分析。其主要关注点包括：专家路由开销，专家负载均衡，token 分发与聚合效率，All-to-All 通信优化，避免热点专家和负载倾斜 等。

与致密模型相比，MoE 推理更容易受到动态路由和通信模式影响，因此需要专门设计。

小结

从流程看，大模型推理包括请求接入、批处理调度、Prefill 计算、Decode
生成和结果返回等阶段；从系统实现看，推理优化可分为服务调度、图编译、底层算子、显存管理、解码策略、分布式执行、模型压缩和 MoE 专项优化等层次。

后续各类优化方法虽然形式不同，但都可以放在这一框架下理解：有些优化请求组织方式，有些提升算子效率，有些聚焦缓存和显存控制，也有些用于提升分布式扩展能力或降低模型本身成本。这样的分层视角有助于将推理优化理解为面向完整系统链路的协同优化过程，而不是零散技巧的堆叠。

6.6.3.3 调度与服务侧优化

在大模型推理系统中，调度与服务层负责将动态到达的用户请求组织成高效执行的计算任务，并驱动底层执行引擎运行。与训练不同，推理请求具有到达时间不确定、输入长度不同、生成时长变化大、阶段状态持续演化等特点，难以依靠固定批处理和同步执行方式获得高效率。

因此，推理性能不仅取决于模型计算效率，也高度依赖请求如何接入系统、如何组成 batch、如何在 Prefill 与 Decode
之间切换，以及如何在多用户场景下平衡吞吐、时延和公平性。调度与服务侧优化是现代推理框架的基础能力，主要包括连续批处理、异步调度、动态批处理、流式返回和负载均衡等机制。

6.6.3.3.1 连续批处理（Continuous Batching / inflating batch）

传统推理系统通常采用静态批处理：在固定时间窗口内收集请求，组成 batch 后统一执行，只有当前 batch 全部完成后，新请求才能进入下一轮。该方式实现简单，但容易导致请求排队、GPU
空转，以及短请求被长请求拖慢。

连续批处理的核心思想是让 batch
在执行过程中动态变化。生成循环推进时，新请求可以在合适时机加入执行队列，已完成请求则立即移出，batch 的大小和成员会随着 Decode
步骤持续变化。因此，连续批处理本质上是将“批处理”从一次性构造，转变为持续演化的执行过程，也常被称为 in-flight batching。

与静态批处理相比，连续批处理的主要优势包括：

提高吞吐：请求持续流入、完成请求及时移出，使 GPU 更容易保持工作状态。

降低首 token 时延：新请求不必等待完整 batch 构造完成，可以更早进入执行流程。

改善尾延迟：长请求不再容易拖慢整批任务，短请求通常能更快完成。

连续批处理特别适合高并发在线服务、请求长度差异较大的交互式场景、流式生成任务，以及需要兼顾吞吐和响应体验的通用推理服务。

限制：

虽然连续批处理收益显著，但其会显著增加服务层实现复杂度。由于 batch 成员持续变化，系统需要：

更细粒度的请求状态管理 更灵活的 KV Cache 分配与释放机制 更高效的 batch 重组与执行编排能力
更完善的公平性控制，避免部分请求长期得不到执行机会

因此，它通常需要与异步调度、缓存管理和流式返回协同设计。

6.6.3.3.2 异步调度（Asynchronous Scheduling）

如果说连续批处理解决的是“batch
如何动态变化”，那么异步调度解决的是“请求接入、调度决策和模型执行如何解耦”。在高性能推理系统中，请求会持续进入系统，调度器根据当前队列和资源状态不断做出决策，执行引擎则异步推进 Prefill 或 Decode 任务，从而避免被全局同步点阻塞。异步调度通常包括三个相互解耦的部分：

请求接入：持续接收外部请求并写入待调度队列。
调度决策：根据队列状态、资源占用和优先级选择待执行任务。 执行推进：底层执行引擎异步执行 Prefill 或 Decode，并回传状态。

这种解耦机制使推理系统不必等待某个阶段“整体结束”后再进入下一阶段，而是可以持续地在不同请求、不同阶段之间切换和推进。

Prefill / Decode 异步化

异步调度的重要动机之一是 Prefill 与 Decode 的计算特性不同。Prefill 更偏计算密集，适合较大 batch 和高效算子；Decode
更偏访存密集，单步计算较小但需要频繁迭代。如果两者完全混在同步流程中，Prefill 可能阻塞 Decode，Decode 也可能持续占用资源，导致新请求迟迟无法进入 Prefill，进而拉高 TTFT。

因此，很多推理系统会在调度层面对 Prefill 和 Decode 进行一定程度的分离，例如：

为 Prefill 和 Decode 维护不同的待执行队列

允许调度器根据负载状态动态选择优先推进哪一类任务

在资源允许时，对两类任务进行交替推进甚至分离部署

其目标是减少阶段间阻塞，使请求既能尽快开始响应，也能持续稳定生成。

调度队列与策略设计

异步调度依赖明确的队列结构和策略设计，包括：

按阶段拆分的多队列结构 按优先级组织的请求队列 面向交互式场景的低时延优先策略
面向离线批量任务的吞吐优先策略

其本质是一个多目标平衡问题，需要同时考虑：

公平性：避免某些请求长期得不到执行机会 时延敏感性：保障交互式请求的响应体验
吞吐最大化：尽量让硬件始终保持高效工作状态 稳定性：避免系统在高负载下出现抖动或极端尾延迟

总体来看，异步调度使推理系统从“分阶段、整批次、同步推进”转变为“持续接入、持续决策、持续执行”的动态系统，是连续批处理、流式返回和负载均衡等机制的重要基础。

6.6.3.3.3 动态批处理（Dynamic Batching）

连续批处理强调 batch 可以在执行过程中变化，而动态批处理更关注系统应如何组织请求，使 batch 执行效率尽可能高。

真实服务中的请求往往差异明显：有的 prompt 很短，有的上下文很长；有的只生成少量 token，有的持续输出较长内容。由于输入和输出长度都不确定，固定组 batch 方式很难适应所有请求。因此，动态批处理的目标是在保证执行效率的同时，将不同特征的请求高效组织到同一执行流程中。

不同长度请求的批处理策略

首先，动态批处理需要处理请求长度差异。如果长度差异过大的请求直接放入同一 batch，短请求可能被长请求拖慢，同时 padding 会降低有效计算比例。常见策略包括：

按阶段分组：将 Prefill 请求和 Decode 请求分开组织，避免不同计算模式互相干扰
按长度分桶（Length-based Bucketing）：将长度相近的请求划入同一批次，减少不必要的对齐浪费
最小化 padding：尽量控制 batch 内部长度差异，提高有效计算比例

Batch 的重组能力

其次，动态批处理要求系统具备 batch 重组能力。在线请求持续到达，batch 需要在调度过程中不断调整，例如新请求并入、完成请求移出，必要时进行拆分或重排。

为了支持这种重组，系统通常需要额外维护一系列与 batch 一致性相关的信息，例如：

token 的位置和对齐关系 请求到 KV Cache 的映射关系 batch 内部样本顺序与状态索引
不同请求所处的生命周期阶段

吞吐与公平性的平衡

此外，动态批处理不能单纯追求 batch 越大越好。更大 batch
通常有利于吞吐，但可能使短请求等待更久，或让长请求持续占用资源。因此，系统需要在吞吐和公平性之间平衡，常见做法包括：

限制最大 batch size，防止单批过大带来调度僵化 引入 aging
机制，使等待时间较长的请求逐步提升优先级 区分交互式请求与离线请求，采用不同调度目标
针对时延敏感请求设置更积极的准入策略

总体来看，动态批处理能够在不规则请求流下提升 batch
利用率和资源效率，是高并发推理系统提升吞吐、减少空转的重要手段。但它也会增加请求分组、batch 重组、缓存映射和公平性控制的复杂度，需要与连续批处理、异步调度、KV Cache
管理和流式返回协同设计。

6.6.3.3.4 流式返回优化（Streaming Optimization）

在在线服务中，用户通常不希望等待模型完整生成后再看到结果，而是希望生成过程中持续获得输出。流式返回即在 Decode 阶段每生成一个或一组 token，就尽快发送给客户端展示。

流式返回不会减少模型总计算量，但能够显著改善用户感知时延，是聊天、代码生成、问答助手等交互式场景中的关键机制。其核心价值是将“等待完整结果”转化为“尽快看到部分结果”，让用户更早感知系统响应，并在生成过程中获得连续反馈。

流式体验主要受两个指标影响：

首 token 时延（TTFT）：决定用户多久能看到模型开始响应。
输出间隔的稳定性：决定生成过程是否自然流畅。

因此，流式优化不仅要更早返回首 token，也要让后续 token 稳定输出。系统通常需要尽快完成 Prefill、优化 Decode 链路、减少调度波动，并控制服务端缓存和发送策略，避免不必要等待。

流式返回还需要服务端与客户端协同。服务端需要决定何时写入发送缓冲区、按单 token 还是多 token flush、如何平衡实时性与网络开销；客户端则需要实时接收增量内容、处理网络抖动、保证内容拼接正确，并在异常中断时支持恢复或重试。

因此，流式优化不是简单地“生成一个 token
就立即发送”，而是需要在时延、稳定性、带宽开销和内容一致性之间权衡。它主要改善用户感知层面的响应体验，并不能替代底层计算优化。只有与调度、执行和缓存管理协同，才能真正发挥价值。

6.6.3.3.5 负载均衡与多实例调度

生产环境中的大模型推理服务通常不会只运行单个实例，而是会在多个
GPU、节点或模型副本之间共同承载请求。因此，服务层不仅要优化单实例内部调度，还需要从全局视角决定请求应路由到哪个副本、如何避免局部过载，以及如何在流量增长时保持系统稳定。

负载均衡与服务扩展可以看作调度优化在更大范围内的延伸，核心目标是提升系统稳定性、可扩展性和资源利用率。

多副本负载分发

在多副本部署中，常见负载分发策略包括：

轮询（Round Robin）：按顺序分发请求，实现简单，适合负载均匀场景。

最少连接（Least
Connections）：优先分发给活动连接数较少的副本，适合请求生命周期差异较大的场景。

基于负载的调度（Load-aware Scheduling）：综合 GPU
利用率、显存占用、队列长度和活跃请求数进行分发，更适合大模型推理这类动态负载场景。

对于大模型服务，简单轮询往往难以反映真实负载状态。短请求和长上下文生成请求在显存、执行时间和 KV Cache 占用上差异很大，因此生产系统通常更倾向于采用运行时感知的负载均衡策略。

热点模型与热点请求治理

生产环境还需要治理热点模型和热点请求。某些热门模型、长上下文请求、多轮长输出请求或复杂推理任务，容易导致局部实例过载。常见措施包括请求限流、热点隔离和分级调度，以避免少数高负载请求破坏整体服务质量。

服务层扩展策略

随着请求量增长，系统还需要服务层扩展能力。常见方式包括：

横向扩展（Scale-out）：增加推理副本或计算节点，提升整体承载能力。
模型分片部署：将超大模型切分到多设备或多节点上运行。 多级调度架构（Gateway +
Worker）：由入口网关负责全局路由，后端 worker 执行具体推理任务。

其中，横向扩展解决服务容量问题，模型分片解决单模型规模问题，多级调度架构提升平台化治理能力。实际系统中，这些方式通常组合使用。

负载均衡与服务扩展的价值在于，将推理优化从“单实例效率”提升为“全局服务能力”。但它也面临负载状态变化快、请求资源差异大、实例异构、故障恢复复杂等挑战。若缺乏合理路由和治理机制，扩容可能只是将局部瓶颈扩散到更大范围。

小结

从服务视角看，大模型推理优化并不只是底层算子加速，更关键的是如何将动态到达、特征各异的请求组织成高效、稳定、可扩展的服务流程。

连续批处理、异步调度、动态批处理、流式返回和负载均衡分别从请求组织、执行推进、结果呈现和全局分发等角度构成服务层优化的核心内容。这些机制并非彼此独立：连续批处理依赖异步调度持续推进执行，动态批处理决定 batch
组织效率，流式返回影响用户体验，负载均衡与扩展决定系统能否在更大规模下稳定运行。

因此，对于现代推理框架而言，服务层优化已经不只是外围逻辑，而是决定整体性能上限的重要组成部分。

6.6.3.4 KV Cache 与显存优化

在大模型推理中，KV Cache 是连接 Prefill 与 Decode 阶段的核心数据结构。它保存历史 token 在各层

Attention 中的 Key 和 Value，使模型在自回归生成时无需重复计算历史上下文，从而显著降低 Decode 阶段的计算开销。

但 KV Cache
也带来了显著的显存和访存压力。随着模型层数、上下文长度、并发请求数和输出长度增加，KV Cache 往往会快速占用大量显存，并在 Decode
阶段产生频繁读取。因此，推理瓶颈常常不再只是算力，而是转向 KV Cache
的存储规模、访问模式和管理效率。尤其在长上下文、高并发和流式生成场景下，KV Cache
已成为影响吞吐、时延和资源利用率的关键因素。

6.6.3.4.1 KV Cache 机制概述

KV Cache 的基本作用

在 Transformer 中，每层 Attention 计算当前 token 输出时，都需要访问历史 token 的 Key 和 Value。如果每生成一个新 token 都重新计算完整历史序列的 K/V，重复计算量会随生成过程快速增长。

KV Cache 的作用是缓存历史 token 已生成的 Key 和 Value，使 Decode 阶段只需计算当前 token 的新 K/V，并与历史缓存共同参与 Attention
计算。其本质是一种“以空间换时间”的机制：通过增加显存占用，换取更高的自回归生成效率。

KV Cache 在 Prefill 与 Decode 中的不同角色

在 Prefill 阶段，模型对输入 prompt 执行完整前向计算，并将每个 token 对应的 Key 和 Value 写入缓存，因此该阶段以 KV 生成和写入为主。

在 Decode 阶段，模型每次生成一个新 token，主要从已有 KV Cache 中读取历史信息，并追加当前 token 的新 K/V，因此该阶段以高频读取和追加为主。

因此，Prefill 更关注大批量写入效率，Decode 更关注高频读取效率和显存带宽利用率。KV Cache 优化不仅要解决“如何存”，还要解决“如何高效读”和“如何持续追加”。

KV Cache 的主要开销来源

在实际系统中，KV Cache 带来的压力主要体现在以下几个方面。

显存占用：每层都需要为每个历史 token 保存 Key 和 Value，缓存规模会随模型层数、序列长度、head 数量、head dimension、并发请求数和输出长度增长。在长上下文和高并发场景下，KV Cache 往往成为显存消耗主体，直接限制单卡可承载请求数。

显存带宽压力：Decode 阶段每一步新增计算量不大，但需要频繁读取历史 KV。上下文越长，单步 Decode 访问的缓存越多，系统越容易从计算受限转为带宽受限。

访存模式效率：如果 KV Cache
布局不合理，或不同请求的缓存分布离散，就会导致访存不连续、局部性较差，降低带宽利用率。因此，KV Cache 优化既是容量优化，也是内存访问模式优化。

KV Cache 开销示意

单请求 KV Cache 大小可近似表示为：

```
KV Cache Size ≈ num_layers × seq_len × num_heads × head_dim × 2 × dtype_size
```

其中，2 表示同时存储 Key 和 Value。若考虑在线服务中的并发请求，总开销还需要乘以 batch size 或并发请求数：

```
Total KV Cache ≈ concurrency × num_layers × seq_len × num_heads × head_dim × 2 × dtype_size
```

因此，KV Cache
压力由模型规模、上下文长度、生成长度、并发请求数和缓存精度共同决定。其中，“长上下文 × 高并发”通常最容易触发 KV Cache 瓶颈。

6.6.3.4.2 共享前缀缓存（Prefix Cache / Radix Cache）

在真实应用中，不同请求常常共享相同输入前缀，例如固定 system
prompt、相似对话历史或模板化工作流。如果这些公共前缀都重复执行 Prefill，会增加计算开销、首 token 时延和 KV Cache 构建成本。

共享前缀缓存的核心思想是：对于已经计算过的相同前缀，后续请求直接复用其 KV
Cache，而不是重新计算。这是一种面向 Prefill 阶段的跨请求缓存复用机制。

基本原理

如果两个请求的前半段输入完全相同，那么该前缀对应的 KV Cache
也是相同的。系统可以直接复用已有缓存，仅对新增部分继续计算。这样可以减少重复 Prefill、重复 KV 写入以及由重复前缀带来的额外 TTFT。

共享前缀的组织方式

为了高效识别和管理公共前缀，系统通常使用 Radix Tree
或类似前缀树结构维护“前缀—缓存”映射关系，因此工程上也常称为 Radix Cache。

典型实现（如 Radix Tree）：

```
    Root
     |
   "You are"
   /   \
" assistant"   " helpful bot"
```

其基本思想是将请求 token 前缀组织成共享路径的树。相同前缀对应公共路径，共享同一段 KV Cache；例如，若多个请求共享前缀 "You
are"，则它们在树中的这一路径可以共用；后续若某个请求继续扩展为 "assistant"，另一个扩展为 "helpful bot"，那么系统只需要分别为分叉后的增量部分构建新缓存，而无需重复处理公共前缀。

从机制上看，不同请求只在分叉点之后继续计算和追加缓存。也就是说，Prefix Cache
是上层优化思想，而 Radix Tree 是支持该机制的一种典型实现结构。

命中率与实际收益

Prefix Cache 的收益取决于业务中的前缀复用程度。如果请求之间存在大量公共前缀，它可以显著减少 Prefill 开销；如果请求差异很大，则命中率和收益都会下降。它通常适合以下场景：

聊天系统：大量请求共享 system prompt 和对话模板。 Agent 框架：系统指令、工具调用模板较固定。 RAG 场景：提示词、检索模板和公共指令前缀较常见。

其主要收益包括减少重复前向计算、降低 TTFT、减少相同前缀下的 KV
构建开销，并在高复用场景下提升吞吐。

不过，Prefix Cache
也会增加前缀索引、引用关系、缓存有效性和淘汰策略的管理成本。在多租户或复杂会话场景中，还需要处理隔离性和一致性，避免缓存复用破坏正确性。

6.6.3.4.3 Paged Attention / Paged KV Cache

随着推理系统需要支持更长上下文和更高并发，传统连续式 KV Cache
管理会逐渐暴露问题。在线请求长度动态变化、持续进入和退出，如果始终要求每个请求占用一整块连续显存，容易导致分配困难、显存碎片和容量浪费。

Paged KV Cache 的核心思想是：不再要求单个请求的 KV Cache
存储在连续大块显存中，而是将其拆分为多个固定大小的 page 管理。Attention
计算再通过索引访问这些 page，这类机制通常称为 Paged Attention。

页式管理的基本思想

传统 KV Cache
通常按“请求—序列”分配连续空间。随着序列增长，系统需要预留较大连续区域，或在空间不足时重新分配和搬移。这种方式虽然直观，但在动态服务场景下并不高效。

```
[KV][KV][KV][KV][KV]...
```

分页式 KV Cache 借鉴虚拟内存分页思想，将缓存划分为固定大小
page。一个请求的逻辑上下文仍然连续，但物理存储可以分散在多个 page 中。请求只需要维护逻辑 token 位置到物理 page 的映射关系。这样，系统可以按需分配新
page，而不必一次性准备大块连续显存。

```
Page1 | Page2 | Page3 | Page4
```

相比连续内存管理的优势

连续式管理在请求长度增长、生命周期差异较大时容易出现问题，包括：

难以为新请求找到足够大的连续空间 请求结束后留下不规则空洞，形成显存碎片
请求动态扩容时可能需要重新分配或搬移 长短请求混合时，整体显存利用率下降

而在分页式管理下，则以 page 为粒度分配和回收缓存。请求增长时逐步追加 page，请求结束时按 page 回收资源，因此更适合动态在线负载。

对显存碎片和调度效率的改善

Paged KV Cache 的主要价值包括：

降低对大块连续显存的依赖，减少分配失败风险。 缓解显存碎片，提升离散空闲空间复用能力。
支持动态序列扩展，避免提前分配过多缓存。

当 KV Cache 被分页存储后，Attention 计算不能再假设历史 K/V 物理连续，因此需要根据 page 映射关系读取缓存并完成计算。Paged Attention 可以理解为与 Paged KV Cache 配套的执行机制：Paged KV Cache 解决“如何分页存储”，Paged Attention 解决“如何在分页布局上高效计算”。

总体来看，Paged KV Cache 不直接减少单 token 理论计算量，但能显著提升显存利用率、系统稳定性和可承载并发规模，是现代推理框架中的核心设计之一。

6.6.3.4.4 Chunked Prefill

在长上下文场景下，输入 prompt 可能达到数万甚至数十万 token。如果一次性完成完整 Prefill，会带来较高的瞬时显存占用和计算压力，也可能长时间占用执行资源，阻塞其他请求。

Chunked Prefill 的核心思想是：不再将长输入作为整体一次性 Prefill，而是切分为多个较小 chunk，按块逐步处理并构建 KV Cache。它不改变模型计算逻辑，而是通过调整执行粒度，降低资源峰值并提高调度灵活性。

长输入分块处理

对于超长输入，一次性 Prefill 会带来两个问题：

单次前向计算规模过大，容易推高显存峰值 Prefill 执行时间过长，容易阻塞其他请求，尤其是正在进行的 Decode 请求

Chunked Prefill 将长输入拆成多个片段，例如将 32K token 输入切分为多个 4K 或 8K chunk。系统按顺序逐块执行前向计算，并逐步写入 KV Cache。这样可以平滑资源消耗，避免单个长请求形成过大的瞬时压力。

流程示意

从执行过程上看，Chunked Prefill 可以理解为将原本的单阶段 Prefill 拆成多个连续的小阶段。例如，对于一个长度为 32000 token 的输入，系统可以按如下方式处理：

```
Input: [Token1 ... Token32000]
→ Chunk1 (0-4K)
→ Chunk2 (4K-8K)
→ ...
→ Chunk N：处理剩余部分
```

在每个 chunk 完成后，对应部分的 KV Cache 会被写入并保留，后续 chunk 再在此前上下文基础上继续推进。从逻辑上看，模型最终仍然完成了对整段输入的 Prefill；但从执行角度看，这一过程被拆解成了多个更容易调度和管理的子步骤。

对显存与调度的影响

Chunked Prefill 的直接收益是降低长输入 Prefill 的显存峰值。由于每次只处理一个 chunk，激活和中间状态压力更可控。

同时，它也改善调度行为。若长输入必须一次性完成 Prefill，可能长时间独占

GPU，导致其他请求无法及时执行。分块后，调度器可以在 chunk
之间插入其他任务，缓解长请求对短请求和 Decode 请求的阻塞。

因此，Chunked Prefill 的主要优势通常包括：

降低长输入 Prefill 的显存峰值 减少单次大 Prefill 对系统的冲击 提高调度器在混合负载下的灵活性
缓解长请求对短请求和 Decode 请求的阻塞

从系统层面看，这种机制特别适合在线服务场景，因为它能让长上下文请求更容易与其他请求共存，而不是成为整个系统的“重负载孤岛”。

代价与挑战

Chunked Prefill 会引入更多状态管理和阶段切换逻辑。系统需要知道请求当前处于哪个
chunk、是否可被抢占、何时继续执行下一块。此外，分块会增加任务切换和调度开销。

因此，chunk
粒度需要合理设计：过大则分块效果有限，过小则调度成本上升、执行效率下降。实际系统需要在显存控制、调度灵活性和执行开销之间平衡。

Chunked Prefill 通常与 Paged KV Cache、连续批处理、异步调度和长上下文 Attention
优化配合使用，是长序列推理系统中的关键组成部分。

6.6.3.4.5 KV Cache 量化（KV Cache Quantization）

在长上下文和高并发场景下，即使模型权重已经量化，KV Cache 仍可能占据大量运行时资源。因此，对 KV Cache 进行低精度存储，是降低显存和带宽压力的重要手段。

KV Cache 量化的核心思想是使用更低精度格式存储 Key 和
Value，以降低缓存容量和访问带宽开销。它本质上是在数值精度和资源效率之间做权衡，目标是在尽量保持生成质量稳定的前提下，降低运行成本并提升承载能力。

KV Cache 量化的目标

KV Cache 量化主要有两个目标。

首先是降低显存占用。KV Cache
大小会随层数、序列长度和并发请求数增长，降低存储精度可以压缩每个 token
的缓存体积，从而支持更长上下文或更多并发请求。

其次是降低 Decode 阶段带宽压力。自回归生成每一步都需要读取历史
KV，缓存体积越小，数据搬运量越低，有助于缓解带宽瓶颈。

因此，KV Cache 量化不仅是一种“节省显存”的手段，同时也是一种“缓解访存瓶颈”的方法。

常见量化形式

KV Cache 通常从 FP16/BF16 压缩到 FP8、INT8，甚至更低比特格式。不同实现可以对 Key 和 Value 使用相同精度，也可以采用不同压缩策略。

从量化方式看，工程上通常不会简单地对整块缓存使用统一缩放，而是会结合更细粒度的 scale

设计，例如：

按张量整体量化 按 head 量化 按 token 或按 block 量化 配合单独的 scale 或元数据进行反量化恢复

这些设计的目的，是在压缩率和数值稳定性之间取得平衡。粒度越细，通常越有利于降低量化误差，但也会带来更多元数据管理和实现复杂度。

收益与限制

当 KV Cache 从 16-bit 压缩到 8-bit
时，理论存储体积接近减半；更低比特格式可进一步压缩。在带宽受限场景下，低精度缓存也可能改善 Decode 性能。

但收益并非无条件成立，还取决于底层 kernel
是否支持低精度访问、量化/反量化开销是否足够低、scale
管理是否高效，以及系统瓶颈是否确实在显存带宽。

此外，KV Cache 直接参与 Attention
计算，并会在长上下文和长输出中反复读取，量化误差可能累积放大。不同层、Key/Value、上下文长度和任务类型对误差的敏感性也不同。因此，KV Cache
量化不能只追求最低比特，通常需要结合校准数据、细粒度
scale、敏感层保留高精度、误差补偿或混合精度策略。

6.6.3.4.6 GQA / MQA / MLA：面向 KV 开销的注意力结构优化

除了优化缓存管理和存储格式，也可以从注意力结构本身降低 KV Cache 开销。Attention
结构会直接决定 Key/Value 的存储规模和 Decode
阶段访存模式，因此会影响显存占用、带宽压力和系统吞吐。

GQA、MQA 和 MLA 都可以视为面向 KV
开销的结构性优化方法。它们不仅是模型结构差异，也会直接改变推理系统中的 KV Cache
大小和访问行为。

从 MHA 到 GQA、MQA：基于 KV 共享的压缩思路

在标准的 MHA（Multi-Head Attention） 中，每个 attention head 都拥有独立的 Key 和 Value
表示。这种方式表达能力最完整，但代价也最直接：随着 head 数量增加，KV Cache
的存储量也会相应增大，因此显存占用和 Decode
阶段的访存压力都较高。为了降低这一开销，后续出现了一类基于 KV 共享
的结构优化思路，其中最典型的就是 GQA 和 MQA。

MHA：

MHA 是传统多头注意力的标准形式，其特点是： 每个 query head 对应独立的 K/V
表达能力强，结构最完整 KV Cache 占用最大，Decode 带宽压力也最高 因此，MHA 可以看作后续各类 KV 压缩结构的参照基线。

GQA：

GQA 的核心思想是：多个 query head 共享同一组 Key 和 Value。也就是说，系统不再为每个 query

head 都维护独立的 KV，而是将多个 head 归为一个 group，在组内共享 K/V
表示。这样做的直接结果是： KV Cache 大小明显下降 Decode 阶段需要读取的数据量减少 相比
MQA，通常能更好兼顾性能与精度 因此，GQA
往往被视为一种在表达能力和系统效率之间较平衡的折中方案。

MQA：

MQA 则进一步强化了共享思想：所有 query head 共享同一组 Key 和 Value。与 GQA 相比，MQA 在 KV 压缩上更激进，因此能够带来更明显的缓存和带宽收益。其主要特征是：

KV Cache 占用进一步降低 Decode 阶段访存压力最小
对系统部署最友好，尤其适合长上下文和高并发场景

不过，MQA
的共享程度最高，也意味着其表达能力约束更强。在部分模型和任务中，这种更激进的共享方式可能带来一定质量折损。因此，从实际工程角度看，MQA 通常代表“极致压缩”，而 GQA
更常被视为“更稳妥的折中”。

GQA 与 MQA 的系统意义

从推理系统角度看，GQA 和 MQA 的价值在于通过减少 KV 数量降低运行时缓存成本。其收益包括：

降低 KV Cache 显存占用 减少 Decode 阶段的显存读取流量 提升长上下文场景下的可部署性
提高高并发服务中的请求承载能力

这类结构优化虽然源于模型设计，但收益会直接传导到推理系统，尤其适合显存预算紧张和带宽敏感的场景。

MLA：基于潜在表示的 KV 压缩

MLA 与 GQA/MQA 的思路不同。GQA/MQA 通过共享 K/V 降低副本数量，而 MLA
更接近通过紧凑潜在表示压缩 KV 信息。其核心是对 KV
进行更紧凑的表示，通常通过压缩或低秩形式减少存储和 Decode 阶段的数据搬运。

从系统角度看，MLA
有助于进一步降低缓存规模、缓解长上下文带宽瓶颈，并为更高并发或更长上下文提供空间。但它也对模型实现、推理 kernel 和框架适配提出更高要求。

小结

GQA、MQA 和 MLA 的共同点是从模型结构层面降低 KV
开销，并进一步改善推理系统的资源使用方式。它们可以降低显存占用、缓解 Decode
带宽压力、提升带宽受限场景下的吞吐，并增强长上下文部署能力。

不过，这类优化的最终收益不仅取决于理论压缩率，还取决于模型精度表现、底层 kernel
支持和推理框架适配程度。更准确地说，它们为降低 KV 成本和改善 Decode
效率提供了结构基础，而实际效果依赖模型设计与系统实现的协同。

6.6.3.5 模型压缩与低比特推理

在大模型推理优化中，除了调度、显存管理和算子优化外，还可以从模型表示本身入手，通过压缩参数或降低数值精度来减少存储、访存和计算开销。这类方法更接近模型本身，能够从源头降低推理成本。

其中，量化（Quantization）是当前最主流、工程价值最高的技术之一。其基本思想是将 FP16/BF16 等高精度表示映射为 INT8、INT4、FP8 等低比特格式，从而减少模型权重显存占用，并在硬件和 kernel 支持良好的情况下提升执行效率。

前文讨论的 KV Cache
量化主要面向运行时缓存，目标是降低长序列生成中的显存和带宽压力；本节更关注模型参数和前向计算过程中的低比特表示，包括权重量化、激活量化以及实际部署中的精度与性能权衡。

6.6.3.5.1 Weight 量化

量化的基本思想

权重量化的目标是用更低精度的数值格式近似表示原始高精度权重，从而减少模型存储开销，并降低推理阶段的数据搬运成本。其基本形式可表示为：

$$W \approx s \boxtimes Q$$

其中：

W 表示原始高精度权重 Q 表示量化后的低比特整数或低精度表示 s 表示缩放因子（scale）

也就是说，系统将浮点权重映射到更小的离散数值空间中，再通过 scale
近似恢复参与计算。从工程角度看，权重量化的意义在于模型不再必须以高精度完整存储和传输，从而显著降低显存占用与访存压力。

常见位宽与量化形式

不同量化方法本质上都是用更少 bit
表示权重，但在压缩率、精度保持和实现复杂度之间存在差异。常见低比特形式包括：

INT8：压缩率较高，精度损失通常相对可控，是较成熟的工程方案
INT4：压缩率更高，适合极致节省显存的场景，但对量化误差更敏感
FP8：兼顾一定动态范围与低精度表示，通常更依赖硬件支持
NF4：针对特定分布特性设计的编码形式，常用于更激进的低比特压缩场景

量化方案的关键不仅是“用几 bit 表示”，还包括如何降低精度损失。例如，GPTQ
通过误差补偿减少低比特近似损失，AWQ
通过重要性加权保护关键权重信息，一些方法还会结合逐层或逐组缩放来改善局部误差。

因此，权重量化并不是简单地把 FP16 改成
INT4，而是围绕压缩率、数值恢复质量和执行效率进行综合设计。

显存与速度收益

权重量化的收益主要体现在显存节省和执行效率改善。

首先，模型权重通常是推理部署中的主要静态存储开销。将权重从 FP16 压缩到
INT8，理论上可将存储量降至约一半；压缩到

INT4，则可降至约四分之一。这使系统能够在相同显存预算下部署更大模型，或为 KV Cache 和并发请求留出更多空间。

其次，低比特权重可以减少数据搬运量，降低显存带宽压力。在具备低比特计算支持的硬件上，还可能使用更高效的执行路径。

但量化并不保证线性加速。实际收益取决于硬件是否原生支持对应格式、kernel 是否针对量化格式优化、反量化和 scale 处理是否高效，以及系统瓶颈究竟在算力、带宽还是其他环节。因此，更准确地说，权重量化通常能显著降低显存占用，并在合适实现条件下改善推理性能。

对精度的影响

量化会不可避免地引入近似误差。当高精度权重被映射到低比特离散空间后，原始分布中的细节会被压缩或截断，并进一步影响模型前向计算。在实际模型中，量化误差的影响通常并不均匀。

这种误差在模型中并不均匀：不同层、不同模块对量化误差的敏感性不同；位宽越低，潜在精度损失通常越明显。因此，实际工程中通常不会采用全模型统一的粗粒度量化，而会结合分层量化、分组量化和离线校准等方法，在压缩率与模型精度之间取得平衡。

工程上的关键认识

权重量化不是简单追求 bit 越低越好，而是需要同时考虑显存节省、kernel 成熟度、端到端速度和模型效果。某些极低比特方案虽然理论压缩率更高，但如果 kernel 支持不足、精度波动明显或部署链路复杂，未必适合生产环境。很多时候，稍保守但稳定、内核成熟、精度可控的方案更具工程价值。

6.6.3.5.2 激活量化与 W/A 量化（Activation Quantization & W/A Quantization）

除了权重量化，还可以进一步对前向计算过程中的激活值进行量化。权重量化主要降低模型静态存储成本，而激活量化关注运行时计算过程中的数据表示成本。当系统同时对权重（Weight）和激活（Activation）进行低比特表示时，通常称为 W/A 量化，例如 W8A8、W4A8。

激活量化的基本特点

与权重量化相比，激活量化更具动态性。权重在推理阶段固定，可以提前离线分析量化参数；激活值则由输入动态产生，其分布会随层、token、上下文和 batch 构成变化。这使得激活量化通常比权重量化更困难。激活量化的主要挑战包括：

数据分布动态变化：不同输入下激活范围可能差异很大 scale
难以提前固定：很难像权重那样进行完全静态的离线量化
更容易影响数值稳定性：激活直接参与当前层计算，误差更容易在网络中传播

因此，从工程实践上看，激活量化通常对量化粒度、scale 设计和 kernel 实现提出更高要求。

为什么需要 W/A 量化

仅进行权重量化虽然能降低模型存储开销，但推理性能瓶颈还可能来自运行时激活的读写和计算。如果激活仍保持高精度，系统在实际 GEMM 和算子执行中仍可能受高精度数据搬运和计算路径限制。因此，进一步引入激活量化的意义在于：

降低前向计算过程中的数据搬运成本 让更多算子能够走低比特执行路径 提升端到端低比特推理的一致性 在合适硬件支持下进一步释放低比特计算性能

也就是说，权重量化解决“模型怎么存”，而 W/A 量化进一步关注“模型怎么以低比特方式运行”。

W/A 量化的常见形式

实际系统中，权重和激活不一定采用相同位宽，而是根据精度风险和硬件支持进行组合。常见配置包括：

W8A8：权重和激活都采用 8-bit，是较常见、也较容易落地的低比特配置 W4A8：权重进一步压缩到 4-bit，而激活仍保持 8-bit，兼顾较高压缩率与相对可控的稳定性 更低比特或浮点低精度组合：例如结合 FP8 路径，面向特定硬件进行优化

通常来说，权重可以压得更激进，激活需要更保守；位宽越低，对 kernel、校准和 scale 管理的要求越高。因此，W/A 量化不是固定方案，而是一类围绕压缩率、执行效率和精度稳定性展开的配置空间。

与 KV Cache 量化的关系

激活量化与 KV Cache 量化都属于低精度技术，但作用对象不同。

激活量化 面向前向计算过程中临时产生的中间结果，重点在于降低运行时计算成本 KV Cache 量化 面向 Attention 历史缓存，重点在于降低长序列生成中的显存和带宽开销

KV Cache 更特殊：它不像权重那样静态，也不像普通激活那样短生命周期，而是会随 token 增长，并在 Decode 中长期保存、反复读取。因此，KV Cache 量化更适合放在缓存优化章节中讨论。

工程中的常见组合策略

实际部署中，推理系统通常不会盲目追求全链路最低 bit，而会根据硬件能力、kernel 支持和精度要求选择组合策略。常见路线包括：

权重低比特 + 激活高精度：这是较稳妥、也最常见的工程路线，适合优先降低显存压力 权重低比特 + 激活低比特：进一步降低运行时成本，但对实现和稳定性要求更高 权重低比特 + KV Cache 低精度：适合长上下文和高并发场景，重点缓解缓存压力 统一低精度执行路径：在新一代硬件支持下，尝试实现更完整的低精度推理链路

这些组合之间没有绝对统一的最优解，需要在精度、性能、显存带宽和工程复杂度之间平衡。

6.6.3.5.3 量化带来的工程挑战

从理论上看，模型量化能够降低显存占用、减少数据搬运，并在合适条件下提升推理效率。但实际低比特推理并不是简单地把模型从 FP16 改成 INT8/INT4。真正影响系统效果的，往往是硬件支持、kernel 实现、数据布局、精度稳定性和端到端执行链路是否适配低比特表示。

理论压缩率不等于实际加速比

从 FP16 到 INT8 理论上可减少约 2 倍存储，从 FP16 到 INT4 理论上可减少约 4 倍存储，但位宽压缩率不会自动转化为等比例端到端加速。原因在于，推理性能不仅由权重大小决定，

还受到以下多个因素共同影响：

计算本身是否是主要瓶颈 数据搬运是否占主导 量化后的 kernel 是否足够高效
是否引入了额外的反量化或格式转换开销 端到端流程中是否仍有大量操作保持高精度执行

因此，低比特推理通常能稳定带来显存收益，但速度提升依赖具体实现条件。

硬件与 Kernel 支持决定实际收益

低比特推理是否高效，很大程度上取决于硬件和 kernel
对相应格式的支持。如果硬件不擅长某种低比特计算，或框架没有高效算子，即使模型已量化，也难以获得明显加速。常见问题包括：

某些位宽格式仅在特定硬件上有原生加速能力 不同低比特格式对应的 kernel 成熟度差异较大
部分算子支持低比特，部分算子仍需回退到高精度路径
权重虽为低比特存储，但执行前仍需转换为更高精度参与计算

这意味着，低比特推理的真实效果往往不是“格式决定一切”，而是“格式、硬件和 kernel
三者共同决定”。

量化与反量化开销可能抵消部分收益

低比特权重通常需要结合 scale、zero-point
或其他参数恢复近似值。如果这一过程设计不够高效，就会引入额外开销：

量化参数的加载与管理 反量化操作本身的计算成本 数据格式转换带来的额外访存
中间张量重排、打包或解包开销

在小
batch、短序列或交互式场景中，这些开销更容易被放大。因此，一个低比特方案是否有效，不仅取决于压缩率，还取决于是否实现了完整高效的低比特执行链路，而不只是低比特存储。

不同模块对量化的敏感性并不一致

模型中不同层和模块对量化误差的敏感程度并不一致。某些层在低位宽下仍能保持效果，而另一些层可能迅速成为精度瓶颈。常见情况包括：

某些层对低比特误差更敏感 Attention 与 MLP 模块的容忍度可能不同
输出层、归一化相关模块常常更需要谨慎处理 位宽越低，模块间的不均匀敏感性越明显

因此，实际部署通常会采用混合精度、分层或分组量化，并对关键模块保留更高精度。这说明低比特推理是细粒度的精度—效率协同优化，而不是简单统一压缩。

端到端系统链路未必完全低比特化

即使模型权重已经量化，整个推理系统也未必实现完整低比特执行。常见情况是：

权重是低比特存储，但激活仍为高精度 GEMM 走低比特路径，但部分前后处理算子仍用高精度 KV Cache、采样、后处理等模块仍未量化 框架调度和数据搬运部分没有针对低比特特别优化

因此，模型“被量化”不等于系统实现了“全链路低比特推理”。很多情况下，显存节省明显，但吞吐或时延收益不如预期。

精度评估与线上稳定性验证更复杂

低比特推理的收益必须通过实际任务和线上场景验证，而不能只看压缩比或单点 benchmark。系统需要关注生成质量是否稳定、是否在特定输入分布下退化，以及长上下文、复杂任务或长输出是否放大量化误差。

因此，低比特推理通常需要更严格的验证流程，包括离线任务评测、长上下文稳定性测试、多场景性能对比、线上灰度验证和回退机制。它不仅是模型压缩问题，也是系统上线和质量保障问题。

6.6.3.6 解码路径优化

在大模型推理中，Decode 阶段通常是影响生成速度和交互体验的关键路径。与 Prefill 相比，Decode 每一步计算规模较小，但由于自回归生成存在严格的 token 依赖，系统必须逐 token 推进，难以像训练阶段那样进行大规模并行。

因此，在长文本生成、流式对话和高并发在线服务中，用户体验不仅取决于首 token 返回速度，还取决于后续 token 是否能持续、稳定且快速生成。围绕 Decode 路径的优化，已成为现代推理系统的重要方向。其中，推测解码（Speculative Decoding）通过“候选生成 + 主模型验证”的方式，在尽量保持生成质量的前提下减少主模型逐 token 解码次数，是最具代表性的 Decode 加速技术之一。

6.6.3.6.1 Decode 阶段为何成为瓶颈

自回归生成天然具有串行性

大语言模型通常按自回归方式生成文本。当前位置 token 的预测依赖此前所有已生成 token：

```
P(x_t | x_1, x_2, ..., x_{t-1})
```

这意味着每个新 token 都必须等待前一个 token 生成完成，token 之间存在严格的前后依赖。因此，Decode 无法像训练阶段那样并行计算大量未来位置，时间维度上的并行能力非常有限。

单 token 生成虽小，但仍需经过完整前向路径

在 Decode 阶段，模型每一步通常只生成一个 token，但这并不意味着执行代价可以忽略。虽然当前步只新增一个位置，但模型仍然需要完成一次完整的前向推理过程，其中通常包括：

基于历史 KV Cache 执行 Attention 计算 执行各层 MLP 和归一化等模块 计算 logits，并完成采样或选择下一 token

Decode 更容易受到访存和调度开销影响

Decode 阶段的瓶颈往往不完全来自算力，而更容易来自 KV Cache 访存和高频调度开销。

首先，每一步生成都需要读取历史 KV Cache，随着上下文增长，显存带宽压力持续增加。

其次，由于每次只处理一个新 token，单步计算粒度较小，GPU 难以充分摊薄 kernel

启动和框架调度成本。

因此，Decode
是典型的高频、小粒度执行路径：单步计算不重，但执行次数多、访存占比高、调度频繁，整体容易成为系统瓶颈。

Decode 为何会成为系统级瓶颈

Decode
阶段的瓶颈可以概括为：不是“每一步算得太多”，而是“每一步算得太碎，并且必须串行重复很多次”。其主要特征包括：

严格串行：难以做时间维度上的大规模并行 重复次数多：生成越长，重复执行次数越多
单步粒度小：GPU 计算资源难以充分利用 访存敏感：高度依赖对历史 KV Cache 的频繁读取
调度频繁：小步迭代使框架和 kernel 开销更显著

因此，Decode 优化的核心思路通常不是单纯压缩单步计算，而是减少主模型逐 token
执行次数，提高单次 Decode 的有效工作量，并降低 KV Cache 访问与小粒度调度带来的开销。

6.6.3.6.2 推测解码（Speculative Decoding）

推测解码的目标是缓解 Decode 阶段逐 token
串行推进的瓶颈。其核心思想是：先由更便宜的机制提前生成多个候选
token，再由主模型进行验证，从而减少主模型逐步解码的次数。这一方法常被概括为“先猜再验证”。

从系统角度看，推测解码并不改变主模型作为最终判定者的角色，而是在保持输出一致性的前提下，让多个 token 的生成和验证以更高效的方式完成。

基本原理

推测解码的核心思想是：

用一个更快的草稿生成器一次性提出多个候选 token，再由原始主模型对这些候选进行验证和接受。

标准自回归解码中，主模型每一步只能生成一个 token。而在推测解码中，流程通常为：

由草稿模型根据当前上下文一次生成多个候选 token

主模型对这段候选序列进行验证

接收通过验证的最长前缀

对未通过的部分回退，并由主模型继续生成或重新进入下一轮推测

从执行视角看，这相当于将“主模型一步生成一个
token”，改为“草稿模型先试探多步，主模型再批量确认”。如果草稿候选足够接近主模型输出，就可以一次接受多个 token，从而减少主模型 Decode 轮数。

草稿模型与主模型的协同机制

推测解码涉及两个模型：

Draft Model（草稿模型）：负责快速生成候选 token
序列，通常规模更小、执行更快，不负责最终决策。

Main Model（主模型）：负责验证候选序列，决定哪些 token
可被接受，并在候选不匹配时接管后续生成。

因此，推测解码的本质不是“用小模型替代大模型”，而是“让小模型负责试探，大模型负责确认”。

验证、回退与接收率

草稿模型生成长度为 k
的候选序列后，主模型会验证其中能够接受的最长连续前缀。一旦某个位置不匹配，系统从该处分叉，由主模型重新接管后续生成。因此，推测解码并不是整段候选“全对或全错”，而是尽可能接受前缀，并回退未通过部分。

其收益高度依赖接收率（Acceptance Rate）。接收率越高，每轮推测能接受的 token
越多，主模型回退和重新生成的次数越少，加速越明显；接收率越低，草稿模型带来的额外计算越难被抵消。因此，推测解码的效果不仅取决于草稿模型有多快，更取决于草稿候选是否足够接近主模型。

适用场景与局限

推测解码通常更适合以下场景：

长文本生成：生成步数多，更容易摊薄额外验证成本 高吞吐推理服务：主模型 Decode
成本高，加速空间更明显 对响应速度敏感的交互式应用：希望提升 token 输出速度
主模型较大、草稿模型明显更便宜的部署环境

虽然推测解码具有明显潜力，但它并不是没有代价。

首先，草稿模型本身会引入额外计算。如果草稿生成质量不高，那么这部分额外开销可能无法通过主模型加速被充分抵消。
其次，接收率通常不是稳定常数。它可能受到任务类型、输入分布、生成阶段以及草稿模型与主模型匹配程度的影响，因此实际收益往往会随场景变化。
再次，推测解码的系统实现复杂度明显高于普通自回归解码。系统不仅要同时维护草稿生成和主模型验证两条路径，还需要处理前缀接收、回退、状态同步以及缓存管理等问题。

因此，推测解码虽然是非常有代表性的 Decode
加速方向，但其落地效果高度依赖模型组合、系统实现和实际业务场景。

6.6.3.6.3 MTP / Draft 机制与实现变体

推测解码是“先猜再验证”的通用框架，但实际实现并不唯一。不同方案的差异主要在于候选 token 由谁生成、验证路径如何组织，以及额外开销如何控制。外部 Draft Model、Self-Speculative 和 MTP 都可以看作这一目标下的不同实现变体。

外部 Draft Model：最典型的推测解码形式

最经典的推测解码实现，是引入一个独立的 Draft
Model。该模型通常规模更小、运行更快，专门负责为主模型生成候选 token
序列。这一方案的基本特点是：

草稿路径与主模型路径明确分离 小模型负责快速生成候选 大模型负责最终验证与接收
两个模型之间通常具有相同或兼容的 tokenizer 与输出空间

这种方案职责清晰、实现直观，只要小模型足够快且与主模型输出一致性较高，就可能获得明显加速。但代价是系统需要维护两套模型执行链路，包括模型加载、显存占用、缓存管理，以及推测与验证之间的状态同步。因此，它更适合资源较充足、主模型较大、草稿模型开销容易被摊薄的场景。

Self-Speculative：在主模型内部产生草稿

除了额外引入一个独立小模型之外，另一类思路是让主模型自身承担草稿生成能力，也就是常说的 Self-Speculative 或“自推测”机制。这类方法的基本出发点是：
既然额外引入一个草稿模型会增加系统复杂度，那么是否可以直接利用主模型内部已有的表示或中间层信息，提前生成候选
token，再由更完整的主路径进行验证？从系统实现上看，这类方法通常具有以下特点：

不需要额外加载一个独立小模型 草稿能力来自主模型自身的轻量路径或局部结构
更容易减少双模型协同带来的工程复杂度 草稿质量与主模型天然具有更强相关性

它的优势是部署链路更统一，可以减少双模型协同带来的
tokenizer、缓存状态和输出分布不匹配问题。其关键在于内部草稿路径是否足够便宜；如果轻量路径本身代价较高，则整体加速效果可能有限。

MTP：多 token 预测作为草稿机制

MTP（Multi-Token Prediction）
可以理解为与推测解码密切相关的一类实现思路。它的核心想法不是用单步只预测下一个 token 的方式推进，而是让模型在一次前向中直接预测未来多个位置的 token 候选。从这一角度看，MTP 可以被视为一种“让模型自身具备多 token 草稿生成能力”的机制。它与传统独立 Draft Model 的区别在于：

候选 token 不一定来自单独的小模型 候选可能由主模型附加的多 token 预测头或相关结构直接产生
更强调“一次前向预测多个未来位置”的能力

因此，MTP
并不一定等同于推测解码本身，但它可以很好地作为推测解码中的候选生成机制。更准确地说，推测解码是更上层的加速框架，而 MTP 是其中一种重要的草稿生成路线。

从系统角度看，MTP
可以减少外部小模型带来的部署负担，使候选生成与主模型结构更紧密耦合，并可能提高候选与主模型最终结果的一致性。但它依赖模型本身是否支持多 token
预测结构，以及推理框架能否高效利用这一能力。

不同实现路线的主要权衡

不同 Draft 机制本质上都在解决同一个问题：如何以尽可能低的额外成本，提前提出足够高质量的候选 token。主要权衡包括：

首先是额外计算开销。外部 Draft Model 需要额外运行一个模型；Self-Speculative 和 MTP 则在主模型内部生成候选，但未必一定更便宜。

其次是工程复杂度。双模型方案更直观，但需要维护两套执行路径；内部草稿方案链路更统一，但对模型结构、推理图和 kernel 支持要求更高。

再次是候选质量与接收率。候选越接近主模型输出，推测解码越容易获得收益；候选质量差时，额外机制可能变成负担。

因此，这几类方法没有绝对优劣，实际系统需要结合模型规模、硬件资源、部署复杂度和业务目标选择。

从推理系统视角看这些变体

从推理框架角度看，无论采用哪种草稿机制，都需要解决候选生成与主模型验证的组织、草稿路径与主路径状态一致性、接收和回退时的 KV Cache
更新、额外草稿开销评估，以及与连续批处理、流式返回和调度系统的兼容问题。

因此，MTP 和 Draft 机制不应被看作孤立的模型技巧，而是 Decode
路径优化在系统侧的重要组成部分。其真正价值不仅取决于模型能否预测多个
token，还取决于推理系统能否将这种能力转化为端到端加速收益。

6.6.3.7 并行与分布式推理优化

随着模型参数规模、上下文长度和在线服务负载持续增长，单卡甚至单机往往难以满足实际部署需求。此时，推理系统需要引入并行与分布式机制，将模型计算、参数存储、KV Cache
或请求负载分散到多个设备上协同完成。

与训练不同，推理并行的目标不仅是“把模型跑起来”，还要在时延、吞吐、显存约束和通信开销之间取得平衡。训练通常可以通过大 batch 和反向传播摊薄通信成本，而推理阶段尤其是 Decode
阶段，具有小粒度、高频率和强串行依赖特征，更容易暴露通信与同步开销。

总体来看，大模型推理中的并行与分布式优化，主要围绕以下几个问题展开：

如何让更大模型突破单卡显存限制 如何在多设备条件下提升整体吞吐
如何控制并行带来的通信与同步开销 如何让长上下文和高并发场景在分布式环境下仍保持可用性能

围绕这些目标，现代推理系统通常会结合 Tensor Parallel、Pipeline Parallel、Data Parallel、Expert Parallel、Context Parallel 等策略，并进一步通过 Prefill-Decode 分离优化不同阶段的资源配置。

6.6.3.7.1 为什么推理需要并行与分布式

在小模型或低并发场景下，单卡推理通常已经足够。但当模型参数超过单卡显存、KV Cache
占用过高、单机吞吐不足、单设备无法满足时延要求，或
MoE、长上下文等场景需要跨设备协同时，系统就需要并行与分布式能力。

从系统角度看，推理走向分布式通常有两类根本动因。

第一类是容量问题，即模型或缓存太大，单卡无法容纳。此时并行首先解决的是“能不能部署”。

第二类是性能问题，即模型虽然能运行，但需要多卡或多机提升吞吐、降低时延。此时并行关注的是“如何更高效服务”。

但并行并不天然意味着更快。尤其在 Decode

阶段，如果每一步计算很小却需要频繁跨设备通信，通信和同步可能反而成为瓶颈。因此，推理并行优化的关键不是简单增加设备数量，而是合理选择并行粒度和协同方式。

6.6.3.7.2 张量并行（Tensor Parallel）

Tensor Parallel
是大模型推理中常见的模型并行方式。它将同一层中的参数矩阵和计算任务按张量维度切分到多个设备上，由多个 GPU 共同完成一次前向计算。

在大模型中，线性层和 Attention 投影通常占据大量参数和计算开销。TP
可以将这些大矩阵按列或按行切分，从而分担单层的存储和计算压力。其主要价值包括：

让单层参数规模突破单卡显存限制 提升单层大矩阵计算的并行能力 支持更大尺寸的致密模型部署

TP 的优势是不会改变模型层次结构，也不需要像 PP
那样重新组织层间流水，因此较容易与现有模型实现结合。但其代价是设备之间需要频繁交换中间结果，常见通信包括 all-reduce、all-gather 和 reduce-scatter。

因此，TP 的收益高度依赖设备互联带宽和通信延迟。Prefill
阶段计算量较大，通信成本更容易被摊薄；Decode 阶段每步计算粒度较小，TP
通信更容易成为瓶颈。总体来看，TP 适合单层参数规模大、单机 GPU
互联较快、需要部署大尺寸致密模型，并能接受一定跨卡通信开销的场景。

6.6.3.7.3 流水线并行（Pipeline Parallel）

Pipeline Parallel 的核心思想是将模型不同层划分到不同设备上，前向计算按层级顺序跨设备传递。相比 TP 是“切开一层”，PP 是“纵向切开模型”。PP
的直接价值是解决模型整体过深、参数总量过大导致单卡或单机无法容纳的问题。每个设备只保存部分层，从而自然分摊显存压力。

但从执行角度看，PP 存在明显的层间依赖。请求必须依次流过多个
stage，单请求很难充分利用流水并行；Decode 阶段每步粒度更小，流水气泡更明显；如果 stage
划分不均，还会导致负载不平衡。训练中通常可以依靠大量 micro-batch
填充流水，但推理尤其是交互式 Decode 阶段，未必总能构造足够多的并发工作。因此，PP
在推理中的收益通常更依赖：

请求并发是否足够高 Prefill 与 Decode 是否可以共同填充流水 各 stage 是否划分均衡
是否存在明显跨机传输延迟

总体来看，PP
更适合模型整体较深、单机难以容纳完整网络，且系统更关注部署能力和总体吞吐，而非极致单请求时延的场景。

6.6.3.7.4 数据并行（Data Parallel）

推理中的数据并行与训练不同。训练 DP 需要多个副本处理不同 batch

后同步梯度；推理没有反向传播和参数更新，因此更常见形式是副本并行（Replica Parallel），即部署多个完整模型副本，让不同请求分别落到不同副本上执行。这种方式的核心价值不在于单请求加速，而在于：

提升整体并发承载能力 提升系统吞吐 增强服务可用性与弹性扩展能力

这种方式实现简单，各副本之间几乎不需要频繁同步执行状态，也适合与服务侧负载均衡结合，通过路由系统将请求分发到不同副本。不过，这种方式的限制也很清楚：

每个副本都需要一整套完整模型参数 无法解决单模型本身放不下的问题 对显存总量要求较高 更偏向横向扩展，而非模型切分

因此，它更适合解决吞吐扩展，而不是超大模型部署。实际系统通常会先用 TP、PP 等模型并行让单个副本能够运行，再通过副本并行提升整体服务能力。

6.6.3.7.5 上下文并行（Context Parallel）

随着长上下文需求增长，KV Cache 和上下文状态也成为分布式优化对象。Context Parallel 的核心思想是将长序列上下文及其计算、缓存状态分散到多个设备上处理。CP 不像 TP 那样切分单层矩阵，也不像 PP 那样切分网络层，而是面向长序列本身进行并行化和负载分摊。其主要意义包括：

降低超长上下文带来的单设备 KV Cache 压力 分摊长序列 Attention 的计算和访存成本

当上下文长度达到 32K、128K 甚至更高时，系统瓶颈往往不只是模型参数，而是序列本身带来的显存和带宽压力。因此，CP 特别适合长上下文推理系统。

但 CP 实现复杂。Attention 仍然依赖全局上下文信息，系统需要在“序列被切分”和“模型仍需看到完整上下文”之间建立高效协同机制，这会引入跨设备通信、序列重组和缓存映射问题。尤其在 Decode 阶段，如果历史 KV 分散在多个设备上，如何高效读取与组织是关键难点。因此，CP 更适用于：

长上下文需求非常强的推理场景 KV Cache 已经成为系统主要瓶颈 系统愿意通过更复杂的跨设备管理换取上下文能力扩展

6.6.3.7.6 专家并行（Expert Parallel / EP）

Expert Parallel 是 MoE 模型推理中的关键并行方式。在 MoE 中，token 不会经过完全相同的参数路径，而是由路由器选择若干专家参与计算。因此，系统可以将不同专家分布到不同设备上，让多个 GPU 分担专家参数和计算开销。从结构上看，EP 的主要意义在于：

支持更大规模的专家集合部署 分散单设备上的专家参数存储压力 让 MoE 模型的稀疏激活特性真正转化为系统收益

但与致密模型的 TP/PP 不同，EP 的核心挑战不仅是“专家参数如何分布”，还包括“token 如何路由”。由于不同 token 可能被分配到不同专家，系统需要进行跨设备 token 分发与聚合，通常会带来明显的 all-to-all 通信开销。因此，EP 在推理中的关键问题通常包括：

专家路由是否均衡 热点专家是否导致部分设备过载 token 分发与回收的通信代价是否可控 Decode 阶段的小粒度请求是否会放大 all-to-all 开销

总体来看，EP 是与路由机制、负载均衡和通信优化高度耦合的分布式执行方案，主要适用于 MoE 模型、专家数量较多、单机难以容纳专家集合，且稀疏激活能够降低单 token 实际计算量的场景。

6.6.3.7.7 Prefill-Decode 分离（PD Disaggregation）与并行策略组合

在大模型推理中，Prefill 与 Decode
虽然属于同一请求流程，但二者在计算特性、资源需求和最优并行方式上差异明显。

Prefill 需要对完整 prompt 执行前向计算，更偏计算密集型，容易利用大
batch、强张量并行和高效算子。Decode 每步只生成少量
token，更偏小粒度、高频率和强串行执行，对 KV Cache 访问、显存带宽和调度开销更敏感。

因此，Prefill 和 Decode 未必适合绑定在同一组设备、使用同一套并行配置。Prefill-Decode 分离（PD Disaggregation）的核心思想是将二者视为不同工作负载，分别部署在不同资源池或并行拓扑上，再通过 KV Cache 传输、状态同步或中间结果交接连接起来。

PD 分离的基本思路

PD 分离的核心思想是：将 Prefill 与 Decode 作为两类不同服务阶段分别部署，由 Prefill
节点负责快速完成长输入前向和初始 KV 构建，再将相关状态交给 Decode
节点继续执行后续生成。从逻辑上看，请求链路会被拆成两段：

请求首先进入 Prefill 资源池，完成输入编码与 KV Cache 构建 生成所需的缓存状态被转移或映射到 Decode 资源池 Decode 节点基于接收到的上下文状态继续逐 token 生成

这样，系统可以针对两个阶段分别选择更合适的资源和并行配置，而不必让同一套部署同时兼顾两类负载。

PD 分离与并行策略的结合方式

PD 分离真正有价值的地方，不只是“把两个阶段拆开”，而是拆开之后可以分别匹配不同的并行策略。

Prefill 侧更偏向计算并行由于 Prefill
计算量大、并行度更高，因此它通常更适合采用偏计算导向的并行方式，例如： 较强的 Tensor Parallel，用于切分大矩阵计算 必要时结合 Pipeline Parallel，支撑更大模型 较大的 batch 组织方式，以提高硬件利用率

也就是说，Prefill 侧更强调“把大计算尽快做完”。

Decode 侧更偏向缓存与时延优化。它通常更适合采用对通信更克制、对缓存管理更友好的配置，例如：更关注 KV Cache 局部性与显存带宽效率 减少过重的 TP 通信链路，避免小步解码被频繁同步拖慢 与连续批处理、流式返回和缓存管理机制更紧密结合 在 MoE 场景下关注 EP
路由与解码稳定吞吐之间的平衡

也就是说，Decode 侧更强调“让逐 token 生成尽量稳定、平滑且低开销”。

与 TP / PP / EP / CP 的组合关系

PD 分离不会替代 TP、PP、EP、CP，而是提供一个更高层的部署框架，使系统能在 Prefill 和 Decode 阶段分别组合不同并行策略。

与 TP 结合：Prefill 侧更容易容忍较强 TP；Decode 侧需要谨慎评估频繁通信是否值得。 与 PP 结合：超大模型仍可能需要 PP，但两个阶段的流水填充效果和时延收益不同。 与 EP 结合：MoE 场景下仍需处理专家路由和 token 分发，Decode 侧尤其敏感于 all-to-all 开销。 与 CP 结合：长上下文场景下，Prefill 构建出的长序列 KV 如何传递给 Decode，是重要问题。

因此，PD 分离更像是一种阶段级部署框架，允许系统在 Prefill 和 Decode 两个阶段分别选择和组合具体并行方式。

PD 分离的核心收益

从工程上看，PD 分离的主要收益通常包括：

让 Prefill 和 Decode 可以分别使用更合适的资源配置 降低两类负载互相干扰的程度
提高长上下文和高并发场景下的整体系统弹性 更容易对 Prefill 吞吐和 Decode 时延分别做独立优化
为异构资源部署提供空间，例如让不同阶段运行在不同类型节点上

尤其在在线服务中，PD 分离可以缓解长 prompt Prefill 阻塞大量 Decode 请求的问题，对服务稳定性有明显价值。

PD 分离的挑战

不过，PD 分离并不是没有代价。它最大的挑战在于：Prefill 与 Decode 之间不再只是逻辑阶段切换，而变成了跨资源池、跨设备甚至跨节点的状态交接问题。

这通常会引入以下挑战：

KV Cache 或上下文状态如何高效传输 两侧并行配置不同时，缓存布局如何匹配 Prefill 输出状态与 Decode 输入状态如何正确映射 跨节点传输是否会引入新的网络瓶颈
调度器如何协调两个阶段之间的流量平衡

因此，PD 分离是否有效，关键不只是“能否拆开”，更在于“拆开后的交接成本是否足够低”。如果 KV Cache 传输和状态重组成本过高，阶段分离带来的收益可能被抵消。

6.6.3.8 编译与 Kernel 优化

在大模型推理系统中，当服务调度、显存管理、缓存组织和分布式通信等优化逐步完善后，系统性能往往会进一步受限于单个算子的执行效率，以及高频小粒度调用带来的调度开销。Decode 阶段尤其明显：单步计算较小，但执行频率极高，Python 调度、kernel launch 和中间访存开销更容易被放大。

因此，编译与 Kernel 优化可以看作推理性能提升的“最后一公里”。它并不直接改变推理流程，而是通过更高效的执行图、更少的 launch 开销、更强的算子融合和更贴近硬件的 kernel 实现，将上层系统设计转化为实际性能收益。

从实现角度看，这一层优化大体可以分为两类：

编译优化：通过图捕获、代码生成和执行图重写，减少 Python/eager 模式下的调度损耗，并生成更高效的执行路径。 Kernel 优化：针对 Attention、Norm、GEMM 和 fused ops 等关键算子设计高效实现，减少访存、提升并行度和硬件利用率。

其中，torch.compile 是 PyTorch 2.x 推荐的编译优化路径，默认后端为 Inductor，可缓存编译结果并在 guard 条件满足时复用。CUDA Graph 则将一串 GPU 操作封装为可重放图，减少 CPU 侧 launch 开销，特别适合高 CPU overhead 或小计算粒度场景。

6.6.3.8.1 torch.compile

基本原理

torch.compile 是 PyTorch 2.x 提供的核心编译优化机制。它的基本思路是：从原本的 eager 执行方式出发，追踪代码中的 PyTorch 运算，将可追踪部分编译为更高效的执行图和优化后 kernel。PyTorch 官方文档指出，torch.compile 会追踪 Python 代码中的 PyTorch 操作，并将代码 JIT 编译为优化后的 kernel；如果代码难以追踪，则会出现 graph break，从而丢失部分优化机会，而不是静默地产生错误结果。

从概念上看，它的执行路径可以理解为：

```
PyTorch eager
  ↓
Graph capture（TorchDynamo）
  ↓
Graph optimization（AOTAutograd / Inductor）
  ↓
生成高效 kernel（Triton / CUDA）
```

对于推理系统而言，torch.compile 的价值不是改变模型数学逻辑，而是将碎片化、解释式的 eager 执行尽量转化为更适合 GPU 高效运行的形式。

对推理场景的主要收益

在推理场景中，torch.compile 的收益主要体现在三个方面。

首先，它可以减少 Python 调度开销。eager 模式下，许多小算子需要频繁由 Python 驱动下发；编译后，这些调用更有机会被整体优化，减少解释器参与。

其次，它可以促进算子融合和执行图优化。多个逻辑相邻的小操作在 eager 模式下往往会拆成多个 kernel；编译后，系统更有机会将其合并为更少、更高效的执行单元，从而减少 launch 次数和中间访存。

再次，它对 Decode 这类小粒度高频路径尤其有价值。Decode 每步工作量较小，Python overhead 和 launch overhead 更容易成为显性瓶颈，因此编译优化往往更容易体现收益。

适用范围与限制

torch.compile 并不是对所有推理路径都同样理想。官方文档指出，编译结果会基于 frame 进行缓存；若后续输入条件不满足已有 guard，就可能触发重新编译，重复次数过多后还可能回退到 eager。这意味着，在推理系统中，以下场景通常更适合使用编译：

模型结构稳定 执行路径相对规则 输入 shape 变化有限或可控 Prefill 这类较大计算块更容易形成稳定图

而以下情况则更容易削弱收益：

变长 Decode 带来频繁 shape 变化 复杂控制流导致 graph break
某些自定义算子或特殊路径缺少良好编译支持

虽然 PyTorch 支持动态 shape
编译，但其本质是通过符号化维度适应不同输入尺寸，并不能完全消除动态形状带来的优化难度。

在推理系统中的常见使用方式

工程上更稳妥的做法不是“全链路无差别 compile”，而是优先编译稳定、重复度高的区域。例如：

对结构规则、计算块大的 Prefill 路径优先使用 torch.compile 对 Decode
中重复出现的局部模块做选择性编译 对 Transformer
层等深层重复结构使用嵌套编译区域，减少重复编译开销。

因此，torch.compile
更适合作为局部到全局逐步推进的编译优化手段，而不是“一键打开必然最优”的机制。

6.6.3.8.2 CUDA Graph

** 基本原理**

CUDA Graph 的核心思想是将一段原本逐个发射的 CUDA
操作录制成可重复执行的图，并在后续迭代中通过一次 replay 统一发射。它不改变计算内容，而是优化 GPU 任务的提交方式。当执行路径高度重复时，graph replay 可以显著减少 CPU 参与和 kernel launch 抖动。

对推理场景的价值

CUDA Graph 特别适合以下类型的推理路径：

执行序列高度重复 每次迭代 kernel 组成基本相同 小 kernel 较多，CPU launch overhead 明显 希望提升 token 级生成路径的稳定性

Decode 阶段单步工作量小、调用频率高，如果每一步都由 CPU 重新发起大量
kernel，调度成本会被放大。CUDA Graph 能将多次 GPU 操作压缩为一次图 replay，因此在 Decode 场景中通常很有价值。

约束和限制

CUDA Graph 的代价在于：它要求执行路径足够稳定。PyTorch 文档明确列出了其重要限制，包括：

控制流不适合 graph capture 会触发 host-device sync 的操作（如 .item()）会出错 kernel
的输入参数在录制时是固定的 CUDA memory addresses 在图中也是固定的，仅地址上的值可以变化

这意味着，CUDA Graph
并不适合高度动态、路径频繁变化的推理流程。因此，在工程上更常见的做法是：

对固定 shape 或少量 shape bucket 的路径进行 capture 对 Prefill 或 Decode 中最稳定的热点路径单独 graph 化 与 batch bucket、静态缓存地址和固定内存池配合使用

与 torch.compile 的关系

torch.compile 和 CUDA Graph 关注的不是同一个层面：

torch.compile 更偏向重写和优化执行图 CUDA Graph 更偏向减少 GPU 工作的发射开销

二者并不冲突。常见组合方式是先通过编译得到更高效、更稳定的执行片段，再对重复性强的热点路径进行 CUDA Graph capture，从而获得更完整的收益。

6.6.3.8.3 算子融合

为什么算子融合重要

在大模型推理中，许多逻辑相邻的操作如果按 eager 方式逐个执行，会带来大量 kernel 启动和中间结果读写。即使单个算子本身不慢，多个小算子串联后也可能产生明显开销。

算子融合的目标是将多个相邻小操作合并为更少的执行单元，从而减少 launch 次数和中间访存。Decode 阶段由于操作粒度小、调用频率高，对这类优化尤其敏感。

常见融合对象

在推理系统中，适合融合的操作通常包括：

线性层后的 bias、激活函数 残差加法与 Norm Attention 前后的 reshape、scale、mask、softmax 等局部链路 一些连续的 elementwise 操作

这些操作单独看通常不重，但如果拆得过细，会增加调度和访存开销。编译器自动融合和手写 fused kernel 本质上都在解决这一问题，只是自动化程度和针对性不同。

例如：

```
LayerNorm → Linear → Activation
```

融合为：

```
Fused Kernel（一次完成）
```

与编译优化的关系

算子融合既可以由编译器自动完成，也可以由工程侧通过自定义 kernel 手动实现。

自动融合的优点是通用性强、接入方便 手工融合的优点是能更贴合具体模型结构和硬件特征

因此，在实际推理框架中，常见策略通常是：

先依赖编译器做通用融合 再对少数最关键热点算子使用专门的 fused kernel

这样既能保留系统通用性，也能把有限优化精力集中在真正影响最大的路径上。

6.6.3.8.4 Attention Kernel 优化

为什么 Attention 是 Kernel 优化重点

Attention 是大模型推理中最核心的计算路径之一。尤其在长上下文和 Decode 场景中，Attention 不仅计算频繁，还高度依赖 KV Cache 读写，因此同时是算力热点和访存热点。

这意味着，Attention 的性能往往不只由数学复杂度决定，还高度取决于：

中间结果是否被重复写回显存 访存路径是否连续 softmax、mask、scale 等步骤是否能高效组织 不同 head、不同 token 维度上的并行度是否充分

FlashAttention 代表的优化方向

FlashAttention 是这一方向中最有代表性的工作之一。从系统视角理解，这类方法的重要意义在于：

不是简单改变 Attention 数学结果 而是通过更 IO-aware 的方式减少中间访存和不必要的数据搬运 同时改进并行组织，让 GPU 执行更加高效

这类方法对推理尤其重要，因为 Decode 阶段往往是带宽敏感路径。减少 Attention 中无效写回和重复读取，通常能带来显著收益。

推理场景下的关注点

推理系统中的 Attention Kernel 优化不仅包括更快的 dense attention，还需要关注：

面向长上下文的高效实现 与分页式 KV Cache / Paged Attention 配合 与 GQA / MQA / MLA 等结构兼容 对不同硬件架构和精度格式做适配 在 Prefill 与 Decode 两个阶段分别优化

因此，Attention Kernel 是模型结构、缓存布局、精度格式和硬件能力的交汇点，也是推理系统中最需要专门工程化优化的热点之一。

6.6.3.8.5 Kernel 级性能优化

核心目标

Kernel 级优化的目标不是简单“手写 kernel”，而是减少无效开销，主要包括：

提高单次 kernel 的有效计算比例 减少不必要的全局内存访问 尽量降低 launch 和同步开销

因此，优化时需要关注是否减少了中间张量读写、是否改善了 memory layout、是否提高了并行粒度和 occupancy，以及是否减少多个小 kernel 之间的往返切换。

常见优化方向

在推理系统中，kernel 级优化常见会围绕以下方向展开：

访存优化：减少 global memory 往返，改善 coalesced access 数据布局优化：让 kernel 更适配目标硬件的高效访问模式 并行映射优化：合理映射线程块、warp 和 tile 大小 融合优化：把多个小步骤合并到一个 kernel 中 精度路径优化：针对 FP16、BF16、FP8、INT8/INT4

等不同格式做专门实现 热点定制：对最常出现、最耗时的路径手工实现专门 kernel

在推理系统中的实际落点

真正有价值的 kernel 优化通常集中在少数热点路径上，例如 Attention、GEMM 前后的小操作、Norm / RMSNorm、量化与反量化、KV Cache 读写与重排，以及采样前后的局部后处理。

因此，kernel 优化并不意味着所有地方都要手写 kernel。更合理的工程策略是：

先用 profiler 找到真正瓶颈 再判断是编译器足够，还是需要专门 kernel 最后只对高频热点投入定制优化

这样才能避免大量低层代码开发，却无法带来明显端到端收益。

6.6.3.9 MoE 模型专项推理优化

Mixture-of-Experts（MoE）模型通过稀疏激活机制，使每个 token
只经过少数专家，从而在扩展总参数规模的同时，将单 token
的实际计算量控制在较低水平。然而，从推理系统角度看，MoE 并不会天然降低部署复杂度。它将 dense
模型中相对规则的前向计算，转化为动态路由、跨设备分发、专家执行和结果聚合组成的复杂链路。

因此，MoE 推理的瓶颈往往不再只是矩阵计算，而更多来自 token 路由、设备间传输、专家负载均衡和 All-to-All 通信。前文已介绍 EP（Expert Parallel）作为 MoE
的基础并行方式，本节进一步从系统角度分析 MoE 推理的专项挑战与优化方法。

6.6.3.9.1 MoE 推理的特殊挑战

稀疏计算并不意味着低系统成本

MoE 的核心特点是每个 token 只激活少量专家，例如 top-k 路由下只经过 2
个专家，而不是全部专家。从理论计算量看，这比同等总参数规模的 dense
模型更高效。但系统执行链路变得更复杂。一个 token 通常需要：

先由 Router 决定应分配到哪些专家 再将 token 发送到对应专家所在设备
完成专家计算后，再将结果收集并组合回来

因此，MoE 实际上是将一部分 dense 计算成本转化为路由、分发、聚合和同步成本。

** 路由稀疏，但通信往往更密集**

在 dense 模型中，token 执行路径相对固定，通信主要由 TP、PP 等并行策略决定。而在 MoE
中，token 会根据输入动态路由到不同专家，通信模式随 batch 的 token 分布变化。例如，一个 token 可能被路由到第 3 和第 7 个专家：

```
Token → Router → Expert 3, Expert 7
```

如果这些专家分布在不同设备上，那么系统就必须把 token
的表示从当前设备发送到目标专家所在位置。这意味着，虽然真正执行计算的专家更少了，但为了让 token 到达这些专家，跨设备通信反而显著增加。因此，MoE 推理中一个非常典型的现象就是：

计算变少了，但通信不一定变少，甚至可能变得更重。

这也是 MoE 推理和 dense 模型推理最本质的系统差异之一。

专家负载不均是常态而不是例外

由于 Router 根据 token
特征动态选择专家，不同专家被命中的频率通常差异明显，容易出现少数热点专家持续过载、部分专家空闲的情况。从系统角度看，这种负载倾斜会带来明显问题：

某些 GPU 上的专家非常繁忙 某些 GPU 上的专家利用率较低 整体执行时间被最慢的热点专家拖住
系统吞吐受制于局部 straggler，而不是平均负载

因此，MoE 推理性能往往不是由平均负载决定，而是由最忙专家或最忙设备是否形成 straggler 决定。

All-to-All 成为关键通信模式

MoE 推理中的核心通信通常是 All-to-All。典型链路为：

```
Token → Router → All-to-All Dispatch → Expert Compute → All-to-All Combine→Output
```

从这个过程可以看出，MoE 推理通常至少包含两次关键的跨设备数据交换：

Dispatch：将 token 分发到目标专家 Combine / Gather：将专家输出重新收集并合并

因此，MoE 推理效率高度依赖互联带宽、网络拓扑、All-to-All 实现效率、token 分发均衡性，以及小 batch / Decode 场景下通信是否被放大。尤其在 Decode 阶段，单步粒度小，叠加 All-to-All
后通信成本更容易压过专家计算本身。

MoE 的推理流程比 dense 模型更“系统化”

如果从完整执行链路来看，MoE 推理的复杂度明显高于 dense 模型。dense
模型更多是在固定图结构上重复前向，而 MoE 推理则通常包含以下几个连续阶段：

Router 决定专家分配 token 根据路由结果进行跨设备分发 专家在本地执行前向计算
不同专家输出被重新聚合 系统继续后续层计算或进入下一步 Decode

因此，MoE 优化不是单个算子优化，而是路由、通信、负载均衡和专家执行的端到端协同优化。

6.6.3.9.2 计算-通信重叠优化（TBO / SBO）

在 MoE 推理中，降低系统效率的常常不是专家计算本身，而是 dispatch / combine
通信。如果系统严格按照“先分发、再计算、再回收”的串行方式执行，GPU
很容易在等待通信时空转。

计算-通信重叠优化的核心思想是：通信不可避免，但应尽量避免计算和通信串行等待。SGLang
中具有代表性的机制包括 TBO（Two-Batch Overlap）和 SBO（Single-Batch Overlap）。

为什么 MoE 需要重叠优化

一个 MoE 层通常包含 Router、dispatch、expert compute 和

combine。如果这些阶段完全串行，通信阶段 GPU
算力闲置，计算阶段网络链路闲置。重叠优化的目标是将通信和计算组织成更连续的执行流水，从而隐藏部分通信时延，提高硬件利用率。

TBO：Two-Batch Overlap

BO 的核心思想是将请求拆成更细粒度的 micro-batches，并让相邻 micro-batch
的不同阶段交错执行。例如，当前 micro-batch 执行 attention 或 MLP 计算时，前一个或后一个 micro-batch 的 dispatch / combine 可以同时推进。

TBO 本质上是一种跨 batch 的流水化调度机制，主要收益包括：

通信等待时间被部分隐藏 GPU 计算阶段与网络传输阶段更连续 整体吞吐更容易提升

官方文档中甚至直接提到，启用 TBO 在某些场景下可解锁最高约 2
倍吞吐提升，但这类收益显然依赖具体模型、负载和通信条件，不能视为固定结论。从适用性上看，TBO 更适合：

batch 规模较大或可进一步拆分的场景 Prefill 或吞吐优先的工作负载 需要通过 micro-batch 流水化摊薄 All-to-All 通信成本的部署

但它也会增加调度、状态管理、micro-batch 切分和缓存管理复杂度。

SBO：Single-Batch Overlap

与 TBO 通过“跨 batch 交错”来实现重叠不同，SBO 更强调：在单个 batch
内部，就尽量把可并行的计算和通信重叠起来。SGLang 官方文档将 SBO 描述为一种基于 dispatcher hook 的机制，它能够在单个 batch 内实现诸如“shared experts computation 与 communication 的重叠”，并且这种机制被设计成在 dispatch / combine 前后插入 hook，而不是直接侵入修改核心 MoE 模块逻辑。

从系统设计角度看，SBO 的重点不再是构造跨 batch 流水，而是识别一个 batch
内哪些阶段之间存在可并行空间。例如：

某部分共享专家计算已经可以开始 与此同时，另一部分 token 的通信还在进行 系统通过 hook 或异步机制把二者叠起来执行

SBO 不依赖跨 batch 流水，因此更适合低时延、单 batch 响应性更重要的场景。尤其在 Decode
中，如果 batch 较小、难以构造稳定 micro-batch 流水，SBO 往往更有现实意义。

但 SBO 需要系统准确识别单 batch 内部的依赖边界，确保异步化和 hook 化不会破坏 dispatch / combine 的正确性。

TBO 与 SBO 的区别

TBO 和 SBO 都属于计算-通信重叠，但粒度不同：

TBO 重叠不同 micro-batch 的不同阶段，偏向跨 batch 流水化，重点提升吞吐。 SBO 重叠同一 batch 内可并行的阶段，偏向局部细粒度优化，重点减少单 batch 等待。

因此，二者并不是互斥关系，而更像是面向不同场景和不同粒度的两种重叠思路。

适用场景与系统权衡

计算-通信重叠适合大规模 EP、All-to-All 已成为瓶颈、专家计算和通信都足够重、需要减少 Decode 或混合负载下 GPU 空转的场景。但这类优化并不是“白捡性能”。无论是 TBO 还是 SBO，本质上都在引入更复杂的执行编排，因此系统需要额外处理：

更细粒度的阶段状态管理 dispatch / combine 与计算之间的同步关系 KV Cache、token 布局和专家结果回写的一致性 重叠带来的调试复杂度和性能不确定性

因此，是否采用 TBO 或 SBO，往往取决于系统当前瓶颈是否真的在通信等待上。如果专家计算本身已占主导，或者 batch 太小、通信难以形成稳定重叠，那么复杂的 overlap 机制未必一定划算。

6.6.3.9.3 专家负载均衡优化（EPLB）

在 MoE 推理中，专家负载不均是由动态路由自然引出的系统性问题。Router 根据 token 特征选择 top-k 专家，不同专家被命中的频率通常差异明显，导致热点专家过载、部分专家空闲。

SGLang 的 EPLB（Expert Parallelism Load Balancer）通过分析 expert activation statistics，计算更优的专家放置或复制方案，以降低 GPU 利用率方差、减少空转并提升扩展性。

为什么 MoE 需要专门的专家负载均衡

MoE 的负载来源不是静态模型结构，而是运行时路由结果。即使专家数量和 GPU 数量看起来均衡，实际请求流仍可能让少数专家成为热点。一旦热点专家所在 GPU 成为 straggler，dispatch、专家计算和 combine 都会被拖慢，进而放大 All-to-All 的尾部等待。

因此，专家负载均衡的目标不是让每张卡平均放置相同数量的专家，而是让实际被访问的专家热点更均匀地摊到系统中。

EPLB 的基本思想

EPLB 的核心是根据一段时间内观察到的专家激活统计，重新安排专家在 GPU 上的位置，并在必要时复制热点专家，从而缓解路由倾斜导致的局部过载。

它并不改变 Router 的数学逻辑，而是改变“被 Router 选中的专家最终落在哪些设备上”。Router 仍然输出相同的专家选择，EPLB 则通过专家布局和副本策略提升系统承载这些路由结果的能力。

EPLB 优化的对象：不是 Router，而是专家布局

EPLB 不是训练时给 Router 加辅助 loss，也不是在线修改 top-k 路由规则，而是系统层面的布局优化器。它通过 expert placement / replication 降低 GPU utilization variance，而不侵入模型语义。这也是为什么 EPLB 在推理系统里很重要：

它让系统不必侵入模型语义层，只通过资源布局与副本策略，就能对 MoE 的运行时热点进行缓解。这类方法通常比直接改 Router 更容易在 serving 框架中落地，也更适合和已有的 EP、dispatch/combine 后端、重叠优化等机制协同工作。SGLang 把 EPLB 作为 EP 文档中的 Workload Balancer 单独介绍，也说明它在 serving 栈里被视为系统优化组件，而不是模型结构本身的一部分。

热点专家复制与专家重排

从实现上看，EPLB 的优化动作通常主要有两类。

第一类是专家重排（expert rearrangement）。根据激活统计，将高访问频率或高相关专家分散到不同 GPU / 节点，避免热点集中。

第二类是热点专家复制（redundant expert replication）。为特别热门的专家放置冗余副本，让请求流量有更多承载点，减轻单点过载。

这说明，EPLB 的关键关键问题不是复制越多越好，而是复制哪些专家、复制到哪里，以及是否与硬件拓扑匹配。如果复制位置不合理，可能增加显存占用，却无法真正降低跨节点流量。

收益与时间尺度

EPLB 最直接的收益，是改善不同 GPU 之间的工作量均衡程度。从系统视角看，这些收益可以理解为：

热点专家不再过度集中，局部 straggler 更少 不同设备在 dispatch / compute / combine 三个阶段中的等待更少 All-to-All 两端更容易保持均衡推进 系统吞吐不再被最忙的一小部分专家严重拖累

同时，EPLB 通常不是一次性静态优化，而是周期性运行的控制环。由于线上请求分布和热门专家会随时间变化，系统需要定期根据新的 activation statistics 重平衡。重平衡过少会导致布局滞后，过于频繁又会带来搬迁和管理成本，因此需要在统计稳定性和更新开销之间权衡。

限制与权衡

虽然 EPLB 对 MoE 推理通常很有价值，但它并不是没有代价。

首先，热点专家复制会占用更多显存，尤其在大规模 MoE 模型中，副本策略本身会消耗额外资源。其次，专家重排与重平衡需要依赖激活统计，而统计窗口过短可能不稳定，过长又可能滞后。

此外，EPLB 的收益还会受到硬件拓扑影响。如果专家虽然被复制了，但复制位置没有考虑节点边界、NVLink / RDMA 差异，那么负载表面上可能更均衡，但跨节点流量未必真的下降。

因此，EPLB 本质上是同时考虑激活统计、专家放置、复制策略和拓扑结构的系统优化机制。

6.6.3.9.4 MoE 推理中的专家放置与通信协同

在 MoE 推理中，专家放置不是独立的静态部署问题。同样一组专家和路由结果，如果放在不同节点、GPU 或互联域上，实际通信成本可能完全不同。尤其在多机 EP 场景下，关键不只是专家是否均匀分布，而是 token 访问目标专家时走本地高速互联，还是跨节点远程通信。

专家放置的核心目标不是均匀，而是降低访问代价

从系统角度看，专家放置至少同时面对两个目标：

让热点专家不会过度集中到少数设备 让 token 访问目标专家时尽量走更低代价的链路

因此，专家放置不应只关注每张卡放多少专家，还要关注热点专家副本覆盖哪些节点，以及 token 更可能从哪里访问这些专家。如果热门专家只存在于远端节点，即使专家计算本身不慢，也会因频繁跨节点访问拖慢整个 MoE 层。

拓扑感知放置比静态平均放置更重要

在多机系统中，GPU 之间并不是“等距”的。同一节点内部通常具有更高带宽、更低时延的互联，而跨节点访问则更容易受到 RDMA、交换结构和网络拥塞影响。这说明，对于 MoE 推理而言，专家放置的关键已经不是抽象的“平衡”，而是拓扑感知的平衡。也就是说，系统要尽量做到：

热点专家副本覆盖更多节点 常被一起访问的专家不要都堆在同一远端位置 token 更常访问的专家尽可能在本地或近端可达 跨节点流量只用于真正不可避免的部分

从这个意义上说，专家放置实际上就是在做一种“通信代价最小化”的布局问题。

专家放置需要和通信模式一起设计

MoE 推理的核心通信并不是普通的点对点传输，而是 dispatch / combine 这类近似 All-to-All 的模式。因此，专家放置是否合理，最终要看它能否让这两轮关键通信变得更便宜、更稳定。

DeepEP 官方仓库将其定位为一个面向 MoE 和 Expert Parallelism 的专用通信库，提供高吞吐、低时延的 all-to-all GPU kernels，也就是面向 MoE dispatch 和 combine 的专门实现。其 README 明确指出，DeepEP 同时提供：

面向高吞吐的正常模式，适合训练与推理 Prefill 面向低时延的纯 RDMA 低延迟模式，适合推理解码阶段 针对 NVLink 域到 RDMA 域这种异构带宽场景的优化转发能力

这意味着，不同通信后端会影响专家布局策略。Prefill 更关注大 batch 高吞吐，Decode 更关注小 batch 低时延，因此专家放置需要结合阶段目标、通信模式和硬件拓扑共同决定。

与 EPLB 和重叠优化的关系

专家放置不能脱离 EPLB 和 TBO/SBO 单独讨论：

EPLB 解决哪些专家更热、是否需要重排或复制。 专家放置与通信协同 决定专家副本放在哪里访问代价更低。 TBO / SBO 在给定布局和通信路径基础上隐藏通信时延。

如果专家放置不合理，例如热点专家副本仍集中在错误节点上，那么 EPLB 的收益会被削弱，TBO/SBO 也只能重叠本不该如此昂贵的通信路径。

6.6.3.9.5 MoE 推理优化的整体协同

MoE 推理优化不是若干技巧的简单叠加，而是高度耦合的系统协同问题。一个 MoE 层通常包括路由、分发、专家计算和结果回收四个阶段。任一环节成为瓶颈，都会削弱其他优化收益。

从局部优化到链路优化

如果只看单点优化，MoE 系统中可以分别做很多事情：

用 TBO / SBO 隐藏通信等待 用 EPLB 缓解热点专家过载 用更合理的 专家放置 降低跨节点访问成本
用更高效的 dispatch / combine 后端 降低 All-to-All 开销

但这些优化相互依赖。通信重叠需要通信路径稳定，负载均衡需要与专家放置结合，通信后端收益也受专家布局和路由分布影响。因此，MoE 推理优化更适合被理解为链路优化，而不是单点优化。

MoE 推理的三个核心目标

从系统角度看，MoE 推理优化最终通常围绕三个目标展开：

第一，降低通信代价。减少跨节点传输，提高本地命中，并隐藏不可避免的通信时延。

第二，缓解负载倾斜。通过负载均衡、专家复制和拓扑感知放置，避免热点专家形成 straggler。

第三，维持执行连续性。通过
overlap、异步调度和阶段协同，让路由、通信和计算持续推进，减少等待和空转。

这三个目标并不是并列分开的，而是相互制约、相互支撑的。

一个更合理的理解框架

如果用更清晰的方式概括，MoE 推理优化大致可以看成三层协同：

路由与负载层：决定 token 去哪些专家，以及热点是否会集中 放置与通信层：决定 token
到专家的路径是否高效，通信代价是否可控
执行与重叠层：决定通信和计算能否被连续组织，减少等待和空转

在这个框架下：

EPLB 更偏向解决第一层和第二层之间的问题 专家放置与通信协同 更偏向第二层 TBO / SBO
更偏向第三层

6.6.3.10 典型场景下的优化组合

前文分别介绍了调度与服务、KV Cache 与显存、低比特推理、Decode
路径优化、并行与分布式、编译与 Kernel 优化，以及 MoE
专项优化等技术。从系统实现角度看，这些方法并不是孤立使用的，而是需要结合具体业务场景进行组合设计。

不同场景的主要瓶颈不同：高并发服务更关注低时延和吞吐，长上下文场景更受 KV Cache
和显存限制，超大模型部署更依赖多机多卡并行，MoE 服务则重点面向路由、All-to-All
通信和专家负载均衡。因此，实际部署中不存在统一最优方案，而需要围绕场景目标选择合适的优化组合。

6.6.3.10.1 高并发在线服务

场景特点

高并发在线服务通常表现为请求持续流入、长度差异较大、用户对首 token 时延敏感，同时系统吞吐和尾延迟都受到严格约束。在这类场景中，单个请求未必很重，但整体流量动态性强，因此服务层组织效率往往比单算子极限性能更重要。

推荐优化组合

连续批处理（Continuous Batching） 异步调度（Asynchronous Scheduling） 动态批处理（Dynamic Batching） Prefix Cache CUDA Graph 或选择性编译优化

组合逻辑

该场景的关键是让系统在动态流量下持续保持高利用率。连续批处理和动态批处理提升 batch 利用率，异步调度减少 Prefill 与 Decode 相互阻塞，Prefix Cache 降低重复前缀带来的 Prefill 开销，CUDA Graph 或编译优化则用于降低 Decode 阶段的小粒度执行成本。

适用边界与权衡

该组合适合交互式服务和高并发请求流，但会增加调度和缓存管理复杂度。如果请求模式过于离散、前缀复用率较低，Prefix Cache 收益会下降；如果 shape 变化过大，编译和 CUDA Graph 的收益也会受限。

6.6.3.10.2 长上下文场景

场景特点

长上下文场景的主要特征是输入序列长、Prefill 开销大、KV Cache 占用和带宽压力快速上升。此时，系统通常更容易从 compute-bound 转向 memory-bound，瓶颈也从单层计算转向缓存规模、访存效率和长序列执行粒度。

推荐优化组合

Chunked Prefill Context Parallel（CP） Paged KV Cache GQA / MQA / MLA KV Cache 量化

组合逻辑

Chunked Prefill 用于降低长输入带来的瞬时显存峰值，并减少大 Prefill 对系统的阻塞；Context Parallel 分摊超长序列负载；Paged KV Cache 提升缓存分配与回收效率；GQA / MQA / MLA 从模型结构上降低 KV 规模；KV Cache 量化进一步降低显存占用和带宽压力。

适用边界与权衡

该组合适合长文本理解、长文生成和 RAG 等场景，但系统复杂度较高。CP、分页缓存、量化和长上下文 kernel 需要良好协同，否则可能引入额外通信和实现成本。

6.6.3.10.3 超大模型部署

场景特点

当模型规模超过单卡甚至单机能力时，系统首先要保证模型能够稳定部署，并具备可接受的吞吐和时延

。此时，多机多卡协同、参数切分和通信控制成为系统设计核心。

推荐优化组合

Tensor Parallel（TP） Pipeline Parallel（PP） 必要时结合 Expert Parallel（EP） 权重量化
通信重叠与通信优化 编译优化与算子融合

组合逻辑

TP、PP 和 EP
用于突破单设备容量限制，使模型能够在多设备环境下运行；权重量化进一步降低部署门槛；通信优化和重叠机制减少并行带来的同步成本；编译与算子融合提升单路径执行效率，避免分布式条件下仍被底层算子开销拖慢。

适用边界与权衡

该组合适合超大参数模型和多节点部署，但通信开销往往是主要限制。尤其在 Decode
阶段，过强的模型并行可能因小粒度通信导致收益下降，因此需要在可扩展性和单请求时延之间权衡。

6.6.3.10.4 MoE 大模型服务

场景特点

MoE 模型具有计算稀疏性，但推理系统中往往会出现高通信、专家负载不均和 All-to-All
开销显著等问题。其瓶颈通常不在单个专家计算，而在于路由、通信和专家放置能否协同。

推荐优化组合

Expert Parallel（EP） 计算-通信重叠优化（TBO / SBO） 专家负载均衡（EPLB）
专家放置与通信协同优化 All-to-All 通信后端优化

组合逻辑

EP 支撑大规模专家部署；TBO / SBO 隐藏 dispatch 和 combine 的通信等待；EPLB
缓解热点专家过载；专家放置与通信协同减少跨节点访问代价；高效 All-to-All
后端则将这些策略转化为实际吞吐收益。

适用边界与权衡

该组合适合大规模 MoE 服务，但复杂度明显高于 dense
模型。其收益高度依赖路由分布、节点拓扑和通信后端实现。如果负载倾斜严重或专家副本布局不合理，单独依赖某一项优化往往难以获得稳定收益。

6.6.3.10.5 PD 分离架构

场景特点

在许多在线推理系统中，Prefill 与 Decode 的资源需求差异明显：Prefill 更偏计算密集，Decode
更偏缓存和带宽敏感。如果二者长期绑定在同一资源池，容易导致利用率不均和阶段间相互干扰。

推荐优化组合

Prefill / Decode 分离部署 针对阶段特性的异构资源配置 KV 传输优化 Speculative Decoding 与调度系统协同的阶段级负载控制

组合逻辑

PD 分离的关键是让 Prefill 和 Decode 分别运行在更适合自身特性的资源上。Prefill 侧更关注计算吞吐，Decode 侧更关注 KV Cache 管理和低时延生成。KV 传输优化用于降低阶段交接成本，Speculative Decoding 则进一步提升 Decode 路径效率。

适用边界与权衡

该架构适合长 prompt、多轮对话和高并发在线服务，但核心挑战是阶段间状态交接成本。如果 KV 传输和布局映射效率不足，PD 分离收益可能被跨资源池传输开销抵消。

6.6.3.10.6 场景优化总览

总体来看，不同推理场景并没有统一最优的技术栈，而是需要围绕各自的主瓶颈进行组合设计：

高并发在线服务更关注调度、批处理和低时延响应 长上下文场景更关注 KV Cache、显存和带宽控制 超大模型部署更关注模型切分、通信和多机扩展 MoE 服务更关注路由、All-to-All 和负载均衡 PD 分离架构更关注阶段解耦和资源异构利用

因此，推理优化的实际落地通常不是单独选择某一种技术，而是根据业务目标、模型结构和硬件条件，构建面向场景的优化组合。

6.6.3.11 各类优化技术的权衡与挑战

前文介绍了大模型推理中的多类优化方法。但在实际系统设计中，这些优化并不能无限叠加，每一种方法都会引入新的约束与代价。因此，推理系统优化的本质不是追求单一指标极致，而是在性能、精度、资源占用和工程复杂度之间进行多维权衡。

6.6.3.11.1 性能与精度的权衡

在推理优化中，性能提升往往伴随一定程度的近似计算或信息损失，从而影响模型输出质量。

量化引入的误差

低比特量化，如 INT8、INT4，通过压缩权重、激活或 KV Cache 表示来降低显存和带宽开销，但也可能带来数值精度下降、分布失真和局部误差放大。尤其在长文本生成、代码生成和高精度任务中，误差可能随生成过程累积，进一步影响输出质量。

推测解码的接收率问题

推测解码通过“生成 + 验证”机制提升速度，其性能高度依赖接收率（acceptance rate）：

接收率高 → 加速显著 接收率低 → 频繁回退，收益下降

同时：

接收率波动 → 性能不稳定 不同输入 → 行为差异较大

过度优化带来的精度波动

当量化、KV
压缩、推测解码等优化同时使用时，误差可能出现非线性叠加，不同模块之间也可能相互影响，导致精度波动更难预测。因此，优化越激进，越需要精细的精度评估、校准和回退机制。

核心结论：

性能优化越激进，越需要精细的精度控制与评估机制。

6.6.3.11.2 吞吐与时延的权衡

吞吐（Throughput）与时延（Latency）是推理系统中的两个核心指标，但二者往往存在天然冲突。

Batch 大小的影响

较大的 batch 可以提高 GPU 利用率和系统吞吐，但请求往往需要等待 batch 形成，从而增加首 token 时延。较小的 batch 有利于低延迟响应，但硬件利用率可能不足，整体吞吐受限。可以概括为：

```
Batch ↑ → Throughput ↑ → Latency ↑
```

调度公平性问题

在动态调度中，长请求可能长期占用资源，短请求也可能因调度策略不当被延迟。如果系统过度追求吞吐，可能导致尾延迟上升，影响用户体验。

典型权衡策略

实际系统通常需要根据业务目标选择不同策略：小 batch 更适合低延迟交互，大 batch
更适合高吞吐任务；分级队列、优先级调度和 aging 机制则可用于在吞吐与公平性之间取得平衡。

6.6.3.11.3 显存与计算的权衡

推理优化中，显存与计算往往可以相互交换。很多方法通过增加一定计算开销，换取更低的显存占用或更小的带宽压力。

KV 压缩 vs 计算开销

KV Cache
量化或压缩可以降低显存占用和带宽压力，但通常需要额外的反量化、重构或格式转换操作。如果这些开销过大，可能抵消缓存压缩带来的收益。

缓存策略的取舍

更激进的缓存复用、分页管理或压缩策略可以提升显存利用率，但也会增加索引管理、映射维护和访问调度成本。缓存策略越灵活，系统管理复杂度通常越高。

带宽 vs 计算的交换

在 Decode 阶段，系统常常是
memory-bound，瓶颈主要来自显存带宽而非算力。此时，通过压缩数据、减少访存，哪怕引入少量额外计算，也可能比直接读取更多数据更高效。

核心结论：

在 memory-bound 场景中，“多算一点”有时比“多存一点”更高效。

6.6.3.11.4 工程复杂度与可维护性的权衡

随着优化手段不断叠加，系统复杂度会快速上升，这也是实际工程中最容易被低估的问题。

优化深度与系统复杂度

简单系统通常更易维护、行为更可预测；复杂优化系统虽然可能获得更高性能，但也会带来多模块耦合、状态管理复杂和运行行为难以预测等问题。

多优化叠加的排障问题

当 continuous batching、KV Cache 管理、speculative decoding、CUDA Graph、TP/EP
等机制同时启用时，系统问题往往更难定位。性能波动可能来自调度、缓存、通信、kernel
或模型路径中的任一环节，复现和排查成本都会显著增加。

可维护性挑战

复杂优化会带来代码复杂度上升、硬件适配困难、调优成本增加以及 fallback
路径维护压力。性能提升越依赖底层细节，系统对硬件、驱动、框架版本和模型结构的敏感性也越高。
可以概括为：

```
性能 ↑ → 系统复杂度 ↑ → 可维护性 ↓
```

工程实践建议

为了降低复杂度风险，推理系统应尽量采用分层设计，保留稳定的 fallback
路径，控制优化粒度，并建立完善的监控、profiling
和回归测试体系。这样才能在追求性能的同时，保证系统行为可控、问题可定位、线上运行稳定。

核心结论：

最优系统不一定是“最快的”，而是“可控且稳定的”。

6.6.4 算子级MOE优化实战

6.6.4.1 前置准备工作

AITER MoE 是面向大语言模型的高性能稀疏专家层计算框架，核心依托 Token 路由重排 + 两阶段融合GEMM
实现高效稀疏计算，支持多后端、多精度量化与专家并行，可完整覆盖大模型推理全场景。

AITER 作为 AI 张量计算引擎，可通过 C++ / PyTorch API 统一整合 CK、Triton、ASM、HIP 等底层加速能力，最大化 AMD GPU 硬件性能。

1. 代码仓库初始化

AITER 算子库依赖 CK、moe_c 子模块，首次使用必须初始化子模块：

git submodule update –init

1. 清理历史编译产物

若存在旧版本编译 / 安装文件，需先彻底清理，避免冲突：

cd aiter

rm -rf aiter/jit/aiter_.so

rm -rf aiter/jit/build

rm -rf aiter/jit/*.so

rm -rf aiter/jit/jit/build

rm -rf build

rm -rf aiter.egg-info

rm -rf aiter_meta

卸载旧版 aiter

pip uninstall aiter -y

1. 开发模式安装

根据目标 GPU 架构指定编译参数，以 develop 模式 安装：

gfx936 对应 BMZ，gfx938 对应 NMZ

GPU_ARCHS=“gfx936;gfx938” python setup.py develop

6.6.4.2 MoE 核心基础结构

1. 从 Llama MLP 到 MoE 架构

FFN（前馈网络）是大模型的核心计算单元，MoE 对标准 FFN 进行了专家化改造：

Llama 系列：单 Expert 结构，使用统一 MLP 处理所有 Token，无路由逻辑

DeepSeek MoE：多 Expert 架构，每个 Token 仅激活 Top-K
个专家计算，在提升模型容量的同时控制计算量

单专家计算逻辑（与 Llama MLP 完全一致）：

输入隐状态 → Gate 投影 + Up 投影 → SiLU 激活 → 逐元素相乘 → Down 投影 → 残差融合

高层数据流向：

输入隐状态 [M, model_dim]

→ Top-K 专家选择

→ Token 按专家分组重排

→ Stage1 GEMM（Gate+Up 融合投影）

→ 激活融合（SiLU/GELU + 点乘）

→ Stage2 GEMM（Down 投影）

→ 路由权重加权聚合

→ 最终输出

1. 两种计算视角

MoE 的计算效率，完全由 数据组织方式 决定：

（1）Token-by-Token

计算逻辑：逐个 Token 处理，每个 Token 执行 K 次独立小 GEMM

缺陷：

小 GEMM 频繁启动，GPU 算力利用率极低

专家权重重复加载，显存带宽浪费严重

需要大量临时缓存存储中间结果

（2）Expert-by-Expert

计算逻辑：先通过排序重排，将 Token 按所属专家分组

优势：

每个专家仅执行 1 次 GEMM，计算规整高效

专家间可并行计算

无需额外临时缓存，通过原子操作完成结果聚合

显存带宽利用率大幅提升

1. MoE 核心计算流水线

高性能 MoE 可抽象为 4 个核心步骤：

Scatter-GEMM → SiLU 激活 + 逐元素乘 → Mul-Reduce → Gather-GEMM

权重优化：多专家权重可连续拼接存储，单次 GEMM 完成批量计算

分布式适配：支持专家并行（EP）/ 张量并行（TP），最终通过 moe_sum 完成多卡结果归约

1. 核心组件

组件	核心功能	代码位置
fused_moe	MoE 主入口，统筹全流程调度行	aiter/fused_moe.py
topk_softmax	单 Token Top-K 专家选择 + 权归一化	aiter/ops/moe_op.py
moe_sorting	Token 按专家分组重排，ID 编与对齐	aiter/fused_moe.py
get_aiter_moe_config	MOE 后端配置选择器	aiter/moe.py
aiter_moe	MOE 执行入口	aiter/moe.py

6.6.4.3 MoE 核心模块详解

1.MoE 主入口：fused_moe

fused_moe 是 AITER MoE 的统一调用接口，定义如下：

def fused_moe(

hidden_states, # [M, model_dim]

w1, # [E, inter_dim*2, model_dim] Gate+Up 权重

w2, # [E, model_dim, inter_dim] Down 权重

topk_weight, # [M, topk] 路由权重

topk_ids, # [M, topk] 专家索引

activation=ActivationType.Silu,

quant_type=QuantType.No,

w1_scale=None, # 权重量化 Scale

w2_scale=None,

...

) -> torch.Tensor

1. Token 路由模块

Token 路由是 MoE 稀疏计算的前置核心，负责完成专家选择与数据重排，由两个子模块构成：

(1) TopK 专家选择：topk_softmax

功能

对每个 Token 计算专家得分，筛选 Top-K 专家并做权重归一化。

计算流程

门控投影：输入隐状态通过线性层得到 [M, num_experts] 专家得分

Softmax 归一化：将得分转为概率分布

Top-K 筛选：保留得分最高的 K 个专家，输出索引与权重

输入输出

输入：hidden_states [M, model_dim]

输出：

topk_ids [M, topk]：每个 Token 选中的专家编号

topk_weight [M, topk]：路由权重

函数接口

def topk_softmax(

topk_weights: Tensor,

topk_indices: Tensor,

token_expert_indices: Tensor,

gating_output: Tensor,

need_renorm: bool,

)

1. Token 重排：moe_sorting

moe_sorting 是路由模块的核心，将 Token 视角 数据转为 Expert 视角 连续数据，直接决定后续 GEMM 效率。

核心功能

视角转换：Token 路由结果 → 专家分组连续数据

块对齐：按 block_size_M 补齐，适配 GPU 计算块

复合编码：紧凑存储 Token ID + TopK 序号，节省内存

专家过滤：支持 expert_mask，适配多卡专家并行

Padding 标记：用无效值标记填充位，计算时自动屏蔽

关键技术机制

复合 ID 编码

32 位整数存储：低 24 位 = Token ID，高 8 位 = TopK 序号

编码宏：MOE_SORTING_MOCK_ID(token_id_, topk_id_)

作用：减少内存占用、支持量化、快速解析路由信息

块对齐与 Padding

按 block_size_M（32/64/128）补齐 Token 数，保证 Grouped GEMM 计算规整。

专家并行（EP）掩码过滤

通过 expert_mask 过滤非本地专家，仅计算当前 GPU 负责的专家。

LDS 共享内存加速

基于 AMD GPU 共享内存实现并行统计与排序，提升处理速度。

调用接口

```
aiter.moe_sorting_fwd(
topk_ids,
topk_weights,
sorted_ids,
sorted_weights,
sorted_expert_ids,
tokens_positions_per_expert,
```

num_valid_ids,

moe_buf,

num_experts,

block_size,

expert_mask,

)

1. MOE执行核心部分

在MOE实际计算前，先通过获取Backend（aiter）对当前MOE问题支持情况，再根据config状态调用实际融合算子，实际应用分 Get config 和 Run moe 两步接口。

(1) **get_aiter_moe_config **

作用

该函数根据给定的 MOE（Mixture of Experts）问题规模 和
量化类型，在多个可用的底层后端实现中选择最优方案，并返回对应的执行配置。

def get_aiter_moe_config(

M: int, # token 数量（batch size）

E: int, # experts数量

N1: int, #GEMM1 gate + up 的合并维度（通常 N1 = 2 * N2）

N2: int, # GEMM2 down / intermediate 维度

K: int, # model 维度

top_k: int, # 每个 token 激活的 expert 数

block_size: int, # backends 量化 block size

dtype: torch.dtype, # 激活数据类型

quant_type: str, # 量化类型 (e.g. “w4a16”, “w8a8”)

activation: str = “silu”, # “silu”/“gelu”/“relu2”/…

gated: Optional[bool] = None, # True=GLU-gated (N1=2*inter), False=non-gated (N1=inter); None=auto from activation

) -> Tuple[bool, AiterMoeConfig]:

返回值

bool：True 表示找到了可用的后端配置；False 表示没有合适的实现

AiterMoeConfig：包含 quant_type、solution_type（后端类型）、config（后端特定的 kernel 参数字典）

(2) aiter_moe

该函数是 Aiter 库中量化 MOE 前向计算的统一入口。它接收由 get_aiter_moe_config 返回的 AiterMoeConfig，根据其中的 solution_type 自动分派到对应的后端 kernel 执行实际的 MOE 计算。

```
def aiter_moe(

hidden_states: torch.Tensor, # 输入激活 [M, K]

w1: torch.Tensor, # gate/up 权重（量化后）

w2: torch.Tensor, # down 权重（量化后）

topk_weights: torch.Tensor, # 每个 token 选中 expert 的权重

topk_ids: torch.Tensor, # 每个 token 选中 expert 的索引

moe_config: AiterMoeConfig, # 由 get_aiter_moe_config 返回的配置对象

inplace: Optional[bool] = False, # Write output into hidden_states directly.

activation: str = “silu”, # 激活函数(“silu”, “gelu”, …).

w1_scale: Optional[torch.Tensor] = None,

w2_scale: Optional[torch.Tensor] = None,

w1_zp: Optional[torch.Tensor] = None,

w2_zp: Optional[torch.Tensor] = None,

a1_scale: Optional[torch.Tensor] = None,

a2_scale: Optional[torch.Tensor] = None,

block_shape: Optional[list] = None, # Optional [block_n, block_k] shape for block-based quantization backends

global_num_experts: int = -1,

expert_map: Optional[torch.Tensor] = None, # Optional mapping tensor from local expert indices to global expert indices

routed_scaling_factor: Optional[float] = 1.0,

) -> torch.Tensor:
```

6.6.4.4 MoE算子优化

算子（Operator）是构成模型计算逻辑的最小功能单元，本质是实现特定数学运算或数据处理逻辑的代码模块。通用框架的原生算子为了兼容性，往往无法适配特定硬件、模型或场景，而需要通过定制化优化解决这些痛点。常见的技术路线有：

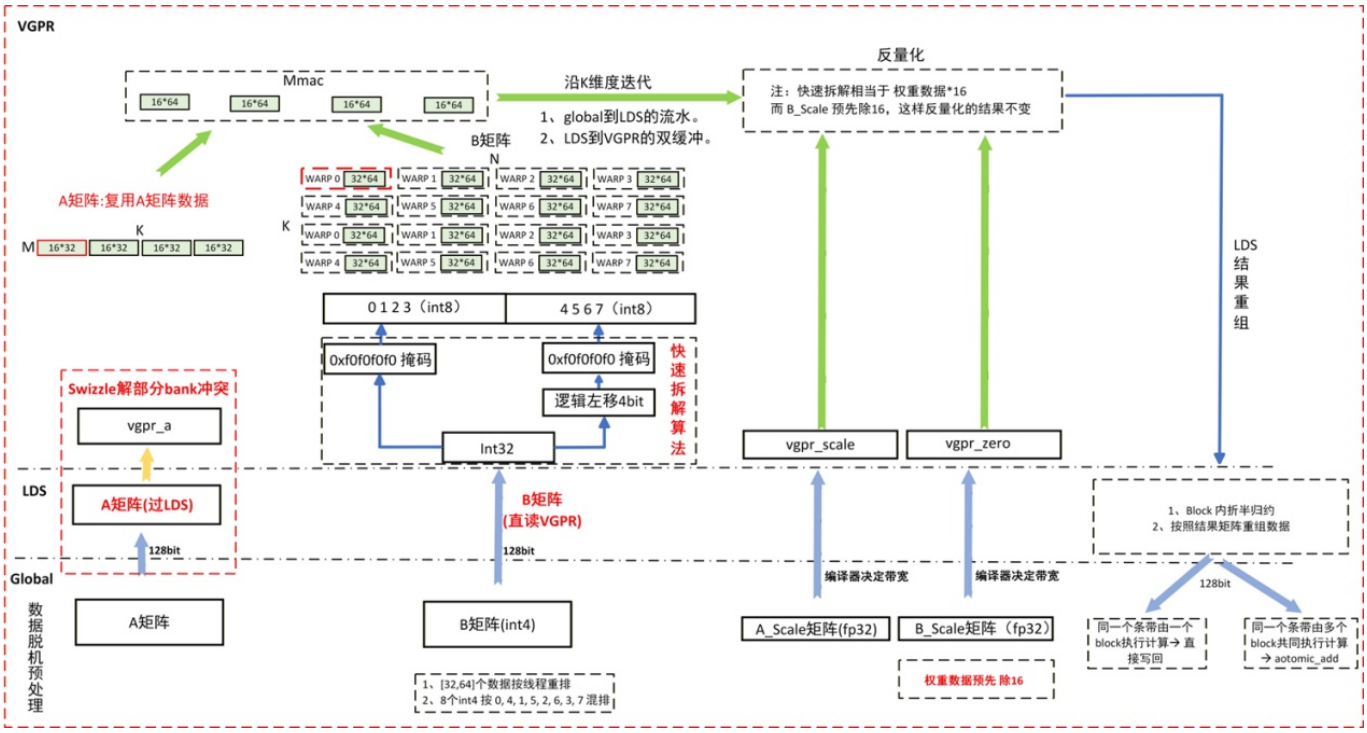
降低延迟（Latency）：实时场景（自动驾驶、工业质检、语音识别）需毫秒级响应（如≤50ms）。通过算子优化（如 C++/汇编 优化），可将关键算子耗时降低 50%
以上。例如：某自动驾驶感知模型的3D点云体素化，原生耗时 200ms，定制优化后降至
50ms，满足实时决策需求。

提升吞吐量（Throughput）：云端推理（推荐系统、AI
绘画）需处理大量请求。通过算子融合，多个算子合并为 1 个算子，减少数据读写，单卡吞吐量可提升 2-3 倍。例如：qwen3 235B优化后吞吐提升100%。

减少显存占用：大模型，通过算子优化（如激活值重计算、模型并行算子，权重低精度int4/int8/fp8量化、kvcache fp8量化），显存占用可降低75%以上。例如deeepseek
671B使用int4量化，可以使用显存64GB的GPU单机8卡运行，大大降低多机通信带来的延迟。

以fused_moe_marlin为例，对权重、访存、计算、数据搬运四大维度做深度融合与硬件级优化，最终消除冗余访存、解冲突、提升计算与搬运重叠度。

[图片]



权重重排和快速拆解算法

对权重做离线预处理与重排，适配 GPU 访存与计算 Layout

用快速拆解算法提升权重读取与分发效率

配合掩码（如 0xf0f0f0f0）做高效数据重组

LDS 访存优化解 Bank 冲突

输入 A 矩阵经过LDS（共享内存），专门解决 Bank 冲突

权重 B 矩阵不经过 LDS，直接加载，减少数据搬运开销

计算与 Load 深度重叠（Overlap）

全局内存（Global）数据加载与计算并行执行

LDS → VGPR 数据搬运与计算流水重叠

最大化掩盖访存延迟，提升硬件利用率

数据加载与计算流水化

把数据预加载、寄存器搬运、Tensor Core 计算做成连续流水线

避免计算单元空等数据，提升指令吞吐

线程与数据排布优化

按线程粒度精细化分配数据（单线程处理 16 份数据）

用逻辑左移、掩码、VGPR 排布实现高效数据重组

数据按 128bit/32bit 对齐，提升向量访存效率

6.7 通用模型DCU推理

6.7.1 MIGraphX简介

MIGraphX是一款用于DCU上的高性能深度学习推理引擎。MIGraphX能将深度学习框架（Tensorflow,Pytorch等）训练好的算法模型转换为MIGraphX
IR表示的计算图，并提供端到端的模型优化、代码生成以及推理业务部署能力。MIGraphX致力于为用户提供高性能、灵活、易用的编程接口以及配套工具，让用户能够专注于推理业务开发和部署本身
，而无需过多关注底层硬件细节，显著提高用户的开发效率。

6.7.1.1 MigraphX 适用场景

图像处理，比如分类(VGG16，ResNet50)、检测(SSD，YOLO)、分割(UNet)等

视频处理

自然语言处理，比如BERT

搜索、推荐

6.7.1.2 MIGraphX特性

支持多种精度推理，比如FP32，FP16，INT8

支持多语言API，包括C++和Python

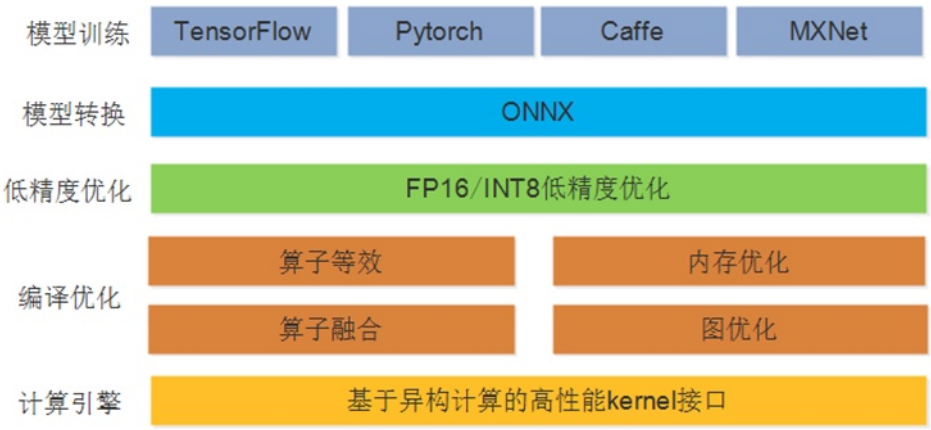
支持模型序列化

支持调试

提供性能分析工具

6.7.1.3 MIGraphX整体架构

[图片]



MIGraphX架构图

MIGraphX整体架构主要分为三层：

中间表示层：用户训练好的算法模型会统一转换为用MIGraphX IR
表示的计算图，后续的模型优化和代码生成都基于该计算图完成。

编译优化层：基于MIGraphX
IR完成各种优化，比如常量折叠，内存复用优化，算子融合，代码生成等，提高推理性能。

计算引擎层：主要包含了底层计算库的接口，包括MIOpen和rocblas。

6.7.1.4 MIGraphX IR

AI编译中的IR从层级上分一般可以分为两种类型：多级IR和单级IR。使用多级IR可以使得系统优化更加灵活，各级IR只需要负责本级优化，多级IR的代表就是MLIR，但是多级IR会带来如下的问题：

需要在不同IR之间进行转换，IR转换做到完全兼容很难而且工作量大。

不同IR转换可能带来信息的损失。

多级IR有些优化既可以在上一层IR进行， 也可以在下一层IR进行， 让系统开发者很难选择。

MIGraphX采用了单级IR的设计，MIGraphX IR以基于SSA的线性IR为基本组成形式
，这种形式的IR可以表达计算图中的控制流信息和数据依赖关系，方便后面的编译优化。

6.7.1.5 编译

MIGraphX采用静态图模式，在编译阶段，MIGrahpX会执行如下操作：

机器无关优化：比如删除公共子表达式，删除无用的代码，常量传播，常量折叠，代数化简等。

内存复用优化：MIGraphX采用了图着色的方法实现无计算依赖的节点间的内存复用，显著降低内存消耗
。

指令调度：根据计算图分析指令之间的依赖关系，根据这些依赖关系优化各指令的执行顺序，从而提高计算性能。

代码生成：MIGraphX利用代码生成技术生成算子的实现，然后利用JIT编译为可执行代码，MIGraphX主要针对访存密集型算子(比如Pointwise和Reduce)做代码生成和自动算子融合，对于计算密集型算子(比如Conv和Gemm)采用调用计算库的方式。

6.7.1.6 支持模型

目前 MIGraphX支持常用的 CNN 、LSTM 、Transformer模型：

Classification：AlexNet，VGG，Inception，ResNet，DenseNet，EfficientNet等

Detection ： SSD，YOLO，DBNet等

Segmentation ： FCN，UNet，MaskRCNN等

LSTM：CRNN等

Transformer：Vision Transformer（ViT）等

BERT：BERT-Base，BERT-Large等

GPT：GPT2等

Stable Diffusion

6.7.2 安装MIGraphX

可以选择不同的方式安装并使用MIGraphX：使用MIGraphX镜像、使用rpm安装包、使用deb安装包、使用run安装包。推荐使用镜像和run包的方式。

6.7.2.1 使用MIGraphX镜像

MIGraphX镜像下载地址：https://sourcefind.cn/#/image/dcu/migraphx?activeName=overview，根据需要选择合适的镜像，比如需要在ubuntu20.04上使用DTK24.04.1和MIGraphX4.3.0，可以通过如下步骤安装MIGraphX。

拉取镜像：

```
docker pull image.sourcefind.cn:5000/dcu/admin/base/migraphx:4.3.0-ubuntu20.04-dtk24.04.1-py3.8
```

使用上面拉取到的镜像开启并运行容器：

```
docker run --shm-size 16g --network=host --name=migraphx --privileged --device=/dev/kfd --device=/dev/dri --group-add video --cap-add=SYS_PTRACE --security-opt seccomp=unconfined -v /opt/hyhal:/opt/hyhal:ro -v <path you want to map>:/home/<path you want to map to> -it <Your Image ID> /bin/bash
```

进入容器后，在使用MIGraphX之前，需要设置容器中的环境变量：

```
source /opt/dtk/env.sh
```

如果需要在python中使用MIGraphX，还需要设置PYTHONPATH：

```
export PYTHONPATH=/opt/dtk/lib:$PYTHONPATH
```

验证MIGraphX是否可用：

执行下面的命令：

```
/opt/dtk/bin/migraphx-driver onnx -l
```

如果能显示下面的输出，则表明安装成功：

```
Where
Min
Max
Transpose
TopK
Squeeze
Softsign
Softplus
Slice
Size
Selu
Upsample
Scatter
RoiAlign
Softmax
ReduceSumSquare
ReduceLogSumExp
ReduceLogSum
ReduceSum
ReduceProd
ReduceMax
ThresholdedRelu
```

```
Range
RandomUniformLike
RandomUniform
RandomNormalLike
Resize
Greater
...
```

6.7.2.2 使用run安装包安装

本节主要介绍如何使用 run包安装 MIGraphX。

6.7.2.2.1 dtk安装

安装MIGraphX之前，需要先安装dtk，有两种方式安装dtk：

使用dtk镜像（推荐），镜像中已经安装好dtk环境，dtk镜像下载地址：https://sourcefind.cn/#/image/dcu/dtk?activeName=overview，根据需要选择合适的镜像。

下载dtk安装包，dtk安装包下载地址：https://download.sourcefind.cn:65024/1/main，根据需要选择合适的版本，然后将下载好的安装包安装到/opt目录下，最后创建一个软连接/opt/dtk，使得该软连接指向dtk的安装目录，注意：一定要创建软连接/opt/dtk，否则MIGraphX无法正常使用。

例如，如果是ubuntu系统，下载DTK-23.10.1-Ubuntu20.04.1-x86_64.tar.gz，并解压：

```
tar -zxvf DTK-23.10.1-Ubuntu20.04.1-x86_64.tar.gz -C /opt
```

创建软连接：

```
ln -s /opt/dtk-23.10.1/ /opt/dtk
```

6.7.2.2.2 下载安装包并安装MIGraphX

MIGraphX的安装包下载地址：https://download.sourcefind.cn:65024/4/main/migraphx，选择合适的安装包下载，比如下载了DAS1.1中的MIGraphX4.3.0版本，可以通过如下命令安装：

```
bash migraphx-4.3.0+das1.1.git0a2480f.abi0.dtk2404-cp38-cp38-linux_x86_64.run
```

安装时会出现两次询问，第一次询问为

```
请输入安装路径（直接回车将安装到默认路径: /opt/dtk）
```

一般情况下，默认安装到/opt/dtk目录下即可。第二次询问为：

```
是否安装migraphx-diffusers?（安装需要环境pip能访问网络。默认不安装，直接回车将不进行安装） [y/n]:
```

migraphx-diffusers是一款支持文生图任务的Python插件，其使用可参考 [13 Stable Diffusion 支持](#13 Stable Diffusion

支持)。如果需要使用MIGraphX执行文生图任务，则建议安装该插件。如果安装过程报错，可能原因是设备无网络，无法安装migraphx-diffusers库所需的依赖。

6.7.2.2.3 设置环境变量

安装完成后，需要设置环境变量：

```
source /opt/dtk/env.sh
```

如果需要在python中使用migraphx，还需要设置PYTHONPATH：

```
export PYTHONPATH=/opt/dtk/lib:$PYTHONPATH
```

6.7.2.2.4 验证是否安装成功

执行下面的命令：

```
/opt/dtk/bin/migraphx-driver onnx -l
```

如果能显示下面的输出，则表明安装成功：

```
Where
Min
Max
Transpose
TopK
Squeeze
Softsign
Softplus
Slice
Size
Selu
Upsample
Scatter
RoiAlign
Softmax
ReduceSumSquare
ReduceLogSumExp
ReduceLogSum
ReduceSum
ReduceProd
ReduceMax
ThresholdedRelu
Range
RandomUniformLike
RandomUniform
RandomNormalLike
Resize
```

```
Greater
...
```

6.7.3 MIGraphx推理

本章以ResNet50为例来说明如何通过C++ API加载ONNX模型进行图像分类模型的推理。

6.7.3.1 将模型转换为ONNX格式

使用MIGraphX进行推理前需要将训练好的ResNet50模型转换为ONNX格式，本示例使用如下的ResNet50模型：https://download.pytorch.org/models/resnet50-19c8e357.pth，下载该模型后使用如下代码可以转换为ONNX格式（本示例代码基于Pytorch1.10）：

```
# Pytorch模型文件
pathOfPytorchModel = "resnet50-19c8e357.pth"
# 创建ResNet50模型
net = torchvision.models.resnet50(pretrained=False)
# 定义输入
input = torch.randn(32,3,224,224)
# 生成的ONNX模型的路径
pathOfONNX = "ResNet50.onnx"
net.load_state_dict(torch.load(pathOfPytorchModel))
net.eval()
# 导出ONNX模型
torch.onnx.export(net,input,pathOfONNX,input_names = ["input"])
```

生成好ResNet50.onnx模型后就可以进行推理了，如果没有特殊说明本教程使用的ResNet50模型都是使用的该模型。

6.7.3.2 使用C++ API进行分类模型的推理

使用C++ API加载ONNX模型进行推理主要包含三个步骤：

加载ONNX模型

编译模型

执行推理

主要步骤示例代码如下：

```
// 头文件
#include <migraphx/onnx.hpp>
#include <migraphx/gpu/target.hpp>

// 加载模型
```

```
migraphx::program net = migraphx::parse_onnx("path/to/your/onnx/model");

// 编译模型
migraphx::compile_options options;
net.compile(migraphx::gpu::target{},options);

// 加载数据
std::unordered_map<std::string, migraphx::argument> inputData;
...

// 执行推理
std::vector<migraphx::argument> results = net.eval(inputData);
```

下面以ResNet50.onnx模型为例说明如何使用C++ API进行分类模型的推理：

```
#include <migraphx/onnx.hpp>
#include <migraphx/gpu/target.hpp>
#include <opencv2/opencv.hpp>

int main(int argc, char *argv[])
{
  // 加载模型
  migraphx::program net = migraphx::parse_onnx("ResNet50.onnx");

  // 获取模型输入/输出节点信息
  std::cout << "inputs:" << std::endl;
  std::unordered_map<std::string, migraphx::shape> inputs = net.get_inputs();
  for (auto i : inputs)
  {
    std::cout << i.first << ":" << i.second << std::endl;
  }
  std::cout << "outputs:" << std::endl;
  std::unordered_map<std::string, migraphx::shape> outputs = net.get_outputs();
  for (auto i : outputs)
  {
    std::cout << i.first << ":" << i.second << std::endl;
  }
  std::string inputName = inputs.begin()->first;
  migraphx::shape inputShape = inputs.begin()->second;
  int N = inputShape.lens()[0];
  int C = inputShape.lens()[1];
  int H = inputShape.lens()[2];
  int W = inputShape.lens()[3];

  // 编译模型
  migraphx::compile_options options;
  options.device_id = 0; // 设置GPU设备，默认为0号设备
  options.offload_copy = true;
  net.compile(migraphx::gpu::target{}, options);
```

```
  // 数据预处理并转换为NCHW格式
  int batchSize = N;
  cv::Mat srcImage = cv::imread("Test.jpg");
  std::vector<cv::Mat> srcImages;
  for (int i = 0; i < batchSize; ++i)
  {
    srcImages.push_back(srcImage);
  }
  cv::Mat inputBlob;
  cv::dnn::blobFromImages(srcImages, inputBlob, 0.0078125, cv::Size(W, H), cv::Scalar(127.5, 127.5,
127.5), false, false);

  // 创建输入数据
  std::unordered_map<std::string, migraphx::argument> inputData;
  inputData[inputName] = migraphx::argument{inputShape, inputBlob.data};

  // 推理
  std::vector<migraphx::argument> results = net.eval(inputData);

  // 获取输出节点的属性
  migraphx::argument result = results[0];                // 获取第一个输出节点的数据
  migraphx::shape outputShape = result.get_shape();      // 输出节点的shape
  std::vector<std::size_t> outputSize = outputShape.lens(); // 每一维大小，维度顺序为(N,C,H,W)
  int numberOfOutput = outputShape.elements();           // 输出节点元素的个数
  float *resultData = (float *)result.data();            // 输出节点数据指针

  // 打印推理结果
  for (int i = 0; i < numberOfOutput; ++i)
  {
    std::cout << resultData[i] << ",";
  }
  std::cout << std::endl;

  return 0;
}
```

首先通过parse_onnx()函数加载onnx模型，加载好模型之后，可以通过program的get_inputs()和get_outputs()函数获取模型的输入和输出节点信息，两个函数返回值类型都是std::unordered_map类型，每个输入节点或者输出节点都有一个节点名和shape，由于ResNet50模型只有一个输入，所以这里可以通过inputs.begin()获取模型的输入。

如果需要采用FP16模式进行推理，可以通过quantize_fp16()函数实现，具体实现见4.3小节，MIGraphX同时也支持int8推理，具体实现见4.4小节。

加载onnx模型之后，需要使用compile()方法编译模型，这里将模型编译为GPU模式，如果需要编译为CPU模式，需要使用migraphx::cpu::target{}，注意：如果你的输入数据在host端，则在设置编译选项的时候，需要设置offload_copy为true。第6章会说明如何直接使用device端的数据进行推理。

编译好模型之后，需要输入数据，输入数据需要经过预处理并转换为NCHW的格式，这里使用了OpenCV的blobFromImage函数将图像转换为了NCHW格式。并通过构造一个std::unordered_map类型的对象保存输入数据，模型的每个输入都会对应一个输入数据，输入数据使用argument保存。

最后通过program的eval()方法执行推理计算，推理结果是一个std::vector< migraphx::argument >类型，由于offload_copy参数设置为true，所以推理的结果是host端数据，然后我们就可以通过argument提供的方法访问推理结果了，如果需要指定输出节点，可以设置eval()中的outputNames参数，具体实现参考4.7小节。

完整的ResNet50示例程序参考[ModelZoo](https://www.sourcefind.cn/)。

6.7.3.3 使用FP16推理

本节主要说明如何在MIGraphX中使用FP16进行推理，在MIGraphX中可以通过下面两种方式实现FP16推理：

方式1：使用FP32格式的ONNX模型，然后通过调用migraphx::quantize_fp16()实现FP16的推理

方式2：先将FP32格式的ONNX模型转换为FP16格式模型，然后使用FP16格式的ONNX模型执行推理

下面是两种方式的具体使用说明。

6.7.3.3.1 使用migraphx::quantize_fp16

实现FP16推理可以直接使用FP32格式的ONNX模型，然后在编译前调用migraphx::quantize_fp16()。这种方式的优点是不需要转换模型格式，只需要修改少量代码，推荐使用该方式。具体使用方法如下：

```
#include <migraphx/quantization.hpp> // FP16头文件
// 使用FP16
migraphx::quantize_fp16(net);
// 编译模型(下面步骤跟FP32推理相同)
...
```

6.7.3.3.2 将模型转换为FP16格式

除了调用quantize_fp16的方式外，还可以通过将模型转换为FP16格式来实现FP16的推理。通过下面的方法可以将FP32格式的模型转换为FP16格式：

安装onnx和[onnxconverter-common](https://github.com/microsoft/onnxconverter-common)

```
pip install onnx onnxconverter-common
```

通过convert_float_to_float16函数转换模型

```
import onnx
from onnxconverter_common import float16
 model = onnx.load("path/to/model.onnx")
```

```
model_fp16 = float16.convert_float_to_float16(model)
onnx.save(model_fp16, "path/to/model_fp16.onnx")
```

转换好之后，可以直接使用model_fp16.onnx文件进行FP16的推理，使用方式与FP32推理一致，注意：由于模型为FP16格式，所以输入数据需要转换为FP16类型。

注：这种方式可能会导致MIGraphX加载FP16格式的模型报错，如果加载FP16格式的模型报错，可以采用4.3.1中的方式实现FP16的推理。

6.7.3.4 使用随机数作为模型输入

有的时候我们希望使用随机数作为模型的输入，MIGraphX提供了生成随机数的函数migraphx::generate_argument，使用方法如下：

```
migraphx::argument data = migraphx::generate_argument(inputShape);
```

返回的data就是一个包含随机数的argument，可以作为模型的输入。

6.7.3.5 查看推理需要的显存大小

如果需要查看模型推理过程中需要使用的显存大小，可以使用下面的方法：

```
...
// 编译模型
net.compile(migraphx::gpu::target{},options);
// 查看显存，单位为字节
std::size_t memoryUsage = net.get_memory_usage();
```

6.7.3.6 在Python中使用MIGraphX

通过以上章节知道如何使用C++ API进行分类模型的推理，本章介绍如何在python中使用MIGraphX。

6.7.3.6.1 设置环境变量

将MIGraphX库路径加入PYTHONPATH：

```
export PYTHONPATH=/opt/dtk/lib:$PYTHONPATH
```

6.7.3.6.2 在Python中使用MIGraphX

下面的示例展示了如何使用python进行ResNet50分类模型的推理。

```
# -*- coding: utf-8 -*-
import cv2
```

```
import numpy as np
import migraphx

def ReadImage(pathOfImage,inputShape):
  srcImage = cv2.imread(pathOfImage, cv2.IMREAD_COLOR)

  # resize并转换为CHW
  resizedImage = cv2.resize(srcImage,(inputShape[3], inputShape[2]))
  resizedImage_Float = resizedImage.astype("float32") # 转换为float32
  srcImage_CHW = np.transpose(resizedImage_Float, (2, 0, 1)) # 转换为CHW

  # 预处理
  mean = np.array([127.5, 127.5, 127.5])
  scale = np.array([0.0078125, 0.0078125, 0.0078125])
  inputData = np.zeros(inputShape).astype("float32") # NCHW
  for i in range(srcImage_CHW.shape[0]):
    inputData[0,i, :, :] = (srcImage_CHW[i, :, :] - mean[i]) * scale[i]

  for i in range(inputData.shape[0]):
    if i!=0:
      inputData[i,:, :, :]=inputData[0,:, :, :]

  return inputData

if __name__ == '__main__':
  # 加载模型
  model = migraphx.parse_onnx("ResNet50.onnx")

  # 获取模型输入输出节点信息
  print("inputs:")
  inputs=model.get_inputs()
  for key,value in inputs.items():
    print("{}:{}".format(key,value))
  print("outputs:")
  outputs=model.get_outputs()
  for key,value in outputs.items():
    print("{}:{}".format(key,value))

  inputName=list(model.get_inputs().keys())[0]
  inputShape=inputs[inputName].lens()

  # 编译模型
  model.compile(t=migraphx.get_target("gpu"),device_id=0) # device_id: 设置GPU设备，默认为0号设备

  # 数据预处理并转换为NCHW格式
  pathOfImage ="Test.jpg"
  image = ReadImage(pathOfImage,inputShape)
```

```
# 推理
results = model.run({inputName:image})

# 获取输出节点属性
result=results[0] # 获取第一个输出节点的数据,migraphx.argument类型
outputShape=result.get_shape() # 输出节点的shape,migraphx.shape类型
outputSize=outputShape.lens() # 每一维大小，维度顺序为(N,C,H,W),list类型
numberOfOutput=outputShape.elements() # 输出节点元素的个数

# 转换为numpy
result = np.array(results[0])

# 打印结果
print(result)
```

Python程序的流程与C++基本一致

更多Python示例程序参考[ModelZoo](https://www.sourcefind.cn/)。

6.7.3.6.3 在Python中使用opt_compile编译

如果想要获取模型在不同数据布局下的最优性能，只需要将编译语句compile替换为opt_compile即可

```
# 例如，在fp16下获得最优性能的计算图，opt_compile传入编译的精度选
precision=migraphx.target_precision.fp16
# 默认会在nhwc，nchw32，和没有任何融合的布局之间选择最优融合策略
migraphx.quantize_fp16(model)
model = migraphx.opt_compile(model,t=migraphx.get_target("gpu"),
offload_copy=False,precision=migraphx.target_precision.fp16)
# 如果想要指定在无融合和nhwc之间选择，可以指定fusion_mode编译选项
model = migraphx.opt_compile(model,t=migraphx.get_target("gpu"),
offload_copy=False,precision=migraphx.target_precision.fp16,mode=migraphx.fusion_mode.nhwc)
...
```

6.7.3.6.4 在Python中使用指定优化编译选项编译

如果想要获取模型在指定数据布局下编译，可以给compile传入相应的编译选项。

```
// 在cpp中，编译选项定义为如下
struct compile_options
{
    bool offload_copy = false;
    bool fast_math       = true;
    bool exhaustive_tune  = false;
    std::size_t device_id = 0;
    enum class fusion_mode {
        none,
```

```
    nchw32_int8,
    nchw32_fp16,
    nhwc
  };
  fusion_mode mode = fusion_mode::none;
  enum class target_precision { fp32, fp16, int8};
  target_precision precision = target_precision::fp32;

  tracer trace{};
};
// 传入compile的compile_options可以设置为：
compile_options options;
options.mode = compile_options::fusion_mode::nhwc;
options.precision = compile_options::target_precision::fp16;
...
```

python中使用如下

```
migraphx.quantize_fp16(model)
precision=migraphx.target_precision.fp16
mode=migraphx.fusion_mode.nhwc
# 使用opt_compile
model = migraphx.opt_compile(model,t=migraphx.get_target("gpu"),
offload_copy=False,precision=precision,mode=mode)
# 使用compile
model.compile(t=migraphx.get_target("gpu"), offload_copy=False,precision=precision,mode=mode)
...
```

6.7.3.6.5 在Python中使用FP16推理

如果需要在python中使用FP16进行推理，只需要在编译前面加上如下语句即可：

```
# 使用FP16
migraphx.quantize_fp16(model)
# 编译模型
...
```

6.7.3.6.6 在Python中获取MIGraphX版本号

在Python中可以通过 version
属性来获取MIGraphX版本号，在命令行中输入如下命令可以打印出MIGraphX版本号：

```
>>> import migraphx
>>> migraphx.__version__
```

6.7.4 算子级卷积优化实战

6.7.4.1 算子结构定义

所有 conv 类算子都定义在 src/include/migraphx/op/ 下的头文件中，核心是 POD-like struct，通过 reflect 暴露成员，实现几个约定接口：

1.convolution类型

MIGraphX/src/include/migraphx/op/convolution.hpp为例：

struct convolution

{

std::vectorstd::size_t padding = {0, 0};

std::vectorstd::size_t stride = {1, 1};

std::vectorstd::size_t dilation = {1, 1};

int group = 1;

padding_mode_t padding_mode = default_;

template

static auto reflect(Self& self, F f)

{

return pack(f(self.padding, “padding”),

f(self.stride, “stride”),

f(self.dilation, “dilation”),

f(self.group, “group”),

f(self.padding_mode, “padding_mode”));

}

std::string name() const { return “convolution”; }

value attributes() const { return {{“normalize_padding”, “padding”}}; }

shape normalize_compute_shape(std::vector inputs) const { … }

argument compute(shape output_shape, std::vector args) const

{

…

migraphx::convolution(output, input, weights, new_padding, stride, dilation, group);

return result;

}

};

关键接口：

name()：全局唯一算子名字符串，用于 make_op(“convolution”) 查找。

reflect：序列化/反序列化成员变量，支持自动的 to_value / from_value。

attributes()：附加元数据，例如 “normalize_padding” 告诉框架在 shape 计算前要把 padding 做 normalize。

normalize_compute_shape()：根据输入 shape 和成员属性推导输出 shape。

compute()：参考实现（Reference implementation），在 Ref target 或缺少硬件后端时直接执行。

2. quant_convolution类型

结构与 convolution 几乎相同，但：

name() 返回 “quant_convolution”。

normalize_compute_shape() 会检查输入类型必须是 int8_type 或 fp8 类型，输出为 int32_type（int8 输入时）或 float_type（fp8 输入时）。

attributes() 里额外标注了 {“general_data_type”, “convolution”}。

3. convolution_backwards类型

结构类似，但 shape 推导公式对应反卷积（stride × (input - 1) + …），name() 为 “convolution_backwards”。

6.7.4.2 核心计算模板

MIGraphX/src/include/migraphx/convolution.hpp为例：

所有 conv 算子的 compute() 最终都调用同一个模板函数：

template

void convolution(Output output, T input, T weights, Padding padding, Stride stride, Dilation dilation, int group)

{

par_for(output_shape.elements(), [&](auto i) {

// 按 (N, C_out, *spatial) 遍历输出每个点

// 滑动窗口计算 dot(input_window, weights)

});

}

这是一个通用的 N-D 组卷积参考实现，使用 par_for 做并行，shape_for_each 遍历卷积窗。

6.7.4.3 算子注册到全局表

MIGraphX 没有手动写一个巨大的 switch-case，而是通过 静态自动注册 把算子 struct 注册到全局 op_map：

MIGraphX/src/include/migraphx/register_op.hpp

template

void register_op()

{

register_op_init();

static auto op_h = detail::op_handler(T{});

register_op(op_h.op);

}

struct register_op_action

{

template

static void apply() { register_op(); }

};

template

using auto_register_op = auto_register;

#define MIGRAPHX_REGISTER_OP(…) MIGRAPHX_AUTO_REGISTER(register_op_action, VA_ARGS)

在编译时，RegisterOp.cmake 会扫描 src/include/migraphx/op/*.hpp，为每个算子生成一条 MIGRAPHX_REGISTER_OP(op::xxx) 的注册代码，并编译进一个 .cpp 中。因此只要新建一个符合约定的 struct 放在 op/ 目录下，就会自动注册。

注册后，就能通过 make_op(“convolution”) 全局构造该算子实例。

6.7.4.3.1前端解析器中添加 Conv 节点

以 ONNX 为例，MIGraphX 把 ONNX 的 Conv、ConvInteger、NhwcConv 映射到内部算子。

1. 解析器 struct

MIGraphX/src/onnx/parse_convolution.cpp

struct parse_convolution : op_parser

{

std::vector operators() const

{

return {{“Conv”，“convolution”},

{“ConvInteger”，“quant_convolution”},

{“NhwcConv”，“convolution”}};

}

instruction_ref parse(const op_desc& opd,

const onnx_parser& parser,

onnx_parser::node_info info,

std::vector args) const

{

auto op = make_op(opd.op_name); // 构造 convolution 或 quant_convolution

auto values = op.to_value();

auto x = args[0];

auto weights= args[1];

// 1. 解析 strides / dilations / pads / auto_pad / group 等 ONNX 属性

// 2. 填充到 values[“stride”], values[“padding”], values[“group”] …

// 3. 处理量化相关的 zero_point、symmetric shift 等（仅 quant_conv）

// 4. 处理 NhwcConv 的 layout transpose

op.from_value(values);

ret = info.add_instruction(op, x, weights);

// 普通卷积：如果有 bias（args[2]），通过 add_bias 加入

// 量化卷积：通过 handle_quant_bias 处理 zero_point 残差

return ret;

}

};

op_parser 基类会自动调用 register_op_parser()，把该解析器注册到 ONNX parser 的查找表中。当 ONNX 图里出现 Conv 节点时，就会走到这个 parse() 函数，最终调用：

info.add_instruction(op, x, weights);

把 convolution / quant_convolution 算子插入到 migraphx::module 的指令流中。

6.7.4.4 后端 Lowering（Target 映射）

算子被加入 IR（module）后，在编译阶段会根据目标硬件（Ref / GPU / CPU）做 lowering。

1. Ref Target

MIGraphX/src/targets/ref/lowering.cpp

Ref backend 没有为 convolution 单独写一个 wrapper，而是走通用路径：

struct ref_op

{

operation op = op::identity{};

argument compute(context&, const shape& output_shape, const std::vector& args) const

{

return op.compute(output_shape, args); // 直接调用 op::convolution::compute

}

};

lowering 时，所有未被特殊处理的算子都会通过 apply_ref_op(ins) 包成 ref_op{ins->get_operator()}，因此 convolution 会直接执行其 compute() 中的参考实现。

2. GPU Target (MIOpen)

MIGraphX/src/targets/gpu/lowering.cpp

GPU backend 把 convolution、convolution_backwards、quant_convolution 映射为 gpu::miopen_op 的 wrapper：

void add_convolution_op(const std::string& name)

{

```
apply_map.emplace(name, [=](instruction_ref ins) {
// 1. 构造 gpu::convolution 算子，保存原始 op 属性
operation conv = make_op(“gpu::” + name, {{“op”, ins->get_operator().to_value()}});
// 2. 插入输出 buffer 分配
auto output = insert_allocation(ins, ins->get_shape());
// 3. 替换为 gpu::miopen_op，后续由 MIOpen 编译
return mod->replace_instruction(ins,
make_op(“gpu::miopen_op”, {{“op”, to_value(conv)}}),
ins->inputs().at(0),
ins->inputs().at(1),
output);
});
}
```

随后 compile_miopen.cpp 会把 gpu::miopen_op 进一步编译为具体的 MIOpen convolution descriptor，并在运行时调用 MIOpen kernel。

6.8 AI 编译器编程

6.8.1 AI编译器提出的背景

6.8.1.1 深度学习计算的挑战

随着深度学习模型规模的急剧扩张，从早期的 AlexNet 到如今的 GPT-4、LLaMA 等超大规模语言模型（LLM），模型的参数量已从数百万飙升至数千亿甚至万亿规模。这一趋势带来了前所未有的计算挑战，传统的软件工程与手工优化方法已逐渐触及天花板。

在深度学习发展的早期，工程师通常直接调用 NVIDIA 提供的 cuBLAS、cuDNN 等高度优化的底层算子库来完成矩阵乘法、卷积等核心运算。然而，随着算法研究的深入，计算的瓶颈逐渐从纯粹的浮点运算能力（FLOPs）转移到了显存带宽（Memory Wall）上。为了突破这一瓶颈，学术界和工业界涌现出大量新颖的算子结构，如 Flash Attention、稀疏注意力（Sparse Attention）、混合专家（MoE）机制等。

这些复杂的算子往往涉及密集的显存读写与非标准的计算模式，难以直接复用现有的标准算子库。如果

依然依赖专家级的 CUDA
工程师从零开始进行手工优化，不仅开发周期长达数周乃至数月，且极易引入隐蔽的并发
Bug，严重拖慢了算法创新的迭代速度。

6.8.1.2 硬件生态的多样化

除了算法复杂度的增加，底层硬件生态的日益碎片化也是推动 AI 编译器发展的重要驱动力。现代 AI 计算早已不再由单一厂商垄断，各类异构硬件加速器层出不穷：

通用 GPU（NVIDIA、AMD）：通用并行计算的霸主，拥有最成熟的软件生态（CUDA/ROCm）。

专用张量处理器（Google TPU、AWS Trainium）：专为深度学习矩阵运算和大规模集群通信设计的 ASIC 芯片。

端侧与边缘 NPU（苹果 Neural
Engine、高通、华为昇腾）：集成于移动设备，追求极致的能效比（Perf/Watt）。

FPGA：具备高度可编程性，常用于超低延迟的特定推理场景。

国产 AI 芯片（海光、寒武纪、燧原、摩尔线程等）：在自主可控战略下蓬勃发展的多元化算力生态。

每类硬件都拥有截然不同的指令集架构（ISA）、内存层次结构和并行执行模型。如果为每款硬件单独手写并维护一套算子库，不仅工程成本呈指数级上升，还会导致模型被锁定在特定硬件上。AI
编译器的出现，正是为了跨越这些“算力孤岛”，实现「一次编写，多端部署」的工程诉求。

6.8.1.3 AI 编译器的演进历程

AI
编译器的发展经历了若干个关键阶段，每一次范式跃迁都代表着业界对“深度学习计算优化”认知的深化：

第一阶段：图优化编译器 (Graph-level Compilers)

代表作：TensorFlow XLA、ONNX Runtime、TensorRT。

核心技术：在粗粒度的计算图（Computational
Graph）层面进行宏观优化。主要手段包括算子融合（Operator Fusion）、常量折叠（Constant Folding）和死代码消除。

局限性：优化空间受限于底层已有算子库的覆盖范围。如果遇到完全创新的算子，图编译器往往无能为力，无法突破底层手工 Kernel 的性能上限。

第二阶段：张量编译器 (Tensor-level Compilers)

代表作：TVM、Halide。

核心技术：引入了“算法（Compute）与调度（Schedule）分离”的编程范式。允许开发者在统一的中间表示（IR）上描述算子语义，随后通过自动调优（Auto-tuning）在目标硬件上搜索最优的循环展开、

分块（Tiling）和向量化等调度策略。

局限性：极大地降低了跨硬件部署成本，但在面对极度复杂的算子时，调优过程极其耗时，且寻找全局最优解的难度极大，往往仍需编译器专家介入。

第三阶段：Kernel 级抽象范式 (Kernel-level Abstractions)

代表作：Triton（由 OpenAI 推广并广泛应用）。

核心技术：开创了一种全新的 GPU 核函数编程范式。它以 Python 为前端语言，在「程序块」（Tile/Block）抽象层次上提供编程模型，隐去了复杂的线程同步和共享内存管理。

产业影响：Triton 构建了多层中间表示（从 Triton IR 到 Triton GPU IR，再映射至 LLVM IR），兼顾了极高的开发效率与极致的执行性能。目前，它已被 PyTorch 2.0 引入的默认后端 TorchInductor 广泛采用，极大地推动了 AI 编译器在工业界算子开发中的普及。

关键洞察 AI 编译器的核心价值在于将「算法创新」与「硬件微架构优化」彻底解耦。它让算法研究员可以专注于数学表达，让编译器替人类接管繁琐的底层映射与流水线编排，从而以最低的成本压榨出硬件的极限算力。

6.8.1.4 AI 编译器的技术栈全景

现代 AI 编译器并非一个简单的翻译黑盒，而是由多个高度解耦的抽象层次构成的复杂编译流水线（Compilation Pipeline）。一段人类可读的深度学习代码（如 Python），需要经历如同“漏斗”般的层层解析与转化，最终才能变成驱动硬件硅片运转的电信号指令。

为了管理这种极其复杂的转化过程，业界广泛采纳了分层架构。以下是该流水线的核心层次与关键职责：

层次	代表技术	主要职责
前端（Frontend）	PyTorch FX、JAX tracing、Triton DSL	捕获计算图结构或提取核函数的学语义，剥离宿主语言（如 Python）的冗余信息。
中间表示（IR）	MLIR、StableHLO、Triton IR	提供多层次的语法抽象，作为衔前后端的核心枢纽，承载各类优计算。
优化 Pass	内存分析、循环变换、指令流水	在 IR 层面执行架构无关或架构关的优化，提升计算强度与内存部性。
代码生成（Codegen）	LLVM、NVPTX、AMDGPU ISA、MSL	将优化后的目标 IR 转换为目标件的底层机器码或汇编指令。

运行时（Runtime）	CUDA Runtime、ROCm HSA	负责异构设备间的内存拷贝、Kernel 调度执行以及多（Stream）并发管理。

前端（Frontend）语义捕获

前端的主要任务是“听懂”开发者想做什么。在图编译器中，前端（如 PyTorch FX 或 JAX jaxpr）会追踪整个神经网络的前向与反向传播过程，将其转化为一个有向无环图（DAG）。而在 Triton 这样的 Kernel 编译器中，前端（Triton DSL）则通过 Python 的抽象语法树（AST），精准捕获函数内部的数据块切片和数学运算逻辑，同时摒弃掉 Python 本身的动态类型开销。

中间表示（IR）与渐进式降级

IR 是 AI 编译器的“通用语言”。传统的编译器通常只有一种 IR，但现代 AI 编译器（尤其是依托 LLVM/MLIR 基础设施的编译器）普遍采用渐进式降级（Progressive Lowering）策略。 以 Triton 为例，代码并不会一步到位变成机器码，而是要经历多级降维：

高层 IR（Triton IR）：保留了数据块（Block）和张量操作的语义，独立于具体硬件，方便进行通用的算法逻辑优化。

底层 IR（Triton GPU IR）：引入了特定硬件的并行概念（如 Warp 划分、共享内存布局），此时代码的形态已经为目标 GPU 量身定制。

优化 Pass (Optimization)

在每一层 IR 上，编译器都会执行一系列被称为 “Pass” 的代码变换插件。

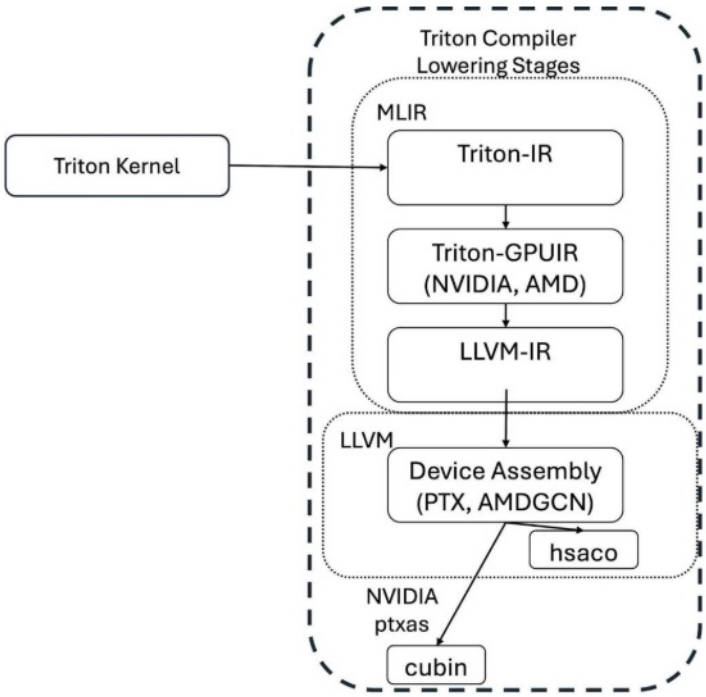
架构无关 Pass：例如死代码消除（去除没有被使用的计算结果）、常量折叠（在编译期提前计算好常数部分）。

架构相关 Pass：例如访存合并（将内存密集的读写操作合并，以打破“内存墙”）、自动流水线编排（Software Pipelining，隐藏内存延迟）。这些 Pass 是编译器能够媲美甚至超越人类手工优化的核心所在。

代码生成与运行时调度

经过重重优化的 IR 最终会被交给 LLVM 等后端基础设施，生成诸如 NVIDIA 的 PTX 或 AMD 的 AMDGCN 等底层汇编代码。最后，通过宿主机的运行时（Runtime） API，将这些编译好的二进制模块动态加载到 GPU 显存中，并分配相应的计算网格（Grid）启动执行。

[图片]



6.8.2 Triton 简介

6.8.2.1 Triton 的诞生背景

在 Triton 出现之前，编写高性能 GPU 核函数需要深入掌握 CUDA C++ 的线程层次结构（Grid/Block/Thread）、共享内存管理、Warp 级同步原语等底层细节。这对于深度学习研究人员而言门槛极高，严重制约了新型算子从理论构想到工程落地的速度。

Triton 的核心理念最早由 Philippe Tillet 在哈佛大学攻读博士期间提出并完成原型开发。其代表性论文《Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations》发表于 2019 年的 MAPL 研讨会。随后，Philippe 加入 OpenAI 并持续推进 Triton 的工程化发展。2021 年，OpenAI 将 Triton 正式开源，使其逐步走向深度学习工业界。该工作提出了一种以「分块」（Tile/Block）为核心抽象的 GPU 核函数编程语言，旨在弥合高层深度学习框架与底层 CUDA 编程之间的鸿沟。

6.8.2.2 Triton 的设计目标

Triton 的设计精准切中了 AI 工程界的痛点，遵循以下核心原则：

生产力优先 (Python 友好)：全盘采用 Python 语法（借助 AST 解析）。让熟悉 NumPy 和 PyTorch 的算法研究员能够以最符合直觉的向量化思维编写核函数，摆脱 C++ 繁琐的编译构建流程。

隐式并发与性能逼近手工 CUDA：开发者无需显式管理线程。Triton

编译器会自动在底层处理共享内存的分配与调度、异步内存传输（如 Tensor Core
所需的访存流水线）以及 Warp 级指令优化，使生成的机器码性能逼近甚至偶尔超越手工 CUDA 实现。

跨硬件可移植性 (基于 MLIR 生态)：Triton 构建在现代化的 MLIR 编译器基础设施之上。其前端 Python 代码会被转换为硬件无关的 Triton IR，随后降级为包含硬件特性的 Triton GPU IR，最终通过 LLVM 为 NVIDIA（PTX）、AMD（AMDGPU）甚至各种国产 AI
芯片生成机器码，实现真正的「一次编写，到处运行」。

极致的可组合性：Triton 核函数可与 PyTorch 的 Tensor 无缝互操作，并原生支持通过 torch.autograd 实现自动微分机制，完美融入现有的训练流水线。

6.8.2.3 Triton 在深度学习生态中的定位

Triton 在现代 AI
软件栈中处于“承上启下”的关键腰部位置。为了理解其价值，可以从宏观的生态定位与微观的技术范式两个维度来剖析。

宏观生态：承上启下的编译器枢纽

上层赋能高阶框架：PyTorch 2.0 引入的默认图编译器后端 TorchInductor，正是通过自动生成 Triton 代码来实现图层面的算子融合与极致加速。

下层映射底层硬件：Triton 编译器将前端 Python 代码转换为硬件无关的 Triton
IR，随后降级为包含硬件特性的 Triton GPU IR，最终通过 LLVM 为
NVIDIA（PTX）、AMD（AMDGPU）等硬件生成机器码。

抽象层级	代表技术	抽象层次	性能可控性	开发效率
高层框架	PyTorch/TensorFlow	算子级	低	高
中间编译器	Triton DSL	Tile级	中高	中高
底层内核	CUDA C++/HIP	线程级	极高	低

从横向工业界应用来看，Triton 已被广泛用于以下场景：

Flash Attention 系列（Flash Attention 1/2/3）：高效注意力机制的 Triton 实现

矩阵乘法（GEMM）：等价于 cuBLAS 的高性能实现

激活函数融合：GeLU、SiLU 与前后矩阵乘法的融合核函数

归一化算子：LayerNorm、RMSNorm 的高效实现

稀疏计算：稀疏注意力、MoE 路由等非规则计算

微观范式：Triton 与底层 CUDA 的核心差异

Triton 虽构建于对底层硬件的抽象之上，但其编程范式与原生的 CUDA C++
存在本质差异。简而言之，传统的 CUDA 编程要求开发者必须具备深厚的 GPU

微架构思维，以进行极度精细的软硬件资源控制；而 Triton
则将这种面向硬件底层的调度工作交由智能化编译器代劳。

区别于 CUDA 直接向开发者暴露的「单指令多线程」（SIMT）标量计算模型，Triton
确立了一种以「张量块」（Tile /
Block）为核心抽象的编程范式。在此模型下，开发者只需聚焦于高维度的数据分块逻辑，而无需陷入复杂的线程间交互与内存生命周期管理。

两者在核心技术维度的详细对比如下：

特性	CUDA C++	Triton
编程界面与生态	依赖 C++ 及其扩展体系，需借 NVCC 独立编译流水线。	采用 Python 领域特定语（DSL），基于 JIT 实时编译，PyTorch 生态无缝融合。
并行抽象粒度	极细粒度的单线程（Thread）量级操作。	较粗粒度的程序块（Tile / Block）张量级操作。
共享内存管理	需开发者手动声明内存容量、计地址偏移并显式分配与管理。	编译器基于数据流的生命周期分，自动化完成内存的分配与布局划。
同步与并发机制	高度依赖显式的 __syncthreads() 等同步原语，避免复杂的数据竞争（Data Race）。	隐式同步机制，编译器通过分析量操作前后的依赖关系，自动插必要的内存屏障。
学习门槛	极其陡峭，需深刻理解线程束发（Warp Divergence）、存储体突（Bank Conflict）等底层微构细节。	相对平缓，开发者可维持较高层向量化思维，仅需建立基本的内局部性（Locality）概念。
性能上限	能够触达理论上的硬件物理极限能。	编译器高度优化，性能接近手工 CUDA。
可移植性	深度绑定于 NVIDIA 专有且闭源软硬件生态。	拥抱开放编译器标准（基于 MLIR/LLVM），可原生平滑迁移 AMD 等异构硬件后端。

6.8.3 Triton 核心编程模型

6.8.3.1 分块并行

Triton 遵循单程序多数据（SPMD, Single Program Multiple
Data）执行模型，其核心抽象是「程序块」（Program / Tile）。在 Triton

中，开发者编写的是单个程序实例（Program Instance）的逻辑，GPU
启动时会同时并行运行大量这样的程序实例。每个程序实例通过 tl.program_id(axis)
获取自身在启动网格（Grid）中的位置索引，进而确定自己负责处理的数据块。

这一模型与原生 CUDA 的「线程」模型有本质区别：在 CUDA
中，开发者需要微观地管理线程块（Thread Block）内的每一个具体线程；而在 Triton
中，开发者只需关注「程序块」级别的张量操作，线程级别的并行映射与协同由编译器自动接管。

CUDA 实现：基于「单线程」的微观控制

在原生的 CUDA C++ 中，开发者必须深入到最微观的线程级别，需要手动组合 blockIdx 和 threadIdx 来计算出当前这唯一的一个线程应该处理哪个具体的元素。

```
__global__ void cuda_add_kernel(float* x_ptr, float* y_ptr, float* output_ptr, int n_elements) {
  // 1. 线程级寻址：我是全局网格中的第几个“线程”？
  // 组合 Block ID 和 Thread ID
  int tid = blockIdx.x * blockDim.x + threadIdx.x;

  // 2 & 3. 标量级掩码：使用标量 if 语句进行越界保护
  if (tid < n_elements) {
    // 4. 标量级访存与计算：当前线程只负责读写 1 个具体的浮点数
    float x = x_ptr[tid];
    float y = y_ptr[tid];
    output_ptr[tid] = x + y;
  }
}
```

Triton 实现：基于「张量块」的宏观控制

在 Triton 中，开发者不需要关心底层的线程（Thread）是如何划分的，只需要获取当前的「程序块 ID」，生成一块连续的偏移量，然后对整个张量块进行批量读写。

```
import triton
import triton.language as tl

@triton.jit
def triton_add_kernel(x_ptr, y_ptr, output_ptr, n_elements, BLOCK_SIZE: tl.constexpr):
  # 1. 块级寻址：我是第几个“程序块”？
  pid = tl.program_id(axis=0)

  # 2. 张量块展开：生成当前块内所有元素的偏移量数组 [0, 1, ..., BLOCK_SIZE-1]
  block_start = pid * BLOCK_SIZE
  offsets = block_start + tl.arange(0, BLOCK_SIZE)

  # 3. 块级掩码：使用向量化的布尔数组进行边界保护
  mask = offsets < n_elements

  # 4. 块级访存与计算：一次性加载、计算并存回整个张量块
  x = tl.load(x_ptr + offsets, mask=mask)
  y = tl.load(y_ptr + offsets, mask=mask)
```

```
    output = x + y
    tl.store(output_ptr + offsets, output, mask=mask)
```

6.8.3.2 核心语言原语

Triton 的核心设计哲学是将程序员从繁琐的底层线程管理（如 CUDA 中的 SIMT 线程同步、共享内存分配等）中解放出来，转而采用基于块（Block-based）的编程范式。在这个范式下，操作的最小逻辑单元是多维数据块（Tensor）。为了实现这一目标，Triton 提供了一套精简而强大的核心语言原语。理解这些原语，是编写高性能 Triton Kernel 的第一步。

并行执行的坐标系：tl.program_id** 与 **tl.num_programs

在 Triton 中，所有的并行计算都是通过启动多个程序实例（Program Instances）**来实现的。这些实例在一个一维、二维或三维的网格（Grid）中并行执行。

tl.program_id(axis)：返回当前程序实例在指定维度（axis=0, 1, 2）上的索引。它的作用与 CUDA 中的 blockIdx 完全一致。通过它，每个实例能够知道“我是谁”，从而计算出自己应该处理哪一部分数据。

tl.num_programs(axis)：返回指定维度上的程序实例总数，类似于 CUDA 的 gridDim。通常用于步幅（Stride）计算或网格级的循环。

实践要点：与 CUDA 不同，Triton 没有 threadIdx 的概念。在 Triton 程序的视角中，直接编写一个处理整个数据块的逻辑，编译器会自动将这个数据块的计算任务映射到 GPU 的 Warp 和 Thread 上。

```
import triton
import triton.language as tl

@triton.jit
def demo_kernel(...):
    # 获取当前实例在第 0 维的 ID
    pid = tl.program_id(axis=0)
    # 计算当前程序实例应该处理的起始数据位置
    block_start = pid * BLOCK_SIZE
```

构建内存指针的基石：tl.arange

tl.arange(start, end)：用于生成一个从 start 到 end-1 的一维整数张量（Tensor）。

重要限制：其生成的张量长度（end - start）在编译期必须是一个严格的 2 的幂次（如 64, 128, 256）tl.arange 很少单独作为计算数据使用，它最核心的作用是生成块内偏移量（Offsets），并与基础指针结合，构建出指向全局内存中特定数据块的指针张量（Pointer Tensor）。

```
# 生成块内相对偏移量: [0, 1, 2, ..., BLOCK_SIZE-1]
    offsets = tl.arange(0, BLOCK_SIZE)

    # 结合全局偏移，计算实际数据的绝对索引
    global_indices = block_start + offsets
```

```
# 构建指针：基础地址 + 数据索引
# 假设 data_ptr 是传入的张量首地址
ptrs = data_ptr + global_indices
```

显式内存操控与越界保护：tl.load** 与 **tl.store

Triton 摒弃了传统语言中隐式的数组解引用（如 array[index]），采用了完全显式的内存访问语义。这种设计让编译器能够精确捕捉到内存的“读”与“写”操作，进而生成高度优化的异步内存拷贝（Async Memory Copy）和流水线（Pipelining）指令。

tl.load(ptr, mask=None, other=0.0)：从全局内存按指针张量 ptr 加载数据块。

tl.store(ptr, value, mask=None)：将数据块 value 写回全局内存。

关键实践：Mask 掩码机制 在实际业务中，张量的长度往往不能被 BLOCK_SIZE 整除，这会导致最后一个程序实例产生越界访问（Out-of-bounds Access），引发段错误。mask 参数正是为了优雅地解决边界问题而设计的：

```
# 假设 N 是数组的总长度
# 创建一个布尔张量，超出 N 的部分为 False
mask = global_indices < N

# 加载数据：如果 mask 为 False，则不访问内存，而是直接返回 other 指定的默认值
# 这在向量加法、Reduce 求和等场景中，用 0.0 填充尾部是非常安全的
data = tl.load(ptrs, mask=mask, other=0.0)

# 业务计算...
result = data * 2.0

# 写回数据：同样使用 mask，防止污染未分配或他人的内存区域
tl.store(ptrs, result, mask=mask)
```

驱动极致优化的引擎：编译期常量 tl.constexpr

在 Triton 的 JIT（即时编译）机制中，如果一个参数在函数签名中被标记为 tl.constexpr，或者它本身就是诸如 BLOCK_SIZE 这样的常数，Triton 编译器会将其视为编译期决定的常量。这意味着，当 Python 端传入不同的 constexpr 值时（例如 BLOCK_SIZE=128 和 BLOCK_SIZE=256），Triton 会在后台编译出两份完全独立、高度特化的 DCU 机器码。

形态确定：Triton 中的张量操作（如 tl.load, tl.dot）需要确切知道数据的形状，以便分配寄存器和共享内存（Shared Memory）。

暴力优化：编译器可以基于已知的值进行深度优化，例如循环展开（Loop Unrolling）、剔除死代码、决定向量化指令的位宽（如使用 128-bit 的访存指令而非单字节加载）。

实践原则：

适用场景：块大小（BLOCK_SIZE）、特征维度大小（如 Transformer 中的 head_dim，通常是固定的 64 或 128）、控制内核执行路径的布尔开关（如 IS_CAUSAL）。

反模式：绝对不要将动态变化的值（如批次大小 batch_size 或每次调用都不同的标量权重）设置为

constexpr，这会导致引发灾难性的重新编译风暴（Recompilation Storm），极大地拖慢程序运行。

深度学习的计算引擎：tl.dot

如果说前面的原语是为了把数据搬运到正确的地方，那么 tl.dot 就是 Triton 存在的最大意义。Triton 最初就是为了能够高效、平滑地调用 NVIDIA GPU 的 Tensor Core（以及其他硬件的矩阵乘法加速单元）而设计的，在DCU上可以调用对应的硬件矩阵乘法指令加速计算。

tl.dot(a, b, out=acc, allow_tf32=True)：执行两个二维张量的高效矩阵乘法。

设计哲学：在 CUDA 中，手写高效的矩阵乘法需要极其复杂的共享内存（Shared Memory）切片和寄存器级别的微调。而在 Triton 中，只需要保证张量 a 的最后一维和张量 b 的第一维大小一致，编译器会自动完成底层内存的平铺（Tiling）、流水线编排和 Tensor Core 指令发射。

Python

```
# 假设 a 的形状是 [BLOCK_M, BLOCK_K]，b 的形状是 [BLOCK_K, BLOCK_N]
# a 和 b 必须提前加载到 SRAM (即 Triton 的张量)
# 执行矩阵乘法得到 [BLOCK_M, BLOCK_N] 的结果
# allow_tf32 允许在支持的硬件上为了速度稍微牺牲精度
accumulator = tl.dot(a, b, allow_tf32=None)
```

维度折叠与统计：归约操作 (Reductions)

在实现 Softmax、LayerNorm 或计算 Loss 时，需要在特定维度上进行求和或寻找最大值。Triton 提供了一组在块内进行并行归约的原语：

tl.sum(input, axis=0) / tl.max(input, axis=0) / tl.min(input, axis=0)：沿指定轴折叠张量。

实践要点：如果不指定 axis，原语会将整个张量展平并归约成一个标量。指定 axis 时，编译器会自动生成高效的 Warp 内归约（Warp Shuffle）或跨 Warp 归约代码。axis=0 沿行归约（结果保留列维），axis=1 沿列归约（结果保留行维）。

Python

```
# data 的形状为 [BLOCK_SIZE, HEAD_DIM]
# 沿 HEAD_DIM 维度求和，得到形状为 [BLOCK_SIZE] 的一维张量
row_sums = tl.sum(data, axis=1)
```

规避分支发散的利器：tl.where

在 GPU 的 SIMT（单指令多线程）架构中，if-else 控制流是性能杀手，因为它会导致同一个 Warp 内的线程发生执行分支发散（Branch Divergence）。Triton 鼓励使用数据掩码（Masking）来代替控制流。

tl.where(condition, x, y)：根据布尔张量 condition 的真假，逐元素地选择返回 x 还是 y。它的作用类似于 numpy 中的 np.where。

Python

```
# 实现 ReLU 激活函数：如果 data > 0 则保持原值，否则变为 0
```

```
# 这比写 if data > 0 更加符合底层硬件的并行逻辑
relu_out = tl.where(data > 0, data, 0.0)
```

全局跨实例同步：原子操作 (Atomic Operations)

由于 tl.program_id
启动的各个程序实例是完全异步且无法预测执行顺序的，当多个实例需要写入同一个全局内存地址时（例如在向后传播中累加梯度），直接使用 tl.store 会导致数据竞争和覆盖。

tl.atomic_add(pointer, value, mask=None) / tl.atomic_max
等：确保对指定指针的“读取-修改-写入”过程是不可分割的。tl.atomic_add
返回的是操作前的旧值，这一点和 CUDA 的 atomicAdd 行为一致，在某些算法中可以加以利用。

性能警告：原子操作会锁定显存地址，导致多个并发实例串行排队。在性能敏感的代码中，应尽量通过重构算法（如使用归约树）来避免使用原子操作；如果不可避免，也要尽量减少原子操作的触发次数。

Python

```
# 多个实例可能同时计算出了属于同一个类别的梯度，需要累加
# 必须使用 atomic_add 而不是 load -> add -> store
tl.atomic_add(grad_ptr + index, local_grad, mask=mask)
```

形状重塑与广播：tl.expand_dims 与 tl.broadcast_to

在进行张量计算（如一个 2D 矩阵加上一个 1D 向量）时，需要对齐它们的维度，其中shape
参数要求是编译期常量（constexpr），不能是运行时动态值。

tl.expand_dims(tensor, axis)：在指定位置插入一个大小为 1 的新维度。

tl.broadcast_to(tensor, shape)：将张量沿着大小为 1 的维度复制扩展，直到匹配目标
shape。注意，Triton
中很多运算（如加法）支持自动广播，但在需要显式对齐内存布局时，这些原语非常关键。

注意 以上仅列举了编写高性能 Triton Kernel
时最核心、最基础的语言原语。实际上，为了满足复杂多变的深度学习算法需求，Triton
提供了极其丰富的标准库与扩展 API。欲了解完整的语言原语支持情况、详细的参数说明以及最新的 API 变动，强烈建议读者前往 Triton 官方网站（https://triton-lang.org/）查阅其最新的 API 参考手册（API Reference）。

6.8.3.3 内存抽象与编译降级流程

要透彻理解 Triton 程序的极致性能，必须深入“引擎盖”之下，搞清楚物理硬件的存储层次，以及 Triton 编译器是如何通过多级降维将高层抽象精准映射到这些物理单元上的。

内存层次与 Triton 数据流

在 Triton
的编程模型中，底层硬件复杂的存储结构被极大地简化了。开发者只需要关注数据块（Tile）的流动，而编译器会围绕以下三个物理层次进行资源的自动分配：

层次	容量	延迟	Triton访问方式
全局内存（HBM）	大（GB级）	高（数百时钟周期）	开发者通过 tl.load / tl.store 显式访问。
共享内存（SRAM）	中（KB级，64KB/CU）	低（数十时钟周期）	隐式管理。编译器通过命周期分析，自动将需复用的数据搬运并缓存此。
寄存器 (Registers)	小（KB级/线程）	极低	自动分配。程序块层级张量计算与累加器（如阵乘法的结果）默认驻于此。

核心性能原则

高性能 Triton
核函数的灵魂在于：最大化共享内存的复用率，掩盖或最小化全局内存的访问延迟。这与手工 CUDA 的优化目标完全一致，但区别在于，Triton
将繁重的内存规划与流水线编排工作交给了其强大的编译器后端。

“前端统一，后端特化” 的编译降级流水线

Triton 的强大之处在于并未直接将 Python 源码生硬地翻译为机器码，而是依托一条基于 MLIR 构建的多级降级流水线（Lowering
Pipeline）。在保留高级计算语义的同时，逐步注入目标硬件（如异构计算加速器 DCU 或 NVIDIA GPU）的微架构特征。针对 DCU 平台，其编译降级流程被精细地划分为以下五个核心阶段：

[图片]

阶段	IR 名称	抽象层次与特征	主要优化 Pass
①	Triton IR (TIR)	Tile 级（硬件无关）：保留分块张量语义、控制流。	内联展开、死代码消除、常量
②	Triton GPU IR (TTGIR)	Wavefront/Block 级（硬件相关）：显式表达线程组布局与 LDS（局部数据共享）。	Layout 转换、LDS 内存分析、
③	LLVM IR	Workitem/SIMD 级：标量化与向量化，注入特定的内置函数。	指令选择、VGPR 寄存器分配
④	DCU ISA	汇编级 (LLVM Backend)：针对特定指令集（如gfx928）生成机器指令。	指令调度、消除流水线停顿
⑤	机器码	机器码：直接运行在计算单元（CU）上的最终二进制文件。	硬件级别的微架构执行

点击图片可查看完整电子表格

Triton IR (TIR) —— 硬件无关的 Tile 抽象

前端解析器将带有 @triton.jit 的 Python 代码转换为 Triton IR。此阶段完全屏蔽了 GPU 的硬件细节（如 Wavefront 大小、CU 结构），纯粹以「分块张量」（Blocked Tensor）为单位描述计算逻辑。

Triton GPU IR (TTGIR) —— 智能优化的核心

这是 Triton 针对 GPU 架构进行特化优化的关键层。TIR 在此被映射为具体的 GPU 执行单元模型，引入了四大核心机制：

布局编码（Layout Encoding）：为张量附加布局属性，决定数据如何在 Wavefront（波前） 的 Workitem 间分布。对于DCU 后端，这里会使用 #blocked（常规访存）和 #mfma（针对 DCU Matrix Core 的专属布局，匹配 mmac 指令）。

LDS 自动分配：编译器自动分析跨 Wavefront 复用的数据，自动在 LDS (Local Data Share，等同于 NVIDIA 的 Shared Memory) 中分配空间，并处理相关的读写请求。

软件流水线（Pipeline）：依据 num_stages 参数，将全局内存（Global Memory / HBM）的加载提前，实现访存延迟与 Matrix Core 计算的异步重叠。

LLVM IR —— 标量化与向量化

分块张量在此层被转化为底层 LLVM 指令：

向量化合并：编译器尽可能合并连续的访存行为，例如将 4 个 float32 的加载合并为一条 128-bit 的向量指令，以最大化带宽。

硬件指令注入：将布局信息转化为显式的 Workitem ID 计算（注入如 llvm.amdgcn.workitem.id.x 等内置函数）。

ISA —— 底层验证与剖析

与 NVIDIA 经过 PTX 虚拟汇编不同，LLVM 的 DCU 后端直接生成目标微架构（如 gfx928）的物理汇编指令。对于高级开发者，阅读这些汇编指令是验证 Triton 优化是否生效的最佳手段：

```
// ISA 示意：关注核心指令是否符合预期
// 期望看到 128-bit 向量化加载 (dwordx4 = 4 * 32bit)
global/buffer_load_dwordx4

// 标量向量加法
v_add_f32_e32 v0, v0, v4

// 期望看到 128-bit 向量化存储
global/buffer_store_dwordx4
```

6.8.4 Triton 算子的高级优化技术

要写出一个能跑的 Triton Kernel 并不难，但要写出一个能压榨出硬件 90% 以上极限性能（Peak FLOPs 或 Peak Bandwidth）的 Kernel，就需要深入理解并综合运用多种优化机制。

6.8.4.1 自动调优（Auto-tuning）

不同的硬件型号、矩阵尺寸和数据精度，往往对应截然不同的最优超参数组合。例如，在 DCU 上对 M=4096、N=4096 的矩阵乘法，BLOCK_M=128 可能比 BLOCK_M=64 快 30%；但当矩阵变为细长形（M=64、N=4096）时，结论可能完全相反。手工枚举所有组合既费时又不可靠，@triton.autotune 正是为解决这一问题而设计的。autotune允许开发者声明一组超参数配置，由框架在目标硬件上自动搜索最优配置。这是 Triton 实现接近手工 CUDA 性能的关键机制之一。

```
@triton.autotune(
  configs=[
    triton.Config({'BLOCK_M': 128, 'BLOCK_N': 256, 'BLOCK_K': 64,  'GROUP_M': 8}, num_stages=3,
num_warps=8),
    triton.Config({'BLOCK_M': 64,  'BLOCK_N': 256, 'BLOCK_K': 32,  'GROUP_M': 8}, num_stages=4,
num_warps=4),
    triton.Config({'BLOCK_M': 128, 'BLOCK_N': 128, 'BLOCK_K': 32,  'GROUP_M': 8}, num_stages=4,
num_warps=4),
    triton.Config({'BLOCK_M': 128, 'BLOCK_N': 64,  'BLOCK_K': 32,  'GROUP_M': 8}, num_stages=4,
num_warps=4),
  ],
  key=['M', 'N', 'K'], # 当这些维度变化时触发重新调优
)
@triton.jit
def matmul_kernel_autotuned(...):
  ...
```

自动调优的关键参数包括：

BLOCK_M / BLOCK_N / BLOCK_K：沿三个维度的分块大小，直接影响共享内存利用率和 Tensor Core 效率；

num_stages：软件流水线（Software Pipelining）的深度，控制内存加载与计算的重叠程度；

num_warps：每个程序块使用的 Warp 数量，影响占用率（Occupancy）和延迟隐藏效果；

GROUP_M：L2 缓存分组优化参数，用于提升矩阵乘法的 L2 命中率；

@triton.autotune 的工作流程分为三步：首先，框架在首次调用时，对 configs 列表中的每个配置分别执行预热（warmup）和计时，记录各配置的实际运行耗时；其次，选出耗时最短的配置缓存下来；最后，当 key 中列出的维度（如 M、N、K）发生变化时，重新触发搜索。这意味着调优开销只发生在”冷启动”阶段，后续调用均使用缓存的最优配置，几乎没有额外开销。

6.8.4.2 软件流水线（Software Pipelining）

软件流水线是 GPU 高性能编程中最重要的延迟隐藏技术之一。GPU 全局内存访问延迟高达数百个时钟周期，如果计算与内存访问顺序执行，大量时间会浪费在等待数据上。

Triton 通过 num_stages 参数控制流水线深度。当 num_stages=3 时，编译器会自动生成如下重叠执行模式：

```
# 概念示意（Triton 编译器自动生成，无需手动编写）
# Stage 1: 预取第 1 块数据
prefetch(block_0_data)
# Stage 2: 计算第 0 块，同时异步预取第 2 块
compute(block_0_data)
async_prefetch(block_1_data)
# Stage 3: 计算第 1 块，同时异步预取第 3 块
compute(block_1_data)
async_prefetch(block_2_data)
# ... 循环展开
#对于矩阵乘法等计算密集型算子，增加num_stages数量可以有效提高内存带宽的效率
```

软件流水线的本质是用共享内存（LDS）换取计算与访存的时间重叠。每增加一个 stage，编译器就需要在 LDS 中额外保留一份数据缓冲区。以矩阵乘法为例，若 BLOCK_K=64、数据类型为 FP16，单份缓冲区占用约 (BLOCK_M + BLOCK_N) × BLOCK_K × 2 bytes。当 num_stages=4 时，LDS 占用翻倍，可能导致每个 CU 能同时驻留的 Wavefront 数量（即占用率 Occupancy）下降，反而抵消了流水线收益。

6.8.4.3 L2 缓存优化：程序块分组（Group Swizzling）

在大矩阵乘法中，朴素的程序块调度会导致 L2 缓存命中率极低。通过对程序块的执行顺序进行重排（称为 Swizzling），可以显著提升 L2 缓存的复用效果。为了实现 Swizzling，启动网格（Grid）将从二维展平为了一维。pid 现在代表的是 1D 线性索引，而后续那几行复杂的取模和整除数学运算，本质上就是一种自定义的二维解码器，将这个 1D 索引重新映射回 (pid_m, pid_n) 的二维坐标，只不过映射的顺序不再是朴素的行主序或列主序，而是以 GROUP_M 为单位的“Z字形”或“回字形”遍历顺序。

```
@triton.jit
def matmul_with_swizzle(... GROUP_M: tl.constexpr):
    pid = tl.program_id(axis=0)
    num_pid_m = tl.cdiv(M, BLOCK_M)
    num_pid_n = tl.cdiv(N, BLOCK_N)
    num_pid_in_group = GROUP_M * num_pid_n
    group_id = pid // num_pid_in_group
    first_pid_m = group_id * GROUP_M
    # 当前组内的实际行数（考虑边界）
    group_size_m = min(num_pid_m - first_pid_m, GROUP_M)
    # 重映射 pid 到 (pid_m, pid_n)
    pid_m = first_pid_m + ((pid % num_pid_in_group) % group_size_m)
    pid_n = (pid % num_pid_in_group) // group_size_m
    ...
```

Swizzling 的核心思想是改变硬件执行单元（CU）获取数据的顺序。通过将连续的 pid 限制在一个 GROUP_M 宽度的组内执行，使得同一组内并发执行的程序块能够共享大量的输入数据（例如矩阵 A 的相同行块），从而大幅减少 L2 缓存的抖动（Thrashing）。在实际情况下，需要考虑 CU 的个数，L2

缓存的大小等因素，很难从第一性原理出发计算出最好的 raster 方向和
swizzle_size，一般是作为参数去 autoune。

6.8.4.4 融合算子（Operator Fusion）

算子融合是提升深度学习推理和训练效率的重要手段。在现代 GPU
架构中，计算极其廉价，而访存极其昂贵。通过将多个逻辑上独立的算子融合进一个 Triton
核函数，可以彻底消除中间结果（Intermediate
Tensors）在全局内存（HBM）中“写回再读取”的巨大开销，从而大幅减少内存带宽压力。

以 Softmax 为例，朴素的 PyTorch
实现需要多次遍历数据（求最大值、求指数和、归一化），每次都涉及高延迟的全局内存读写。而利用 Triton 的 tl.max 和
tl.sum，可以将整行数据一次性加载到片上寄存器/共享内存中，所有操作全在极速的片内完成（即 Fused Softmax）：

```
@triton.jit
def fused_softmax_kernel(input_ptr, output_ptr, stride, n_cols, BLOCK_SIZE: tl.constexpr):
    row_idx = tl.program_id(0)
    row_ptr = input_ptr + row_idx * stride

    # 获取该行的列索引
    cols = tl.arange(0, BLOCK_SIZE)
    mask = cols < n_cols

    # 唯一的一次全局内存读取：加载整行数据到寄存器（越界部分补负无穷）
    row = tl.load(row_ptr + cols, mask=mask, other=float("-inf"))

    # 以下所有计算均在片内寄存器/SRAM中完成，无全局内存开销
    row_max = tl.max(row, axis=0)
    row = row - row_max  # 减去最大值以保证数值稳定性 (Safe Softmax)
    numerator = tl.exp(row)
    denominator = tl.sum(numerator, axis=0)
    softmax_output = numerator / denominator

    # 唯一的一次全局内存写回
    tl.store(output_ptr + row_idx * stride + cols, softmax_output, mask=mask)
```

注：上述代码要求序列长度 n_cols 必须小于等于 BLOCK_SIZE。对于超长序列，则需要引入循环并结合 Online Softmax 算法分块处理。

6.8.5 在 DCU 上使用 Triton 的实践

6.8.5.1 Triton环境搭建与安装验证

DCU已适配多个版本triton，推荐从光合开发者社区下载相关镜像，默认会包含triton组件，可直接使用；或根据需要直接下载triton的wheel包，上传到容器后可直接进行pip安装（https://download.sourcefind.cn:65024/4/main/triton）。注意：triton作为torch inductor后端需要的组件，建议与对应版本torch搭配使用，如torch2.5.1对应triton3.1.0、torch2.7.1对应triton3.3.0、torch2.9对应triton3.5.1。

```
# 下载wheel包
wget
https://download.sourcefind.cn:65024/file/4/triton/DAS1.8/triton-3.1.0+das.opt1.dtk2604.torch271-cp310-cp310-manylinux_2_28_x86_64.whl
# 使用pip 进行安装
pip install triton*.whl
# 查看当前安装的triton版本
pip list | grep triton
# 查看是否成功安装triton
python -c "import triton; print(triton.__version__)"
```

6.8.5.2 kernel开发流程

向量加法（Z = X + Y）是 GPU 编程领域的“Hello World”。它虽然在计算上极其简单，但却是一个典型的内存密集型（Memory-bound）任务。本节将以官方教程为准，完整走通一个 Triton 算子的开发生命周期：编写 Kernel、编写 Host 调度逻辑、验证正确性以及执行标准化基准测试。

一个完整的 Triton 工程通常包含三个部分：

Device 端代码（Kernel）：在 GPU 上实际执行的并行逻辑。

Host 端代码（Wrapper）：在 CPU 上分配内存、计算网格大小并启动 Kernel 的 Python 包装函数。

基准测试（Benchmark）：验证算子正确性并评估性能的代码。

编写 Compute Kernel（设备端计算核心）

首先使用 @triton.jit 装饰器定义在 GPU 上运行的核函数。Triton 采用 SPMD（单程序多数据）编程模型，多个“程序实例”会同时运行，处理不同的数据块。

```
import torch
import triton
import triton.language as tl

@triton.jit
def add_kernel(x_ptr,    # 指向第一个输入向量的指针
        y_ptr,    # 指向第二个输入向量的指针
        output_ptr, # 指向输出向量的指针
        n_elements, # 向量的总元素个数
        BLOCK_SIZE: tl.constexpr, # 每个程序实例应处理的元素个数
        ):
  # 获取当前程序实例在 0 轴上的 ID（因为使用的是 1D 启动网格）
```

```
    pid = tl.program_id(axis=0)

    # 计算当前程序实例需要处理的数据起始偏移量
    # 例如：向量长度 256，BLOCK_SIZE 为 64，则各个实例处理的切片为 [0:64, 64:128, 128:192, 192:256]
    block_start = pid * BLOCK_SIZE

    # 构造一个指针偏移量列表
    offsets = block_start + tl.arange(0, BLOCK_SIZE)

    # 创建一个掩码，防止越界访问内存
    mask = offsets < n_elements

    # 从 DRAM 中加载 x 和 y。如果输入长度不是 BLOCK_SIZE 的整数倍，掩码会过滤掉多余的元素
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)

    # 计算加法
    output = x + y

    # 将结果写回 DRAM
    tl.store(output_ptr + offsets, output, mask=mask)
```

核心设计点：

指针与偏移：Triton 传递给 Kernel 的是张量的首地址指针（_ptr），需要利用 tl.arange 加上 block_start，显式计算出每个实例负责的绝对内存地址。

安全掩码 (Mask)：在实际业务中，张量长度极少能被 BLOCK_SIZE 完美整除。mask = offsets < n_elements 是保护程序不发生内存段错误的关键。

编写 Host 端包装函数

Triton Kernel 编写完成后，不能直接在 Python 中像普通函数一样调用，需要一个 Python 包装函数来分配内存、计算并行网格（Grid）大小，并触发计算。

```
def add(x: torch.Tensor, y: torch.Tensor):
    # 预先分配输出张量
    output = torch.empty_like(x)

    # 确保所有张量都在同一个正确的设备上
    assert x.is_cuda and y.is_cuda and output.is_cuda

    n_elements = output.numel()

    # 定义 SPMD 启动网格（Grid），决定并行运行的 Kernel 实例数量
    # 类似于 CUDA 中的 grid_size。使用 triton.cdiv 进行向上取整除法
    grid = lambda meta: (triton.cdiv(n_elements, meta['BLOCK_SIZE']), )

    # 启动 Kernel
    # 注意：传入的 torch.tensor 会被隐式转换为指向其首元素的指针
```

```
    add_kernel[grid](x, y, output, n_elements, BLOCK_SIZE=1024)

    # 返回结果。注意：此时 GPU 计算是异步的，如果没有显式同步，返回时计算可能仍在进行
    return output
```

正确性验证

在进行性能测试前，必须确保计算结果与原生 PyTorch
完全一致。代码示例刻意选择一个不规则的张量大小（98432）来验证 mask 边界处理逻辑是否健壮。

```
torch.manual_seed(0)
size = 98432 # 故意选择不能被 1024 整除的尺寸
x = torch.rand(size, device='cuda')
y = torch.rand(size, device='cuda')

output_torch = x + y
output_triton = add(x, y)

print(f'Torch 输出与 Triton 输出的最大差异为: '
    f'{torch.max(torch.abs(output_torch - output_triton))}')
# 如果输出差异为 0.0，则说明逻辑完全正确！
```

自动化性能基准测试

Triton 提供了极为方便的 @triton.testing.perf_report
装饰器，专门用于绘制算子在不同问题规模下的性能曲线。对于向量加法，主要关注内存带宽吞吐量
(GB/s)。

```
@triton.testing.perf_report(
    triton.testing.Benchmark(
        x_names=['size'], # 用作图表 X 轴的参数名
        x_vals=[2**i for i in range(12, 28, 1)], # X 轴的取值范围
        x_log=True, # X 轴使用对数刻度
        line_arg='provider', # 用于区分不同折线的参数
        line_vals=['triton', 'torch'], # `provider` 的取值列表
        line_names=['Triton', 'Torch'], # 图例名称
        styles=[('blue', '-'), ('green', '-')], # 折线样式
        ylabel='GB/s', # Y 轴标签
        plot_name='vector-add-performance', # 图表名称与保存文件名
        args={}, # 不在 x_names 和 line_arg 中的其他常量参数
    ))
def benchmark(size, provider):
    # 准备测试数据
    x = torch.rand(size, device='cuda', dtype=torch.float32)
    y = torch.rand(size, device='cuda', dtype=torch.float32)

    # 记录 50% (中位数), 20% 和 80% 的分位数
    quantiles = [0.5, 0.2, 0.8]

    # 使用 do_bench 执行算子，它会自动处理预热 (warmup) 并进行多次重复采样
```

```
    if provider == 'torch':
        ms, min_ms, max_ms = triton.testing.do_bench(lambda: x + y, quantiles=quantiles)
    if provider == 'triton':
        ms, min_ms, max_ms = triton.testing.do_bench(lambda: add(x, y), quantiles=quantiles)

    # 计算 GB/s：读取 x 和 y，写入 output，共传输 3 个张量
    # 公式：3 * 元素总数 * 每个元素的字节数 * 1e-9 (转为GB) / (时间 ms * 1e-3 (转为秒))
    gbps = lambda ms: 3 * x.numel() * x.element_size() * 1e-9 / (ms * 1e-3)

    return gbps(ms), gbps(max_ms), gbps(min_ms)

# 执行并输出结果
if __name__ == '__main__':
    # print_data=True 将在终端打印 Markdown 表格
    # show_plots=True 将弹出性能折线图
    benchmark.run(print_data=True, show_plots=True)
```

运行结果预期：

当运行这段代码时，会发现如下输出， Triton 的性能与 PyTorch 几乎重合。这是因为向量加法已经达到了硬件内存带宽的物理瓶颈。这个例子虽然简单，但它提供了一套完整的测试脚手架，可以将这套模板直接复用到后续开发的高级算子（如 Flash Attention 或融合算子）的性能评估中。

YAMLTorch 输出与 Triton 输出的最大差异为: 0.0vector-add-performance: size Triton Torch0 4096.0 5.688889 5.6888891 8192.0 11.377778 11.3777782 16384.0 22.341818 23.1849053 32768.0 44.683637 44.6836374 65536.0 89.367273 89.3672735 131072.0 175.542852 172.4631666 262144.0 327.679984 333.2338927 524288.0 604.947691 604.9476918 1048576.0 925.214105 925.2141059 2097152.0 1209.895381 1219.27437710 4194304.0 1463.129253 1463.12925311 8388608.0 1617.340853 1617.36689412 16777216.0 1709.634745 1698.09876713 33554432.0 1759.855521 1750.05735014 67108864.0 1786.710921 1793.71920515 134217728.0 1770.682390 1761.546461

6.8.5.3 调试 Kernel

在实际开发中，编写 Triton Kernel 往往不是一帆风顺的。当遇到逻辑错误或性能瓶颈时，需要像外科医生一样剖析 Kernel 的内部状态。本节将继续以向量加法为例，演示如何在设备端打印变量、分析生成的各级中间表示（IR），以及如何直接修改并编译执行 TTIR（Triton IR）。

设备端运行时调试：tl.device_print** 与 **tl.static_print

由于 Triton Kernel 是在 GPU 的成百上千个核心上异步并行的，传统的 print 函数或断点调试（如 pdb）无法直接使用。Triton 提供了专门的打印原语：

tl.static_print：在编译期打印 Python 环境中的常量或变量状态（例如 BLOCK_SIZE）。

tl.device_print：在运行期由 GPU 线程将动态计算的张量数据打印到 Host 端的控制台。

调试最佳实践：单次执行 (Single-block Execution)

如果成千上万个并发的 GPU 实例同时触发
tl.device_print，海量的输出不仅会瞬间淹没控制台，还极易导致终端卡死甚至引发 GPU
崩溃。为了安全地进行调试，通常采用两种手段结合：

Kernel 内部截流：使用 if pid == 0: 限制只有第一个程序实例执行打印逻辑。

Host 端强制单实例：在定位 Bug
期间，不要使用完整的数据集，而是构造一个极小规模的“玩具张量”（Toy Tensor），并强制将 Host
端的启动网格（Grid）修改为单次执行。

```
@triton.jit
def debug_add_kernel(x_ptr, y_ptr, output_ptr, n_elements, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements

    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    output = x + y

    # 【调试技巧 1】编译期打印，检查常量值
    tl.static_print(f"Compiling Kernel with BLOCK_SIZE = {BLOCK_SIZE}")

    # 【调试技巧 2】运行期打印，只让 pid == 0 的第一个线程块打印前 4 个元素
    if pid == 0:
        # device_print 会打印出张量的形状和具体数值
        tl.device_print("First block X: ", x)
        tl.device_print("First block Y: ", y)
        tl.device_print("First block Output: ", output)

    tl.store(output_ptr + offsets, output, mask=mask)

# --- Host 端调试代码 ---
def debug_add():
    # 1. 构造小规模测试数据
    size = 128
    x = torch.rand(size, device='cuda')
    y = torch.rand(size, device='cuda')
    output = torch.empty_like(x)

    # 2. 调试时，强行将 Grid 写死为 1，确保绝对只有一个 Block 在运行
    # 避免真实场景下复杂的块间交错执行干扰调试日志
    grid = (1, )

    debug_add_kernel[grid](x, y, output, size, BLOCK_SIZE=128)
```

查看不同层级的生成 IR

Triton 编译器采用了多级 IR 设计：Python AST -> TTIR (Triton IR) -> TTGIR (Triton GPU IR) -> LLVM IR

-> AMDGCN (底层汇编)。想要查看这些中间表示以及编译器的优化过程，有两种主要途径：

途径 A：使用环境变量实时拦截

由于 Triton 的编译栈重度依赖
MLIR（多层中间表示），当想要弄清楚某一段具体的优化逻辑（如循环展开、常量折叠）是如何发生时，可以使用以下底层的环境变量，可以将不同层级的 IR 或编译过程直接打印到控制台。

```
export MLIR_ENABLE_DUMP=1    # 打印 MLIR 层面每个 Pass 执行前后的变换细节
export LLVM_IR_ENABLE_DUMP=1  # 打印转译为 LLVM IR 阶段的详细生成与优化过程
```

注意：开启这两个 Pass
级别的追踪会产生极其庞大的日志输出，建议仅配合前面提到的“单实例执行（Grid=1）”和极其简单的代码块一起使用。

途径 B：从 Triton 缓存目录中提取

在实际开发中，Kernel 生成的 IR
和汇编代码往往长达数千行，通过终端标准输出查看非常痛苦（尤其是开启了 MLIR_ENABLE_DUMP 后）。事实上，Triton
会在后台自动将每次编译的最终产物缓存到本地磁盘中。默认情况下，这些文件保存在 ~/.triton/cache/ 目录下。

当运行过一次自定义的 Kernel 后，可以执行以下操作：

打开 ~/.triton/cache/ 目录。

在这个目录下会看到一些由长串哈希值（Hash）命名的文件夹。找到修改时间最新的那个文件夹。

在该文件夹中，可以看到 Triton 完整保存了该 Kernel 编译流水线各个阶段的最终文件：

*.ttir (高层 Triton IR)

*.ttgir (带 Layout 信息的 GPU IR)

*.llir (LLVM IR)

*.amdgcn（汇编文件）

6.8.5.4 Triton IR 执行

在高阶算子开发与底层性能调优中，仅依赖上层 Python API
往往不足以应对所有的极端场景。当开发者需要验证特定的编译器 Pass 行为、排查底层降级 Bug，或尝试绕过当前编译器优化策略的局限性时，直接干预中间表示（IR）便成为了一项必备的底层工程技能。

Triton 的架构设计提供了极高的灵活度：其底层的 triton.compile API 不仅能够接收 Python 抽象语法树（AST），还可以直接解析并编译多级中间表示（如 .ttir 或
.ttgir）文件。本节将通过一个算子行为注入的案例，演示从 IR 拦截到手动执行的完整标准工作流。

提取与定制化修改中间表示

首先，通过运行前文的向量加法 Kernel 生成初始的编译缓存。随后，进入 ~/.triton/cache/ 目录，提取对应的 *.ttir 文件至当前工作区，并重命名为 modified_kernel.ttir。

在此阶段，将进行两处核心的修改，以验证底层指令的替换逻辑：

指令逻辑替换：将原本的加法指令 arith.addf 替换为乘法指令 arith.mulf。

符号重命名（避免缓存命中）：由于 Triton 的 JIT 机制强依赖哈希缓存，若仅修改内部指令而保持函数签名不变，编译器将直接加载旧的汇编二进制文件。因此，必须将函数符号从 @add_kernel 修改为 @mul_kernel，强制触发重新编译。

```
// modified_kernel.ttir 核心逻辑片段
module {
 // 【规范 1】修改函数符号名，使得当前 JIT 缓存失效
 tt.func public @mul_kernel(%arg0: !tt.ptr<f32> {tt.divisibility = 16 : i32} ... ) attributes {noinline = false} {
  // ... 前置的内存指针推导与 Mask 计算逻辑 ...

  %x = tt.load %ptr_x, %mask : tensor<128x!tt.ptr<f32>>
  %y = tt.load %ptr_y, %mask : tensor<128x!tt.ptr<f32>>

  // 【规范 2】在 IR 层面直接修改算子行为：由 addf 变更为 mulf
  %out = arith.mulf %x, %y : tensor<128xf32>

  tt.store %ptr_out, %out, %mask : tensor<128x!tt.ptr<f32>>
  tt.return
 }
}
```

基于底层 API 的 Host 端调度

脱离了 @triton.jit 装饰器的上层封装后，Host 端的调度代码需要严格遵循底层 API 的调用规范。对于 triton.compile 实例化的 CompiledKernel 对象，在启动网格构建与参数传递时存在严格的约束：

```
import torch
import triton
# 1. 准备测试数据：使用 2.0 和 3.0 进行计算验证
def run_modified_ir():
  size = 128
  x = torch.full((size,), 2.0, device='cuda', dtype=torch.float32)
  y = torch.full((size,), 3.0, device='cuda', dtype=torch.float32)
  output = torch.empty_like(x)

  # 2. 编译底层 IR
  print("Compiling modified TTIR...")
  # 【调用规范 A】：直接传入文件路径字符串。Triton 内部会自动解析文件系统
  # 若强行传入 f.read() 读取的全量文本，会引发 FileNotFoundError。
  compiled_kernel = triton.compile("modified_kernel.ttir")

  # 3. 构造启动网格 (Grid)
  # 【调用规范 B】：底层 C++ 运行库无自动维度补齐机制。
```

```
    # 必须显式定义三维网格 tuple (X, Y, Z)，即使 Y 和 Z 维度为 1，否则将触发 IndexError。
    grid = (1, 1, 1)

    # 4. 执行 Kernel# 【调用规范 C】：传入参数的顺序与数据类型，必须与 IR 中的 %arg0, %arg1... 严格对
。
    compiled_kernel[grid](x, y, output, size)

    # 5. 结果校验
    print("Execution complete. First 4 elements of output:")
    print(output[:4])

if __name__ == "__main__":
    run_modified_ir()
```

执行验证与工程启示

运行上述调度脚本，控制台输出如下：

```
Compiling modified TTIR...
Execution complete. First 4 elements of output:
tensor([6., 6., 6., 6.], device='cuda:0')
```

测试数据 2.0 与 3.0 的计算结果为 6.0，证明我们成功在 Python 抽象与底层汇编之间进行了逻辑注入。

工程启示： 这种直接操作 IR
的工作流在实际算子开发中极具价值。例如，当处理极其复杂的布局转换（Layout
Conversion）或特定架构的特殊指令（如针对底层 Matrix Core 的特殊排布）时，如果高层 Python
代码无法诱导编译器生成最优的机器码，开发者便可以借助这一流程，在 TTGIR 或 LLVM IR
层面进行人工介入与微调，最终实现对 DCU 硬件资源的极致压榨。

附录

A 错误检查

当我们在编写DCU异构程序时，出现意外错误是难免的，这个时候需要借助一些方法或工具来快速定位问题。

下面几类错误是在开发过程中经常遇到的：

- API返回错误
- kernel函数非法地址访问

A.1 API返回错误

API返回错误是在开发中最常见的一种错误类型，一般情况下我们可以通过判断API的返回值来确定出错的原因。返回值的类型一般是hipError_t的枚举类型，开发者可以在hip/include/hip/hip_runtime_api.h的头文件中查找对应的错误类型。

```
typedef enum __HIP_NODISCARD hipError_t {
  hipSuccess = 0,  ///< Successful completion.
  hipErrorInvalidValue = 1,  ///< One or more of the parameters passed to the API call is NULL
                    ///< or not in an acceptable range.
  hipErrorOutOfMemory = 2,
  // Deprecated
  hipErrorMemoryAllocation = 2,  ///< Memory allocation error.
  hipErrorNotInitialized = 3,
  // Deprecated
  hipErrorInitializationError = 3,
  hipErrorDeinitialized = 4,
hipErrorProfilerDisabled = 5,
……
} hipError_t;
```

A.2 kernel函数非法地址访问

DCU程序的Kernel函数非法地址访问错误与CPU程序的segment fault非常相似，出现的原因一般为操作了非法的内存地址，执行的机器指令异常等等。

在DCU中，一个进程的kernel函数只能访问这个进程显式申请的DCU内存，如果在kernel内访问了不属于该进程的DCU内存Runtime便会终止进程的运行，同时为用户输出kernel函数的VMFault信息。

下面的kernel代码中引入了一个非法内存访问的错误。

传给kernel的out指针为0x7f9f24400000大小为4194304字节，in指针为0x7f9f24a00000大小为4194304字节，width为1024

```
hipLaunchKernelGGL(
matrixTranspose,
dim3(256, 256),
            dim3(4, 4),
0,
0,
out,
in,
1024);
```

kernel函数matrixTranspose的具体实现如下：

```
__global__ void matrixTranspose(float* out, float* in, const int width) {
  int x = blockDim.x * blockIdx.x + threadIdx.x;
  int y = blockDim.y * blockIdx.y + threadIdx.y;

  out[y * width + x + 1000000] = in[x * width + y];//此处访问了非法内存
```

```
}
```

当DCU执行这个kernel时必然会触发非法内存访问的错误，此时Runtime会返回给用户如下所示的VMFault信息并终止程序继续执行。

```
Invalid address access: 0x7f9f24800000, Error code: 3.

>>>>>>>> KERNEL VMFault !!!! <<<<<<

>>>>>>>> PID: 2396 !!!! <<<<<<
=====> STREAM <0x174bad0>: VMFault HSA QUEUE ANALYSIS <======
STREAM <0x174bad0>: get hsa queue W/R ptr: write index: 5, read index: 3
STREAM <0x174bad0>: >>>>>>>> DUMP KERNEL AQL PACKET <<<<<<<<<
STREAM <0x174bad0>: header: 770
STREAM <0x174bad0>: setup: 3
STREAM <0x174bad0>: workgroup: x:4, y:4, z:1
STREAM <0x174bad0>: grid: x:1024, y:1024, z:1
STREAM <0x174bad0>: group_segment_size: 0
STREAM <0x174bad0>: private_segment_size: 0
STREAM <0x174bad0>: kernel_object: 140335074075968

SUCCESS: FIND SAME KERNEL OBJECT COMMAND IN USE LIST. useIdx: 3
STREAM <0x174bad0>: >>>>>>>> FIND MATCH KERNEL COMMAND <<<<<<<<<
STREAM <0x174bad0>: kernel name: _Z15matrixTransposePfS_i
STREAM <0x174bad0>: >>>>>>>> DUMP KERNEL ARGS: size: 20 <<<<<<<<<

00 00 40 24 9f 7f 00 00 00 00 a0 24 9f 7f 00 00
00 04 00 00

STREAM <0x174bad0>: >>>>>>>> DUMP KERNEL ARGS PTR INFO <<<<<<<<<
STREAM <0x174bad0>: ptr arg index: 0, ptr: 0x7f9f24400000
STREAM <0x174bad0>: host ptr: 0x7f9f24400000, device ptr: 0x7f9f24400000, unaligned ptr:
0x7f9f24400000
STREAM <0x174bad0>: size byte: 4194304
STREAM <0x174bad0>: ptr arg index: 1, ptr: 0x7f9f24a00000
STREAM <0x174bad0>: host ptr: 0x7f9f24a00000, device ptr: 0x7f9f24a00000, unaligned ptr:
0x7f9f24a00000
STREAM <0x174bad0>: size byte: 4194304

>>>>>>>> KERNEL VMFault Analysis END !!!! <<<<<<
```

对VMFault信息的解释如下：

第一行Invalid address
access后面的地址表示kernel非法访问的具体地址，在这里是0x7f9f24800000。Error code:
3表示非法内存地址访问。

DUMP KERNEL AQL
PACKET这一栏详细说明了出错kernel的的DCU参数，用户只需要关心workgroup和grid这两个关于网格大小的参数就可以了，其中grid为hipLaunchKernelGGL中blockDim与gridDim相乘得出的。

FIND MATCH KERNEL
COMMAND这一栏中有最为重要的kernel函数名的信息，这里的kernel的函数名都是C++命名格式，需要使用c++file命令来转换为我们代码中的函数定义。

```
# c++filt _Z15matrixTransposePfS_i
# matrixTranspose(float*, float*, int)
```

DUMP KERNEL
ARGS为用户展示了传给kernel的参数信息，其中size为总的参数大小，后面的16进制数据是堆参数内容的输出，前16个字节是传给kernel的out和in这两个指针，每个指针各占8字节，以小端方式存储。最后四个字节是传入的int类型的立即数1024。

DUMP KERNEL ARGS PTR INFO展示了参数中指针的信息，ptr arg
index和ptr表示参数中的第几个指针的信息。由于在开发过程中有可能预先分配一大段DCU内存然后自己做DCU内存管理，所以后面的device
ptr信息展示了这个指针属于申请哪块DCU内存（开发者只需要关注device
ptr的信息就可以了），以及这块DCU内存的大小（size byte）

总结

本节主要为开发过程中最常遇到的两类错误做了简单的介绍，这两类错误对应的检查方法也可以帮助开发者做初步的问题定位。但对错误和问题的定位并没有银弹，开发者实际遇到的问题可能更为棘手，这时，开发者对异构系统和代码的深入理解才是解决根本问题的关键所在。

B HIP调试工具

B.1 使用hipgdb调试程序

hipgdb是DCU Toolkit的调试分析工具，在GDB（GNU
Debugger）原有功能的基础上，增加了对DTK平台程序的调试功能，能够跟踪kernel执行流程，以及定位程序的异常错误。hipgdb主要提供两类重要功能：kernel执行过程的控制，包括断点插入、单步执行等；执行过程中环境信息的打印，包括线程、变量、寄存器、汇编指令等。

在正确加载DTK环境并及进入hipgdb后，可以使用help查看hipgdb的帮助信息。hipgdb的基本用法如下：

1.编译kernel时添加-ggdb、-O0选项；

```
hipcc –ggdb –O0 MatrixTranspose.cpp –o MatrixTranspose
```

2.使用hipgdb启动Host应用程序；

```
hipgdb ./MatrixTranspose
```

3.在kernel的入口函数打断点，matrixTranspose为核函数名称；

```
(gdb) break matrixTranspose
```

4.run执行应用程序，hipgdb会命中断点并停在核函数入口处。

```
(gdb) run
```

B.2 核函数调试具体过程

下面的核函数matrixTranspose实现了矩阵的转置运算，预设的kernel配置为[1,1,1]/(16,16,1)。

```
__global__ void matrixTranspose(float* out, float* in, const int width) {
  int x = hipBlockDim_x * hipBlockIdx_x + hipThreadIdx_x;
  int y = hipBlockDim_y * hipBlockIdx_y + hipThreadIdx_y;
  out[y * width + x] = in[x * width + y];
}
```

1. 编译并用hipgdb运行程序

```
$ hipcc –ggdb –O0 MatrixTranspose.cpp –o MatrixTranspose
$ hipgdb ./MatrixTranspose
GNU gdb (GDB) 11.1
Reading symbols from ./MatrixTranspose…
(gdb)
```

2. 在核函数内部插入断点

由于无法在Host代码端单步进入核函数，需要主动在核函数内部添加断点。核函数调试一般利用核函数名称或者源代码行号进行断点设置。

需要注意的是，由于设备端代码执行情况复杂，核函数的跟踪调试过程和CPU程序存在较大差异。一般设备端代码会被多个线程执行，断点也会命中多次，且以线程束warp为单位发送中断信号。在已命中断点的情况下，继续运行或者单步调试，会依次命中其它线程束warp的中断信号，导致无法跟踪核函数的执行过程。为了解决上述问题，最好按照如下四种方法进行断点处理，然后切换选定warp进行跟踪调试，不然可能造成异常情况的发生。

- 使用tbreak

使用tbreak来代替break添加断点，使断点只命中一次。

```
(gdb) tb matrixTranspose
Function "matrixTranspose" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Temporary breakpoint 7 (matrixTranspose) pending.
(gdb) c
Continuing.
[New Thread 0x7fffeb70f700 (LWP 774)]
[New Thread 0x7ffbc2bff700 (LWP 775)]
Device name Z100SM
[Switching to warp 1, lane 0 (HYDCU Lane 1:8:1:1/0 (0,0,0)[0,0,0])]
Warp 1 "MatrixTranspose" hit Temporary breakpoint 7, with lanes [0-63], matrixTranspose
(out=0x7ffbc0c00000, in=0x7ffbc1200000, width=0) at MatrixTranspose.cpp:40
40      int x = hipBlockDim_x * hipBlockIdx_x + hipThreadIdx_x;
(gdb) s
41      int y = hipBlockDim_y * hipBlockIdx_y + hipThreadIdx_y;
```

• 手动删除断点

在第一次命中断点后，手动禁用或者删除断点。

```
(gdb) b matrixTranspose
Function "matrixTranspose" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 12 (matrixTranspose) pending.
(gdb) c
Continuing.
[New Thread 0x7fffeb70f700 (LWP 780)]
[New Thread 0x7ffbc2bff700 (LWP 781)]
Device name Z100SM
[Switching to warp 1, lane 0 (HYDCU Lane 1:8:1:1/0 (0,0,0)[0,0,0])]
Warp 1 "MatrixTranspose" hit Breakpoint 12, with lanes [0-63], matrixTranspose
(out=0x7ffbc0c00000, in=0x7ffbc1200000, width=0) at MatrixTranspose.cpp:40
40      int x = hipBlockDim_x * hipBlockIdx_x + hipThreadIdx_x;
(gdb) info b
Num     Type           Disp Enb Address            What
12      breakpoint     keep y   0x00007ffff7e913dc in matrixTranspose(float*, float*, int) at
MatrixTranspose.cpp:40
    breakpoint already hit 1 time
(gdb) d 12
(gdb) s
41      int y = hipBlockDim_y * hipBlockIdx_y + hipThreadIdx_y;
```

• 设置条件断点

利用核函数的局部变量来设置条件断点，可以对感兴趣的线程行为进行跟踪。不过，核函数内部的条件断点只有进入核函数后才能设置。因此，一般先断到核函数后，再设置条件断点。而在命中条件断点后，最好也删除或禁用断点，再进行单步调试，不然会造成调试过程的卡顿。

```
(gdb) tb matrixTranspose
Function "matrixTranspose" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Temporary breakpoint 7 (matrixTranspose) pending.
(gdb) c
Continuing.
[New Thread 0x7fffeb70f700 (LWP 774)]
[New Thread 0x7ffbc2bff700 (LWP 775)]
Device name Z100SM
[Switching to warp 1, lane 0 (HYDCU Lane 1:8:1:1/0 (0,0,0)[0,0,0])]
Warp 1 "MatrixTranspose" hit Temporary breakpoint 7, with lanes [0-63], matrixTranspose
(out=0x7ffbc0c00000, in=0x7ffbc1200000, width=0) at MatrixTranspose.cpp:40
40      int x = hipBlockDim_x * hipBlockIdx_x + hipThreadIdx_x;
(gdb) b 43 if x==3
(gdb) c
Continuing.
[Switching to warp 1, lane 3 (HYDCU Lane 1:8:1:1/0 (0,0,0)[3,0,0])]
Warp 1 "MatrixTranspose" hit Breakpoint 3, with lanes [3 7 11 15], matrixTranspose
(out=0x7ffbc0c00000, in=0x7ffbc1200000, width=0) at MatrixTranspose.cpp:43
43      out[y * width + x] = in[x * width + y];
```

・ 设置scheduler-locking

和调试CPU程序的thread类似，如果只想调试当前warp，阻止其它warp执行，可以使用“set scheduler-locking on”等相关命令。

```
(gdb) b matrixTranspose
Function "matrixTranspose" not defined.
Make breakpoint pending on future shared library load? (y or [n]) y
Breakpoint 17 (matrixTranspose) pending.
(gdb) c
Continuing.
[New Thread 0x7fffeb70f700 (LWP 787)]
[New Thread 0x7ffbc29ff700 (LWP 788)]
Device name Z100SM
[Switching to warp 1, lane 0 (HYDCU Lane 1:8:1:1/0 (0,0,0)[0,0,0])]
Warp 1 "MatrixTranspose" hit Breakpoint 17, with lanes [0-63], matrixTranspose
(out=0x7ffbc0e00000, in=0x7ffbc1400000, width=0) at MatrixTranspose.cpp:40
40      int x = hipBlockDim_x * hipBlockIdx_x + hipThreadIdx_x;
(gdb) set scheduler-locking on
(gdb) show scheduler-locking
Mode for locking scheduler during execution is "on".
(gdb) s
41      int y = hipBlockDim_y * hipBlockIdx_y + hipThreadIdx_y;
(gdb) s
42      out[y * width + x] = in[x * width + y];
```

1. 环境信息查询

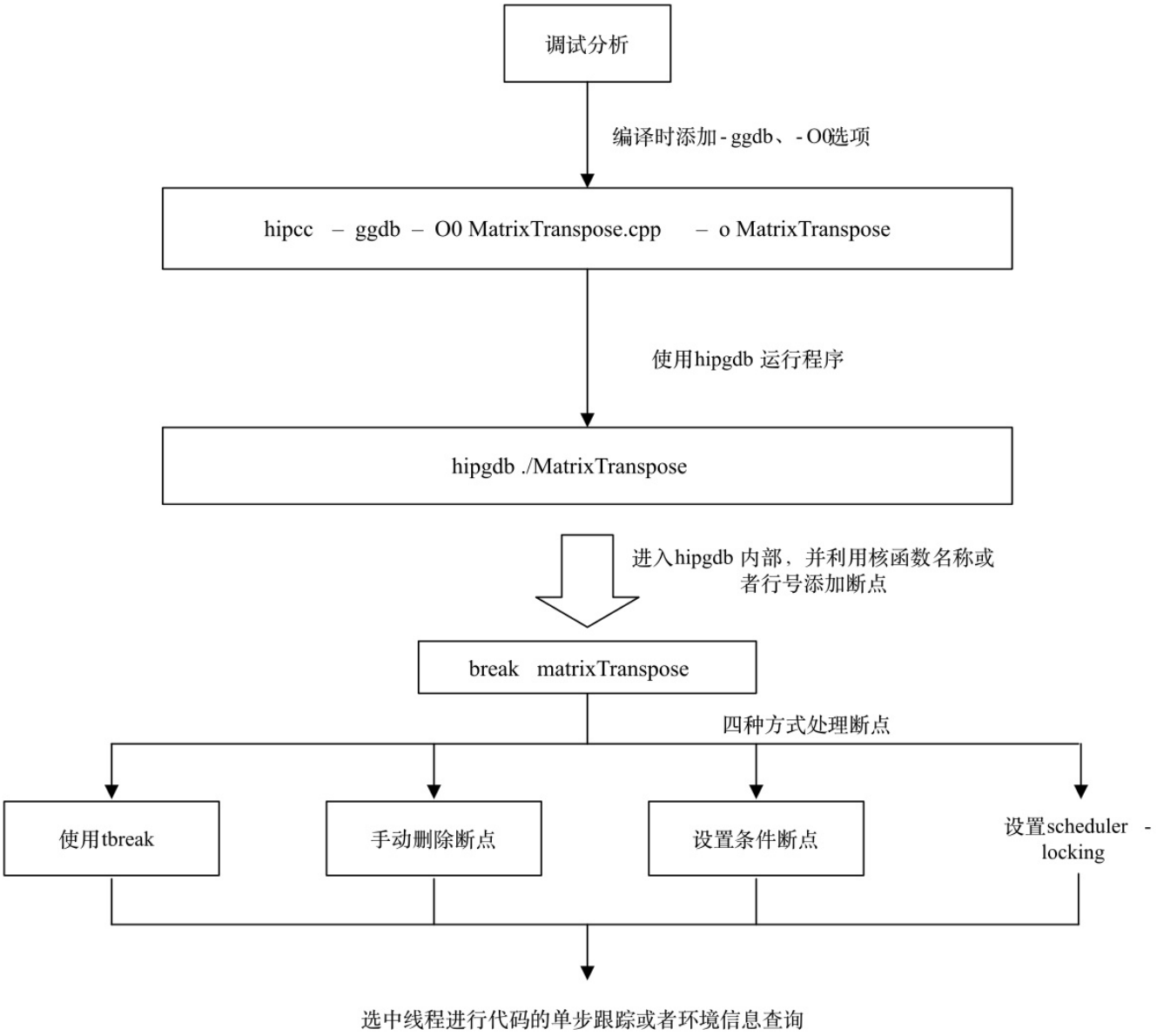
在命中kernel断点后，可直接利用warp和lane进行环境切换；也可以利用hipblock和hipthread的三维坐标进行环境切换。在切换选定线程束warp后，通过2中的方法处理断点，就可以跟踪调试选定的warp。注意在后续调试过程中不能再切换其它的warp，可以进行lane的切换。以下常见命令能够提供当前各种环境信息。

命令	描述
info devices	打印设备信息
show architecture	显示当前的架构信息
info queues	打印队列信息
info kernels	打印核函数配置信息
info registers	打印主要的寄存器信息。当寄存器存储重复的值，会用repeats简写，例如v0 {0x28 }
info all-registers	打印出所有寄存器，包括浮点寄存器和向量寄存
info registers regroup …	打印出某一组的寄存器，可以通过maint print regroups指令查询
info registers regname …	打印出指定名称的寄存器

print $valname	打印变量
info warps	打印线程束warp信息
warp id	切换线程束warp
info lanes	打印lane信息
lane id	切换lane
info hipblocks	打印block分配信息
hipblock x y z	依据坐标(x, y, z)，切换线程块
info hipthreads	打印thread分配信息
hipthread x y z	依据坐标[x, y, z]，切换线程
backtrace	在cpu端只会显示cpu端的调用栈，在gpu端只会示gpu端的调用栈
list	显示当前文件的源代码
info functions	显示出所有的函数名和类型
info variables	显示出非当前局部变量的变量类型和名称
info locals	显示出当前局部变量的变量类型和名称
info files	显示出当前hipgdb正在使用的文件名
info source	显示当前代码文件的信息
display /10i $pc	每次中断都自动打印10条指令

B.3 总结

[图片]



C hipGraph

C.1 hipGraph简介

hipGraph是一种静态计算图，由一系列操作（包括内核启动），这些操作是HIP中的基础操作，操作之间存在一定的关联关系，这些关联关系描述了操作之间是相互依赖的，还是相互并行独立的。这允许一个graph被定义一次，然后重复启动。

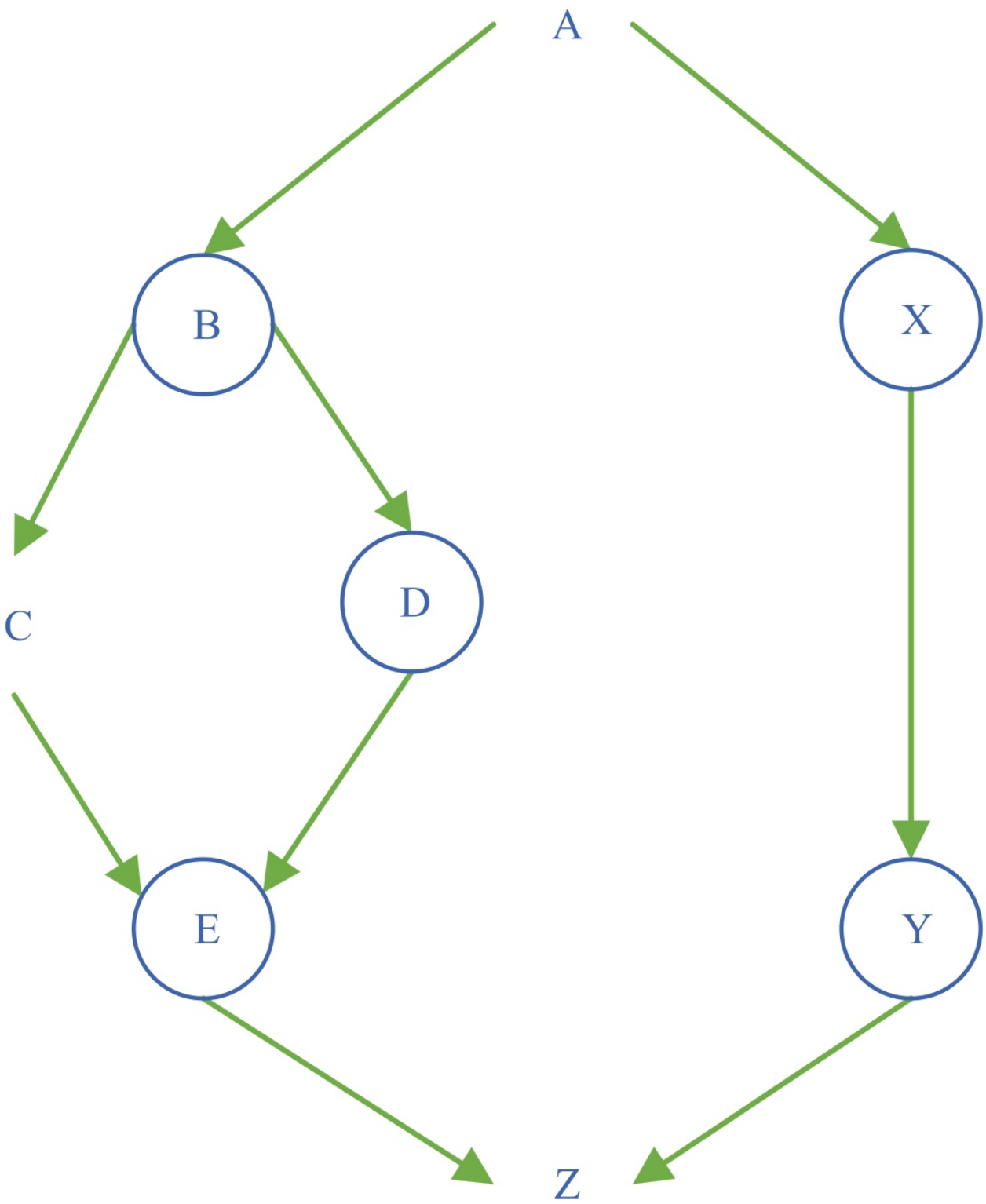
使用graph的工作提交分为三个不同的阶段：定义、实例化和执行。

在定义阶段，程序创建graph中操作的描述以及node之间的依赖关系。

在实例化阶段，获取graph模块的快照，对其进行验证，并执行大部分工作的设置和初始化，目的是最大限度的减少启动时需要完成的工作，生成的实例称为可执行图。

在执行阶段，可执行图启动到流中，类似其他异步操作，它可以在重复实例化的情况下启动任意次数。

[图片]



图C-1 hipGraph节点构成

一个操作在graph中形成一个节点；操作之间的依赖关系是边；这些依赖关系限制了操作的执行顺序。

一个操作可以在它所依赖的节点完成后随时调度。

C.2 函数接口说明

- hipGraph创建

```
hipError_t hipGraphCreate(hipGraph_t* pGraph,unsigned int flags);
```

参数说明：

pGraph： 返回新创建的graph, 输出参；

flags： graph创建的flags【必须是0】；

返回值：

hipSuccess、hipErrorInvalidValue、hipErrorMemoryAllocation

作用：创建一个空的graph，赋值给pGraph

使用：

```
hipGraph_t graph = nullptr;
hipError_t status = hipGraphCreate(&graph, 0);
```

- hipGraph实例化

```
hipError_t hipGraphInstantiate(hipGraphExec_t *pGraphExec, hipGraph_t graph,
hipGraphNode_t *pErrorNode,
char *pLogBuffer,
size_t bufferSize);
```

参数说明：

pGraphExec : 返回实例化后的graph 输出参；

graph : 将要实例化的graph

pErrorNode : 在实例化错误的情况下，可以修改它以指示导致错误的节点

pLogBuffer : 用于存储诊断消息的字符缓冲区

bufferSize： log buffer size

返回值：

hipSuccess or hipErrorInvalidValue

使用：

```
hipGraphExec_t graphExec = nullptr;
hipError_t status = hipGraphInstantiate(&graphExec,
```

```
graph, NULL, NULL, 0);
```

- hipGraph执行

hipError_t hipGraphLaunch(hipGraphExec_t graphExec,
hipStram_t stream);

参数说明：

graphExec : 将要启动的实例化后的graph

stream ： 在指定的stream上启动graph,需要提前申请创建stream；

返回值：

hipSuccess or hipErrorInvalidValue

使用：

```
hipError_t status = hipGraphLaunch(graphExec, streamForGraph);
```

- 释放资源

1. Graph exec的释放
```
hipError_t status = hipGraphExecDestory(graphExec);
```
2. graph的释放
```
hipError_t status = hipGraphDestroy(graph);
```

- kernel

hipError_t hipGraphAddKernelNode(
hipGraphNode_t *pGraphNode,
hipGraph_t graph,
const hipGraphNode_t *pDependencies,
size_t numDependencies,
const hipKernelNodeParams *pNodeParams);

参数说明：

pGraphNode : 返回参，创建kernel类型的graph node

graph : 将memcpy node添加到的graph

pDependencies : 依赖node集合 [可为NULL]

numDependencies : 依赖项个数

[如果pDependencies为NULL，依赖个数须为0]

pNodeParams : GPU执行函数参数，核函数相关参数

参数结构体定义如下：

```
typedef struct hipKernelNodeParams {
```

```
dim3 blockDim;
void** extra;
void* func;
dim3 gridDim;
void** kernelParams;
unsigned int sharedMemBytes;//对于每个线程block可共享的内存
} hipKernelNodeParams;
```

返回值：

hipSuccess、hipErrorInvalidValue、hipErrorInvalidDevice

使用：

```
std::vector<hipGraphNode_t> nodeDependencies;
hipGraphNode_t kernelNode = nullptr;
hipKernelNodeParams kernelNodeParams = {0};
void* kernelArgs[4] = {(void*)&inputVec_d,
(void*)&outputVec_d,
&inputSize, &numOfBlocks};
kernelNodeParams.func = (void*)add; //kernel function name
kernelNodeParams.gridDim = dim3(gridSize, 1, 1);
kernelNodeParams.blockDim = dim3(blockSize, 1, 1);
kernelNodeParams.extra = NULL;
kernelNodeParams.sharedMemBytes = 0;
kernelNodeParams.kernelParams = (void**)kernelArgs;
hipError_t status = hipGraphAddKernelNode(&kernelNode, graph,
NULL, 0,
&kernelNodeParams);
```

如有依赖项，则：

```
nodeDependencies.push_back(memcpyNode);
nodeDependencies.push_back(memsetNode);
hipError_t status = hipGraphAddKernelNode(&kernelNode, graph,
nodeDependencies.data(),
nodeDependencies.size(),
&kernelNodeParams);
```

- 接口支持

表C-1 DTK支持的hipGraph接口

DTK支持函数名	接口描述
hipGraphAddDependencies	添加依赖项
hipGraphAddEmptyNode	添加empty node
hipGraphAddEventRecordNode	添加event record node
hipGraphAddEventWaitNode	添加event wait node

hipGraphAddHostNode	添加host execution node
hipGraphAddKernelNode	添加kernel exection node
hipGraphAddMemcpyNode1D	添加1D memcpy node
hipGraphAddMemcpyNodeFromSymbol	添加memcpy node 通过设备的symbol拷贝
hipGraphAddMemcpyNodeToSymbol	添加memcpy node 拷贝到设备的symbol
hipGraphAddMemsetNode	添加memset node
hipGraphCreate	创建graph
hipGraphDestroy	释放graph
hipGraphEventRecordNodeGetEvent	获取关联node的event
hipGraphEventRecordNodeSetEvent	设置记录node的event
hipGraphEventWaitNodeGetEvent	获取关联等待node的event
hipGraphEventWaitNodeSetEvent	设置等待node的event
hipGraphExecDestroy	释放可执行graph
hipGraphExecKernelNodeSetParams	为可执行graph指定的内核节点设置参数
hipGraphMemsetNodeSetParams	为可执行graph指定的memset 节点设置参数
hipGraphGetEdges	获取graph中依赖关系
hipGraphGetNodes	获取graph的节点信息
hipGraphGetRootNodes	获取graph中根节点信息
hipGraphHostNodeGetParams	获取graph中host node的参数
hipGraphHostNodeSetParams	为graph中指定的host node设置参数
hipGraphInstantiate	实例化
hipGraphKernelNodeSetParams	为graph中指定的kernel node设置参数
hipGraphLaunch	启动graph
hipGraphMemcpyNodeGetParams	获取graph中memcpy node的参数
hipGraphMemcpyNodeSetParams	设置graph 中memcpy node的参数
hipGraphMemsetNodeGetParams	获取graph中memset node的参数
hipGraphMemsetNodeSetParams	设置graph 中memset node的参数

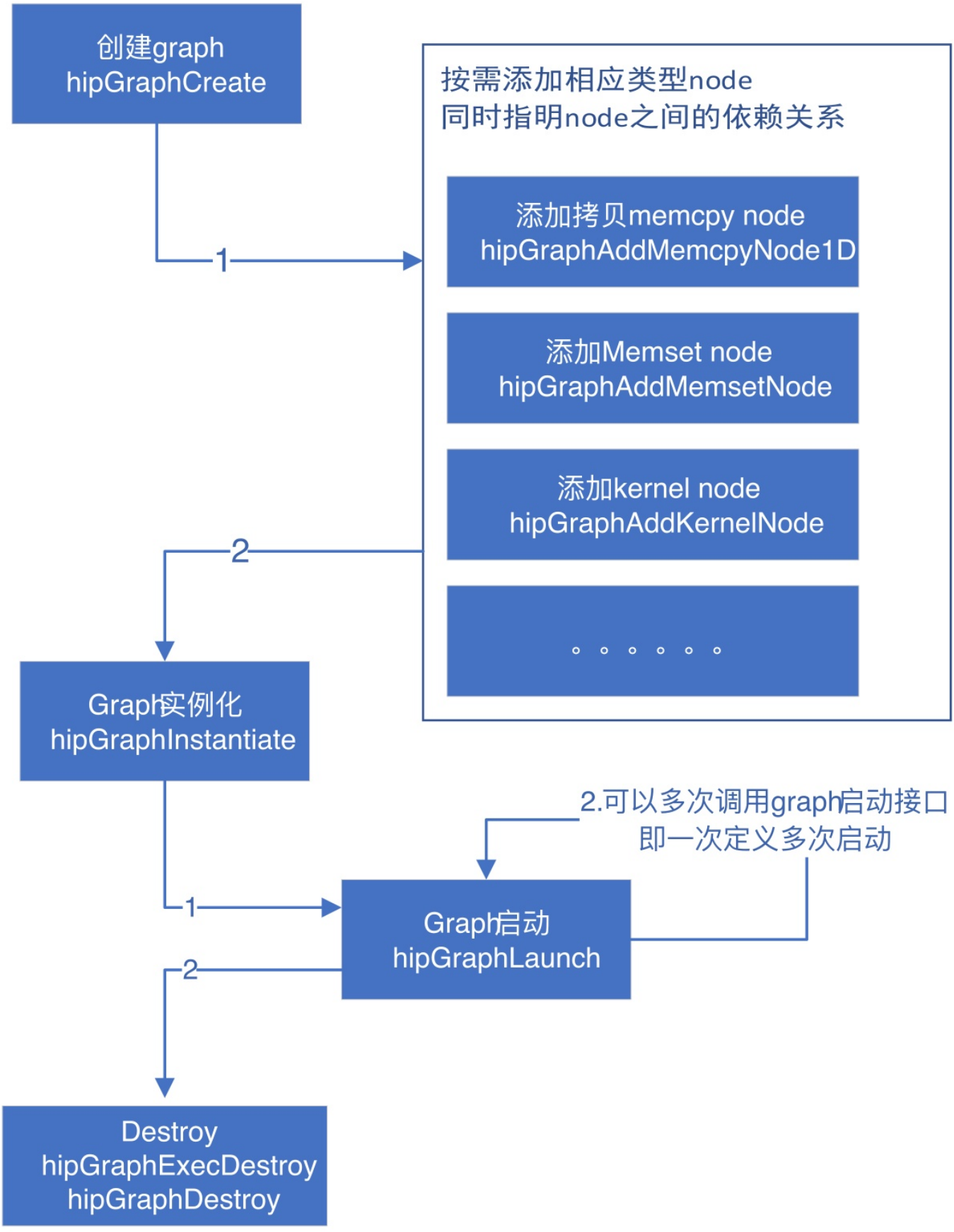
C.3 示例

- 重复计算场景

使用场景：适用于重复执行完整程序，即多次计算获取结果

编程流程概括：

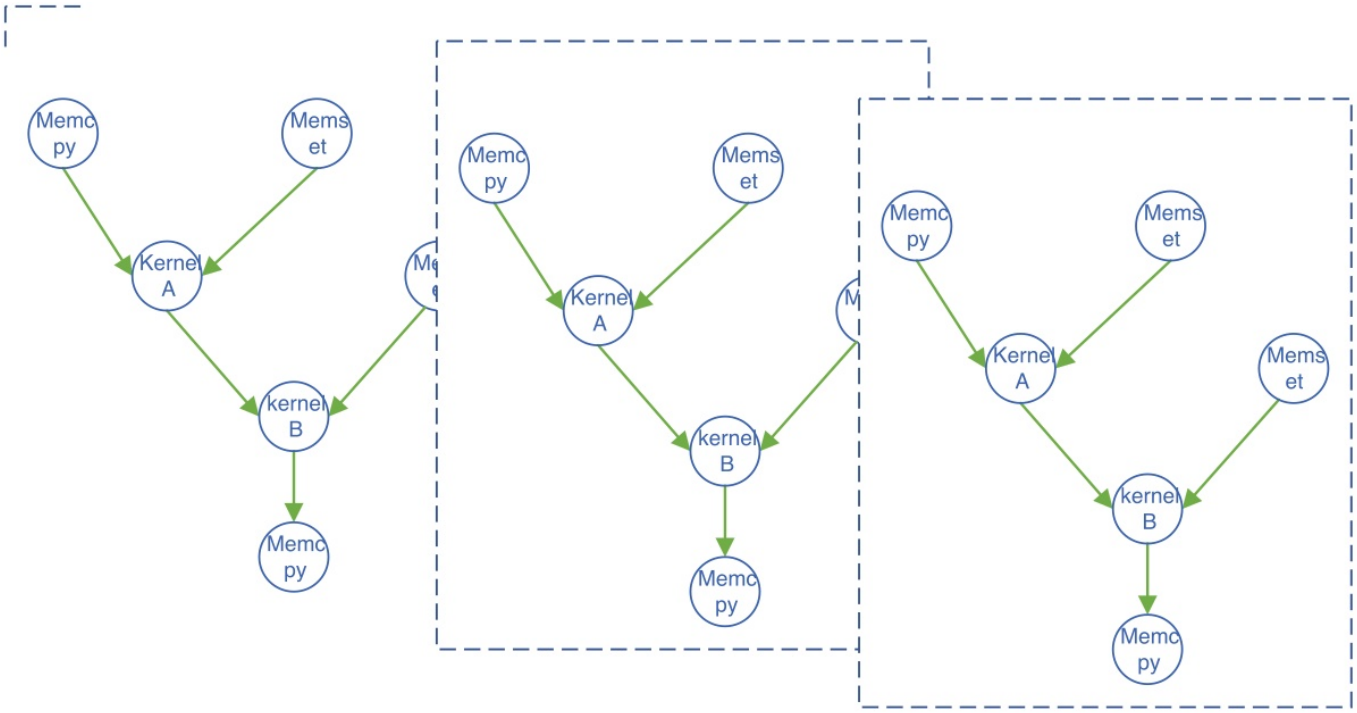
[图片]



图C-2 hipGraph针对重复启动编程流程

模型如下：

[图片]



图C-3 重复计算启动Graph模型

参考编程如下：

```
#define HIP_CHECK(stat)                    \
({                                  \
hipError_t restat = stat;                \
if(restat != hipSuccess){                \
printf("Error: hip error in line %d,stat:%d\n",__LINE__,
restat) ;                                \
}                                  \
})
//创建graph
hipGraph_t graph;
hipStream_t streamForGraph;
std::vector<hipGraphNode_t> nodeDependencies;
hipGraphNode_t memcpyNode, kernelNode, memsetNode;
double *result_h = NULL;
//申请资源
HIP_CHECK(hipHostMalloc(&inputVec_h, sizeof(float) * size,
                hipHostMallocDefault));
HIP_CHECK(hipMalloc(&inputVec_d, sizeof(float) * size));
HIP_CHECK(hipMalloc(&outputVec_d, sizeof(double) *maxBlocks));
HIP_CHECK(hipMalloc(&result_d, sizeof(double)));
HIP_CHECK(hipHostMalloc(&result_h, sizeof(double),
 hipHostMallocDefault));
```

```
//初始化主机端
for (int i = 0; i < size; i++) {
inputVec_h[i] = 1.0f;
inputVec_2_h[i] = 2.0f;
result_h[i] = 0.0f;
}
//创建stream
HIP_CHECK(hipStreamCreate(&streamForGraph));
//创建hipGraph
HIP_CHECK(hipGraphCreate(&graph, 0));
//拷贝 H2D
HIP_CHECK(hipGraphAddMemcpyNode1D(&memcpyNode, graph,
NULL, 0,
inputVec_d, inputVec_h,
sizeof(float)*inputSize,
hipMemcpyHostToDevice));
//Memset接口
hipMemsetParams memsetParams = {0};
memsetParams.dst       = (void*)outputVec_d;
memsetParams.value     = 0;
memsetParams.pitch     = 0;
memsetParams.elementSize = sizeof(float);
memsetParams.width     = numOfBlocks * 2;
memsetParams.height    = 1;
HIP_CHECK(hipGraphAddMemsetNode(&memsetNode, graph, NULL, 0,
&memsetParams));
//添加依赖
nodeDependencies.push_back(memcpyNode);
nodeDependencies.push_back(memsetNode);
//核函数添加，有两个依赖项
hipKernelNodeParams kernelNodeParams = {0};
void* kernelArgs[4] = {(void*)&inputVec_d,
(void*)&outputVec_d,
&inputSize,
&numOfBlocks};
kernelNodeParams.func = (void*)reduce;
kernelNodeParams.gridDim = dim3(inputSize/THREADS_PER_BLOCK,1,1);
kernelNodeParams.blockDim = dim3(THREADS_PER_BLOCK, 1, 1);
kernelNodeParams.sharedMemBytes = 0;
kernelNodeParams.kernelParams  = (void**)kernelArgs;
kernelNodeParams.extra        = NULL;
HIP_CHECK(hipGraphAddKernelNode(&kernelNode, graph,
nodeDependencies.data(),
nodeDependencies.size(),
&kernelNodeParams));
//添加依赖
nodeDependencies.clear();
nodeDependencies.push_back(kernelNode);
```

```
//Memset接口
memset(&memsetParams, 0, sizeof(memsetParams));
memsetParams.dst = result_d;
memsetParams.value = 1;
memsetParams.elementSize = sizeof(double);
memsetParams.width = 1;
memsetParams.height = 1;
HIP_CHECK(hipGraphAddMemsetNode(&memsetNode, graph, NULL, 0,
&memsetParams));
nodeDependencies.push_back(memsetNode);

//核函数添加，两个依赖项
memset(&kernelNodeParams, 0, sizeof(kernelNodeParams));
kernelNodeParams.func     = (void*)reduceFinal;
kernelNodeParams.gridDim  = dim3(1, 1, 1);
kernelNodeParams.blockDim = dim3(THREADS_PER_BLOCK, 1, 1);
kernelNodeParams.extra    = NULL;
kernelNodeParams.sharedMemBytes = 0;
void* kernelArgs2[3] = {(void*)&outputVec_d,
(void*)&result_d,
&numOfBlocks};
kernelNodeParams.kernelParams = kernelArgs2;
HIP_CHECK(hipGraphAddKernelNode(&kernelNode, graph,nodeDependencies.data(),
nodeDependencies.size(),&kernelNodeParams));
//拷贝，D2H
nodeDependencies.clear();
nodeDependencies.push_back(kernelNode);
HIP_CHECK(hipGraphAddMemcpyNode1D(&memcpyNode, graph,
nodeDependencies.data(),
nodeDependencies.size(),
result_h, result_d,
sizeof(double),
hipMemcpyDeviceToHost));
//获取node信息
hipGraphNode_t hostNode;
hipGraphNode_t nodes[5] = {};
size_t numNodes = 5;
HIP_CHECK(hipGraphGetNodes(graph, nodes, &numNodes));
HIP_CHECK(hipGraphGetRootNodes(graph, nodes, &numNodes));
//hipGraph实例化接口
hipGraphExec_t graphExec;
HIP_CHECK(hipGraphInstantiate(&graphExec, graph, NULL, NULL, 0));

//启动hipGraph，可多次调用 [GRAPH_LAUNCH_ITERATIONS 启动次数]
for(int i = 0; i < GRAPH_LAUNCH_ITERATIONS; i++){
HIP_CHECK(hipGraphLaunch(graphExec, streamForGraph));
}
//stream 同步
```

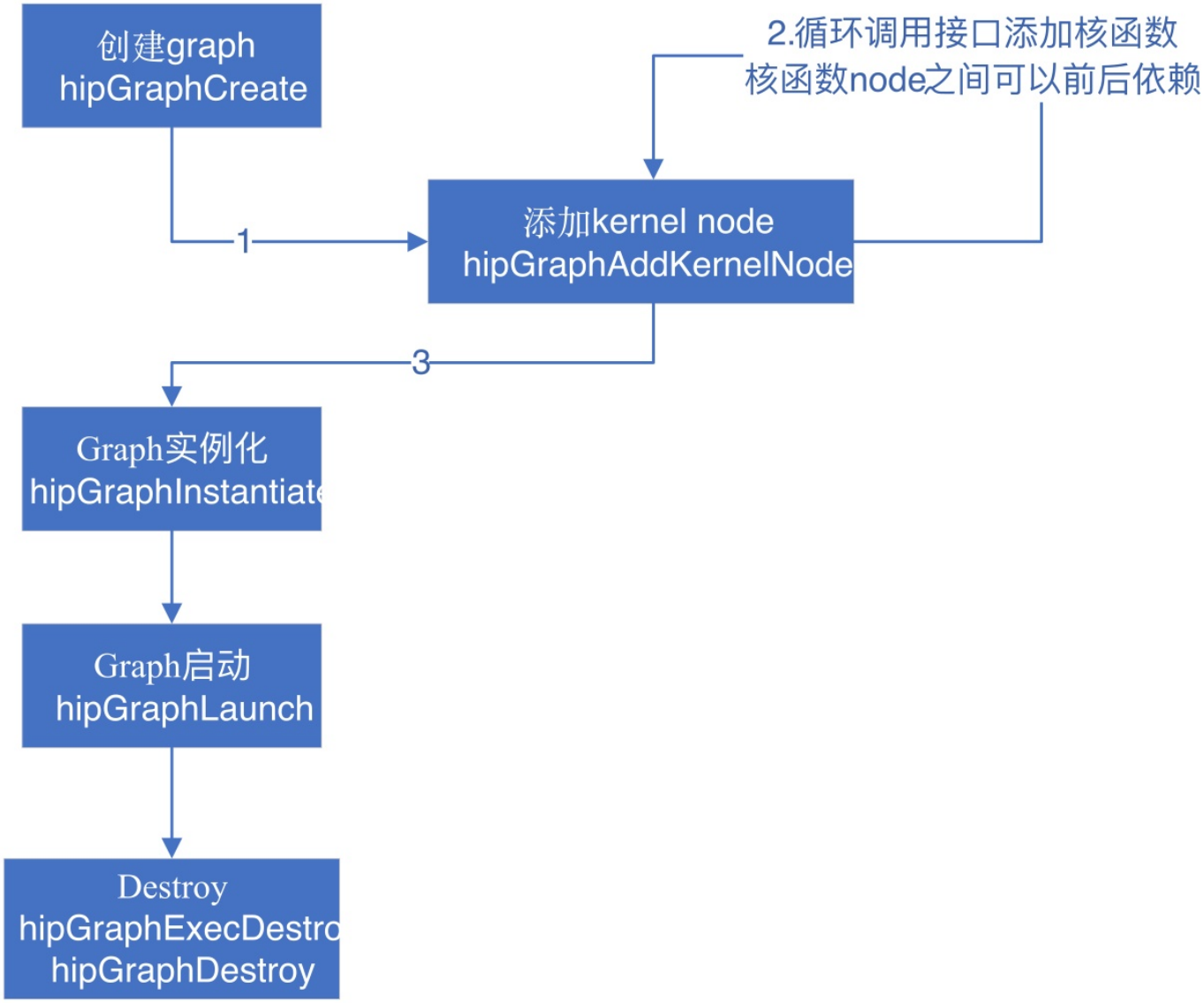
```
HIP_CHECK(hipStreamSynchronize(streamForGraph));
//destroy (graph & graphExec & streamForGraph)
HIP_CHECK(hipGraphExecDestroy(graphExec));
HIP_CHECK(hipGraphDestroy(graph));
HIP_CHECK(hipStreamDestroy(streamForGraph));
//释放资源
HIP_CHECK(hipHostFree(inputVec_h));
HIP_CHECK(hipFree(inputVec_d));
HIP_CHECK(hipFree(outputVec_d));
HIP_CHECK(hipFree(result_d));
```

2.大量轻型kernel场景

适用场景：对大量轻型kernel处理

编程流程概括：

[图片]



图C-4 hipGraph对大量轻型kernel函数编程概括

参考编程如下：

```
#define failed(...)                                 \
printf("%serror: ", KRED);                          \
printf(__VA_ARGS__);                                \
printf("\n");                                       \
printf("error: TEST FAILED\n%s", KNRM);             \
abort();
#define HIPCHECK(error)                             \
{                                           \
hipError_t localError = error;                      \
if ((localError != hipSuccess) &&
(localError != hipErrorPeerAccessAlreadyEnabled)) {\
printf("%serror: '%s'(%d) from %s at %s:%d%s\n", KRED, hipGetErrorString(localError),  localError, #
error, __FILE__, __LINE__, KNRM);                   \
failed("API returned error code.");                 \
}                                           \
}
__global__ void add(float *d_x, float *d_y, float *d_z, size_t N) {
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < N) {
d_z[i] = d_x[i] + d_y[i];
}
}
//申请资源，并对主机端进行初始化
HIPCHECK(hipHostMalloc(&inputVec_h, sizeof(float) * size,
hipHostMallocDefault));
HIPCHECK(hipHostMalloc(&inputVec_2_h, sizeof(float) * size,
hipHostMallocDefault));
HIPCHECK(hipHostMalloc(&result_h, sizeof(float) * size,
hipHostMallocDefault));
for (int i = 0; i < size; i++) {
inputVec_h[i] = 1.0f;
inputVec_2_h[i] = 2.0f;
result_h[i] = 0.0f;
}
HIPCHECK(hipMalloc(&inputVec_d, sizeof(float) * size));
HIPCHECK(hipMalloc(&inputVec_2_d, sizeof(float) * size));
HIPCHECK(hipMalloc(&outputVec_d, sizeof(float) * size));
//创建stream
hipStream_t streamForGraph;
HIPCHECK(hipStreamCreate(&streamForGraph));
//拷贝接口，H2D
HIPCHECK(hipMemcpyAsync(inputVec_d, inputVec_h,
sizeof(float) * inputSize,
hipMemcpyHostToDevice,
streamForGraph));
HIPCHECK(hipMemcpyAsync(inputVec_2_d, inputVec_2_h,
sizeof(float) * inputSize,
hipMemcpyHostToDevice,
```

```
streamForGraph));
//创建graph
hipGraph_t graph;
HIPCHECK(hipGraphCreate(&graph, 0));
//核函数添加
hipGraphNode_t kernelNode;
hipKernelNodeParams kernelNodeParams;
void *kernelArgs[4] = {(void *)&inputVec_d,
(void *)&inputVec_2_d,
(void *)&outputVec_d,
&size};
memset(&kernelNodeParams, 0, sizeof(kernelNodeParams));
//循环添加kernel
for (int i = 0; i < kernelNum; i++) {
kernelNodeParams.func = (void *) add;
kernelNodeParams.gridDim = dim3(gridSize, 1, 1);
kernelNodeParams.blockDim = dim3(blockSize, 1, 1);
kernelNodeParams.sharedMemBytes = 0;
kernelNodeParams.kernelParams = kernelArgs;
kernelNodeParams.extra = NULL;
HIPCHECK(hipGraphAddKernelNode(&kernelNode, graph, NULL, 0,
&kernelNodeParams));
}
//hipGraph实例化
hipGraphExec_t graphExec;
HIPCHECK(hipGraphInstantiate(&graphExec, graph, NULL, NULL, 0));
//hipGraph 启动
HIPCHECK(hipGraphLaunch(graphExec,streamForGraph));
//stream 同步
HIPCHECK(hipStreamSynchronize(streamForGraph));
//拷贝接口，D2H
HIPCHECK(hipMemcpyAsync(result_h, outputVec_d,
sizeof(float) * inputSize,
hipMemcpyDeviceToHost,
streamForGraph));
//destroy (graph & graphExec & streamForGraph)
HIPCHECK(hipGraphExecDestroy(graphExec));
HIPCHECK(hipGraphDestroy(graph));
HIPCHECK(hipStreamDestroy(streamForGraph));
//释放资源
HIPCHECK(hipFree(inputVec_d));
HIPCHECK(hipFree(inputVec_2_d));
HIPCHECK(hipFree(outputVec_d));
HIPCHECK(hipHostFree(inputVec_h));
HIPCHECK(hipHostFree(inputVec_2_h));
HIPCHECK(hipHostFree(result_h));
```

D Cooperative Groups

D.1 Cooperative Groups简介

Cooperative
Groups（cg）用于组织通信线程。cg允许开发人员调整线程通信的粒度，帮助他们表达更丰富、更有效的并行分解。

前期编程模型为同步协作线程提供了一个单一、简单的构造：线程块的所有线程之间的屏障，如使用__syncthreads()
内部函数实现的那样。如果想以其他粒度定义和同步线程组，以“集体”组范围功能接口的形式实现更高的性能、设计灵活性和软件重用。cg通过提供安全且面向未来的机制来启用高性能代码解决这个问题。

Cooperative Groups是在现有3D
Grid结构，即grid-block-thread结构下，重新对计算线程划分的功能，其特点是可以进行跨block线程组定义和block内线程组定义，即可以将blcok内线程进行划分，也可以将不同blcok内的线程进行合并。

Coopreative
Groups机制，用于线程块内和跨线程块的同步。该机制为开发者提供了自定义线程组的方式，并提供了相应的同步函数，同时还包括一个新的kernel启动API（hipLaunchCooperativeKernel），该API保证了cooperative groups同步的安全性。

D.2 块内同步

- thread_block

数据结构： thread_block，即线程块。

获取并初始化：this_thread_block()；

```
thrad_block tg = this_thread_block();
```

thread_block继承自结构：thread_group

thread_group提供如下公开成员函数：

```
void sync(); //同步组内的所有线程，这个tg.sync() 等价于__syncthreads();
uint32_t size();  //获取组内的线程数目
uint32_t thread_rank(); //获取线程的组内索引值[0, size);
bool is_valid();  //判断本组是否违背了任何APIConstraints（API限制）
dim3 group_index(); //网格内3维索引（block索引）
dim3 thread_index();//块block内3维索引（thread索引）
```

注：以上所有操作组内所有线程都要确保执行到，否则行为未定义。

优点：

使用tg.sync()的好处在于避免隐式同步隐患，如：

```
__device__ int sum(const thread_group& tg, int *x, int n){
//……
tg.sync();
return total;
}
__global__ void parallel_kernel(float *x){
//……
//entire thread block must call sum
sum(this_thread_block(), x, n);
}
```

- 分片

函数：tiled_partition(const thread_group parent, unsigned int tilesz);

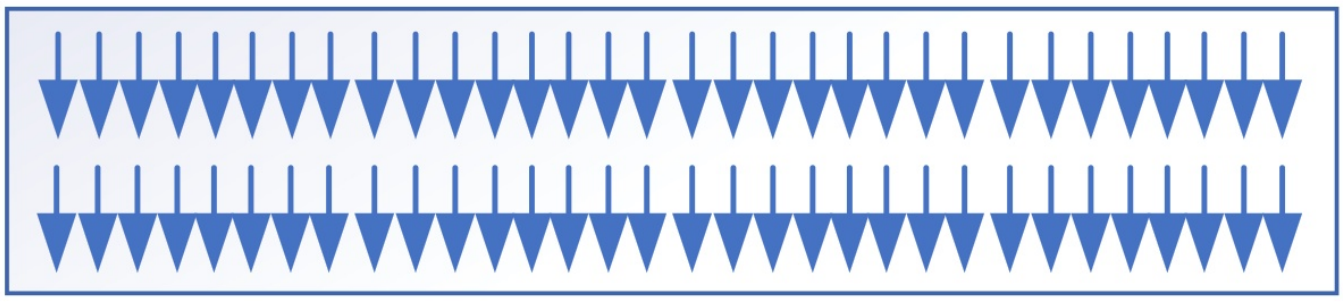
作用：用于将一个线程块分解为多个小的协同线程组（tiled subgroups）

参数：第二参数Size必须是2的幂且小于或等于64

例如：

```
thread_blcok wholeBlock = this_thread_block();  //获取线程块
```

[图片]

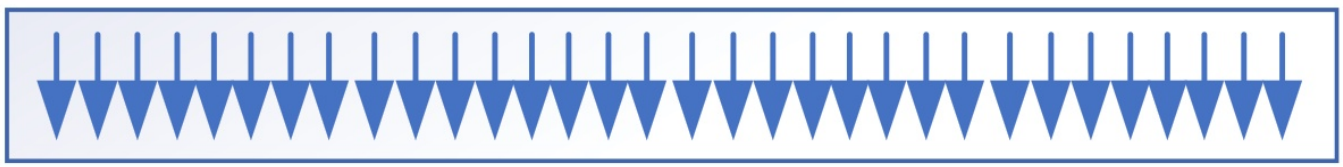


图D-1 获取整个线程块 共64个线程

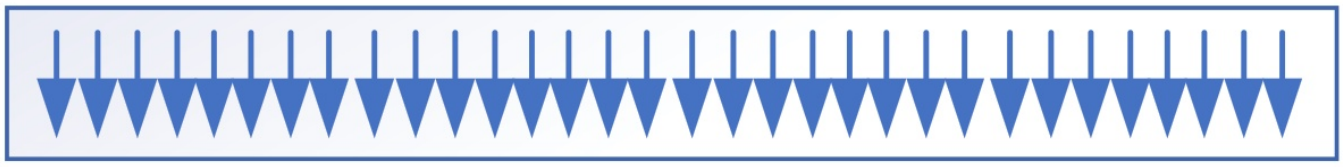
//将线程块分解为若干个大小为32的小线程组：

```
thread_group tile32 = tiled_partition(wholeBlock, 32);
```

[图片]



[图片]

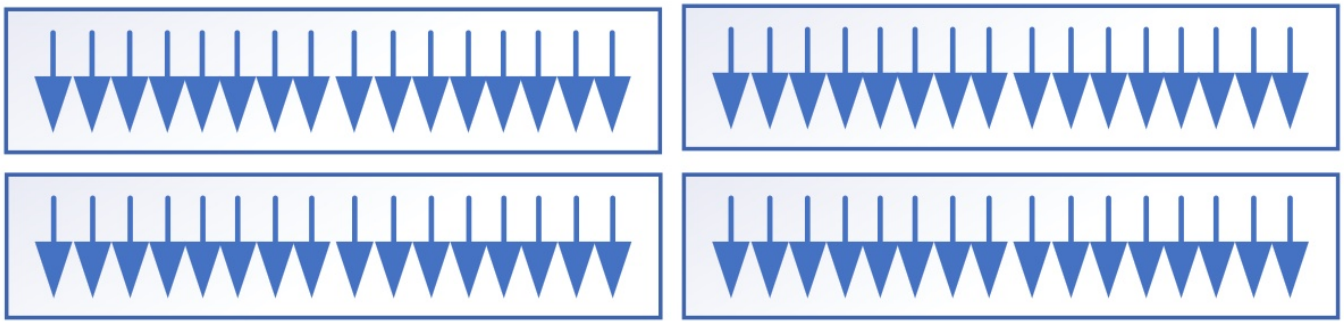


图D-2 分解成大小为32的线程组

//将tile32分解为更小的若干个大小为16的小线程组

```
thread_group tile16 = tiled_partition(tile32, 16);
```

[图片]

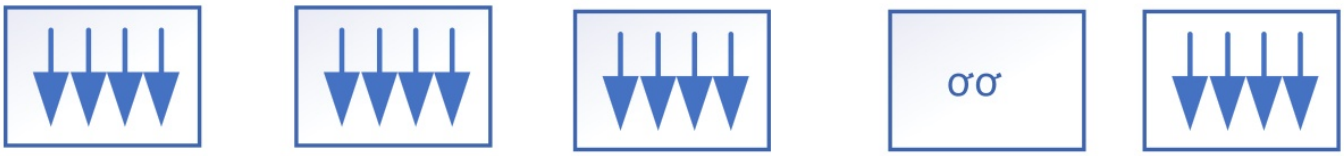


图D-3 分解成大小为16的线程组

//将tile32分解为更小的若干个大小为4的小线程组

```
thread_group tile4 = tiled_partition(tile32, 4);
```

[图片]



图D-4 分解成大小为4的线程组

//通过如下操作可以让线程0，4，8，12…等线程打印“hello world”

```
if ( 0 == tile4.thread_rank() ){
printf( “hello world \n” );
}
```

- 分片（模板化）

模板函数：tiled_partition

模板参数用来指定分片的大小—-其在编译期就确定下来，从而有着更多的执行优化空间。

template class thread_block_tile;

通过以下构建：

```
Template <unsigned int Size, typename ParentT = void>
thread_block_tile<Size, ParentT> tiled_partition(const ParentT& g);
```

Size必须是2的幂且小于或等于64

ParentT是从其中划分该组的父类型。它是自动推断的，但void的值会将此信息存储在该组句柄中而不是类型中

示例：

```
thread_block_tile<32> tile32 = tiled_partition<32>(this_thread_block());
thread_block_tile<4>  tile4  = tiled_partition<4>(this_thread_block());
```

注：这里使用thread_block_tile模板化数据结构，而且组大小是以模板参数而非函数参数传给tile_partition()函数的。

thread_block_tile 提供如下成员函数用于协同同步（即集合函数、洗牌指令函数）：

```
.shfl();                    //等价__shfl_sync();
.shfl_down();                 //等价__shfl_down_sync();
.shfl_up();                  //等价__shfl_up_sync();
.shfl_xor();                  //等价__shfl_xor_sync();
.any();                    //等价__any_sync();
.all();                   //等价__all_sync();
.ballot();                   //等价__ballot_sync();
.match_any();                 //等价__match_any_sync();
.match_all();                //等价__match_all_sync();
void sync() const;           //协作组内线程同步
unsgined long long num_threads(); //协作组内线程总数
unsigned long long thread_rank(); //调用线程在组内索引[0, num_threads)
```

示例： 设置Per-Tile of 16 threads

```
template <unsigned size>
__device__ int tile_reduce(thread_block_tile<size> g, int val){
//Each iteration halves the number of active threads
//Each thread adds its partial sum[i] to sum[lane+i];
for (int i = g.size() / 2; i > 0; i /= 2) {
val += g.shfl_down(val, i);
}
return val;    //note: only thread 0 will return full sum
}
template < unsigned tile_sz>
__global__ void sum_kernel_tile_shfl(int *sum, int *input, int n){
int my_sum = thread_sum(input, n);
//tile_sz = 16
auto tile = tiled_partition<tile_sz>(this_thread_block());
int tile_sum = tile_reduce<tile_sz>(tile, my_sum);
if(tile.thrand_rank() == 0){
atomicAdd(sum, tile_sum);
}
}
```

- coalesced_threads

  若同一线程束内的线程出现条件分化（通常由if语句导致），那么程序将序列化运行：既在执行某分支线程时停止其他分支线程的执行，直到所有分支执行完毕。

对当前执行分支上的活跃线程称为coalesced
thread，线程束内所有活跃线程组成的线程组即为coalesced

groups，coalesced_group也是继承类thread_group，通过coalesced_threads()构造组句柄是伺机的，即在某时刻返回一组活跃线程，并且不保证返回哪些线程（只要它们是活跃的）或者它们在整个执行过程中保持合并（它们将被重新组合在一块以执行一个集合，但之后可以再次发散）

通过coalesced_threads函数获取：

```
coalesced_group active = coalesced_threads();
```

提供如下成员函数用于协同同步（即集合函数、洗牌指令函数）：

```
.shfl();                    //等价__shfl_sync();
.shfl_down();                  //等价__shfl_down_sync();
.shfl_up();                   //等价__shfl_up_sync();
.shfl_xor();                   //等价__shfl_xor_sync();
.any();                    //等价__any_sync();
.all();                   //等价__all_sync();
.ballot();                   //等价__ballot_sync();
.match_any();                  //等价__match_any_sync();
.match_all();                 //等价__match_all_sync();
void sync() const;               //协作组内线程同步
unsgined long long num_threads() const; //协作组内线程总数
unsigned long long thread_rank();//调用线程在组内索引[0, num_threads)
```

示例场景：

假设一个线程块有8个线程，此线程块中的第2、4、8线程为分支上的活跃线程，对应索引为1、3、7，其他线程被阻塞，通过coalesced_threads获取活跃线程名为cg的线程组，此时cg包含三个活跃线程（组内id分别为0、1、2），如下图：

[图片]

图D-5 coalsced groups示意

D.3 网格级同步

cooperative Groups最强大的能力在于跨线程块同步。

之前不同线程块仅能在kernel执行结束时同步，现在可以通过grid_group结构执行网格级同步，该组对象表示在单个网格中启动的所有线程。继承自thread_group即：

```
__global__ void myKenrel(…) {
grid_group grid = this_grid();
//load data
//TODO:loop – compute, share data
grid.sync();
```

```
}
```

网格级同步必须通过hipLaunchCooperativeKernel API配置并启动kernel， 函数接口如下：

```
hipError_t hipLaunchCooperativeKernel(
const T *func,                    //kernel函数指针
dim3 gridDim,
dim3 blockDime,
void **args,                      //kernel参数数组
size_t sharedMem = 0,
hipStream_t stream = 0
)
```

注：为保证所有协同线程块能安全的常驻GPU，gridDim和blockDim的值需要慎重考虑，可以通过SIMD的最大活跃线程块数目来最大化并行率，如：

```
hipOccupancyMaxActiveBlocksPerMultiprocessor(&numBlocks,
myKernel,
numThreads, 0);
hipGetDeviceProperties(&deviceProp, device);
// initialize , then  launch
hipLaunchCooperativeKernel( (void *)myKernel,
deviceProp.multiProcessorCount * numBlocks,
numThreads,args);
```

Cooperative
Groups目前不支持任务抢占和调度，若一次启动的block数超过了设备驻留的的极限，则报错“too many blocks in cooperative launch
hipLaunchCooperativeKernel”,此时需要检查一下启动的block数、使用的共享内存大小、使用的寄存器大小。

查询是否支持网格级同步方式：

```
hipGetDeviceProperties(&deviceProperties, deviceId);
```

对deviceProperties.cooperativeLaunch 进行判断

当deviceProperties.cooperativeLaunch == 1时 表明设备支持网格级同步。

公开成员函数：

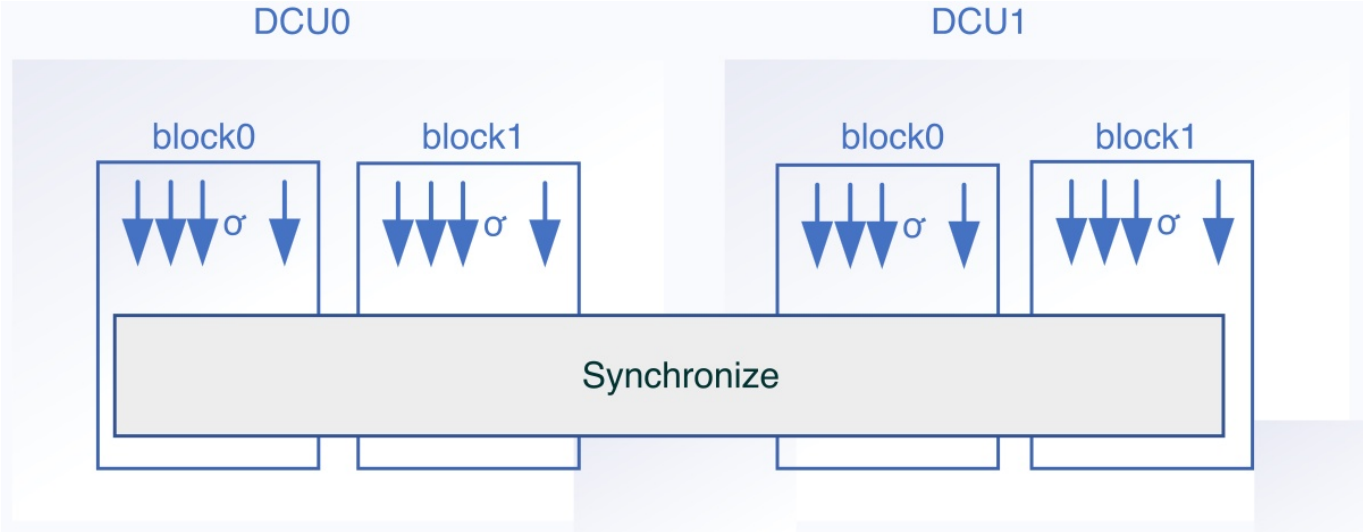
```
bool is_valid() const;    //返回grid_group是否能进行同步
void sync() const;        //网格内部的线程同步
unsigned long long thread_rank();//获取线程的组内索引值[0, num_threads)
unsigned long long block_rank();//获取线程块的组内索引值[0, num_blocks)
unsigned long long num_threads();//在网格内线程的总数
dim3 dim_blocks();        //启动网格的尺寸（以block为单位）
dim3 block_index();       //在启动网格内block的3维索引
unsigned long long size();  //group内的线程总数（num_threads()的别名）
dim3 group_dim();         //启动网格的尺寸（dim_blocks()的别名）
```

D.4 多设备同步

类似网格级同步，多设备同步通过mult_grid_group结构执行，该组对象表示跨设备协作组启动的所有设备启动的所有线程，继承自thread_group：

```
__global__ void myKernel(…) {
multi_grid_group multi_grid = this_multi_grid();
//load data
//TODO: loop – compute, share data
multi_grid.sync();
}
```

[图片]



图D-6 多设备同步示意图

启动kernel方式API：hipLaunchCooperativeKernelMultiDevice，接口信息如下：

```
hipError hipLaunchCooperativeKernelMultiDevice{
HIP_LAUNCH_PARAMS *launchParamsList,
unsigned int numDevices,
unsigned int flags = 0};
```

相应编程示例如下：

```
struct hipLuanchParams params[numDevices];
for (int i = 0; i < numDevices; i++){
params[i].func      = (void *)myKernel;
params[i].gridDim   = dim3(…);
params[i].blockDim  = dim3(…);
params[i].sharedMem = …;
params[i].stream    = …;   //cannot use the NULL stream
params[i].args      = …;
}
hipLaunchCooperativeKernelMultiDevice(params, numDevices);
```

注意事项：

・
该API将确保一个launch操作是原子性，例如当API调用成功时，相应数目的线程块在所有指定设备上的launch成功。
・ 对应所有设备，该API调用的Kenrenl函数必须是相同。
・ 同一设备上的launchParamsList参数必须是相同的。
・ 所有设备的计算能力必须是相同（major and minor version）
・
对于所有设备，配置的网格大小{girdDim）、块大小（blockDim）和每个网格的共享内存大小必须是相同。
・ 自定义的__device__\ __constant__ \
managed全局变量在每个设备上都是独立实例化的，因此需要开发这对该类变量赋初值。
公开成员函数：

```
bool is_valid() const;          //返回multi_grid_group是否可用
void sync() const;                //协作组内同步
unsigned long long num_threads() const;  //协作组内总线程数
unsigned long long thread_rank()；//调用线程在组内索引[0, num_threads)
unsigned int grid_rank() const; //协作组内grid索引[0, num_grids)
unsigned int num_grids() const; //启动的grid总数
unsgined long long size() const; //组内总线程数（num_threads()的别名）
```

E 开发者社区（https://developer.sourcefind.cn/）

面向国产加速卡DCU的官方一站式开发者服务平台，聚焦国产异构计算、AI
与科学计算生态，为开发者、企业、科研院所提供全链路工具、资源、文档与社区支持。

E.1社区定位与核心价值

1、社区定位

光合开发者社区是DCU 生态的核心入口，以自主可控、开放开源、高效易用为理念，打通 “芯片 — 工具 — 框架 — 应用 — 人才” 全链条，降低 DCU 开发、移植、优化与部署门槛，支撑 AI
训练推理、科学计算、工业仿真、智算中心等场景规模化落地。

2、核心价值

国产化算力底座：基于海光 DCU 加速卡，提供全精度通用 AI 与高性能计算能力

一站式开发环境：从驱动、工具链、框架到模型、应用，开箱即用

开放生态兼容：支持主流 GPGPU 生态，源码平滑迁移

产学研协同：连接开发者、企业、高校，推动技术落地与人才培养

E.2核心功能与服务模块

1、光源：

社区光源覆盖科研教育、金融等多行业近400
关键场景，超400+款主流大模型完成全面适配与精调。用户可在开发者社区首页导航栏进入光源界面。

1）模型仓库：适配DCU加速卡的优化模型库，助力开发者快速部署 AI 与科学计算任务

镜像仓库：用于存储、管理DCU支持的容器镜像，支持获取与分发的仓库。

2）数据集：为大模型提供“通用智能”提供标准化、全面化的评估框架，覆盖多种任务类型。

3）代码仓：驱动、HIP运行时系统增加BW平台支持。

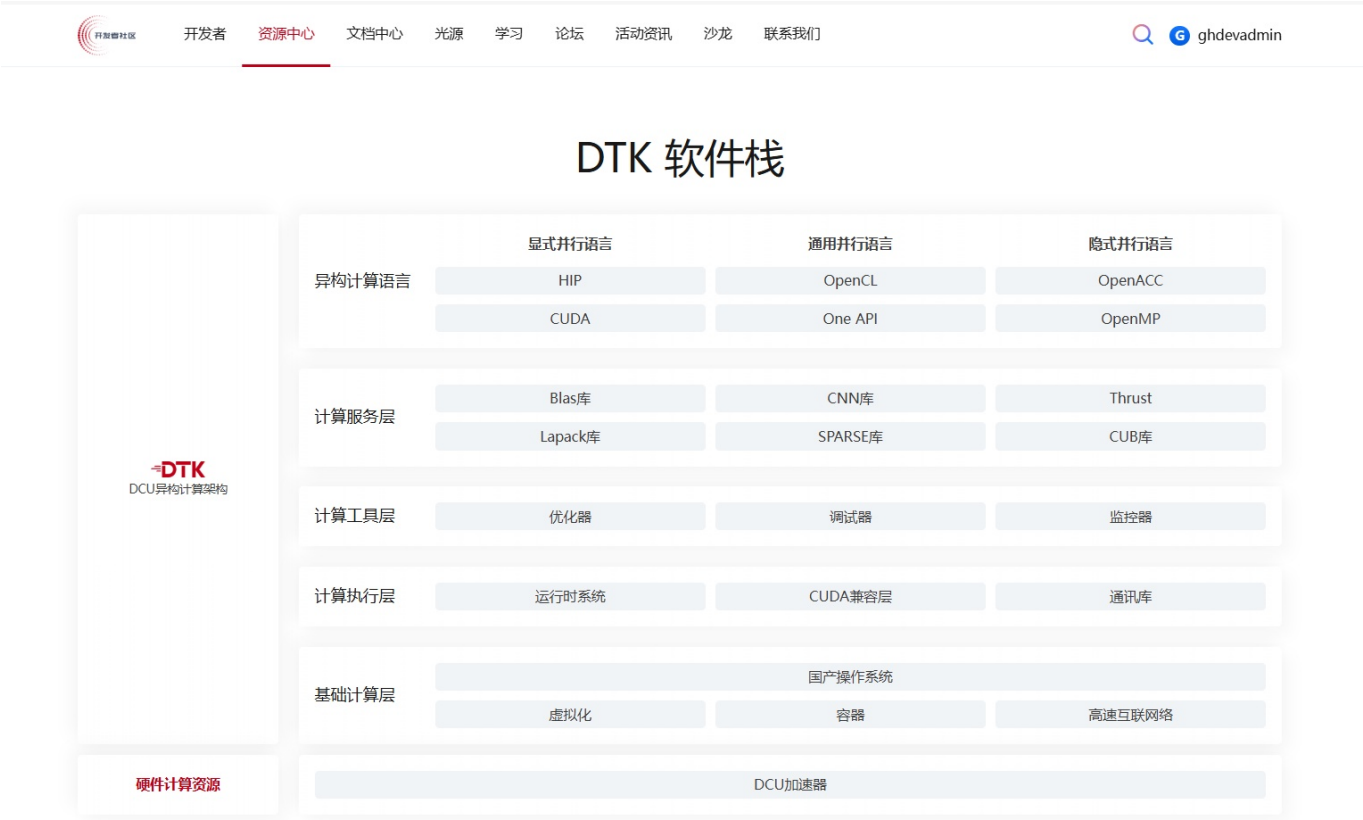
[图片]

2、 DTK 与DAS

目前最新版DTK及DAS版本都已上线社区，用户可通过社区资源中心—DTK/DAS专栏直接下载。

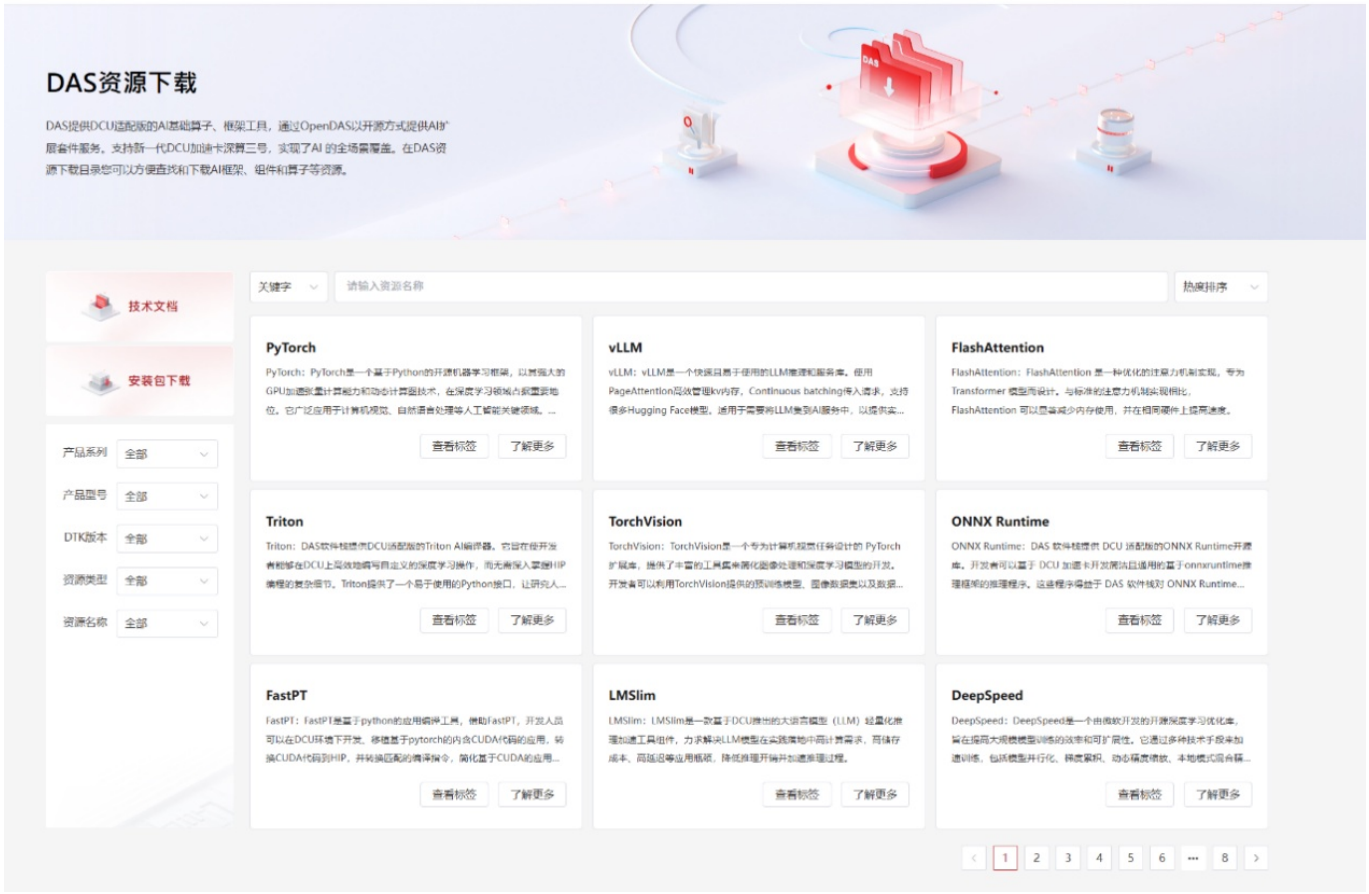
DTK(DCU ToolKit):
DTK工具包为创建高性能的DCU应用程序提供了一个完整的开发套件。借助它，您可以在DCU的CentOS、Ubuntu、Rocky、麒麟、方德等操作系统上开发、优化和部署您的应用程序。该工具包包括数学库、通讯库、调试和优化工具、Fortran/C/C++ 编译器和运行时库等。

[图片]



DAS（DCU AI Software Stack，DCU 人工智能基础软件系统）:是针对DCU
推出的一套完整的人工智能基础软件。DAS提供DCU适配版的AI基础算子、框架工具，通过OpenDAS以开源方式提供AI扩展套件服务，支持最新一代的DCU加速卡。在社区DAS页面用户可以方便查到和下载AI框架、组件和算子资源。

[图片]

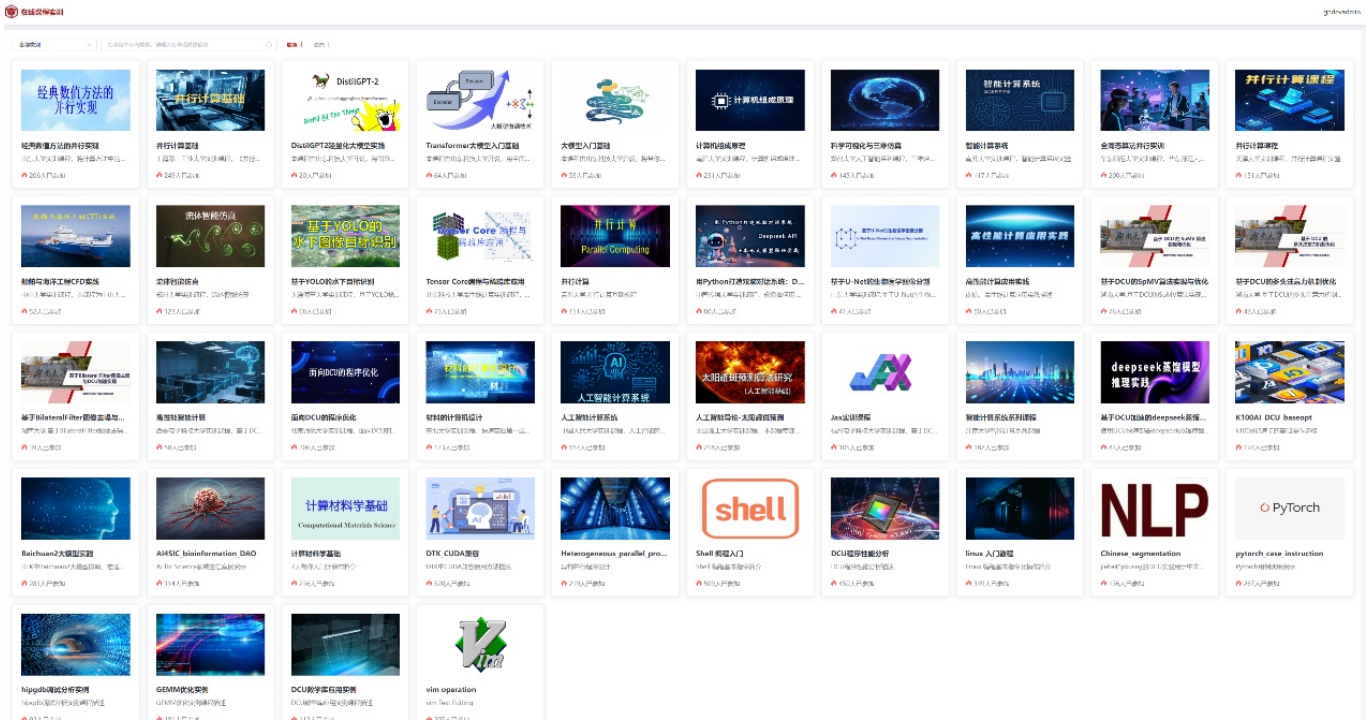


3、实训平台:

针对AI、科学计算学习者的在线教育实训平台；平台底层由DCU提供免费算力支撑；课程由中国人民大学、南开大学、中山大学、上海第二工业大学、苏州大学、中国民航大学等多个国内顶尖高校合作共建实训教学课程开发，已开展多次高校培训，同时在线实训人数近百人，覆盖智能计算、大模型、并行计算等多个领域，已超6000人次学生学习。目前已有课程：大模型入门基础、船舶与海洋工程CFD实践、用Python打造双脑对话系统：Deepseek API +
本地大模型联动实战、计算材料学基础等四十余门精品课程。

用户可通过开发者社区—学习—实训平台进入学习。

[图片]

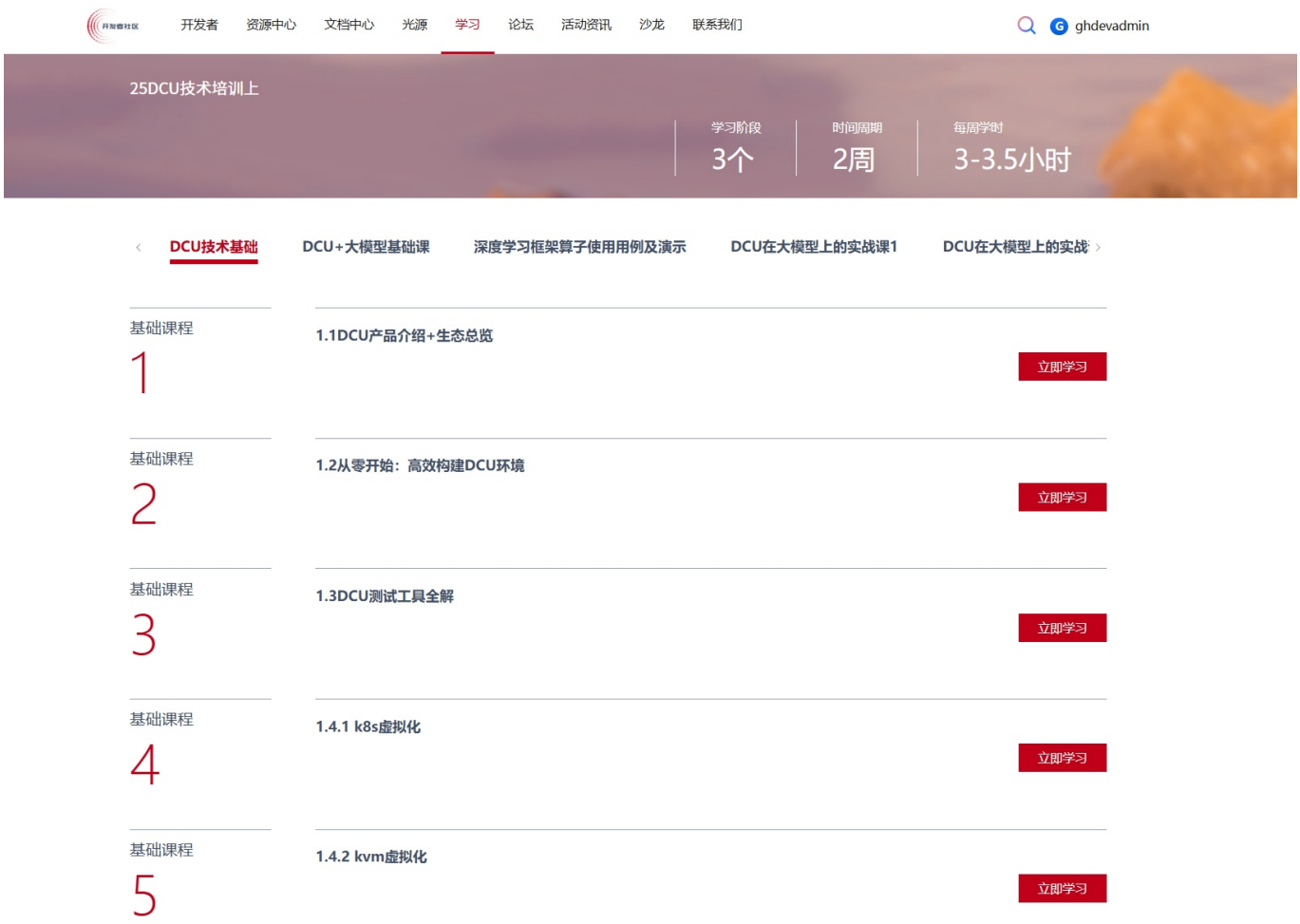


4、课程培训：

针对人工智能、科学计算等相关技术内容，提供众多视频及文档资源供用户下载学习。包括但不限于国产加速卡的异构编程优化课程，各领域常用模型，框架、算子介绍等内容。

用户可通过开发者社区—学习—课程视频进入学习。

[图片]



5、论坛

为用户提供沟通交流论坛。在社区论坛中定期发布官方技术指导文章。针对用户关心的技术问题，一对一进行技术答疑。

用户可通过开发者社区—论坛页面进入交流。

我的关注
模型适配进展
所有版块
官方技术指导
模型讨论区
DeepSeek
Qwen
其他
模型适配进展
DTK开发
数学库
运行时
编译器
AI人工智能
框架组件
模型应用
性能优化
DCU工具

综合排序 最新发布 最新回复 精华

提示 pytorch混合精度库 amp 找不到 GradScaler属性

可以在nvidia平台上正常运行的模型训练代码在东方超算的海光平台上运行报告代码出错。错误信息报告 pytorch混合精度库 amp 找不到...

菜菜 2026-05-31 14:56:51 0 框架组件

K100-AI无法直通使用问题 新人帖

我已经看到Kernel driver in use: vfio-pci，但是没法直通使用，补丁也安装了。应该如何检查呢OS版本

网友vnew3G 2026-05-29 11:24:25 1 DCU工具

有回复 **K100ai有适配的语音识别转文字的模型方案吗?** 新人帖

K100ai有适配的语音识别转文字的模型方案吗?

我要分享 我要提问

热门话题 更多 >

SBATCH -N 1，两个节点
vllm,llama,任意模型
先导杯大赛
False
K100-ai

本周热帖

1 关于适配大模型，给海光的建议
12 19

2 按照论坛大佬们给的方式部署deepseek-v4-flash版在1台K100AI...
2 6

3 【光源上线】Qwen3.6
1 8